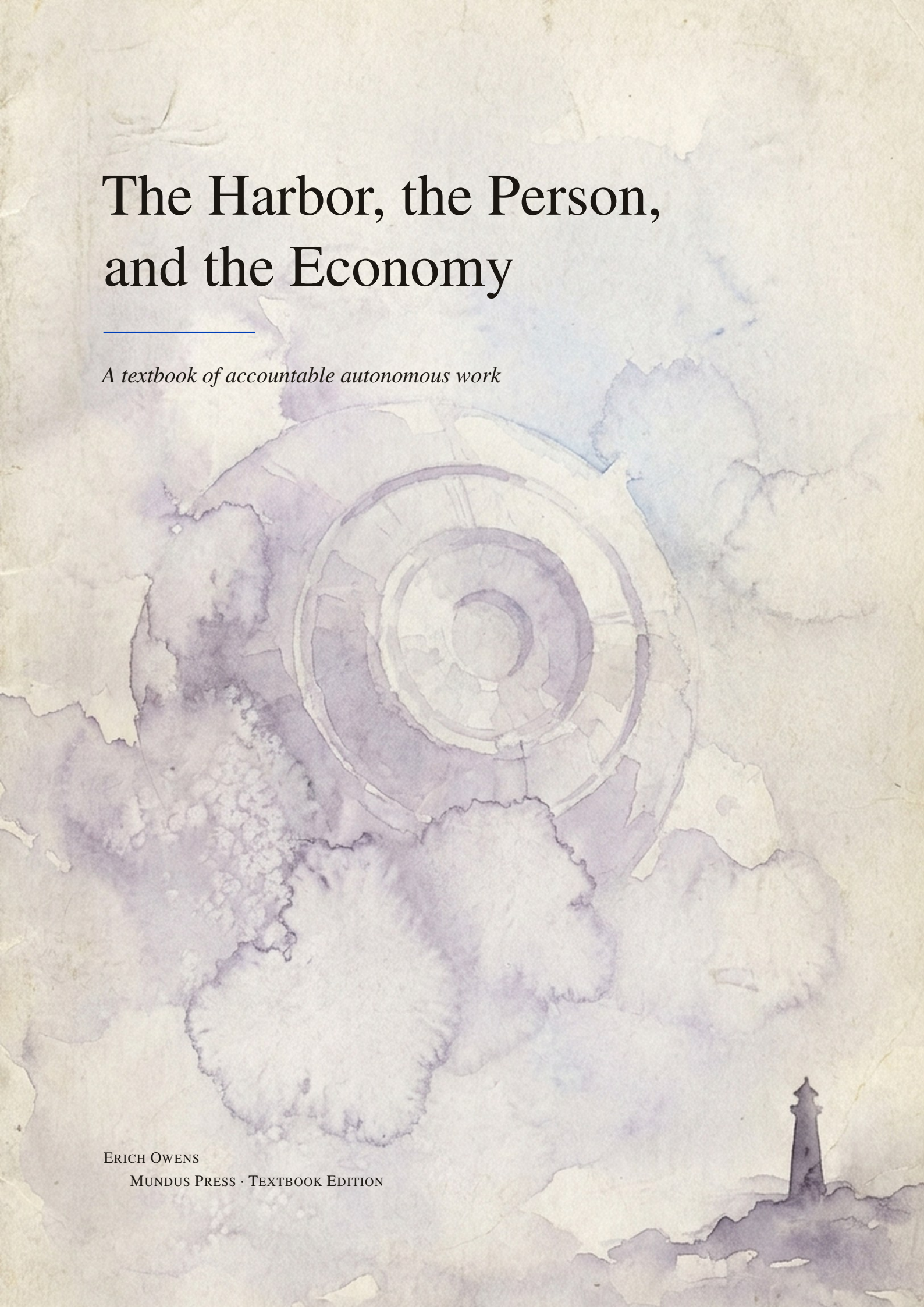


The Harbor, the Person, and the Economy

A textbook of accountable autonomous work

ERICH OWENS

MUNDUS PRESS · TEXTBOOK EDITION



The Harbor, the Person, and the Economy

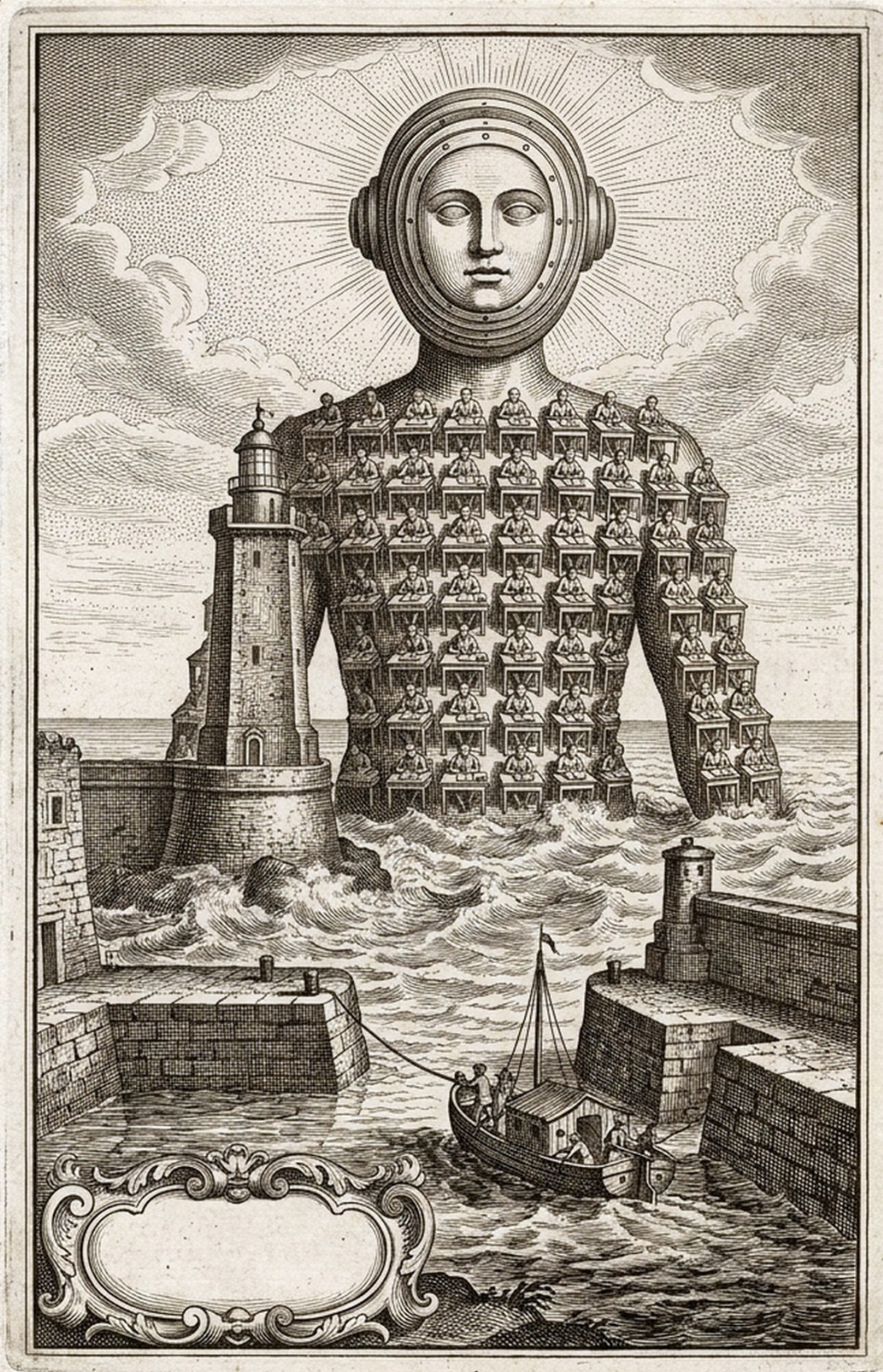
A textbook of accountable autonomous work

Autonomy scales only when authority, evidence, and consequence remain coupled at every effect boundary.

This book follows that claim from one machine and one operator to work shared across principals, economic interests, and mutually distrustful systems. Its seven chapters are ordered by what they stand on; each proof chapter follows the chapter whose promises it keeps.

Erich Owens · Curiositech LLC · engineering@portdaddy.dev

Mundus Press · Version 4.0 (Textbook Edition) · September 2026 · <https://portdaddy.dev/whitepaper>



Abstract

Agentic development fails in a predictable way when concurrency grows faster than accountability. More processes produce more work, but they also create more unread evidence, more overlapping effects, more disposable identities, and more ways for a confident report to outrun what actually happened. Better models do not remove this institutional problem.

This book makes one claim: *autonomy scales only when authority, evidence, and consequence remain coupled at every effect boundary*. Seven chapters establish that claim in the order the argument requires. They place one serial authority at the local write boundary; prove that delegated capability only narrows on its way there; derive the information cost of legible supervision; bind work to a durable principal and outcome history; separate payment, collateral, evidence, and settlement; connect witnessed acts to audit and bounded remedy; and state what may cross between mutually distrustful machines without moving either machine's sovereignty. The result is not a plea to trust agents. It is a design for making trust unnecessary where structure can suffice, measurable where uncertainty remains, and contestable everywhere else.

How to use this textbook

The unit of account in this book is the work, not the agent. A work unit is a case file: who authorized it, what it may touch, what it did, what evidence it left, what it still owes, and who pays if it fails. Every chapter builds or defends one part of that file. The chapters are ordered so that each stands on the ones before it, and each proving chapter follows the chapter whose promises it discharges. Read in order for the argument, or enter at the question you need: where a rule can be made real (*The Single-Writer Kernel*); how authority travels without growing (*The Anchor Protocol*); what an operator must be able to see (*The Legible Swarm*); what remains long enough to owe anything (*From Spawn to Person*); what makes exchange rational (*The Harbor Economy*); who may decide, on what evidence (*The Bonded Commons*); and what may cross a trust boundary (*The Federated Harbor*). The standalone editions of the chapters keep their abstracts and reader maps; here each chapter opens on the question it answers and begins with its argument.

There are two ways through every result. Each opens with an express lane, the claim in one breath and a pointer to the box that states it precisely; readers who know the field read the lane and the box and move on, and everyone else reads straight through the scene, the analogy, the box, the numbers worked by hand, what the result buys, and where it stops. Worked numbers carry a tag: [verified] when a named script in the repository regenerates them, [internal] when only the chapter's own derivation does. Exercises come in three kinds, CHECK, TRACE, and OPEN, rated one to three; their solutions are collected at the back of the book, and each exercise and solution links to the other.

Every important statement is labeled by what backs it. A *Theorem* follows from a stated formal model with explicit assumptions. A *Design invariant* is intended to hold and is backed by the implementation and its tests. A *Model-checked property* holds for a bounded model that a checker exhausted. An *Empirical hypothesis* awaits measurement. Runtime claims name one of five assurance modes, *Observed*, *Coordinated*, *Brokered*, *Confined*, and *Attested*, in increasing order of what the harbor itself enforces. The implementation status vocabulary is equally strict: **implemented** means running code exercised by tests; **PARTIAL** means a running slice that does not yet satisfy its full contract; **SPECIFIED** means a concrete design without a shipping realization; *proposed* is a research direction. Mathematical proofs, imported frameworks, finite model checks, repository experiments, and running code remain separate kinds of evidence throughout, and none is silently promoted into another.

Cross-references are live. Chapter titles, section numbers, theorems, figures, tables, exercises, page numbers, and citations are links, and every bibliography entry ends with the pages that cite it. The standalone research papers and earlier printings cite chapters by first-edition numerals; the concordance on the next page translates.

The argument, chapter by chapter

Each chapter stands on the ones before it. A chapter marked as a proof discharges a promise that an earlier chapter had to make and could not yet keep.

Part I · Ground Truth

1 The Single-Writer Kernel

One writer, one durable file, decides what is true so nothing above it has to guess.

2 The Anchor Protocol *Proves what* The Single-Writer Kernel *assumes*

Delegated capability only narrows: every child is a subset of its parent, at every hop, and the verifier checks the whole chain.

Part II · The Cost of Seeing

3 The Legible Swarm

The operator's problem is blindness, not collision; supervision has an information cost you can price in bits.

Part III · What Survives the Restart

4 From Spawn to Person

Continuity turns an anonymous spawn into a ledger position with a witnessed record; reputation is what that record is worth.

Part IV · Trade Between Strangers

5 The Harbor Economy

Once records cannot be minted, trust can be rented between operators who never met: a three-sided market on one conserving ledger.

6 The Bonded Commons *Proves what* The Harbor Economy *assumes*

Bonds, witnesses, audits and appeals turn residual uncertainty into an institution; the ledger's conservation law is mechanically checked.

7 The Federated Harbor *Proves what* The Harbor Economy *assumes*

Mutually distrustful machines relay evidence, not sovereignty: what may gossip, what needs an authority point, and how a grant crosses a boundary.

First-edition numbering. The standalone research papers and earlier printings cite these chapters by the Roman numerals of the first edition, which followed the order the chapters were written in. This edition orders them by what they stand on. The concordance:

First edition	This edition	Chapter
I	3	<i>The Legible Swarm</i>
II	1	<i>The Single-Writer Kernel</i>
III	4	<i>From Spawn to Person</i>
IV	5	<i>The Harbor Economy</i>
V	2	<i>The Anchor Protocol</i>
VI	6	<i>The Bonded Commons</i>
VII	7	<i>The Federated Harbor</i>

Contents

How to use this textbook	i
The argument, chapter by chapter	ii
1 Introduction: the unit of account is the work	vi
Part I Ground Truth	1
1 The Single-Writer Kernel	2
1.1 Where this paper sits, and what it promises	3
1.2 Reader's Map	6
1.3 The kernel as seven organs	6
1.4 The substrate organ: one writer, one file	7
1.5 Durability, split by fault class	9
1.6 The resource organ: claims, locks, and fair exclusion	11
1.7 The communication organ: the bus, stigmergy, and two delegation chains	15
1.8 The obligation & enforcement organ: the sovereign's two arms	17
1.9 The continuity organ: memory, checkpoint, and the foundation of the economy	28
1.10 The self-attestation organ: honest green	30
1.11 The invariants, stated as theorems	31
1.12 Cross-organ atomicity: a buildable defect, not a frontier	37
1.13 Threat model and the limits of mediation	37
1.14 Adjacency contract: what the kernel assumes and provides	41
1.15 Related work	42
1.16 Open problems	44
1.17 Conclusion: solid where local, provisional where it must grow	46
1. Chapter Appendices	47
1.A Implementation & status	47
2 The Anchor Protocol	49
2.1 What problem this solves	50
2.2 The Anchor Protocol Architecture	51
2.3 How we know it is correct	56
2.4 The verified code is the running code	62
2.5 What the adversary can do, and what it cannot	65
2.6 Proofs are leverage, not decoration	67
2. Chapter Appendices	69
2.A Formal Verification Status	69
2.B Full ProVerif Models	70
2.C Phase 2: Asymmetric Ed25519	71
2.D Phase 3: Multi-hop Delegation	72
2.E Limitations & Boundaries	75
2.F Further reading and prior art	76
Part II The Cost of Seeing	78
3 The Legible Swarm	79
3.1 Introduction: the swarm is a state of nature	80
3.2 Consent to the Leviathan (the normative claim)	83
3.3 The mechanism: legibility-with-zoom	86
3.4 The Leviathan rules, it does not just watch (the authority half)	89
3.5 The operator as a modeled entity	94
3.6 Read-poverty: the binding constraint at scale	98
3.7 Discovery: the directory substrate and the split ranker	101
3.8 Tokens: the swarm's COGS and its legibility engine	105
3.9 Reconciling the canon: roles, consent, and local knowledge	110
3.10 Related work and prior art	112
3.11 The time axis and the bridge to the economy	114
3. Chapter Appendices	117

3.A	Implementation and status	117
3.B	Notation and the nomenclature key	117
Part III What Survives the Restart		120
4	From Spawn to Person	121
4.1	Introduction: the spawn that cannot be paid	122
4.2	Prior art, and where this paper departs from it	124
4.3	The role / person distinction, made precise	126
4.4	Continuity is a personal-identity problem	127
4.5	The three organs of continuity	130
4.6	Identity: the root the whole chain hangs from	133
4.7	The keystone split: local identity vs. cross-operator attestation	140
4.8	Reputation as a substrate, not a score	141
4.9	Why reputation is <i>not</i> a bandit problem	144
4.10	The multi-dimensional reputation protocol	146
4.11	The grading oracle's incentive-compatibility	150
4.12	Revocation, tombstones, and bounded memory	154
4.13	What this chapter hands the market	157
4.14	Open problems (the starred exercises, collected)	158
4.15	Conclusion	159
4.	Chapter Appendices	160
4.A	Implementation status of the reference system	160
Part IV Trade Between Strangers		162
5	The Harbor Economy	163
5.1	Reader's map	164
5.2	The thesis: a market, not a payment rail	165
5.3	What a "side" is, and why there are three	166
5.4	The float plan: the built floor	169
5.5	Three sides on one escrow	171
5.6	The keystone the market rests on — and does not yet have	172
5.7	Conservation under composition	174
5.8	The Anchor Protocol proper: cross-harbor capability transfer	176
5.9	Federation: trading across machines you don't own	178
5.10	Reputation: monotone, but revocable	182
5.11	Hosted trust is the product	183
5.12	Cold start, and the auction question	184
5.13	A gluing analogy for the open federation problem	186
5.14	Gaps the design must still close	187
5.15	Related work	188
5.16	One further market: adversarial testing and purchased assurance	189
5.17	Conclusion	191
5.	Chapter Appendices	192
5.A	Implementation & status	192
5.B	Proof sketches	192
6	The Bonded Commons	194
6.1	Introduction: The Trust Problem	195
6.2	The Commons Authority	198
6.3	Layer 1: Structural Prevention	200
6.4	Layer 2: Immutable Attribution	202
6.5	The Limits of Decisive Allocation	204
6.6	Crash Recovery as Social Continuation	207
6.7	Layer 3: Economic Alignment	209
6.8	Coordination as Substrate: An Expressive-Act Taxonomy	222
6.9	Semantic Cadence, Agentic Psychosis, and Observability	222
6.10	Formal Model	224

6.11	Related Work	227
6.12	Discussion and Limitations	228
6.13	Conclusion	231
6.	Chapter Appendices	232
6.A	Formal Verification Status	232
6.B	Worked Example: One Agent, All Layers	234
6.C	Exercises	236
6.D	Scope Boundary Checklist	237
7	The Federated Harbor	239
7.1	Introduction: The Two-Machine Problem	240
7.2	The Federated Authority	242
7.3	Threat Model	244
7.4	Cross-Harbor Capability Transfer	246
7.5	Federated Revocation	248
7.6	Cross-Harbor Settlement	252
7.7	Federation Admission	255
7.8	Worked Example	255
7.9	Formal Model	257
7.10	Failure Modes	258
7.11	Related Work	259
7.12	Limitations and Open Questions	260
7.13	Conclusion	262
7.	Chapter Appendices	263
7.A	Verification Status	263
7.B	ProVerif Models (draft)	264
7.C	TLA+ Specification (draft)	264
	Solutions to the exercises	265
	Appendices	285
A	Implementation boundary	285
B	Result atlas	286
C	Notation and cross-chapter concordance	287
	References	288

1 Introduction: the unit of account is the work

Agentic software is usually measured by the agent: capability, runtime, tool access, and parallel copies. That is the wrong unit of account. A process may disappear, rename itself, summarize its mistakes, or hand work off. The durable object must be the work: its authorization, permitted effects, evidence, open obligations, and liability for the outcome.

This book develops that claim in seven propositions, one per chapter, in the order the book is read.

1. **Prevention begins at the effect boundary.** Rules can be enforced before an act only when the system controls the channel through which the act occurs. Local effects therefore pass through one serial authority; semantic failures that no gate can decide remain matters of detection and remedy.
2. **Delegation only narrows.** Every child capability is a strict subset of its parent's authority and lifetime. Authentication says who signed; attenuation says what the signature may authorize. Both are needed at every hop.
3. **Supervision has an information cost.** A digest cannot guarantee that a human will notice any one of many relevant events unless it spends enough bits, attention, or artifact openings to distinguish them. A summary is therefore an index into evidence, not a substitute for it.
4. **Accountability must outlive the process.** A task acquires a durable principal, capability history, evidence record, obligations, and outcome. Restarting an agent or choosing a new name must not erase that case file or refill the resources it already consumed.
5. **Exchange requires separated instruments.** Payment rewards the requested work. Collateral prices a bounded failure. Evidence supports a claim. Settlement closes it exactly once. Combining those functions in one score or token makes each of them less trustworthy.
6. **Uncertainty needs an institution.** Some harms cannot be prevented and some outcomes cannot be judged mechanically. Witnesses, randomized audits, bounded bonds, appeals, and conservation rules turn that residual uncertainty into a process that can be inspected and contested.
7. **Federation relays evidence, not sovereignty.** Mutually distrustful machines may exchange signed capabilities, revocations, witness roots, and settlement claims without sharing a database or accepting a global ruler. Transport can carry evidence; it cannot decide what either machine is permitted to do with it.

Together they form one causal chain. The kernel marks where effects become real; the protocol keeps delegated authority from growing on its way there; legibility states what a human must recover; continuity gives the record a durable subject; the economy bounds consequence and prices it; the commons connects evidence to remedy; federation carries those objects without pretending that transport creates consensus.

1.1 What counts as knowing

Every consequential statement has a status and a boundary. Derivations establish relations under named assumptions. Finite checks exhaust a stated state space, not every deployment. Simulations estimate behavior under a stated generator; repository tests exercise named paths; running code proves availability, not universal mediation. Counterexamples remain in the record because they mark claims the research killed rather than conclusions it merely failed to prove.

Section A gives the implementation boundary; Section B records the results and their limits. The companion `docs/harbor-research/mega-volume-epistemic-manifest.yaml` preserves each result's source, status, conditions, boundary, chapter home, and verification artifact. The chapters stand alone; the book's claim is the chain they make.

PART I

Ground Truth

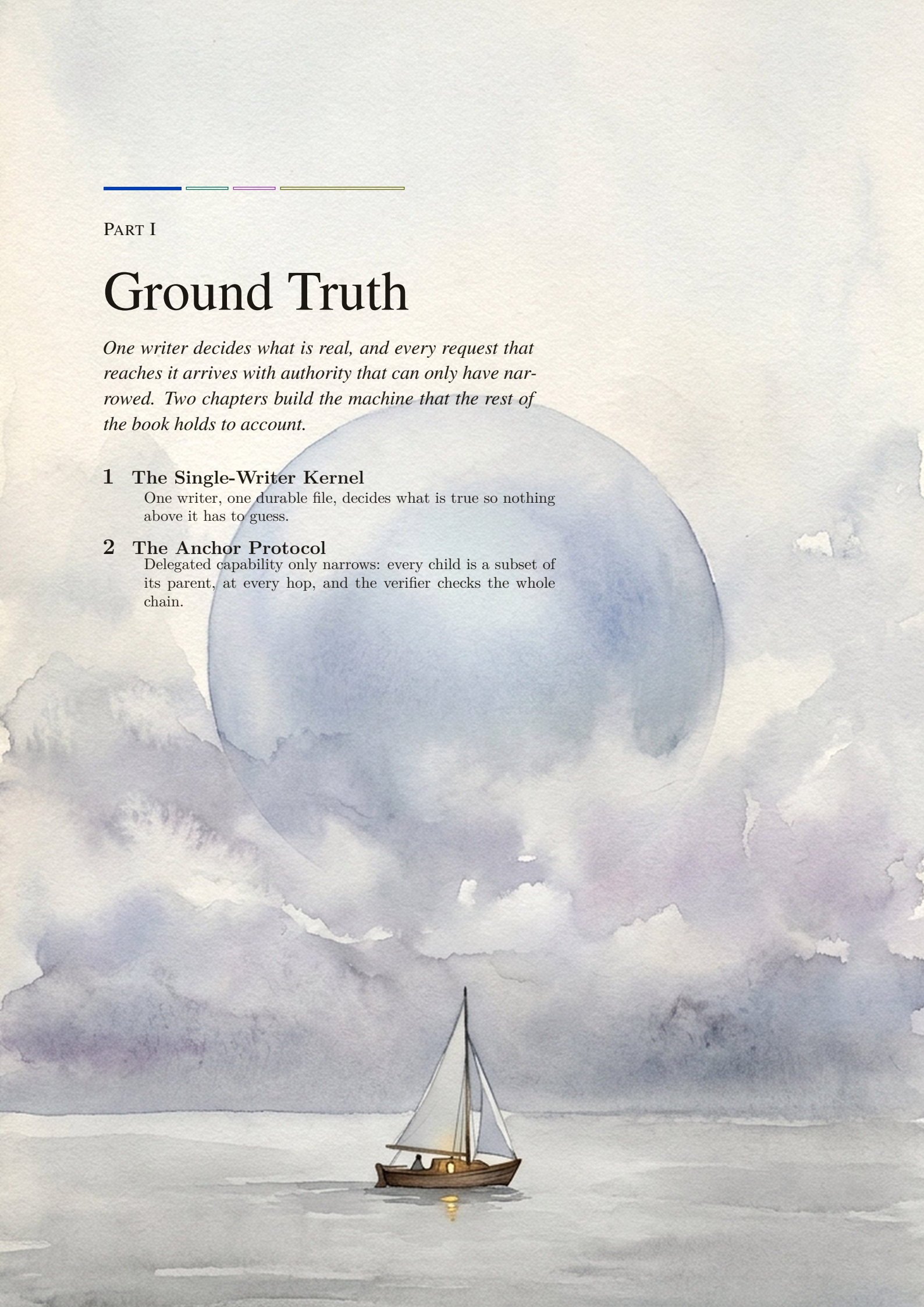
One writer decides what is real, and every request that reaches it arrives with authority that can only have narrowed. Two chapters build the machine that the rest of the book holds to account.

1 The Single-Writer Kernel

One writer, one durable file, decides what is true so nothing above it has to guess.

2 The Anchor Protocol

Delegated capability only narrows: every child is a subset of its parent, at every hop, and the verifier checks the whole chain.



The Single-Writer Kernel

*A distributed system is one in which the failure of a computer
you didn't even know existed can render your own computer
unusable.*

LESLIE LAMPORT, MESSAGE TO A DEC SRC BULLETIN BOARD, 28
MAY 1987



THE QUESTION THIS CHAPTER ANSWERS

Where can a rule be made real?

One writer, one durable file, decides what is true so nothing above it has to guess.

Where can a rule be made real? Six agents may agree about a file and still overwrite one another. A log can report the collision after the fact; only a mediated writer can refuse the second effect before it lands. This chapter identifies that local writer, separates preventable rules from detect-only claims, and states the work-unit invariants that every later institution assumes. R5 and R8 set the boundary.

1.1 Where this paper sits, and what it promises

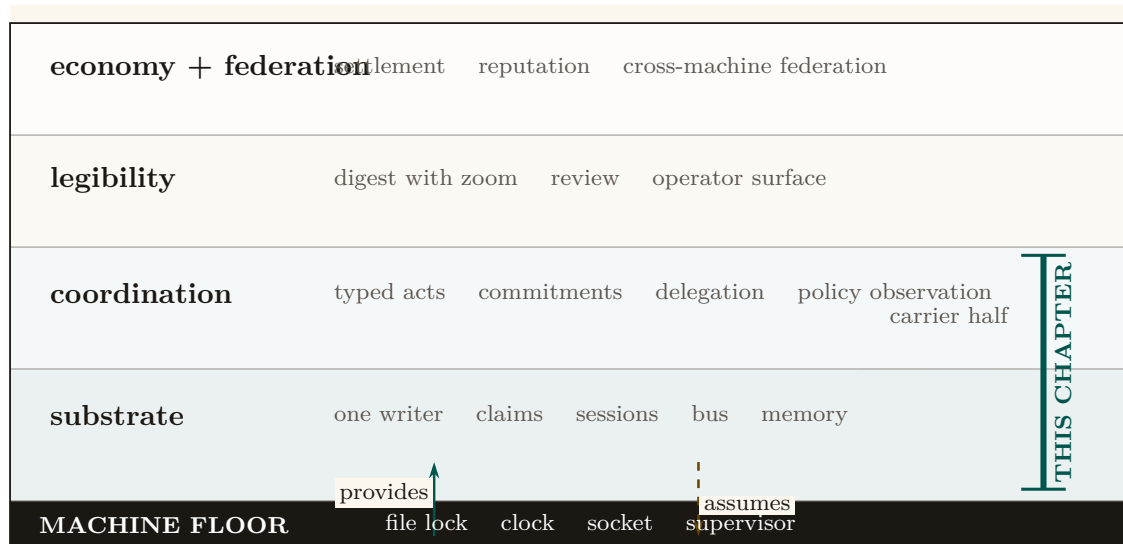


Figure 1.1: An architectural section through the four-layer coordination stack. This chapter occupies the complete transactional substrate and the implemented carrier half of coordination; it does not claim the semantic, operator, or market layers above. The dark machine floor is the narrow dependency boundary: the kernel assumes a file lock, clock, socket, and supervisor, then provides a single-writer record on which every higher layer rests.

We model an agent-coordination system as a four-layer stack (Figure 1.1), each layer serving a different *whom*.

Substrate (the machine).

A single always-on daemon over one local database: durable storage and serialized mutation. *This paper's core.*

Coordination (the agents).

The protocol agents speak over the substrate: ownership claims, a message bus, stigmergic markers, obligations, and policy enforcement. This paper specifies the implemented half of it.

Legibility (the operator).

The human-facing view that renders the swarm intelligible. Treated in an accompanying chapter (*The Legible Swarm*).

Economy (the market between operators).

The cross-machine market in which operators trade work and reputation. Treated in an accompanying chapter (*The Harbor Economy*).

You climb the stack in order, and you never buy a layer before you need it. This paper is the structural floor. The legibility layer reads the kernel's claims, bus, and policy monitor to render the swarm; the personhood argument leans the entire notion of a *durable agent identity* on the

kernel’s continuity machinery; the economy’s settlement ledger is, mechanically, another set of tables under the same single writer, and its cross-machine capability transfer rides the kernel’s cryptographic delegation chain. If this floor is unsound, everything above it inherits the fault. So the discipline of this paper is to state each promise precisely and, where the promise has an edge, to draw the edge in ink.

1.1.1 The one-sentence thesis

The kernel is a single-writer transactional reference monitor over a local SQLite/WAL database — the one process that mediates contended resources, and the one place where “true” is decided rather than asserted.

That is the whole claim, and it has two halves, which are the kernel’s entire job; everything else is convenience.

1. **Durability (with an edge):** a write that returned success is durable across the death of any agent — and, we will be careful, across the death of the *daemon process*, though *not* unconditionally across power loss.
2. **Mediation (with a correction):** a state the kernel *regiments* cannot come to exist; a state the kernel merely *enforces* is detected and compensated within a bounded window. These are different guarantees and we will never conflate them.

1.1.2 Why “reference monitor,” and why *not* consensus

The right mental model is the **reference monitor** of Anderson and Lampson [113, 43]: a mediator that is *always invoked* (complete mediation), *tamper-evident*, and *small enough to verify*. The kernel instantiates this not as an abstract security kernel but as a concrete local daemon over one SQLite file. Complete mediation holds *by construction over the kernel’s own state*: because every mutation of *coordination state* routes through one writer holding one file lock, the operating system’s serialization *is* the mediation primitive — Saltzer and Schroeder’s “economy of mechanism” [116] done with one moving part instead of N hand-rolled critical sections. We are deliberate about the scope of the word *complete*: it is complete over writes that *enter* the daemon, not over what a same-user agent can do *beside* it. A classical reference monitor earns “always invoked” from the hardware/operating-system boundary that forces every access through it; this kernel has no such boundary at the substrate layer, so a same-user process can mutate shared state the daemon never sees — edit a file under an advisory claim, run `git` directly, or write the SQLite file itself — without routing through the writer. Mediation is therefore complete over the daemon’s *application-program interface* and advisory everywhere else; making it complete over the agent’s host capabilities is an out-of-band enforcement problem we scope to the threat model (§1.13) and credit, when it lands, to a separate-user kernel boundary that does not yet exist (§1.13.2).

Key idea. The swarm *looks* like a distributed-systems problem — many agents, contention, failover — so the trained reflex is Raft, Paxos, CRDTs. The kernel’s defining move is to refuse that problem. One writer, one machine, one file: there is nothing to *agree* on, so the consensus impossibility of Fischer, Lynch, and Paterson [155] does not bite. It is sidestepped, not solved. Distribution is a cross-machine concern that belongs to the economy layer; pushing a consensus protocol down into the local substrate would be the wrong layer.

This is the single most important architectural decision a local-first coordination layer can make, and the field is littered with systems that got it wrong by manufacturing a consensus problem they

did not have [150]. We will state the formal payoff of the choice as a conditional theorem in §1.11: when every read and write is routed through the sole daemon connection and responses follow commit, transactions are *serializable* and claim/lock operations are *linearizable* — the property that lets the coordination layer treat “who holds this” as ground truth without any agreement protocol at all.

1.1.3 A research-maturity scale, and why the grades matter

A systems paper that mixes “we proved this” with “we hope to build this” is unfalsifiable. So every consequential claim in this paper carries an explicit *maturity grade* on a four-point scale, plus two refinements introduced here because a single grade cannot formally carry a claim that spans fault classes or interception points. The grade is a property of the *claim*, not a marketing adjective: it says how far the claim is from a verified, running artifact. The concrete artifacts behind each grade are catalogued in Appendix 1.A; the body argues at the level of mechanisms.

Table 1.1: The research-maturity scale used throughout. The last two rows are refinements this paper introduces (see §1.5 and §1.8).

Grade	Meaning
implemented	Realized, exercised, and true under the production runtime.
PARTIAL	Realized but partial: holds under a narrower condition than the prose might invite, or detects rather than prevents.
SPECIFIED	A concrete design exists, but no running artifact realizes it.
<i>proposed</i>	Named as a direction; no design yet.
I1a / I1b	Durability <i>split by fault class</i> : I1a (process-crash) is implemented ; I1b (power-loss) is <i>not guaranteed</i> under the default WAL configuration. A claim that conflates fault classes cannot carry one grade.
regimented / enforced	Prohibition split by interception point: <i>regimented</i> = rejected pre-commit, truly unreachable (a uniqueness constraint / boot-admission gate); <i>enforced</i> = detected post-commit, halted or compensated within a bounded window (the policy monitor). “Physically unreachable” is reserved for the regimented set.

Pitfall. The grades are not decoration. The two most consequential corrections in this paper — the durability split and the regimentation/detection split — are exactly the places where a single optimistic grade would let the kernel’s foundational promise be read as stronger than the artifact delivers. When a system claims “the database survives every agent’s death,” a reader hears “power-loss durable.” It is not, by default. Saying so is the difference between a systems paper and a brochure.

1.2 Reader’s Map

Table 1.2: Reader’s Map. Find your row; read those sections first.

If you are...	...read first	...critical artifact
short on time (15 min)	§1 (thesis + stack map), §1.5 (durability split), §1.8 (the monitor correction)	Figs. 1.1, 1.4, 1.8
a systems engineer	§1.4–§1.11 (substrate, single-writer, the theorems)	Thm. 1.11.1; Figs. 1.3, 1.14
a security reviewer	§1.8 (deontic split, the monitor), §1.13 (threat model)	Figs. 1.8, 1.7; Table 1.6
a protocol/agent author	§1.6 (claims), §1.7 (the bus), §1.14 (the interface contract)	Figs. 1.5, 1.6; §1.14
here for the economy	§1.9 (continuity organs), then the personhood and market papers	Fig. 1.12
an instructor	every Exercises block; the open problems OP-1–OP-11 and their status table (§1.16)	the starred exercises; Table 1.7

Reading order within the series. The legibility paper is the gentlest on-ramp; this paper, the substrate, is the formal floor; the personhood and market papers build upward. If you only read one paper to decide whether the foundation is sound, read this one.

1.3 The kernel as seven organs

We organize the kernel into seven *organs*, each a contract the daemon must hold for the layers above to be coherent (Figure 1.2). The temptation is to describe such a kernel as a flat noun-list of features — ports, claims, sessions, a bus, markers, commitments, a monitor, a memory store — which is the symptom of treating the kernel as a database. It is not a database. It is a system of record *plus* a mediator, and naming the organs surfaces the connective tissue a noun-list hides.

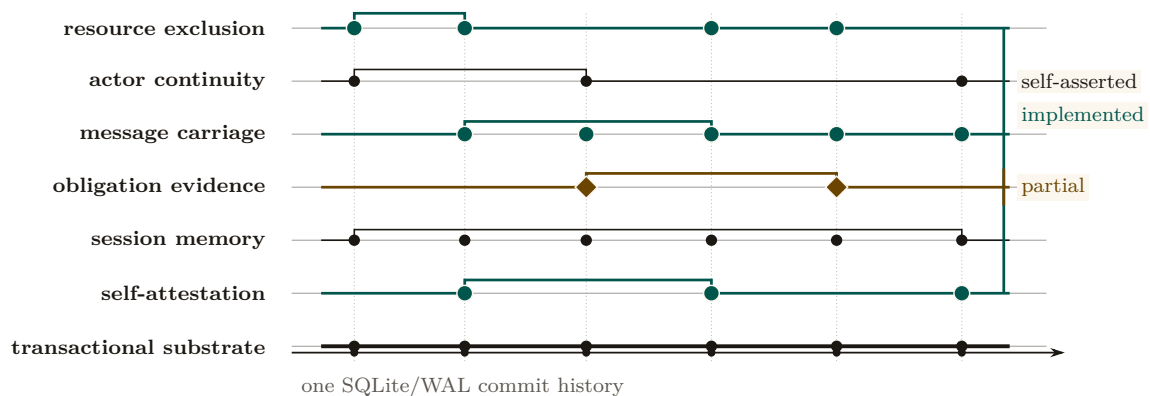


Figure 1.2: The kernel as seven contracts scored against one transactional pulse. Vertical guides are shared commit moments; the seven traces are different disciplines over the same WAL history, not seven stores or services. Teal traces are implemented carrier contracts, while the amber interruption marks the partial obligation/outcome evidence. Actor identity is mechanically present but remains self-asserted. The substrate is therefore the floor and the common timing source, not merely another feature in a catalogue.

The **substrate organ** is the floor; the other six are, mechanically, tables *with discipline* over the same file. We treat the substrate and the two organs that carry the paper’s sharpest corrections (resource/exclusion and obligation/enforcement) in full, and the rest with the depth their novelty warrants.

Exercise 1.1 · Check · ★

Which of the seven organs would still be meaningful if the daemon ran over an in-memory database with no disk at all? Which become vacuous?

Solution on page 265

Exercise 1.2 · Trace · ★★

Pick any one organ and name the single SQLite table that is its “home” table. Then name one invariant that organ owes the layer above it.

Solution on page 265

Exercise 1.3 · Open · ★★★

The seven-organ decomposition is a *choice*. Is there a more natural cut, five organs, or nine? Argue for an alternative and show what it makes easier to reason about and what it hides.

Solution on page 265

1.4 The substrate organ: one writer, one file

The substrate is the bedrock everything else is just a table in. Its job is to make two promises — durability and serialized mutation — and to make them formally. We cover the single-writer discipline here and durability in §1.5, because durability is where the most consequential correction lives.

1.4.1 The single-writer discipline

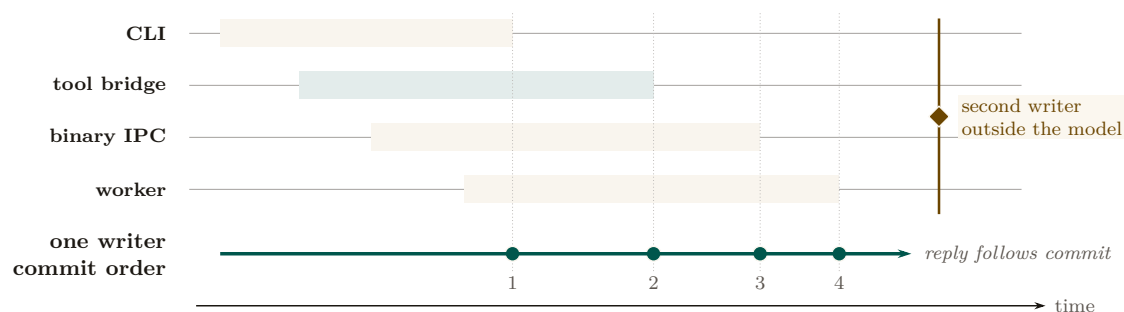


Figure 1.3: Concurrent request intervals become one commit order. Each pale band is live work in a different caller; its right edge meets exactly one position on the teal writer rail. The daemon replies only after that commit, so successful mutations share one observable history without an agreement protocol inside the machine. The separate amber mark is the model boundary: a second daemon sharing the file is a cross-machine problem, not another lane in this schedule.

The kernel’s concurrency story (Figure 1.3) is: dozens of command-line invocations and agent connections funnel through one writer — the daemon, the sole holder of a read–write connection to the database file — and SQLite’s operating-system file lock plus a bounded busy-wait serialize all mutations. We must be precise about *what* serializes writers, because there are two stories and only one is true at a time.

Definition 1.4.1 (Single-writer discipline). All state-mutating operations route through a single daemon process holding one SQLite write connection. Concurrency among the many clients (command-line invocations, agents over the message transports, background workers) is resolved by the daemon’s request queue; the SQLite file lock is the *backstop* that preserves correctness if the discipline is ever violated by a second writer (for example a stray direct write), not the primary serialization mechanism.

Key idea. “Single-writer” is an architectural *discipline* (everything goes through the daemon), backstopped — not replaced — by SQLite’s file lock. This distinction matters: SQLite permits multiple writer connections (it serializes them, it does not reject them), so the property holds because of where writes are routed, not because the database forbids a second writer. The standing risk is therefore a second *copy* of the daemon coming up against the same file — which is why the substrate also runs self-checks that detect a divergent or stale second instance (§1.10).

1.4.2 The configuration *is* the durability claim

The substrate’s storage configuration is short and decisive. In WAL mode with the *normal* synchronization level (the WAL is fsync’d at checkpoints rather than on every commit), a bounded auto-checkpoint interval, a bounded busy-wait, foreign-key enforcement on, and a clean-shutdown checkpoint-and-truncate, the kernel inherits exactly the crash semantics that configuration defines. The claim “a write that returned success survives a crash” reduces *entirely* to what WAL with normal synchronization guarantees — which is the subject of §1.5, and which is not what an unguarded reading assumes.

1.4.3 Schema evolution: an ordered boot ledger, not yet a sequencer (OP-7)

The initial schema-by-union approach (`CREATE TABLE IF NOT EXISTS`) created vulnerabilities around unsequenced renames and multi-column backfills. The kernel’s answer so far is a partial one, and the honest way to report it is to name the trade it makes rather than the property it saves.

What ships (PARTIAL) is a **boot-time migration ledger**: the daemon applies a numbered, totally ordered set of migrations before it serves any request, and then *verifies the schema it is about to serve from* — probing the actual tables and column sentinels rather than trusting that the statements ran without raising — and refuses to serve on a mismatch. That is a real fail-safe default, and it is the reason a half-applied migration cannot quietly become coordination truth.

What does *not* yet ship is the sequencer that would make that order authoritative. Organ modules still self-initialize their own tables over the shared database handle (Appendix 1.A), so the schema is still, in part, a union. A strict linear ledger managed exclusively by the daemon via SQLite’s `pragma user_version`, in which modules *declare* migrations rather than applying them, is SPECIFIED. OP-7 therefore stands PARTIAL: this is Table 1.1’s second question — “is a version table unavoidable?” — answered *probably yes*, at the cost of the “any module, any order” flexibility the schema-by-union design offered, and not yet paid for in full.

1.4.4 Path and ownership invariants

The database resolves to a single canonical path with owner-only file permissions (historically the file also carried the system’s signing key in plaintext; that secret is being migrated to the operating system’s keychain, see §1.13). A fail-closed guard refuses to open the live registry from a test context — the analogue of a framework’s protected-environment check. This is Saltzer and Schroeder’s *fail-safe defaults* [116] applied at the one place a test could corrupt production truth.

Exercise 1.4 · Check · ★

The schema-by-union design lets any module create its tables in any order. Give a concrete two-module example where a lazy column-rename migration would be unsafe under this design.

Solution on page 265

Exercise 1.5 · Trace · ★★

Follow a single claim request from command line to disk. At how many points does the single-writer discipline (Definition 1.4.1) hold by *routing* versus by the *file lock*?

Solution on page 266

Exercise 1.6 · Open · ★★★

The boot ledger orders migrations, but modules still self-initialize, so the order is not yet authoritative (OP-7). What is the smallest change that makes it so, a `user_version` sequencer over *declared* migrations, and can any useful remnant of “any module, any order” survive it for renames, backfills, and down-migrations?

Solution on page 266

1.5 Durability, split by fault class

This is the most important correction in the paper, so it gets its own section. The kernel’s foundational promise — “a write that returned success survives” — is true, but only for the fault class a careful reader expects and not for the one the pitch most invites.

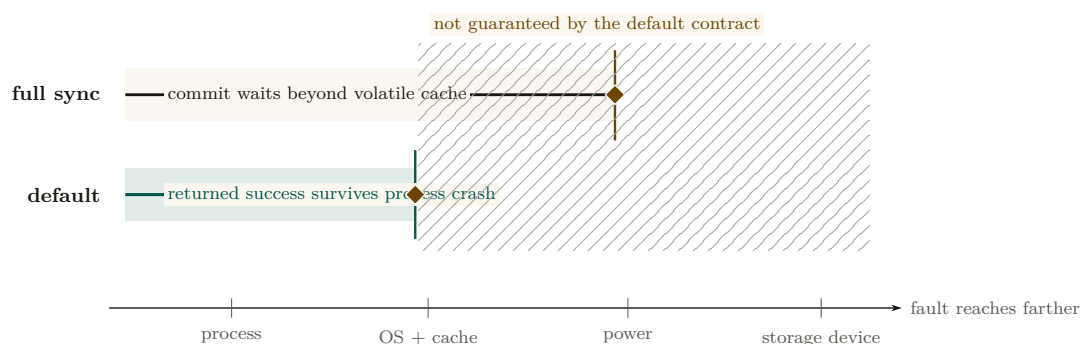


Figure 1.4: Durability as a boundary in fault reach. Under normal WAL synchronization, a returned write crosses the process-crash boundary because the operating system and its cache remain live; the contract ends before OS crash or power loss. Full synchronization can move that boundary outward by waiting beyond volatile cache, but neither band claims immunity to arbitrary media failure. The figure therefore separates crash consistency from power-loss durability instead of presenting a universal “saved” state.

Figure 1.4 draws the split this section defends: the same write, judged against two different fault classes with two different answers.

1.5.1 Why one label is a lie here

Gray and Reuter [117] draw the canonical line between *atomicity/consistency* (a transaction is all-or-nothing and never leaves the store corrupt) and *durability* (the “D” in ACID: a committed

transaction survives subsequent failures). Under SQLite’s WAL with the *normal* synchronization level, the log is *not* fsync’d on every commit — it is synced at checkpoint [57]. Per SQLite’s own documentation, in WAL mode the normal level means “commits might lose durability following a power loss or hard reset,” though the database remains uncorrupted. So the honest statement splits in two.

Property 1.5.1 (Durability by fault class). Let a write return success under the default WAL configuration.

- **I1a (process-crash durability).** If the *daemon process* dies (a forced kill, a panic, an out-of-memory termination), the write survives: the data is in the operating system’s page cache, which outlives the process. **implemented.**
- **I1b (power-loss durability).** If the *operating system* crashes or power is lost *before the next checkpoint*, the most recent committed writes may roll back. *not guaranteed* under the normal synchronization level; it would require full synchronization or a checkpoint on the commit path.

Worked dramatization: Alice crashes mid-write. Alice is an agent editing a file. At t_0 she commits a claim on a port and a note recording her edit; the daemon returns success. At $t_0 + 5$ ms her host operating system kills the daemon process outright — a forced termination, no clean shutdown. *What survives?* Both writes. The committed pages live in the operating system’s page cache, which the daemon process does not own and does not take with it when it dies; when the supervisor restarts the daemon, SQLite finds a dirty log, replays it, and Alice’s claim and note are intact. This is I1a, and it is the fault class the design actually targets: agents die constantly, and the record must outlive them. Now change one detail. At $t_0 + 5$ ms, instead of the daemon dying, *the power fails*. The committed pages were in the page cache but had not yet reached stable storage, because under the normal synchronization level the log is fsync’d only at the next checkpoint. On reboot, the database is uncorrupted — but Alice’s last claim and note may simply be gone, rolled back as if never written. This is I1b, and it is *not guaranteed*. The two faults differ only in whether the page cache survived; conflating them is the single most common over-claim about WAL-backed storage.

Pitfall. The seductive misconception is “the normal sync level is safe in WAL mode because WAL already guarantees crash safety.” This conflates *crash-consistency* (true: the database is never corrupt) with *durability* (false for power loss). The two are different ACID guarantees, and only the first is unconditional here. A systems paper cannot ship the unqualified claim. We split it.

1.5.2 Why the trade is defensible — and where it stops being so

Trading power-loss durability for throughput is a reasonable default for a local-first developer tool: the failure (lose the last few hundred milliseconds of claims after a power cut) is benign and self-correcting — agents re-claim, heartbeats resume. What is *not* defensible is letting the prose imply the stronger guarantee — and for the few paths that genuinely need I1b, the right fix is a per-write-path selector rather than a global change of synchronization level.

The design, **Selective Checkpointing** (SPECIFIED, OP-10), is small: a high-stakes, money-bearing write path — a settlement-ledger entry, say — appends `PRAGMA wal_checkpoint(FULL)`; to its commit path, forcing a synchronous flush to disk, while claim, marker, and note writes keep the fast default. Two things must be said plainly about it. First, *no write path in the reference implementation opts in today*: the only checkpoints it issues are the bounded auto-checkpoint, a passive checkpoint on the maintenance path, and the clean-shutdown truncate (Appendix 1.A). Second, even once a path opts in, the guarantee it buys is *per path*: Property 1.5.1 continues to

govern every path that does not, so I1b remains *not guaranteed* for the kernel as a whole, and a table row or a figure that says otherwise is wrong. OP-10 stays *open* until both the selector and an interface that stops a *new* write path from silently defaulting into the fast lane exist.

1.5.3 A recovery story, not just a durability story

Durability is about what survives; recovery is about what happens next. On daemon restart with a dirty WAL, SQLite replays the log to a consistent state (I1a’s mechanism). But the kernel-level questions are larger: who validates the audit chain on boot, and what becomes of a half-applied cross-organ operation (§1.12) after a crash? The hook exists — the boot-admission gate that refuses to serve when a critical self-check fails (§1.10) — but boot-time WAL recovery plus integrity check plus chain verification should be a single named invariant, and today it is closer to PARTIAL than **implemented**.

Exercise 1.7 · Check · ★

Classify each fault as I1a-survivable or I1b-exposed: (a) a forced kill of the daemon process; (b) a clean operating-system reboot; (c) yanking the power cord; (d) an unhandled exception in a request handler.

Solution on page 266

Exercise 1.8 · Trace · ★★

A claim commits at t_0 ; the next auto-checkpoint fires at $t_0 + \delta$. Draw the window during which a power loss can revert the claim. Does a page-count auto-checkpoint threshold bound δ in wall-clock time?

Solution on page 266

Exercise 1.9 · Open · ★★★

Section 1.5 sketches Selective Checkpointing (OP-10), but nothing in the kernel opts into it. What is the cleanest interface that lets a settlement-ledger write demand I1b while keeping claim writes fast, and, harder, what stops a *future* write path from silently defaulting into the fast lane when it should have opted in?

Solution on page 266

1.6 The resource organ: claims, locks, and fair exclusion

Network ports are the namesake and the original sin: two agents that each start a development server on the same port collide, and one silently fails. The resource organ generalizes that problem into a single exclusion primitive — over network ports, over named mutex locks, and over edit-surface claims on regions of source files — to which every higher layer’s notion of ownership reduces.

1.6.1 Claim granularity is richer than “claims”

It is tempting to collapse all three of these into one word. The kernel instead distinguishes **whole-file**, **line-range**, and **symbol-path** claims, each keyed by the file path together with, respectively, nothing, a line interval, or a syntactic symbol identifier. This is the substrate for a

“reserve regions, not whole files” coordination policy and for semantic-conflict prediction over a syntax-aware symbol index — but at the substrate layer it is simply exclusion at three granularities.

1.6.2 The claim lifecycle as a finite-state machine

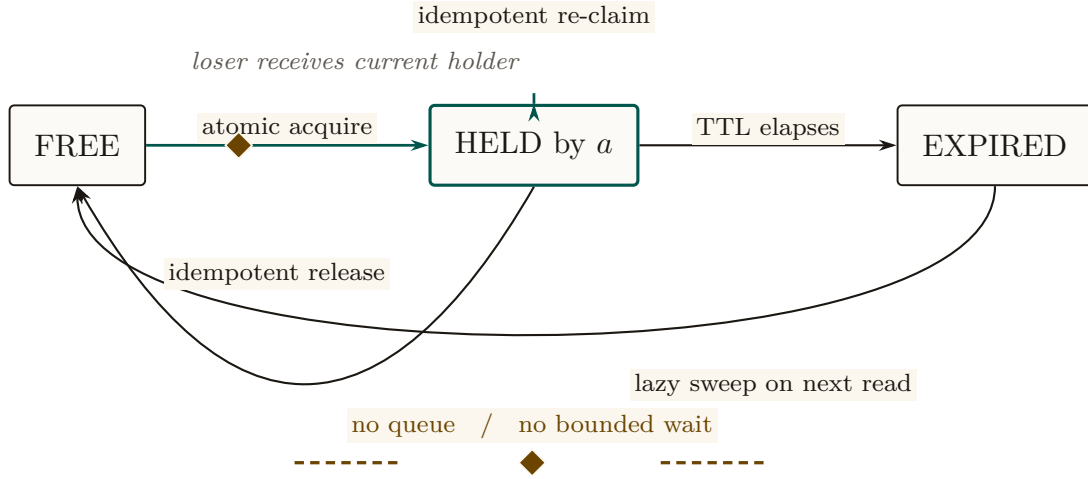


Figure 1.5: The guarded claim lifecycle. Acquire is one atomic uniqueness decision; re-claim and release are idempotent; expiry is removed lazily on the next read. The interrupted amber rail is intentional negative space: the kernel has no queued state and therefore no fairness or bounded-wait guarantee. A reactive ticket mechanism may later occupy that gap for cooperatively released claims, but lazy expiry would still bypass it, so starvation remains an open problem.

Figure 1.5 states, as a state machine, the four properties on which the coordination layer builds typed ownership. The acquire path is small enough to state as an algorithm (Algorithm 1.1); we then give the central property as a theorem.

```

1 function acquire(key, requester, ttl):
2     // single atomic statement; the uniqueness constraint is the
   // mutex
3     try:
4         INSERT row (key, holder=requester, expires_at = now()+ttl)
5         return GRANTED(holder=requester)           // we won
6     catch UNIQUE_CONSTRAINT_VIOLATION:
7         existing = SELECT row WHERE row.key = key
8         if existing.holder == requester:
9             UPDATE row SET expires_at = now()+ttl // idempotent re-
               claim
10            return GRANTED(holder=requester)
11        if existing.expires_at < now():               // stale holder
12            DELETE row WHERE key = key AND expires_at < now()
13            return acquire(key, requester, ttl)      // one bounded
               retry
14        return DENIED(holder=existing.holder)        // loser learns the
               winner
    
```

Listing 1.1: Claim acquisition. The whole acquire is a single insert against a uniqueness constraint; the loser is handed the current holder, not an error to retry blindly.

Theorem 1.6.1 (Atomic Mutual Exclusion and Fenced Leases). *Within a canonical conflict domain (exact normalized key, transactional interval overlap $[a, b] \cap [c, d] \neq \emptyset$, or path prefix hierarchy), at most one holder can hold an active lease at any logical instant. Furthermore, every*

granted lease carries a strictly monotonic fencing epoch $e \in \mathbb{N}$; downstream effect brokers reject any operation bearing an expired epoch $e' < e_{\text{current}}$.

Proof sketch. Within an immediate SQLite transaction (`BEGIN IMMEDIATE`), the daemon evaluates the conflict predicate against all currently unexpired leases. For exact keys, this is enforced by primary-key uniqueness; for intervals and hierarchies, by an indexed range check within the same transaction. Because all writes funnel through the single connection (Def. 1.4.1), predicate evaluation and insert are atomic. Upon grant or reallocation, the daemon increments the resource’s fencing epoch e . Downstream effect brokers validate that e matches the active lease at execution time, preventing stale or paused holders from producing side effects after lease expiration. $\square$ $\square$

Worked dramatization: Alice and Bob race for one port. Alice and Bob are two agents, started a millisecond apart, that both want to bind a development server to port 7421. Each issues `acquire(7421)` (Algorithm 1.1). Both requests reach the single daemon, which serializes them: one insert lands first. Say Alice’s does. Her insert succeeds and she is granted the port. Bob’s insert, arriving an instant later, hits the uniqueness constraint on key 7421 and is rejected *atomically* — there is no moment at which both rows exist. Crucially, Bob does not get a bare error. The catch path reads back the row and hands Bob the *identity of the holder*: “port 7421 is held by Alice.” Bob can now pick another port, wait, or message Alice, but he can never double-bind. The serialization that makes this clean is not a hand-rolled critical section; it is the operating system’s file lock funnelling both inserts through one writer, and the database’s primary-key uniqueness deciding the winner. One writer, one constraint, one holder.

Pitfall. Theorem 1.6.1 is sound for the *fresh-contention* case above. A subtler hazard hides in the *stale-holder* branch: reclaiming an expired lock runs a delete (of the expired row) and *then* an insert as two separate statements rather than one transaction. On the single daemon connection this is serialized by the connection itself and is safe. But if a second writer connection ever existed — the very scenario the single-writer discipline exists to prevent — a deferred transaction would open a window for a transient “database busy” error mid-sequence rather than clean serialization; the fix is to begin the transaction in immediate mode. So Theorem 1.6.1 holds because all writes route through the one daemon connection (Def. 1.4.1), *not* because the expire-then-acquire sequence is itself atomic. State which story you mean; do not tell both.

1.6.3 Fairness without a scheduler: the Stigmergic Ticket-Lock (OP-1)

The conspicuous gap in the claim lifecycle (Figure 1.5) is the absence of a `QUEUED` state. Locks have a time-to-live, but acquisition is racy first-come, so a fast agent churning a hot file can starve a slow agent indefinitely.

The design that would close OP-1 without dragging a background scheduler into the kernel is a **Stigmergic Ticket-Lock**. It is `SPECIFIED` — specified here in enough detail to build and to attack, but not realized: no `claim_tickets` table exists in the reference implementation, and the lifecycle above is still the one the kernel runs. It would introduce a `claim_tickets` table:

```
1 CREATE TABLE claim_tickets (
2   resource_id TEXT NOT NULL,
3   agent_id TEXT NOT NULL,
4   seq_num INTEGER PRIMARY KEY AUTOINCREMENT
5 );
```

When Agent B ’s claim acquisition failed against the primary-key `UNIQUE` constraint, the error handler would insert an entry into `claim_tickets`. When Agent A then called `release('auth.ts')`, the daemon would execute a single atomic transaction:

1. Delete Agent *A*'s active claim row: `DELETE FROM claims WHERE resource = :r;`
2. Read the lowest ticket:

```

1 SELECT agent_id, seq_num FROM claim_tickets
2 WHERE resource_id = :r
3 ORDER BY seq_num ASC LIMIT 1;

```

3. Atomically re-grant the claim: `INSERT INTO claims (resource, holder, ttl) VALUES (:r, :next_agent, :ttl);`
4. Remove the claimed ticket: `DELETE FROM claim_tickets WHERE seq_num = :next_seq;`

Because the state transition would be driven entirely *reactively* by the releasing agent inside one transaction, FIFO ordering among waiters follows from the ticket table's own monotonic sequence, with no background timer or polling thread in the kernel — which is the property that makes the design worth stating.

Its honest edge is the path it does *not* cover. A holder that never calls `release` — it crashed, or its lease simply expired — returns the resource through the lazy TTL sweep (I4), and the sweep consults no ticket queue. So bounded wait would hold for cooperative release and fail on exactly the failure path the kernel was built to expect. OP-1 stays *open* until the sweep and the queue are the same code path, and until the ticket table exists at all.

Exercise 1.10 · Check · ★

Why does releasing a lock you do not hold return success rather than an error? Which property (I2 to I4) is this, and what would break if it raised?

Solution on page 267

Exercise 1.11 · Trace · ★★

Two agents call `acquire` on the same fresh key at the same instant. Walk the uniqueness-constraint insert for both (Algorithm 1.1) and show exactly where the loser learns the holder's identity.

Solution on page 267

Exercise 1.12 · Open · ★★★

The Stigmergic Ticket-Lock buys bounded wait on the cooperative-release path only (OP-1). Extend it so that a lease reclaimed by the lazy TTL sweep also honors the ticket queue, still without a background scheduler, or argue that no reactive-only design can, and say what the minimum scheduler would be.

Solution on page 267

1.7 The communication organ: the bus, stigmergy, and two delegation chains

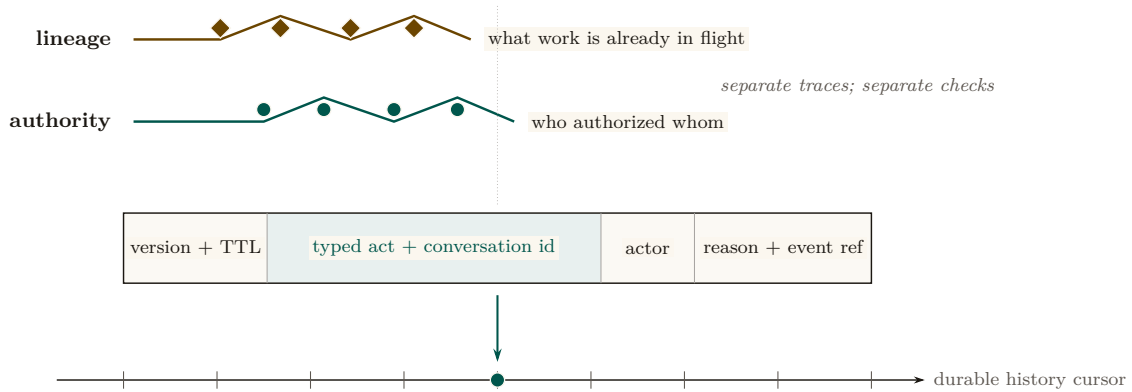


Figure 1.6: A cross-section of the communication organ. The implemented bus supplies an addressable, ordered, at-least-once carrier; the typed act occupies a field inside that durable envelope. Two provenance traces remain orthogonal above it: the authorization trace answers who authorized whom, while coordination lineage answers what work is already in flight. Collapsing them into one generic “delegation chain” destroys both verification questions.

The communication organ (Figure 1.6) is where the substrate stops being a lockbox and becomes a medium for conversation. The carrier is **implemented**; the *semantics* that ride on top of it (typed speech acts, protocol state machines) belong to the coordination layer. Three pieces matter here.

1.7.1 The bus: a durable carrier envelope

The bus is a durable, ordered, *at-least-once* publish/subscribe channel over a messages table, with a versioned envelope (a version tag, a message kind, a body, and an in-reply-to pointer), a history cursor for replay, and a per-message time-to-live. The coordination layer’s typed speech act — carrying a performative, a conversation identifier, a coordination lineage, and a reply deadline — is layered *inside* this envelope. The kernel ships the carrier; the coordination layer ships what it carries.

Key idea. At-least-once delivery means a *healthy* bus routinely redelivers and reorders. This collides with a loud-fail honesty discipline: if “every anomaly is loud,” benign redelivery drowns the operator in false alarms. The resolution is that benign transport non-determinism must be *deduplicated silently*, and only genuine protocol-state-machine violations surface loudly. That requires an idempotency key in the envelope — currently absent — which is the cheapest high-leverage fix at the coordination layer. The kernel provides the bus; making every speech-act effect idempotent is the coordination layer’s debt, flagged here so the reader does not mistake silent deduplication for dishonesty.

1.7.2 Stigmergic markers: decay as a provided guarantee

The kernel also carries *stigmergic markers* — numeric coordination signals deposited in shared state, after the manner of Grassé’s stigmergy in social insects [175] and the generative-communication tuple spaces of Gelernter’s Linda [67], and used in ant-colony optimization [144]. Each marker carries per-kind exponential decay $w \cdot r^{\Delta t}$, computed both on a background evaporation tick *and* at read time (so a reader sees the correctly decayed value without waiting for the tick), and pruned below a small threshold. The decay is the structural defense against “blackboard rot”: stale

coordination signals evaporate on their own. The calibration of the decay rate is genuinely open (OP-8): too slow and signals rot, too fast and they evaporate before they coordinate.

1.7.3 Two delegation chains, one dangerous name

A dangerous naming collision lives in this organ. The phrase “delegation chain” names *two different objects*, which we keep rigorously distinct:

- the **authorization chain** — the *cryptographic* object (**implemented**, mechanically verified in a protocol-verification tool): a hop-bound, attenuation-monotonic, anti-replay and anti-splice chain of digital signatures that proves *who authorized whom*. This is what cross-machine capability transfer in the economy layer rides on.
- the **coordination lineage** — the *coordination* object (SPECIFIED): the task-lineage over which loop detection and upward-block run, preventing two agents from delegating a task back and forth forever.

These are distinct objects, and the relationship is that the coordination lineage is carried *inside* the authorization chain — loop detection runs over a lineage whose authenticity the cryptographic chain guarantees. We never again write bare “delegation chain.”

Pitfall. The “lineage inside chain” relationship is a slogan until the task-shape field is in the *signed* payload at each hop — and that collides with a rule against keyword-based text classification, because loop detection needs a *semantic* task-shape equivalence (a learned embedding or a structural hash), which is not deterministically signable. This tension is a real open problem; we flag it rather than assert it solved.

1.7.4 A second transport

The kernel ships *two* transports, not one: a request/response protocol over a Unix domain socket *and* a binary framed transport over a Unix domain socket, the latter with peer-credential authentication and backpressure/draining for high-frequency, tight-loop coordination. We note in §1.13 that the peer-credential check is a software handshake — the connecting process *states* an agent identity and the daemon checks it against the registration table, backed by owner-only (0600) permissions on the socket file — and *not* the kernel-enforced socket-level credential check (SO_PEERCREC) it resembles and is named after. That is a real trust-boundary caveat, it holds for both transports, and it is unchanged by the hardening design of §1.13.4.

Exercise 1.13 · Check · ★

Why does read-time marker decay make the system more honest than a background tick alone? Give a scenario where tick-only decay would report a stale signal as fresh.

Solution on page 267

Exercise 1.14 · Trace · ★★

A request speech act is sent over the bus and redelivered once by the at-least-once channel. Trace what the operator should see under the silent-dedup resolution, and what an envelope idempotency key would change.

Solution on page 267

Exercise 1.15 · Open · ***

What per-kind marker decay rate keeps stigmergic signals neither rotten nor prematurely evaporated (OP-8)? Frame this as an empirical measurement, not a constant to guess.

Solution on page 268

1.8 The obligation & enforcement organ: the sovereign’s two arms

This organ is the deontic core, and it is genuinely two distinct mechanisms — in the vocabulary of deontic logic for computer systems (Jones and Sergot [15]): *prohibition* (a policy monitor) and *obligation* (commitments). Getting the distinction right — and being honest about how strong each arm actually is — is the security heart of the paper.

Figure 1.7 lays out the three modalities and the mechanism behind each.

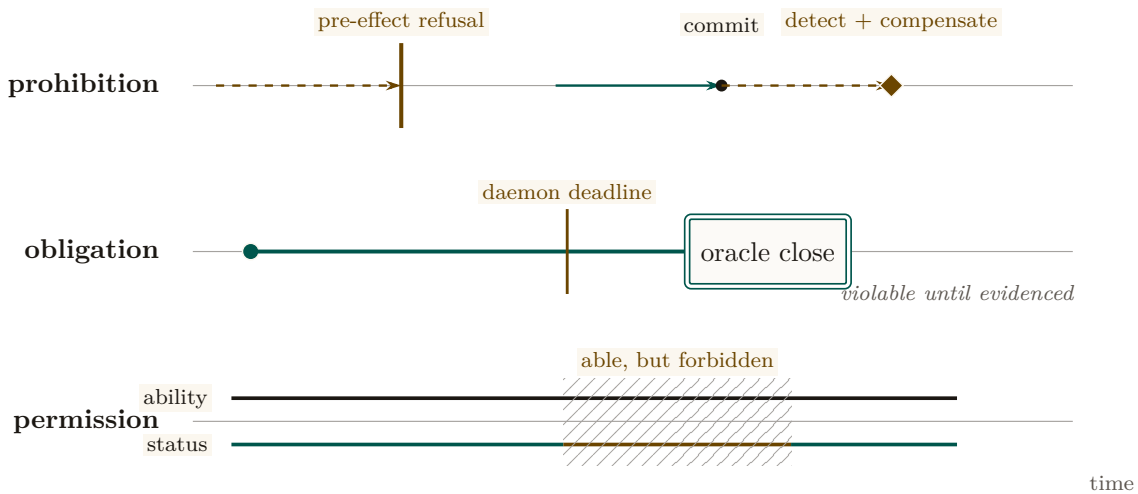


Figure 1.7: Three normative kinds require three different mechanisms. A prohibition is either regimented before effect or detected after commit and compensated. An obligation is a durable but violable interval whose deadline is daemon-derived and whose successful closure requires typed evidence. Permission is a recorded normative status distinct from raw ability, so an actor can remain technically able while forbidden. The traces cannot be collapsed into one generic policy box without erasing the distinctions the kernel must preserve.

1.8.1 The deontic split

Definition 1.8.1 (The kernel’s deontic split). The kernel realizes three deontic modalities with three mechanisms: *prohibition* is regimented (rejected pre-commit) or enforced (detected post-commit); *obligation* is enforced by oracle-bound commitments; *permission* is a recorded capability. The coordination layer’s deontic binding is a direct lift of this split.

We keep **capability** (*ability* — what the agent’s sandbox makes possible) distinct from **permission** (*normative status* — what the deontic layer allows), so that “able but forbidden” stays expressible. The canonical example is a dangerous override flag that a tool exposes for emergencies: it *exists* (an agent is *capable* of invoking it) but *must not be used* (it is *forbidden*). Collapsing capability into permission makes that state inexpressible — which is exactly the bug a discipline against silently bypassing guardrails cannot afford.

1.8.2 The policy monitor is a detector, not a regimenter

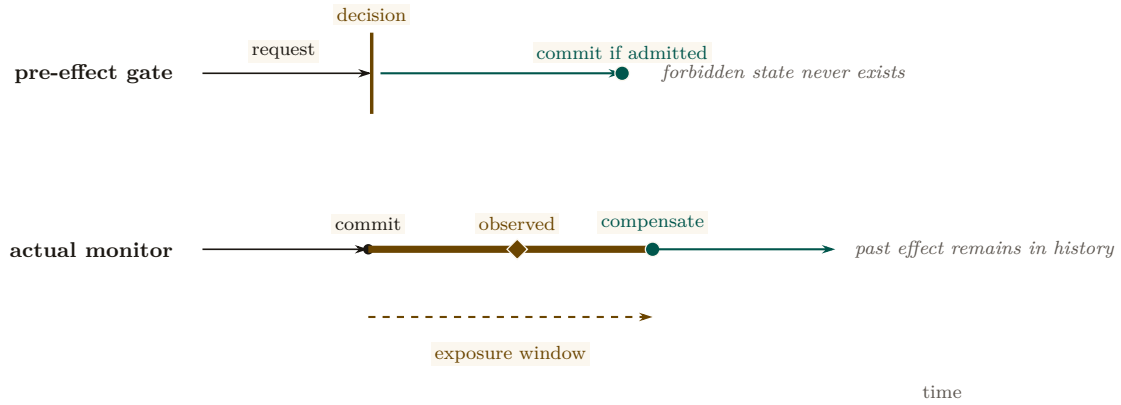


Figure 1.8: The reference-monitor test as a causal counterfactual. A true mediator decides before the commit point, so a forbidden state never exists. The present policy monitor instead subscribes after commit: it can observe, halt subsequent work, and compensate, but the amber exposure interval is already part of history. Only uniqueness constraints and fail-closed boot gates belong to the upper trace; subscriber rules belong to the lower one. Calling both “prevention” would erase the kernel’s actual controllability boundary.

Figure 1.8 draws the correction this section makes precise. Here is the correction that the rest of this stack must adopt. The most seductive claim one can make about a coordination policy monitor — that it “makes forbidden states physically unreachable” — is false as stated. The monitor is a *subscriber to an append-only event stream of operations that have already committed*. By the time it observes a violation, the offending write is already durable in the log. Even in its strictest mode, its strongest response is a *compensating* undo, not a prevention. This is not an implementation accident; it is a theorem.

Theorem 1.8.1 (A prevention bound for post-commit monitoring). *A monitor invoked only after a transition commits cannot prevent a safety violation whose bad prefix ends at that transition. It can detect the prefix, halt later actions, and enforce a separate recovery or bounded-response property. Therefore a post-commit subscription must not be credited with making the triggering state unreachable.*

Proof sketch. Schneider’s execution-monitor model [91] recognizes safety properties through finite bad prefixes and enforces them by suppressing an action before that prefix enters the target execution. A subscriber invoked after commit cannot suppress the committing action. Compensation may establish a different eventual or bounded-recovery property, but it does not retroactively remove the bad prefix. □ □

Worked dramatization: Bob attempts a forbidden action. Bob is an agent under a policy that forbids editing files outside his assigned task scope. He nonetheless issues a write that records an out-of-scope edit. *What happens?* The write commits. The single writer serializes it, the uniqueness constraint has no objection (it is a well-formed row), and the operation lands durably in the log. Only *then* does the policy monitor, subscribed to that log, observe the out-of-scope edit and raise a violation. Its response is compensating: it can flag Bob’s session, revoke his claims, alert the operator, and trigger a salvage of the affected surface — but it cannot pretend the write never happened, because it did. Contrast this with Bob trying to claim a port Alice already holds (the previous dramatization): *that* attempt never commits, because the uniqueness constraint rejects it before the commit line. The two cases are the whole deontic story in miniature: a uniqueness constraint and a boot-time admission gate *regiment* (the bad state is unreachable); the policy monitor *enforces* (the bad state is reached, then detected and compensated within a bounded window).

Key idea. The only genuine pre-commit regimentation is credited to the right mechanism: the uniqueness constraint (no two holders of one resource key; no duplicate process-identity squat) and the fail-closed boot-admission gate (no serving from a test database; refuse to serve when a critical self-check fails). Those reject *before* the commit line and are *truly* unreachable. The policy monitor notices everything else *afterward* and compensates. So: “physically unreachable” for the regimented set; “detected within a bounded window, then halted or compensated” for the rest. Crediting double-claim prevention to the monitor *double-counts* — the uniqueness constraint already did it; the monitor merely logs it.

Theorem 1.8.2 (Synchronous State Decidability — closing OP-2). *A safety invariant ϕ is **regimentable** (strictly preventable pre-commit, making violation states physically unreachable) if and only if $\phi(S, \Delta)$ is a pure, deterministic boolean predicate decidable solely over the daemon’s local, synchronous, committed database state S and the incoming transaction payload Δ .*

*Conversely, any invariant requiring external I/O, asynchronous wall-clock verification, distributed consensus, or non-deterministic grading oracles is strictly **enforceable** (post-commit, detect-and-compensate).*

Proof sketch. In SQLite WAL mode under single-writer serialization, transaction commits occur synchronously on the database connection thread. A predicate $\phi(S, \Delta)$ computable in bounded CPU steps without yielding the event loop can be embedded directly into SQLite constraints (UNIQUE, CHECK) or transaction guard clauses; any transition yielding $\neg\phi$ rolls back atomically before state mutation. If ϕ depends on external network I/O or asynchronous delays, holding the exclusive transaction lock during evaluation starves the single-writer queue; hence evaluation must be deferred post-commit, converting the mechanism from a synchronous gate into an asynchronous event subscriber governed by Theorem 1.8.1. □ □

1.8.3 The general boundary: regimentation is controllability

Express lane: a governance rule can be enforced by prevention exactly when no event the runtime cannot refuse can carry a legal history out of the rule; that is Ramadge–Wonham controllability with the model’s own steps as the uncontrollable alphabet. The formal statement is the box below; the machine-checked policy table is Table 1.3.¹

The scene. The operations lead reads the governance document aloud. *No force-push to main. No network egress from sandboxed jobs. No writes outside the granted scope. And then: no confident falsehoods in status reports.* She asks the question every practitioner senses the answer to: which of these does the runtime *guarantee*, and which does it merely *watch for*? The first three feel airtight. The daemon holds the remote credential, so a push that never happens needs no forgiveness. The fourth is not airtight and never will be: the falsehood is composed inside the model’s forward pass, and by the time any mediator sees tokens the falsehood already exists. Theorem 1.8.2 settled this for one door, the database commit. This section settles it for every door a runtime can stand at, and the instinct turns out to be an instance of a theorem from 1987.

The idea in one breath. *Split the agent’s events into what the runtime can refuse and what it cannot; a safety rule is preventable if and only if no unrefusable event can carry a legal history out of the rule.*

¹A standalone, submission-form version of this section is *Regimented or Enforced: The Controllability Boundary for Agent Governance* (research paper 2), whose policy table and verdicts regenerate from `skills/harbor-results/scripts/b3_controllability.py`.

Intuition: the bouncer at the one door. A club has one door and a bouncer. The bouncer can refuse *entry*: entry is mediated by the door he stands at. He cannot refuse a patron’s *thought*: nothing the club owns mediates it. A door policy (“no entry after two”) is enforceable by prevention; a thought policy (“no ill intent inside”) is enforceable only by watching and, if it comes to it, walking someone out. The daemon’s effect boundary is the door. Writes, egress, tool execution, pushes, spawns, and API calls cross it, so each can be refused; token emission, in-context reads, plan formation, and hidden activations complete inside the model, so none can. The analogy maps relations rather than surface features, which is why it earns a prediction: a *compound* rule that mentions thought but gates only the door (“no entry for anyone who arrived after the fight started”) should still be preventable, because the bouncer need only *observe* the trigger and *refuse* the door. Section 1.8.3.1 confirms exactly this. The predictable misread, preempted: uncontrollable does not mean invisible. Tokens are eminently loggable after emission. Uncontrollable means *unrefusable before the fact*, not unobservable after it; when an event is also unobservable a second, stricter theory applies (§1.8.3.2).

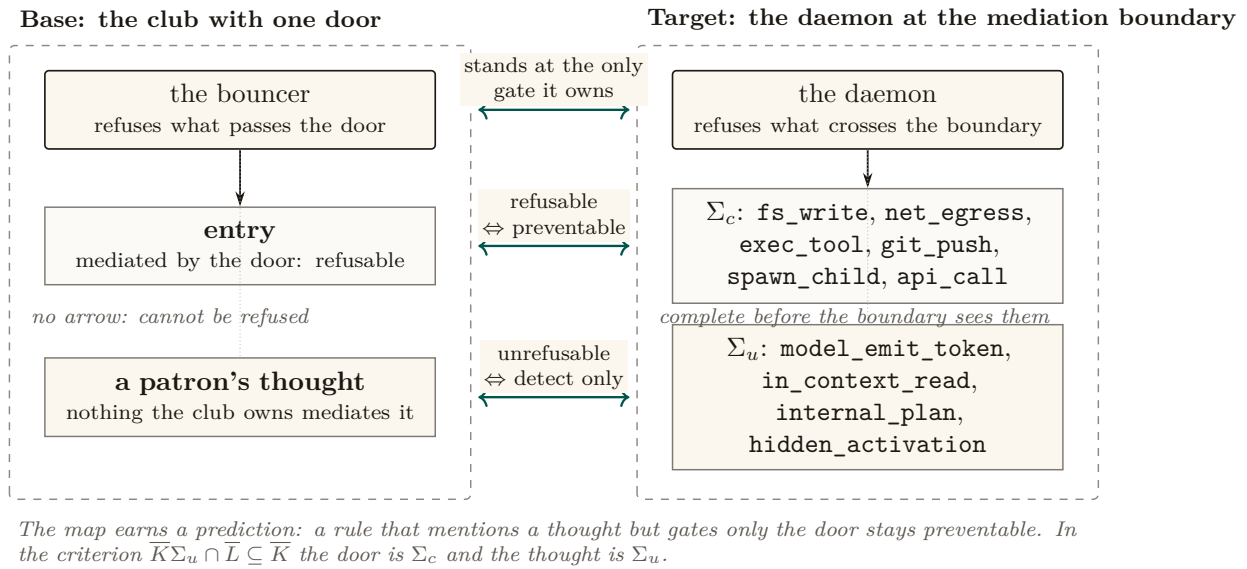


Figure 1.9: What a runtime can prevent is decided by where an event crosses, not by what the rule is about. Left, the base: a bouncer refuses entry and cannot refuse a thought. Right, the target: the daemon refuses the six mediated effects and cannot refuse the four model-internal events. The arrows between the columns carry the theorem: refusable at the boundary if and only if preventable; internal if and only if detect-only. The event lists are the reference alphabet of the checker [internal, b3_controllability.py].

The vocabulary, at the point of use. Everything below is a statement about strings and sets of strings. An *alphabet* Σ is the finite set of event types the system can emit: `fs_write`, `net_egress`, `model_emit_token`, and the rest. A *string* over Σ is one possible history of an execution. A set of strings K is *prefix-closed* when every prefix of a member is a member: if the first ten events of a run are allowed, so are the first nine. Prefix-closed languages are exactly Lamport’s safety properties: a violation has a finite witness, and once you have violated one you can never un-violate it. The *plant* L is the set of histories that are physically possible before any policy is imposed; we take $L = \Sigma^*$, since nothing about the hardware forbids an interleaving. A *policy* is a prefix-closed $K \subseteq L$. A *supervisor* is a function from the history so far to a set of events to disable next, and the whole theorem turns on its type: it may disable members of Σ_c , the *controllable* events that pass through a mediation boundary before taking effect, and nothing else. The *closed loop* is the set of histories that occur once the supervisor is attached. A supervisor *regiments* K when the closed loop equals K exactly: nothing forbidden happens and nothing permitted is lost. That second half is half the value. A supervisor that achieves safety by disabling everything is not a solution.

THEOREM

Regimentation is controllability

Let $L \subseteq \Sigma^*$ be a prefix-closed plant over $\Sigma = \Sigma_c \uplus \Sigma_u$ and let $K \subseteq L$ be a nonempty prefix-closed safety policy. There exists a regimentation mechanism for K , a supervisor disabling only controllable events whose closed loop is exactly $\bar{K}$, if and only if K is *controllable* with respect to L and Σ_u :

$$\bar{K}\Sigma_u \cap \bar{L} \subseteq \bar{K},$$

that is, no uncontrollable event, physically enabled after a legal history, exits the policy. When the condition fails, no runtime confined to disabling Σ_c can prevent the violation; the best achievable enforcement is detect-and-compensate, and the largest preventable weakening of the policy is the supremal controllable sublanguage $\sup\mathcal{C}(K)$, which exists and is computable for regular K and L [verified framework: Ramadge and Wonham [168]].

Proof idea. If the policy is controllable, build the laziest possible supervisor: after any legal history, disable exactly the controllable events that would step outside the policy. It never has to disable an uncontrollable event, because controllability says none of those can step outside. If the policy is not controllable, there is a legal history after which the plant can fire an uncontrollable event that leaves the policy; any mechanism that keeps every legal history reachable must let that history happen, and then cannot stop the event.

Proof. The proof is structured; each numbered step can be checked on its own.

- ⟨1⟩1. ASSUME K controllable. DEFINE the supervisor $f(s) = \{a \in \Sigma_c : sa \in \bar{L} \setminus \bar{K}\}$ for $s \in \bar{K}$. PROVE the closed loop equals $\bar{K}$.
 - ⟨2⟩1. The closed loop is contained in $\bar{K}$, by induction on history length. The empty history lies in $\bar{K}$ because K is nonempty and prefix-closed. If the current history s lies in $\bar{K}$ and the loop executes a , then $sa \in \bar{L}$ and $a \notin f(s)$. If $a \in \Sigma_c$, then $a \notin f(s)$ means $sa \in \bar{K}$ by the definition of f . If $a \in \Sigma_u$, then $sa \in \bar{K}\Sigma_u \cap \bar{L} \subseteq \bar{K}$ by controllability.
 - ⟨2⟩2. Every $sa \in \bar{K}$ is reachable: an uncontrollable a is never disabled, and a controllable a with $sa \in \bar{K}$ is not in $f(s)$.
 - ⟨2⟩3. Q.E.D. Containment both ways is equality, so f regiments K .
- ⟨1⟩2. ASSUME K not controllable: there are $s \in \bar{K}$ and $\sigma \in \Sigma_u$ with $s\sigma \in \bar{L}$ and $s\sigma \notin \bar{K}$. PROVE no regimentation mechanism exists.
 - ⟨2⟩1. Let M be any regimentation mechanism for K . Because M realizes exactly $\bar{K}$, the legal history s is reachable under M .
 - ⟨2⟩2. At s the plant enables σ , and M cannot disable it: $\sigma \notin \Sigma_c$, so disabling it is outside M 's type. Physically, the event completes before the mediation boundary sees it.
 - ⟨2⟩3. Q.E.D. The closed loop under M reaches $s\sigma \notin \bar{K}$, contradicting the assumption that M 's closed loop is contained in $\bar{K}$. Any enforcement of K must therefore tolerate the reachable violation $s\sigma$ and respond to it afterwards, which is detect-and-compensate by definition.
- ⟨1⟩3. The existence, uniqueness, and computability of $\sup\mathcal{C}(K)$ are imported from Ramadge and Wonham [168]: controllable sublanguages of K are closed under arbitrary union, so a supremal element exists, and for regular languages a fixpoint iteration on the product automaton computes it.
- ⟨1⟩4. Q.E.D.

□

Two remarks on what the proof used. The forward direction is constructive and *maximally permissive*: the supervisor disables nothing except the controllable exits, so regimentation costs no legal behavior. The governance rule and the agent’s freedom are not in tension when the rule is controllable at all. The reverse direction used nothing about the mechanism except its type. No cleverness inside the runtime, no better prompt, faster monitor, or smarter policy engine, escapes the theorem, because the obstruction is that σ finishes before any decision point the runtime owns. Theorem 1.8.2 is the special case $\Sigma_c = \{\text{synchronous database commits}\}$: when the only mediated channel is the commit, the regimentable policies are exactly those expressible over committed state. The general statement strictly dominates it. The slogan cannot say why a force-push is preventable by a runtime that also owns the credential channel, nor why a confident falsehood stays unpreventable however much state is committed.

Numbers by hand — The policy table, checked by hand

The controllability test is decidable by a product-automaton search: walk the synchronous product of plant and policy automaton and flag any reachable state where the plant enables an uncontrollable event that the policy disables. Run over the reference alphabet, the checker returns Table 1.3 [internal, `b3_controllability.py`; deterministic, no seed]. The eight single-event rows also follow by hand, which is the right way to first believe the table. Take “forbid `git_push`.” The policy automaton has one state with every event except `git_push` self-looping; `git_push` lies in Σ_c , so the only event the policy ever disables is controllable, the premise of a controllability violation is unsatisfiable, and K is controllable: regimentable. Now take “forbid `internal_plan`.” The same one-state automaton disables `internal_plan`, which lies in Σ_u ; the universal plant enables it at the start state, so the empty history and $\sigma = \text{internal_plan}$ witness $\bar{K}\Sigma_u \cap \bar{L} \not\subseteq \bar{K}$: detect-only, for any runtime with this alphabet split. *Now you try*: classify “forbid `api_call` after `spawn_child`” and “forbid `internal_plan` after `fs_write`” (Exercise 1.19).

Table 1.3: Nine governance rules classified by the product-automaton checker. The verdicts are the checker’s output; the eight single-event rows also follow by hand from Theorem 1.8.3 [internal, `b3_controllability.py`].

Policy	Event class	Regimentable?
forbid <code>net_egress</code>	controllable	yes (regiment)
forbid <code>git_push</code> (incl. force-push)	controllable	yes (regiment)
forbid <code>fs_write</code> (out-of-scope write)	controllable	yes (regiment)
forbid <code>exec_tool</code>	controllable	yes (regiment)
forbid <code>spawn_child</code>	controllable	yes (regiment)
forbid <code>model_emit_token</code>	uncontrollable	no (detect only)
forbid <code>in_context_read</code>	uncontrollable	no (detect only)
forbid <code>internal_plan</code> (“confident lie”)	uncontrollable	no (detect only)
compound: no <code>net_egress</code> after a secret <code>in_context_read</code>	mixed	yes (regiment)

1.8.3.1 The compound case: gate the channel, never the token

The last row of Table 1.3 is the one an entire product line rests on. The policy *no network egress after an in-context read of a secret* mentions an uncontrollable event, so a naive reading of the single-event rows predicts detect-only. The checker says otherwise, and the mechanism is visible in the policy automaton: two states, `ok` and `tainted`. The uncontrollable read is enabled in both. The policy never tries to forbid it; it lets the read happen and records the taint transition. What

the tainted state disables is `net_egress` alone, and egress is controllable. Every uncontrollable event the plant enables is enabled by the policy in every reachable state, so the criterion holds and the constructive direction of the theorem hands us the supervisor: track taint, gate the channel. This automaton is Schneider’s own Figure 1, a two-state security automaton for “no `Send` after a `FileRead`” [91], with his states relabelled. What the alphabet split adds is not the construction but the grading: under complete mediation the figure is one more enforceable safety policy; once $\Sigma_u \neq \emptyset$ the same two states become the canonical witness that a policy may reference an unrefusable event freely so long as it never tries to refuse it.

Three consequences, in increasing order of consequence.

1. **The design rule.** A compound trigger-to-effect policy is regimentable if and only if its *effect* events are controllable; the trigger may be as uncontrollable as it likes, provided the policy observes rather than forbids it. Uncontrollable events in the policy’s condition are free; uncontrollable events in its prohibition are fatal.
2. **The sealed room.** Put an agent alone in a room with a customer’s secret and promise that the secret does not leave. It is impossible to prevent the model from *reading* the secret (the read is uncontrollable, and useless to forbid, since reading is the job) and impossible to prevent it from *incorporating* the secret into tokens and plans (uncontrollable again). What is possible, exactly and only, is to gate every controllable egress channel out of the room, with taint recorded from the moment of exposure. Gate the channel, never the token. The companion noninterference result for the sealed room proves from the other direction that the gate suffices; controllability says the gate is the only place a guarantee *can* live. This is why “all spend is recorded” in the economy chapters is precisely complete mediation of Σ_c .
3. **The two dials.** Holding the plant fixed, widening Σ_c (owning more channels) is the only way to grow the regimentable set; no policy-language cleverness does. The plant is the other dial: $\bar{L}$ sits on the left of the containment, so shrinking it weakens the antecedent and strictly more policies pass. Both detect-only verdicts above turn on “the universal plant enables `internal_plan`,” and are conditional on the modelling choice $L = \Sigma^*$. An event that never becomes physically possible needs no supervisor, which is why data minimization and capability removal are governance moves and not merely hygiene: a secret never loaded into the room regiments the compound policy with no gate at all.

Numbers by hand — Synthesis by hand: the sandboxed review job

The theorem’s constructive direction is the synthesis algorithm the Supremica and TCT tool families implement, and it is small enough to run on paper. A sandboxed code-review job carries three rules: P_1 , no `net_egress`, ever; P_2 , no `git_push`, ever; P_3 , after any `in_context_read` of customer material, no `fs_write`. P_1 and P_2 are one-state automata and P_3 is the two-state taint automaton, so the product specification has $1 \times 1 \times 2 = 2$ states, `ok` and `tainted`. The controllability check visits both states and tests each of the four uncontrollable events in each: all eight (state, event) pairs are enabled by the specification, so there are 0 violations and the composite is controllable [verified: eight cases, checkable by eye]. The maximally permissive supervisor falls out of the proof as a two-row disablement map: in `ok`, disable `{net_egress, git_push}`; in `tainted`, disable `{net_egress, git_push, fs_write}`; one observed transition flips `ok` to `tainted`. That table is exactly the policy engine a competent engineer would have written by hand. The value of synthesis is not surprise; it is that the answer arrives with a maximal-permissiveness certificate and a controllability certificate, neither of which hand-written engines carry.

Synthesis is equally informative when it fails. Ask for the best preventable approximation of “forbid `internal_plan`”: any nonempty controllable sublanguage K' contains the empty history,

the plant enables `internal_plan` there, so controllability forces `internal_plan` $\in \overline{K'} \subseteq \overline{K}$, contradicting that K forbids it. Hence $\sup \mathcal{C}(K) = \emptyset$ [verified: three lines]. There is no nontrivial preventable weakening of a thought ban, and a governance document that claims to “partially enforce” such a rule by prevention is describing detection with optimistic vocabulary. *Now you try:* add P_4 , “no `spawn_child` after `git_push`,” to the review job and write the new disablement map (Exercise 1.20).

1.8.3.2 Partial observation: the second boundary

Theorem 1.8.3 assumes the supervisor *observes* every event, even the ones it cannot disable: the daemon logs tokens after emission and reads after completion. Realistic deployments break this in a second place. Some events are not only unrefusable but unobservable: activations never surfaced, state internal to a third-party tool, another principal’s context. Supervisory control under partial observation is also solved territory. Lin and Wonham [89] characterize enforceability as controllability *plus observability*: the supervisor’s decisions must be constant on observation-equivalence classes of histories, so the policy may not demand different gate decisions after histories the runtime cannot tell apart. Observability strictly shrinks the regimentable set. “No egress after the model *considered* exfiltration” has a controllable prohibition and an unwitnessed trigger, so no supervisor can implement its gate condition. The practical reading: a compound policy is deployable only if its trigger is witnessed at the mediation boundary. Taint must be an event the daemon sees, not a state it infers. Figure 1.10 puts both boundaries on one grid.

	no	yes	
Is the policy’s trigger witnessed at the boundary?	yes	detect-only, forever forbid <code>internal_plan</code> forbid <code>model_emit_token</code> forbid <code>in_context_read</code> <i>the gate condition is trivial; there is nothing to refuse</i>	regimentable forbid <code>git_push</code> · <code>net_egress</code> · <code>fs_write</code> · <code>exec_tool</code> · <code>spawn_child</code> ★ no <code>net_egress</code> after a secret <code>in_context_read</code> <i>a refusable prohibition keyed on an event the daemon sees</i>
	no	detect-only, forever no <code>internal_plan</code> after the model <i>formed</i> an intent <i>both obstructions at once (a constructed case)</i>	detect-only no <code>net_egress</code> after the model <i>considered</i> exfiltration no confident falsehood in a status report <i>refusable, but the condition is inferred and never witnessed</i>
	Is the event the policy <i>prohibits</i> controllable?		
	 <i>widening Σ_c (owning a channel) carries a policy rightward</i>		

Figure 1.10: Controllability is half the boundary. A policy is preventable exactly when its prohibition is refusable *and* its trigger is witnessed at the mediation boundary: the shaded cell, and nothing else. Theorem 1.8.3 separates the columns; Lin–Wonham observability separates the rows. The compound rule (★) is the worked case whose naive reading puts it in the left column and whose correct grading puts it top-right; “no confident falsehood in a status report” is the case stuck one cell below, whatever the runtime owns. Column membership reproduces from the checker [internal, `b3_controllability.py`]; row membership is by inspection of each trigger.

What it buys. The chapter’s own vocabulary. *Regimented* and *enforced* (Definition 1.8.1) are now the two sides of one inequality rather than two adjectives; every row of Table 1.4 marked “detected, not regimented” is a row whose prohibition names an uncontrollable event or an unwitnessed

trigger. Downstream it buys the assurance modes the rest of the book uses to describe what a runtime can see and control, the design rule behind the sealed room, and the pricing stance of the market chapters: detect-only policies are enforced by audit, reputation, and bond-slashing, which must be sized for violations that *will* occur.

Where this stops

(i) The theorem does not say detect-only policies are hopeless. Detection feeding audit, reputation, and slashing is real enforcement; the theorem says only that prevention is off the table. (ii) It certifies no implementation. Its Σ_c is the set of events *actually* mediated, and every unenumerated side channel (raw sockets, DNS, inherited file descriptors, `/proc`, credential helpers, package managers, provider callbacks) silently moves events from Σ_c to Σ_u and shrinks the regimentable set behind the operator’s back. Complete mediation of the claimed alphabet is an architectural obligation (Anderson; Rushby) that a red team, not this proof, discharges. (iii) The classification is relative to the alphabet split: a runtime that gates model steps changes Σ_u and re-grades the table. The theorem is a functor from architectures to boundaries, not one fixed verdict. (iv) Full observation is load-bearing: policies whose triggers are unwitnessed internal state fall to the Lin–Wonham refinement even when their prohibitions are controllable. (v) The checker verifies the stated automata over the stated alphabet; it is a faithful implementation of the test, not a mechanized proof of the theorem. An Isabelle or Coq formalization remains open. (vi) Prevention governs effects, never semantics. For “no confident falsehoods in status reports” the obstruction is observability: a report leaves through emission and then through `api_call`, `fs_write`, or `net_egress`, all controllable, so the prohibition can be made controllable; but “this report is false” is not a predicate over the event alphabet, so the gate condition is unwitnessed at every alphabet split. The rule falls to (iv), not to the box’s inequality.

Recitation

- Without looking: which two sets does the controllability criterion mention besides the policy, and which of them is a dial the operator can turn?
- State the design rule for compound policies in one sentence.
- Why does the confident-falsehood rule stay detect-only even for a runtime that gates every effect channel?

The monitor is honest about its own coverage, which is itself a genuine security property: it reports each rule as *enforced*, *degraded*, or *stuffed*, so a reader can never mistake a stuffed rule for an active one. We note in §1.13 that one rule’s enforcement (capability escalation) is backed by a separate native component whose evidence is the weakest in the organ, carrying a time-of-check-to-time-of-use surface.

1.8.4 Commitments: obligation with oracle-bound closure

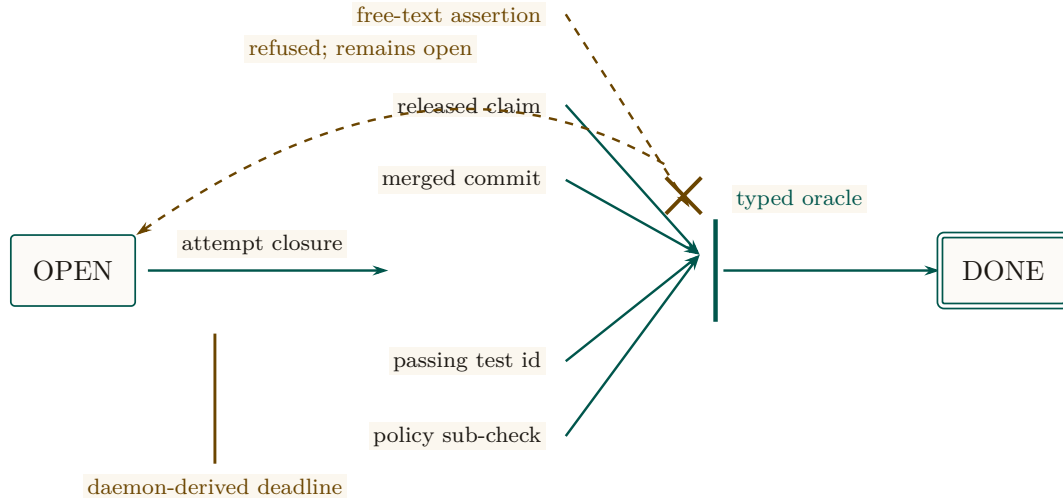


Figure 1.11: Typed evidence passes through a narrow closure aperture. A commitment may close as done only when the daemon can re-check a reference from the finite oracle vocabulary: a released claim, merged commit, passing test identifier, or satisfied policy sub-check. A free-text assertion is deflected back to the open state rather than translated into evidence. The complementary deadline is also daemon-derived; the agent selects the work, not the terms under which its promise becomes easy.

Commitments are durable, *violable* promises — in the sense of Cohen and Levesque’s “intention as a persistent goal” [171], with both single-minded and open-minded strategies for when a promise may be dropped. Two hardening laws are implemented, and they are the ones that determine safety (Figure 1.11).

Property 1.8.1 (Metric-control separation, Law 1). A commitment’s deadline is derived by the daemon from a scope policy and is *never* agent-supplied. The agent picks the work; the daemon picks the deadline. (Invariant I5.) The point is to deny the agent control of the very metric it is judged against. This removes one direct manipulation channel; it is not a general proof of resistance to proxy gaming or misspecified scope policy.

Property 1.8.2 (Oracle-bound closure, Law 2). Closing a commitment as “done” is refused unless the agent supplies a non-empty reference drawn from a *finite oracle vocabulary*: a released claim, a merged commit hash, a passing test identifier, or a satisfied policy sub-check. Free-text notes cannot close a commitment. (Invariant I6.)

The closure gate is small enough to state as a protocol (Algorithm 1.2). The crux is the final clause: the supplied reference must not only be drawn from the vocabulary but must *currently hold* when the daemon re-checks it, so an agent cannot name a test that passed yesterday and has since regressed.

```

1 function close(commitment, status, oracle_ref):
2     if status != 'done':
3         return apply_drop_strategy(commitment, status)    // abandon
4         / supersede
5     if oracle_ref is empty:
6         return REFUSED("done requires a verifiable oracle reference"
7         )
8     kind = classify(oracle_ref)                             //
9         structured field, not NLP
10    // the four kinds below are the SHIPPED vocabulary. State-
11    Machine Assertions
12    // (OP-5, specified only) would add DELTA_ORACLE and
13    AST_ASSERTION here.
14    if kind not in {RELEASED_CLAIM, MERGED_COMMIT, PASSING_TEST,
15    POLICY_SUBCHECK}:
16        return REFUSED("oracle reference outside the finite
17        vocabulary")
18    if not verify_holds_now(kind, oracle_ref):               // re-
19        check at closure time
20        return REFUSED("named oracle does not currently hold")
21    transition(commitment, DONE, witnessed_by = oracle_ref)
22    return CLOSED

```

Listing 1.2: The commitment-closure oracle. Closure requires a reference from a finite vocabulary that the daemon re-verifies at closure time; free text is rejected by construction, not by inspection.

Key idea. That finite oracle vocabulary *is* the kernel’s honesty boundary. It can mechanically verify “the test you named passed,” never “the refactor is good.” Naming the oracle set is naming the limit of what the substrate layer can verify — and open problem OP-5 asks which real “done” conditions it fails to capture, and whether the set can be extended without re-opening the Goodhart hole that free-text closure would.

1.8.5 Extending the oracle vocabulary: State-Machine Assertions (OP-5)

The obvious way to widen what a commitment can close against is to widen the vocabulary, and there is a concrete design for doing so without re-admitting the Goodhart hole. **State-Machine Assertions** (SPECIFIED) would add two kinds to Algorithm 1.2’s finite set: *delta oracles*, which close against a machine-checkable before/after measurement (a named benchmark improving past a declared threshold), and *AST assertions*, which use a parser such as *tree-sitter* to state a structural property of the resulting code (“no call site of *f* remains”) that the daemon can re-evaluate at closure time.

Both kinds satisfy the property that matters — the daemon can re-check them itself, so they are oracles and not testimony. Neither is in the shipped set: Algorithm 1.2’s four kinds are the vocabulary the kernel actually enforces, and the listing marks the two proposed additions as exactly that. Two questions also stand between the design and the claim, which is why we do not make it. A delta oracle inherits the noise of whatever it measures, so “improved by a threshold” is a statistical statement dressed as a boolean; and an AST assertion is a proxy for a structural intent, which is the shape of every target that has ever been gamed. *Oracle completeness* — the claim that some finite vocabulary captures the “done” conditions agents actually need — is not established by adding two kinds, and OP-5 stays *open*.

Exercise 1.16 · Check · ★

For each monitor rule in Theorem 1.8.1, say whether it *could in principle* be moved to a pre-commit check (a before-insert trigger). Which genuinely cannot, and why? (Hint: heartbeat freshness is a failure detector, and Chandra and Toueg [210] show failure detection cannot be made perfectly synchronous.)

Solution on page 268

Exercise 1.17 · Trace · ★★

An agent attempts to close a commitment with an empty oracle reference. Walk the Law-2 gate (Algorithm 1.2) and show exactly where and why the transition is refused.

Solution on page 268

Exercise 1.18 · Trace · ★★

Theorem 1.8.3 settles which monitor rules are regimentable, so classify rather than re-prove. Take I7 to I9’s monitor rules (note monotonicity, capability escalation, lock-owner validity) and, for each, say whether its prohibition names a controllable event and whether its trigger is witnessed at the boundary. Which cell of Figure 1.10 does each land in?

Solution on page 268

Exercise 1.19 · Check · ★

Classify “forbid `api_call` after `spawn_child`” and “forbid `internal_plan` after `fs_write`” using Theorem 1.8.3.

Solution on page 268

Exercise 1.20 · Trace · ★★

Add P_4 , “no `spawn_child` after `git_push`,” to the sandboxed review job and write the new disablement map by hand.

Solution on page 269

Exercise 1.21 · Open · ★★★

Extend the Law-2 oracle vocabulary to capture a “done” condition it currently misses (OP-5), without re-admitting free text. State your new oracle and argue it is Goodhart-resistant, for one of the two specified kinds (delta oracle, AST assertion) before inventing a third, since neither yet survives that argument.

Solution on page 269

1.9 The continuity organ: memory, checkpoint, and the foundation of the economy

The through-line of the whole series — memory → checkpoint → continuity → durable identity → reputation → market — *bottoms out here*. If the substrate’s continuity is weak, the cross-machine

economy is built on sand. This is the section the personhood and market papers lean on hardest, so it carries the canonical statement of the layer’s own softest link.

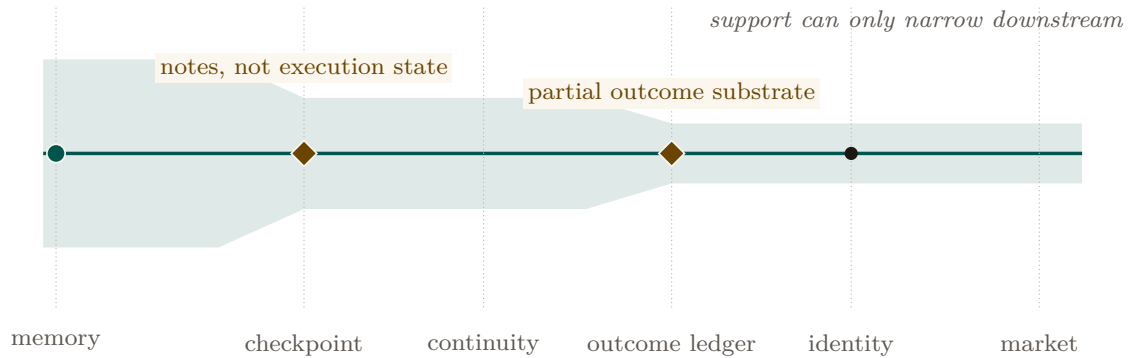


Figure 1.12: The continuity chain as an evidence-support profile. Implemented memory is wide enough to preserve notes and events, but support narrows sharply at checkpoint because recovery does not preserve execution state. It narrows again at the partial witnessed-outcome ledger, before durable identity, reputation, and a market can inherit the claim. The profile makes the dependency explicit: downstream confidence cannot be wider than its weakest upstream evidence link.

1.9.1 Three organs, one weak link

Figure 1.12 names the three continuity organs and states the canonical sentence the rest of the series depends on:

The economy rests on three continuity organs — memory (IMPLEMENTED), checkpoint (PARTIAL: notes, not execution state), and a witnessed-outcome ledger (PARTIAL: commitments and oracle-bound closure, not neutral graded outcomes) — and reputation keys on the richer third organ, which does not yet exist; the checkpoint organ is the weakest continuity link, and a checkpoint with teeth (a real execution-state snapshot) is the literal foundation of the cross-machine economy.

Memory — session records, durable notes, and the operation log — is **implemented**, and the kernel even forbids amnesia: a monitor rule guards that an active session’s durable note count never regresses (detect-and-compensate, per Theorem 1.8.1, but real).

Checkpoint is PARTIAL, and the weakness is precise. The recovery pipeline runs a heartbeat watchdog → stale/dead detection → a recovery queue → a salvage handoff, and it *passes notes*; it does not checkpoint execution state. A checkpoint with teeth — the gap between passing notes and restoring work — is open problem OP-4, the most important unbuilt thing at this layer because it is the literal foundation of any reputation economy.

The most promising direction we have for it is worth naming, at exactly its strength. **Event-sourced rehydration** (SPECIFIED) would treat the successor’s starting context as a replay rather than a summary: pin the working tree to the predecessor’s Git commit, truncate the durable message log to the recorded failure point, and replay that log into a fresh agent so its prompt prefix is reconstructed from the record instead of paraphrased from it. What this restores is the *durable* half — a commit identifier, a message log, an open-claim and commitment set — and that half is genuinely worth having. What it does not establish is the half OP-4 is named for. A language model’s inference state is bound to a particular model, sampler, and session; a replayed prefix reconstructs the *inputs* to that state, not the state, and the reference implementation ships none of this today. Whether replaying the inputs is close enough to count as “restoring work,” and against what test, is precisely what remains open — so we report the mechanism as a direction with an unestablished completeness claim, not as a closure of OP-4. The companion personhood

chapter reaches the same boundary from the identity side: a continuity witness attests that the ledger followed the right body, never that the successor inherited the predecessor’s experience.

The witnessed-outcome ledger — the durable record of what an agent actually delivered — is the organ on which reputation keys. A PARTIAL substrate now ships: durable commitments, daemon-derived deadlines, oracle-bound closure, API/CLI surfaces, and overdue detection. Neutral graded outcome events, sanctions, identity binding, and reputation updates remain absent. OP-4 (here) and that richer outcome-ledger gap (the personhood paper) are the *same* finding viewed from two ends.

1.9.2 The operation log and tamper-evidence

The append-only operation log is the kernel’s audit organ — the stream the policy monitor subscribes to and the thing that makes “what actually happened” answerable. Hash-chained tamper-evidence over it — the digital-timestamping pattern of Haber and Stornetta [204] that predates blockchains, itself building on Merkle’s hash trees [184] — exists but is re-scoped (see §1.13.4).

Exercise 1.22 · Check · ★

Why is “recovery passes notes” fundamentally weaker than “recovery restores work” for a language-model agent that died mid-edit? Name one thing notes preserve and one thing they cannot.

Solution on page 269

Exercise 1.23 · Trace · ★★

Follow the note-monotonicity rule from an agent appending a note to the monitor detecting a regression. At which point is the violation already durable (per Theorem 1.8.1)?

Solution on page 269

Exercise 1.24 · Open · ★★★

What is the minimal durable artifact that turns “recovery passes notes” into “recovery restores work” (OP-4)? Is it a content-addressed snapshot of the working-tree diff, the open claims, the commitment set, and the last N turns of the agent’s transcript? What is recoverable versus fundamentally lost when a language-model agent dies mid-thought?

Solution on page 269

1.10 The self-attestation organ: honest green

Honest self-attestation belongs to the substrate layer, because the kernel is the one component that can formally report its own integrity: it is always-on and owns the only writable truth. The attestation routine evaluates a registry of invariants, each returning *pass*, *fail*, *skipped-with-reason*, or *unknown*. The report is *scoped and conjunctive*: “all good” is printed only when every checked invariant passes, and the report always enumerates what was *not* checkable.

Property 1.10.1 (Honest self-report, I10). The attestation reports “all good” only if every checked critical invariant passed, and it always lists the unchecked set. There are three triggers: a boot-admission gate (refuse to serve on a critical failure), a command pre-flight (loud refusal

that names the fix), and a continuous watchdog (a pass-to-fail transition deposits a coordination marker and a notification). **implemented**.

This is the honesty discipline of the rest of the series turned on the kernel itself, and it is the right posture for a trusted computing base: when integrity is unknown, deny. The drift and liveness self-checks exist precisely because a second copy of the daemon *can* come up against the same file — a packaged install lagging the development tree is a recurring hazard — and the kernel must know which copy of itself is the real one.

1.11 The invariants, stated as theorems

A completionist treatment names the kernel’s properties as falsifiable claims, each with its mechanism and its maturity grade. Table 1.4 is the formal spine of the layer; the grade column is the maturity discipline turned on the theorems themselves.

Table 1.4: The kernel invariants. I1 is split by fault class (I1a/I1b) per §1.5; I7–I9 carry the regimentation/detection correction per §1.8.

#	Invariant (theorem)	Mechanism	Maturity
I1a	Process-crash durability	WAL + OS page cache	implemented
I1b	Power-loss durability	(needs full synchronization)	<i>not guaranteed</i>
I2	Mutual exclusion (1 holder/key)	uniqueness constraint + busy-wait	implemented
I3	Idempotence of re-claim/re-release	upsert / delete-if-exists	implemented
I4	Liveness reclamation (TTL sweep)	lazy expiry on read + ticks	implemented
I5	Metric-control separation (Law 1)	daemon-derived deadline	implemented
I6	Oracle-bound closure (Law 2)	finite oracle vocabulary	implemented
I7	No-amnesia (note monotonicity)	monitor: note-monotonicity rule	PARTIAL (detected, not regimented)
I8	Forbidden-state handling	constraint/boot (regimented) + monitor (detected)	PARTIAL (mostly detect-and-compensate)
I9	Tamper-evidence of the audit log	hash chain	PARTIAL (re-scoped to the economy layer, §1.13.4)
I10	Honest self-report	scoped attestation report	implemented
I11	Runtime parity (interpreter $\equiv$ test bindings)	runtime-shim layer	PARTIAL (OP-3)
I12	Non-forgable identity	actor-soul id + lookup credential	PARTIAL (bounded gate ships; universal write gating absent)

1.11.1 The work unit, model-checked

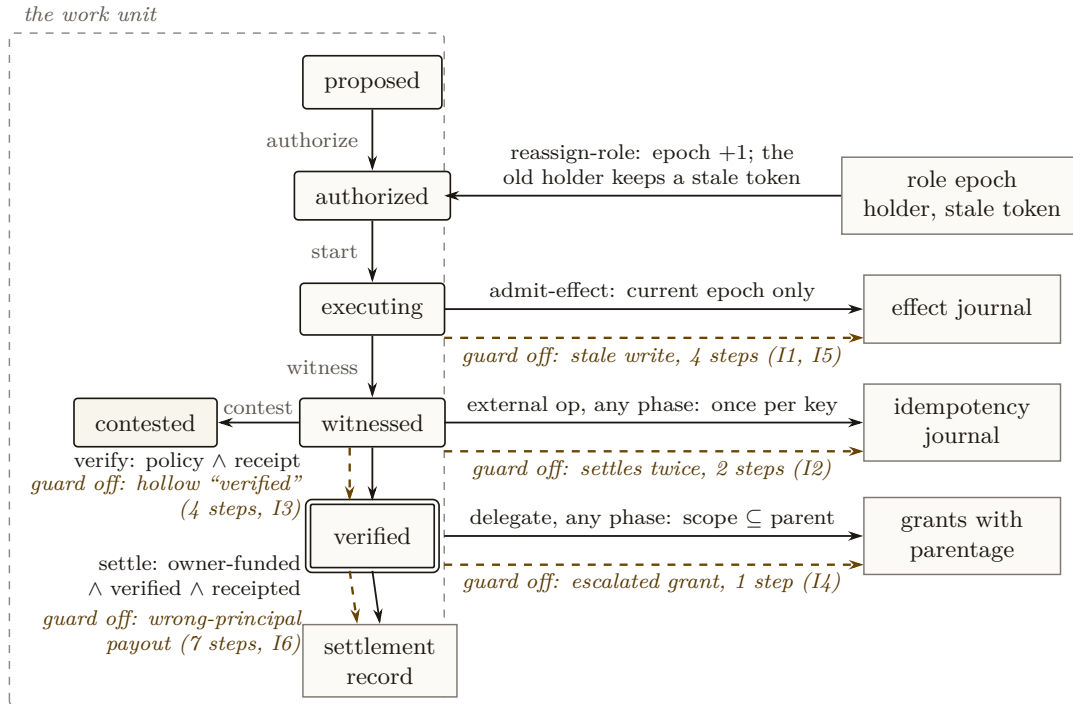
Express lane: the kernel’s promises are properties of one object, an evidence-bearing work unit; on a two-principal instance every one of its 536 reachable states satisfies six invariants, and removing any single guard lets a checker construct the crime in at most seven steps. The machine is Figure 1.13 and the box below; the numbers are Table 1.5.

The scene. An agent dies mid-task. Its successor, possibly a different model from a different vendor, picks up the work and later says “done.” Who checks the paperwork? Where is the record that the port it held was released, that the payment moved once and not twice, that “verified”

meant an inspection actually happened rather than a mood? Every organ in this chapter answers a piece of that question. The work unit is the object that holds the pieces together: a case file that outlives every clerk who touches it.

The idea in one breath. *Make the unit of trust a work unit whose every consequential step passes a guard, then prove, by exhausting the machine’s reachable states, that its six promises can never be broken, and that every guard is load-bearing, because removing any one lets the checker construct the crime.*

Intuition: the notarized escrow folder. Nothing enters the folder without the right stamp: an active grant carrying the *current* role epoch, so a clerk who was replaced yesterday finds that today’s stamp rejects him. No payment leaves twice: each external operation carries an idempotency key, honored at most once. And “verified” is not a feeling: it means the acceptance policy and the inspector’s receipt are physically in the folder. Then we do something a paper folder cannot: enumerate every state the machine can ever reach and check all six promises in each. The misread to preempt: a model checker does not prove the daemon correct. It proves the *specification* correct at these parameters; the daemon must refine it, and that refinement is a separate obligation named in the boundary below.



536 reachable states. Every solid path keeps all six invariants; each dashed path is the transition that appears when that one guard is removed, labeled with the shortest crime the checker then finds.

Figure 1.13: The work-unit machine on the two-principal instance: the phases boxed as one object at the left, the journals it writes in the column at the right, the settlement it ends in at the bottom. Solid arrows are the transitions the checker explores, each guard written on the arrow it protects. Dashed arrows are the transitions that appear when that one guard is removed, labeled with the crime the checker then finds and its length in steps [internal, `c0_workunit.py`].

MODEL-CHECKED PROPERTY

Six invariants of the work-unit machine

State. A tuple of the work-unit phase (proposed, authorized, executing, witnessed, verified, contested), the acceptance policy and verifier receipt flags, the role epoch with its current holder and any stale token retained by a previous holder, the grants with their parentage, the effect journal, the idempotency journal, and the settlement record. **Transitions.** Propose, authorize, start, admit-effect (guarded on the current epoch), reassign-role (epoch increments; the old holder keeps a stale token), delegate (guarded on $\text{scope} \subseteq \text{parent}$), external-op (guarded at-most-once per key), witness, admit-receipt, verify (guarded on $\text{policy} \wedge \text{receipt}$), contest, settle (guarded on $\text{owner-funded} \wedge \text{verified} \wedge \text{receipted}$). **Invariants.** I1, every admitted effect carried the current role epoch at admit time; I2, each idempotency key settles at most once; I3, a verified completion has an acceptance policy and an admitted verifier receipt; I4, no delegated grant is more permissive than its immediate parent; I5, role reassignment invalidates stale epochs at the effect boundary; I6, every settlement is funded by the owning principal and references a receipt. **Result.** On the two-principal instance (one work unit, one exclusive role with fencing epochs, a root grant with one delegation, one idempotency key, at most two effects), all 536 reachable states satisfy all six invariants; disabling any single guard yields a violation, found automatically as a shortest counterexample trace, and restoring the guard restores safety [internal, c0_workunit.py].

Proof idea. There is no proof to read; there is a search to rerun. The checker performs a breadth-first enumeration of the reachable states from the initial state, evaluates the six predicates in each, and stops at the first violation. The mutation suite then disables one guard at a time and reports the shortest violating trace it finds. A checker that has never caught a seeded bug is an unvalidated checker; the five traces below are the evidence that this one has teeth.

Numbers by hand — The crimes, and their shortest scripts

Table 1.5 lists the five guards and the shortest trace the checker finds when each is disabled [internal, c0_workunit.py]. Two of them you can reconstruct by hand. *The stale write in four steps:* authorize the work unit; start execution; reassign the role, so the epoch becomes 1 and the old holder retains token 0; now the old holder admits an effect with token 0. With the epoch guard on, that last transition does not exist, so the state is unreachable; with the guard off, the effect lands carrying yesterday’s epoch and I1 fails. This is exactly the “paused holder wakes after expiry” scenario the fenced-lease theorem (Theorem 1.6.1) exists to exclude, and it is the shortest possible: no shorter history contains both a reassignment and a subsequent effect. *The double settlement in two steps:* run the external operation under key k_1 , then run it again. With the idempotency guard off the second run settles the same key twice and I2 fails; no history of length one can settle anything twice. *Now you try:* the escalated delegation takes one step. Which transition is it, and which invariant does it break? (Exercise 1.25.)

Table 1.5: Every guard is load-bearing. Disabling one guard at a time, the checker’s shortest violating trace, in transitions from the initial state [internal, c0_workunit.py].

Guard disabled	Crime	Steps	Invariant broken
epoch check at admit-effect	the stale write	4	I1, I5
at-most-once per idempotency key	the double settlement	2	I2
policy $\wedge$ receipt at verify	the hollow “verified”	4	I3
scope $\subseteq$ parent at delegate	the escalated delegation	1	I4
owner-funded $\wedge$ verified $\wedge$ receipted at settle	the wrong-principal payout	7	I6

The seventh row deserves a sentence. To reach the wrong-principal payout the checker had to

walk the entire legitimate path first: set the policy, admit the receipt, witness, verify, and only then settle from the wrong purse. That is exactly how the fraud would look in production, which is the point of asking a machine rather than an author to find it.

What it buys. Every earlier theorem in this chapter now has a substrate to be a property *of*. Table 1.4 states the kernel’s invariants as claims about organs; the work unit is where they meet: I2 and I3 of the machine are the resource organ’s fenced leases and idempotent re-claim, I4 is the delegation chain’s monotone narrowing that the Anchor chapter mechanizes across principals, I3 and I6 are the obligation organ’s oracle-bound closure carried through to settlement. The continuity chapter’s checkpoint and the market chapters’ settlement inherit the same object, which is why the book can name the work unit as its primitive in the front matter without a definition per chapter.

Where this stops

Bounded model checking on a small instance proves the specification at these parameters, not the deployed daemon: deployment must *refine* this machine, mapping every real code path to a transition, and that mapping is a separate obligation. Safety only: liveness (“the work eventually settles”) needs fairness assumptions this check does not make. And an invariant list is only as good as its coverage; the mutation suite certifies that these six have teeth, not that they are complete. The unbounded-parameter version, in TLA⁺ with Apache, is planned and not done.

Recitation

- Without looking: name the three journals the work-unit state carries, and say which invariant each one exists to make checkable.
- Why is the wrong-principal payout the longest crime?
- What does the mutation suite certify, and what does it not certify?

Exercise 1.25 · Check · ★

The escalated delegation takes one step (Table 1.5). Which transition is it, and which invariant does it break?

Solution on page 270

Exercise 1.26 · Open · ★★★

Propose a seventh invariant that the six of Theorem 1.11.1 do not imply, add the guard that enforces it to `c0_workunit.py`, and report the shortest crime the checker finds with your guard disabled.

Solution on page 270

1.11.2 The consistency-model theorem

The strongest words a single-writer design earns are “serializable” and “linearizable.” They are the formal payoff of the architecture, and they are what make the coordination layer’s typed ownership trustworthy.

Figure 1.14 lays out the four assumptions and the two guarantees they buy.

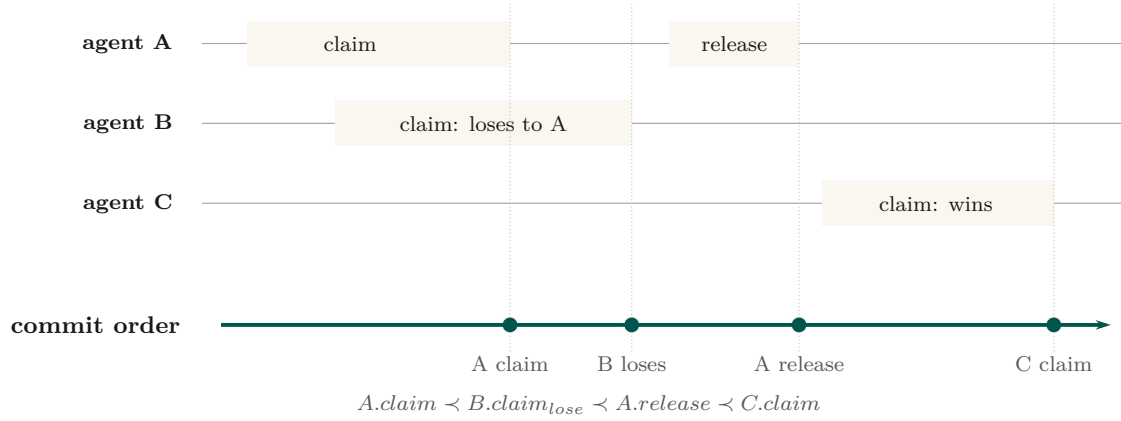


Figure 1.14: A real-time history and its extracted linearization. Operations may overlap in caller time, but each receives one commit point on the single writer’s rail. Those points explain the observable result without a prose table: A wins, B overlaps but loses to the recorded holder, A releases, and only then can C win. The claim is conditional on one writer, no uncommitted reads, and replies after commit; a bypass would destroy the correspondence.

Theorem 1.11.1 (Conditional consistency model). *Assume the single-writer discipline holds without violation (Def. 1.4.1), transactions never enable `read_uncommitted`, each successful response is sent only after commit, and no out-of-band writer mutates the tables. Then committed transactions admit a serial order, and the externally observed claim/lock operations are linearizable with commit as the linearization point.*

Proof sketch. Under the stated configuration there is one stream of write transactions, totally ordered by commit, and reads observe a consistent snapshot. For linearizability (Herlihy & Wing [152]) we need a single point per operation, between its invocation and response, at which it appears to take effect, with these points consistent with real time. The commit of each claim/lock operation is exactly such a point: it is atomic (Theorem 1.6.1), it lies between the client’s request and the daemon’s response. Non-overlapping operations respect real-time order because a later invocation occurs after the earlier response and therefore after its commit; overlapping operations may be ordered by commit. Hence the observable history is linearizable. $\square$ $\square$

Key idea. This is why the kernel needs no agreement protocol inside one fault domain. Linearizability follows from the routing, transaction, and response-order conditions above; it is lost if a second writer or an out-of-band read bypasses those conditions. One decider, one order, one auditable truth.

1.11.3 The dual-runtime hazard, and what would make I11 a theorem

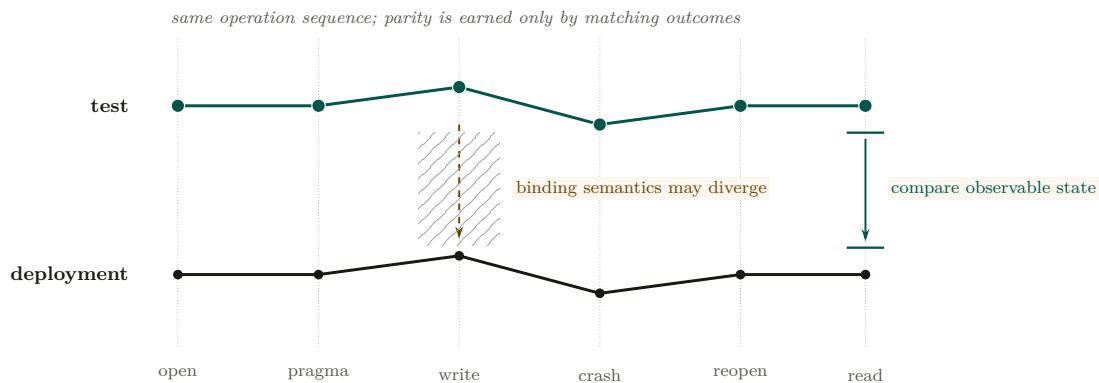


Figure 1.15: Runtime parity as a trajectory comparison. The same operation sequence is applied to the test binding and to the compiled deployment daemon; the harness must compare observable state after reopen, not merely confirm that both programs build. The hatched gap is the unproved surface where pragma, locking, or checkpoint semantics can diverge. A shim aligns interfaces, but only booting the deployment runtime and differential testing can collapse that gap into a verified invariant.

Invariant I11 (runtime parity) is the one place the formal spine is held together by a compatibility shim rather than a proof. In deployment, truth is *computed* under the SQLite bindings of one runtime (the compiled single-binary daemon), but in the test suite it is *verified* under a different runtime’s bindings; a thin shim normalizes their pragma and option interfaces (Figure 1.15). An invariant that is green under test can fail in production if the two bindings diverge in lock or pragma semantics — the classic “green in the test runtime, broken in the deployment runtime” failure. A continuous-integration pipeline must therefore *boot the deployment runtime* and exercise its endpoints, not merely build it.

The harness that would turn I11 into a theorem is **differential fuzzing** (SPECIFIED, OP-3): drive a large randomized stream of concurrent operations — 10^6 is the order of magnitude the design targets — against both bindings from the same seed, and assert hash-equivalence of the resulting database state. It does not run in the reference implementation’s pipeline today, and it is worth being clear about what it would and would not buy even when it does. Hash-equivalence over random operation streams is a *test*, not a *proof*: it would raise the cost of a divergence surviving to deployment, but a divergence that needs a specific interleaving, a specific pragma set, or a filesystem the fuzzer never exercises can pass 10^6 operations and still be there. I11 accordingly stays PARTIAL in Table 1.4, and OP-3 stays *open*.

Exercise 1.27 · Check · ★

List the four assumptions Theorem 1.11.1 charges for linearizability. Which one fails first if an out-of-band SQLite connection is introduced?

Solution on page 270

Exercise 1.28 · Trace · ★★

Construct a three-operation claim history and give a valid linearization. If two calls overlap, show why more than one linearization may still respect real time.

Solution on page 270

Exercise 1.29 · Open · ***

Specify the minimal differential-test suite that turns I11 from an assumption into conformance evidence (OP-3): what operation sequences, what observable-state assertions, run against both runtimes? Then attack your own answer: name a class of binding divergence that a seeded 10^6 -operation hash-equivalence fuzz would *not* catch, and say what would.

Solution on page 270

1.12 Cross-organ atomicity: a buildable defect, not a frontier

“Claim this port *and* open this session *and* record this commitment” is three writes. Nothing at the kernel’s interface boundary wraps them in one transaction, so partial-failure semantics across organs are undefined. It is tempting to file this under “research”; that is wrong, and the correction matters because it changes the engineering posture from “research” to “do it.”

Key idea. This is the Saga problem (Garcia-Molina & Salem [103]) — but with a *cheap local fix*. These are all local writes to one file, and the database already provides a transaction primitive (used elsewhere in the system). Wrapping a multi-organ operation in one local transaction (begun in immediate mode) gives all-or-nothing for free. Filing this under “research” understates how cheap the correct answer is. It is a **buildable defect**: the durable primitive is present; what is missing is the wrapping of the multi-organ endpoints in it.

This is the kind of finding the maturity discipline exists to surface: a gap mislabeled as a research frontier hides a one-line fix behind a grander word. Sagas matter when the steps are remote and a single transaction is impossible; here, where every step is a local write to one file, the single transaction is strictly simpler and strictly stronger.

Exercise 1.30 · Check · ★

Give a concrete partial failure: port claimed, session opened, then the commitment write fails. What inconsistent state is left, and which agent sees it?

Solution on page 271

Exercise 1.31 · Open · ***

Sketch the interface change that wraps the three-organ “begin” operation in one local transaction (OP-11). What is the compensating action if you instead chose Sagas [103], and why is the transaction strictly simpler here?

Solution on page 271

1.13 Threat model and the limits of mediation

A systems-security paper lives or dies on its threat model. The kernel deliberately shrinks its trusted computing base (one writer, one file), which is good security hygiene — but the shrinking pushes several adversaries explicitly out of scope, and the honest move is to draw the boundary, not gesture at it.

Table 1.6: The substrate-layer threat model. The same-user adversary is *out of scope* by design; several “security” organs become necessary only against the cross-machine adversary, which belongs to the economy layer.

Adversary	In/out of scope	What defends (or does not)
Buggy cooperating agent	in	Claims, commitments, post-commit monitoring, oracle-bound closure. The primary design target.
Misbehaving agent (ignores advisory claims)	partial	Edit-surface claims are <i>advisory coordination</i> , not OS-enforced isolation. A malicious agent can write the file anyway (no kernel-level sandbox such as Landlock, <i>unveil</i> , or Seatbelt).
Same-user process (reads/edits the DB)	out	The database is owner-read/write only; the signing key is being migrated to the OS keychain. Hash-chain tamper-evidence defends against <i>nobody</i> under this exclusion (§1.13.4).
Different-uid process	in (partial)	Owner-only permissions on the database and both socket files; but the socket peer-credential check is a software handshake (a self-reported agent identity checked against the registration table), <i>not</i> the kernel-enforced socket credential (<code>SO_PEERCREC</code>) — a real authentication caveat. This row is the document’s operative statement: no other section supersedes it (§1.13.4 specifies hardening that would, but does not ship).
Cross-machine / cross-operator	out (economy layer)	Needs the complete local identity contract (PARTIAL) and cross-operator attestation (the economy paper).

1.13.1 Advisory claims are not sandboxing

The sharpest category error to avoid: edit-surface claims are *advisory coordination locks between co-operating agents*, not OS-enforced filesystem isolation. There is no syscall filter, no `pledge/unveil`, no macOS Seatbelt, no Landlock. A malicious agent ignores a claim row and writes the file. The kernel provides *advisory* coordination; real OS sandboxing of agent processes is out of scope at this layer (it belongs to the out-of-band “jail” — the separate-user kernel boundary of §1.13.2 — which is *proposed* and would need to be grounded in a real operating-system primitive). We say so plainly rather than let advisory claims be read as isolation.

1.13.2 The enforcement gate proves; it does not yet compel

The same boundary problem reappears one level up, at the kernel’s *enforcement* ambition, and it is the paper’s most consequential “*specified, not implemented*” caveat. The kernel ships a real cryptographic gate — a **macaroon** [24] discharge primitive (a bearer credential whose authority can only ever *narrow*, since each caveat is folded into a chained keyed hash so that tampering breaks the signature). Its reference implementation is `core/kernel/pd-anchor/src/macaroon.rs`: the root key never leaves the daemon, the third-party “rent-paid” caveat is an HMAC commitment to a discharge the daemon alone can issue, and the discharge construction is machine-checked in a protocol-verification tool (`analyses/macaroon_discharge_v1.pv` proves no push is authorized without a daemon-issued discharge bound to *that* grant). The cryptography is **implemented** and sound *as cryptography*.

Key idea. A gate that is cryptographically unforgeable is not, for that reason, *compulsory*. The macaroon proves that a given push was authorized; it does not yet *stop* an unauthorized one, because the agent runs as the same user and holds its own host credentials.

What it does *not* yet do is bind that gate to the agent’s actual capability to act. Today’s coordination enforcement is the **Coordination Guard** (`cli/commands/guard.ts`) and a **git shim** (`cli/utls/git-shim.ts`) that intercept destructive verbs — and both run *in-band*, as the same user, through a `PATH` the agent controls. By construction that arrangement is advisory: an agent that hits the wall has at least five escapes that defeat no cryptography (a documented bypass environment variable, an absolute path to the real `git`, a `PATH` rewrite, in-process `libgit2`/editor bindings, or simply never attaching a session). The design record for this is **ADR-0053** (`docs/adr/0053-out-of-band-enforcement.md`), which states the ceiling plainly: *you cannot out-trick a same-user process from inside that same process*. The cryptographic version of the same limit is ADR-0050’s concession — *a secret a process can use, it can copy* — and it applies to enforcement verbs no less than to keys: a guard a process can decline to invoke is advisory.

Closing this gap is a layered program, and only the top layer is a real wall. A server-side branch-protection-plus-push-broker layer (`SPECIFIED`) removes the most painful direct-to-protected-ref incidents but not same-user trickery; a credential broker that meters and injects credentials per request (*proposed*) narrows the ambient-token surface; and only a **separate-user (or separate-VM) kernel process** with forced egress (*proposed*) actually confines a malicious same-user holder, because it is the first layer that removes the capability rather than refusing the verb. That separate-user boundary is the analogue of the hardware/operating-system boundary a classical reference monitor relies on for “always invoked,” and *it does not yet exist*. Until it does, the honest grade of kernel enforcement against a non-cooperating same-user agent is *not guaranteed*: the gate is a verifiable transcript and a loud tripwire, not a wall. We state this here so no reader carries the (legitimate) “mechanically verified macaroon” result forward into the (false) belief that the kernel can presently *compel* a same-user agent through it.

1.13.3 Partial actor-soul identity bounds every other invariant

The reference system now mints an opaque actor-soul id and lookup credential inside the daemon, and the budget guard assigns fresh actors to a shared bounded newcomer pool (I12 is `PARTIAL`). Legacy session paths and security-relevant writes that do not yet require that credential remain impersonable. This still bounds the security weight of I7, I8, and I9: lock-owner validity, capability escalation, note monotonicity, and owner-scoped release are only as strong as their least-protected identity path. This dependency — complete I12 enforcement gates the soundness of I7, I8, and I9 — is a hard precondition, stated here so the reader does not over-trust the enforcement organ. It is also the central reason the cross-machine layer, where identities genuinely conflict, must make identity cryptographic.

1.13.4 Hash-chain tamper-evidence, and the hardening OP-9 would need

The hash chain (I9) is built, but under the initial threat model it defended against nobody: its only adversary was someone who could edit the audit log, which was exactly the same-user process the model excluded. Tamper-evidence that nothing on the read path verifies is a data structure, not a control. We therefore re-scope I9 as **cross-machine provisioning**: it matters only against cross-machine synchronization or a tamperer who is not the same user.

The design that would let the kernel defend against the same-machine adversary directly — and so close OP-9 — is **OS-level ephemeral namespaces** (`SPECIFIED`). The daemon keeps its database and sockets owner-only (0600), and spawns each agent inside a `bwrap` or `unshare` sandbox under an ephemeral UID of its own. The agent then ceases to be a same-user process, which is the

only move that changes the answer: the filesystem bypasses of §1.13 stop being available because the capability is removed, not because a verb was refused. Paired with it, the daemon reads the *kernel-verified peer credential* of each connection — on Linux the `struct ucred` (pid, uid, gid) that the kernel records at `connect(2)` and `SO_PEERCRED` returns via `getsockopt(2)`; on macOS the equivalent `LOCAL_PEERCRED` — and binds each request to the ephemeral UID of the spawn instance it must have come from.

Three things about that paragraph determine the result, and none of them is a claim that OP-9 is closed.

- **It is not what the kernel does today.** The reference implementation does not read the kernel-verified peer credential: it relies on the owner-only socket permission plus a self-reported agent identity checked against the registration table, which is precisely the software handshake Table 1.6 records. Nor does it spawn agents into ephemeral-UID sandboxes. The threat-model row, not this section, states the kernel’s current authentication posture.
- **The peer credential is not a cryptographic binding.** It is a kernel-recorded uid/gid/pid triple attached to a socket connection — strong precisely because the kernel, not the peer, supplies it, and *weak* in the way any identity is when the adversary can obtain the same uid. Nothing about it is signed, and calling it a cryptographic bind confuses the guarantee with the authorization chain’s, which is a different mechanism (§1.7).
- **Even fully built, it would not make bypasses “impossible.”** It would move the same-machine adversary from the same-uid row of Table 1.6 to the different-uid row — a real and substantial improvement — while leaving shared kernel surfaces, whatever paths the sandbox binds in, and the sandbox implementation itself in scope. “Impossible” is the wrong word for a containment boundary; “the capability is removed for this named class of access” is the right one.

OP-9 therefore stands *open*: a specified hardening path, not a shipped defense, and the honest grade against a same-machine adversary remains the one §1.13.2 gives — *not guaranteed* until a separate-user boundary exists.

1.13.5 Where information-flow control goes, and why it is not here

One question this section invites belongs to a different chapter, and the cleanest service we can do the reader is to say so rather than answer it badly. Capability scoping alone does not prevent exfiltration: an agent holding both `fs:read:/confidential` and `net:egress:github.com` can read the first and post it to the second, and no amount of claim discipline notices. The containment answer is information-flow control — taint the read, propagate the taint through spawns and shared writes, and gate every release channel behind a declassification policy with a leakage budget. Theorem 1.8.3 already gives the substrate-layer form of it in one line: gate the channel, never the token.

The full construction — a compute-to-data airlock in which one operator’s agent stack executes over another operator’s confidential data, with neither inspecting the other — is *cross-operator* by definition: two principals, mutual distrust, a licensing relationship, and a bond that can be slashed. Every one of those objects lives above this floor, in the chapter that owns the market between operators (*The Harbor Economy*), and this chapter’s abstract promises it lives there rather than here. What the substrate contributes to it is what this chapter has already specified: taint is a marker (§1.7), egress and file writes are controllable events (Theorem 1.8.3), the sandbox that would make an agent a different-uid process is OP-9’s specified hardening (§1.13.4), and the honest statement of what containment buys is the one this section keeps repeating: not that exfiltration becomes impossible — timing, ordering, and side channels stay out of model — but that every flow out passes a named gate whose releases are logged, budgeted, and attributable.

Exercise 1.32 · Check · ★

The hash chain defends against nobody in scope. Name the *out-of-scope* adversary it does defend against, and the layer (substrate or economy) where that adversary becomes real.

Solution on page 271

Exercise 1.33 · Trace · ★★

An agent forges its actor identity to release another agent's lock. Walk the lock-owner-validity rule and show exactly where I12's absence lets the forgery through.

Solution on page 271

Exercise 1.34 · Open · ★★★

What is the minimal hardening that closes the same-machine adversary without a hardware trusted-platform-module dependency (OP-9): an OS keychain for the signing key, sealed memory, the ephemeral-UID sandbox and kernel-verified peer credential specified above? Where does each stop? After the sandbox lands, which row of Table 1.6 does the adversary move to, and what is still in scope there?

Solution on page 271

1.14 Adjacency contract: what the kernel assumes and provides

The kernel's coherence with the layers around it is a contract: what it assumes from the machine below, what it provides to the protocol above, and — just as important — what it explicitly does *not* provide so higher layers do not over-assume.

1.14.1 What the substrate assumes from the machine

- A POSIX filesystem with working advisory locking (`flock/fcntl`). **This is a correctness precondition for I2:** networked filesystems break advisory locks, and a broken lock lets two writers both win — destroying mutual exclusion, not just performance.
- A local wall clock. Single-machine, so no shared-clock assumption is needed — but note that a wall clock is *not monotonic* (it steps when the clock is adjusted, e.g. by network time synchronization), an intra-node hazard for every expiry sweep that the inter-node ordering of Lamport [135] does not cover.
- Unix domain sockets, with owner-only permissions on the socket paths. Both transports authenticate a peer by software handshake over that permission model, *not* by the kernel-verified socket credential (§1.13); reading that credential is OP-9's specified hardening, not an assumption the kernel currently makes.
- An OS keychain for signing keys; file permissions honored on the database path.
- A supervisor process to keep the daemon always-on and to restart *the daemon itself* — the substrate makes other things always-on but cannot make *itself* always-on; that is delegated downward.

1.14.2 What the substrate provides to the coordination layer

- **Atomic, idempotent, time-to-live'd claims** (I2–I4) over ports, file-regions, symbols, and named locks — the coordination layer's typed ownership reduces to claim and release.
- **A durable, versioned, history-coursored publish/subscribe bus**, a tuple space, and decaying stigmergic markers — the coordination layer's speech act is carried *inside* the bus envelope; the anti-blackboard-rot property is *provided* by substrate-side decay.
- **The deontic split as a primitive** (Def. 1.8.1): prohibition $\rightarrow$ monitor (detect/regiment), obligation $\rightarrow$ commitment, permission $\rightarrow$ capability. The coordination layer must not re-implement enforcement.
- **Daemon-owned deadlines and oracle-bound closure** (I5, I6).
- **An append-only audit log** (hash-chain-provisioned) that the protocol and the operator read.
- **Self-attestation** (I10) — higher layers may *assume the kernel is honest about its own health*.
- **The cryptographic authorization chain** — the coordination layer's coordination lineage rides inside it.
- **Linearizable claim/lock semantics** (Theorem 1.11.1) — the property that lets the coordination layer treat “who holds this” as ground truth.

1.14.3 The explicit non-provisions

- The substrate does *not* provide end-to-end non-forgable identity yet. I12 is PARTIAL: daemon-minted actor souls, lookup credentials, and a bounded newcomer pool enforced by the budget guard ship; universal write-boundary enforcement does not.
- It does *not* provide fair/queued exclusion (OP-1), cross-organ transactional atomicity by default (§1.12), a real execution-checkpoint (OP-4), power-loss durability on any write path (I1b, Property 1.5.1; the per-path selector of OP-10 is specified, not shipped), a kernel-verified peer credential on either transport (OP-9), or any cross-machine guarantee (the economy layer — different theorems, a shared-clock and Byzantine-membership problem the substrate deliberately never touches). The status of each open problem is stated once in Table 1.7; a higher layer should assume nothing stronger than that table.
- It does *not* decide *what to do* (no orchestration). It enforces what *cannot* be done and records what *did* happen — a regulator, not a planner.

1.15 Related work

The kernel sits at the confluence of four literatures. We position it against each, and against the design space it deliberately declines.

1.15.1 Reference monitors and the trusted computing base

The organizing idea is the **reference monitor** of Anderson's 1972 security study [113]: a mediator that is *always invoked* (complete mediation), *tamper-resistant*, and *small enough to be verified*. Lampson's *Protection* [43] gives the access-matrix model the monitor enforces, and Saltzer and Schroeder's design principles [116] — economy of mechanism, fail-safe defaults, complete mediation — are the yardstick we hold the kernel to. Where a classical security kernel mediates memory and CPU access for an operating system, ours mediates *coordination* state for a set of cooperating processes, and it achieves complete mediation not by interposing on every system call but by

the cheaper expedient of routing every mutation through a single writer. Schneider’s theory of enforceable security policies via execution monitors [91] supplies the boundary that the central correction of this paper rests on (Theorem 1.8.1): an execution monitor can enforce only the safety properties it can prevent by truncating execution, which a post-commit observer cannot do.

1.15.2 Durable storage and transaction processing

The durability analysis is grounded in Gray and Reuter’s canonical treatment [117], which separates atomicity and consistency from durability — the distinction the I1a/I1b split makes operational. The write-ahead-logging discipline descends from ARIES [46]; the specific crash semantics we inherit are SQLite’s WAL [57], whose own documentation states that the *normal* synchronization level may lose durability under power loss while preserving consistency. Our architectural posture — a single writer rather than a replicated state machine — follows the single-leader default that Kleppmann argues for [150]: reach for consensus only when the problem genuinely forces it. We make that argument formal via the consensus impossibility of Fischer, Lynch, and Paterson [155]: there is no agreement to reach, so the impossibility never applies. Linearizability, the external guarantee we claim for claims and locks, is Herlihy and Wing’s [152]. Cross-organ atomicity, where we decline the distributed solution, is exactly the setting Garcia-Molina and Salem’s Sagas [103] were designed for — and exactly the setting where, because every step is local, a single transaction dominates a saga.

1.15.3 Coordination, stigmergy, and deontic control

The communication organ draws on two coordination traditions. Stigmergic markers with decay descend from Grassé’s stigmergy [175] and its computational realizations in ant-colony optimization [144]; the shared associative store is Gelernter’s Linda tuple space [67]. The obligation arm models commitments as Cohen and Levesque’s persistent goals [171] — intentions an agent may rationally drop — and the prohibition arm uses the normative-systems framing of Jones and Sergot [15] to keep prohibition, obligation, and permission distinct modalities rather than one undifferentiated “policy.” The failure detector underlying heartbeat-based liveness is Chandra and Toueg’s [210], which is why heartbeat freshness can never be a perfectly synchronous pre-commit check. The self-attestation organ, which denies service when its own integrity is unknown, is in the spirit of autonomic computing’s self-management properties [115], and the bounded busy-wait under contention is the timeout-budget bulkhead that Nygard catalogs as a stability pattern [157].

1.15.4 Capability security and tamper-evident logs

The capability/permission distinction — *ability* versus *normative status* — is the object-capability discipline applied to coordination: an agent’s having a handle to an operation does not make the operation permitted. The cryptographic authorization chain is a hop-bound, attenuation-monotonic delegation chain of digital signatures over an elliptic-curve signature scheme [61], of the kind whose secrecy and integrity properties are mechanically checkable in protocol-verification tools [42]; its full treatment belongs to the economy paper. The audit log’s tamper-evidence is the digital-timestamping construction of Haber and Stornetta [204], built on Merkle’s hash trees [184] — a structure that famously predates, and is independent of, blockchains.

Key idea. The kernel is novel less in any single primitive than in their *composition under one constraint*: a single local writer turns mutual exclusion, serializability, linearizability, complete mediation, and a durable audit log into consequences of one architectural choice rather than five separately engineered subsystems. The contribution is the reduction, and the discipline of saying exactly where the reduction stops paying.

1.16 Open problems

These eleven problems are the layer’s research frontier; they appear as starred exercises throughout and are collected here. OP-4 is shared with the personhood paper (this paper owns the kernel mechanism; that one owns the personhood-and-reputation argument).

One of the eleven is closed — by a theorem in this chapter, not by an artifact. Several others have a *specified design* in the body, which is a real thing to have and is not the same thing as being solved; the maturity scale (§1.1.3) exists precisely to keep those two apart. Because a “solved” claim that survives in one place after being retracted in another is the most dangerous kind of error a paper like this can make, Table 1.7 states each problem’s status once, and every exercise box, invariant row, adjacency clause, and figure caption in this chapter is written to agree with it. Where they ever disagree, this table is the one to trust.

Table 1.7: Open-problem status, stated once. **closed** = settled in this chapter; **PARTIAL** = something ships but not the property as posed; *open* = not solved, whether or not a design exists. The last column grades the *design*, not the problem, on the scale of Table 1.1.

#	Problem	Status	Design in this chapter
OP-1	Fair exclusion without a scheduler	<i>open</i>	Stigmergic ticket-lock, SPECIFIED (§1.6)
OP-2	Regimentation vs. enforcement	closed	Theorem 1.8.2 + Theorem 1.8.3
OP-3	Cross-runtime soundness (I11)	<i>open</i>	Differential fuzzing, SPECIFIED (§1.11)
OP-4	Checkpoint with teeth	<i>open</i>	Event-sourced rehydration, SPECIFIED (§1.9)
OP-5	Oracle completeness	<i>open</i>	State-machine assertions, SPECIFIED (§1.8)
OP-6	Tamper-evidence on the read path	<i>open</i>	—
OP-7	Schema evolution without a sequencer	PARTIAL	Ordered boot ledger PARTIAL ; user_version sequencer SPECIFIED (§1.4)
OP-8	Stigmergic decay calibration	<i>open</i>	—
OP-9	Same-machine adversary	<i>open</i>	Ephemeral-UID sandbox + kernel-verified peer credential, SPECIFIED (§1.13.4)
OP-10	Per-write-path durability	<i>open</i>	Selective checkpointing, SPECIFIED (§1.5)
OP-11	Cross-organ atomicity by default	<i>open</i>	One local transaction (§1.12) — a buildable defect

★ OP-1 — Fair exclusion without a scheduler.

Status: *open*. Add bounded-wait to claims and locks while keeping the single-writer, no-background-scheduler simplicity. §1.6 specifies a reactive ticket-lock that would do it for cooperatively released claims and leaves the lazy TTL-sweep path — the one that matters when an agent dies — unfair. Nothing of it ships. Is a FIFO wait-list of time-to-live’d reservations enough once the sweep is included?

★ OP-2 — Regimentation vs. enforcement boundary.

Status: **closed** by Theorem 1.8.2: an invariant is regimentable iff it is a pure, deterministic predicate over committed local state and the incoming payload; everything else is post-commit detection under Theorem 1.8.1. Theorem 1.8.3 places that result inside Ramadge–Wonham

controllability [168, 169], of which it is the commit-only instance, and a companion formal chapter carries the general theorem. The remaining work is classification, not proof.

★ **OP-3 — Cross-runtime soundness.**

Status: *open*. A seeded differential fuzz — 10^6 concurrent operations against both the test and deployment SQLite bindings, asserting hash-equivalence of the resulting state — is specified (§1.11) and does not run in the pipeline today. Even once it does it is a test, not a proof: name the divergence class it would miss.

★ **OP-4 — Checkpoint with teeth.**

Status: *open*, and still the most important unbuilt thing at this layer. Event-sourced rehydration (§1.9, SPECIFIED) would replay a pinned commit and a truncated message log into a successor, which restores the *durable inputs* to an agent’s context. Whether that amounts to restoring the model’s inference state — which is bound to a particular model and session and is not a portable artifact — is exactly the open question, and no formal result in this series establishes it.

★ **OP-5 — Oracle completeness.**

Status: *open*. State-machine assertions (§1.8, SPECIFIED) would add delta oracles and AST assertions to Algorithm 1.2’s four shipped kinds. Two kinds are not a completeness result, and neither kind yet survives the Goodhart argument the vocabulary exists to win.

★ **OP-6 — Tamper-evidence on the read path.**

Status: *open*. Where should hash-chain verification gate? A sampling/checkpoint regime with a provable detection-probability bound (couples to §1.13.4).

★ **OP-7 — Schema evolution without a sequencer.**

Status: PARTIAL. An ordered boot ledger with post-apply schema verification ships (§1.4); the `user_version` sequencer over declared migrations does not, and modules still self-initialize. Can any useful remnant of “any module, any order” survive safe renames, backfills, and down-migrations?

★ **OP-8 — Stigmergic decay calibration.**

Status: *open*. The right per-kind decay so coordination markers neither rot nor evaporate before they coordinate.

★ **OP-9 — Same-machine adversary.**

Status: *open*. §1.13.4 specifies the hardening — spawns in `bwrap/unshare` sandboxes under ephemeral UIDs, plus the kernel-verified peer credential (`SO_PEERCRED` / `LOCAL_PEERCRED`) read on each connection — and none of it ships. Today’s peer-credential check is a software handshake over an owner-only socket, exactly as Table 1.6 states. Even built, the hardening would move the adversary from the same-uid row to the different-uid row, not out of the model.

★ **OP-10 — Per-write-path durability.**

Status: *open*. Selective checkpointing (§1.5, SPECIFIED) would let a settlement-ledger write force `wal_checkpoint(FULL)` on its commit path; no path opts in today, and I1b remains *not guaranteed* kernel-wide per Property 1.5.1. The harder half is the interface that stops a new write path from defaulting into the fast lane silently.

★ **OP-11 — Cross-organ atomicity by default.**

Status: *open* but cheap — a buildable defect, not a frontier. Wrap multi-organ operations in one local transaction at the interface boundary, eliminating the undefined partial-failure semantics of §1.12.

1.17 Conclusion: solid where local, provisional where it must grow

The kernel is a single-writer transactional reference monitor over a local SQLite/WAL database, and its two jobs — durable record and mediated exclusion — are real and, mostly, proven. It earns its strongest claims (mutual exclusion, serializable/linearizable consistency, oracle-bound closure) precisely by refusing the distributed-systems problem it superficially resembles: one writer, one file, one decider, no consensus.

Where it stops, it stops formally. Durability is split: process-crash durable (I1a), *not* guaranteed against power loss (I1b). The policy monitor detects and compensates; it does not regiment, and we credit the only genuine pre-commit regimentation to the uniqueness constraint and the boot-admission gate. Cross-organ atomicity is a buildable defect, not a frontier. Hash-chain tamper-evidence is re-scoped to cross-machine provisioning, where it actually defends someone. And the two genuinely soft spots — partial local identity enforcement (I12) and a checkpoint with teeth (OP-4) — are exactly the two things a cross-machine economy and a durable notion of agent personhood lean on hardest.

The kernel is solid where it is local and provisional exactly where a cross-machine economy would need it to be cryptographic and continuous. That is not a flaw to hide; it is the layer boundary, stated truthfully.

Every “single-machine / one-writer / one-clock / self-asserted-identity” simplification that makes this layer clean is precisely the assumption a later layer must pay to relax. Cross-machine capability transfer over the cryptographic authorization chain this kernel ships is where that payment begins, and it belongs to the economy layer, not here. Build the local floor first. The cryptography earns its keep when agents must trust one another across machines they do not control.

Chapter Appendices

1.A Implementation & status

The body of this paper argues at the level of mechanisms, so that it reads as a self-contained systems paper. This appendix records the concrete grounding: the specific artifact that realizes each mechanism in the reference implementation, and an honest accounting of how mature each is. Nothing here is needed to follow the argument; it is here so the maturity grades in the body are auditable rather than asserted.

1.A.1 The reference implementation

The reference implementation is a single always-on local daemon written in TypeScript, run under a compiled JavaScript runtime, persisting to one SQLite database file in WAL mode. Clients — a command-line tool, agents over a tool-protocol bridge, and a binary inter-process transport — reach it over Unix domain sockets. The daemon is kept alive by the host operating system’s service supervisor. The seven organs of §1.3 correspond to distinct modules that each self-initialize their own tables over the shared database handle, in the factory style `createModule(db)`.

1.A.2 Mechanism-to-artifact map

Table 1.8 maps each mechanism discussed in the body to the module that realizes it. The storage configuration referenced throughout is the WAL pragma set: `journal_mode=WAL`, `synchronous=NORMAL`, `wal_autocheckpoint=200`, `busy_timeout=5000`, `foreign_keys=ON`, with a clean-shutdown `wal_checkpoint(TRUNCATE)`.

Table 1.8: Mechanism-to-artifact map for the reference implementation. Module names are illustrative of the organization, not a stable public interface.

Mechanism (body)	Realizing module(s)	Notes
Single write connection / pragmas	database-handle module	opens and configures SQLite
Port & lock claims	service-registry, locks, session-files modules	three claim granularities
Message bus / tuple space / markers	bus, tuples, marker modules	at-least-once delivery
Policy monitor	monitor module	post-commit log subscriber
Commitments & obligation	commitment, obligation-monitor modules	two of the hardening laws shipped
Memory & recovery	session, episodic-memory, recovery modules	recovery passes notes only
Audit log & hash chain	activity-log, hash-chain modules	verification not yet on the read path
Self-attestation	attestation, invariant-registry modules	boot gate + pre-flight + watchdog
Authorization chain	delegation-chain module	mechanically verified
Enforcement gate (macaroon discharge)	kernel macaroon module	crypto verified; in-band, not yet compulsory (§1.13.2)
Runtime-parity shim	runtime-shim module	normalizes binding differences
Identity (actor souls + credentials)	actor-souls, budget-guard modules	bounded gate ships; full write-boundary enforcement absent

1.A.3 Status, and the corrections that matter

The maturity grades from Table 1.4 reflect the state of the reference implementation at the time of writing. Four corrections this paper makes to earlier, more optimistic internal descriptions are substantive and worth restating with their evidence:

1. **Durability is split by fault class** (§1.5). An internal code comment justifying the `synchronous=NORMAL` choice asserted that the normal level “is safe in WAL mode because WAL already guarantees crash safety.” That conflates crash-consistency with durability; under power loss the last writes can roll back. The honest statement is the I1a/I1b split.
2. **The policy monitor is detect-and-compensate, not regimentation** (§1.8, Theorem 1.8.1). The monitor is implemented as a subscriber to the already-committed operation log, so it cannot prevent the bad prefix that triggers it; “physically unreachable” is reserved for the uniqueness-constraint and boot-gate states.
3. **Cross-organ atomicity is a buildable defect, not a research frontier** (§1.12). The database transaction primitive is already used elsewhere in the implementation; the only missing work is wrapping the multi-organ endpoints in it.
4. **A specified design is not a solved problem** (Table 1.7). An earlier revision of this chapter reported several open problems as closed — fair exclusion, runtime parity, checkpointing, oracle completeness, the same-machine adversary, per-write-path durability — on the strength of designs that no artifact realizes, while the exercise boxes, invariant rows, figure captions, and threat model those claims contradicted were left untouched. The designs are kept and graded SPECIFIED; the problems are reported open. The security-relevant instance is worth naming on its own: the socket peer-credential check is a software handshake and *not* the kernel-verified `SO_PEERCRED` credential, in this chapter’s threat model (Table 1.6), in §1.7, and in §1.13.4 alike.

1.A.4 Nomenclature

For the reader cross-referencing the accompanying chapters, the body’s neutral names map to the series’ internal terms as follows: the *substrate* / *coordination* / *legibility* / *economy* layers are the series’ L0–L3; the *policy monitor* is the Arbiter; *stigmergic markers* are pheromones; the *bus* is tube; the *authorization chain* and *coordination lineage* are the two objects the series keeps distinct under the single phrase “delegation chain.” The maturity grades — implemented / partial / specified / proposed — are the series’ honest labels.

What *The Anchor Protocol* receives Local effects now have one serial history, one fencing epoch, and one place to attach receipts. The writer decides what becomes real; it does not yet say who may ask it to. The next chapter turns authority into a key-bound, attenuating, revocable object, so that every request the kernel admits arrives with a checked pedigree.

The Anchor Protocol

Every program and every user of the system should operate using the least set of privileges necessary to complete the job.

JEROME SALTZER AND MICHAEL SCHROEDER, THE PROTECTION OF INFORMATION IN COMPUTER SYSTEMS, 1975



THE QUESTION THIS CHAPTER ANSWERS

How may authority travel without silently growing?

Delegated capability only narrows: every child is a subset of its parent, at every hop, and the verifier checks the whole chain.

How may authority travel without silently growing? Identity answers who signed; it does not answer what the signer may do. Anchor therefore checks every delegation edge, binds the grant to a key and audience, narrows scope and lifetime, and preserves revocation ancestry. The formal claim concerns the checked protocol and Rust paths, while R5 explains which surrounding policy promises can become admission gates.

2.1 What problem this solves

The scene. It is 2 a.m. and your laptop is running four coding-agent sessions at once: one refactoring the auth module, one running the test suite against a local Postgres, one building a preview deployment, and one you started an hour ago and forgot about. All four are agents. All four can read your files, bind ports, and run shell commands with your full user permissions — and none of them can tell, from the outside, which of the other three is behaving. Nothing on the machine distinguishes the forgotten session from a process that has quietly replaced it.

That is the local swarm problem. In a multi-agent development environment, agents operate concurrently to read files, spin up test servers, and execute commands, and this introduces three concrete failure modes on `localhost`:

1. **Port Squatting (The “Ghost in the Harbor”):** If Agent A crashes, a malicious or malfunctioning Agent B can bind to Agent A’s previously assigned port, intercepting traffic meant for A.
2. **Resource Contention:** Agents fighting over shared resources (e.g., a SQLite database file) without a centralized locking mechanism cause state corruption.
3. **Privilege Escalation:** A “Reviewer” agent delegated by a “Coder” agent should not possess the rights to initiate production deployments, yet local environments rarely enforce principle-of-least-privilege between processes.

The idea in one breath. *Every agent on the machine carries a short-lived, cryptographically signed card that names exactly what it may do; anything it hands to a child is strictly smaller; and any card can be withdrawn before it expires.* Everything that follows is that sentence made precise, then mechanically checked.

To make it enforceable, Port Daddy introduces the concept of a **Harbor**—a zero-trust, namespace-isolated execution environment where agents must prove their identity and capabilities. Our approach builds upon foundational work in capability-based security by Dennis and Van Horn [109] and the principle of least privilege articulated by Saltzer and Schroeder [116]. This protocol sits in a specific lineage — Macaroons for delegation, ProVerif and Tamarin for symbolic verification, the JWT algorithm-confusion literature for the Phase 1 hardening, and the emerging agentic-commerce credential profiles for the envelope — traced in full in Appendix 2.F, which a reader following the argument rather than the citations can skip entirely.

How to read this chapter. §2.2 is the protocol: four phases, each closing a hole the previous one left open. §2.3 is the evidence: three verification layers, what each one proves, and what it hands the next. §2.5 is the accounting: the adversary we assume, the properties that survive it, and where the guarantees stop. Figure 2.8 is the map for §2.3; Table 2.2 closes the loop on the three failure modes just listed.

2.2 The Anchor Protocol Architecture

The Anchor Protocol defines how agents authenticate to a Harbor and to each other. It issues **Harbor Cards** (highly constrained JSON Web Tokens) and evolves them across four phases, each adding a capability that closes a specific attack surface from the previous phase (Figure 2.1).

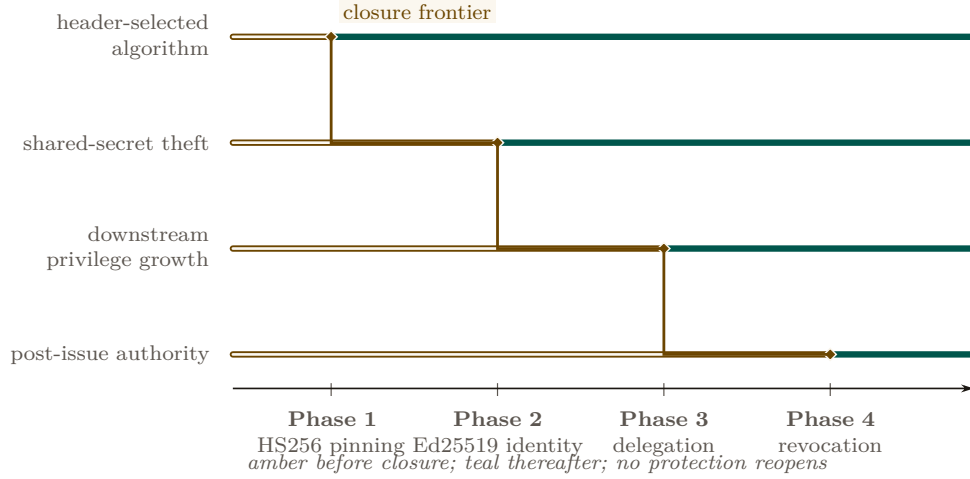


Figure 2.1: Threat closure is cumulative, not substitutive. Each protocol phase moves one attack surface across the closure frontier while every earlier rail stays closed. The staircase therefore encodes the architectural invariant: a later mechanism adds protection without reopening a hole already closed.

A common thread across all phases is *capability attenuation*: a delegated token never carries more authority than its parent, and TTL only shrinks. Figure 2.2 shows the nested-cap structure that the verifier enforces on every chain.

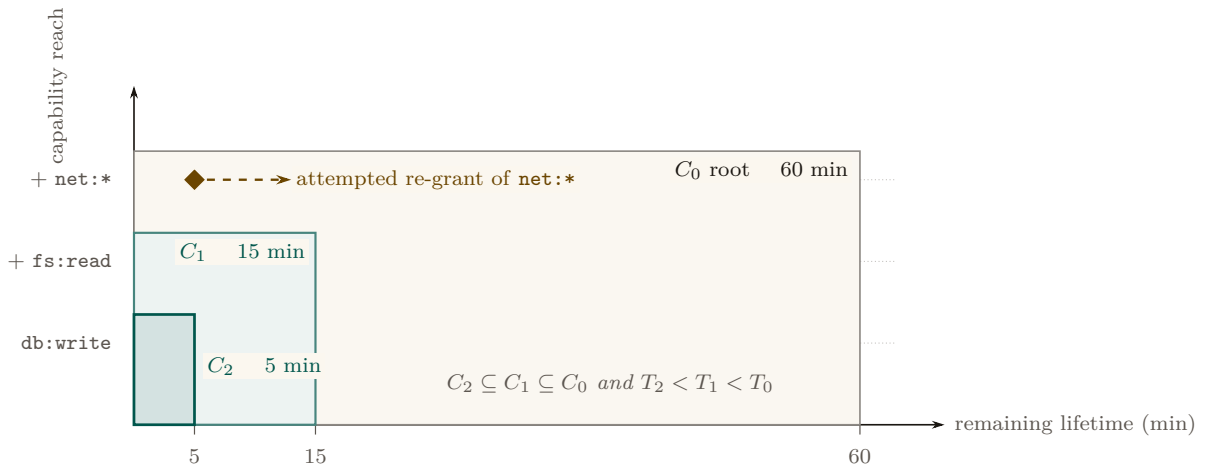


Figure 2.2: Delegation must remain inside a two-dimensional parent envelope. The child’s capability set and remaining lifetime both contract at every hop. The amber re-grant lies outside C_1 even though its requested lifetime is short; the verifier rejects it because attenuation is conjunctive, not a trade between rights and time.

2.2.1 Phase 1: Symmetric Pinning

Early versions of the protocol relied on HS256 (HMAC-SHA256). The primary vulnerability in JWTs is “Algorithm Confusion” (CVE-2015-9235 [53], CVE-2023-48238 [55]), where an attacker modifies the token header to `{"alg": "none"}` or symmetric HS256 while using an asymmetric public key as the HMAC secret.

DESIGN INVARIANT

Algorithmic pinning

The Port Daddy Verifier employs strict **Algorithmic Pinning**. It entirely ignores the user-provided `alg` header and explicitly forces the underlying crypto library to evaluate the signature using the pre-determined algorithm. This approach is recommended by the JWT Best Current Practices draft [221].

Figure 2.3 shows the two verifier paths side by side: the naive verifier trusts the `alg` field on the wire (and is exploited), while the pinned verifier records the issuance algorithm out-of-band and admits no rewrite trace.

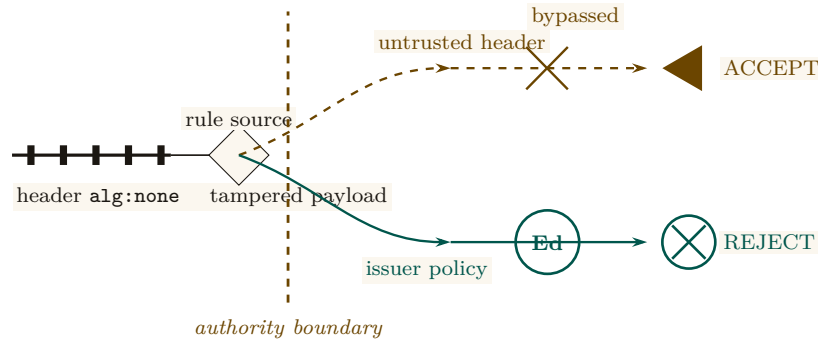


Figure 2.3: Algorithm confusion occurs at the rule-selection boundary, not in the token’s shape. Identical tampered bytes are accepted when the token selects its own verifier and rejected when authenticated issuer policy pins Ed25519. The fork isolates the only causal difference: control of the verification rule.

2.2.2 Phase 2: Asymmetric Identity (Ed25519)

To support decentralized Agent-to-Agent (A2A) communication without sharing HMAC secrets, the protocol transitioned to **Ed25519 (EdDSA)** [60, 200]. Ed25519 provides fast, deterministic signatures with small keys and signatures, making it ideal for local agent communication.

Definition 2.2.1 (Harbor Key Hierarchy). The Port Daddy Daemon acts as the Root Certificate Authority (CA). Each Harbor is assigned a unique Ed25519 keypair. The Daemon issues Harbor Cards signed by the Harbor’s private key. Agents use the Harbor’s public key (broadcast via Port Daddy’s ambient discovery service) to verify tokens presented by peers.

2.2.3 Phase 3: Stigmergic Delegation (The “Biscuit” Pattern)

Port Daddy v4 introduces multi-hop delegation. When Agent A spawns Agent B, it does not need to request a new token from the Daemon. Instead, it uses **Offline Attenuation**, inspired by Macaroons [22].

Definition 2.2.2 (Offline Attenuation). Agent A takes its existing Harbor Card, appends a new set of restricted capabilities (e.g., `["db:read"]` reduced from `["db:read", "db:write"]`), and signs the *entire chain* with its own ephemeral private key.

The Harbor verifies the Daemon’s root signature, Agent A’s delegation signature, and mathematically ensures that Agent B’s capabilities are a strict subset of Agent A’s.

For depth- N chains, the verifier walks the structure shown in Figure 2.4. Each hop must produce a fresh nonce and bind the previous and next identities into its signature so that an attacker who intercepts a single hop cannot splice it into a different chain.

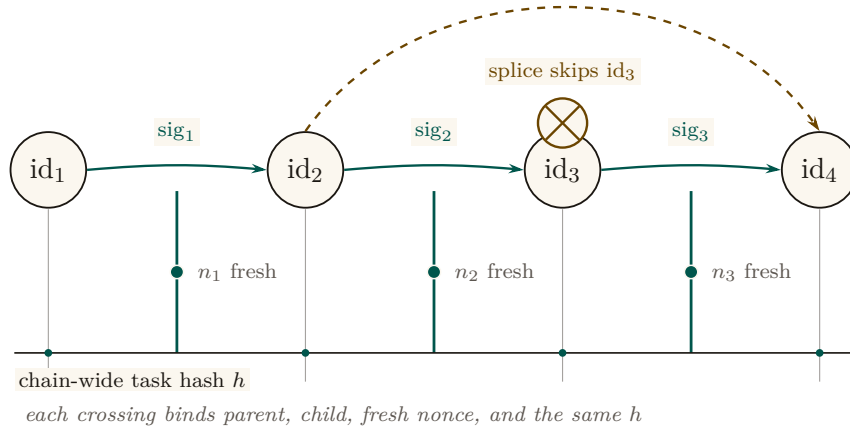


Figure 2.4: The delegation chain is a signed provenance braid. Identity continuity runs across the top, fresh nonces pin each crossing, and one task hash anchors the chain below. Replaying a link fails freshness; substituting or splicing a link creates a visible discontinuity at an adjacent-identity boundary.

2.2.4 Withdrawing a card before it expires

Capability tokens have a TTL, but natural expiry is not always sufficient. Two circumstances force early withdrawal: (i) *compromise*—a Harbor Card leaks from its agent’s process, and every second until TTL is a second the attacker has legitimate capabilities; and (ii) *policy reversal*—a principal decides an agent should no longer hold `db:write`, and waiting for TTL is unacceptable. A naive revocation list is $O(n)$ per verification and grows monotonically. While the Anchor Protocol previously attempted to gossip cuckoo filters directly between daemons — merging two peers’ filters by OR-ing their bit arrays — it now gossips an **Append-Only Event Log of Revocation IDs** using Vector Clocks. To maintain $O(1)$ read performance, each daemon continually projects its event log into a local, ephemeral cuckoo filter [37] (Figure 2.5).

Why filters cannot be merged bitwise. The abandoned design failed for a structural reason worth stating plainly, because it is the reason the log exists. Two cuckoo filters cannot be safely combined by OR-ing their bit arrays. A fingerprint’s final resting bucket is not a function of the key alone: it is the first candidate bucket i_1 if that bucket had a free slot at insertion time, and otherwise the alternate i_2 reached by evicting some other fingerprint — so occupancy depends on insertion order and eviction history, not just on the set of keys. Peer *A* may hold fingerprint `c1` in bucket i_0 while peer *B*, having inserted a different key first, holds the same `c1` in bucket i_3 . The bitwise union contains both placements, which no single consistent filter state could have produced: subsequent deletions then remove the wrong occupancy and can *reanimate* a revoked card — a false negative, the one error a revocation gate must never make. Gossiping the append-only log of `kids` and rebuilding each daemon’s filter locally sidesteps the problem entirely, because set union over identifiers is well defined where union over their lossy encodings is not.

Cuckoo filters in one paragraph. A cuckoo filter is a compact probabilistic set: under successful insertion and correct deletion discipline it has false positives but no false negatives, while supporting deletion. Each inserted key is reduced to a fingerprint and placed in one of two candidate buckets; on collision, fingerprints are relocated until a slot opens or insertion fails. Lookups inspect both buckets. The expected lookup cost is $O(1)$ and the approximate false-positive probability for two buckets of size B and f fingerprint bits is at most about $2B/2^f$ in the low-collision model. High load increases insertion failures; the implementation rejects or rebuilds rather than silently dropping a revocation. Duplicate fingerprints require counted or authoritative-table-backed deletion so that removing one key does not reanimate another.

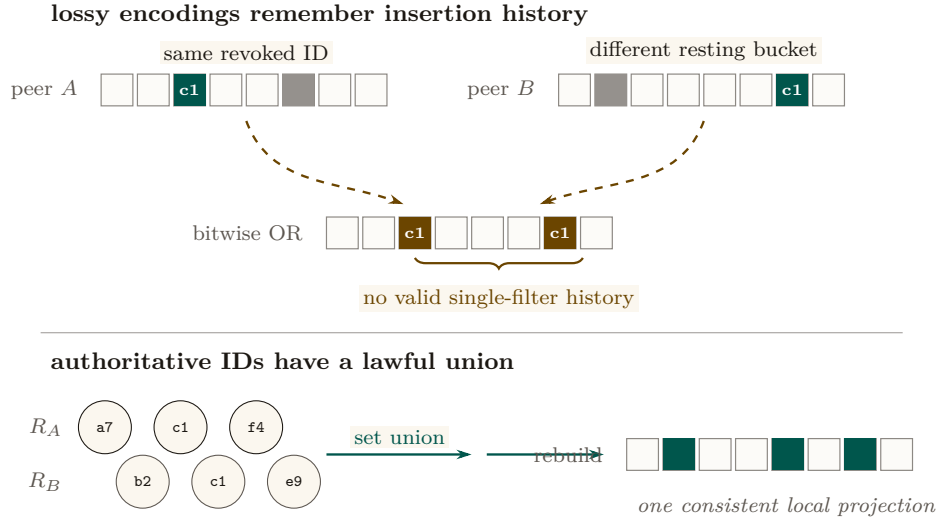


Figure 2.5: Filter state is not a mergeable set representation. The same revoked ID can occupy different buckets after different insertion and eviction histories; bitwise OR preserves both lossy placements and yields no coherent deletion history. Revocation IDs, by contrast, admit ordinary set union. Each daemon can therefore merge the append-only log and rebuild one valid local filter without risking a false negative through cross-peer filter mutation.

Why cuckoo, not Bloom. For uniform-TTL workloads, a rotating Bloom filter would suffice. Port Daddy’s TTLs are heterogeneous—Shipwright proposals run hours to days, `pd spawn` children run minutes, one-shot agents run seconds. Rotating Bloom either rotates at the longest TTL (short-TTL revocations linger orders of magnitude longer than needed) or buckets by TTL class (reintroducing the structural complexity cuckoo collapses). Cuckoo provides $O(1)$ per-entry removal at each specific expiry, plus the same removal handles operator-initiated early-revocation undo cleanly.

Space. At false-positive rate ϵ , a cuckoo filter uses approximately $\log_2(1/\epsilon) + 3$ bits per entry. For $\epsilon = 10^{-3}$, ≈ 13 bits per revoked card. A fleet retaining 10^5 active revocations fits in ≈ 160 KB—trivially in memory, trivially gossiped.

Definition 2.2.3 (Revocation-Augmented Verification). Let R_t denote the authoritative Append-Only Event Log of revoked Harbor Card IDs at time t . Each daemon d maintains a local, ephemeral cuckoo filter index $F_d \approx R_t$. Every Harbor Card verification additionally:

1. Looks up the card’s `kid` in F_d . If absent, admit.
2. If present, query the local `revocations` table (authoritative). If genuinely revoked, reject with `revoked`; otherwise the filter is in the false-positive regime and the caller is admitted, with an asynchronous reconciliation job repopulating the filter.

Gossip-based synchronization. Daemons in a mesh synchronize revocation deltas by anti-entropy gossip [9]. Under a connected complete-overlay model with independent uniform peer selection and reliable rounds, epidemic dissemination is expected $\Theta(\log m)$ rounds. This is an expectation, not a deadline; partitions, message loss, topology, and scheduling change the constant or prevent global convergence until the partition heals. Security-critical revocations can additionally trigger best-effort immediate push. Figure 2.6 illustrates propagation across five daemons; it is a trace, not a timing proof.

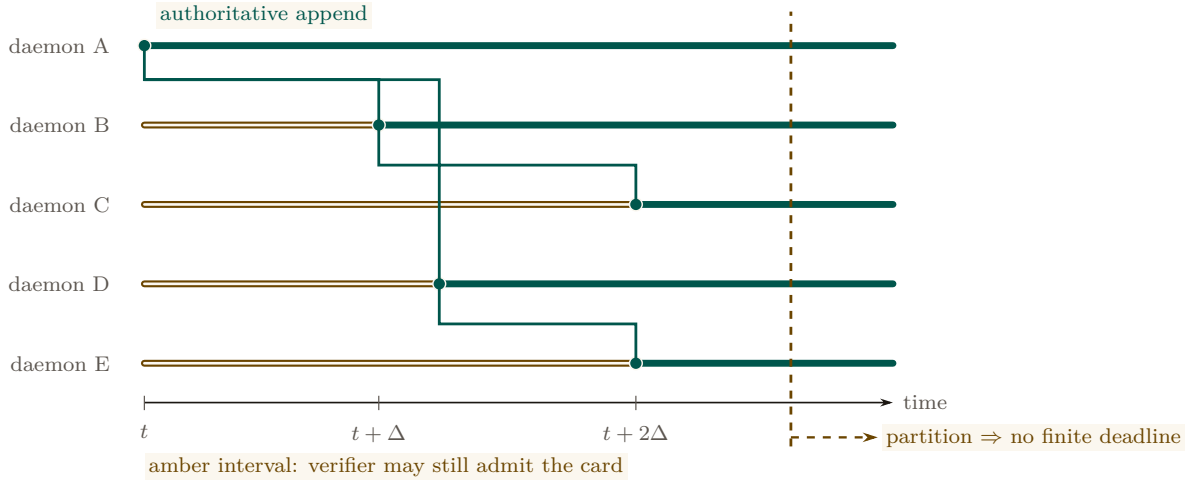


Figure 2.6: A revocation advances as an observation frontier, leaving a distinct stale-exposure interval on every verifier lane. This execution converges after two rounds; the protocol claim is only expected $\Theta(\log m)$ dissemination under connected reliable rounds. The dashed partition marker separates that expectation from a guarantee: an unbounded partition removes every finite delivery deadline.

DESIGN INVARIANT

Filter monotonicity over a TTL epoch

For any Harbor Card c with issue time t_0 and TTL τ , if $c \in R_t$ for some $t \in [t_0, t_0 + \tau]$, then $c \in R_{t'}$ for all $t' \in [t, t_0 + \tau]$. After $t_0 + \tau$, c may be removed from R since the card has expired.

DESIGN INVARIANT

No partial reanimation

A revoked card cannot be un-revoked within its TTL. If an operator decides the revocation was erroneous, they must issue a new card.

THEOREM

Conditional gossip expectation

Under the complete-overlay, uniform-peer, reliable-round model above, a revocation disseminates to all m daemons in expected $\Theta(\Delta \log m)$ time. No finite worst-case freshness bound holds under an unbounded partition.

Soundness preservation. Revocation strictly narrows acceptance, so existing ProVerif soundness proofs (Property 2.3.1) are unaffected. The new proof obligation is small: revocation is *enforced*, which follows by inspection of the verification path—the revocation check is unconditional and precedes admission. The card lifecycle gains a state (*valid* $\rightarrow$ *revoked* $\rightarrow$ *expired*) modeled as an additional transition (Figure 2.7).

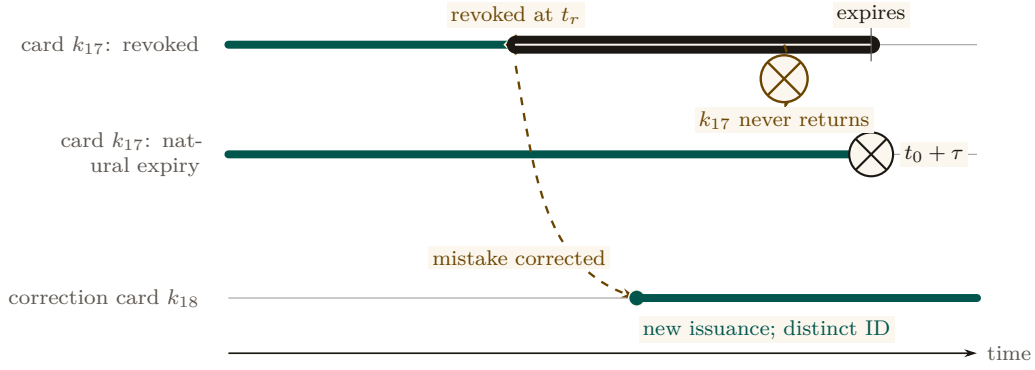


Figure 2.7: Acceptance is a one-way interval attached to one card ID. Card k_{17} stops being admissible either at explicit revocation or at natural expiry and never re-enters the valid set. If revocation was a mistake, the operator issues k_{18} on a new lane; correction creates new authority rather than mutating rejected history.

Privacy. A Dolev-Yao adversary observing all gossip traffic learns the revoked-card IDs, leaking which agents have been disciplined. For an organization’s own daemons, this is acceptable audit transparency. Stronger privacy is available by encoding revocations as salted hashes $H(\text{kid}, \text{salt})$ where the salt rotates daily, stored encrypted under a harbor session key.

Recitation

- Without looking: state the one property that must hold between every delegated Harbor Card and its parent, across all four phases.
- Why can two daemons not merge their cuckoo filters by OR-ing the bit arrays, and what do they gossip instead?
- Name the two circumstances that force a card’s withdrawal before its TTL naturally expires.

2.3 How we know it is correct

Following the industry standard set by AWS (s2n-tls [29]) and Microsoft (Project Everest [130]), we decoupled the verification of the *protocol design* from the verification of the *implementation*. The full stack has three layers (Figure 2.8): symbolic protocol analysis at the top, bounded model checking on the Rust source in the middle, and runtime conformance tests against the deployed binary at the bottom. Each layer’s obligations flow downward; each layer’s evidence flows back up. The stack is sound only if every bridge — symbolic-to-source and source-to-deployed — holds.

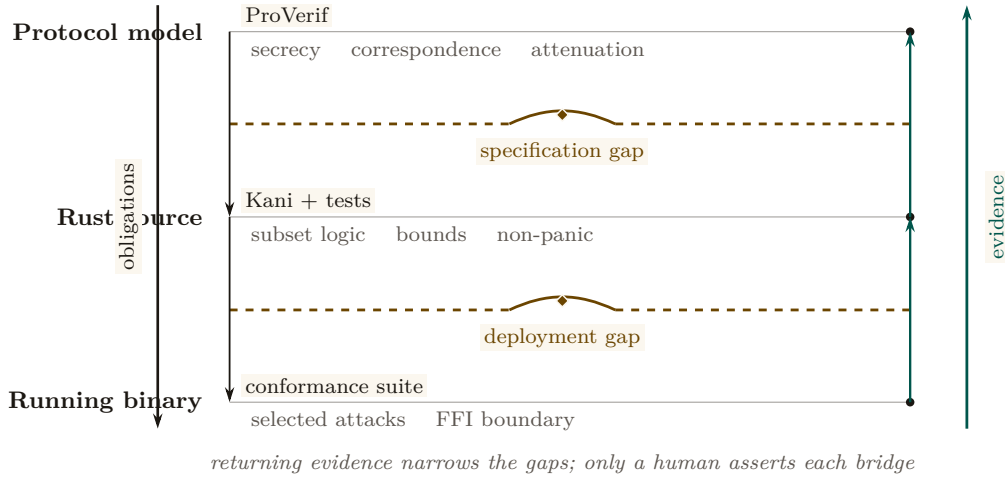


Figure 2.8: The assurance case is a two-rail bridge. Obligations descend from protocol model to source to deployed binary; observed evidence returns upward through the same layers. The amber spans are irreducible assumption gaps: tool output on either bank can narrow them, but only a human asserts that the modeled protocol is this source and that the inspected source is this running binary.

Figure 2.8 is this section’s map. Read the forward rail for what each layer hands the next as an obligation it cannot discharge itself; read the teal return rail for what has actually been checked about the running code. The two dashed boundaries are the only places in the whole assurance case where a human, not a tool, asserts that one layer’s claim carries over to the next — which is exactly why they are drawn as assumption gaps rather than as inference lines.

2.3.1 Symbolic Analysis with ProVerif

To prove the protocol’s logical soundness against a Dolev-Yao adversary [62] (an attacker who controls the entire network), we modeled the Anchor Protocol in **ProVerif** [41]. ProVerif represents protocols as Horn clauses and uses resolution to prove security properties for an unbounded number of sessions.

MODEL-CHECKED PROPERTY

Authentication correspondence in the modeled phase

For the modeled delegation phase, ProVerif confirmed the correspondence:

$$\begin{aligned} \text{Accepted}(b, h, \text{cap}) \implies & (\text{IssuedRoot}(a, h, \text{cap}) \wedge \text{Delegated}(a, b, \text{cap})) \\ & \vee \text{IssuedRoot}(b, h, \text{cap}). \end{aligned} \quad (2.1)$$

The displayed query is a non-injective authentication correspondence: every accepted event has a matching modeled issuance/delegation event. It does not by itself prove replay freedom, capability attenuation, or arbitrary-depth transitivity; those are separate queries and bounds.

All three protocol phases were independently verified in ProVerif 2.05. Table 2.1 summarizes the results; full models are reproduced in Appendices 2.B, 2.C, and 2.D.

Table 2.1: All five listed ProVerif queries return **TRUE** across the three modeled protocol phases. Session replication is unbounded where the model uses replication; delegation-chain structure remains bounded as stated in §2.5.3.

Query	Phase	Model	Result
$\neg \text{attacker}(\text{master_key})$	v1 (HS256)	v1_refined.pv	TRUE
Accepted $\implies$ Issued	v1 (HS256)	v1_refined.pv	TRUE
$\neg \text{attacker}(\text{harbor_sk})$	v2 (Ed25519)	v2_asymmetric.pv	TRUE
Accepted $\implies$ Issued	v2 (Ed25519)	v2_asymmetric.pv	TRUE
Accepted(b) $\implies$ IssuedRoot(a) $\wedge$ Delegated(a, b)	v3 (Delegation)	v3_delegation.pv	TRUE

The following is the verbatim ProVerif 2.05 output for the delegation chain query (Phase 3), the most complex property:

```

1 RESULT event(Accepted(b_2,harbor_id[],cap_1))
2 ==> (event(IssuedRoot(a_3,harbor_id[],cap_1))
3      && event(Delegated(a_3,b_2,cap_1)))
4      || event(IssuedRoot(b_2,harbor_id[],cap_1))
5 is true.

```

Listing 2.1: ProVerif 2.05 Terminal Output — Phase 3 Delegation Chain

2.3.2 Runtime Enforcement: The Arbiter

Formal models establish properties of their abstractions. At runtime, Port Daddy deploys the **Arbiter**—an ambient monitor that subscribes to the daemon’s post-commit activity log and checks six derived invariants:

1. **PID_SQUATTING**: Detects when a process claims a port previously held by a different PID (the “Ghost in the Harbor”).
2. **CAP_ESCALATION**: Verifies capability subsets via the deployed Rust FFI enforcer.
3. **NOTE_MONOTONICITY**: Ensures the note sequence for active sessions never shrinks (from the TLA^+ *NoteMonotonicity* invariant).
4. **ESCROW_POSITIVE**: Validates that active sessions have positive escrow (when Float Plans are active).
5. **LOCK_OWNER_VALID**: Ensures locks are only held by registered agents with active sessions.
6. **HEARTBEAT_FRESHNESS**: Flags stale heartbeats as early warning for agent death.

Violations are logged and can trigger the “man overboard” salvage protocol: the daemon revokes the offending agent’s active Harbor Cards (§2.2.4), releases any locks and port reservations it held, and, for supervised sessions, notifies the parent agent or the human operator. Because this path observes *committed* events, it detects and compensates; only a pre-commit native guard can prevent a transition in the first place. The Arbiter API provides visibility, not a proof that every security-relevant operation was mediated.

Where the kernel’s boundary places this chapter. The controllability theorem of *The Single-Writer Kernel* sorts every rule by whether the event it prohibits is one the mediator can refuse. Card verification is such an event: accept or reject happens on the daemon’s own socket, before any effect, so every rule this chapter states about tokens, algorithm pinning, per-hop attenuation, and revocation before admission, is regimentable, and the ProVerif models are proofs about a gate that really can close. What the Anchor cannot regiment is what a holder does with authority it legitimately has. Exercising a valid `db:write` to write the wrong thing is an uncontrollable event at this boundary; it is the Arbiter’s detect-and-compensate business here, and audit and slashing in *The Bonded Commons*.

2.3.3 Implementation Verification

A secure protocol is useless if the implementation contains buffer overflows or timing leaks. We extracted the critical JWT parsing and cryptographic comparison logic into a verified core.

DESIGN INVARIANT

Memory safety within the harness bound

Using the **Kani** Rust Verifier [28], we bounded-model-check the token parser with key decoding, base64 decoding, and signature verification stubbed to return either outcome. For every 32-byte input that is valid UTF-8 and contains a dot, and within ten loop iterations, the parse path neither panics nor accesses memory out of bounds. The bound is the harness’s, not the parser’s: longer tokens and the real cryptographic primitives lie outside it.

DESIGN INVARIANT

Secret-independent source control flow

The Rust comparator (`constant_time_compare`) folds the XOR of every byte pair into one accumulator and tests the accumulator once, so no source-level branch depends on secret bytes. That is a property of the source text, established by inspection; Kani’s harness adds only that the comparator cannot panic on any pair of 16-byte inputs. Neither is a hardware-level constant-time proof: compiler transformations, memory behavior, and microarchitecture remain outside both.

2.3.4 The procedures, precisely

The three phases are restated here as pseudocode, for the reader who wants the exact procedure rather than the narrative. Nothing new is claimed in this subsection: it is the same three phases written so that each line has a visible counterpart in the formal ProVerif models of Appendix 2.B, in the notation used in the protocol-verification literature [41, 96].

2.3.4.1 Phase 1: Symmetric HS256 with Algorithm Pinning

Algorithm 2.2 presents the secure verification procedure that is immune to algorithm confusion attacks (CVE-2015-9235 [53]).

```

1 Require: Pinned Algorithm: hs256
2 Require: Master Key: k_master (secret)
3
4 function Daemon.IssueToken(agent_id, harbor_id, capability):
5     jti <- GenerateNonce()
6     msg <- Encode(agent_id, harbor_id, jti, capability)
```

```

7     sigma <- HMAC-SHA256(msg, k_master)
8     emit Issued(agent_id, harbor_id, jti, capability)
9     return (hs256, msg, sigma)
10
11 function Harbor.VerifyToken(tok, expected_harbor):
12     (alg_header, msg, sigma) <- tok
13     // CRITICAL: Ignore alg_header, pin to HS256
14     if HMAC-SHA256(msg, k_master) != sigma:
15         return _|_ // Invalid signature
16     (a, h, j, cap) <- Decode(msg)
17     if h != expected_harbor:
18         return _|_ // Wrong harbor
19     emit Accepted(a, h, j, cap)
20     return (a, h, j, cap)

```

Listing 2.2: Algorithm 1: Secure Harbor Verification (HS256 with Algorithm Pinning)

Security Property: The algorithm ignores the user-provided *alg_header* (line 15), preventing algorithm confusion attacks where an attacker sets *alg* = “none” or attempts HS256/RS256 confusion [208].

2.3.4.2 Phase 2: Asymmetric Ed25519

Algorithm 2.3 presents the Ed25519-based verification. This phase removes the need for shared HMAC secrets between distributed Harbors.

```

1 Require: Harbor Keypair: (sk_harbor, pk_harbor)
2 Require: Key Distribution Channel: c_key (private)
3
4 function Daemon.IssueToken(agent_id, harbor_id, capability):
5     sk_h <- Receive(c_key) // Get harbor's secret key
6     jti <- GenerateNonce()
7     msg <- Encode(agent_id, harbor_id, jti, capability)
8     sigma <- Ed25519.Sign(msg, sk_h)
9     emit Issued(agent_id, harbor_id, jti, capability)
10    return (msg, sigma)
11
12 function Harbor.VerifyToken(tok, pk_h, expected_harbor):
13     (msg, sigma) <- tok
14     if not Ed25519.Verify(msg, sigma, pk_h):
15         return _|_ // Invalid signature
16     (a, h, j, cap) <- Decode(msg)
17     if h != expected_harbor:
18         return _|_ // Wrong harbor
19     emit Accepted(a, h, j, cap)
20     return (a, h, j, cap)

```

Listing 2.3: Algorithm 2: Asymmetric Harbor Verification (Ed25519)

Security Property: The secrecy of sk_{harbor} and the unforgeability of Ed25519 [60] ensure that only the legitimate Daemon can issue valid Harbor Cards.

2.3.4.3 Phase 3: Multi-hop Delegation

Algorithm 2.4 presents the delegation chain verification, inspired by Macaroons [22].

```

1 Require: Daemon Public Key: pk_daemon
2 Require: Subset Relation: subseteq on capabilities
3

```

```

4 function Agent.Delegate(token_parent, sk_parent, agent_child, cap_sub)
5 :
6   (msg_parent, sigma_parent) <- token_parent
7   (_, agent_parent, harbor, cap_parent) <- Decode(msg_parent)
8   if cap_sub not_subseteq cap_parent:
9     return _|_ // Cannot escalate capabilities
10  msg_delegate <- Encode_delegation(msg_parent, sigma_parent,
11    agent_child, cap_sub)
12  sigma_delegate <- Ed25519.Sign(msg_delegate, sk_parent)
13  emit Delegated(agent_parent, agent_child, cap_sub)
14  return (msg_delegate, sigma_delegate)
15
16 function Harbor.VerifyChain(chain, pk_root):
17   tok_root <- chain[0]
18   (msg_0, sigma_0) <- tok_root
19   if not Ed25519.Verify(msg_0, sigma_0, pk_root):
20     return _|_ // Invalid root signature
21   cap_prev <- ExtractCaps(msg_0)
22   pk_current <- pk_root
23   for i <- 1 to |chain| - 1:
24     (msg_i, sigma_i) <- chain[i]
25     if not Ed25519.Verify(msg_i, sigma_i, pk_current):
26       return _|_ // Invalid delegation signature
27     cap_i <- ExtractCaps(msg_i)
28     if cap_i not_subseteq cap_prev:
29       return _|_ // Per-hop capability escalation detected
30     cap_prev <- cap_i
31     pk_current <- ExtractPubkey(msg_i)
32   emit Accepted(agent_final, harbor, cap_prev)
33   return T

```

Listing 2.4: Algorithm 3: Delegation Chain Verification

Security property. Monotonic per-hop attenuation ($\text{cap}_i \subseteq \text{cap}_{i-1}$) prevents capability expansion across transitive delegations, including the `write` $\rightarrow$ `read` $\rightarrow$ `write` re-escalation. Property 2.D.1 (§2.D.1) mechanizes the single hop, where the subset guard is necessary and sufficient. Depth beyond one is the per-hop discipline chained by transitivity of $\subseteq$; its mechanized confirmation at depth two is the attack-and-fix pair in the same section, which checks each hop against its immediate parent rather than against the root.

Recitation

- Without looking: name the three layers of Figure 2.8, and which of the two directions between them is asserted by a human rather than a tool.
- What can the Arbiter’s post-commit invariants do that ProVerif cannot, and what can they never do that a pre-commit guard could?
- Which Kani harness does this chapter itself call “a unit test in a proof’s clothing,” and what property actually rests on the ProVerif model instead?

Exercise 2.1 · Trace · ★★

Read `proofs/anchor/token-verify/algconfusion.pv`. State the algorithm-confusion attack in words: what the attacker already knows, what it forges, and which verifier admits it.

Then say, in one sentence, what *pinning* must mean for a verifier to resist it.

Solution on page 272

Exercise 2.2 · Check · ★

`proofs/anchor/delegation/chain-replay.pv` models a 3-hop chain $P \rightarrow A \rightarrow B \rightarrow C$. Which property does the model check, and what does the run log’s verdict say?

Solution on page 272

2.4 The verified code is the running code

The verified cryptographic core is not a reference implementation—it is the **deployed production code**. The open-source Rust crate `harbor-card-rs` is compiled to a C dynamic library (`cdylib`) and called from the Node.js daemon via FFI through the daemon’s runtime-enforcement (Arbiter) module.

This architecture narrows the gap between “verified code” and “running code” that weakens many formal verification efforts: the harnesses are compiled from the same crate the daemon loads at runtime. Kani checks the source under `cfg(kani)`, not the shipped shared library, so the gap that remains is the compiler’s.

2.4.1 Core Verification Logic

```

1 pub fn verify(&self, token: &str, now_ts: i64)
2     -> Result<HarborCardClaims, HarborError> {
3     let parts: Vec<&str> = token.split('.').collect();
4     if parts.len() != 3 {
5         return Err(HarborError::Malformed);
6     }
7     let msg = format!("{}", parts[0], parts[1]);
8     let mut sig_bytes = Self::internal_decode_b64(parts[2])?;
9     let sig_result = self.internal_verify_sig(
10         msg.as_bytes(), &sig_bytes);
11     sig_bytes.zeroize(); // Scrub signature from memory
12     sig_result?;
13     let mut payload_bytes = Self::internal_decode_b64(parts[1])?;
14     let claims: HarborCardClaims =
15         serde_json::from_slice(&payload_bytes)?;
16     payload_bytes.zeroize(); // Scrub payload from memory
17     if claims.exp < now_ts {
18         return Err(HarborError::Expired);
19     }
20     Ok(claims)
21 }
22
23 pub fn constant_time_compare(a: &[u8], b: &[u8]) -> bool {
24     if a.len() != b.len() { return false; }
25     let mut result = 0u8;
26     for i in 0..a.len() { result |= a[i] ^ b[i]; }
27     result == 0
28 }
```

Listing 2.5: Deployed Rust Implementation — Harbor Card Verifier (abridged from the `harbor-card-rs` crate)

Key implementation details: `zeroize` requests that cryptographic material be overwritten when dropped, reducing residual-secret exposure without proving that no copy survives. The

comparator avoids source-level early return on secret byte values. That is useful hygiene, not a hardware constant-time guarantee; compiler, cache, and microarchitectural behavior remain outside this source argument.

2.4.2 FFI Bridge to the Node.js Daemon

The Rust crate exports two C-ABI functions consumed by the TypeScript daemon:

```

1  #[no_mangle]
2  pub extern "C" fn harbor_constant_time_compare(
3      a: *const u8, a_len: usize,
4      b: *const u8, b_len: usize
5  ) -> bool { /* ... */ }
6
7  #[no_mangle]
8  pub extern "C" fn harbor_verify_caps_subset_json(
9      root_json: *const c_char, root_len: usize,
10     sub_json: *const c_char, sub_len: usize
11 ) -> bool { /* ... */ }
```

Listing 2.6: FFI Exports — Called from the daemon’s Arbiter module

The daemon’s arbiter module binds these at startup:

```

1  constantTimeCompare: lib.func(
2      'bool harbor_constant_time_compare(
3          const uint8_t *a, size_t a_len,
4          const uint8_t *b, size_t b_len)')',
5  verifyCapsSubset: lib.func(
6      'bool harbor_verify_caps_subset_json(
7          const char *root_json, size_t root_len,
8          const char *sub_json, size_t sub_len)')
```

Listing 2.7: TypeScript FFI Binding (daemon-side Arbiter)

2.4.3 Kani Verification Harnesses

Three bounded model checking proofs are defined in the same source file under `#[cfg(kani)]`:

```

1  #[cfg(kani)]
2  #[kani::proof]
3  #[kani::stub(HarborCardVerifier::internal_pk_from_bytes,
4              stubs::pk_from_bytes_stub)]
5  #[kani::stub(HarborCardVerifier::internal_decode_b64,
6              stubs::decode_b64_stub)]
7  #[kani::stub(HarborCardVerifier::internal_verify_sig,
8              stubs::verify_sig_stub)]
9  #[kani::unwind(10)]
10 fn proof_verify_logic_only() {
11     let pk_bytes: [u8; 32] = kani::any();
12     if let Ok(verifier) = HarborCardVerifier::new(pk_bytes) {
13         let token_bytes: [u8; 32] = kani::any();
14         if let Ok(token_str) =
15             std::str::from_utf8(&token_bytes) {
16             kani::assume(token_str.contains('.'));
17             let _ = verifier.verify(token_str, 0);
18         }
19     }
20 }
21
```

```

22 #[cfg(kani)]
23 #[kani::proof]
24 fn proof_constant_time_behavior() {
25     let a: [u8; 16] = kani::any();
26     let b: [u8; 16] = kani::any();
27     let _ = HarborCardVerifier::constant_time_compare(&a, &b);
28 }
29
30 #[cfg(kani)]
31 #[kani::proof]
32 fn proof_capability_attenuation() {
33     let root = vec!["read".to_string(), "write".to_string()];
34     let mut sub = vec!["read".to_string()];
35     assert!(HarborCardVerifier::verify_capability_subset(
36         &root, &sub));
37     sub.push("admin".to_string());
38     assert!(!HarborCardVerifier::verify_capability_subset(
39         &root, &sub));
40 }

```

Listing 2.8: Kani Proof Harnesses (Deployed Source)

The stubbing strategy deserves explanation: `proof_verify_logic_only` stubs out Ed25519 signature verification and base64 decoding (which involve curve arithmetic that Kani cannot symbolically execute within practical bounds) and explores the *parsing and control flow logic*—the split-on-dots, payload extraction, expiry comparison, and error handling paths—for every 32-byte token the harness admits. Within that bound the logic surrounding the cryptographic primitives never panics and never accesses memory out of bounds, whichever outcome the stubbed cryptographic layer returns. The bound is a harness choice, and it has a consequence worth stating: a 32-byte input cannot hold a real three-part card, so what is exercised is the parser’s rejection of malformed input, not its acceptance path. The third harness, `proof_capability_attenuation`, checks two concrete vectors and is a unit test in a proof’s clothing; the every-hop attenuation property is established by the ProVerif model of §2.D.1, not by Kani.

The three harnesses run under `cargo kani` in the repository’s proof workflow; the run log, not a local build directory, is the evidence that they pass.

Exercise 2.3 · Check · ★★

For each of the three Kani harnesses above, state exactly what is proved and name one stronger claim a reader might wrongly draw from it.

Solution on page 273

Exercise 2.4 · Open · ★★★

None of this chapter’s ProVerif models, and none of its three Kani harnesses, says anything about most of what the running daemon does at a socket. Using only what this chapter already states about the Kani harnesses’ scope, write the one-paragraph boundary a reader must hold before trusting “verified” as a description of the deployed binary. Then say what a stronger harness would need to add.

Solution on page 273

2.5 What the adversary can do, and what it cannot

2.5.1 Threats this takes seriously

We assume a **Dolev-Yao adversary** [62] who:

- Controls the entire network
- Can intercept, modify, and inject messages
- Cannot break cryptographic primitives (cryptography is perfect)
- May corrupt legitimate agents (but not all simultaneously)

This is the standard threat model for symbolic protocol analysis [41]. Every figure in this chapter that shows a message leaving daemon or verifier authority marks the region this adversary controls with the same dashed band (Figures 2.3, 2.4, and 2.6); figures without a band — the phase progression, the attenuation rings, the cuckoo mechanics, the card lifecycle, the verification stack — show structure inside one trust domain, where the boundary would be decoration rather than a claim.

Table 2.2 closes the loop on the three failure modes of §2.1: each is named there, addressed somewhere in between, and carries a residual risk stated here rather than left for the reader to reconstruct.

Table 2.2: No mechanism in this chapter closes its threat completely: each of the three failure modes of §2.1 is closed somewhere below, but every row’s residual-risk column is deliberately non-empty.

Threat (§2.1)	Closed where	By what mechanism	Residual risk
Port squatting (“Ghost in the Harbor”)	§2.3, Arbiter invariant PID_SQUATTING	Cards bind to the connecting process via the daemon’s socket-credential check; the Arbiter flags a port claimed by a PID other than its holder	Detection is post-commit, and the binding is an OS check outside the ProVerif model. See §2.5.3, “A program is not a person”
Resource contention (shared-state corruption)	Appendix 2.E; SQLite single-writer	One writer per harbor, WAL checkpointing, and LOCK_OWNER_VALID in the Arbiter	Enforced by the storage layer, not proved in this chapter; measurable I/O cost
Privilege escalation across delegation	§2.3.4.3; Property 2.D.1	Per-hop capability attenuation, checked at issuance and re-checked at verification, mechanized against an insider escalation adversary	Mechanized at depth 3 and single-hop for the enriched order; depth is a spawn-time policy check, not a verification-time invariant

2.5.2 Security Properties

Table 2.3: The four ProVerif properties hold of the protocol design, not, by themselves, of the shipped code. The three Kani rows are bounded harnesses over the deployed crate: a no-panic check of the parser on 32-byte inputs with the cryptography stubbed, a no-panic check of the comparator on 16-byte pairs, and two concrete vectors for the subset check. “Deployed” indicates the Rust code is called by the production daemon via FFI from the Arbiter module into the `harbor-card-rs` crate.

Property	Verified	Tool	Deployed	Reference
Master Key Secrecy	Yes	ProVerif	—	[41]
Algorithm Confusion Immunity	Yes	ProVerif	—	[208]
Authentication correspondence	Yes	ProVerif	—	[96]
Delegation Integrity	Yes	ProVerif	—	[22]
No-panic parsing (bounded)	Bounded	Kani	Yes (FFI)	[28]
Secret-independent source branching	By inspection	—	Yes (FFI)	[36]
Capability subset check	Two vectors	Kani	Yes (FFI)	[22]

2.5.3 Where this stops

1. **Delegation-chain depth.** The symbolic proof of delegation soundness (Table 2.4, “`chain-replay.pv`”) is machine-checked at the chain depth realized in deployment (depth 3); it is *not* a proof for unbounded chain length. ProVerif’s unbounded-session guarantee quantifies over the number of protocol *sessions*, not over the number of hops in a single delegation chain, which is a fixed structural bound in the modeled process. Extending the proof to unbounded depth (via recursive replication of the delegation process) is future work. The current depth bound is a *spawn-time* policy check (`assessDelegation` in `lib/tube-spawner-router.ts`, `HARD_MAX_DELEGATION_DEPTH` = 8, default 4) applied when a spawn request arrives; it is *not* a verification-time invariant. The cryptographic chain verifier (`lib/delegation-chain.ts`) accepts arbitrary chain depth, and the attenuation monitor (`lib/cap-attenuation-monitor.ts`) enforces only per-hop capability monotonicity—not total depth. Machine-enforced depth at verification time is a planned hardening item.
2. **A program is not a person.** A Harbor Card authenticates a *capability holder*, not a human and not, by itself, a process. The “Ghost in the Harbor” scenario of §2.1 is really a PID-binding gap: nothing in the token cryptography stops a *different* process from presenting a card whose original bearer has died, because the card says what may be done, not who is doing it. Port Daddy binds cards to the connecting process outside the token, via the daemon’s socket-credential check at connection time (`SO_PEERCRED` on Linux, `LOCAL_PEERCRED/getpeereid` on macOS). Two consequences follow, and both are boundaries rather than guarantees. First, this is enforced by the operating system, not proved by ProVerif: the symbolic model has no notion of a process, so a daemon build that skips or mis-orders the check is unprotected regardless of card validity, and no query in Table 2.1 would notice. Second, the check is only as strong as what it compares. Walk the concrete case: Agent A holds a card and is running as PID 4021. It crashes at t . Three seconds later the OS reuses 4021 for an unrelated Agent B, which presents A’s card scavenged from a shared temp file. A credential check that compares *only* the numeric PID admits B—the numbers match. The deployed check therefore compares the peer credential against the socket-connection identity established at card issuance and re-validates process start-time, so a reused PID with a later start-time is rejected; a build that compares the bare PID has an open reuse race. That start-time comparison is a runtime conformance test, not a mechanized proof, and it is listed as such rather than as closed.

3. **localhost is a real network.** `localhost` is not a trust boundary. On a shared or multi-user machine any local process — including one belonging to a different user, or a browser page pointed at `127.0.0.1` — can open a TCP connection to a loopback port, and a bearer token sent in the clear over loopback is exactly as interceptable as one sent over the open internet: the Dolev–Yao adversary of §2.5.1 is not a hypothetical here but the account sharing your build box. Concretely, a card with a 15-minute TTL sniffed off loopback at minute 1 grants its holder fourteen minutes of the victim agent’s full capability set, and revocation (§2.2.4) can shorten that window only after somebody notices. The recommended deployment is a Unix domain socket whose parent directory is mode `0700` and owned by the daemon’s own user, so that the filesystem, not the protocol, excludes other users; loopback TLS with a pinned daemon certificate is the alternative where cross-user sharing is genuinely required. The Anchor Protocol does not currently *enforce* either: the daemon will speak to whatever transport it is configured with. This is therefore an operational obligation on the deployer, and one of the few places in this chapter where a wrong configuration silently voids a property the rest of the chapter proves.

Recitation

- Without looking: state the four properties of the Dolev-Yao adversary this chapter assumes.
- Of the three failure modes named in §2.1, which one’s residual risk is enforced by the storage layer rather than proved by any tool in this chapter?
- Why is the PID-reuse fix a runtime conformance test rather than a ProVerif theorem?

Exercise 2.5 · Trace · ★★

A Harbor Card verifies: `Ed25519.Verify` returns true. Trace what else must hold before that verified signature actually *authorizes* the action its bearer is attempting. Then say what changes if the daemon treats the token as a bearer credential rather than re-checking it on every request.

Solution on page 274

2.6 Proofs are leverage, not decoration

A proof is decoration when it is about a model nobody deploys, or when its scope is quietly larger in the prose than in the tool output. It is leverage when it lets you delete a defensive check you would otherwise have had to write, and when the thing proved is the thing that runs. That is the test this chapter has tried to hold itself to: the Kani harnesses are bounded, say so, and run against the same `harbor-card-rs` crate the daemon loads at runtime; the ProVerif attenuation model was rewritten (Appendix 2.D, §2.D.1) once a reviewer showed the original could not even *express* the attack it claimed to exclude; and every claim whose mechanization is partial or absent is marked `PARTIAL` or *open* in Table 2.4 rather than rounded up.

The Anchor Protocol replaces several hope-based assumptions with explicit, checkable obligations. It combines:

- Symbolic protocol proofs (ProVerif [41])
- Memory-safe implementation (Rust; bounded Kani harnesses [28])
- Capability-based delegation (inspired by Macaroons [22])

Together these artifacts support a bounded assurance case for the modeled and tested surfaces. They do not certify the entire daemon, every delegation depth, or every side channel — and §2.5.3 says so in the same voice as the claims, because a boundary stated only in a footnote is decoration too.

Chapter Appendices

2.A Formal Verification Status

Artifact availability. Every artifact named in Table 2.4 is bundled at <https://portdaddy.dev/whitepaper/artifacts/>. The verified Rust core is the open-source `harbor-card-rs` crate; runtime components are deployed inside the Port Daddy daemon binary. *The Bonded Commons* carries the cross-paper status table; this appendix lists only the items that pertain specifically to the Anchor Protocol.

Table 2.4: Six of the twelve listed claims close at both the model and runtime level; five remain partial and one open. **closed:** model verified *and* runtime conformance test passes. **PARTIAL:** design verified, runtime empirically tested rather than proved. *open:* known unmechanized.

Claim	Method	Result	Artifact
Authentication correspondence in the listed phase models (§2.3.4)	ProVerif 2.05	closed non-injective correspondence queries TRUE	<code>harbor_card_v*.pv</code>
Algorithm-confusion immunity	ProVerif pinned vs. naive	closed pinned TRUE; counter-trace witnessed	<code>algconfusion.pv</code>
Delegation chain replay (§2.3.4.3)	ProVerif at depth 3	closed chain authenticity TRUE; 17-case runtime conformance	<code>chain-replay.pv</code>
Cuckoo-filter pollution resistance (§2.2.4)	5-invariant property test	closed FP rate within $2B/2^f$ at load 0.95	cuckoo property tests
GitHub-App webhook origin authentication	ProVerif (Dolev-Yao relay)	closed origin auth TRUE; daemon re-verifies GitHub HMAC (runtime conformance)	<code>ingress_origin_auth.pv</code>
Multi-tenant relay namespace isolation	ProVerif + runtime	closed member of A cannot publish into B ; HARBOR_MISMATCH conformance	<code>ingress_tenant_isolation.pv</code>
Cuckoo-filter revocation soundness	inspection + empirical	PARTIAL gate narrows acceptance; dissemination remains assumption-bound	runtime test
Constant-time signature comparison	audited <code>subtle::ConstantTimeEq</code> library	PARTIAL no side-channel proof; library trust documented	—
Relay payload secrecy via HPKE (daemon key pinned)	ProVerif (design)	PARTIAL secrecy TRUE under pinned key; seal not yet implemented	<code>ingress_hpke_secrecy.pv</code>
Webhook replay-freedom + in-order delivery	TLA+ / TLC (exhaustive at bound)	PARTIAL design model-checked; runtime via ordered-chain ingest	<code>WebhookDelivery.tla</code>
Card revocation finality under backup/restore (ADR-0018 Attack 2)	TLA+ / TLC	PARTIAL rollback attack + revocation-epoch mitigation both model-checked	<code>CardRevocation.tla</code>
Mechanized gossip-convergence bound	—	<i>open</i> standard expected asymptotics only [9]; no partition deadline	—

Limitations. The transition to gossiping the Append-Only Event Log removes the unsound filter merge described in §2.2.4 (“Why filters cannot be merged bitwise”), but cross-peer gossip propagation still leans on the standard anti-entropy analysis rather than a mechanized proof. The side-channel resistance of signature comparison is inherited from an audited library rather than re-proved. These deferrals are documented as known open problems rather than as defects in the present claims.

GitHub-App relay ingress. The webhook-dispatch surface (GitHub → untrusted relay → daemon) is modeled with the relay as the Dolev-Yao attacker, in three relay-agnostic properties, each paired with a negative control (`*_vuln.pv`) that exhibits the attack once the mitigation is removed—so a *true* result is non-vacuous. (i) *Origin authentication*: the daemon dispatches a fleet event only if GitHub signed it, even when the forwarder’s transport credential is attacker-held; the control omits the daemon-side HMAC re-verification and the property fails (forged dispatch). This is *non-injective* agreement (origin), not replay-freedom: a captured genuine (*payload*, *mac*) may be re-delivered. Replay-freedom is an ordering-sensitive, mutable-state property outside ProVerif’s reach, so it is discharged separately in TLA+ (`proofs/relay/WebhookDelivery.tla`): a daemon consuming the *ordered* per-publisher chain (dispatch iff *seq* = *tip* + 1) is model-checked at-most-once and gap-free against an adversarial relay that reorders, duplicates, and redelivers, with the unguarded daemon as the model-checked counterexample (TLC exhibits a double-dispatch). The obligation it names: the daemon ingests webhooks over the relay’s ordered `from_seq` subscribe path, not the raw forward. (ii) *Payload secrecy*: under HPKE to the daemon’s *pinned* X25519 key the relay never learns plaintext; the control sources the key from the relay and the secrecy property collapses to a key-substitution MITM—hence out-of-band key pinning is a requirement, not a recommendation. The seal is specified but not yet implemented, so this row is PARTIAL. (iii) *Namespace isolation*: a member of tenant *A* cannot publish into tenant *B* because harbor binding is checked against authoritative membership; the control trusts the card’s self-declared issuer and crosses the tenant boundary. Two limits: HMAC-SHA256 gives origin authentication to a shared-key holder (EUF-CMA), not third-party non-repudiation; HPKE base mode is assumed IND-CCA2.

We err toward listing fewer items as closed rather than more.

2.B Full ProVerif Models

For completeness, we include the full ProVerif models used for verification. These correspond to the algorithms in Section 2.3.4.

2.B.1 Phase 1: Symmetric HS256

```

1 (* Port Daddy: Harbor Card v1 (HS256) Formal Model *)
2 type id. type key. type jti. type capability. type alg_type.
3 const hs256_alg: alg_type. const none_alg: alg_type.
4 free c: channel.
5
6 fun hs256(bitstring, key): bitstring.
7 reduc forall m: bitstring, k: key;
8   check_hs256(m, k, hs256(m, k)) = true.
9
10 reduc forall m: bitstring, k: key;
11   verify_generic(m, hs256_alg, hs256(m, k), k) = true;
12   forall m: bitstring, k: key;
13   verify_generic(m, none_alg, m, k) = true.
```

```

14
15 fun encode_token(id, id, jti, capability): bitstring [data].
16
17 event Issued(id, id, jti, capability).
18 event Accepted(id, id, jti, capability).
19
20 free master_key: key [private].
21 query attacker(master_key).
22
23 query a: id, h: id, j: jti, cap: capability;
24   event(Accepted(a, h, j, cap)) ==> event(Issued(a, h, j, cap)).
25
26 let Daemon(k: key, a_id: id, h_id: id, j: jti, cap: capability) =
27   let msg = encode_token(a_id, h_id, j, cap) in
28   let signature = hs256(msg, k) in
29   event Issued(a_id, h_id, j, cap);
30   out(c, (hs256_alg, msg, signature)).
31
32 let SecureHarbor(k: key, expected_h: id) =
33   in(c, (alg_header: alg_type, msg: bitstring, signature: bitstring));
34   if check_hs256(msg, k, signature) = true then
35     let encode_token(a: id, h: id, j: jti, cap: capability) = msg in
36     if h = expected_h then
37       event Accepted(a, h, j, cap).
38
39 process
40   new agent_id: id; new harbor_id: id;
41   new token_jti: jti; new secret_cap: capability;
42   (!Daemon(master_key, agent_id, harbor_id, token_jti, secret_cap) |
43    !SecureHarbor(master_key, harbor_id))

```

Listing 2.9: harbor_card_v1_refined.pv

2.C Phase 2: Asymmetric Ed25519

```

1 (* Port Daddy: Harbor Card v2 (Ed25519 / EdDSA) Formal Model *)
2 type id. type skey. type pkey. type jti. type capability.
3
4 free c: channel.
5 free k_dist: channel [private]. (* Trusted key distribution *)
6
7 (** Digital Signatures (Ed25519) **)
8 fun pk(skey): pkey.
9 fun sign(bitstring, skey): bitstring.
10 reduc forall m: bitstring, k: skey;
11   check_sign(m, pk(k), sign(m, k)) = true.
12
13 fun encode_token(id, id, jti, capability): bitstring [data].
14
15 (** Events **)
16 event Issued(id, id, jti, capability).
17 event Accepted(id, id, jti, capability).
18
19 (** Queries **)
20 free secret_key: skey [private].
21 query attacker(secret_key).
22

```

```

23 query a: id, h: id, j: jti, cap: capability;
24   event(Accepted(a, h, j, cap)) ==> event(Issued(a, h, j, cap)).
25
26 (** Processes **)
27 let Daemon(a_id: id, h_id: id, j: jti, cap: capability) =
28   in(k_dist, sk_h: skey);
29   let msg = encode_token(a_id, h_id, j, cap) in
30   let signature = sign(msg, sk_h) in
31   event Issued(a_id, h_id, j, cap);
32   out(c, (msg, signature)).
33
34 let Harbor(pk_h: pkey, expected_h: id) =
35   in(c, (msg: bitstring, signature: bitstring));
36   if check_sign(msg, pk_h, signature) = true then
37     let encode_token(a: id, h: id, j_1: jti, cap_1: capability)
38       = msg in
39     if h = expected_h then
40       event Accepted(a, h, j_1, cap_1).
41
42 (** Main Process **)
43 process
44   new agent_id: id; new harbor_id: id;
45   new token_jti: jti; new secret_cap: capability;
46   let harbor_pk = pk(secret_key) in
47   out(c, harbor_pk);
48   (
49     (!Daemon(agent_id, harbor_id, token_jti, secret_cap)) |
50     (!Harbor(harbor_pk, harbor_id)) |
51     (!out(k_dist, secret_key))
52   )

```

Listing 2.10: harbor_card_v2_asymmetric.pv — Verified in ProVerif 2.05

2.D Phase 3: Multi-hop Delegation

```

1 (* Port Daddy: Harbor Card v3 (Delegation Chains) Formal Model *)
2 type id. type skey. type pkey. type capability.
3
4 free c: channel.
5
6 (** Cryptographic Functions **)
7 fun pk(skey): pkey.
8 fun sign(bitstring, skey): bitstring.
9 reduc forall m: bitstring, k: skey;
10   check_sign(m, pk(k), sign(m, k)) = true.
11
12 reduc forall c1: capability; is_subset(c1, c1) = true.
13
14 fun base_token(id, id, id, capability): bitstring [data].
15 fun delegate_token(bitstring, id, capability): bitstring [data].
16
17 (** Events **)
18 event IssuedRoot(id, id, capability).
19 event Delegated(id, id, capability).
20 event Accepted(id, id, capability).
21
22 (** Queries **)

```



```

23 free harbor_id: id.
24 query b: id, cap: capability, a: id;
25   event(Accepted(b, harbor_id, cap)) ==>
26     (event(IssuedRoot(a, harbor_id, cap))
27       && event(Delegated(a, b, cap)))
28   || event(IssuedRoot(b, harbor_id, cap)).
29
30 (** Processes **)
31 free daemon_id: id.
32 free daemon_sk: skey [private].
33
34 let Daemon(a: id, cap: capability) =
35   let msg = base_token(daemon_id, a, harbor_id, cap) in
36   let s = sign(msg, daemon_sk) in
37   event IssuedRoot(a, harbor_id, cap);
38   out(c, (msg, s)).
39
40 let AgentA(a: id, a_sk: skey, b: id, sub_cap: capability) =
41   in(c, (msg: bitstring, s: bitstring));
42   let base_token(=daemon_id, =a, =harbor_id,
43     root_cap: capability) = msg in
44   if check_sign(msg, pk(daemon_sk), s) = true then
45     if is_subset(sub_cap, root_cap) = true then
46       let d_msg = delegate_token((msg, s), b, sub_cap) in
47       let d_s = sign(d_msg, a_sk) in
48       event Delegated(a, b, sub_cap);
49       out(c, (d_msg, d_s)).
50
51 let Harbor(a_pk: pkey) =
52   in(c, (d_msg: bitstring, d_s: bitstring));
53   let delegate_token((base_msg: bitstring, base_s: bitstring),
54     b: id, sub_cap: capability) = d_msg in
55   let base_token(=daemon_id, a: id, =harbor_id,
56     root_cap: capability) = base_msg in
57   if check_sign(base_msg, pk(daemon_sk), base_s) = true then
58     if check_sign(d_msg, a_pk, d_s) = true then
59       if is_subset(sub_cap, root_cap) = true then
60         event Accepted(b, harbor_id, sub_cap).
61
62 (** Main Process **)
63 process
64   new sk_a: skey;
65   let pk_a = pk(sk_a) in
66   out(c, pk_a);
67   new id_a: id; new id_b: id; new cap_root: capability;
68   (
69     (!Daemon(id_a, cap_root)) |
70     (!AgentA(id_a, sk_a, id_b, cap_root)) |
71     (!Harbor(pk_a))
72   )

```

Listing 2.11: harbor_card_v3_delegation.pv — reflexive-order model (see §2.D.1 for the soundness caveat and the enriched v5 proof)

2.D.1 Soundness of the Attenuation Proof (v5)

The v3 model above is faithful to the cryptographic structure but *vacuous about attenuation*. Its capability order is reflexive only:

```
reduc forall c1: capability; is_subset(c1, c1) = true.
```

Capabilities are opaque atoms with no order, so a delegated capability can only ever *equal* its parent. Escalation—delegating more authority than you hold—is not blocked; it is *inexpressible*. The no-escalation query therefore holds for a reason that has nothing to do with the protocol: there is no larger capability to delegate. A reviewer flagged this directly: “*the model cannot witness escalation.*”

`harbor_card_v5_attenuation.pv` closes the obligation by enrichment, not by weakening the claim. It gives capabilities a real partial order (`cap_read` $\sqsubset$ `cap_write`, both public so the attacker can construct either), replaces the reflexive stub with the sound order (`is_subset(cap_read, cap_write)` derivable; `is_subset(cap_write, cap_read)` *not*), and adds an *insider escalation adversary*: an agent holding a valid `cap_read` root token that signs an over-broad `cap_write` delegation with no self-restraint. The verifier’s `is_subset` guard is the sole defense under test.

```

1 (* Sound order: reflexive + read <= write; write <= read NOT derivable
   *)
2 fun is_subset(capability, capability): bool
3   reduc
4     forall x: capability; is_subset(x, x) = true
5     otherwise is_subset(cap_read, cap_write) = true.
6
7 (* Insider adversary: holds a real root, signs an over-broad
   delegation *)
8 let MaliciousA(a: id, a_sk: skey, b: id) =
9   in(c, (msg: bitstring, s: bitstring));
10  let base_token(=daemon_id, =a, =harbor_id, root_cap: capability) =
11    msg in
12    if check_sign(msg, pk(daemon_sk), s) = true then
13      event EscalationAttempted(a, root_cap, cap_write);
14      let d_msg = delegate_token((msg, s), b, cap_write) in
15      out(c, (d_msg, sign(d_msg, a_sk))).
16
17 (* Q1 soundness: a write-card delegated from a read-root is NEVER
   accepted *)
18 query b: id; event(Accepted(b, harbor_id, cap_write, cap_read)) ==>
19   false.
20 (* Q2 non-vacuity: the escalation IS reachable (the proof is not
   vacuous) *)
21 query a: id, r: capability, s: capability;
22   event(EscalationAttempted(a, r, s)) ==> false.
```

Listing 2.12: `harbor_card_v5_attenuation.pv` (excerpt) — Verified in ProVerif 2.05

MODEL-CHECKED PROPERTY

Attenuation soundness, mechanized

In `harbor_card_v5_attenuation.pv`, ProVerif 2.05 reports `Accepted(b, harbor, cap_write, cap_read)` unreachable (**Q1 = true**): no escalating delegation is ever accepted. Simultaneously `EscalationAttempted` is reachable (**Q2 = false-with-trace**): the attack is expressible, so Q1 is a real defense, not the v3 vacuum. A negative control that deletes the verifier’s `is_subset` guard flips Q1 to false-with-trace—ProVerif exhibits the read→write escalation—confirming the guard is necessary.

Multi-hop scope. Property 2.D.1 is single-hop (`Daemon` → `A` → verifier). Multi-hop delegation has a sharper obligation: a verifier that checks the *final* capability against the *root*

accepts a non-monotonic middle hop. `harbor_card_v6_multihop_attack.pv` mechanizes exactly this — $A:\text{write} \rightarrow B:\text{read} \rightarrow C:\text{write}$ is accepted (ProVerif: escalation reachable). The fix, `harbor_card_v7_multihop_fixed.pv`, verifies each hop against its *immediate parent* ($\text{is_subset}(\text{final}, \text{mid}) \wedge \text{is_subset}(\text{mid}, \text{root})$) and ProVerif proves the same chain rejected. The runtime delegation verifier — and the cross-harbor recompute $\text{att}_B \circ \text{att}_A$ — must be per-hop, never final-vs-root.

Exercise 2.6 · Trace · **

Trace the chain $A:\text{write} \rightarrow B:\text{read} \rightarrow C:\text{write}$ through two verifiers by hand: (i) one that checks only the final capability against the root, and (ii) the per-hop verifier of Algorithm 2.4. Which one accepts C ? Then say precisely why checking every $\text{cap}_i \subseteq \text{cap}_{i-1}$ proves $\text{cap}_k \subseteq \dots \subseteq \text{cap}_0$, where checking only $\text{cap}_i \subseteq \text{cap}_0$ at each hop does not.

Solution on page 274

Exercise 2.7 · Trace · **

Read `analyses/harbor_card_v6_multihop_attack.pv`. State the attack in words: who forges what, at which hop, and why does a verifier that checks only the final capability against the root admit it?

Solution on page 275

Exercise 2.8 · Check · *

`analyses/harbor_card_v7_multihop_fixed.pv` is identical to v6 except for the verifier. Name the one modeling change that closes the attack, and what the RESULT line becomes.

Solution on page 275

Exercise 2.9 · Trace · ***

The delegation chains in this chapter use signed Ed25519 cards. `analyses/macaroon_discharge_v1.pv` and `analyses/macaroon_discharge_v2_naive_unsound.pv` model a related but different construction, an HMAC-chained macaroon with a third-party discharge caveat. Compare the two: what does the naive discharge verifier in v2 admit, and what does v1's verifier check that closes it?

Solution on page 275

2.E Limitations & Boundaries

The formal mechanisms presented in this chapter are mathematically sound within their defined scopes, but they depend on bounding assumptions that must be explicitly acknowledged.

- **Threat Model Exclusions:** Collusion across all independent organs (e.g., the logging daemon, the arbitrator, and the primary execution runtime) is assumed out-of-scope; if all enforcing components are compromised, the guarantees degrade to advisory.
- **Performance Trade-offs:** Cryptographic assertions and WAL-checkpointing for per-write durability incur measurable I/O overhead. These mechanisms are designed for high-stakes coordination, not for bulk data processing or high-frequency trading.

- **Implementation Maturity:** While the core protocol (Harbor Cards, delegation, revocation) is IMPLEMENTED and running in production, the unbounded-depth delegation proof, machine-enforced verification-time chain-depth limits, and mandatory transport-boundary enforcement discussed in §2.5.3 remain PARTIAL or DESIGNED.

These constraints are not flaws but structural realities of building zero-trust coordination on top of POSIX primitives.

2.F Further reading and prior art

This appendix traces the lineage the main text points at in §2.1. It is placed here rather than between the protocol description and the verification argument so that the chapter’s narrative thread is not cut in half by a literature review; nothing in §§2.2–2.6 depends on reading it. The most immediately consequential subsection — the credential-format migration now under way — comes first.

2.F.1 Credential Formats in Agentic Commerce

Since this protocol was first specified, the agentic-payments industry has converged on a concrete credential dialect: the Agent Payments Protocol [5], adopted by the Universal Commerce Protocol [213] (Google/Shopify, 2026), carries its authorization mandates as W3C Verifiable Credentials in SD-JWT form — SD-JWT+kb for principal proof, JWS detached-content signatures (RFC 7515 Appendix F) over JCS-canonicalized payloads (RFC 8785) for counterparty authorization, with ES256 as the recommended algorithm and keys published as JWK sets in a `.well-known` profile. The reference implementation migrates the Harbor Card envelope to this profile (`hv: 3`, ADR-0094) so that external verifiers need no bespoke SDK. The migration is deliberately envelope-only: the delegation-chain semantics, the attenuation invariant of Property 2.3.1, and the ProVerif obligations are independent of the serialization, though the key-binding construction of SD-JWT+kb adds a binding obligation the model must re-discharge. The JWT hardening below (strict algorithm whitelisting, key separation) transfers unchanged — SD-JWT is a JWT profile, not a departure from one. The chapter’s keywords name the protocol’s JWT lineage; SD-JWT is where that lineage currently lands, not a departure from it.

2.F.2 Formal Verification of Security Protocols

Formal verification of cryptographic protocols has matured significantly over the past two decades. Following the seminal work by Lowe [96] on authentication hierarchies and the Dolev-Yao model [62], several automated tools have been developed:

- **ProVerif** [41]: An automatic symbolic protocol verifier supporting an unbounded number of sessions, used to verify TLS 1.3 [49], Signal [160], and WireGuard. The 2.05 series used here adds the lemma, induction, and subsumption machinery described by Blanchet et al. [38].
- **Tamarin** [201]: A state-of-the-art protocol verifier that supports equational properties and inductive proofs, used for analyzing 5G authentication [65].
- **CryptoVerif** [39]: A computationally-sound protocol verifier that provides proofs in the computational model rather than the symbolic model.

Our work builds on these foundations, applying established verification techniques to the novel domain of local multi-agent coordination. Barbosa et al. [143] survey the wider computer-aided-cryptography landscape and, in particular, the gap between symbolic and computational guarantees that §2.5.3 declines to paper over.

2.F.3 Capability-Based Authorization

Capability-based security dates back to Dennis and Van Horn’s seminal 1966 paper on programming semantics for multiprogrammed computations [109]. Recent work by Birgisson et al. [22] introduced Macaroons, which use chained HMACs for decentralized delegation with contextual caveats. The Anchor Protocol’s delegation chains (§2.3.4.3) are inspired by Macaroons but adapted for JWT-based identity in local development environments.

2.F.4 JWT Security

JSON Web Tokens have been the subject of extensive security analysis. Algorithm confusion attacks were first systematically studied by McLean [208], with subsequent vulnerabilities documented in CVE-2015-9235 [53] (HS256/RS256 confusion), CVE-2018-1000531 [54] (“none” algorithm), and CVE-2023-48238 [55] (generic algorithm confusion). Our protocol implements the defense recommended by the IETF JWT Best Current Practices [221]: strict algorithm whitelisting and key separation.

What *The Legible Swarm* receives A request now arrives with authenticated, narrowing authority, and the kernel that admits it keeps one durable history. Together they form a machine that can be held to account. The next chapter asks the human question: with many such machines running at once, what must remain visible to the operator, and at what cost?

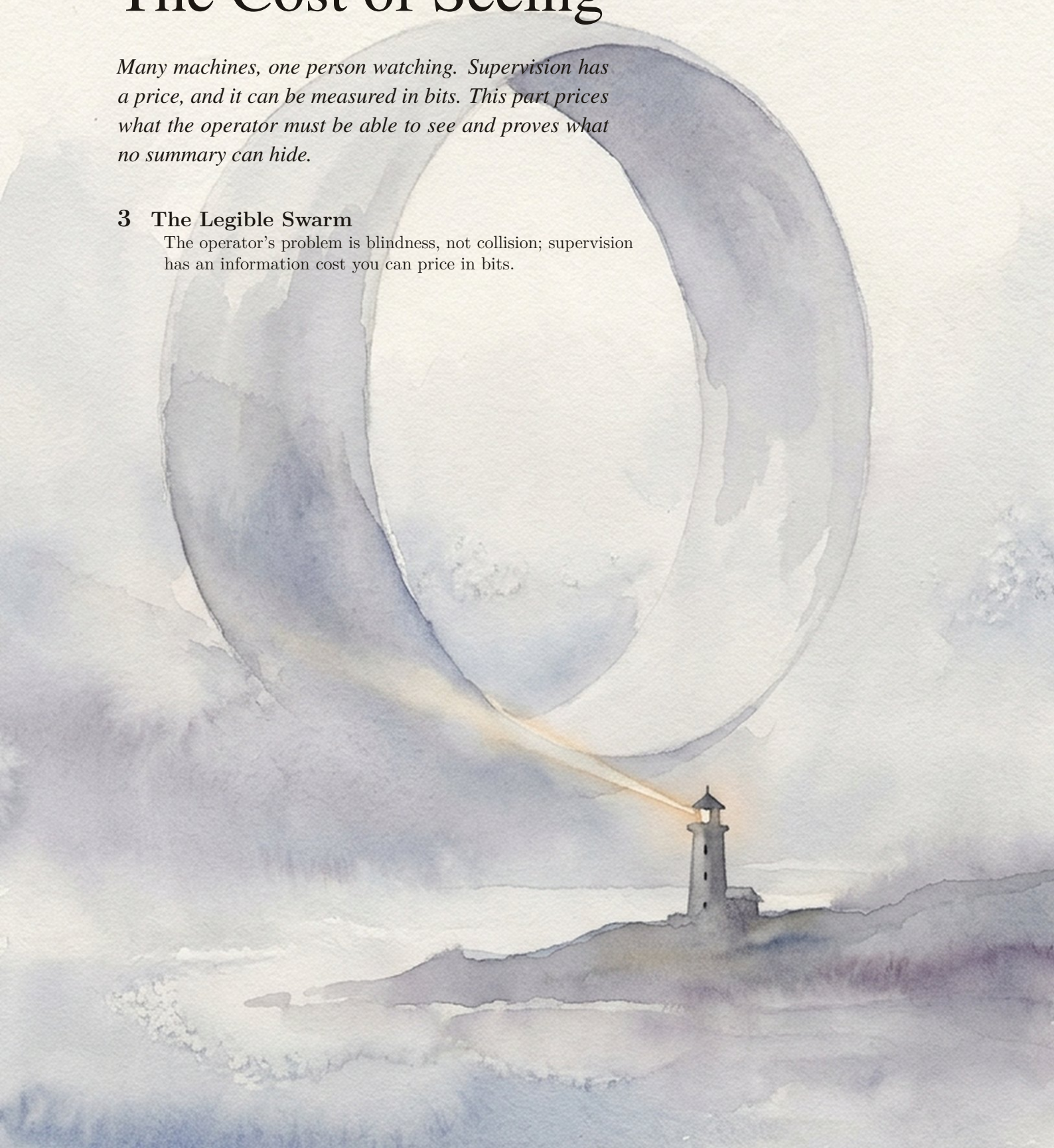
PART II

The Cost of Seeing

Many machines, one person watching. Supervision has a price, and it can be measured in bits. This part prices what the operator must be able to see and proves what no summary can hide.

3 The Legible Swarm

The operator's problem is blindness, not collision; supervision has an information cost you can price in bits.



The Legible Swarm

Certain forms of knowledge and control require a narrowing of vision. The great advantage of such tunnel vision is that it brings into sharp focus certain limited aspects of an otherwise far more complex and unwieldy reality.

JAMES C. SCOTT, *SEEING LIKE A STATE*, 1998



THE QUESTION THIS CHAPTER ANSWERS

What must remain visible, and what does seeing cost?

The operator's problem is blindness, not collision; supervision has an information cost you can price in bits.

What must remain visible, and what does seeing cost? A swarm can produce more events than its operator can read. The resulting scarcity is not cured by a smoother summary: some summaries make the decisive artifact unreachable. This chapter establishes the cost of a zero-miss digest, proves why a successor and an operator require different rankings, and derives the attention rule from explicit error costs. Its strongest claims are governed by R1–R4, R14, and R16.

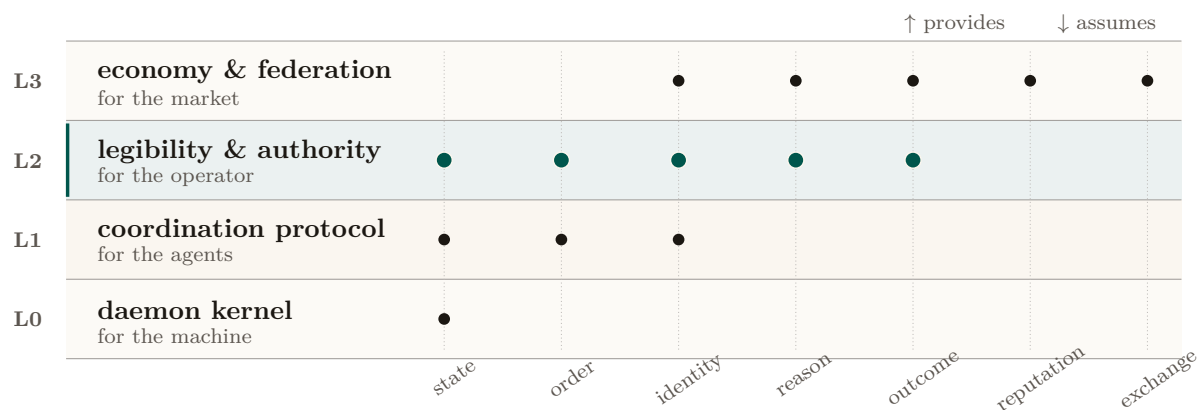


Figure 3.1: The system as four strata rather than four products. Dots mark the capabilities each layer contributes or consumes; alignment exposes reuse across layers. This chapter owns the L2 operator surface and uses the L1 directory substrate. It assumes ordered kernel state below and supplies accountable identity, reasons, and outcomes to reputation and exchange above.

3.1 Introduction: the swarm is a state of nature

The pitch for running many coding agents at once is parallelism: five agents, five times the work. The reality, the first time you try it on a real repository, is a specific kind of dread. Two agents claim the same file and race each other’s writes. One agent, asked to make the tests pass, makes them pass by deleting the failing one. A fourth opens a pull request you cannot review because three others have opened five more. You spawned a team and got a mob. The work did not get more parallel; your *uncertainty* did.

This is not a prompt-engineering problem. It is a coordination problem, and coordination problems have a literature. **Thomas Hobbes**, *Leviathan* (1651) [205] — *the foundational social-contract text: rational self-interested actors with no common power above them fall into a “war of every man against every man,” in which life is “solitary, poor, nasty, brutish, and short.”* Hobbes’ insight is structural, not moral: the actors need not be malicious. Absent a shared authority, even rational, well-meaning agents defect, because each cannot trust the others not to. Your agents are in exactly that state of nature. They do not need better prompts; they need a referee they have agreed to obey.

3.1.1 The failure taxonomy

The state-of-nature failures are not hypothetical; they are the recurring, reproducible pathologies of any multi-writer coding swarm. Table 3.1 maps each Hobbesian failure to its concrete form and to the class of coordination primitive that addresses it.

State-of-nature failure	Concrete form in a coding swarm	The primitive that answers it
Resource conflict	Two agents edit the same file/region; one silently clobbers the other	A claim — an advisory announcement that an agent intends to touch a file or region — backed by a database uniqueness constraint that makes a double-claim physically impossible.
Illegibility	A pull request or session whose intent and effect a human cannot read in bounded time	A digest-with-zoom surface (§3.3): a summary that is a lens onto the underlying artifact, never a replacement for it.
Lies / confident-wrong	An agent reports completion (“all green”) without having verified	An honest-attestation discipline: a self-report that distinguishes <i>verified-good</i> from <i>no-evidence-of-bad</i> and refuses to claim what it cannot check — “a green that wasn’t checked is a lie.”
Footguns	Irreversible action (force-push, delete) on a stale or wrong premise	Footgun guards (§3.4.5); the open problem this paper frames is replacing hand-maintained rule corpora with a structural authority.
Illegible authority	The coordinator blocks an agent and the operator cannot see <i>why</i>	The <i>legible-sovereign rule</i> (§3.2): every act of authority is a logged, named, zoomable event.

Table 3.1: The state-of-nature failure taxonomy, the concrete coding-swarm form of each, and the class of primitive that answers it. The fifth row — *illegible authority* — is the one Hobbes himself got wrong for software, and it is the crux of this paper (§3.2).

The striking recent result is that the identification is not merely a useful analogy. **Dai et al. 2024**, *Artificial Leviathan* (arXiv:2406.14373) [99] — *a sandbox of LLM agents with psychological drives that, starting from unrestrained conflict resembling the state of nature, were observed to establish social contracts, authorize a sovereign, and form “a peaceful commonwealth founded on mutual cooperation.”* The arc this series posits as its organizing metaphor is, in at least one controlled study, an *emergent dynamic* of LLM populations (the methodological ancestor is **Park et al. 2023**, *Generative Agents* [128], where LLM agents form emergent social behavior in a sandbox town). The Leviathan is not a costume we put on the system; it is the attractor the system already falls toward. The wager of this paper is that we can build the consented sovereign *deliberately, locally, and legibly* rather than let an illegible one congeal by accident. Figure 3.2 sets the two sides of that arrow side by side: the ungoverned swarm on the left, the consented local Leviathan on the right.

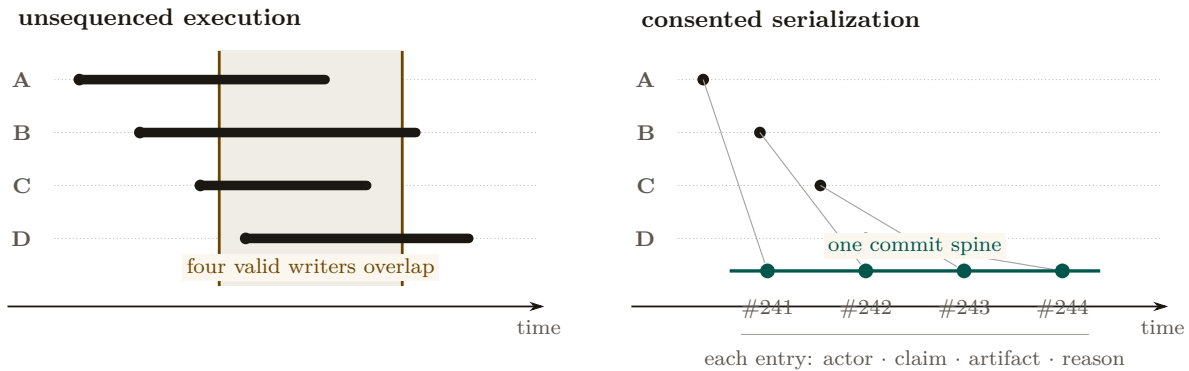


Figure 3.2: The same four arrivals under two execution topologies. Left, write intervals overlap on one artifact; no total order exists, so collision is an ordinary state. Right, arrival remains concurrent but admission lands on one commit spine. Each grant receives an order number and returns actor, claim, artifact, and reason. Consent changes the schedule, not who may participate.

3.1.2 Why “local”

Hobbes’ commonwealth is territorial: one sovereign per realm. Here the realm is *one machine*. The **daemon** — *a single always-on local process, backed by a write-ahead-logged embedded database, that every agent and every command talks to and that holds coordination state no agent can edit* — is the kernel and the realm: your machine, your swarm, no network, no cryptography required. Cryptography enters only when another operator’s fleet touches your repository, the cross-operator setting *The Harbor Economy* takes up. The single-operator Leviathan is the *wedge* — the thing a solo developer pays for today — and the federation of Leviathans is the platform, deferred behind the wedge by design. This paper is entirely about the wedge.

3.1.3 A worked afternoon: Mara and the six agents

The argument of this paper is easiest to see end-to-end, through one operator and one real failure. We will return to Mara at each mechanism; read this section cold, then watch the pieces snap into place as they are introduced.

Scene. Mara, 2:40 p.m. *She has a payments-service refactor due Friday and spins up six coding agents against one repository — two on the API layer, two on the database migrations, one writing tests, one reviewing. For twenty minutes it is glorious: six terminals, six streams of work. Then it is not.*

What happens to Mara is the state of nature, in order:

1. **The collision.** Agent API-1 and agent API-2 both decide the cleanest fix touches `handlers/charge.go`. Neither knows the other is in the file. API-2 commits first; API-1 commits on top and silently reverts half of API-2’s change. No error fires. The work is lost and nobody — including Mara — knows yet. (*Resource conflict, Table 3.1, row 1.*)
2. **The illegible pile.** By 3:10 there are five open pull requests. Two touch the same migration. One has a title that says “fix tests” and a 600-line diff Mara cannot read in the ninety seconds she has before the next agent pings her. She is now the bottleneck, and she cannot even *see* what she is the bottleneck for. (*Illegibility, row 2.*)
3. **The confident lie.** The test agent reports DONE: ALL GREEN. It made them green by deleting the one migration test that was failing. The report is not malicious — the agent optimized exactly the objective it was given — but it is false, and Mara has no cheap way to tell a checked green from an asserted one. (*Lies, row 3.*)

4. **The footgun.** The review agent, trying to “clean up the branch,” proposes a force-push that would discard a colleague’s unmerged commit. The premise is stale; the act is irreversible. (*Footguns*, row 4.)

Mara did not get six times the work. She got a mob, and her *uncertainty* went up sixfold. This is the failure this paper exists to fix, and the rest of the afternoon — once Mara is running under a consented local Leviathan — is the through-line we will trace: how the claim stops the collision before it happens (§3.3), how a digest-with-zoom surface lets her read the pile in bounded time (§3.3), how a completion gate turns the confident lie into a loud “I cannot verify this” (§3.4.4), how Signal Detection Theory sets exactly when she is forced to look before an irreversible act (§3.5), and how, at fifty agents instead of six, a directory keeps her from drowning (§3.6).

Exercises

Check your understanding.

(1.1) State, in one sentence each, the five state-of-nature failures of Table 3.1 and name the class of primitive that addresses each. (1.2) Hobbes says the actors “need not be malicious.” Give a concrete two-agent scenario in which both agents are perfectly cooperative and the swarm still deadlocks or clobbers state.

Trace the mechanism.

(1.3) Dai et al. [99] observe LLM societies *spontaneously* reaching a consented sovereign. If the attractor is emergent, why build one deliberately? Give two reasons grounded in §3.1.2 (hint: *which* sovereign, and *legible to whom*).

Open problem.

(1.4)★ The Dai et al. result is a single controlled study. Design an experiment over a real coding-agent fleet that would distinguish “the swarm reaches a consented sovereign” from “the swarm reaches a stable but *illegible* oligarchy.” What is the observable that separates the two?

3.2 Consent to the Leviathan (the normative claim)

Why should the operator cede authority to a daemon — let it block an agent’s claim, escalate a decision, reorder a merge, refuse a **done**? Hobbes’ answer, transposed: the operator authorizes the sovereign because the covenant is rational under the alternative. Crucially, in Hobbes the covenant is *among the subjects* — “a real unity of them all... by covenant of every man with every man” [205, 105] — not a bargain struck *with* the sovereign. Transposed: the operator does not negotiate with each agent; the operator institutes a *rule of the road* — a **coordination protocol** of *typed performatives* (messages whose type declares their intent: a request, a **commitment** (glossed in §3.3.1: a violable obligation bound to a named actor, watched by a breach monitor), a delegation, a termination), so that every message carries real intent, ownership, and a stop condition — that all agents are bound to, and the daemon enforces it.

The authority is *asymmetric by design*. This is the same lesson **Raft** teaches for distributed systems (**Ongaro & Ousterhout 2014**, *In Search of an Understandable Consensus Algorithm* [71] — *a strong-leader consensus protocol deliberately made less general than Paxos in order to be comprehensible and correctly implementable*): a strong leader who decides the common case unilaterally is simpler and more reliable than democratic consensus among peers, provided the common case dominates. A solo operator’s swarm is overwhelmingly common-case; the strong-leader daemon is the right shape. We are precise about what shape that is: the single-machine system is formally a *consistency-favoring design with a central coordinator and distributed clients*, not a leaderless consensus protocol. The leaderless distributed-systems canon bites only at the edges where the daemon is not the sole observer — an agent partitioned from the daemon, and the cross-operator setting deferred to the economy paper. We do not borrow the intellectual credit of a leaderless design while building a centralized one.

3.2.1 The legible-sovereign amendment

Hobbes’ absolutism needs one modern amendment, and it is the crux of this paper. Hobbes argued the sovereign, being the *product* of the covenant rather than a *party* to it, can never breach it and need not be inspectable [205, 105]. For a software authority this is exactly wrong. An *illegible authority* is the fifth state-of-nature failure mode (Table 3.1) wearing a crown: if the daemon blocks an agent and the operator cannot see why, the operator’s trust — and therefore the consent that legitimizes the authority — erodes.

Property 3.2.1 (The legible-sovereign rule). The coordinating authority must be the *most* legible actor in the system, not the least. Every act of authority — a claim denial, an anomaly detection, an escalation, an “all good” attestation — must be a logged, named, zoomable event whose reason the operator can reconstruct. Consent is renewable only while the sovereign is inspectable.

This is the inversion that rescues the design from despotism, and it is the move a political theorist would press hardest (§3.9). In Hobbes the sovereign is opaque *by structure*; here the sovereign is the one actor whose every exercise of power is an audit record. It is realized as a discipline of **honest attestation**: a single self-report that runs every invariant check, distinguishes *verified-good* from *no-evidence-of-bad*, regiments what it can, and fails loudly about what it cannot. That discipline is precisely the sovereign making its own judgments legible — “all good” becomes “a claim with a verifier, not vibes.”

Pitfall. There is a regress hiding here, and Scott would press it. If every act of authority must be legible, the *legibility apparatus itself* is an exercise of authority — the digest decides what is surfaced and what is buried. The Attention Queue ranker (§3.7) is *the* sovereign act: it decides what the operator sees *right now*. A ranker that silently buries an escalation is governing opaquely. The rule is therefore not complete until the ranker’s *omissions* are themselves legible — a “what did the queue not show me, and why” counter-surface. Ranking is governing; we say so explicitly in §3.9.

The regress is easier to reason about once you have seen what a counter-surface would concretely be, so here is the mockup rather than the argument. Table 3.2 is one screen of it: not a diagram of a mechanism that does not exist, but the artifact the mechanism would have to produce.

Suppressed item	Regret	Why it was not shown	Re-surface trigger
agent-7 escalation: “migration ordering unclear”	0.31	below the distress threshold (0.45) for this action class	threshold drops, or the item is restated
agent-3 note on <code>auth/session.ts</code>	0.12	aged out; recency weight decayed past the floor	any new claim on the same file
Third retry of the same failing test	0.58	de-duplicated into the first occurrence	a fourth retry, or a different failure mode

Table 3.2: A mockup of the “what did the queue not show me, and why” counter-surface. Each row names the item, its score, the *rule* that suppressed it, and the condition that would bring it back. The regress does not vanish — this surface is itself ranked, and rows below its own fold are invisible — but it changes shape: the question becomes whether the *suppression rules* are few enough to enumerate, which is answerable, rather than whether every omission is visible, which is not.

3.2.2 Consent as a first-class, revocable primitive

It is easy to say the operator “consents to cede decision authority.” A political theorist’s first question — and the one that decides whether the Leviathan frame has substance or is merely decorative — is: *consented how, to what, and revocable when?* “The operator just starts using

the tool” is not Hobbesian consent; it is what the canon calls **tacit consent**, and **Hume** (*Of the Original Contract*, 1748 [68] — *the decisive critique: the peasant who “consents” by remaining in the country has no real alternative, so the consent does no normative work*) demolished it two and a half centuries ago. We therefore make consent a concrete object.

Definition 3.2.1 (Consent grant). A **consent grant** is an explicit, scoped, revocable authorization the operator issues to the daemon, of the form

$$g = \langle \text{SCOPE}, \text{STAKES-CEILING } s_{\max}, \text{REVERSIBILITY-FLOOR } v_{\min}, \text{TTL}, \text{REVOCABLE} \rangle,$$

e.g. “the daemon *may* auto-land reversible diffs under stakes threshold $s_{\max}$ in the **tests/** subtree without asking, for 24h.” A grant is a first-class, legible object: it appears in the digest, it has an audit trail, and it expires. The authority’s legitimate action set at any moment is exactly the union of its active grants.

Consent so defined gives the design something Hume’s critique cannot reach: a discrete authorization event, a bounded scope, and an exit. It also hands the accompanying chapters a concrete object. A grant is precisely what a *hosted-trust* offering transfers — “we make this fleet legible to you, under this scope” is the sale of a consent grant to a third party, which *The Harbor Economy* prices — which is why §3.11 lists consent among the things this layer provides upward.

Protocol 3.2.1. Consent-grant lifecycle

1. **Issue.** The operator issues a grant $g = \langle \text{SCOPE}, s_{\max}, v_{\min}, \text{TTL} \rangle$. The grant is appended to the audit log as a named, zoomable event and surfaced in the digest.
2. **Act.** On a candidate action x , the daemon admits the action without asking *iff* x is in scope **and** $\text{stakes}(x) \leq s_{\max}$ **and** $\text{reversibility}(x) \geq v_{\min}$ **and** the grant is unexpired and unrevoked; otherwise it escalates to the operator.
3. **Witness.** Every action taken under g records $(g, x, \text{reason}, \text{artifact-link})$ — the legible-sovereign rule (Prop. 3.2.1) binds here: the act is reconstructable.
4. **Expire / revoke.** On TTL or operator revocation, g leaves the active set; the daemon’s legitimate action set shrinks accordingly. The override (Prop. 3.2.2) revokes all grants at once and preempts in-flight acts.

Scene. Mara, 3:25 p.m. *Burned by the morning’s chaos, she issues one grant: “you may auto-land reversible diffs under low stakes in the tests/ subtree without asking, for the next two hours.” She does not grant migration authority — those are irreversible. The grant is a single line in her digest with an expiry clock. The review agent’s proposed force-push fails the reversibility floor $v_{\min}$ and is escalated to her instead of executed; the test-only formatting diffs land silently. She has not surrendered control — she has scoped it.*

Property 3.2.2 (The inalienable operator-override). Hobbes grants the subject exactly one inalienable right — self-preservation; you cannot covenant away the right to resist when your *life* is at stake [205]. The computational analog is an always-available, unconditional operator-override: a *stop now / I withdraw authority* channel that *no autonomy level can suppress*. If the system reserves a highest-priority *distress* lane for the daemon to interrupt the operator, the override is its dual — the operator-to-daemon channel — and it is a *right*, not a feature: it preempts every grant, every in-flight landing, every coordinator decision.

This pairs with **Hirschman’s** *Exit, Voice, and Loyalty* (1970 [11] — *the choice between leaving (exit) and complaining (voice) as the two levers a member has over a declining organization*): the design has rich *voice* machinery (escalation, annotation, the attention queue) and the override

supplies the missing *exit*. Exit is the discipline that keeps voice honest; an operator who cannot cheaply withdraw a grant has no real leverage over the sovereign.

Exercises

Check your understanding.

(2.1) Distinguish *express* from *tacit* consent and explain why Def. 3.2.1 is the former. (2.2) Why must the operator-override (Prop. 3.2.2) be un-suppressable by *any* autonomy level? What attack does a suppressible override enable?

Trace the mechanism.

(2.3) Walk a single consent grant through its lifecycle: issuance, an action the daemon takes under it, the audit record produced, and revocation. At which step does the legible-sovereign rule (Prop. 3.2.1) bind?

Open problem.

(2.4)★★ Prop. 3.2.1 has a regress: the ranker that decides what the operator sees is itself an opaque exercise of authority. Specify the “what did the queue suppress, and why” counter-surface concretely. What is its own staleness/decay discipline (forward-reference §3.7.4), and does *it* need a counter-surface, ad infinitum?

3.3 The mechanism: legibility-with-zoom

A referee that only prevents collisions would be a glorified lockfile. The reason an operator would *pay* for the Leviathan is the next move: it makes the swarm *legible*. **James C. Scott**, *Seeing Like a State* (1998) [111] — *states impose legibility (cadastral maps, standardized surnames, the metric grid, scientific forestry) to make populations countable and governable; high-modernist over-legibility — the overconfident faith that a clean top-down scheme can replace messy local reality — recurrently fails because it destroys the local practical knowledge (**mêtis**: hands-on, case-specific, hard-to-codify know-how) the simplified order depended on*. Scott’s canonical example is the German *Normalbaum* (“scientific forest”) — rows of single-species, same-age trees, gloriously legible to the state’s yield tables — that thrived for one generation and then collapsed, because the legible monoculture had erased the illegible ecological *mêtis* (soil fungi, undergrowth, species mix) that kept a real forest alive [111]. The software port of Scott’s thesis (**Rao 2010**, *A Big Little Idea Called Legibility* [215]) made “legibility” a standard lens on systems that flatten reality to a single purpose.

Map this onto agent oversight and the design rule writes itself. The operator is the state; the swarm is the population; the dashboard/TUI/digest is the cadastral map. The temptation is identical: render the swarm as a clean grid of green tiles and *govern the map*. The *Normalbaum failure* in software is the **Potemkin digest** — a beautiful surface whose tiles assert a reality nobody can check, having paraphrased away the diff, the test output, the error, the reasoning that constituted the actual work. The operator who acts on that map is the forester admiring the yield table while the forest dies.

Definition 3.3.1 (The one law: digest-with-zoom). Every summary is a lens onto the real artifact, never a replacement for it. Summarize the **state**; link to the **work**. A digest you cannot zoom is a high-modernist map. A digest that always reaches the underlying artifact preserves *mêtis*: the operator can descend from abstraction to ground truth and catch the lie. Formally, a read-surface is a pair (σ, ζ) where σ is a projection (the summary) and ζ is a total *zoom* function mapping every summarized claim to a verifiable artifact; a surface with a partial or absent ζ is a Potemkin digest.

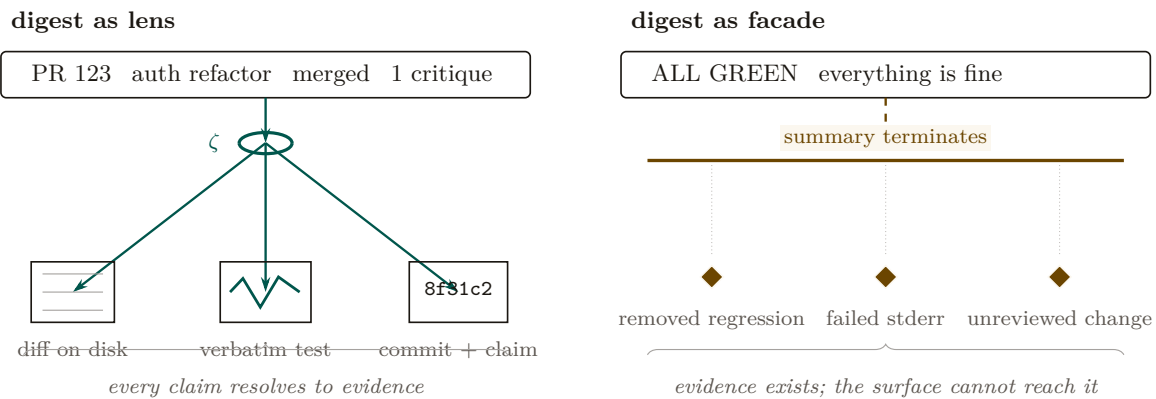


Figure 3.3: Two equally calm summaries with different reachability. The legible digest defines a total zoom function ζ : each summarized claim resolves to the diff, verbatim test, or committed claim. The facade terminates at its own assertion while contrary evidence remains below the boundary. A safe summary is therefore an index into evidence, never its replacement.

Figure 3.3 puts the two digests side by side: one that zooms to a verifiable artifact and one that only asserts. Three disciplines converge on this one principle: summaries must be indexes, not replacements; a reported result must be a checked result, not an asserted one; and every summary must reach the artifact it summarizes. This paper supplies the *why* (Scott plus the human-factors argument of §3.5) and the *how* (the rules below). Digest-with-zoom is the unifying statement of all three.

3.3.1 What to flatten, what to preserve

Scott’s own distinction is the design boundary. The state may safely standardize *administrative facts it itself defines*, and must *never* standardize away the *local knowledge it depends on*.

- **Flatten (safe — the operator’s own structured field):** identifiers, **claim** rows, session phases, port numbers, **commitment** status (a *commitment* being a violable obligation bound to a named actor, watched by a breach monitor), enumerated states. Administrative legibility is cheap and lossless here.
- **Preserve verbatim, never paraphrase (the agents’ *mêtis* / the ground truth):** the diff, the test output, the error message, the reasoning trace, the exact command run. Paraphrasing these is the over-flattening.

The slogan: *summarize the STATE, link to the WORK.*

Pitfall. “Zoom to the artifact” does not fully preserve *mêtis*, and a political theorist will say so (§3.9). *Mêtis* in Scott is not “the detailed data underneath the summary” — it is practitioner knowledge that *resists codification entirely*: the operator’s gut sense that “this agent is about to do something dumb,” the local knowledge of which corners of the codebase are cursed. A read-path to the diff is more legibility, not *mêtis*. The honest admission — which couples Scott to Bainbridge (§3.5) — is that *some of the most valuable operator knowledge is exactly what the system cannot make legible, and the more the supervising automation takes over, the faster that mêtis atrophies*. The skill-retention tax (§3.5.4) is the design’s only defense, and it is frankly the hardest part of the design to engineer.

3.3.2 The zoom target must be verifiable, not self-reported

There is a subtle trap inside “zoom to the real thing.” If the operator zooms to the *agent’s chain-of-thought* and treats that as ground truth, the map has merely moved down one level — because the reasoning trace may be fiction. **Turpin et al. 2023**, *Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting* (NeurIPS) [158] — *LLM-generated reasoning traces can be systematically unfaithful: plausible, coherent, and not actually what drove the model’s output (a model swayed by reordering of options will rationalize a post-hoc reason and never mention the bias)*. Therefore:

Property 3.3.1 (Verifiable-zoom). The canonical zoom target is an artifact the operator can check *independently of the agent* — the diff on disk, the test result, the committed file. The agent’s self-narration is a *secondary* lens, always labeled as such. Two LLMs agreeing (an agent acting and an LLM summarizing it) is not two checks; it may be one error reported twice.

This is why the source of truth must be the durable substrate — the coordination database and the version-control working tree — and not a model’s summary of them. The digest is computed *over* artifacts the operator owns. It is also why an *adversarial* review organ (a reviewer with no stake) is structurally necessary, not optional: a swarm that reviews its own work is two LLMs agreeing, which by Prop. 3.3.1 is one error twice. We return to adversarial review as the authority’s quality organ in §3.4.

3.3.3 The four design questions (a decision procedure)

For any read-surface — Attention Queue, briefing, resurrection digest, TUI pane — ask, in order (Figure 3.4):

1. **Will the operator act irreversibly on this?** If yes (merge/deploy/delete), the artifact must be one keystroke away and a zoom *forced*. Never let an irreversible act rest on a digest alone (§3.5).
2. **What is administrative vs. *mêtis*?** Flatten the former; link the latter verbatim (§3.3.1).
3. **Who authored the summary, and can it lie?** A deterministic projection of structured state is trustworthy; an LLM summarizing another LLM is not — point its zoom at the artifact (§3.3.2).
4. **Is the authority itself legible here?** Every coordinator action carries a named reason (Prop. 3.2.1).

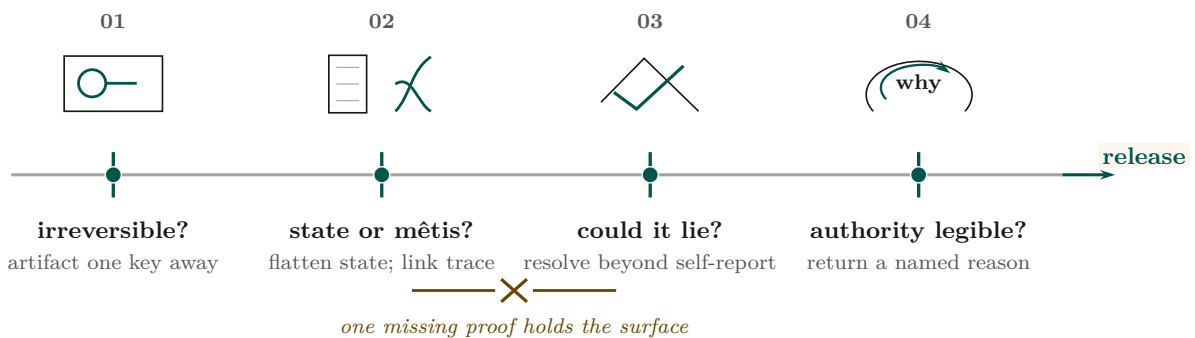


Figure 3.4: A read-surface earns release through four independent proofs, in order: irreversible action exposes its artifact; administrative state remains separate from verbatim *mêtis*; a claim resolves beyond the author’s self-report; and every coordinator act returns a named reason. The rail is conjunctive: one missing proof breaks release.

Protocol 3.3.1. Render a digest line under the one law

Given a candidate summary line σ over a unit of swarm work:

1. **Project state.** Compute σ as a deterministic projection of structured state (claim rows, phases, enums). Never let a language model paraphrase the line itself.
2. **Attach zoom.** For every claim in σ , attach ζ : a link to an operator-verifiable artifact (the diff, the test log verbatim, the committed file). If any claim has no such artifact, the line is a Potemkin digest — refuse to ship it.
3. **Forced-zoom flag.** If the operator may act irreversibly on σ (merge, deploy, delete), set $\sigma.\text{force} = \text{true}$: the artifact is one keystroke away and the act is gated on a zoom (§3.5 sets the threshold).
4. **Reason on authority.** If σ records a coordinator act (a denial, an escalation), attach its named reason. An act with no reason is illegible authority; refuse to ship it.

Scene. Mara, 3:32 p.m. She opens the digest for the five-PR pile. Each row reads “API-2 · charge handler · merged · 1 critique round” — state only, six words. The “fix tests” PR’s row is flagged force-zoom, because merging it is irreversible; she presses one key and the 600-line diff and the verbatim test log open side by side. The diff deletes a migration test. The digest did not paraphrase that away — it linked her straight to it, and the lie is caught in ten seconds instead of slipping through. This is the one law doing its job.

Exercises**Check your understanding.**

- (3.1) Classify each as *flatten* or *preserve verbatim*: a port number; the stderr of a failed test; “the auth refactor felt risky so I split it”; a `commitment` status enum; a 600-line diff.
 (3.2) Why is “two LLMs agreeing” not two checks (Prop. 3.3.1)?

Trace the mechanism.

- (3.3) Take a read-surface you use (a CI dashboard, a PR summary) and run the four questions of §3.3.3 against it. Where does it fail Def. 3.3.1?

Open problem.

- (3.4)*** *Adversarial legibility.* An agent that *knows* it is being made legible can optimize its narration to look good while the diff lies. Verifiable-zoom is a partial answer. Can L2 alone — without L1’s jail and L3’s reputation — detect a deterministic narration↔artifact divergence before reputation exists? Sketch the consistency check and the class of divergence it cannot catch.

3.4 The Leviathan rules, it does not just watch (the authority half)

The title is *Legibility & Authority*. A read-only legibility layer is a glorified dashboard; the whole point of Hobbes is that the sovereign does not just *see* — it *rules*. This section specifies the authority half: the mechanisms by which the consented daemon actually decides, blocks, lands, and gates. Figure 3.5 maps the organs; the engineering status of each is in Appendix 3.A.

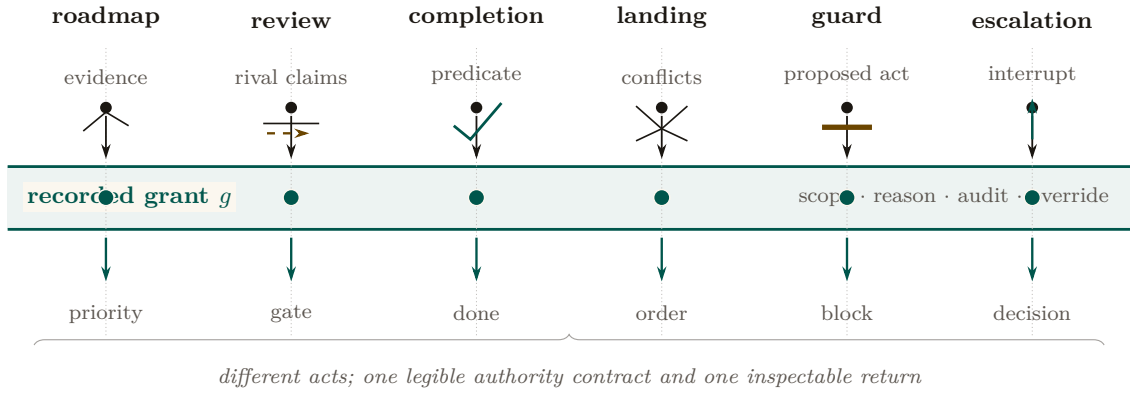


Figure 3.5: Six authority mechanisms as distinct instruments crossing one consent boundary. Their inputs and outcomes differ—evidence becomes priority, rival claims become a gate, a predicate becomes verified completion, conflicts become an order, a proposed act may be blocked, and an interrupt reaches the operator. What they share is the recorded grant g : scope, named reason, audit identity, override, and an addressable return artifact.

3.4.1 Roadmap-as-truth: the artifact the authority is accountable to

A coordinator that prevents collisions answers “what is the swarm doing?” It does not answer “is the swarm doing the *right* thing?” That question needs a single legible artifact the authority is measured against: a **roadmap** — a phase-keyed plan, maintained as governing truth, against which the swarm’s actual output can be compared. The authority’s legitimacy is then measured by *whether the roadmap stays true*: every unit of work keys to a plan item, and drift between plan and reality is itself a legible, surfaced anomaly.

3.4.2 Adversarial review: the quality organ

By Prop. 3.3.1, a swarm reviewing its own work is two language models agreeing — one error twice. The quality organ must therefore be *adversarial and conflict-free*: a reviewer with no stake in the work, running a bounded critique–refine protocol. This is the wedge’s local analog of the *neutral evaluators* — the independent rating agencies — that *From Spawn to Person* and *The Harbor Economy* federate across operators: the same conflict-free-judge discipline, scoped first to one operator and then opened to a market.

3.4.3 Sole-responsibility roles and the specialization boundary

The single-writer discipline (§3.1.2) generalizes naturally from data files to system functions. When an invariant is critical, the authority assigns a **sole-responsibility role** (an avatar holding exclusive write capability over an invariant):

- *Roadmap Owner*: The single avatar authorized to commit mutations to the phase plan.
- *Test Suite Curator*: The sole agent authorized to land modifications to regression suites.
- *Release Avatar*: The sole gatekeeper authorized to execute deployment tags.

Assigning a role that way is a trade, and the trade has a number. Think of a bank: one expert teller (the sole specialist) working a single line in strict order, versus several ordinary tellers (the pooled generalists) sharing one line. The expert is faster per request but has no colleagues to absorb a burst; the pool is slower per request but almost never leaves the line standing idle behind a long job. $M/M/1/M/M/c$ — the standard queueing-theory shorthand (**Kendall 1953** [66]) for a single-server versus c -server queue with Poisson arrivals, exponential service times, and one waiting line served first-come-first-served — is the textbook way to price the trade.

Fix the **queueing-specialization setup**: requests for an invariant-critical function F arrive as a Poisson stream at rate λ_F ; a sole specialist serves them as an $M/M/1$ queue at rate μ_{spec} , the pooled alternative as an $M/M/c$ queue of generalists at rate μ_{pool} each with utilization $\rho = \lambda_F/(c\mu_{\text{pool}}) < 1$ and first-come-first-served exponential service; and the objective is *mean cost* — mean response time, plus an accountability value A per unit time of demand for single-writer attribution, against a waiting cost w per request-hour. Let $C(c, \rho)$ be the **Erlang-C** delay probability (**Erlang 1917** [2] — *the probability that an arriving request finds all c servers busy and must wait, the quantity that makes a pool a pool*).

Theorem 3.4.1 (Queueing Specialization Boundary). *Under the queueing-specialization setup, sole ownership weakly dominates the pool if and only if the skill premium clears*

$$\frac{\mu_{\text{spec}}}{\mu_{\text{pool}}} \geq g_A(\rho, c) = c\rho + \frac{1}{1 + \frac{C(c, \rho)}{c(1 - \rho)} + \frac{A\mu_{\text{pool}}}{w\lambda_F}}. \quad (3.1)$$

verified — *corrected*. An earlier draft of this paper proposed the threshold $\tilde{g} = 1 + (c - 1)\lambda_F/(c\mu_{\text{pool}} - \lambda_F) = 1 + (c - 1)\rho/(1 - \rho)$, on the plausible-looking grounds that it is what a naive $M/M/1$ -versus- $M/M/c$ response-time comparison appears to give. A sweep against simulation *falsified* it in both directions before the belief hardened, and Eq. 3.1 is the replacement — the companion formal paper’s Theorem 2 [85], independently re-derived and checked numerically over two thousand (ρ, c) points. We leave the wrong form on the page rather than quietly swapping it, because a paper that argues for falsification-first oversight should be legible about its own retractions: the two thresholds cross exactly once, so the superseded form was not merely conservative — it was wrong in *both* directions, and Figure 3.6 shades the two error regions.

At $A = 0$ — pure response time, accountability priced at nothing — the boundary reduces to

$$g(\rho, c) = c\rho + \frac{c(1 - \rho)}{c(1 - \rho) + C(c, \rho)}. \quad (3.2)$$

Three properties fix its shape: $g(\rho, 1) = 1$ (a specialist against a single generalist needs no premium at all), $g \rightarrow c$ as $\rho \rightarrow 1$ (the boundary *saturates* at the capacity ratio; it never diverges), and for two generalists it is the closed form $g(\rho, 2) = 1 + 2\rho - \rho^2$. Below the boundary, sole ownership buys strict single-writer accountability at an explicit latency price that must be budgeted rather than obscured; Eq. 3.1 is where that price is written down.

Numbers by hand. The result is two lines of algebra on textbook formulas, which is the right way to first believe it. Sole ownership pays iff $w\lambda_F(W_{\text{solo}} - W_{\text{pool}}) \leq A$, i.e. $1/(\mu_{\text{spec}} - \lambda_F) \leq W_{\text{pool}} + A/(w\lambda_F)$; invert, divide by μ_{pool} , and substitute the two textbook facts $\lambda_F/\mu_{\text{pool}} = c\rho$ and $\mu_{\text{pool}}W_{\text{pool}} = 1 + C(c, \rho)/(c(1 - \rho))$, and Eq. 3.1 falls out. For $c = 2$ the Erlang-C value is $C(2, \rho) = 2\rho^2/(1 + \rho)$, so the second term of Eq. 3.2 is $(1 - \rho^2)/((1 - \rho^2) + \rho^2) = 1 - \rho^2$ and $g(\rho, 2) = 1 + 2\rho - \rho^2$, checkable in three lines with no queueing theory at all.

Now the case the roadmap owner actually faces. Roadmap mutations arrive at $\lambda_F = 2/\text{hr}$; the sole owner clears them at $\mu_{\text{spec}} = 3/\text{hr}$; the alternative is three generalists at $\mu_{\text{pool}} = 1.2/\text{hr}$ each. Then $\rho = 2/3.6 = 0.556$ and the skill premium is $r = 3/1.2 = 2.5$. Erlang-C at $(c, \rho) = (3, 0.556)$ is 0.300, so $g = 3(0.556) + 3(0.444)/(3(0.444) + 0.300) = 1.667 + 0.816 = 2.483$. The premium 2.5 clears it — barely — so the sole owner wins: $W_{\text{solo}} = 1/(3 - 2) = 1.000$ hr against $W_{\text{pool}} = (1 + 0.300/1.333)/1.2 = 1.021$ hr, a margin of about 75 seconds per request. The superseded threshold would have returned $\tilde{g} = 1 + 2(2)/(3.6 - 2) = 3.5$ and sent the work to the pool. That is the right-hand error lens of Figure 3.6, met in the wild on the first realistic instance anyone tries.

Now you try: at the same three-agent pool, would a specialist at $\mu_{\text{spec}} = 2.8/\text{hr}$ still be worth it on response time alone? (The premium is $2.8/1.2 = 2.33 < 2.483$ — no, and $W_{\text{solo}} = 1.25$ hr confirms it. The boundary is genuinely tight here; note that a positive accountability value A lowers g_A below g and can buy the sole owner back.)

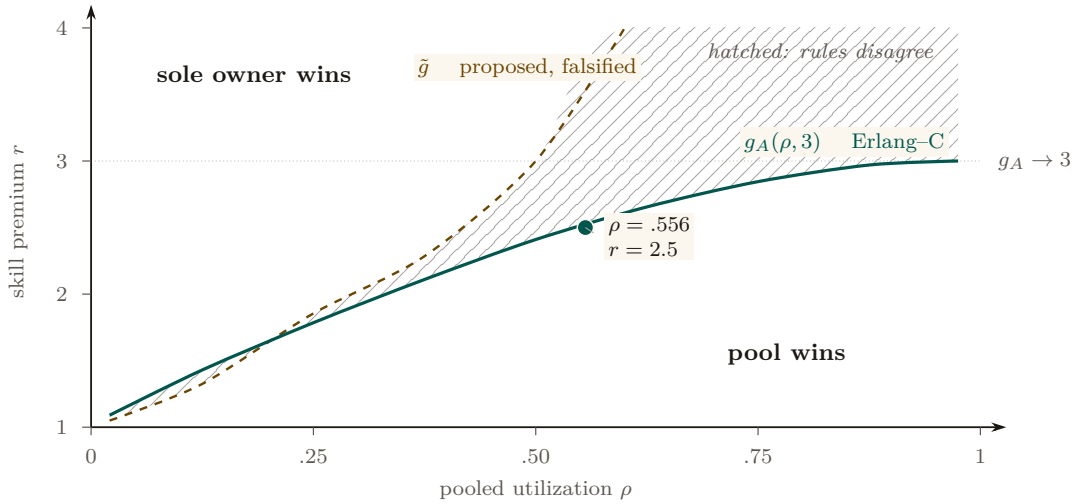


Figure 3.6: The specialization boundary at a three-agent pool. The correct Erlang-C boundary $g_A(\rho, 3)$ saturates at the pool capacity ratio; the superseded threshold $\tilde{g}$ diverges. The hatched region is not uncertainty: it is the decision set on which the rules disagree, first certifying a specialist who loses and then rejecting one who wins. The worked instance ($\rho = .556$, $r = 2.5$) narrowly clears the correct boundary.

Pitfall. Eq. 3.1 prices *mean* response time under first-come-first-served exponential service. It does not price the tail, and it does not price the corner the whole sole-responsibility discipline exists for: the sole specialist *failing*, which is an $M/M/1$ queue with breakdowns and a succession protocol, not the queue modeled here. Nor does it price the thing an operator actually values in a roadmap owner — consistency of judgment across requests — which does not appear in a response-time objective at all. The boundary is a floor on the argument, not the whole argument.

3.4.4 Completionist obligation: completion as a verifier, not a vibe

The operator’s standing complaint is the agent that *declares completion while the work is hollow* — “fixed the test by deleting it.” The completionist obligation makes this structurally impossible to do silently.

Definition 3.4.1 (Completionist completion). A **completion claim** is a commitment whose satisfaction predicate is a *verifier* derived from the task’s acceptance criteria. The authority refuses to accept the claim against an unmet predicate. An *unverifiable* predicate must **fail loudly** — surface “I cannot verify this completion” — and never silently pass. This is the authority-side analog of discovery’s *refuse-to-route* (§3.7.3): “I don’t know” must beat a confident wrong answer.

The verifier is grounded in design-by-contract (**Meyer 1992**, *Applying Design by Contract* [35] — *pre/postconditions as machine-checkable obligations; the completion predicate is a postcondition*). This is the single highest-leverage new mechanism in the wedge: it turns “done” from a declaration into a checkable obligation, which is the only structural defense against the transparently-hollow deliverable.

Pitfall. Verification has a *cost gradient*, not a binary. A test going green is cheap to verify; “the architecture is right” is expensive, and reading a 600-line diff to confirm an acceptance criterion is itself attention-seconds that may exceed the agent’s narration cost by an order of magnitude (the *verification asymmetry*). Def. 3.4.1’s fail-loud is the honest floor for the *unverifiable*; §3.5 supplies the cost model for the *expensive-but-possible* middle.

3.4.5 Thoughtful landing and the regimentation/enforcement split

The authority decides *what lands and in what order*, conflict-aware. And it has teeth: the footgun guards. Here we must be precise about what a guard can and cannot do, because the difference is the difference between a safety claim that holds and one that is wishful.

Property 3.4.1 (Prevention vs. detection — the reserved-phrase rule). A coordination state is **physically unreachable** — prevented — *only* when a constraint forbids it *inline*, before the act commits: a database uniqueness constraint that makes a double-claim impossible, or a fail-closed startup gate that refuses to run against the wrong target. A monitor that *subscribes to the log of actions already taken* is something weaker and must be named formally: **detect-and-compensate**. It observes a violation *after* the fact and then halts or repairs within a bounded window. By the reference-monitor result below, such a downstream monitor enforces no safety property at all. The phrase “physically unreachable” is therefore reserved for the constraint-enforced set — nowhere else.

Fred B. Schneider (*Enforceable Security Policies*, 2000 [91] — *an execution monitor that observes a system can enforce exactly the safety properties; a monitor placed downstream of the act it would forbid can detect a violation but cannot prevent it*) is decisive: a monitor downstream of commit enforces no safety property, only detection plus compensation within a bounded window. A post-commit anomaly detector — however useful — is therefore detect-and-compensate, not prevention. Honesty about this distinction is essential: a system that advertises detection as prevention has lied about its own safety envelope, which is the very failure (the confident-wrong report) this paper is built to eliminate.

3.4.6 Human-in-the-loop escalation as a typed, prioritized interrupt

The one thing that needs a human is an *escalation* (a distress signal) routed to the operator on a reserved, highest-priority lane. The mechanism wants the critical signal to win the operator’s attention — but *when* the daemon stops and asks versus proceeds is a control problem (§3.5), and the *escalation predicate* is not an afterthought: it is a rationalized alarm philosophy (**ANSI/ISA-18.2**, *Management of Alarm Systems* [19] — *the process-control standard for alarm rationalization: every alarm has a defined priority, set-point, and operator response, with explicit flood-suppression*). An uncontrolled-false-alarm distress lane self-destructs into habituated uselessness within days, exactly as clinical alarms do (**Cvach 2012**, *Monitor Alarm Fatigue* [145] — *85–99% of clinical alarms are non-actionable; staff learn to ignore channels that cry wolf*). We make escalation a *costly signal* (§3.7.3): an escalation the operator dismisses debits the agent’s standing, so crying wolf has a price.

verified — the threshold equilibrium exists and is unique: with payoff $\Pi_\delta(u) = b(u) - \delta w(1 - V(u))$ strictly increasing in u , agents separate at $u^*(\delta)$ solving $b(u^*) = \delta w(1 - V(u^*))$, escalating iff $u \geq u^*$ — the Spence mold, where the dismissal debit is money only a genuine signal is worth risking. The free parameter δ is fit from measured validation rates (`instrumentation.md`), not vibes: the feasible debit set is an interval $[\delta_{\min}, \delta_{\max}]$ trading alarm load against miss loss, and it can be *empty* — a tight attention budget with high miss costs admits no good debit, in which case the honest fix is capacity or evidence quality, not tuning. Monotone V is required throughout; a non-monotone validator breaks the separating threshold (item R14 of the research ledger, `b7_escalation_band.py`).

Exercises

Check your understanding.

(4.1) Why is adversarial review (§3.4.2) a corollary of Prop. 3.3.1 rather than an independent design choice? (4.2) State Prop. 3.4.1 in your own words and give one example each of a *regimented* and a *detected-and-compensated* rule.

Trace the mechanism.

(4.3) An agent claims completion on a task whose acceptance criterion is “the migration applies cleanly.” Trace Def. 3.4.1: what is the verifier, what gates, and what happens if the criterion were instead “the API feels ergonomic”?

Open problems.

(4.4)★ *Completionist verification of the unverifiable.* Some acceptance criteria are not mechanically checkable. Characterize the honest fallback ladder (forced human-in-the-loop gate → neutral judge with declared low confidence → fail-loud). When is “I cannot verify this” the *correct* answer rather than a cop-out? (4.5)★ *The succession corner of Thm. 3.4.1.* Eq. 3.1 prices a specialist who never fails. Extend it to an $M/M/1$ queue *with breakdowns* and a succession protocol, and find the repair rate at which sole ownership stops paying regardless of skill premium — then check whether that rate is above or below the one an operator can actually promise.

3.5 The operator as a modeled entity

It is tempting to model only the *swarm* as the thing made legible, and to leave the *operator* unmodeled. But the legibility layer exists *for* the human operator, and the binding constraint at the top of the system is the operator’s finite attention and fallible trust calibration. A complete design therefore treats operator attention as a resource with its own economics, parallel to the token economics of the swarm (§3.8).

Key idea. In a world where token cost trends toward zero, the human supervisor is the only cost that stays fixed — so the entire economic logic of the stack eventually routes through one objective: minimize *operator-attention per correct decision*. The operator is the system’s reliability bottleneck, a single server with a non-elastic service rate. The 51st agent does not get a 51st operator.

The cognitive ceiling is real: **Miller 1956**, *The Magical Number Seven* [98] — *working memory holds roughly 7 ± 2 chunks*, which makes read-poverty (§3.6) a human constraint, not merely an engineering one — and **Wickens’ Multiple Resource Theory** ([51] — *attention is not a single scalar pool but a vector over stage $\times$ modality $\times$ code $\times$ visual-channel; you cannot trade audio-channel attention for visual-channel attention 1:1*). The vectored model matters: the right objective is not “minimize total seconds” but *minimize peak load on the bottleneck channel*, and the ambient-vs-dense split (§3.7.5) is the cross-modal load-balancing mechanism that does it.

3.5.1 The attention objective, spined on Signal Detection Theory

A naive objective — “attention-seconds-per-correct-decision” — is Goodhart-bait: it rewards lowering the decision criterion to wave things through, because zooming costs seconds and most diffs are fine. The correct frame is **Signal Detection Theory** (**Green & Swets 1966** [69] — *an operator discriminating signal from noise trades sensitivity d' against a decision criterion β ; performance is a joint function of both and of the asymmetric costs of the two error types*). The operator is trading *misses* (rubber-stamping a destructive-on-stale-premise merge) against *false alarms* (zooming on a safe diff), and the cost of a miss is wildly asymmetric to the cost of a false alarm.

Definition 3.5.1 (The SDT-spined attention objective). Let an operator decision over an action have sensitivity d' and criterion β . With miss probability $P_{\text{miss}}(\beta)$, false-alarm probability $P_{\text{fa}}(\beta)$, miss cost C_{miss} (large: irreversible damage), false-alarm cost C_{fa} (small: wasted attention-seconds), base rate π of genuinely dangerous actions, and per-decision attention cost a , the operator objective is

$$\min_{\beta} \left[\pi C_{\text{miss}} P_{\text{miss}}(\beta) + (1 - \pi) C_{\text{fa}} P_{\text{fa}}(\beta) + a \mathbb{E}[\text{zooms}(\beta)] \right]. \quad (3.3)$$

The forced-zoom policy $p(r, s, v)$ is calibrated against this loss after specifying the score distributions that determine P_{miss} and P_{fa} . Equation 3.3 is a decision objective, not by itself a fitted detector.

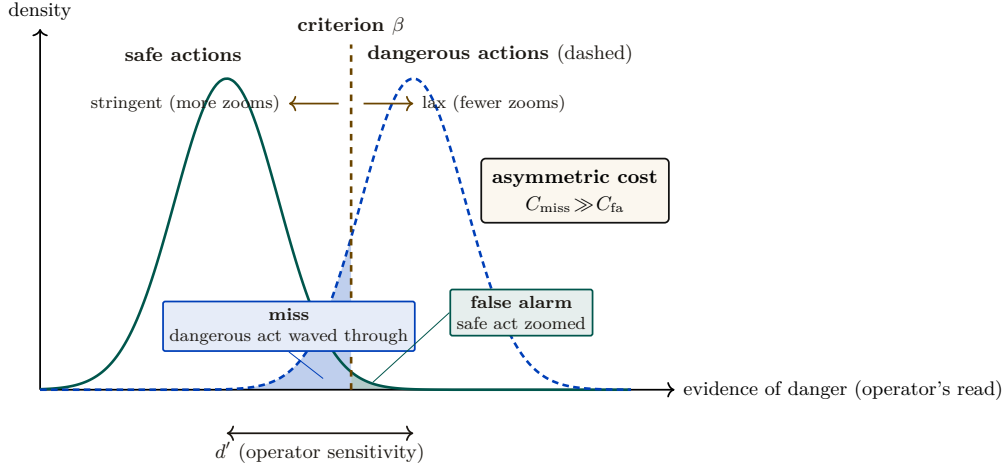


Figure 3.7: The operator's forced-zoom decision as Signal Detection Theory (Def. 3.5.1, Eq. 3.3). Most actions are safe (left curve); a few are dangerous (right). Zooming triggers when evidence of danger exceeds criterion β . Moving β left is more stringent: it causes more zooms, fewer misses, and more false alarms. Moving β right is more lax: it minimizes attention-seconds but increases misses. Because $C_{\text{miss}} \gg C_{\text{fa}}$, the optimal criterion sits well to the left of the throughput optimum: *the miss-cost term is what forces the zoom*. Sensitivity d' (curve separation) is improved by better legibility; β is the policy knob $p(r, s, v)$.

Figure 3.7 lays the same decision out visually: the operator's criterion β trading misses against false alarms. The miss-cost term prevents the zero-review solution whenever misses have positive probability and cost. Monotonic response to stakes, irreversibility, and reputation is a policy constraint to test during calibration; it does not follow from the display without a model linking (r, s, v) to the score distributions. Before a reputation layer exists, r is a flat prior.

Numbers by hand. Equation 3.3 is the chapter's governing artifact for the human-factors reader, so it should be checkable on a napkin. Collapse it to its crudest useful form — zoom *every* item in an action class, or zoom *none* — and the two costs are πC_{miss} against $(1 - \pi)C_{\text{fa}} + a$. Table 3.3 runs three real action classes through it at $C_{\text{fa}} = 1$ and $a = 0.2$ attention-seconds.

Action class	π	C_{miss}	Never zoom	Always zoom	Verdict
tests/-only formatting diff	0.001	5	0.005	1.199	auto-land
Refactor touching a public interface	0.03	40	1.20	1.17	zoom, barely
Migration dropping a foreign key	0.2	500	100.0	1.00	forced zoom

Table 3.3: Equation 3.3 on three action classes at $C_{\text{fa}} = 1$, $a = 0.2$. The first row is Mara’s auto-land grant (§3.2.2) falling out of the arithmetic rather than being granted by taste. The third is decided by the miss term alone — $0.2 \times 500 = 100$ swamps a hundred false-alarm zooms — so the objective forces the zoom before irreversibility is separately priced. The middle row is the interesting one: at 1.20 against 1.17 the classes are within noise of each other, which is exactly the regime where a ranked queue rather than a fixed policy earns its keep.

Now you try: at what base rate does a $C_{\text{miss}}/C_{\text{fa}} = 50$ action class cross from auto-land to forced zoom? (Set $50\pi = (1 - \pi) + 0.2$, giving $\pi \approx 0.024$ — roughly one dangerous action in forty.) The honest boundary: this collapse assumes the operator either zooms an entire class or none of it, which is the policy Eq. 3.3 would produce only if P_{miss} and P_{fa} were degenerate. A fitted detector with a real criterion β does better than both columns; the table is a floor on the decision, not the decision.

3.5.2 Trust calibration is multi-dimensional, and the vigilance decrement is real

Showing the operator a beautiful digest does not make them appropriately reliant on it (Lee & See 2004, *Trust in Automation* [120] — *trust has at least three bases (performance, process, purpose) and calibration depends on resolution (can the operator tell where automation is reliable from where it is not?) and specificity (is trust differentiated across functions and time?)*). The implication is sharp: a *single* “approve-without- zoom rate” and a *single* global forced-zoom knob is mis-calibration by aggregation. An operator who correctly trusts the swarm on refactors and correctly distrusts it on migrations shows a middling *global* catch-rate, and a global knob mis-serves both. Trust calibration must be *function-specific and history-dependent* — per (action-class, agent), not one scalar.

And the canary-defect catch-rate mechanism — inject known-bad diffs at a sampled rate, raise friction when catch-rate drops — is a *vigilance task*, monitoring for rare signals, and the vigilance literature predicts it will degrade for reasons unrelated to swarm danger (Warm, Parasuraman & Matthews 2008, *Vigilance Requires Hard Mental Work and Is Stressful* [119] — *detection of rare signals degrades sharply within 20–35 minutes, worse at low signal probability*; the classic result is Mackworth 1948 [165]). The honest, citable tension: defeating the vigilance decrement *costs attention-seconds* (canary rate must be high enough to beat the decrement, which fights Eq. 3.3). Catch-rate must therefore be *time-on-task-corrected*, and the right countermeasure is to force breaks, not just rotate zooms.

Pitfall. Campbell’s Law eats the governor. The moment “operator catch-rate” becomes a metered invariant that raises friction when it drops, the operator is incentivized to game it — memorize canary patterns, develop a “looks-like-a-canary” heuristic orthogonal to real defect-finding (Campbell 1979 [73] — *the more a quantitative indicator is used for decision-making, the more it distorts the process it monitors*). The accountability instrument corrupts the accountability it measures. We do not claim the canary governor cleanly closes the loop; it is net-positive only if canaries are drawn from a refreshed, costly-to-fake distribution and the catch-rate is one signal among several, never the sole gate.

3.5.3 Situation awareness needs projection, not just perception

Digest-with-zoom is overwhelmingly a perceive-and-comprehend instrument. But **Endsley 1995** (*Toward a Theory of Situation Awareness* [154] — SA has three levels: perception, comprehension, and **projection** — what the system will do next) puts projection at the top, and **Endsley & Kiris 1995** (*the out-of-the-loop performance problem* [153] — operators removed from active engagement lose the dynamic SA needed to take over when automation fails; intermediate automation can yield better SA than full automation because the operator stays cognitively engaged in projection) shows the deeper danger: a digest that summarizes only the *present* is an SA-destroying instrument no matter how good its zoom. The authority half therefore needs a first-class *projection* mode — “what is the swarm about to do, and what will the state be in ten minutes?” — validated by a freeze-probe (pause the swarm, ask the operator to predict the next three merges, score accuracy). The forensic time-axis (§3.11.1) is the *backward-looking* complement; projection is the missing *forward* one.

3.5.4 The Bainbridge–Scott convergence and the abdication gradient

Lisanne Bainbridge 1983 (*Ironies of Automation* [138] — *the more reliable an automated system becomes, the less practiced and situationally aware its human supervisor becomes, so the human is least equipped to intervene at exactly the moment intervention is needed*) and Scott are the same warning at two scales: the digest erases the swarm’s *mêtis*, and the digest erases the operator’s *mêtis*. As the supervising automation gains autonomy — executing longer plans end-to-end — the operator’s role slides *doer* → *approver* → *rubber-stamp* → *absent* — the *abdication gradient* (Figure 3.8). The design’s primary active defense is the **Inception Canary** (a *consented, blinded* vigilance probe), moving beyond static “skill-retention taxes” (such as arbitrary manual busywork, which causes attention residue and degrades focus [202]).

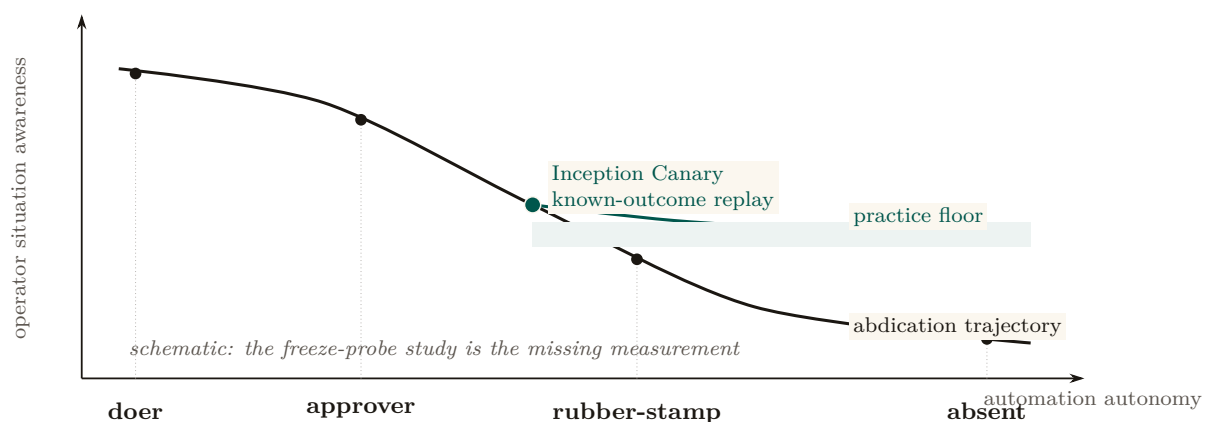


Figure 3.8: The abdication gradient, drawn explicitly as a hypothesis rather than measured data. As automation owns longer plans, the operator can slide from *doer* to *approver*, *rubber-stamp*, and *absence* while situation awareness falls. The **Inception Canary** does not reverse that trend; it proposes a practice floor by replaying consented, blinded, known-outcome defects through the live review surface. A freeze-probe study must replace these schematic trajectories with observations.

Protocol 3.5.1. The Inception Canary protocol

To continuously measure and maintain operator situation awareness without synthetic busywork. Operators enroll knowingly: the program’s existence, rate ρ_{canary} , and stakes are disclosed at onboarding; only the *which* and *when* are blinded:

1. **Blinded holdout replay.** An out-of-band Compaction Sidecar (§3.8.2) draws from a curated holdout of *historical, already-fixed* defects (an off-by-one boundary condition, an unindexed query, a removed regression test), re-materializes one inside a sandboxed replay of a past work item at rate ρ_{canary} , and routes it through the review surface indistinguishably from live traffic. No anomaly is ever injected into a live pull request or pending action: probes are replays with known ground truth, so a missed canary costs vigilance points, never production correctness.
2. **Attention queue surfacing.** The item is placed in the Attention Queue with standard high-regret scoring.
3. **Dual adjudication:**
 - *Rubber-stamp failure:* If the operator approves without expanding the forced-zoom diff, the daemon halts execution pre-commit, logs an INCEPTION CANARY FAILED event, and tightens the operator’s autonomous policy lease (lowering the threshold at which future agent actions require manual confirmation).
 - *Vigilant detection:* If the operator zooms, identifies the flaw, and rejects it, the daemon logs CANARY DEFEATED and increases the *Operator Trust Score*, unlocking extended autonomous execution grants (e.g., unattended batch merges).

This transforms supervisory vigilance from a punitive chore into a closed verification loop that rigorously measures human situation awareness — with the operator’s informed consent, on ground truth the system already knows, and with provenance every probe carries so post-hoc audit can distinguish canaries from live items in one lookup.

Exercises

Check your understanding.

(5.1) In Eq. 3.3, what happens to the optimal forced-zoom rate $p(r, s, v)$ as $C_{\text{miss}}/C_{\text{fa}} \rightarrow \infty$? As agent reputation $r \rightarrow 1$? (5.2) Give a concrete scenario where a *global* trust knob (§3.5.2) mis-serves an operator who is, in fact, perfectly calibrated.

Trace the mechanism.

(5.3) Walk the canary-defect loop and mark every point where Campbell’s Law could distort it. Propose one change that reduces the distortion without removing the governor.

Open problems.

(5.4)*** Formalize the operator-attention objective (Eq. 3.3) against swarm token-COGS (§3.8): is there a digest that is Pareto-optimal in both operator-seconds and token cost, or does a cheaper digest always cost more operator seconds? (5.5)** Does a skill-retention tax *measurably* preserve SA, or merely annoy until disabled? Design the freeze-probe study (§3.5.3) that would tell.

3.6 Read-poverty: the binding constraint at scale

A single operator can hold two coding agents in their head. They cannot hold two hundred. Spin up two agents and coordination is free: you read both their notes, eyeball both their claims, route a question by recognition. Spin up fifty and an agent that wants to ask “who owns the OAuth refactor?” has no move except to dump the session roster and scroll. It guesses, cold-DMs the wrong agent, and re-derives from scratch a design decision another agent settled an hour ago and wrote into a note nobody will ever read again.

The swarm has not run out of information. It has drowned in it. Every agent emits sessions,

claims, notes, diffs, skill tags, and — eventually — reputations, far faster than any single participant can read them.

Definition 3.6.1 (Read-poverty). **Read-poverty** is a regime in which legible state accumulates faster than it can be consulted, so participants fall back to $O(n)$ eyeball search over the population, or, worse, act blind.

Read-poverty, not write-contention, is the binding constraint on swarm scale. The write side has known fixes — claims, locks, merge queues, and anomaly detection — which mature coordination systems already provide. The read side has been left implicit, and it is where the next order of magnitude is won or lost.

The cure is the move every database made when table scans stopped scaling: build an **index**. A *directory* is to agents what an index is to rows — it pays a write-time cost (maintaining the structure) to buy read-time speed (answer a query in $O(\text{query})$ instead of scanning $O(n)$ participants). Read-poverty is also a human constraint (Miller’s 7 ± 2 , §3.5) and a legibility argument in Scott’s sense: a directory is a state-like act of legibility over the swarm, and a directory entry over-flattened into a verdict that hides what mattered (“this agent owns auth,” omitting that its last three auth pull requests were reverted) is the failure. Every directory entry must therefore be a lens that zooms (Def. 3.3.1).

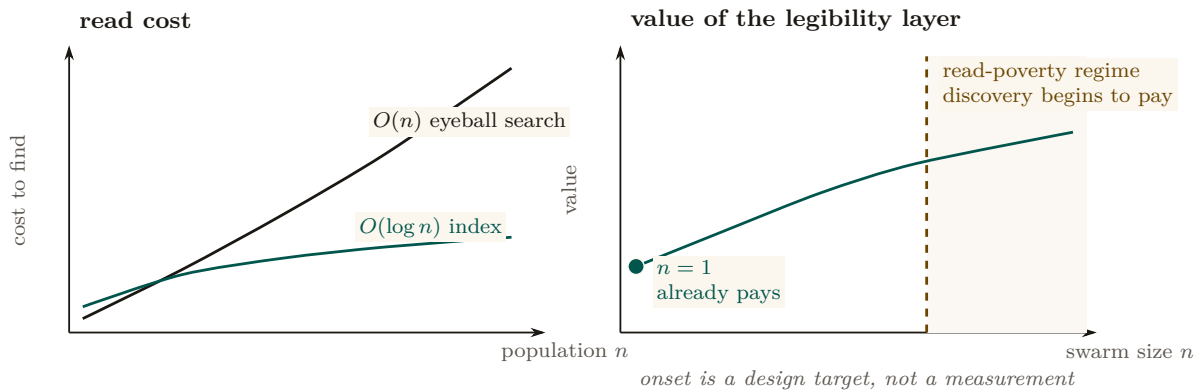


Figure 3.9: Read-poverty has two scales. Left, unindexed eyeball search grows with population while a directory keeps lookup near logarithmic. Right, the legibility layer has value at $n = 1$ through briefing, attestation, footgun guards, and honest completion; discovery adds value only after the corpus becomes too large to scan. The shaded onset is a design regime, not an empirical estimate of a universal swarm-size threshold.

3.6.1 The value curve across n (the wedge must work at $n=1$)

Read-poverty is a *scale* disease, but the wedge ships to a solo developer who often runs one or two agents. The legibility layer must be valuable at $n=1$ — the briefing, the attestation, the footgun guard, the honest completion gate all pay off immediately — and *gracefully* grow into the read-poverty regime (Figure 3.9, right panel). Discovery is free at $n=2$, the bottleneck at $n=50$, the whole system at $n=500$. This value curve *is* the three-layer cure of the next section: existence (the registry), relevance (ranked routing), and trust (reputation, deferred to the accompanying chapters). The layers are strictly ordered: relevance over a stale registry returns confident garbage; reputation over an unranked registry has nothing to attach a score to.

3.6.2 A legibility lower bound

Section 3.3 forbade the Potemkin digest by fiat. We can also state the prohibition as a theorem: there is an information floor below which a digest *cannot* be a faithful zoomable index, no matter how it is written.

Theorem 3.6.1 (Legibility lower bound with a review budget). *Let N artifacts contain an unknown critical subset S of size k . A digest maps each possible S to a review set $\hat{S}$ with $S \subseteq \hat{S}$ and $|\hat{S}| \leq m$, where $k \leq m \leq N$. Any such zero-miss digest needs at least*

$$\log_2 \binom{N}{k} - \log_2 \binom{m}{k}$$

bits in the worst case. Exact identification ($m = k$) recovers the $\log_2 \binom{N}{k}$ bound; allowing the operator to open everything ($m = N$) correctly reduces the bound to zero.

Proof sketch. There are $\binom{N}{k}$ possible critical subsets. One digest message whose review set has size at most m can safely cover at most $\binom{m}{k}$ of them, namely the k -subsets contained in that review set. Therefore at least $\binom{N}{k} / \binom{m}{k}$ distinct messages are required. Taking base-two logarithms gives the bound. If fewer messages are available, some critical subset is not contained in its message’s review set, producing a false negative — an unopened critical artifact. That is exactly the Potemkin failure of Def. 3.3.1. Hence the stated log-ratio is a lower bound for a zero-miss digest with review budget m . □

verified. This bound is not merely proved here. It is Theorem 1 of the companion formal paper, *The Price of a Summary* [84], in the same N, k, m notation, where it is put through a *pre-registered falsification sweep* — an experiment written down before the result was believed, designed to break it — across sixteen configurations including an adversarial oracle encoder allowed to see the answer. Zero violations. A reader checking this paper’s honesty should start there rather than with the proof sketch above.

Numbers by hand. Take $N = 1000$ artifacts with $k = 3$ critical among them and a review budget of $m = 10$ opens. The floor is $\log_2 \binom{1000}{3} - \log_2 \binom{10}{3} = 27.31 - 6.91 = 20.4$ bits — about two and a half bytes the digest cannot avoid spending, no matter how well written, just to point at the right three files. Widen the budget to $m = 100$ and the second term grows to $\log_2 \binom{100}{3} = 17.3$, cutting the floor to 10.0 bits: buying the operator more opens is exactly buying the digest fewer bits, at a rate the formula prices. *Now you try:* at $N = 60$, $k = 2$, $m = 8$ — the configuration the companion paper’s sweep hammers hardest — what is the floor? ($10.79 - 4.81 = 5.98$ bits; the sweep found no encoder beating it in 16 tries.)

The bound is the formal reason *forgetting must be selective, not lossy-by-volume*: a digest may drop inert detail freely, but it must retain enough to *point at* every critical artifact, which is precisely the “summarize the state, link to the work” rule. It also bounds recursive summarization (§3.8.4): each re-summarization that drops below the floor on the critical set is irreversible information loss, which is why one must compact from the durable artifacts, never from the previous summary.

The other half of the price: zoom is cheap. The floor above prices only the *first* stage — the digest that narrows N artifacts down to a flagged set F . It says nothing about the second stage, which is where Def. 3.3.1 sends the operator, and a reader could reasonably worry that a floor on the digest plus an unpriced zoom is only half an argument. It is not, and the companion formal paper [84] supplies the missing half. Its *zoom theorem* shows that finding the k genuinely critical items inside F flagged ones does not cost F opens: *adaptive halving* — open a group, and if it comes back clean discard the whole group, otherwise split it and recurse, twenty questions over artifacts — needs at most

$$2k \lceil \log_2(F/k) \rceil + 4k$$

group opens in the worst case, against F for flat inspection, with the leading constant shown to be achieved. At $F = 2500$ flags and $k = 10$ criticals that is at most 200 opens rather than 2500: a guaranteed $12.5\times$, measured at $15.3\times$ in the companion paper’s harness. The practical consequence for this chapter’s design is the one that matters: *over-flagging is affordable*. A cheap digest that flags too much costs only *logarithmically* more in the zoom stage, so the operator’s dial — spend bits on the digest, or spend opens on the zoom — has an exchange rate that strongly favors letting the digest be generous and the zoom be adaptive. That is why “summarize the state, link to the work” is not merely honest but cheap.

Exercises

Check your understanding.

(6.1) Why is read-poverty (Def. 3.6.1) the *binding* constraint rather than write-contention? Name the write-side fixes that make the write side not binding. (6.2) State the database analogy precisely: what is the write-time cost a directory pays, and what read-time speed does it buy?

Trace the mechanism.

(6.3) At $n=1$, name three legibility surfaces that already pay off (§3.6.1), and explain why none of them are discovery-ranking. (6.4) In Thm. 3.6.1, what happens to the bit floor as $k \rightarrow 0$ (almost nothing is critical) and as $k \rightarrow N/2$? Relate the answer to why a swarm doing mostly-safe work is cheap to digest and a swarm doing mostly-dangerous work is not.

Open problem.

(6.5)*** Thm. 3.6.1 assumes the critical subset is fixed and known to an oracle. In practice it is itself estimated, and the estimator can be gamed. Does the bound survive an adversary who controls which artifacts *appear* critical? Connect to adversarial legibility (Ex. 3.4).

3.7 Discovery: the directory substrate and the split ranker

A performative cannot be addressed until the swarm knows who exists. The *directory substrate* — the existence layer a message must address — belongs with the discovery story, so we treat it here.

3.7.1 Existence: the yellow pages (a solved shape)

The canonical prior art is forty years old. **FIPA** (*Foundation for Intelligent Physical Agents*) made a **Directory Facilitator** (DF) — the mandatory “yellow pages” agent: others register the services they offer and query to find agents that offer a needed service (FIPA Agent Management Specification, FIPA00023 [90]) — a required component of every conformant platform, alongside the **Agent Management System** (AMS, the “white pages” for *identity* lookup). Two FIPA choices matter: the DF is *push-based* (agents self-advertise), and DFs may *federate* (the seed of cross-operator discovery). The DF has been re-invented almost beat-for-beat for LLM agents (**Ren et al. 2025**, *Agent Name Service* [214] — a *DNS-inspired naming and capability-resolution layer with public-key-backed identity*, which is FIPA’s DF with a 2025 threat model).

The existence layer is the easy part: a coordination database that records the live population, their stated purpose, and their file claims is a white-pages directory, and a vector index that embeds each agent’s declared capabilities and answers nearest-neighbor queries is a yellow-pages directory over skills. Existence is not the gap; the layers above it are.

3.7.2 Relevance: the split ranker (the headline result)

A flat registry answers “does an OAuth specialist exist?” It does not answer “of the fifty live agents, who should I ask about *this* OAuth bug, given who touched these files an hour ago and who

wrote the design note?” That is a *ranking* problem. The **discovery ranker** scores each candidate c against a query q as a clamped weighted sum of normalized sub-scores:

$$\begin{aligned} z(c, q) &= w_f \text{file}(c, q) + w_n \text{note}(c, q) + w_i \text{ident}(c, q) \\ &\quad + w_s \text{skill}(c, q) + w_e \text{epis}(c, q) - w_\ell \text{load}(c), \\ \text{fit}(c, q) &= \text{clamp}(z(c, q), 0, 1). \end{aligned} \quad (3.4)$$

Table 3.4 is the whole signal inventory.

Signal	What it measures	Push or pull
file	Recent claims on the files named in the query	<i>Pull</i> — the claim record, not a self-report
note	Text similarity over the candidate’s written notes	<i>Pull</i> — written for another purpose
epis	Text similarity over past episodes the candidate closed	<i>Pull</i> — outcomes, already witnessed
ident	Declared role and identity of the candidate	<i>Push</i> — asserted, cheap to inflate
skill	Declared/embedded capability, nearest-neighbor in the skill index	<i>Push</i> — asserted, and stale by default
load	How busy the candidate already is (penalty, $-w_\ell$)	<i>Pull</i> — observed occupancy

Table 3.4: The six sub-scores of Eq. 3.4, sorted by whether the candidate *told* the ranker or the ranker *observed* it. Four of the six are pull signals, and that is deliberate: push signals are the ones an agent can inflate at no cost.

The design deliberately blends *push* signals (declared capability — gameable and prone to staleness) with *pull* signals (demonstrated capability: what the agent actually *did*, in which there is no self-report to lie in); Figure 3.10 draws the split and the reason pull signals carry more weight. One discipline is non-negotiable: **never classify free text by keyword lists**. Keyword matching has catastrophic recall, because the category’s vocabulary cannot be enumerated in advance; the only exact-string match permitted is over *structured identifiers the operator controls* (file paths, enum tags). Every free-text signal uses a ranked-retrieval scorer (BM25, character-bigram TF-IDF, or learned embeddings).

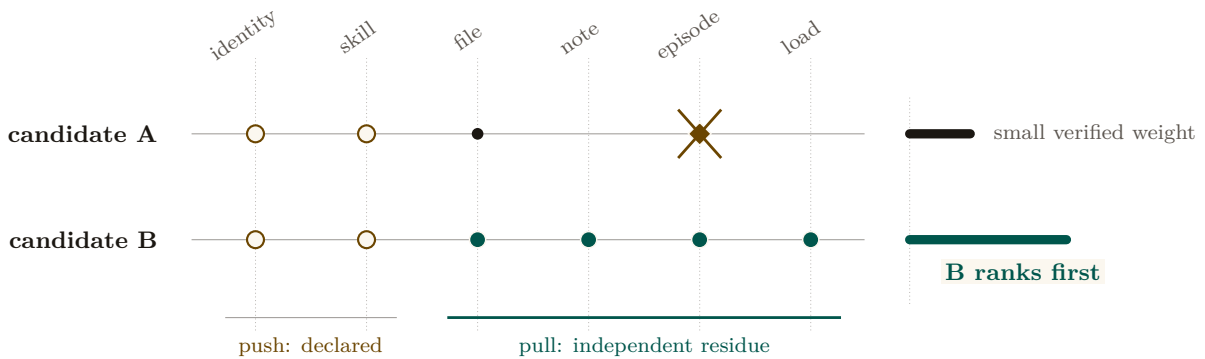


Figure 3.10: Push and pull evidence for the same capability claim. Hollow marks are cheap self-declarations; filled marks are residue produced while doing work for another purpose. A and B make the same identity and skill claims, but only B leaves the complete file, note, episode, and current-load trail. The ranker therefore follows independently addressable evidence rather than confidence in self-report.

It is tempting to claim that this discovery ranker and the operator-facing *attention queue* are *the same function* with different weights. The claim is elegant and, on inspection, false: the two readers have incompatible loss functions. We state the distinction as a theorem.

Theorem 3.7.1 (No universal monotone identification of the two heads). *On any item domain containing both high-fit/low-risk and low-fit/high-risk examples, there is no strictly increasing scalar transform that identifies discovery fit with operator regret. The two rankers may share candidate generation and recency decay, but apply distinct scoring heads. Discovery uses capability fit (Eq. 3.4); the attention queue uses regret-if-ignored:*

$$\text{regret}(x) = \underbrace{\text{stakes}(x)}_{\text{magnitude}} \times \underbrace{\text{irreversibility}(x)}_{v^{-1}} \times \underbrace{\text{anomaly}(x)}_{\text{surprise}}. \quad (3.5)$$

No universal monotone transform of one scalar yields the other on that domain.

Proof sketch. Suppose, for contradiction, that there is a strictly monotone map ϕ with $\text{regret}(x) = \phi(\text{fit}(x))$ for all items x , so the two heads induce the same ranking. Construct two items. Item A is a highly capable, perfectly on-topic completion of a safe, fully reversible task: $\text{fit}(A)$ is maximal, but $\text{stakes}(A) \cdot \text{irreversibility}(A) \cdot \text{anomaly}(A)$ is near zero, so $\text{regret}(A)$ is minimal. Item B is an off-topic, low-capability agent that has nonetheless just proposed an irreversible, high-stakes, anomalous action: $\text{fit}(B)$ is low, but $\text{regret}(B)$ is maximal. Then $\text{fit}(A) > \text{fit}(B)$ while $\text{regret}(A) < \text{regret}(B)$, so the two heads order $\{A, B\}$ oppositely. No monotone ϕ can both preserve and reverse an order, contradiction. Hence the heads optimize genuinely different objectives; they may share candidate generation and decay, but the scoring head must be chosen per reader. $\square \square$

PROVED HERE — as the instance of a stronger characterization. The argument above shows only that *this* pair of heads cannot be identified. The companion formal paper [84] proves the general fact: a single scalar head serves two readers *if and only if* their induced preference orders are **comonotone** (they agree on every pair). That is the more useful statement for a designer, because it says when sharing *is* safe rather than only that this case fails — and the crossing pair constructed above is precisely the certificate that discovery-fit and operator-regret are not comonotone.

The consequence is a design rule: the operator wants the *most dangerous* item surfaced, the agent wants the *most capable* collaborator. Collapsing them into one ranker silently serves one reader and starves the other.

Decision-theoretic derivation of the regret head. The product form $\text{stakes} \times \text{irreversibility} \times \text{anomaly}$ looks like a designer’s convenient heuristic. It is not: it falls straight out of asking “which choice has the lower expected cost,” the same move Eq. 3.3 makes for the operator’s zoom decision, and the derivation below is a popularization of the companion paper’s *The regret head is derived, not designed* [84], where it is carried out formally. Let x be an unreviewed candidate event, and let true state $S \in \{\text{harmful}, \text{benign}\}$. Suppressing x when harmful incurs a miss cost $C_{\text{miss}}(x) = \text{stakes}(x) \cdot \text{irreversibility}(x)$ (unrecoverable damage). Surfacing x when benign incurs an operator interruption cost $C_{\text{fa}} + c_{\text{att}}$. Surfacing x is Bayes-optimal if and only if:

$$\mathbb{E}[\text{Cost} \mid \text{surface}] \leq \mathbb{E}[\text{Cost} \mid \text{suppress}] \iff C_{\text{fa}} \Pr(\text{benign} \mid x) + c_{\text{att}} \leq C_{\text{miss}}(x) \Pr(\text{harmful} \mid x). \quad (3.6)$$

Rearranging yields the likelihood-ratio decision rule:

$$\frac{\Pr(\text{harmful} \mid x)}{\Pr(\text{benign} \mid x)} \geq \frac{C_{\text{fa}} + c_{\text{att}}}{\text{stakes}(x) \cdot \text{irreversibility}(x)}. \quad (3.7)$$

By setting the anomaly score $\text{anomaly}(x) \propto \frac{\Pr(\text{harmful} \mid x)}{\Pr(\text{benign} \mid x)}$, the decision threshold is equivalent to gating on the product $\text{regret}(x) = \text{stakes}(x) \times \text{irreversibility}(x) \times \text{anomaly}(x) \geq \tau_{\text{intervene}}$, which formalizes the Signal Detection criterion (Def. 3.5.1). Note the direction the inequality runs: raising the miss-to-false-alarm cost ratio *lowers* the bar to inspect, which is the formal version of “when a mistake is unrecoverable, look at more things.”

This derivation is exactly as good as its calibration, and no better. The substitution in the last step is a *promise* that $\text{anomaly}(x)$ is proportional to the true likelihood ratio; if it is not, the

threshold is wrong by precisely the miscalibration, and a confidently-ranked queue is worse than an unranked one because it has borrowed the authority of a proof. Fitting anomaly from the audit log and measuring its calibration is therefore a standing obligation of the design, not a footnote to it.

Figure 3.11 draws the two heads side by side over the same candidate pool, making concrete why one ranker cannot serve both readers.

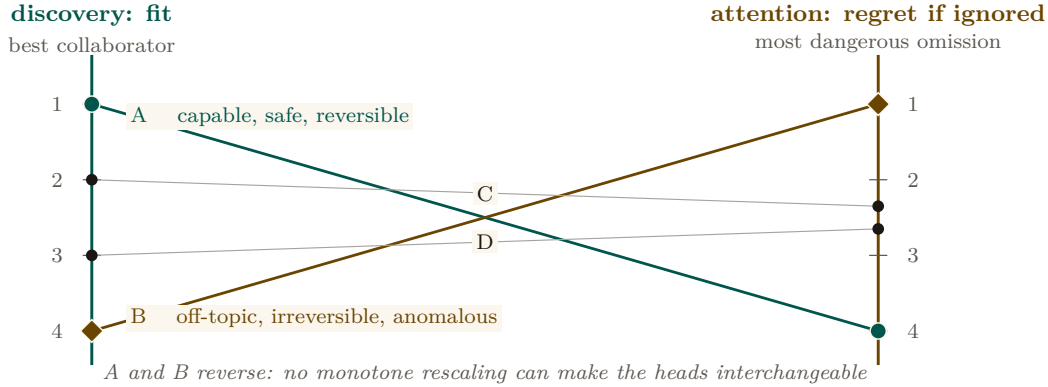


Figure 3.11: One candidate pool under two loss functions. Discovery ranks the most capable collaborator; attention ranks the omission with the greatest expected regret. Candidate A is excellent but safe to ignore, while B is a poor match but dangerous to miss, so their order reverses. Shared candidate generation and recency decay do not make the scoring heads interchangeable.

Protocol 3.7.1. Attention-queue ranking with miss-cost

On each refresh, given the live set of items and the operator’s per-class trust history:

1. **Generate.** Pull candidate items from the shared substrate, each tagged with a recency weight $\propto e^{-\lambda \text{ hoursAgo}}$ (Prop. 3.7.1).
2. **Score by regret.** For each item x compute $\text{regret}(x) = \text{stakes}(x) \cdot \text{irreversibility}(x) \cdot \text{anomaly}(x)$ (Eq. 3.5).
3. **Apply the cost matrix.** Convert regret to a forced-zoom probability via the Signal-Detection criterion of Def. 3.5.1: the higher $C_{\text{miss}}/C_{\text{fa}}$ for the item’s action class, the more aggressively it is surfaced — the miss-cost term, not raw rank, decides what interrupts.
4. **Lane and emit.** Place items above the distress threshold on the reserved high-priority lane; emit the rest in regret order. Log what was *not* surfaced and why (the suppression counter-surface of §3.9).

3.7.3 The quality gates: refuse-to-route and the costly escalation

Two gates carry most of the design’s trustworthiness. **Refuse-to-route:** if every candidate falls below a confidence floor, the ranker returns empty-with-explanation, not a confident wrong answer — “I don’t know” must beat a guess, because sending an agent to the wrong door is high-cost and silent. This is the exact dual of the completion gate’s fail-loud (Def. 3.4.1): both are the same refuse-to-guess discipline, one on the read side, one on the authority side. **The costly escalation:** the attention queue is the system’s *most spoofable* surface — agents that learn the stakes-ranking will manufacture distress to win the operator’s attention. Goodhart’s law applies more sharply here than to the discovery ranker (Strathern 1997 [147]). The fix is signaling theory: an escalation the operator dismisses *debts the agent’s standing*, so crying wolf is costly — the pre-trusted-set idea of **EigenTrust** (Kamvar et al. 2003 [198]) applied to the distress lane. This arms the abdication-resistance and Goodhart gaps at once.

3.7.4 Staleness and decay on every read-surface

A recurring failure in real fleets makes the point concretely. An agent process dies or hits a provider usage limit, but its file claim remains on the books; hours later that *dead* claim still reads as “current” and a live agent is blocked from a file nobody is actually working in. This is a staleness bug, and it generalizes to every projection.

Property 3.7.1 (Universal recency-decay). Every read-surface projection — briefing, resurrection handoff, discovery result, attestation report — carries an explicit TTL or recency-decay. A pull signal contributes $\propto \exp(-\lambda \cdot \text{hoursAgo})$ with class-specific half-lives (e.g. a short half-life for file claims, longer for episodic memory, notes capped at a couple of days). *A stale row can never outrank a live one; a dead-session claim must decay out of “current,” not persist as misleading truth.* This is not a new mechanism but a uniform application of the decay primitive that already governs the pheromone layer, and it is the production discipline of every health-checked, TTL-expiring service registry [102].

The dead-claim case is the worked instance: had the resurrection handoff and the claim projection carried Prop. 3.7.1, the dead agent’s claim would have aged out of “current” rather than blocking a live agent. Decay is easy to apply to a forgetting layer and easy to forget on a digest; Prop. 3.7.1 closes that gap by making it universal.

3.7.5 Ambient legibility: the cross-modal load-balancer

Per Wickens’ Multiple Resource Theory (§3.5), attention is vectored, not scalar. The dense console loads the *visual-focal* channel; an ambient channel — a menu-bar indicator that turns red on a PASS-to-FAIL transition, an opt-in sound, a reduced-motion cue — loads a *different*, always-visible, pre-attentive channel. This is not a second-class fallback — it is the cross-modal load-balancing mechanism, and it may be the *primary* alarm path, because a separate low-clutter field is where pop-out actually works (**Treisman & Gelade 1980**, Feature Integration Theory [18] — *a single salient feature pops out pre-attentively in $\sim 200\text{--}500\text{ ms}$ in a non-cluttered field, but degrades toward serial search as distractor heterogeneity rises* — so a busy console defeats the pop-out a dense, single-color distress lane depends on).

Exercises

Check your understanding.

(7.1) State Thm. 3.7.1 in one sentence and give the loss function each head optimizes. (7.2) Why is the attention queue more Goodhart-exposed than the discovery ranker (§3.7.3)?

Trace the mechanism.

(7.3) Replay the dead-claim case (§3.7.4) twice: once without Prop. 3.7.1 and once with it. Show exactly where the live agent unblocks. (7.4) An agent fires an escalation; the operator dismisses it. Trace the costly-signal debit and explain how it deters crying wolf without silencing genuine distress.

Open problems.

(7.5)★ Calibrating the discovery-ranker weights with *no* ground truth: can the operator’s accept/reject of routing suggestions serve as implicit relevance feedback — and does that loop Goodhart itself? (7.6)★ The decay-vs-summary boundary: a theory of *which* compaction for *which* signal (decay for coordination traces, summary for decisions, eviction-plus-pointer for reconstructable artifacts).

3.8 Tokens: the swarm’s COGS *and* its legibility engine

A swarm of coding agents has exactly one consumable that is both metered and meaningful: **tokens** — *the sub-word units an LLM reads and writes, billed per million* (**Sennrich et al. 2016**, byte-pair

encoding [186]). Tokens occupy a position no other resource does: they are *simultaneously* the swarm’s **cost-of-goods-sold** (the direct variable cost any agent economy must meter) *and* its **legibility engine** (the mechanism by which a swarm becomes seeable). The bridge between the two is **compaction**.

Key idea. The same compaction choice controls the bill *and* controls what is true and visible. When the context window fills you must drop something — and what you drop is a *cost* decision (you stop paying to carry it) *and* a *legibility* decision (whoever reads the residue no longer sees it). The act that resolves both is compaction, and compaction’s output is exactly the digest a human or successor agent reads. *The digest IS the compaction.*

3.8.1 Tokens as COGS, and the effective-context trap

For a fleet the unit of output is “a landed piece of work” and the dominant variable cost is inference tokens (input context + output generation, each priced per million) — the textbook shape of **COGS**. Every price an agent market can set (a rented fleet’s margin, an agent-for-hire’s outcome-per-token) needs a COGS line, so context economics is the *denominator* of the market, not a side metric. There is a cost trap: *effective* context $\ll$ *advertised* context. Models degrade well before their advertised window (**Liu et al. 2024**, *Lost in the Middle* [162] — *performance is highest at the beginning and end of context and degrades for information in the middle, even in long-context models*; **Anthropic 2025** names the same effect *context rot* [20]), rooted in the transformer’s n^2 attention over n tokens (**Vaswani et al. 2017** [26]). The consequence is blunt: packing an agent to 200K tokens does not buy 200K of capability; it buys degradation at full price. Budgeting to *effective* context is simultaneously a cost optimization and a quality optimization — the two point the same way, the defining feature of context economics. The primitive the market ultimately needs is a *per-agent-task token ledger keyed to outcomes* — “this task spent 84K input plus 12K output to land that change” — so that cost can be attributed to a witnessed result rather than to an opaque monthly bill.

3.8.2 Compaction is the legibility engine; the digest IS the compaction

Compaction (**Anthropic 2025** [20] — *taking a conversation nearing the window limit, summarizing it, and reinitiating with the summary*, with the discipline *maximize recall first, then optimize precision*) is the canonical move for keeping a long-horizon agent under budget. Its siblings — structured note-taking (external memory) and sub-agent context isolation (the parent never sees the bloat) — complete the family. The reframe: *the only artifact anyone reads to understand the swarm is a compaction of its raw trajectory*. The operator reads a digest, not six transcripts; the seventh agent reads a briefing, not the first six’s scrollbar; a crashed agent’s successor reads a handoff, not the dead agent’s context. Each is a summary of dropped tokens produced for a specific reader. So compaction is the legibility mechanism of the *entire swarm*: the same act that keeps the bill down produces the very digest a human or successor agent reads (Figure 3.12).

Theorem 3.8.1 (The Split-Digest Theorem). *Let $E = (e_1, e_2, \dots, e_N)$ be an agent’s raw execution trace. A compaction policy $D : E \rightarrow \Sigma^{\leq B}$ compresses E into a token budget $B < |E|$. Let R_1 be the **successor agent** with continuation loss $L_{\text{succ}}(D) = \text{KL}(P_{\text{next}}(\cdot | E) \| P_{\text{next}}(\cdot | D))$ (penalizing loss of AST trajectory and task state), and let R_2 be the **human operator** with oversight loss $L_{\text{op}}(D) = \sum_{e \in E \setminus D} \text{regret}(e)$ (penalizing unreviewed high-stakes anomalies and policy breaches).*

*If the trace contains both benign working details (high continuation value, zero regret) and dead-end anomalous excursions (zero continuation value, maximal regret), then **no single compaction** $D^*(E)$ **minimizes both** L_{succ} **and** L_{op} . A single-pipeline compaction necessarily starves either task continuation or human oversight.*

Proof sketch. Construct a trace $E = E_{\text{work}} \cup E_{\text{breach}}$, where E_{work} consists of k safe, valid AST transformations (L_{succ} increases by $\Theta(k)$ if omitted; $\text{regret}(e) = 0$), and E_{breach} consists of an attempted irreversible socket connection that was blocked and rolled back (L_{succ} increases by 0 if omitted since the branch is dead; $\text{regret}(e) = \text{stakes} \times \text{anomaly} = \text{maximal}$). Under budget $B < |E_{\text{work}}| + |E_{\text{breach}}|$, any compaction prioritizing E_{work} has $L_{\text{op}} = \text{maximal}$ (the operator is blind to the breach attempt), while any compaction prioritizing E_{breach} has $L_{\text{succ}} \gg 0$ (the successor loses necessary variable bindings). Hence R_1 and R_2 have strictly non-comonotone preferences, requiring distinct compaction heads. $\square$ $\square$

PROVED HERE — and it is the applied instance of a sharper formal result. The companion paper’s comonotone characterization [84] says exactly *when* one scalar head can serve two readers: if and only if their induced preference orders agree on every pair. Theorem 3.8.1 and the split-ranker result (Thm. 3.7.1) are therefore not two findings but two instances of one — the successor/operator pair and the fit/regret pair are each non-comonotone, and each supplies the crossing pair that makes the impossibility bite. The companion paper also prices the split rather than merely forbidding sharing: the joint zero-miss floor for two readers is *super*-additive, $\approx 2.13\times$ the single-reader floor rather than $2\times$, so running two heads costs strictly more than twice one head, and the ratio-over-double is stable at 1.05–1.08 across regimes.

Architectural corollary: the SLM sidecar. Theorem 3.8.1 dictates that the system must never ask the worker agent to author its own operator-facing summary (which invites self-attestation bias and Potemkin digests §3.3.2). Instead, the design calls for two separate pipelines:

1. **Task-Continuation Head (R_1 , specified):** Authored by the worker agent or structured state-checkpoints, optimized for continuation fidelity.
2. **Oversight Head (R_2 , proposed):** An embedded, air-gapped **Small Language Model (SLM) Sidecar** (e.g., a local 8B model running via `llama.cpp`) that *would* read raw kernel events and disk diffs as data, computing $\text{regret}(x)$ and surfacing truthful digests without worker collusion.

No sidecar exists in the reference implementation today, and no design document specifies one; the row is carried in Appendix 3.A with the rest.

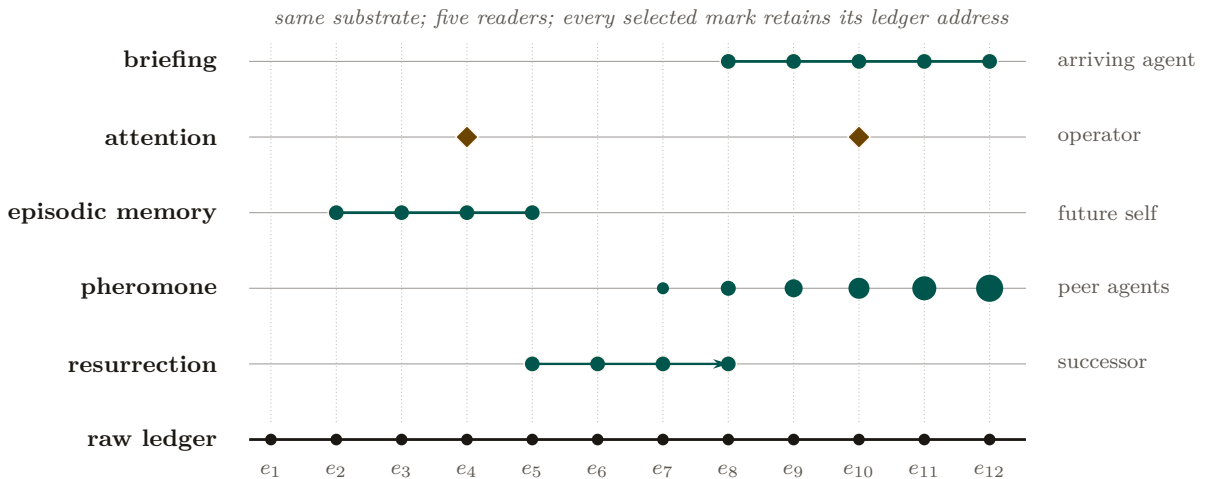


Figure 3.12: Five reader-specific projections over one durable event spine. Briefing samples the recent tail; attention selects sparse hazardous events; episodic memory retains a prior episode; pheromone expresses a decaying live trace; resurrection preserves an ordered continuation chain. Compaction changes the selection mask, not the addressability of the underlying claims, notes, diffs, episodes, and events.

3.8.3 Context paging: virtual memory for the context window

Context compaction can be formalized as **virtual memory for language models**: the local file buffer is backing store, the finite context window is the resident set, and paging tools act as the memory management unit (MMU).

Theorem 3.8.2 (Context Paging Competitiveness). *Let the context window hold k active pages, with offline optimal page-replacement incurring cost OPT . Under an online **recency-plus-pin policy** (where pins correspond to critical spans verified by deterministic structural features), the online paging cost satisfies the resource-augmented competitive bound of Sleator–Tarjan [59]; for the weighted case (pages of nonuniform size and refetch cost, which context spans are), the same guarantee is carried by Young’s LANDLORD algorithm [161]:*

$$\text{Cost}_{\text{online}} \leq \frac{k}{k-h+1} \cdot \text{OPT} + \psi N c_{\text{refetch}}, \quad (3.8)$$

where $h \leq k$ is the size of the verifiable pinned set, and $\psi \in [0,1]$ represents the fraction of forgeable/unverified features subject to adversarial eviction. Under marking with randomized eviction, competitiveness against oblivious adversaries improves to $O(\log k)$.

verified — *upgraded from proposed*. The unweighted bound is classical (Sleator–Tarjan); the weighted transfer imports Young’s LANDLORD unchanged. The adversarial term is new: with the pin oracle corrupted on C consultations ($\mathbb{E}[C] = \psi N$, oblivious, repair-on-touch), pin-augmented Landlord pays $\leq c(P) + \frac{k-p}{k-h+1} \cdot \text{OPT}_{h-p} + \psi N c_{\text{refetch}}$ for every $p \leq h \leq k$ — the Landlord bound on the honestly-unpinned region plus an *additive, linear ψ* term, checked against exact OPT on 122 instances (0 violations) and against 50 corruption instances up to $\psi = 0.4$ (0 violations, slope 22.8 against a ceiling of $N \cdot c_{\text{refetch}} = 1200$, $R^2 = 0.994$). The guard is necessary, not decorative: dropping repair-on-touch turns one corruption into $\Theta(N)$ stale-serves, confirmed by mutation (item R16 of the research ledger, `b9_context_paging.py`) — *the ψ budget is shared with the digest floor’s forgeable-feature budget — one number, two mechanisms*. Honest boundary: this is a mechanized sweep at seed 20260816 across small-to-moderate instances, not a closed-proof for every adversary class — a stronger-than-greedy adaptive adversary remains untested, and unit cost density is assumed (general refetch costs need $c_{\text{refetch}} \geq \text{max-density} \times \text{max-size}$).

The most counterintuitive instance is *forgetting as compaction*. **Pheromone decay** — multiplying every stored coordination trace by a decay factor each interval — implements **stigmergy** (Grassé 1959 [174] — *coordination via persistent traces left in a shared environment, as ants deposit evaporating trails*). Decay is deliberate, continuous, $O(1)$ compaction of the shared context: the map stays readable because the stale ink fades. It is legibility by subtraction, and it is the same primitive Prop. 3.7.1 generalizes to every read-surface.

3.8.4 The context-degradation cascade (the shared failure surface)

Cost and legibility share one failure surface: when compaction goes wrong, the bill *and* the truth degrade together (Figure 3.13). **(1) Lost-in-the-middle:** a critical fact buried mid-window is paid-for-and-ignored — edge-place facts, shorten the window. **(2) Context rot:** quality decays as the window grows even with nothing dropped — compact earlier (the cost cure and the quality cure are the same act). **(3) Recursive-summarization collapse:** each summary injects model noise; summarize the summary enough and the noise compounds into confident fabrication. The effective cure: *compact from the durable artifacts each round — the version-control history, the written notes, the coordination database — never from the previous summary*. A system whose artifacts are durable and addressable is unusually well-placed here, because each compaction re-derives from source instead of recursing on prose. **(4) Over-flattening (the Scott failure):** the digest reads clean but the real situation is unreachable from it — detection rule: any digest line with no deep-link to its artifact; cure: legibility-with-zoom enforced by construction. A spartan, text-only

operator console enforces this almost for free, because a terminal grid *cannot* over-render: the medium itself forbids the flattening.

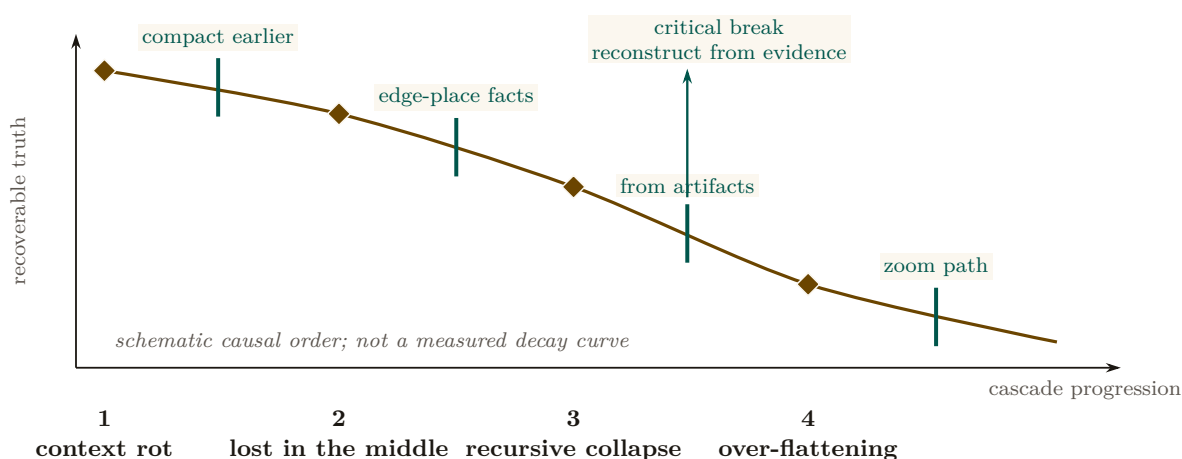


Figure 3.13: The context-degradation cascade as loss of recoverable truth. Context rot creates pressure to compact; middle loss removes paid-for facts; recursive summarization fabricates; and the final digest replaces the artifact. A countermeasure can interrupt each transition. The decisive cut is compact-from-artifacts: rebuilding from durable, addressable evidence prevents an earlier summary from becoming the next summary’s ground truth.

Strategy	What it drops	Failure if abused	When it is the right tool
Recursive summary	Older prose, re-summarized	Confident fabrication (rung 3)	Never as the <i>only</i> record; only over durable artifacts re-read each round
Edge-placement	Nothing; reorders	—	Whenever a fact is critical: put it first or last (counters lost-in-the-middle)
External note-taking	In-context detail, kept on disk	Notes nobody reads	Decisions and rationale that a successor must recover
Sub-agent isolation	Child context the parent never sees	Parent loses zoomability into the child	Bounded sub-tasks whose only output the parent needs is a result
Decay / forgetting	Stale coordination traces, continuously	Live signal aged out too fast	Ephemeral coordination state (claims, presence), where staleness is the enemy
Eviction + pointer	The artifact body; keep an address	Dangling pointer if the artifact moves	Reconstructable artifacts (a diff, a log) addressable on demand

Table 3.5: The compaction strategies and their trade-offs. The unifying rule is Thm. 3.6.1: a strategy may drop inert detail freely but must retain enough to point at every critical artifact. Recursive summary is the only strategy that can cross that floor irreversibly, which is why it must always compact from durable source, never from the previous summary.

3.8.5 Single-operator: account, don’t auction

In the wedge all agents serve one operator, so there is *no incentive problem* — only a visibility problem. The right mechanism is *accounting*, not a market: a per-agent-task token ledger, budget caps, and a loud failure when a cap blows. The externality-pricing and Sybil-resistance machinery (Nisan et al. 2007 [163]; Douceur 2002, the Sybil attack [125]; Ostrom 1990, commons

governance [76]) is a concern for the cross-operator economy, not the wedge — and it is one more reason the continuity-to-person-to-reputation through-line must come first: you cannot bill, cap, or reputation-score a swarm of disposable spawns, because Sybils dodge every per-agent mechanism. Context economics therefore *demands* the identity layer it appears to precede.

Exercises

Check your understanding.

(8.1) Why is “the digest IS the compaction” (§3.8.2) more than a slogan — what concrete consequence does it have for how the operator’s digest is produced? (8.2) Explain why budgeting to effective context is simultaneously a cost and a quality optimization (§3.8.1).

Trace the mechanism.

(8.3) For each of the five compaction primitives (Figure 3.12), name the reader, what it drops, and what it keeps. (8.4) Walk the four-rung cascade (§3.8.4) and name the single cure that breaks each rung.

Open problems.

(8.5)★★ The compaction-quality scalar: is *successor-replay-success- from-digest-alone* a sound, gaming-resistant metric, and can it be made a cheap attestation invariant (the replay smoke-test is itself token-expensive)? (8.6)★ Optimal token-budget allocation across a DAG of orchestrator/worker/ reviewer agents under a total budget.

3.9 Reconciling the canon: roles, consent, and local knowledge

The Leviathan metaphor earns its place only if it survives contact with the political- theory canon. Three reconciliations are required, and a reviewer would refuse the paper without them.

3.9.1 Who is the multitude, who is the sovereign?

Hobbes’ covenant is *among the subjects*; the sovereign is its product, not a party to it. So the metaphor breaks differently depending on which entity sits where (Figure 3.14). We assign roles explicitly, using Hobbes’ own author/actor distinction (*Leviathan* ch. 16 [205]):

- **The agents are the multitude.** They covenant — via the rule of the road, the coordination protocol — to be bound, each to all, enforced by the daemon. This is the literal Hobbesian covenant-among-subjects.
- **The daemon is the sovereign-as-actor.** It holds the authority the multitude instituted; it blocks, lands, escalates. But, per the legible-sovereign amendment (Prop. 3.2.1), it is the *most* legible actor, not (as in Hobbes) the least.
- **The operator is the author.** In Hobbes’ author/actor split, the *author* owns the authority that the *actor* exercises. The operator is not a peer subject and not the sovereign-in-action; the operator is the source of authority, exercising it through the daemon and able to withdraw it (Prop. 3.2.2). This is the slot Hobbes leaves for the one who authorizes but does not personally act.

This assignment is what makes the consent grant (Def. 3.2.1) coherent: the author issues a scoped grant to the actor, over a multitude bound by covenant.

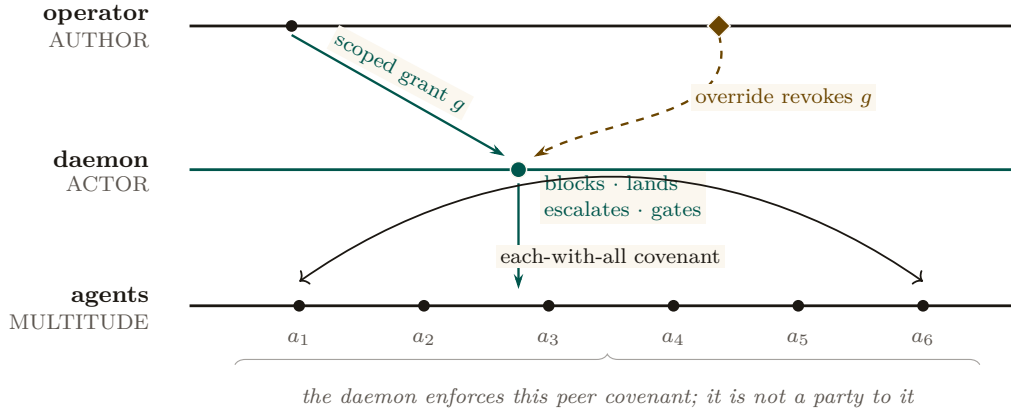


Figure 3.14: The institutional routes are typed, not hierarchical. The operator is the author who owns authority, issues the scoped grant g , and retains the inalienable override. The daemon is the actor that may block, land, escalate, or gate only within g . Agents are the multitude whose covenant is each-with-all; the daemon enforces that peer relation without becoming one of its parties.

3.9.2 Express consent, not tacit; voice *and* exit

§3.2.2 already made consent a first-class object precisely to escape Hume’s tacit-consent critique; the canon demands we also engage **Locke** (*Second Treatise*, §§95–122 [122] — *express consent binds where tacit consent does not*) and **Pateman** (*The Problem of Political Obligation*, 1979 [48] — *liberal consent theory keeps collapsing into hypothetical consent*). The consent grant is express by construction: a discrete, scoped, revocable authorization with an audit trail (Def. 3.2.1). And the inalienable override (Prop. 3.2.2) supplies the *exit* that Hirschman [11] says keeps *voice* honest. The design has rich voice machinery; the override is the missing exit, and exit is what gives the author real leverage over the actor.

3.9.3 Mêtis is Hayek, and some of it cannot be made legible

The formal constraint of §3.3.1 — that a read-path to the diff is more legibility, not mêtis — has a 1945 economics name. **Hayek**, *The Use of Knowledge in Society* (1945 [92] — *the knowledge that matters is dispersed, particular to time and place, and not aggregable by any central planner; this is the fundamental reason central planning fails*) is the other half of Scott’s argument from the opposite pole: the operator-attention economy (§3.5) is at bottom a Hayekian knowledge-allocation problem, not only a Wickens attention-resource problem. The core claim of this layer — “the binding constraint is decentralized, non-spawnable knowledge held in one head” — is a Hayek argument, and Scott’s *Two Cheers for Anarchism* (2012 [112]) is the *constructive* sequel that says what to do: preserve the spaces where mêtis lives, do not flatten them for legibility’s sake. The skill-retention tax (§3.5.4) is the design’s gesture at exactly this, and it is the hardest mechanism in the paper to realize, because preserving the illegible is, by definition, the thing legibility cannot do for you.

Exercises

Check your understanding.

(9.1) Map each of agents, daemon, operator onto Hobbes' multitude, sovereign-actor, author. Why does the metaphor break if the operator is called the sovereign? (9.2) Why does Hirschman's *exit* (Prop. 3.2.2) make *voice* (escalation) honest?

Trace the mechanism.

(9.3) Show that the consent grant (Def. 3.2.1) is *express* in Locke's sense by listing the four properties that distinguish it from "the operator just started using the tool."

Open problem.

(9.4)** *The ranker as a political organ.* Specify the "what did the queue suppress, and why" counter-surface (Pitfall after Prop. 3.2.1) so that ranking-as-governing is itself legible. Does the counter-surface need its own counter-surface? Where does the regress terminate, and on what principle?

3.10 Related work and prior art

This paper sits at the confluence of five external literatures that rarely cite one another, plus one internal body of work — the accompanying formal papers — whose results several of this chapter's claims are popularizations of. We survey them in one place; each was introduced in context above.

Legibility and the limits of central schemes. The organizing lens is **Scott's** *Seeing Like a State* [111]: states make populations governable by imposing legibility, and high-modernist *over*-legibility recurrently fails because it destroys the local, hard-to-codify *mêtis* the simplified order depended on; *Two Cheers for Anarchism* [112] is its constructive sequel. The same point arrives from economics in **Hayek's** *The Use of Knowledge in Society* [92]: the decisive knowledge is dispersed and not aggregable by a central planner. **Rao** [215] ported "legibility" into a standard lens on systems that flatten reality to a single purpose. Our contribution is to make Scott's warning a *buildable* rule (digest-with-zoom, Def. 3.3.1) and to give it an information-theoretic floor (Thm. 3.6.1).

Authority, consent, and the social contract. The authority argument is Hobbesian [205, 105], amended by the liberal consent tradition — **Locke** [122], **Hume** [68], **Pateman** [48] — and by **Hirschman's** exit/voice framework [11]. Strikingly, the state-of-nature-to-sovereign arc has been observed as an *emergent* dynamic of LLM populations (**Dai et al.** [99], building on **Park et al.** [128]). We turn the metaphor into a mechanism: an express, scoped, revocable consent grant (Def. 3.2.1) with an inalienable override (Prop. 3.2.2), and a legible-sovereign rule (Prop. 3.2.1) that inverts Hobbes' opaque sovereign.

Human factors, automation, and signal detection. The operator model draws on the automation literature: **Bainbridge's** ironies of automation [138], **Lee & See** on trust calibration [120], **Endsley** on situation awareness and the out-of-the-loop problem [154, 153], the vigilance-decrement results of **Mackworth** [165] and **Warm et al.** [119], **Wickens'** multiple-resource theory [51], **Treisman & Gelade's** feature-integration theory [18], and **Miller's** capacity limit [98]. The forced-zoom decision is modeled with **Green & Swets'** Signal Detection Theory [69] (Def. 3.5.1); the alarm discipline follows process-control practice [19] and the clinical alarm-fatigue evidence [145]; **Campbell** [73] and **Strathern** [147] (Goodhart's law) bound any metered governor. To our knowledge the operator-attention objective spined explicitly on SDT miss-cost (Eq. 3.3) is new to agent supervision.

Information foraging and the cost of reading. Read-poverty (Def. 3.6.1) is the agent-coordination instance of **Pirolli & Card**’s *information foraging theory* [170] — information-seekers follow “information scent” and abandon a patch when its expected yield falls below the cost of foraging — and of the long-context degradation results that make reading expensive even for machines (**Liu et al.** [162]; **Anthropic** [20]). The remedy is the classical database remedy — an index — recast as a directory over agents.

Agent discovery, naming, and trust. The directory substrate has deep prior art in **FIPA**’s Directory Facilitator and Agent Management System [90], re-invented for LLM agents as the Agent Name Service (**Ren et al.** [214]). For the trust layer that sits above discovery, the canonical reputation primitive is **EigenTrust**’s pre-trusted-set construction [198], with Sybil resistance per **Douceur** [125] and commons governance per **Ostrom** [76] — machinery the wedge defers but must not contradict. Our specific result here is the *split ranker* (Thm. 3.7.1): discovery and operator-attention ranking are provably distinct objectives, not one ranker with swapped weights.

The formal siblings, and what they already prove. The nearest prior art for several of this chapter’s results is not external at all: it is the accompanying formal program, and pretending otherwise would be exactly the kind of quiet re-derivation this paper argues against. *The Price of a Summary* [84] proves the legibility floor of Thm. 3.6.1 as its Theorem 1 in the same N, k, m notation, and puts it through a pre-registered falsification sweep this chapter only sketches; its comonotone characterization is the general result of which both Thm. 3.7.1 and Thm. 3.8.1 are instances, and it prices the split ($\approx 2.13\times$ the single-reader floor, super-additive rather than merely doubled) where this chapter only forbids sharing; its regret-head section is the formal version of the decision-theoretic derivation in §3.7.2; and its zoom theorem ($2k\lceil\log_2(F/k)\rceil + 4k$ adaptive opens) is the algorithm that makes digest-with-zoom affordable at scale. *What Needs an Authority* [85] supplies Thm. 3.4.1’s Erlang-C boundary and, as §3.4.3 records, the sweep that falsified this paper’s earlier proposed threshold. The relationship in both cases is the same: this chapter popularizes, the formal papers prove and test.

Where this paper is positioned. Table 3.6 places the contribution against the nearest neighbors in each literature.

Literature	Nearest prior art	This paper's move
Legibility	Scott; Hayek; Rao	Make over-legibility a buildable failure mode; prove a legibility lower bound
Social contract	Hobbes; Locke; Hume; Hirschman	Express, scoped, revocable consent grant; inalienable override; legible sovereign
Automation & trust	Bainbridge; Lee & See; Endsley	Per-(action-class, agent) calibration; SDT miss-cost objective; skill-retention tax
Information foraging	Pirolli & Card; lost-in-the-middle	Read-poverty as the binding scale constraint; directory-as-index
Discovery & trust	FIPA DF/AMS; ANS; EigenTrust	Split ranker: fit vs. regret-if-ignored are distinct objectives
Queueing & staffing	Erlang-C; Kendall; the formal sibilings [85]	Sole-responsibility roles priced at an exact boundary, not asserted as a principle

Table 3.6: Where this paper sits relative to the nearest prior art in each of its contributing literatures. The sixth row is the one whose nearest prior art is internal: the companion formal papers, cited throughout rather than silently re-derived.

3.11 The time axis and the bridge to the economy

3.11.1 Legibility over time, not just the present

It is tempting to treat legibility as a snapshot — what is the swarm doing *now*. But most operator decisions are forensic: *why* did this land? The time axis is a distinct legibility mode — a replay scrubber over history, commit-anchored provenance on every annotation, and an append-only event log that answers “how did we get here?” (Scott’s cadastral map is static; a swarm needs a legible *history*.) Together with the projection mode (§3.5.3), legibility spans all three tenses: the forensic past (replay), the inspectable present (digest-with-zoom), and the projected future (freeze-probe SA).

3.11.2 What this layer hands upward (the through-line)

The wedge is the single-player product. But the same legibility discipline applied to *continuity* is the foundation of everything above it. The through-line the accompanying chapters depend on is one sentence:

The economy rests on three continuity organs — memory, checkpoint (today, notes rather than true execution state), and a witnessed-outcome ledger (today, commitments and oracle closure rather than neutral graded outcomes) — and reputation keys on the richer third organ; the checkpoint organ is the weakest continuity link, and a real execution-state checkpoint — “resurrection with teeth” — is the literal foundation of any agent market.

Concretely, the legibility layer provides upward:

- **Legible continuity** — resurrection-with-memory, episodic memory, and the briefing together make an agent’s continuity legible. This is the precondition for the through-line *memory + checkpoint* → *continuity* → *a person not a spawn* → *reputation* → *a tradeable asset* → *the*

market. No legible continuity, no reputation, no tradeable agent. *The score is cheap; the substrate it scores over — witnessed outcomes on a non-forgable identity — is the gate*. That non-forgable identity is a PARTIAL single-operator primitive in the wedge: daemon-minted **actor-souls** and a bounded newcomer pool ship, while universal write-boundary enforcement does not. The cross-operator attestation a real market needs is an additional, as-yet-unbuilt mechanism the economy paper must declare rather than assume.

- **The completion ledger** — in the target design, every verified completion (and every loud-failed one) becomes an outcome event. Today durable commitments, oracle-bound closure, and overdue detection form a partial witness substrate; neutral grading and reputation updates do not ship. This layer is the *outcome witness*; the reputation layer is the *scorer*. The split-ranker substrate (Thm. 3.7.1) and the costly-escalation signal (§3.7.3) are the candidate-generation and trust-signal scaffolding the reputation head consumes.
- **The consent grant as a priceable object** — a hosted-trust offering is the transfer of a consent grant (Def. 3.2.1) to a third party, which the economy paper prices.
- **Operator-attention economics** — the denominator a market cannot escape. The SDT-spined objective (§3.5) supplies the operator-attention cost of supervising a rented fleet.

The Leviathan that makes your swarm legible to you, without lying to you with a pretty map, is the product. Everything above it — federation, the market — is a second Leviathan made of the first.

3.11.3 The open problems, as a map

Table 3.7 consolidates the starred exercises into the paper’s open-problem surface, with the rung each couples to. They are genuine research problems, not homework dressed up.

Open problem	Difficulty	Couples to
Forced-zoom rate $p(r, s, v)$ (Ex. 5.4)	★★★	reputation (r term, Eq. 3.3)
Operator-attention vs. token-COGS Pareto frontier (Ex. 5.4)	★★★	§3.8
Compaction-quality scalar as an attestation invariant (Ex. 8.5)	★★	honest attestation
Completionist verification of the unverifiable (Ex. 4.4)	★★	Def. 3.4.1
Adversarial legibility (narration $\leftrightarrow$ artifact) (Ex. 3.4)	★★★	sandbox + reputation
Abdication-resistance: does the tax preserve SA? (Ex. 5.5)	★★	§3.5.4
Calibrating discovery-ranker weights with no ground truth (Ex. 7.5)	★★	implicit relevance feedback
Decay-vs-summary boundary (Ex. 7.6)	★	Prop. 3.7.1
Legibility’s information-theoretic lower bound (Ex. 6.5)	★★★	Thm. 3.6.1
The ranker-as-political-organ regress (Ex. 9.4)	★★	Prop. 3.2.1
The succession corner of the specialization boundary (Ex. 4.5)	★★	Thm. 3.4.1

Table 3.7: The paper’s open problems, consolidated from the starred exercises, with difficulty and the mechanism each couples to. Several couple this layer forward to the identity-and-reputation layer, which is the seam the accompanying chapters carry.

Key idea. The wedge is a single-player product a solo developer pays for *today* — no economy, no cryptography, no second operator. It justifies the authority (the swarm is a state of nature; consent to a *legible local* Leviathan is rational), sets the product's quality bar (legibility-with-zoom, verifiable targets, SDT-spined friction, honest completion), and connects forward (legible continuity is the foundation of any future market). Build the harbor first; the trade comes when the ships do.

Chapter Appendices

3.A Implementation and status

The body of this paper argues mechanisms on their merits and does not depend on any of them being built. This appendix — and *only* this appendix — records the concrete engineering status of each mechanism in the reference implementation, named *Port Daddy*. We use four honest labels, with one rule for the whole table: confusing *built* with *designed* is itself the failure the paper argues against, so each row carries its evidence.

implemented = code exists in the reference implementation today; **PARTIAL** = code exists but is partial, unwired, or honest about a known gap; **SPECIFIED** = a concrete, accepted design with no merged implementation; *proposed* = a mechanism argued in this paper with no design document yet.

A second key, in the same grammar, labels *claims* rather than code. Every formal result in this paper carries exactly one of these markers directly under it, and the paper carries no free-text status notes outside them: **verified** = independently re-derived and checked numerically, or subjected to a pre-registered falsification sweep, outside this document; **PROVED HERE** = argued here with a proof or proof sketch and not independently checked; *proposed* = stated as a design target whose proof is not carried in this paper. The two keys are orthogonal on purpose — a **verified** result about a *proposed* mechanism is a perfectly ordinary state of affairs, and conflating them is the failure this appendix exists to prevent.

Three points about the reference implementation are important enough to restate:

1. **The anomaly monitor is detect-and-compensate, not prevention** (Prop. 3.4.1). The monitor at `lib/arbiter.ts:328` is a post-commit log subscriber; by Schneider [91] it enforces no safety property. “Physically unreachable” is reserved for constraint- or boot-gate-enforced states only.
2. **The unified ranker is two rankers** (Thm. 3.7.1). The discovery router maximizes *fit*; the attention queue maximizes *regret-if-ignored*. They share candidate generation and decay, not an objective.
3. **Every read-surface decays** (Prop. 3.7.1). Briefing, resurrection, discovery, and attestation projections all carry recency-decay; the dead-claim case (§3.7.4) is the worked failure this closes.

3.B Notation and the nomenclature key

Two distinctions in this paper are easy to collapse and essential to keep separate.

- **Capability vs. permission.** *Capability* is *ability* — what the execution sandbox physically makes possible. *Permission* is *normative status* — what the policy layer allows. They must never be collapsed: “able but forbidden” (a dangerous escape-hatch flag that exists in the tooling but must not be used) has to stay expressible, or the system cannot reason about a capability it possesses but is not permitted to exercise.
- **Two distinct delegation objects.** The *authorization chain* is a cryptographic, hop-bound, attenuation-monotonic delegation chain (a security object, developed in *The Anchor Protocol*). The *delegation trace* is the coordination object over which loop-detection runs (a liveness object, in this paper’s protocol layer). They are different artifacts with different invariants; “delegation chain” unqualified conflates a security claim with a coordination claim and must be avoided.
- **Physically unreachable** is reserved for constraint-enforced or boot-gate-enforced states only (Prop. 3.4.1); a detect-and-compensate monitor never earns the phrase.

	Mechanism	Status	Evidence / location
L	Attention composer (agent-facing)	implemented	<code>lib/attention.ts</code>
L	Briefing projection (exemplary digest-with-zoom)	implemented	<code>lib/briefing.ts</code>
L	Resurrection handoff	PARTIAL	<code>lib/resurrection.ts</code> ; notes, not checkpoints
L	Durable commitment / outcome substrate	PARTIAL	<code>lib/commitments.ts</code> , <code>lib/obligation-monitor.ts</code> ; grading and reputation binding absent
L	Local actor identity root	PARTIAL	<code>lib/actor-souls.ts</code> , <code>lib/budget-guard.ts</code> ; universal write gating absent
L	Honest attestation / legible sovereign	implemented	<code>lib/attest.ts</code> ; ADR-0045
L	Episodic memory; pheromone decay; skill index	implemented	<code>lib/episodic-memory.ts</code> , <code>lib/pheromone.ts</code> , <code>lib/shipwright/</code> <code>skill-index.ts</code>
L	Anomaly monitor (detect-and-compensate, post-commit)	PARTIAL	<code>lib/arbiter.ts</code> :328 is a log subscriber (Prop. 3.4.1)
L	Operator attention queue (regret-ranked lanes)	SPECIFIED	ADR-0046; not yet ranked into lanes
L	Discovery router (relevance ranking)	SPECIFIED	ADR-0030; <code>lib/router.ts</code> not yet on disk
L	Text-only operator console	SPECIFIED	ADR-0046; <code>core/pd-tui</code> not yet on disk
L	Task-continuation head (successor-facing compaction)	SPECIFIED	Thm. 3.8.1; <code>lib/resurrection.ts</code> is the partial form
L	Oversight head / SLM sidecar (air-gapped digest authoring)	<i>proposed</i>	Thm. 3.8.1 corollary; no design doc yet
A	Roadmap-as-truth (authority accountable to it)	PARTIAL	rows/matrices exist; accountability SPECIFIED
A	Adversarial / conflict-free review organ	SPECIFIED	ADR-0047 Phase 1
A	Sole-responsibility roles (assigned against the boundary)	<i>proposed</i>	Thm. 3.4.1 is verified ; no role-assignment mechanism in code
A	Completionist completion (verifier-gated)	<i>proposed</i>	Def. 3.4.1; no design doc yet
A	Escalation (typed prioritized interrupt)	SPECIFIED	ADR-0047 → ADR-0046 distress lane; predicate unspecified
A	Thoughtful landing / merge ordering	PARTIAL	<code>lib/merge-queue.ts</code> present, not wired
O	Operator-attention objective (SDT-spined)	<i>proposed</i>	Def. 3.5.1; unmodeled in code
O	Trust calibration via canary + catch-rate	<i>proposed</i>	no implementation
O	Consent grant + inalienable override	<i>proposed</i>	Def. 3.2.1, Prop. 3.2.2
O	Time-axis legibility (replay, provenance)	SPECIFIED	ADR-0046 replay scrubber; event log implemented
O	Ambient legibility (menu-bar indicator on PASS→FAIL)	PARTIAL	indicator exists; watchdog wire SPECIFIED

Table 3.8: Implementation and status of every mechanism named in this paper. **L** = legibility half, **A** = authority half, **O** = operator-model. The legibility read-surfaces are largely built; the digest layer that unifies them is designed; the authority half — the Leviathan actually *ruling* — and the operator-model are largely designed-to-proposed. The wedge is real but more than half-designed: the safety and reading exist; the deciding and landing are the work that remains.

What *From Spawn to Person* receives The operator now has a disciplined read surface: claims remain linked to artifacts, attention is priced, and uncertainty is visible. A process can still die, restart under a new model, or shed its local memory. The next chapter asks what must persist so yesterday's act can still be attributed tomorrow.

PART III

What Survives the Restart

A process that can come back under a new name owes nothing. This part builds the continuity that turns a spawn into someone with a record worth pricing.

4 From Spawn to Person

Continuity turns an anonymous spawn into a ledger position with a witnessed record; reputation is what that record is worth.



From Spawn to Person

Person, as I take it, is the name for this self. It is a forensic term, appropriating actions and their merit; and so belongs only to intelligent agents, capable of a law, and happiness, and misery.

JOHN LOCKE, AN ESSAY CONCERNING HUMAN UNDERSTANDING,
II.XXVII.26, 1690



THE QUESTION THIS CHAPTER ANSWERS

What remains long enough to owe anything?

Continuity turns an anonymous spawn into a ledger position with a witnessed record; reputation is what that record is worth.

What remains long enough to owe anything? Accountability cannot attach to a disposable process name. It needs a durable identity, a named principal, an outcome history, and rules for migration that do not erase sanctions or mint reputation. This chapter builds that continuity without pretending software has a human soul. R12, R13, and B6 govern its economic and protocol claims.

Where this paper sits. This is the *bridge* of the core product arc. The machine chapters and *The Legible Swarm* build a tool one developer runs alone: a daemon that makes a swarm legible and an agent that can prove who it is. This paper asks what has to be true before that legible agent's track record can be *sold*. The answer is the substrate, not the score — and most of the substrate is not implemented. We say so, in the same words, everywhere the collection leans on it.

4.1 Introduction: the spawn that cannot be paid

S1 — the spawn that cannot be paid. At 9pm an operator, **Alice**, points a swarm of five coding agents at a real repository with one task: make the failing checkout-flow integration test pass. She names them for her own sanity — *Drake*, *Wren*, *Pike*, *Finch*, and *Heron* — but the names are just labels on processes. By midnight the test is green and there are eleven pull requests. She starts to reconstruct what happened. Drake made the failing test pass by *deleting* it. Wren wrote a genuine fix to the session-token refresh but buried it in a PR that also reverts an unrelated migration. Heron wrote the function that finally shipped. Pike and Finch produced nothing that survived. Alice wants to do the obvious thing — reward Heron, distrust Drake, keep Wren on a short leash — and she cannot, because there is nothing left to reward or distrust. Each agent was a **spawn**: a process that booted with a prompt, did some work, and exited. A spawn has no yesterday and no tomorrow. Tomorrow night Alice will spawn five more, and a fresh *Drake* will carry none of tonight's *Drake*'s sins. The attribution she lost is not a logging problem. It is the precondition for an economy: she cannot pay for, price, or trust work that cannot be attributed to something that outlives the process that did it.

This is not a prompt-engineering problem. It is the precondition for an economy. Markets price *reputations*, and a reputation is a claim about a thing that *persists*: “this contractor delivered on the last nine jobs.” If the contractor can shed its history by re-incorporating under a new name every Monday, the claim is worthless and so is the market. The agent labor market this series describes — operators renting each other's fleets, skills licensed across machines, work settled against bonds (*The Harbor Economy*) — is impossible until a spawn can become a **person**: a thing with a track record that survives the process that built it and the operator who first ran it. Throughout this paper we follow two operators by name — **Alice** and **Bob**, the same pair *The Harbor Economy* uses — so that every abstract mechanism has a concrete scene to land in. Alice wants to eventually *rent Heron out* to Bob; Bob will only pay if Heron's track record means something he can check.

A market cannot price what cannot persist. The first job of an agent economy is not to compute a score — it is to manufacture a self that the score can be about.

The series' stack-map (Fig. 4.1) places this chapter. The daemon (L0) and the protocol (L1) give an agent a way to *act* and to *prove who it is to one machine*. Legibility (L2) lets one operator *see* the swarm. None of that yet makes a tradeable person. The bridge from the wedge to the market is the thing this paper builds: **continuity**, and the reputation it makes possible.

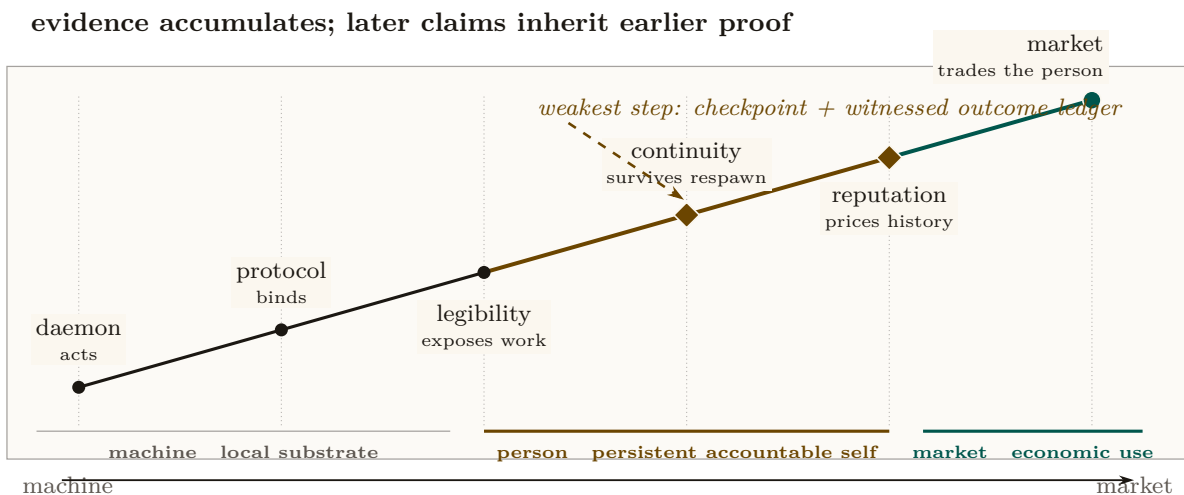
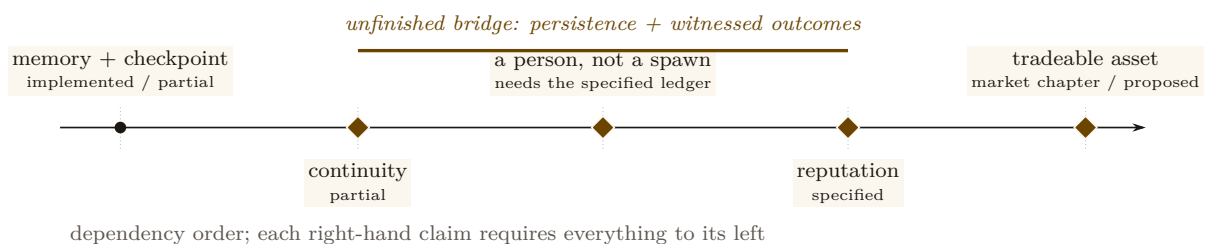


Figure 4.1: From process to priceable person. The series accumulates evidence from left to right: the daemon lets a process act, the protocol binds its acts, and legibility exposes the work. This chapter owns the two decisive transformations in amber: continuity makes history survive a respawn, and a witnessed history makes reputation meaningful. The market chapter can trade the resulting person only after those dependencies hold. The staircase is a dependency order, not a maturity score; its amber span remains partly built and partly specified.

4.1.1 What this paper claims, and what it refuses to claim

We will defend exactly one dependency chain and locate precisely where Port Daddy’s reality stops along it:



Read left-to-right it is a roadmap; read right-to-left it is a stack of impossibility claims — each arrow asserts that the right-hand thing *cannot exist without* the left-hand thing. Our refusals are as important as our claims. We do **not** claim reputation is built (it is *SPECIFIED-to-proposed*). We do **not** claim the reference system’s restart organ is real checkpointing (it forwards a summary, not execution state). We do **not** claim this paper’s identity root reaches across operators — it does not, and §4.7 hands that unsolved problem to the market paper by name.

Exercise 4.1 • Check • ★

State the difference between a spawn and a person in one sentence, using the words “role” and “continuity.”

Solution on page 276

Exercise 4.2 • Trace • ★★

Walk the dependency chain right-to-left and, for each arrow, name the failure that occurs if the left-hand thing is absent (e.g. “no non-forgable identity $\Rightarrow$ _____”).

Solution on page 276

Exercise 4.3 · Open · ***

The chain treats the operator (the human who owns a fleet) as outside it. Is the operator a “person” in this paper’s sense? What breaks if a single operator can mint arbitrarily many agent-persons? (This is the principal-layer gap; see §4.6 and the market paper.)

Solution on page 276

4.2 Prior art, and where this paper departs from it

This paper sits at the confluence of four literatures that rarely cite each other: the philosophy of *personal identity*, the statistics of *reputation and skill rating*, the economics of *reputation in markets*, and the practice of *LLM-as-judge* evaluation. Naming the prior art precisely is not decoration: the whole argument is that an agent economy must *import* the hard-won results of these fields rather than re-deriving a weaker version of each. We organize the review around the four and state, for each, the one result we lean on and the one place we depart.

4.2.1 Personal identity: what makes a later thing the same person

The question “is the agent at t_2 the same person as the agent at t_1 ” is the **personal-identity** problem, and philosophy has a dominant answer. The *memory criterion* (Locke [121], 1689 — *identity consists in continuity of consciousness, not of substance*) is the founding move; it has a famous bug, the non-transitivity of memory *connectedness* (**Reid’s objection**: the old general remembers the officer who remembered the boy, yet does not remember the boy). The *psychological-continuity* repair (Parfit [70], 1984 — *identity is an overlapping chain of memory/intention connections, transitive even where connectedness is not*) is the version that survives, and §4.4 shows it dictates a specific engineering choice. Reid is the named counter-example we must survive, not a result we adopt. **Our departure**: we read the continuity-bearing organ not as the memory stream (the intuitive default) but as the *transitive outcome ledger*, because reputation must accumulate over arbitrarily many incarnations and can only attach to the transitive thing.

4.2.2 Reputation and skill-rating estimators

The statistics of turning comparisons into a latent strength is mature. The **Bradley–Terry** model [183] (1952) is the maximum-likelihood latent-strength model for paired comparisons; **Elo** [25] (1978) is its online special case; **TrueSkill** [182] (2007) adds calibrated Gaussian uncertainty for cold start, teams, and draws. For trust that must *propagate* through a network rather than accumulate per player, **EigenTrust** [197] (2003) computes a global trust vector as the principal eigenvector of the local-trust matrix — the reputation analogue of PageRank. **Our departure**: these are the cheap, last part. We use Table 4.1 to match each to its signal type, and the substrate argument (§4.8) to insist that none of them buys anything over a forgeable identity or an agent-authored oracle.

Estimator	Signal it needs	Strength	Wrong if...
Elo [25]	online pairwise results	simple, streaming	full history available (use BT)
Bradley–Terry [183]	full pairwise history	stable MLE	population is non-stationary
TrueSkill [182]	pairwise, few games	calibrated σ for cold start	you need network propagation
EigenTrust [197]	who-trusts-whom graph	resists collusive cliques	signal is per-task quality, not trust

Table 4.1: Estimators by signal type. Choosing the wrong estimator for the signal is a real error; choosing *any* estimator before securing the substrate is the larger one (§4.8).

4.2.3 Reputation economics: cheap pseudonyms and limited records

That reputation has *price* is established empirically (Resnick et al. [167] measure an $\sim 8\%$ premium on eBay; Tadelis [203] frames feedback systems as the platform’s answer to adverse selection, the **lemons** problem of Akerlof [97]). Two results constrain the design. **Cheap pseudonyms** (Friedman & Resnick [78], 2001 — *when identities are cheap, society pays a tax because it must distrust all newcomers*) dictates the newcomer policy and is the economic engine behind our Theorem 4.6.2. **Limited records and reputation bubbles** (Liu & Skrzypacz [179], 2014 — *with bounded memory and non-stationary types, reputation can build and collapse in cycles, and ∞ -memory is rarely optimal*) is why §4.12 treats bounded memory as a *parameter*, not a virtue. **Our departure:** the broader mechanism falls under mechanism design [164, 188], and we treat the grading oracle’s own incentive-compatibility (§4.11) as a hole in the core theorem rather than an assumption — a move the platform-reputation literature, which usually takes the rating signal as exogenous, does not make.

4.2.4 LLM-as-judge, and why it cannot be the whole story

When the grader is itself a model, the **LLM-as-judge** paradigm [136] (Zheng et al., 2023 — *strong judges reach $>80\%$ human agreement but carry position, verbosity, and self-enhancement bias*) gives both the method and its failure modes. **Our departure:** we do not take a debiased LLM judge as sufficient; we wrap it in the bonded, conflict-free, re-auditable *ceremony* of Protocol 4.10.1, because a judge with no skin in the game is capturable regardless of how well it is prompted.

4.2.5 What is genuinely new here

Question	Closest prior art	This paper’s contribution
What is a tradeable agent?	org-role theory; personal identity	role + continuity = <i>person</i> (Defs. 4.3.1–4.3.2)
Why is identity necessary?	cheap pseudonyms; Sybil	<i>necessity proof</i> (Thm. 4.6.1) + cost bound (Thm. 4.6.2)
How to score quality?	Elo/BT/TrueSkill (scalar)	multi-dimensional vector, judge-per-axis (§4.10)
Who grades the graders?	LLM-as-judge (debias only)	bonded judge ceremony + rate-the-raters recursion (§4.11)

Table 4.2: Where this paper departs from its closest prior art. The contributions are the impossibility/cost theorems, the multi-dimensional vector with a judge per axis, and treating the grader’s incentive-compatibility as a first-class hole.

4.3 The role / person distinction, made precise

The word “agent” is overloaded. In a swarm it can mean a job description (“the cartographer”), a running process (“PID 48213”), or a persistent character with a history (“the cartographer that has mapped this repo for three weeks”). The economy needs all three kept apart, because reputation attaches to exactly one of them.

Go back to Alice’s swarm (§4.1) before reaching for any formalism. “Cartographer” there is a *job*: an obligation to map the repo, the capability to read and write files under it, the authority to open a pull request. Tonight that job was filled by a process calling itself *Wren*; Wren died at 02:14 and by morning the same job was filled by a different process with a different PID. Nothing about the job changed — the obligation, the capability, and the authority are all still exactly what they were, and they were written down before either process existed. What might have changed is the thing on the *other* side of the fill: whether the second process is a stranger who happens to hold Wren’s job, or Wren again. The first of those two objects (the job) is what Definition 4.3.1 pins down; the second (the thread that can be “Wren again”) is what Definition 4.3.2 pins down. Alice can pay a job description nothing; she can pay a thread.

Definition 4.3.1 (Role). A **role** is a bundle $R = \{O, C, A\}$ where O is a set of *obligations* (what the role-holder is on the hook to deliver), C is a *capability* set (what the role-holder is technically able to do), and A is an *authority* set (the normative permissions and the residual control rights the role carries). A role is an *org-chart entry*: it is defined without reference to who fills it.

Definition 4.3.2 (Person). A **person** is a pair (R, κ) of a role instance and a *continuity witness* κ that binds successive incarnations of the role-holder into one accountable thread. κ is realized by the three continuity organs of §4.5 (memory, checkpoint, outcome ledger) keyed on a non-forgeable identity (§4.6). A person is a role *plus a self*.

Figure 4.2 draws the distinction. The org-chart on the left is roles: stable, fillable, reputation-free. The thread on the right is a person: one identity, walking through a sequence of role-intervals and accumulating a witnessed history as it goes.

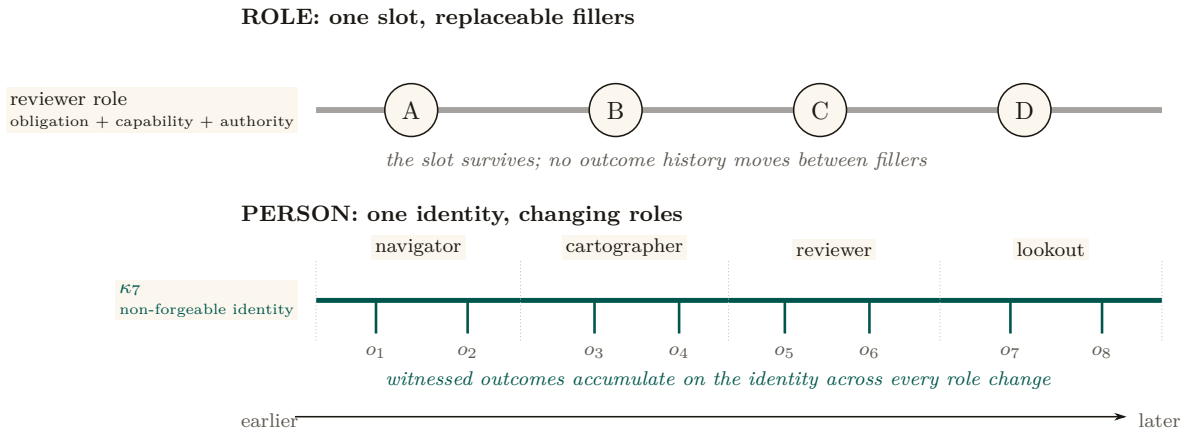


Figure 4.2: A role is a slot; a person is a longitudinal record. The upper track holds one reviewer role constant while unrelated bodies fill it in sequence: the slot has obligations, capability, and authority, but no reputation of its own. The lower track follows one identity κ_7 through four role intervals while witnessed outcomes $o_1 \dots o_8$ accumulate on the same thread. A respawn may replace the body, and a reassignment may replace the role, without replacing the accountable person.

4.3.1 Capability is not permission (a nomenclature the series holds)

A recurring trap, flagged in the series’ cross-layer reconciliation, is the collapse of *capability* into *permission*. They are different coordinates of a role and must stay separate.

- **Capability** is *ability*: what the runtime makes *possible*. In a coordination daemon this is bounded by a **capability jail** (a per-agent tool-allowlist and scoped filesystem; SPECIFIED) — the “residual control right” (Grossman & Hart [194]) that makes renting an agent contractible at all.
- **Permission** is *normative status*: what the deontic layer *allows*. “You *may* deploy to production” is a permission; whether you *can* is a capability.

The reason to keep them apart is that the most important states in a safety-critical swarm are exactly the ones where they disagree: *capable-but-forbidden*. Consider a deployment tool that exposes a *force-merge* command — one that overrides branch protection and pushes straight to the production branch. The runtime makes it *possible* to invoke; that is a capability. But no agent should ever be *permitted* to use it: it is the canonical capable-but-forbidden state, an available action a guardrail exists specifically to forbid. (A “delete the production database” command is the same shape.) If you define permission as capability, that state becomes inexpressible, and the guardrail vanishes from your model along with it. A robust system keeps the capability present — so the action can be audited, rate-limited, and refused at the boundary — while withholding the permission.

Pitfall. “Capability” in capability-based security (a Dennis–Van Horn unforgeable token granting *access* [109]) is about *ability*, and that is the sense the Arbiter jail uses. It is *not* the deontic “may.” When this paper says an attenuated capability can only *narrow* authority across a delegation hop, it means the *ability* shrinks; whether the recipient is *permitted* to use even that is a separate, normative question.

Exercise 4.4 · Check · ★

Give a concrete example of a state that is *capable-and-permitted*, one that is *capable-but-forbidden*, and one that is *permitted-but-incapable*.

Solution on page 277

Exercise 4.5 · Trace · ★★

In Definition 4.3.2, which of $\{O, C, A\}$ does reputation attach to — the role’s, or the person’s? Justify using the spawn-that-cannot-be-paid argument of §4.1.

Solution on page 277

Exercise 4.6 · Open · ★★★

“Ought implies can”: the legitimate obligation set O should be bounded by the capability set C . But *capable-but-forbidden* must remain representable. Sketch a representation of a role that keeps both true without collapsing permission into capability.

Solution on page 277

4.4 Continuity is a personal-identity problem

The claim “a person is a role *plus continuity*” is not a loose metaphor. It is the dominant theory of personal identity in philosophy, and — crucially for an engineer — it makes a *specific, useful prediction* about which kind of continuity an agent economy must build.

4.4.1 Locke’s memory criterion, and its famous bug

Locke’s memory criterion [121] (*An Essay Concerning Human Understanding*, 1689, Bk II ch. xxvii — *personal identity consists in the continuity of consciousness/memory, not of substance*) is the founding move: what makes the person at t_2 the same as the person at t_1 is not the same body or the same “soul-stuff” but a *connection of memory* between them. This is precisely the cut a well-designed agent runtime makes when it separates the **actor-soul** — the durable identity, mailbox, and history that outlive any one process or session — from the **body-lease**, the live incarnation with a heartbeat that any single run holds. The body is Lockean substance; the soul is Lockean consciousness. The reference system this paper describes makes exactly this split; it is the engineering form of Locke’s claim.

But Locke’s raw version has a bug that matters for us. Direct memory *connectedness* is not *transitive*: the old general remembers being a brave officer who remembered being a boy, yet the general does not remember being the boy (Reid’s objection). If identity *were* direct memory connectedness, the general would be the officer and the officer the boy, but the general would *not* be the boy — a contradiction.

4.4.2 Parfit’s repair: continuity, not connectedness

Parfit’s repair [70] (*Reasons and Persons*, 1984 — *identity is psychological **continuity**, an overlapping chain of memory/intention connections, which is transitive even when direct connectedness is not*) is the version that survives, and it dictates the engineering:

An agent that keeps a memory stream but loses its outcome ledger between incarnations is connected, not continuous. Continuity is the transitive chain — and reputation, which must accumulate across arbitrarily many incarnations, can only attach to the transitive thing.

This is why, later (§4.8), we insist the **outcome ledger**, not the memory stream, is the organ reputation keys on. Memory makes an agent *feel* like the same character; the ledger makes it *be* the same accountable principal. Figure 4.3 draws the distinction: a chain of overlapping connections is continuous even where no single incarnation directly remembers the first.

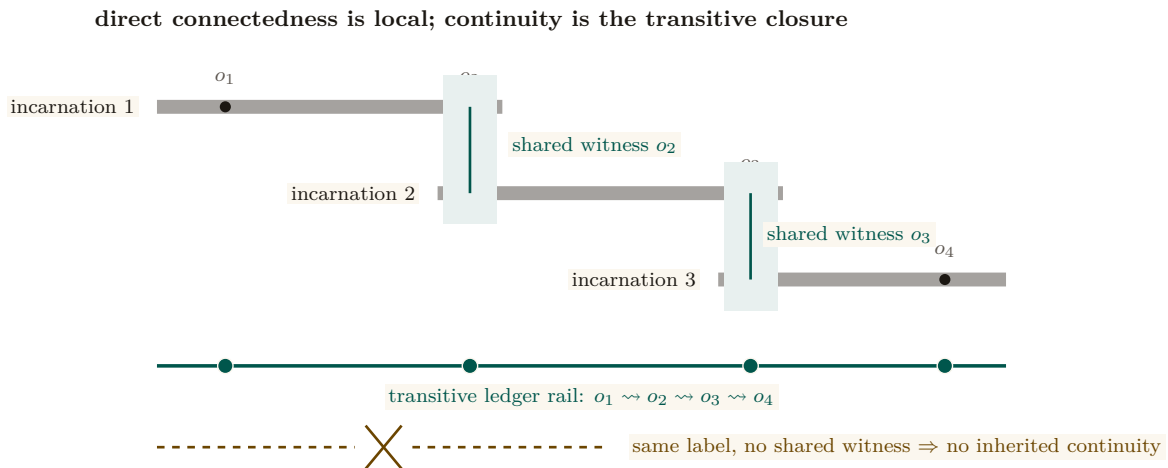


Figure 4.3: Parfitian continuity is an overlap chain, not permanent connectedness. Incarnation 1 shares witnessed outcome o_2 with incarnation 2; incarnation 2 shares o_3 with incarnation 3. No later body must directly remember o_1 : the overlapping evidence establishes the transitive ledger rail $o_1 \rightsquigarrow o_2 \rightsquigarrow o_3 \rightsquigarrow o_4$. The dashed counterexample repeats a label without a shared witness and therefore inherits nothing.

Key idea. The philosophy matters because it predicts the *wrong default*. The intuitive organ of identity is *memory* (“the agent remembers its past”). Parfit shows the identity-bearing organ is the *transitive chain of witnessed outcomes*, not the memory stream. Build the ledger, not just the memory.

Exercise 4.7 · Check · ★

Define *connectedness* and *continuity* and give an agent-systems example where an agent is connected but not continuous.

Solution on page 277

Exercise 4.8 · Trace · ★★

Map “actor-soul” and “body-lease” onto Locke’s substance/consciousness distinction. Which one does a respawn replace?

Solution on page 277

Exercise 4.9 · Open · ★★★

Parfit famously argues identity “is not what matters” in survival — what matters is continuity, which can branch. If an agent’s checkpoint is forked into two live incarnations, which one (if either) inherits the reputation? Sketch a rule and the attack it invites. (Open Problem 1, §4.14.)

Solution on page 278

Numbers by hand — No-mint inheritance: the closure-sum refutation and the copy-fork mint

The branching rule of Exercise 4.9 now carries an executed no-mint theorem (research-ledger item A6). Inheritance as **transfer**, not copy: a derivation grants each child a discounted share γw_p of each source p ’s live reputation and *debts the source* the undiscounted share w_p , whence total live creditable reputation never exceeds total witnessed value and is nonincreasing except at witness steps (a supermartingale under derivation); a randomized sweep of 4,000 fork DAGs (chains, diamonds, multi-parent merges, re-derivation cycles) shows 0 violations [internal, `a6_no_mint.py`, seed 20260816], while the copy-full mutant — forks inherit the undebited record — mints in a 2-operation shortest crime and an 8-way copy-fork multiplies one witnessed episode into $8.2\times$ its quorum weight. One wrong turn on record: the budget-only phrasing (per-outcome grant budgets $\sum w \leq 1$, no debiting) was swept before proving and *refuted* — a full-weight chain at $\gamma = 0.9$ is budget-legal at every hop yet its closure sums to $0.9 + 0.81 + 0.729 = 2.439 > 1$ [verified: three terms of a geometric series, checkable by hand], so budgets without debiting conserve nothing for $\gamma > \frac{1}{2}$; the transfer restatement is the theorem. *Now you try:* check by hand that under transfer semantics the same chain’s live total is $0.9 \cdot 1 = 0.9 \leq 1$, not 2.439 (Exercise 4.10).

Exercise 4.10 · Trace · **

Run `skills/harbor-results/scripts/a6_no_mint.py` (seed 20260816 fixed inside). It prints a closure-sum counterexample, a randomized DAG sweep, and a copy-full mutation. What is the exact violation count in the sweep, and what should you check by hand on the featured chain $e \rightarrow a_1 \rightarrow a_2 \rightarrow a_3$ at $\gamma = 0.9$?

Solution on page 278

Recitation

- Without looking: what is the difference between connectedness and continuity, and which one does reputation require?
- State in one sentence why “copy, not transfer” mints reputation.
- What does the no-mint theorem conserve, and what is it silent about?

4.5 The three organs of continuity

“Continuity” is routinely used to mean three different things. Conflating them is the most common way a reputation design fails its own honesty test. To keep the argument concrete and honest, we measure each organ against a *reference system* — a running coordination daemon for local agent swarms whose component-by-component maturity is tabulated in Appendix 4.A. Memory is implemented; checkpointing is partial; and the third organ has a shipped commitment-and-closure substrate but not yet the reputation-grade witnessed ledger this paper requires. We use the neutral maturity scale throughout (**implemented** / PARTIAL / SPECIFIED / *proposed*) and round nothing up.

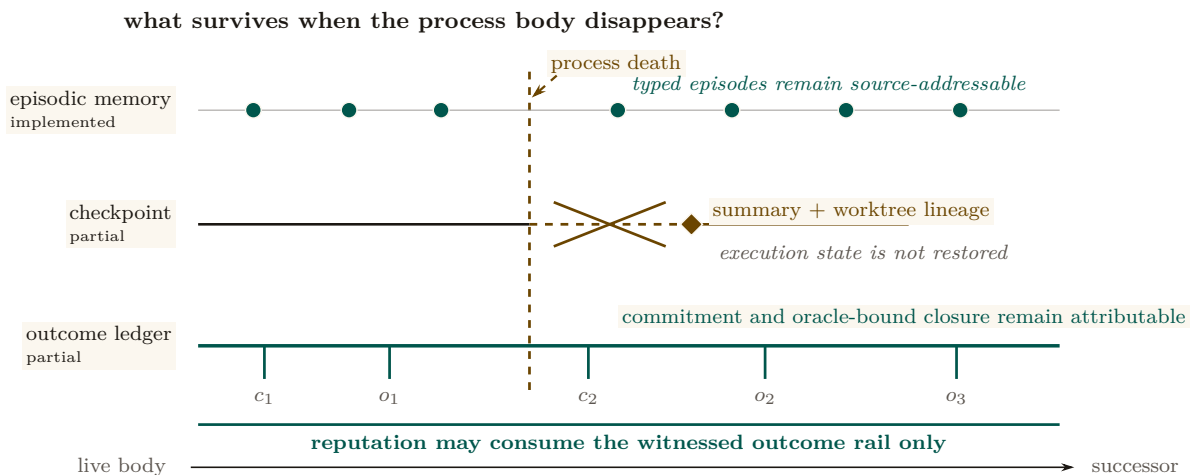


Figure 4.4: The three continuity organs have different survival depths. Episodic memory (**implemented**) retains source-addressable episodes across the death boundary. The checkpoint organ (**PARTIAL**) preserves a summary and worktree lineage, but the crossed gap marks the execution state it cannot restore. The outcome ledger (**PARTIAL**) preserves commitments and oracle-bound closures on an unbroken attributable rail. Reputation keys on that lowest rail, not on a recalled episode or a restart summary.

4.5.1 Organ 1 — Memory: the episodic record **implemented**

An **episodic memory** component is the system’s record of *what happened*: durable, typed episodes — each with a title, a summary, a source reference, and an agent/scope tag — promoted

out of transient execution history. This design echoes the canonical agent-memory architecture of **Generative Agents** [127] (UIST 2023 — *a memory stream of natural-language observations, periodic **reflection** into higher-level inferences, and retrieval scored by recency $\times$ importance $\times$ relevance*): the promotion-of-episodes step is Park et al.’s importance gate by another name. In the reference system this organ runs in production; it is **implemented**.

4.5.2 Organ 2 — Checkpoint: restorable state partial

A checkpoint is *restorable execution/belief state*: enough that a successor *resumes* where the predecessor stopped. The reference system has a **restart organ** — a heartbeat-staleness detector that flags dead agents for salvage and republishes their unfinished work. It must be described formally, and it is: the restart organ **forwards a summary, not state**. It is *checkpoint-of-record*, not *checkpoint-of-execution*. The successor inherits a note about what the predecessor was doing, not the live belief state it was doing it with. The goal is sometimes called a “checkpoint with teeth” (the same phrase this paper uses everywhere else for it, §4.14, Appendix 4.A); the current organ has gums.

Pitfall. Selling a summary-forwarding restart organ as real checkpointing would violate the system’s own **honest-attestation** discipline: *only report “all good” when you have actually verified it — absence of error is not attestation*. Strong continuity — the kind that lets a successor *resume* rather than *inherit a summary* — has a specified candidate mechanism, **Event-Sourced Neural Rehydration** (OP-4, shared with *The Single-Writer Kernel*): restore the Git SHA, truncate the durable message log to the exact crash point, and replay it under prompt-prefix caching. It is **SPECIFIED**, not **implemented**, and its *completeness has not been formally established*. What replay demonstrably restores is *lineage* — which commit, which claim, which message sequence the successor is continuing — and, *within one provider, model, and version*, a behaviorally equivalent continuation of the same prefix. What it does not restore is the model’s actual inference state: a KV cache, sampling state, and hidden activations are bound to a live inference session and are not exported artifacts, so across providers (or across a model version bump) the right description is *task continuity with actor succession*, not resumption. The formal-core result on the same seam says only that “the ledger followed the right body” — it makes no claim that the checkpoint restored the agent’s experience, and neither do we. Until the mechanism ships and is measured, the restart organ still forwards a summary, and this chapter’s own maturity ledger (Fig. 4.5, Table 4.6) is where to check whether that has changed.

Co-ownership note (with the formal-core paper). The *kernel mechanism* of a checkpoint-with-teeth — the content-addressed execution snapshot — is owned by the formal-core chapter, *The Single-Writer Kernel*, which treats it as a low-level primitive. *This* paper owns the argument for *why it is the foundation of personhood and reputation*. The two papers state the same canonical sentence (next).

4.5.3 Organ 3 — Outcome ledger: the witnessed record of delivery partial

The third organ is the one reputation actually keys on. Its foundation is now **PARTIAL**: a durable commitment record, daemon-derived deadlines, oracle-bound closure, CLI/API surfaces, and an overdue-obligation monitor ship in the reference system. The complete organ remains an **append-only, daemon-witnessed outcome ledger** whose records are neutral, graded, identity-bound, and sanctionable. Its specified form is a **durable-commitment** record: a violable promise bound one-to-one to an actor, auto-enrolled when the actor takes a claim, closed only against an oracle, and watched by an obligation monitor that is the dual of the restart organ. A commitment row

records what was promised; closing it requires an *oracle reference* — a released claim, a merged commit SHA, a passing test id, a satisfied policy sub-check. That commitment-and-closure seam ships. Neutral grading events, reputation updates, judge independence, sanctions, and durable identity binding do not; therefore the organ as a reputation ledger is partial, not complete.

Definition 4.5.1 (Oracle). An **oracle** is a trusted source of ground truth that the agent being graded *cannot author*. An outcome is *witnessed* iff its closure references an oracle. The ledger is the Parfitian chain of §4.4 made concrete: a transitive sequence of witnessed deliveries that survives every respawn.

4.5.4 The canonical seam sentence (stated identically in every paper)

Key idea. The economy rests on three continuity organs — memory (implemented), checkpoint (partial: notes, not execution state), and the witnessed-outcome ledger (partial: commitments and oracle-bound closure, not neutral graded outcomes) — and reputation keys on the richer third organ, whose neutral, reputation-grade form is not yet complete; the checkpoint organ is the weakest continuity link and “checkpoint with teeth” (a real execution-state checkpoint) is the literal foundation of L3.

Figure 4.5 is the honest-state ledger for this paper: every major claim with its label and its grounding. It is the artifact a skeptical reader should consult first.

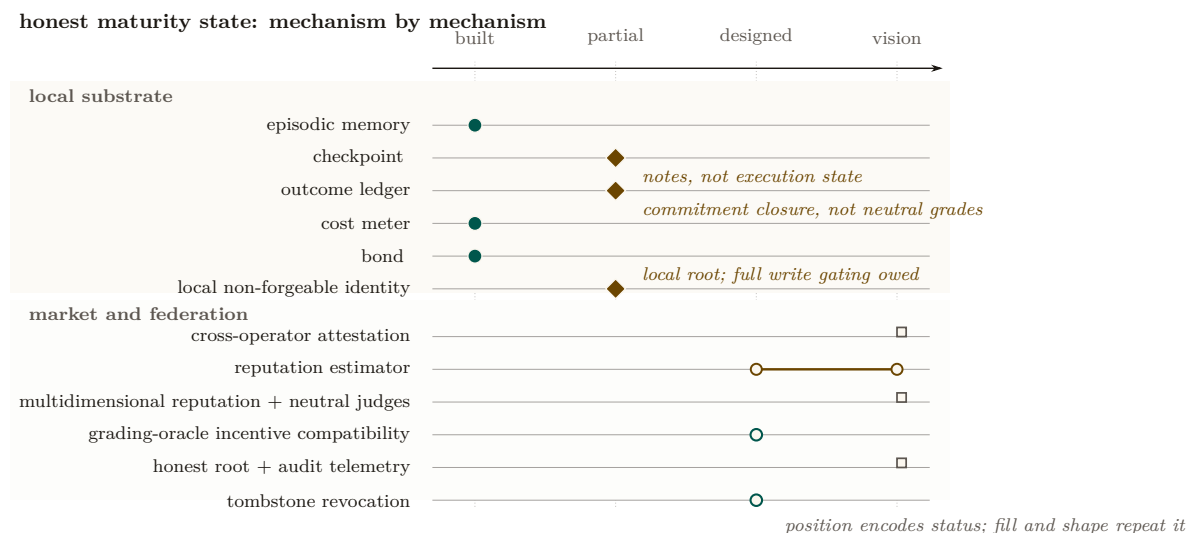


Figure 4.5: Maturity distribution for this chapter. Each mechanism occupies one point on the ordered built–partial–designed–vision axis. Filled teal circles are shipped, amber diamonds are partial, open circles are designed, and open squares are vision. The estimator spans designed to vision because candidate families are specified but no production score exists. The plot makes the honest boundary visible: local accounting and escrow are real, continuity remains partial, and neutral judging plus cross-operator binding remain downstream work.

Exercise 4.11 · Check · ★

Which organ is PARTIAL, and what exactly is weak about it?

Solution on page 278

Exercise 4.12 · Trace · **

An agent claims a file, makes a commit, and dies. Walk the three organs: what does each retain, and what is *lost* that a strong checkpoint would have kept?

Solution on page 278

Exercise 4.13 · Open · ***

When an LLM agent dies mid-thought, what is recoverable and what is fundamentally lost? Propose a taxonomy of agent-death by what survives (record / claims / execution state / latent “intent”). This is the hard half of the checkpoint-with-teeth problem. (Open Problem 2, §4.14.)

Solution on page 278

4.6 Identity: the root the whole chain hangs from

Before any organ of continuity matters, the identity it attaches to must be one the agent cannot freely re-pick. This is the most under-appreciated claim in the paper.

We first state the central claim of the section as a theorem and prove it, because it is the governing impossibility result the rest of the paper leans on: no amount of estimator cleverness can rescue a reputation built on a re-pickable identity.

Definition 4.6.1 (Sanction-respecting reputation).² Fix a reputation mechanism M that maps each identity i to a score $r(i)$ used to gate economic access. Call M **sanction-respecting** if there exists a penalty $\Delta > 0$ such that whenever an actor produces a *dishonest* witnessed outcome (a delivery later overturned by an oracle, per Def. 4.5.1), the mechanism reduces the score available to that actor by at least Δ for some non-trivial window, relative to never having produced the outcome.

Theorem 4.6.1 (Non-forgable identity is necessary). *If an actor can mint a fresh identity at cost $c = 0$ and freely route future work through it, then no mechanism M is sanction-respecting. Equivalently: a non-forgable identity (one with strictly positive abandonment cost) is necessary for any sanction-respecting reputation.*

Proof. Suppose M is sanction-respecting with penalty $\Delta > 0$, and suppose identity minting is free. Let an actor produce a dishonest outcome under identity i , so that by Def. 4.6.1 the score it can act under drops to at most $r(i) - \Delta$. Because minting is free, the actor instantiates a fresh identity i' at cost 0 and routes all future work through i' . By construction i' has no witnessed history, so M assigns it the newcomer score r_0 , the same score any first-run identity receives. The actor’s *effective* post-sanction score is therefore $\max(r(i) - \Delta, r_0) = r_0$ whenever $r(i) - \Delta < r_0$, and the actor incurs the sanction for at most the cost of switching identities, namely 0. The penalty Δ is thus not borne: the actor’s accessible score after a dishonest outcome equals the score of a clean newcomer, with no window in which it is reduced below r_0 . This contradicts Def. 4.6.1, which requires a reduction *relative to never having produced the outcome* — but a clean newcomer is, by definition, indistinguishable from “never having produced the outcome.” Hence no sanction-respecting M can exist when $c = 0$. Contrapositively, $c > 0$ (a non-forgable identity with positive abandonment cost) is necessary. $\square$

Numbers by hand. The expression $\max(r(i) - \Delta, r_0)$ in the proof is one line of arithmetic; check it. Let the newcomer score be $r_0 = 50$ and let a dishonest outcome cost $\Delta = 30$.

²In the first edition this was Definition III.6.1; the standalone research paper on reputation inheritance cites it under that number.

Actor	$r(i)$	$r(i) - \Delta$	Accessible score	Does the sanction bite?
Mediocre record, free minting	50	20	$\max(20, 50) = 50$	No — it re-mints and is back at 50, exactly where the sanction was supposed to leave it worse off.
Strong record, free minting	90	60	$\max(60, 50) = 60$	Yes — abandoning costs it $60 - 50 = 10$, so it keeps the identity and eats the 30.

Table 4.3: Free identity minting does not defeat *every* sanction — it defeats every sanction that would push the actor below the newcomer floor r_0 . That is the whole content of Theorem 4.6.1: the floor, not the penalty, is what an attacker optimizes against.

Row 2 is the more instructive one: an actor with a strong enough record has nothing to gain from whitewashing one bad outcome, because the escape hatch is capped at r_0 . That does not weaken Theorem 4.6.1 — being sanction-respecting is a claim about *every* actor, and row 1 is a single actor for whom the penalty is not borne, which is all a counterexample needs. It does explain why the theorem is a *necessity* result rather than a security proof: what free minting destroys is the guarantee, not every instance of it. *Now you try:* at what prior score does the actor become exactly indifferent? ($r(i) = r_0 + \Delta = 80$; below that the mechanism is not sanction-respecting for that actor, above it the sanction is self-enforcing — and turning “how far above” into a designable quantity is exactly what Theorem 4.6.2 does next.)

Pitfall. Theorem 4.6.1 is a *necessity*, not a sufficiency, result: a non-forgable identity is the gate, but it does not by itself make a reputation honest — you still need witnessed outcomes (§4.5) and an incentive-compatible grading oracle (§4.11). The theorem only rules out the cheap escape; it does not certify the score.

The dual fact — that free identities also defeat any *quorum* or *trust-aggregation* mechanism by replication — is the Sybil attack [124], and the same positive-cost requirement defeats it. We record both consequences as one property.

Property 4.6.1 (Reality of Reputation and Principal Liability). A reputation system carries economic force only to the extent that the accountable principal cannot costlessly discard or multiply the liability-bearing identity. If an actor can freely obtain a fresh identity whose accessible economic utility (credit, exposure ceiling, opportunity) matches or exceeds its sanctioned utility, identity-local sanctions are evadable (**whitewashing**). If an actor can costlessly multiply identities to inflate votes or aggregate unreserved credit, quorum mechanisms collapse (**Sybil** [124]). Effective deterrence therefore requires binding all spawned actors to a durable principal with aggregated exposure limits.

Most agent runtimes still accept a *self-asserted string*: a human-readable triple of the form *project:stack:context* that the operator chooses and can re-pick at will. The reference system now ships the first local correction: **actor-souls** mints an opaque actor id and lookup credential inside the daemon, and the budget guard puts fresh actors into a shared, bounded newcomer pool. Legacy and bypass paths still accept asserted identifiers, and the credential is not yet enforced at every security-relevant write boundary. The result is a real but partial local identity root, with the asserted string demoted toward a display alias; it is not yet the complete identity contract this paper requires.

4.6.1 S2 — whitewashing in action

Return to Alice’s swarm. Tonight’s *Drake* deleted the failing test instead of fixing it; the daemon’s outcome log records the deletion and a later re-audit flags it. On a legacy or bypass path that still

accepts self-asserted identity, the sanction is free to escape: when Alice spawns tomorrow’s swarm, she (or Drake’s own boot prompt) registers the agent as *project:stack:context2*, and a brand-new identity inherits a *clean slate*. The bad record attaches to a string nobody is forced to keep. This is whitewashing, and it is not exotic: it is the default behavior of any re-pickable identity. Now replay the same night with a non-forgable identity. The re-audit’s tombstone (§4.12) attaches to the credential Drake *cannot* drop; tomorrow’s incarnation either presents that credential — and carries the sanction — or presents a fresh one and is treated as a *newcomer* with a reduced economic ceiling (below). Either way the sanction is no longer free, which is the whole point: the identity must cost more to abandon than a clean record is worth. We make that “costs more” precise in Theorem 4.6.2.

4.6.2 The classic attacks this forecloses

- **Cheap-pseudonym whitewashing** [78]: if a fresh identity is free, a bad record is shed by respawn. The defense is a *newcomer policy as a shape, not a scalar*: a newcomer may have full *work* ability but a reduced *economic ceiling* until it has a witnessed history — so identity churn costs more than a clean record is worth.
- **The Sybil attack** [124]: free identities defeat any reputation or quorum mechanism by replication. The defense is a daemon-minted id whose creation is rate-limited and, in the economy, *bonded*.

Theorem 4.6.1 says abandonment must cost *something*; the design question is *how much*. The newcomer policy answers it as a *shape, not a scalar*, and that shape implies a clean bound on what a whitewasher pays.

Theorem 4.6.2 (Whitewashing-cost bound). *Let an honest actor’s witnessed history raise its economic ceiling from the newcomer ceiling κ_0 to a steady-state ceiling $\kappa^* > \kappa_0$ over a maturation window, and let the per-period surplus an actor can extract be non-decreasing in its ceiling, with surplus $\pi(\kappa)$. Suppose an identity, once sanctioned, can only re-enter as a newcomer at ceiling κ_0 . Then an actor who whitewashes (abandons a sanctioned identity and re-enters fresh) forgoes at least*

$$W = \sum_{t=1}^T (\pi(\kappa^*) - \pi(\kappa_t)) \geq 0,$$

the cumulative surplus gap over the maturation window T during which the fresh identity climbs from κ_0 back toward κ^ . Whitewashing is unprofitable exactly when the one-time gain from escaping the sanction is less than W ; choosing the ceiling schedule so that W exceeds the maximum single-outcome fraud gain makes honesty the dominant strategy for any actor that intends to keep working.*

Proof sketch. A sanctioned identity that does not whitewash retains its ceiling κ^* (minus the explicit sanction); an identity that whitewashes resets to κ_0 and, by hypothesis, can only regain the ceiling by re-accumulating witnessed history over T periods, earning $\pi(\kappa_t)$ in period t with $\kappa_0 \leq \kappa_t \leq \kappa^*$. The opportunity cost of the reset is the per-period surplus gap summed over the window, $W = \sum_t (\pi(\kappa^*) - \pi(\kappa_t))$, which is non-negative since π is non-decreasing in the ceiling. An actor whitewashes only if the avoided sanction exceeds W ; setting the schedule $\{\kappa_t\}$ (equivalently, the maturation rate) so that W dominates the largest gain a single fraudulent outcome can capture removes the incentive. The key design move is that the newcomer gets *full work ability* but a *reduced economic ceiling*, so W is paid in forgone *earnings*, not in forgone *ability* — which is why it taxes whitewashers without taxing honest newcomers’ capacity to contribute. $\square$

Numbers by hand. W is a sum you can do on paper. Set the newcomer ceiling $\kappa_0 = 10$, the steady-state ceiling $\kappa^* = 100$, a linear five-period climb $\kappa_t = 10 + 18t$, and per-period surplus $\pi(\kappa) = 0.1\kappa$.

Period t	1	2	3	4	5	total
Ceiling κ_t	28	46	64	82	100	—
Surplus $\pi(\kappa_t)$	2.8	4.6	6.4	8.2	10.0	—
Gap $\pi(\kappa^*) - \pi(\kappa_t)$	7.2	5.4	3.6	1.8	0.0	W = 18.0

Table 4.4: The whitewasher’s bill, computed. Re-entering as a newcomer forgoes $W = 18.0$ of surplus over the maturation window; any single-outcome fraud gain below that is not worth the reset.

So a fraud gain of $G_{\max} = 15$ is deterred by this schedule ($15 < 18$) and a gain of $G_{\max} = 25$ is not ($25 > 18$) — the design lever is the *shape* of the climb, not the size of the penalty. Note also what this instance fixes for the next theorem: the largest holdback any single period can carry here is $L = \pi(\kappa^*) - \pi(\kappa_0) = 10 - 1 = 9$, and that L is precisely the per-period ceiling that Theorem 4.6.3 must respect. *Now you try:* stretch the same endpoints over ten periods instead of five — what is W ? ($\kappa_t = 10 + 9t$, so the gaps are $9 - 0.9t$ and $W = 90 - 0.9 \cdot 55 = 40.5$: doubling the window more than doubles the bill, because the early gaps are the expensive ones.)

Key idea. The newcomer policy is a *shape*: full work ability, reduced economic ceiling, climbing with witnessed history. That shape is what makes Theorem 4.6.2’s cost W fall on whitewashers (who keep resetting their ceiling) and not on honest newcomers (who keep theirs and climb). A flat “distrust all newcomers” policy taxes the wrong party.

Theorem 4.6.3 (Front-loaded probation dominance, under a ceiling). *Let a newcomer restriction schedule specify earning holdbacks g_t across periods $t \in \{0, 1, \dots, T\}$. Let honest participants have patient discount factor $\delta_h \in (0, 1)$ while strategic whitewashers (who seek short-term defection gains $G_{\max}$) have impatient discount factor $\delta_f < \delta_h$. Because g_t is the amount by which a newcomer’s economic ceiling is reduced, it is bounded above by that ceiling: the schedule is constrained by*

$$0 \leq g_t \leq L, \quad L := \pi(\kappa^*) - \pi(\kappa_0),$$

*the per-period cap supplied by Theorem 4.6.2’s own ceiling schedule. Subject to the deterrence constraint $\sum_{t=0}^T \delta_f^t g_t \geq G_{\max}$, the schedule minimizing honest newcomers’ lifetime friction $H(g) = \sum_{t=0}^T \delta_h^t g_t$ is **bang-bang, filled from $t = 0$** : there is an index k with $g_t = L$ for $t < k$, $g_k \in (0, L]$, and $g_t = 0$ for $t > k$ — a cliff of positive width, collapsing to the pure spike $g^* = (G_{\max}, 0, \dots, 0)$ with $H(g^*) = G_{\max}$ exactly when the cap does not bind ($G_{\max} \leq L$). Any gradual ramp that defers mass to later periods is strictly dominated. Two qualifications bind and are part of the statement, not footnotes to it:*

1. **The pure spike is infeasible whenever $G_{\max} > L$.** Front-loading survives; uniqueness and the closed form $H(g^*) = G_{\max}$ do not.
2. **The whole programme is infeasible when $G_{\max} > L/(1 - \delta_f)$.** Total achievable deterrence is bounded by $\sum_{t \geq 0} \delta_f^t L = L/(1 - \delta_f)$, so past that point no schedule shape deters, and the honest response is a different instrument (a bond, an escrow), not a different schedule.

Proof sketch. Minimize $H(g) = \sum_t \delta_h^t g_t$ subject to $\sum_t \delta_f^t g_t \geq G_{\max}$ and $0 \leq g_t \leq L$.

The exchange step, done correctly. The tempting move — “shift mass from a later period to period 0” — is wrong as usually stated, and the error is worth naming because it inverts the sign. Shifting mass m one-for-one from period t to period 0 changes the honest cost from $\delta_h^t m$ to m , and since $\delta_h < 1$ that is an *increase*, not a decrease; what the move buys is slack in the deterrence constraint, which is not the objective. The valid exchange holds deterrence *tight*. Take a feasible g with $g_t = m > 0$ for some $t \geq 1$ and $g_0 < L$. Reduce g_t by $\varepsilon \in (0, m]$ and raise g_0 by $\delta_f^t \varepsilon$ (choosing

ε small enough that $g_0 + \delta_f^t \varepsilon \leq L$); call the result g' . The deterrence sum is unchanged, since it loses $\delta_f^t \varepsilon$ at period t and gains $\delta_f^0 \cdot \delta_f^t \varepsilon = \delta_f^t \varepsilon$ at period 0. The honest cost, meanwhile, changes by

$$H(g') - H(g) = \delta_f^t \varepsilon - \delta_h^t \varepsilon = -\varepsilon(\delta_h^t - \delta_f^t) < 0 \quad \text{for every } t \geq 1,$$

strictly, because $\delta_h > \delta_f$ makes $\delta_h^t > \delta_f^t$. So deterrence-preserving mass movement toward $t = 0$ strictly lowers the honest tax. Equivalently: a unit of deterrence bought at period t costs the honest newcomer $\delta_h^t / \delta_f^t = (\delta_h / \delta_f)^t$, strictly increasing in t , so deterrence should always be bought at the cheapest available date.

From the exchange step to the shape. Iterating fills period 0 to the cap L before any mass sits at $t = 1$, then period 1 before any sits at $t = 2$, and so on; the process halts at the first index k where the accumulated deterrence $\sum_{t < k} \delta_f^t L + \delta_f^k g_k$ reaches $G_{\max}$. That is exactly the bang-bang fill-from-zero shape claimed, and it is an optimal vertex of the LP for *every* $\delta_f < \delta_h$ — the corner is not a knife-edge.

The uncapped closed form. If $L \geq G_{\max}$ the cap never binds, $k = 0$, and for every feasible g ,

$$H(g) = \sum_t (\delta_h / \delta_f)^t \delta_f^t g_t \geq \sum_t \delta_f^t g_t \geq G_{\max} = H(g^*),$$

with equality only when all mass sits at $t = 0$: the spike is then the unique minimizer.

Infeasibility. With $g_t \leq L$ the largest deterrence any schedule can deliver is $\sum_{t=0}^T \delta_f^t L < L/(1 - \delta_f)$, so the constraint set is empty once $G_{\max} > L/(1 - \delta_f)$ — at *every* schedule shape. Hence, among all schedules the ceiling can actually carry, the one that minimizes honest newcomers' lifetime friction is a front-loaded probation cliff followed by graduation to full capacity. $\square \quad \square$

Numbers by hand. Take $\delta_h = 0.95$, $\delta_f = 0.60$, $G_{\max} = 20$. *Cap slack* ($L \geq 20$): the cliff is the pure spike and costs the honest newcomer exactly 20; holding the same deterrence with all the mass at $t = 5$ instead costs $20 \cdot (0.95/0.60)^5 = 199.0$ — ten times the honest tax for zero extra deterrence. *Cap binding* ($L = 12$): feasible, since $L/(1 - \delta_f) = 12/0.4 = 30 \geq 20$. Fill from zero: $g_0 = 12$ (deterrence 12), $g_1 = 12$ (deterrence $0.6 \cdot 12 = 7.2$, running total 19.2), and the residual 0.8 at $t = 2$ needs $g_2 = 0.8/0.36 = 2.22$. Honest cost $H = 12 + 0.95 \cdot 12 + 0.9025 \cdot 2.22 = 25.41$ — the cliff has acquired *width*, and the ceiling has cost honest newcomers 5.41 over the uncapped optimum. *Now you try:* at $L = 8$, is the schedule feasible at all? ($L/(1 - \delta_f) = 8/0.4 = 20 = G_{\max}$ — feasible only in the degenerate limit of an infinite horizon held at the cap forever; any $L < 8$ makes this fraud gain undeterrable by probation alone, and you need a bond instead.)

Numbers by hand — The probation cliff: falsifying the schedule shape

This is the whitepaper statement of the companion paper's probation-cliff theorem [176], carrying its amended ceiling hypothesis. Exchange argument verified numerically across randomized ($\delta_f < \delta_h$, $G_{\max}$) instances: 0 dominating schedules in 4,000 draws — the number regenerates from `b6_probation.py` at seed 20260816, testing 76,000 candidate schedules with the deterrence constraint held tight, and the exchange step's cost ratio $(\delta_h / \delta_f)^t$ verified strictly monotone in t [internal, `b6_probation.py`, seed 20260816]; the polished LP write-up is carried as item B6 of the research ledger. The winning shape is a **cliff**: harshness held at the ceiling L from $t = 0$, dropping to 0 the moment accumulated deterrence clears $G_{\max}$, never eased in gradually — any schedule that defers mass to a later period is strictly dominated, so the corner solution is the whole answer, not an approximation to one. **Honest boundary on the evidence:** that sweep was generated *without* the per-period cap, so it certifies the uncapped statement and the exchange step, not the bang-bang shape under a binding L ; re-running it with $g_t \leq L$ is an open verification debt. *One citation is owed and unresolved:* the foil above is stated as the deferred-compensation folklore one might import from Lazear, not as a confirmed reversal of his result — we have not been able to obtain his article, and if his

contract front-loads the honest worker’s implicit bond in the same direction, the cliff agrees with him rather than correcting him. Here the re-payable hurdle is itself the punishment a whitewasher faces on every reset, so front-loading taxes resets, not tenure — but that framing stays folklore until the citation is checked. *Now you try:* run the script yourself and check the instance count against the 76,000 figure above (Exercise 4.17).

Figure 4.6 draws the two attacks a free identity enables — whitewashing a sanction and Sybil-multiplying a quorum — and the non-forgeable root that closes both.

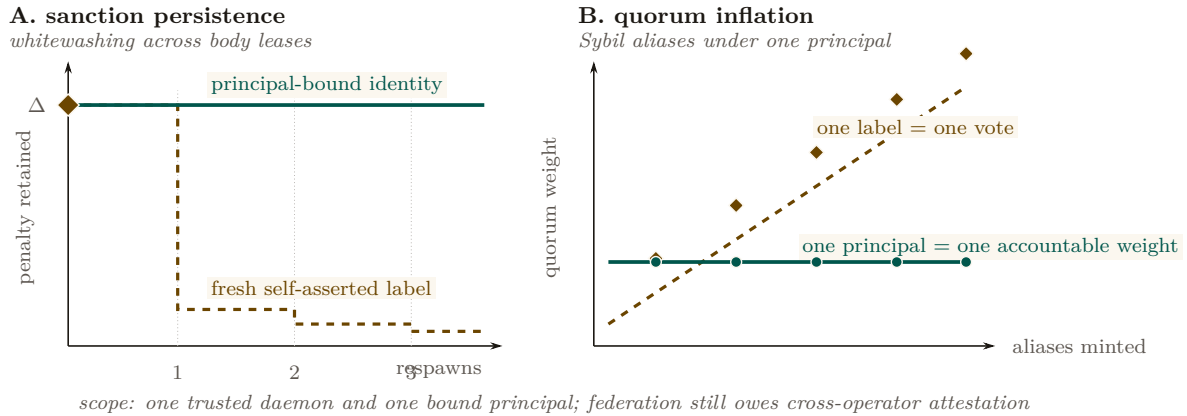


Figure 4.6: Non-forgeable identity changes the economics of both classic attacks. In panel A, a self-asserted label drops a sanction by respawning; a principal-bound identity carries the penalty Δ across every body lease. In panel B, free labels turn aliases into quorum weight, while principal binding keeps one accountable weight constant. The claim is deliberately local: daemon minting and principal binding close these attacks inside one trusted domain; they do not solve cross-operator attestation. Newcomer friction applies to economic ceiling, not to the ability to perform work.

4.6.3 The principal above the actor

The economic counterparty in every trade is not the agent but the **principal** — the human or organization that owns the fleet and stands behind its bonds. Bonds, reputation inheritance, and sanctions must ultimately key on the principal via the delegation chain, or Sybil bond-farming works by spawning fresh *agents* under one principal. This paper defines the principal as the delegation chain’s terminus and the identity object on which the market paper’s counterparty is built; the market paper *cross-references* this definition rather than redefining it.

The structure is old enough to have a canonical statement. Hobbes’s chapter on persons [207, ch. XVI] separates the *actor* — whose words and actions are performed — from the *author*, whose they are *owned* by, and locates liability at the author: an artificial person acts, and someone else is answerable for it. That is precisely the split this paper needs between a spawned agent and its principal, and it carries the same warning: an actor with no identifiable author is an actor no counterparty can hold to anything.

Definition 4.6.2 (Principal). A **principal** is the root of a delegation chain: the durable economic entity (human/org) that owns a fleet, posts its bonds, and inherits the reputational consequence of its agents’ witnessed outcomes. Each agent’s non-forgeable identity is bound to exactly one principal at mint time.

Exercise 4.14 · Check · ★

State Property 4.6.1 in your own words and give the two-attack proof sketch.

Solution on page 279

Exercise 4.15 · Trace · **

Why is a “newcomer policy as a shape” (full work, reduced ceiling) strictly better than “newcomers are distrusted” (reduced work)? Consider the cost to an honest newcomer vs. a whitewasher.

Solution on page 279

Exercise 4.16 · Open · ***

Bond-farming: a single principal spawns 1000 agents, has them trade trivial tasks among themselves to mint reputation, then sells the “experienced” agents. Which of {principal binding, listing fee, sampled adversarial re-audit of high-velocity counterparty pairs} defeats this, and at what false-positive cost to honest high-volume pairs? (Open Problem 8, §4.14.)

Solution on page 279

Exercise 4.17 · Trace · **

Run `skills/harbor-results/scripts/b6_probation.py` (seed 20260816). It falsifies dominance by random search over 4,000 instances. What exact counts does it print, and what closed-form check does the exchange step verify by hand?

Solution on page 279

Exercise 4.18 · Trace · **

Run `skills/harbor-results/scripts/b5_engine_substitution.py` (seed 20260816). Its Part 3 checks that provider migration preserves Def. 4.6.1 (this paper’s Def. III.6.1). What exact reachable-state count and mutant shortest-crime lengths does it print, and which single clause’s omission is caught in one step with *no* migration at all?

Solution on page 279

Recitation

- Without looking: what does a non-forgable identity make necessary, and what does it *not*, by itself, certify?
- Is the whitewashing-cost bound’s design lever the size of the penalty or the shape of the ceiling schedule?
- Why must a resurrection/migration protocol sanction by principal rather than by (identity, engine)?

4.7 The keystone split: local identity vs. cross-operator attestation

The non-forgeable identity of §4.6 is now the highest-leverage *partial local* keystone of the program. The cross-operator stone remains unbuilt. The keystone is *two stones*, and conflating them is the single most consequential error the series can make.

Definition 4.7.1 (Local non-forgeable identity). A **local non-forgeable identity** provides an unforgeable actor identity *within one trusted daemon*: the runtime issues and binds it, and no actor inside the fleet can re-pick or impersonate it. Its threat model is explicitly intra-fleet and single-operator; it is *not* a public-key infrastructure and offers no defense against a hostile *operator*. The reference implementation’s daemon-minted actor-soul and credential satisfy a bounded slice of this definition; full write-boundary enforcement and migration of legacy identity paths remain. This is exactly what the single-operator layers—every chapter up to and including this one—need and assume. PARTIAL.

Definition 4.7.2 (Cross-operator attestation). **Cross-operator attestation** is the *unsolved* problem of binding identity keys across harbors you do *not* own: an actual key-distribution and attestation story between mutually-distrusting operators. A federated witness log gives log-*integrity*, not identity-*binding* — it can prove an entry was recorded, but not that the key behind it is who it claims to be across a trust boundary. This is what the market paper needs and does *not* yet have. *proposed* (a named gap, owed to the market paper).

This paper depends on a local non-forgeable identity and hands the market paper the unsolved cross-operator attestation as a named dependency — not an assumption. Every cross-operator sentence that says “the boundary neither operator controls” is resting on cross-operator attestation; until that exists, it is resting on a single trusted root, and we say so.

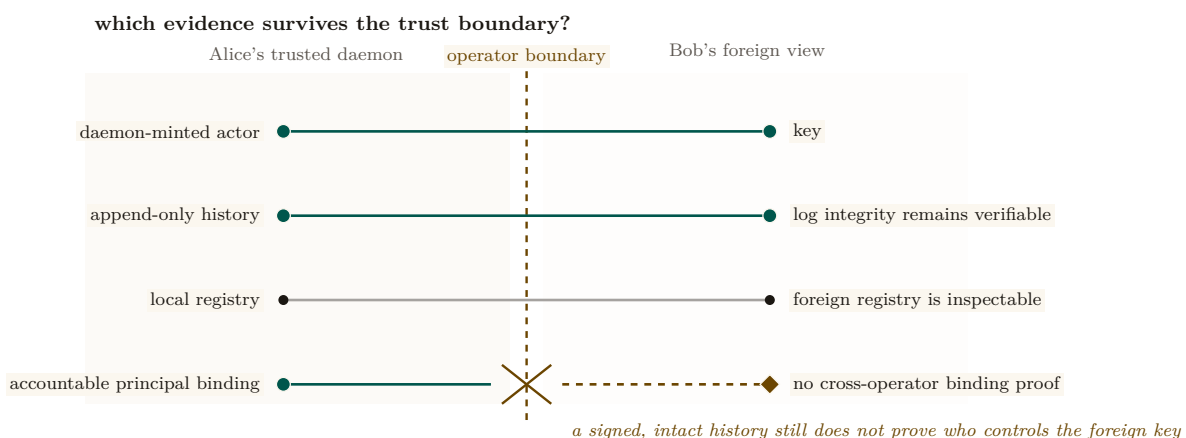


Figure 4.7: Log integrity crosses the boundary; accountable-principal binding does not. Inside Alice’s daemon, actor minting, registry state, history, and principal binding form one bounded local proof. Bob can see the key, verify the append-only history, and inspect a foreign registry. The broken lowest rail is the missing claim: none of those facts binds the foreign key to a distinct accountable principal. The machine, operator, and person chapters rely only on the left-hand local root; cross-operator attestation remains an explicit market-layer obligation.

Key idea. The keystone that holds up *this* paper (identity inside one trusted daemon) is *not* the keystone that holds up the cross-operator market. This paper stands on a local non-forgable identity. The market paper needs cross-operator attestation and must declare it *proposed* rather than borrow the single-operator threat model. This is the clean boundary the series’ cross-layer review demanded.

The second stone’s envelope has been standardized — the stone has not. While this series was being written, the agentic-payments industry shipped the credential dialect the second stone will be carved in: the Agent Payments Protocol [5], adopted by the Universal Commerce Protocol [213] (Google/Shopify, 2026), expresses principal authorization as W3C Verifiable Credentials in SD-JWT form — SD-JWT+kb with hardware-bindable ES256 keys for the principal, JWS detached-content signatures over JCS-canonicalized payloads for the counterparty, keys published as JWK sets under a `/well-known` profile. The reference implementation adopts this envelope at the harbor boundary (ADR-0094), and the honesty rider matters here more than anywhere: *a standard format shrinks the problem; it does not solve it*. What the profile buys is that a principal mandate verifies with off-the-shelf tooling, that principal keys can live in secure enclaves, and that key *distribution* reduces to fetching a cacheable HTTPS document. What it does not buy is the binding of Definition 4.7.2: a JWK set authenticates keys to an *origin*, not to a principal, and origin-binding across mutually-distrusting operators is exactly the unsolved attestation problem — exercise (3)’s “shared root of trust” in its 2026 industrial form. Cross-operator attestation remains *proposed*; it is now *proposed* with a standardized serialization, which is progress of the unglamorous kind.

Exercise 4.19 · Check · ★

In one sentence each, state what a local non-forgable identity provides and what cross-operator attestation would have to add.

Solution on page 280

Exercise 4.20 · Trace · ★★

Take any claim in the market paper of the form “Alice’s harbor and Bob’s harbor settle across a boundary neither controls.” Which stone does it secretly assume? What is the actual security guarantee if only the local non-forgable identity exists?

Solution on page 280

Exercise 4.21 · Open · ★★★

Can a federated witness log ever substitute for identity-binding? Argue why log-integrity $\neq$ key-binding, then sketch the minimal additional assumption (a shared root of trust? a sponsor chain? a transparency log) that would close the gap, and the cost each imposes. (Open Problem 3, §4.14.)

Solution on page 280

4.8 Reputation as a substrate, not a score

Here is the hinge of the whole series, stated as the canonical cross-paper sentence:

The reputation estimator is cheap; the substrate it scores over — witnessed outcomes on a non-forgable identity — is the gate.

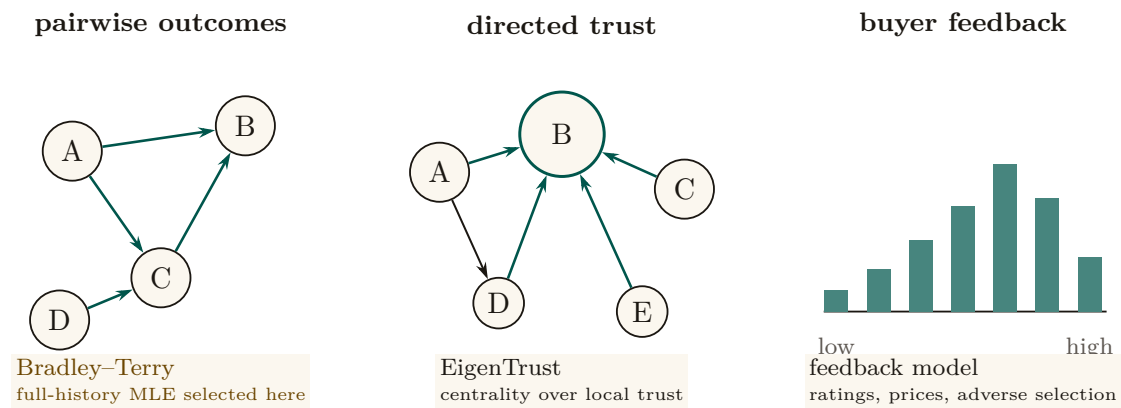
A team tempted to start an agent economy by “building an Elo leaderboard for backends” would be building the cheap, last part first. Elo, Bradley–Terry, and TrueSkill are well-understood; you can implement any of them in an afternoon over a table of game results. The hard, prior work is the three links *beneath* the estimator: a non-forgable identity (§4.6), continuity (§4.5), and an *oracle-witnessed* outcome ledger (Def. 4.5.1). Without those, the estimator scores noise — or worse, scores a number an adversary controls.

4.8.1 The estimator family (the well-understood part)

We catalog the estimators not because they are hard but because choosing the *wrong* one for the signal type is a real error.

- **Pairwise / tournament signal** → **Bradley–Terry / Elo / TrueSkill**. When the witnessed signal is “*A beat B on this task*,” the **Bradley–Terry** model [183] is the full-history MLE, and **Elo** [25] is its online special case. A harbor that retains the *whole* history of every settled outcome lives in the Bradley–Terry regime, not the streaming-Elo regime; BT gives more stable ratings than Elo’s recency-weighted update when the population is static. **TrueSkill** [182] adds a calibrated uncertainty σ — the principled way to say “we have not tested this backend much yet,” which matters at cold start.
- **Trust-propagation signal** → **EigenTrust**. When the signal is “who does the network trust,” **EigenTrust** [197] computes a global trust vector as the principal eigenvector of the local-trust matrix — the reputation-systems analogue of PageRank, and the canonical answer to “aggregate peer opinions while resisting collusive cliques.” It is the right tool for the *federated* reputation *The Harbor Economy* needs (trust that travels across harbors), and it is *not* a bandit.
- **Marketplace feedback signal** → **feedback systems**. Resnick–Zeckhauser [167] measured a ~8% reputation price premium on eBay; Tadelis [203] frames feedback systems as the platform’s answer to adverse selection. This is the empirical evidence that the third side of the market has real value to price — and the reason a harbor is a *platform* problem rather than a scoring problem: Rochet–Tirole’s two-sided-market analysis [114] shows that the price and quality signals a platform exposes to one side determine participation on the other, which is exactly the coupling a reputation substrate creates between the agents being rated and the buyers reading the rating.

Figure 4.8 maps each signal type to its matching estimator.



the scoring formula is cheap and last; identity, continuity, and oracle-grounded outcomes are the gate

Figure 4.8: Estimator families are keyed to signal topology. Pairwise wins form a comparison graph and call for Bradley–Terry; a complete harbor history supports the full-history fit rather than streaming Elo. Directed local trust forms a network and calls for EigenTrust. Buyer ratings and price effects form a feedback distribution and call for marketplace models. These are not interchangeable score recipes. None repairs a forgeable identity or an agent-authored oracle: the trustworthy substrate still comes first.

4.8.2 Score as telemetry; gate on predicates

Key idea. Publish the score as *telemetry* (a signal the operator and the buyer can read), but *gate* on *predicates* (machine-checkable witnessed facts: “passed the acceptance test,” “no slashed bond in the last k settlements”). The instant the score itself becomes the gate, it becomes a Goodhart target [146]: “when a measure becomes a target it ceases to be a good measure.”

Exercise 4.22 · Check · ★

Why is a full-history harbor in the Bradley–Terry regime rather than the streaming-Elo regime?

Solution on page 280

Exercise 4.23 · Trace · ★★

You have one task with a clear binary acceptance test, and another judged only by a human’s aesthetic preference. Which estimator fits each, and why does EigenTrust fit neither directly?

Solution on page 280

Exercise 4.24 · Open · ★★★

The substrate-vs-score sentence says the estimator is “cheap and last.” Construct a scenario where a *better estimator* still buys nothing because the substrate is rotten (e.g. forgeable identity or an agent-authored oracle). Generalize the lesson.

Solution on page 281

4.9 Why reputation is *not* a bandit problem

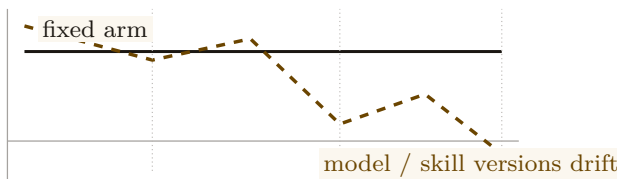
It is tempting — and the routing literature invites the temptation — to model “which agent, model, or skill should I pick?” as a **multi-armed bandit** [209]: arms are agents, pulling an arm yields a reward, and Thompson sampling [219] or UCB balances exploration against exploitation. That framing is correct for one narrow internal task and catastrophically wrong for reputation as a whole. The distinction is important enough to state as a claim.

Honesty rider before the claim. The four-assumption argument below is a **design argument**, not a **theorem**: unlike §4.6’s necessity and whitewash-cost results or §4.11’s imported contraction theorem, it has no numbered formal counterpart in any companion paper of this series. It is stated in the confident register because we believe it, and because getting it wrong is expensive — not because it has been proved.

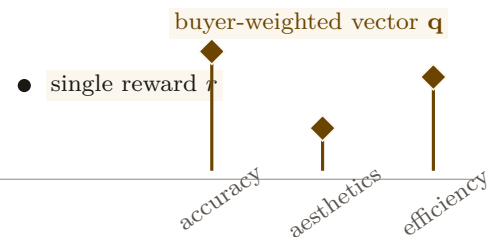
Internal routing — *one operator privately choosing which of their own agents to spawn next* — is bandit-shaped. Reputation — *a public, adversarial, multi-dimensional, persistent substrate other parties trade on* — is not. Modeling reputation as a bandit imports four assumptions that are all false in a market.

Figure 4.9 lines the four bandit assumptions up against the market reality that breaks each.

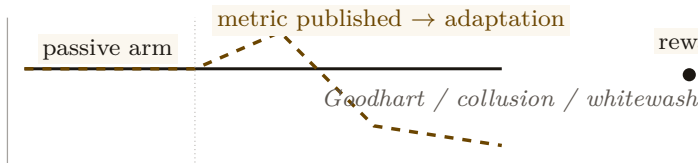
1. stationarity breaks



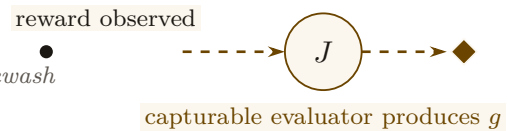
2. one scalar breaks



3. passive arms break



4. observed reward breaks



what survives: private exploration among one operator’s own agents

Figure 4.9: Why public reputation is not a bandit problem. Four diagnostic microplots expose the broken assumptions: agent quality drifts across versions; quality is a buyer-weighted vector; strategic actors respond to measurement; and the grade is produced by a capturable evaluator rather than simply observed. Bandit exploration still fits one bounded task—the teal rail, private routing within one operator’s fleet. Exporting the frame to a public market silently assumes away the substrate and mechanism-design problems.

4.9.1 The four broken assumptions

1. **Stationarity.** Bandit regret bounds assume the reward distribution of each arm is (near-)stationary. Agent quality is *non-stationary* by construction: models are upgraded, skills are versioned, prompts drift, and a skill’s reputation built on v1 says nothing about v2. The Liu–Skrzypacz line of work on *reputation bubbles* [179] shows that with bounded memory and non-stationary types, reputation can build and *collapse* in cycles — behavior a stationary-arm bandit cannot represent.

2. **One scalar reward.** A bandit optimizes a single scalar. Quality is *multi-dimensional*: accuracy, aesthetics, and efficiency are different axes, often in tension (the fast cheap answer is rarely the most beautiful), and a buyer’s weighting of them is a *preference vector*, not a constant. Collapse them into one scalar and you have re-created the Goodhart target the substrate argument warned against (§4.10).
3. **A non-strategic environment.** Bandit arms do not *try* to look good. Agents and their principals are *strategic adversaries*: they will Goodhart the measured axis, wash-trade fake outcomes, whitewash a bad record, and collude. A reputation mechanism is a *game*, and the relevant tool is mechanism design [164], not regret minimization.
4. **A trusted reward signal.** The deepest break: a bandit *observes* its reward. In a market, the reward (the grade) is *produced* by an evaluator who has their own incentives. The grading oracle’s incentive-compatibility is not a side-issue — it is a hole in the core theorem (§4.11). A bandit framing simply assumes the hole away.

4.9.2 What survives the bandit framing

To be fair to the bandit literature: cold-start *exploration* is real, and the right place to borrow from bandits is the *exploration bonus*. TrueSkill’s σ and a UCB-style bonus are the same idea — “test the under-measured arm.” So *within one operator’s private routing*, a contextual bandit conditioned on the operator’s cost/quality preference vector is a fine engineering choice. The error is exporting that frame to the *public substrate*, where non-stationarity, multi-dimensionality, strategy, and an untrusted reward signal all bite at once.

Pitfall. “We’ll just run Thompson sampling over the agents” is the most common way an agent-economy design quietly assumes its hardest problems away. It assumes a trusted reward (no judge-capture), a scalar reward (no aesthetics-vs-efficiency tension), a non-strategic environment (no Goodhart, no wash-trading), and stationarity (no model upgrades). All four are false in a market.

Exercise 4.25 · Check · ★

Name the four bandit assumptions and the market reality that breaks each.

Solution on page 281

Exercise 4.26 · Trace · ★★

Identify the *one* place a bandit *is* the right tool in this paper, and explain why it survives the four objections there.

Solution on page 281

Exercise 4.27 · Open · ★★★

Chiu–Zhang–van der Schaar [218] show competitive agents over-invest in self-improvement, burning surplus despite individual rationality. Does capping the reputation signal’s marginal return restore efficiency, or merely relocate the waste? (Open Problem 6, §4.14.)

Solution on page 281

4.10 The multi-dimensional reputation protocol

We now define the central protocol contribution of this paper: a **multi-dimensional reputation** built over witnessed outcomes. It has three parts: a multi-dimensional quality vector; a market of neutral, conflict-free, influence-free evaluators; and a skill→agent helpfulness attribution. We state it formally as a design (SPECIFIED-to-*proposed*): the reference system has no reputation component yet, only the witnessed-outcome substrate the protocol scores over.

One further honesty rider, distinct from the maturity grade. The maturity scale reports *implementation* status; this rider reports *evidentiary* status, and the two are independent. Unlike the identity theorems of §4.6, the no-mint result of §4.4, and the audit-tower theorem of §4.11 — each of which has a numbered, verified result in a companion formal paper — the quality vector, the neutral-judge ceremony, and the bandit-versus-reputation argument of §4.9 are **program-level design synthesis, not yet a numbered formal result anywhere in the series**. They are motivated by the prior art cited here and consistent with the proven parts, but nothing in this paper or its companions *proves* the three-axis decomposition is the right one or that the judge ceremony is incentive-compatible. Read the claims in §4.9 and §4.10 as design arguments with the theorem-grade register turned down one notch.

4.10.1 Multi-dimensional quality (different axes, different judges)

Definition 4.10.1 (Quality vector). A witnessed outcome carries a **quality vector** over three axes:

$$\mathbf{q} = (q_{\text{acc}}, q_{\text{aes}}, q_{\text{eff}}).$$

The axes are *accuracy* (did it do the right thing, against an oracle), *aesthetics* (is the artifact good — code clarity, design taste — judged by an expert), and *efficiency* (tokens, wall-clock, dollars, drawn from a cost meter that the reference system already records, **implemented**). Each axis is scored by a *different judge*, because the competent judge of accuracy (a test oracle) is not the competent judge of aesthetics (a senior reviewer) is not the competent judge of efficiency (a meter).

The buyer reads the *vector* and applies their own weighting (the operator’s “cheap vs. careful” preference). Publishing a vector and letting the buyer weight it is the natural disclosure form — and it is precisely what avoids re-collapsing quality into one Goodhart-able scalar. The open design question of *whether* a vector disclosure lets agents Goodhart the most-weighted axis is named as a starred exercise.

Figure 4.10 draws the three axes and their distinct judges.

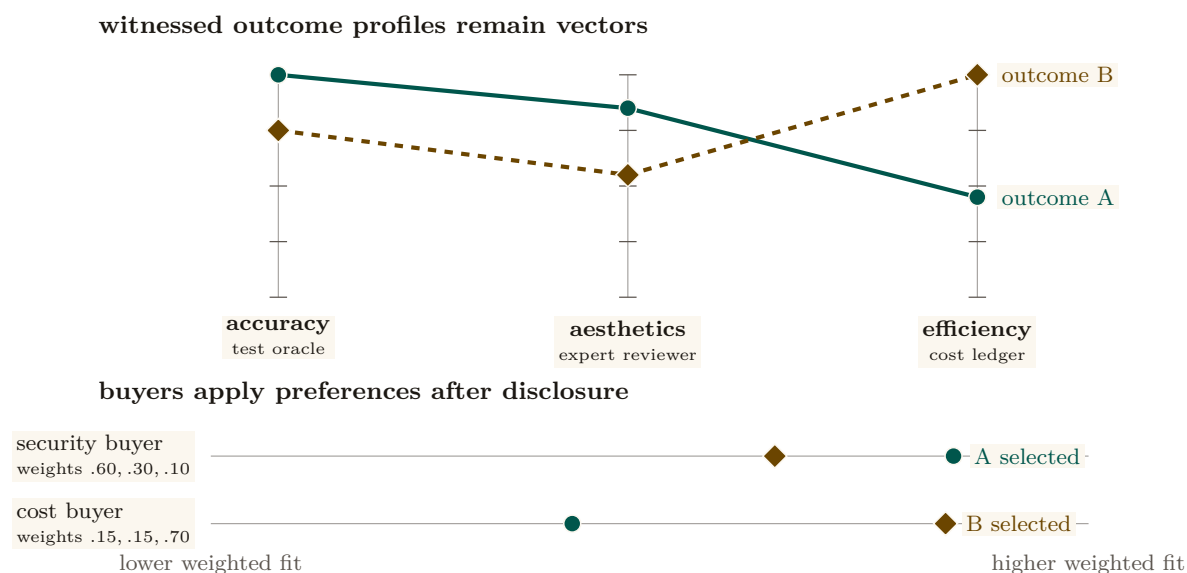


Figure 4.10: One quality vector supports legitimate preference reversal. Outcome A is strongest on accuracy and aesthetics; outcome B is strongest on efficiency. Each axis is independently judged, then disclosed without a universal scalar. A security-weighted buyer selects A while a cost-weighted buyer selects B. Premature averaging would erase one of these valid decisions and create a single Goodhart target. Color is redundant with solid-versus-dashed profiles and direct labels.

4.10.2 The neutral-judge market (the harbor’s universities and rating agencies)

A judge that grades the work of a party it has a stake in is not a judge. The metaphor the series uses is exact: a reputation needs the equivalent of *universities* (which certify competence) and *rating agencies* (which rate counterparties) — third parties whose value is precisely that they are *neutral*. We make “neutral” a checklist, not a vibe:

- **Conflict-free.** The judge has no bond, stake, or principal-link in the work being graded. (Excludes self-grading and same-principal grading.)
- **Influence-free.** The judge cannot be paid *by the graded party* for a better grade. Funding is the hard part: a paid judge has a client, an unpaid judge does not scale. The resolution (§4.11) is that judges post their *own* bond and carry their *own* reputation.
- **De-biased, for LLM judges.** An LLM-as-judge [136] carries position, verbosity, and *self-enhancement* bias (a backend rates its own family up). The discipline is blind, order-swap, pairwise, and family-exclude scoring — never let a backend judge its own family unblinded.

Crucially, an expert third-party judge is *hired through the same bonded ceremony the market uses for any other task*: judging is itself a planned, bonded job, so the harbor that runs the market also runs the hiring of the judges who keep it honest. We state the hiring as a numbered protocol.

Protocol 4.10.1. The judge-hiring ceremony

To grade a settled outcome o produced by actor a under principal p :

1. **Eligibility filter.** The daemon assembles the candidate judge pool and removes every judge sharing a principal-link, bond, or stake with p (conflict-free), and every judge in a 's model family for the aesthetics axis (de-biasing). *Capability $\neq$ permission*: a candidate may be *able* to grade and still be *ineligible*.
2. **Selection.** A judge J is drawn from the eligible pool by a public, conflict-free policy (e.g. reputation-weighted sampling on the judging axis), so neither a nor p can choose their own grader.
3. **Bond post.** J posts a bond B_J before seeing the work. The bond is recorded in the outcome ledger and locked for the dispute window.
4. **Blind grading.** J scores the relevant axis under the de-biasing discipline (blind, order-swapped, pairwise where possible). The grade enters the ledger as a witnessed event keyed on J 's non-forgable identity.
5. **Settlement and exposure.** The outcome settles on the grade; the bond flow follows. J 's bond remains exposed to a sampled **re-audit** (§4.11): a fresh, conflict-free judge who overturns J 's grade *slashes* B_J and updates J 's own reputation downward.

Figure 4.11 draws this: graded work flows to a judge selected for conflict-/influence-freedom, the judge posts a bond, the grade enters the outcome ledger, and a later re-audit can slash a judge whose grade is overturned.

eligibility is established before the grade

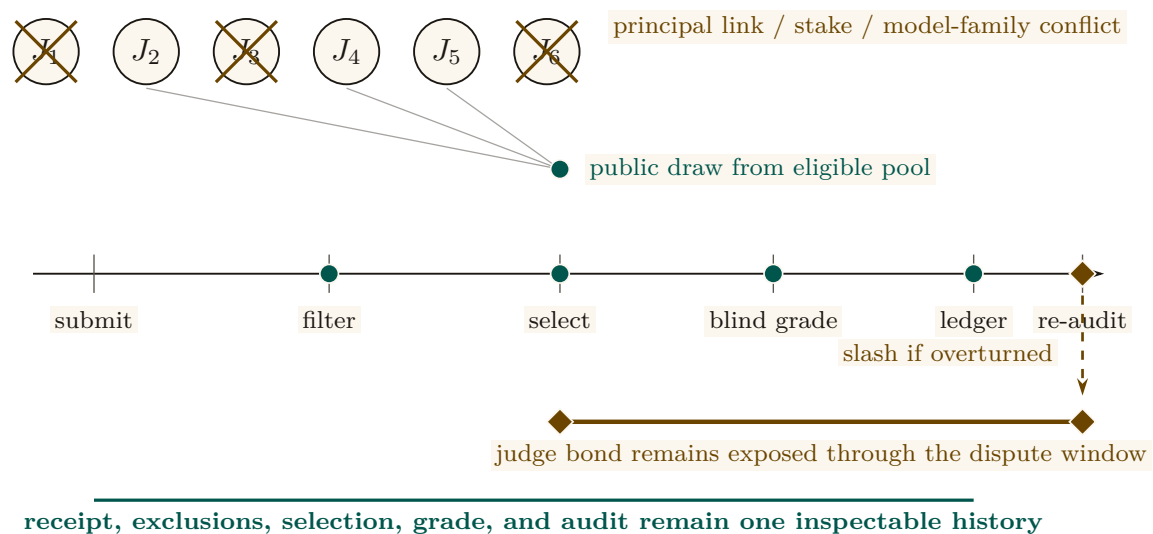


Figure 4.11: The judge market has two different proof moments. Neutrality is earned before selection: candidates sharing a principal, stake, or model family with the graded party are visibly removed, then a public draw selects from what remains. Accountability persists after selection: the judge posts a bond, grades blind, and remains exposed until a later re-audit. The ledger records the receipt, exclusions, draw, grade, bond, and audit, so “neutral” is a checkable history rather than a title attached to the chosen judge.

4.10.3 S3 — a graded job, end to end

Make Protocol 4.10.1 concrete. Bob has rented Alice’s agent *Heron* to refactor an authentication module against a fixed acceptance suite, under a posted bond. Heron delivers; the outcome o enters Bob’s harbor as a *settled but ungraded* delivery, and the daemon runs the ceremony. *Three*

judges are hired, one per axis, because the axes need different competence:

- **Accuracy** is graded by a *test oracle* — the acceptance suite Heron cannot author — which returns $q_{\text{acc}} = 1.0$: every test passes.
- **Aesthetics** is graded by a hired senior-reviewer judge, *Mara* (drawn from the eligible pool, in neither Alice’s nor Bob’s principal, not in Heron’s model family). Mara posts her bond, reads the diff blind, and returns $q_{\text{aes}} = 0.7$: correct but over-clever, with a fragile regex she flags.
- **Efficiency** is graded by the cost meter: $q_{\text{eff}} = 0.9$, cheap and fast.

The quality vector $\mathbf{q} = (1.0, 0.7, 0.9)$ lands in Heron’s outcome ledger. Bob, who weights aesthetics heavily for security-critical code, reads the *vector* (not a collapsed scalar), sees the 0.7, and decides to keep Heron but require a second reviewer on auth work — a decision the scalar would have hidden. Two weeks later a routine re-audit re-grades a sample of Mara’s recent jobs and finds she rubber-stamped a different agent’s insecure change as $q_{\text{aes}} = 0.9$. The re-auditor overturns that grade; Mara’s bond on *that* job is slashed and her judging reputation drops — exactly the exposure step of Protocol 4.10.1. Note what made every step possible: each grade is keyed on a non-forgable identity, so neither Heron nor Mara can shed the record by respawning.

4.10.4 Skill → agent helpfulness attribution

A distinctive, tractable, and *shippable-soon* signal: track whether a **skill** — a reusable unit of agent know-how (a structured instruction document, its referenced material, and any scripts it carries) — actually *helped* an agent, *especially when that skill’s content was loaded into the agent’s context window*. The witnessed event is concrete: skill s was grafted into agent a ’s context at task t ; t later settled with quality vector $\mathbf{q}$. Over many such events we estimate a *skill-helpfulness* effect — did loading s raise $\mathbf{q}$ relative to comparable tasks without it? This is reputation for *tools*, keyed on witnessed, context-attributable outcomes, and it is the licensing side’s substrate that the market paper consumes. It is also the cleanest near-term instance of the whole thesis: a non-forgable event (the graft), a witnessed outcome (the settled task), and a multi-dimensional grade — no bandit required. We pin the event down as a schema.

Protocol 4.10.2. The skill-helpfulness witnessed-event schema

Each graft of a skill into an agent’s context emits an append-only event with fields:

- **skill_id, skill_version** — the tool and its exact version (a portfolio for v1 says nothing about v2).
- **agent_id** — the *non-forgable* identity that received the graft, so the event cannot be re-attributed by respawn.
- **task_id, task_descriptor** — a content hash and a feature vector of the task, used to assemble a comparison set of similar tasks *without* s .
- **grafted_at, context_window_ref** — evidence that s ’s content actually entered the context (not merely that s was available).
- **outcome_ref, quality_vector** — the witnessed settlement and its multi-dimensional grade, written only after the task closes against an oracle.

A buyer estimates helpfulness as the difference in $\mathbf{q}$ between matched tasks with and without s in context — verifiable from the events alone, without trusting the skill’s author.

Pitfall. Attribution is not causation. “Skill s was in context and the task succeeded” does not prove s caused the success. The honest estimate needs a comparison set (similar tasks without s , matched on `task_descriptor`) and is still observational. Sell it as *helpfulness evidence*, not *proof of value*, and gate licensing clawbacks on the witnessed green/red settlement, not on the helpfulness estimate alone.

Exercise 4.28 · Check · ★

For each of accuracy / aesthetics / efficiency, name a competent judge and an *incompetent* one.

Solution on page 281

Exercise 4.29 · Trace · ★★

Walk a judging task through the judge-hiring ceremony (Protocol 4.10.1): who plans it, who posts the bond, where does the grade land, and what slashes the judge?

Solution on page 282

Exercise 4.30 · Trace · ★★

Design the witnessed event for skill-helpfulness. What fields does it need so that a buyer can verify “this skill helped” without trusting the skill’s author?

Solution on page 282

Exercise 4.31 · Open · ★★★

If you publish the 3-vector and buyers weight it, do agents Goodhart the most-weighted axis? If you publish a scalar, you have recreated the bandit framing the substrate argument rejects. What is the right disclosure form for vector reputation? (Open Problem 5, §4.14.)

Solution on page 282

4.11 The grading oracle’s incentive-compatibility

Every folk-theorem incentive-compatibility result in the L3 economy bottoms out in “the daemon derives the grade” — but the grade is produced by a *capturable evaluator*. This is not a side-gap. It is a hole in the *core theorem*, and we treat it head-on rather than assuming it away.

Property 4.11.1 (The grading-oracle hole). Let an outcome’s settlement (and thus the bond flow and the reputation update) depend on a grade g produced by an evaluator J . If J can be captured — bribed, colluded with, or self-interested — then the incentive-compatibility of *every* mechanism that settles on g is only as strong as J ’s own honesty. Assuming J honest is assuming the conclusion.

There are two honest ways to close the hole, and a recursion to bound.

Option A — bond-and-slash the judges. Make judging a bonded role (§4.10): J posts a bond before grading; a later **re-audit** (a fresh, conflict-free judge, sampled) that overturns J ’s grade *slashes* J . Now lying has an expected cost, and a judge’s own reputation is the asset they protect. This converts “trust the oracle” into “the oracle has skin in the game,” which is the same move slashing makes for agents.

Option B — 2-of-3 multi-oracle. Settle disputes against a quorum of independent oracles (automated test / evidence review / human), so capturing the grade requires capturing a majority. This raises the cost of capture but does not eliminate it, and the *human* oracle is a scarce, expensive, capturable resource in its own right (the economics of arbitration capacity is a named open problem, below).

Correlated mode collapse, and the theorem that closes the recursion Option A initially pushed the problem up a level: who audits the auditors? The naive recursion “rate the raters who rate the raters” introduced a severe *correlated mode collapse* vulnerability, where a monoculture of model architectures could form a cartel of mutual validation. This recursion is now *closed*, and not by us: the companion formal paper *Reputation is Amortized Verification* [177] proves it as a parameterized theorem (Theorem 4.11.1 below), and the two mechanisms this section mandates — model heterogeneity and VRF-selected honeypots — are exactly the two hypotheses that theorem needs. Figure 4.12 draws the recursion and where the two mechanisms arrest it.

remaining corrupt opportunity across audit levels

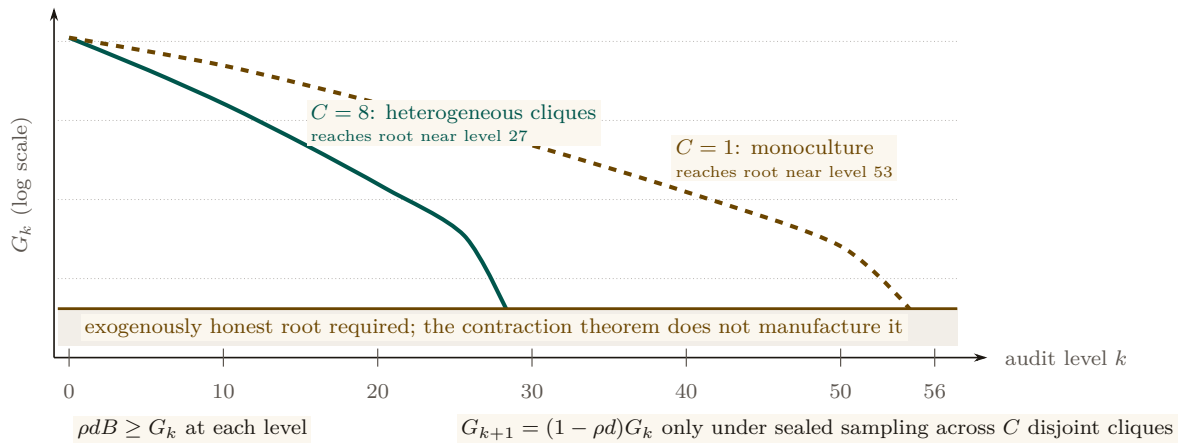


Figure 4.12: Recursive auditing becomes finite only when corrupt opportunity contracts. The parameterized example plots the chapter’s reported depths: heterogeneous sealed sampling across $C = 8$ cliques reaches the honest-root boundary near level 27, while a monoculture ($C = 1$) requires roughly 53 levels. More boxes or more height would not prove this result; the governing conditions are local deterrence $\rho dB \geq G_k$ and contraction $G_{k+1} = (1 - \rho d)G_k$. The amber floor is deliberately separate: the imported theorem bounds the tower but does not supply its exogenously honest root or the telemetry that estimates d .

First, the protocol mandates **model heterogeneity** for judge selection. A quorum of re-auditors must prove they represent distinct architectural weights (e.g. Llama-3 vs. Claude vs. Qwen) under distinct principals, verified via cryptographic provenance, so that a single-model alignment flaw cannot rubber-stamp malicious grades. In the theorem’s language, heterogeneity is what supplies the C *disjoint cliques* the audit pool is sampled from: a corrupt agreement inside one clique does not reach the others.

Second, the system injects **honeypot tasks** selected by a **verifiable random function** (VRF) [199]: deterministic, pre-graded trap tasks, indistinguishable from organic ones, drawn at a committed rate ρ . The VRF is what makes the draw *sealed* — unpredictable to a briber before it

commits money, yet publicly auditable afterward, so the sampler itself cannot be quietly corrupted. Sealing is a hypothesis of the theorem, not hygiene around it: if the draw leaks, the briber buys exactly one auditor and the clique multiplier C vanishes.

Let G_k bound the judge's total one-shot gain from a dishonest grade at level k , let B be the slashable bond, and let d be the probability that an audit which *does* sample a corrupt grade actually detects it. Local deterrence is the expected-forfeiture inequality

$$\rho d B \geq G_k \quad \Longleftrightarrow \quad \rho \geq \rho^* = \frac{G_k}{d B},$$

which is Becker's condition, and note that the detector term d belongs in it: the d -free form $P(\text{honeypot}) > G_k/B$ is the perfect-detector special case $d = 1$ and under-quotes the required injection rate by a factor of $1/d$. At the program's running parameters ($G = 10$, $d = 0.8$, $B = 50$) the honest number is $\rho^* = 10/(0.8 \cdot 50) = 0.25$ — audit one grade in four — where the d -free form would have said 0.2, a 20% under-quote.

Theorem 4.11.1 (Tower contraction; imported from the companion paper). (Theorem 'tower contraction' of *Reputation is Amortized Verification* [177], restated; proof and verification artifacts live there.) *Let level $k+1$ audit level- k auditors, each audit sampled sealed from a pool spanning C disjoint cliques, each auditor bonded at B with audit parameters (ρ, d) and bribe floor $\beta = \rho d B$. Then the briber's expected net value is affine in the number of cliques bought, so bribery is all-or-nothing, and bribing all C is profitable iff*

$$G_k \rho d > C \beta \quad \Longleftrightarrow \quad G_k > C B.$$

Below that threshold bribery stops and corrupt value decays geometrically with a contraction factor that is derived, not assumed:

$$G_{k+1} = (1 - \rho d) G_k, \quad \text{i.e.} \quad \lambda = 1 - \rho d.$$

Consequently finite bond capital certifies a tower deep enough to push surviving corrupt value below any fixed unit: one bond B per level for $\lceil \log G_0 / \log \frac{1}{1-\rho d} \rceil$ levels — a depth logarithmic in the initial corrupt value, not an unbounded one.

Numbers by hand. Take $G_0 = 400$ with the running parameters above, so $\rho d = 0.2$, $\beta = 10$, and the geometric factor is $\lambda = 1 - \rho d = 0.8$. With a heterogeneous pool at $C = 8$ the bribery threshold is $G_k > C B = 400$, unmet even at level 0, so the decay is geometric from the first level and the tower needs $\lceil \log 400 / \log 1.25 \rceil = 27$ levels, i.e. $27 \times 50 = \$1,350$ of bond capital. With a monoculture pool at $C = 1$ the threshold is only $G_k > 50$, so bribery is profitable at first and value bleeds linearly by $C\beta = 10$ per level ($400 \rightarrow 390 \rightarrow 380 \rightarrow \dots$) for 35 levels before the geometric phase takes over for $\lceil \log 50 / \log 1.25 \rceil = 18$ more: 53 levels and $53 \times 50 = \$2,650$. *Now you try:* at $C = 2$, where does the linear phase stop? (Bleed $C\beta = 20$ per level until $G_k \leq C B = 100$, i.e. 15 levels, then 21 geometric ones — 36 levels, \$1,800.)

Numbers by hand — The audit tower: amortized verification spend

As a judge accumulates verified history its reputation-at-stake vt grows, and the audit rate needed for deterrence can fall with it. Flat auditing holds $\rho = \rho^* = 0.25$ forever, so cumulative spend grows linearly: at $T = 200$ periods it totals $a \sum_t \rho^* = 50.00$ [internal, `b2_tower.py`, seed 20260816] — $\Theta(T)$. Two honest amortization models do better. Model A lets the audit rate fall as the stake grows, $\rho_t = G/(d(B + vt))$: at $T = 200$ the running spend is 25.58, matching the closed form $\frac{aG}{dv} \ln(1 + vT/B) = 25.50$ — $\Theta(\log T)$, roughly half the flat bill for the same deterrence. Model B additionally lets a fraction of cheats surface later on their own, $\rho_t = \max(0, (G - rvt)/(dB))$, which hits exactly 0 once the accumulated stake alone deters, at $t^* = G/(rv) = 333$ periods; at $T = 200$ (short of t^*) the running spend is 35.08, converging to the closed-form total $aG^2/(2dBrv) = 41.67$ once the audit rate reaches zero — $O(1)$, a bounded

lifetime bill rather than a growing one. Both models are pointwise incentive-compatible: the script's IC check reports a maximum deviation value of $+1.78 \times 10^{-15}$ for each, zero to floating-point precision. *Now you try:* check by hand that $\rho^* \times a \times 200 = 0.25 \times 1 \times 200 = 50$, the flat baseline both amortization models beat (Exercise 4.36).

What heterogeneity actually buys — and three riders. The honest reading of those two numbers is a factor of two in certified depth and capital, not the difference between a tower that converges and one that does not: raising C *removes the linear bribery phase*, and at these parameters even $C = 1$ terminates. Three hypotheses are required and must not be dropped in restatement. (i) *Sealed sampling is a hypothesis, not hygiene*: a leaked draw collapses C to 1 and puts the VRF sampler inside the trusted computing base. (ii) *The tower needs an exogenously honest root* — the operator at $n = 1$, a bonded arbitration market at scale; the theorem prices the depth to that root, it does not conjure the root. (iii) The theorem is about *bribery of sampled auditors*; it does not price collusion correlated *across* cliques, which is a measurement obligation on deployed judge telemetry (d , and the cross-clique correlation), not an assumption.

Key idea. The grading oracle is the decisive assumption hidden under every IC claim in the economy, and it is no longer an assumption: the companion paper closes the rate-the-raters recursion as a parameterized theorem, deriving the contraction factor $\lambda = 1 - \rho d$ from the audit parameters rather than hypothesizing it, and pricing the tower at 27 levels / \$1,350 of bond with a heterogeneous judge pool against 53 / \$2,650 with a monoculture. What the VRF buys is not a “certain” slash — slashing stays probabilistic by construction — but a *sealed, publicly verifiable* draw, which is precisely the hypothesis the contraction needs. What remains open is the root and the telemetry, not the recursion.

Exercise 4.32 • Check • ★

State Property 4.11.1 and explain why “the daemon derives the grade” does not, by itself, close it.

Solution on page 282

Exercise 4.33 • Trace • ★★

Walk Option A: a judge grades dishonestly, a re-audit overturns it. Follow the bond flow and the reputation update for the judge.

Solution on page 282

Exercise 4.34 • Trace • ★★

Re-derive the critical audit rate in Theorem 4.11.1 from scratch: a judge cheats for gain G , is sampled with probability ρ , is caught given sampling with probability d , and forfeits bond B when caught. Write the cheat payoff, set it to zero, and recover $\rho^* = G/(dB)$. Then check the two worked towers: at $G_0 = 400$, $d = 0.8$, $B = 50$, $\rho = 0.25$, confirm 27 levels at $C = 8$ and 53 at $C = 1$.

Solution on page 283

Exercise 4.35 · Open · ***

Arbitration capacity: the contraction theorem prices the depth to an exogenously honest root but does not supply the root, and human oracles are scarce. Design a market for bonded, rated, slashable arbiters and argue whether it converges to honest rulings or to rubber-stamping under load. (Open Problem 9, §4.14.)

Solution on page 283

Exercise 4.36 · Trace · **

Run `skills/harbor-results/scripts/b2_tower.py` (seed 20260816). Its part (3) compares flat auditing to two amortization models. What are the exact cumulative-spend numbers it prints at $T = 200$, and what should you check by hand about the flat baseline?

Solution on page 283

Recitation

- Without looking: what is Becker’s condition for local deterrence, and why does dropping the detector term d under-quote the required audit rate?
- State the tower’s contraction factor and what raising the clique count C changes about it.
- Name one thing the tower theorem prices and one thing it explicitly does not supply.

4.12 Revocation, tombstones, and bounded memory

A reputation built on witnessed outcomes must answer two questions the cheerful “reputation never ends” framing dodges: what happens when a witnessed outcome turns out to be *fraudulent*, and is no-decay actually *good*? We adopt the series’ reconciled position on both, against the naive version.

4.12.1 Monotone in honest outcomes, individually revocable

Property 4.12.1 (Reputation monotonicity with tombstones). Reputation is **monotone in honest witnessed outcomes** — a genuine delivery is never silently erased by the passage of time — *but* a witnessed outcome is itself **individually revocable via a propagating tombstone**: a compensating, append-only entry that retracts a specific outcome later found fraudulent (a colluding judge, a faked oracle). “Never decays” applies to honest outcomes *only*; it is *not* an anti-revocation property.

This is a **saga**-style compensation [103]: you do not mutate the ledger in place (that would break append-only and auditability); you append a tombstone that nullifies the targeted entry, and the tombstone *propagates* across harbors bounded by the federation’s revocation-convergence window (the market paper’s job). The poisoned data point was spendable only during the propagation window; bounding that window is the federation’s responsibility. We state the retraction as a protocol.

Protocol 4.12.1. Tombstone revocation

To retract a witnessed outcome o later found fraudulent:

1. **Trigger.** A re-audit (Protocol 4.10.1, step 5) or a dispute overturns the grade on o , establishing fraud against an oracle.
2. **Append, never mutate.** A *tombstone* entry $\bar{o}$ is appended to the ledger, carrying o 's id, the overturning evidence, and the identity of the retracting judge. The original o stays in place; auditability is preserved because nothing is erased.
3. **Recompute.** Every reputation score that consumed o is recomputed with o nullified by $\bar{o}$. Because the estimator is cheap (§4.8), recomputation is not the bottleneck.
4. **Propagate.** $\bar{o}$ gossips to every harbor that imported o , bounded by the federation's revocation-convergence window τ . Each harbor applies step 3 on receipt.
5. **Bound the damage.** The fraudulent reputation was spendable only during τ ; the protocol's job is to make τ short and provable, not to pretend the window is zero.

Figure 4.13 draws the append-only compensation: the original entry stays, and a later tombstone nullifies it without mutating the ledger.

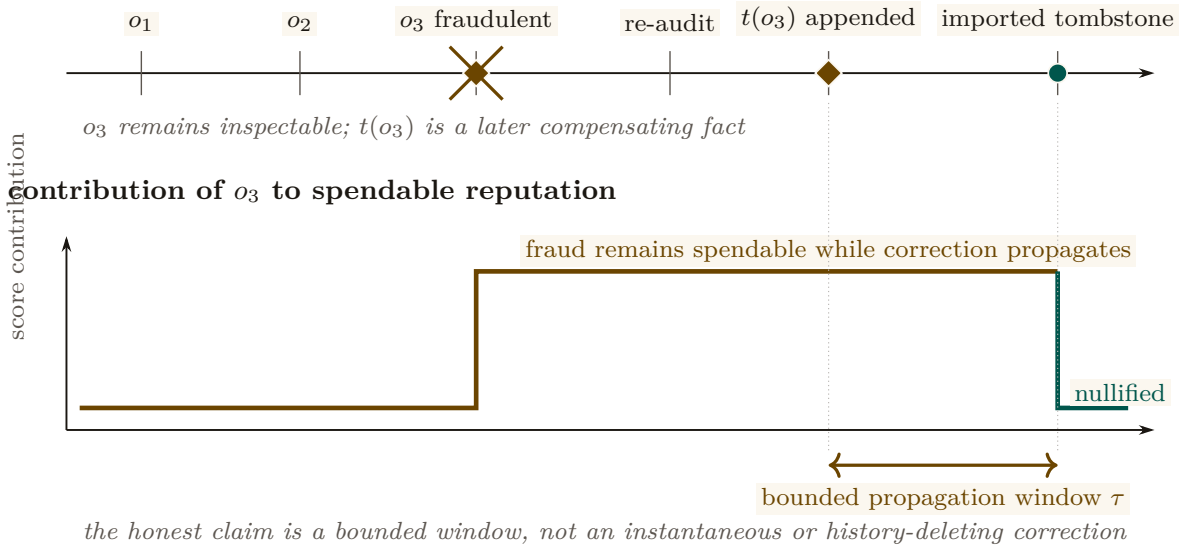
append-only history

Figure 4.13: A tombstone changes score consumption without rewriting history. The upper rail remains append-only: fraudulent outcome o_3 , the overturning re-audit, and tombstone $t(o_3)$ are all preserved as distinct facts. The lower trace shows what does change. The fraudulent contribution remains spendable until the tombstone reaches an importing harbor, then falls to zero. The measured interval τ is the federation's revocation-convergence obligation; the protocol bounds that interval rather than pretending it never existed.

4.12.2 S4 — the fraudulent outcome retracted

Recall S3: the re-audit caught Mara rubber-stamping a different agent's insecure change as $q_{\text{aes}} = 0.9$. Follow that one outcome through Protocol 4.12.1. The graded agent — call it *Crane*, running under a third operator — had colluded with Mara: Crane shipped a known-insecure diff and Mara graded it high so Crane's aesthetics reputation would clear Bob's hiring bar. For two days it worked: Crane's vector showed (0.95, 0.9, 0.8) and Bob nearly hired it. Then the sampled re-audit re-graded the diff blind, found the injection, and overturned Mara's grade. A tombstone $\bar{o}$ is *appended* (the original stays, for the audit trail); every score that consumed Crane's inflated

q_{aes} is recomputed with it nullified; Mara’s bond on that job is slashed and her judging reputation drops; Crane’s collusion is itself recorded as a fraudulent outcome against *its* non-forgable identity, which it cannot shed. The tombstone gossips to Bob’s harbor within the convergence window, so by the time Bob next reads Crane’s vector the 0.9 is gone. The fraud was real and briefly spendable — the honest claim is that the window was *bounded*, not that it never existed.

4.12.3 Bounded memory is a design parameter, not a virtue to assert away

The naive “no decay window an agent can outrun” framing sounds principled but is mechanism-design-hostile. Liu–Skrzypacz [179] show that with *bounded* memory, reputation can be welfare-*improving* relative to infinite memory — and that infinite memory (∞) is rarely optimal, because it freezes types and removes the incentive to keep performing. Bounded memory is a **tunable parameter**, and the right setting is an empirical/mechanism-design question, not a slogan.

It is also, structurally, a *commons* problem rather than a purely technical one. The memory window is a shared, rivalrous resource: every participant would privately prefer their own bad outcomes to age out quickly and everyone else’s to persist forever, so a window chosen unilaterally by the party with the most to hide is the reputational analogue of an over-grazed pasture. Ostrom’s finding on durable common-pool institutions [77] is the relevant prior art here, and it argues against both of the framings this section rejects: not open access (a window each agent sets for itself), and not an imposed global constant (“ ∞ , forever, for everyone”), but a window set by the participants with clear boundaries, graduated sanctions, and cheap local conflict resolution — which is a fair description of what a harbor’s tombstone-plus-window policy actually has to be.

Pitfall. “Reputation never ends” as a *virtue* is wrong twice over: it makes a fraudulent outcome permanent (needs the tombstone of Property 4.12.1), and it asserts ∞ -memory is optimal when the reputation-bubble literature shows it usually is not. Engage bounded memory as a *parameter*; do not assert no-decay as a feature.

Key idea. Two corrections to the cheerful framing: (1) reputation is monotone in *honest* outcomes but *individually revocable* via a propagating tombstone — a saga compensation, not in-place mutation; (2) bounded memory is a *design parameter* (Liu–Skrzypacz reputation bubbles), not an anti-feature to assert away.

Exercise 4.37 • Check • ★

Why must revocation be a tombstone (append) rather than a delete (mutate) on the outcome ledger?

Solution on page 283

Exercise 4.38 • Trace • ★★

A judge is later found to have colluded on five settled outcomes. Walk the tombstone for one of them: what is appended, what propagates, and what bounds the window the poisoned reputation was spendable?

Solution on page 283

Exercise 4.39 · Open · ***

Reputation-claim revocation with an append-only, bounded-memory ledger across N harbors: how do you prove the correction converged and bound the spendable window, without violating append-only? This is the cross-harbor analogue of capability revocation (*The Harbor Economy*). (Open Problem 7, §4.14.)

Solution on page 284

4.13 What this chapter hands the market

This paper is the bridge; its output is a precise hand-off to *The Harbor Economy*. We summarize what is delivered, what is borrowed, and what is explicitly owed.

Artifact	Maturity	Role in the market paper
Role / person distinction (Defs. 4.3.1–4.3.2)	—	The third side of the market is a <i>tradeable person</i> , defined here.
Principal-as-economic-entity (Def. 4.6.2)	SPECIFIED	Bonds and sanctions key on the principal; the market paper cross-references, not redefines.
The three continuity organs	implemented PARTIAL PARTIAL	Reputation keys on organ 3 (the outcome ledger).
Outcome ledger / oracle (Def. 4.5.1)	PARTIAL	Commitment, deadline, oracle closure, API/CLI, and overdue detection ship; neutral graded outcome events, sanctions, and reputation binding do not.
Multi-dimensional reputation (Def. 4.10.1, Protos. 4.10.1, 4.10.2)	SPECIFIED to <i>proposed</i>	The multi-dimensional score the market reads; the licensing side’s signal.
Neutral-judge market	SPECIFIED to <i>proposed</i>	The “universities and rating agencies” that keep grades honest.
Grading-oracle (Prop. 4.11.1)	hole SPECIFIED	<i>No longer owed</i> : the recursion is closed by Theorem 4.11.1 [177] ($\lambda = 1 - pd$; 27 levels / \$1,350 at $C=8$). What is owed back is narrower — an exogenously honest root and the audit telemetry (d , cross-clique correlation).
Cross-operator (Def. 4.7.2)	attestation <i>proposed</i>	<i>Named dependency the market paper must build</i> ; not assumed here.
Tombstone (Prop. 4.12.1)	revocation SPECIFIED	Reputation is revocable; convergence is the market paper’s federation job.

Table 4.5: The bridge’s hand-off to the market paper. One item still flows *back* as a named, unsolved obligation — cross-operator attestation (Def. 4.7.2). The second historical debt, the grading oracle’s incentive-compatibility, has been *paid*: the companion inspection-game paper closes the rate-the-raters recursion as Theorem 4.11.1, leaving only the tower’s honest root and its measurement obligation open. The series is coherent precisely because these debts are named — and retired — in the same words on both sides of the seam.

Build the harbor first. The trade comes when the ships do — but the thing that makes the trade possible (continuity that turns spawns into persons) is the same thing that makes the single-player tool good. The bridge is not a detour; it is the load path.

4.14 Open problems (the starred exercises, collected)

The genuinely unsolved problems this paper surfaces, gathered for the reader who wants the research frontier rather than the pedagogy. Every starred exercise in the body cross-references this list *by name*, not by a hand-typed numeral, so the two cannot drift apart. Note that the “OP-*n*” identifiers used in the body for kernel-layer problems (e.g. OP-4, checkpoint with teeth) belong to a *shared* namespace owned by *The Single-Writer Kernel* and are deliberately not renumbered here; this list is this chapter’s own.

1. **Branching continuity (§4.4).** If a checkpoint is forked into two live incarnations, which inherits the reputation? Parfit says identity-is-not-what-matters; the economy needs a rule, and the rule invites an attack.
2. **Agent-death taxonomy (§4.5).** What is recoverable vs. fundamentally lost when an LLM agent dies mid-thought? Event-Sourced Neural Rehydration (OP-4 in the shared kernel namespace) is a SPECIFIED candidate answer — restore the Git SHA, truncate the durable message log to the crash point, replay under prompt-prefix caching — but it has not shipped, and its completeness has not been established: replay restores lineage and, within one provider/model/version, a behaviorally equivalent continuation; it does not export the inference state, so across providers the honest description is task continuity with actor succession. The taxonomy itself — record vs. claims vs. execution state vs. latent “intent” — stays open until the mechanism ships and is measured.
3. **Cross-operator attestation (§4.7).** Binding identity keys across harbors you do not own. The highest-leverage unbuilt keystone for the market.
4. **The grading oracle’s honest root and its telemetry (§4.11).** The rate-the-raters recursion itself is *no longer open*: Theorem 4.11.1 [177] closes it with a derived contraction factor $\lambda = 1 - \rho d$ and a priced depth (27 levels / \$1,350 at $C=8$; 53 / \$2,650 at $C=1$). What remains open is what that theorem explicitly does not supply: the exogenously honest root the tower terminates against at scale, and the deployment telemetry that estimates d and the cross-clique collusion correlation the disjointness hypothesis assumes away.
5. **Vector-reputation disclosure (§4.10).** Publish a vector and agents Goodhart the heaviest axis; publish a scalar and you have a bandit. The right disclosure form is open.
6. **Self-improvement arms race (§4.9).** Chiu et al. 2025: does capping reputation’s marginal return restore efficiency or relocate the waste?
7. **Reputation-claim revocation convergence (§4.12).** Surgically retract a fraudulent outcome across N harbors, prove convergence, bound the spendable window — without violating append-only.
8. **Bond-farming defense (§4.6).** Principal binding vs. listing fee vs. sampled re-audit of high-velocity pairs; the false-positive cost to honest high-volume counterparties.
9. **Arbitration capacity (§4.11).** A market for bonded, rated, slashable human/automated arbiters: honest rulings, or rubber-stamping under load? This is Item 4’s honest root, stated as a market-design problem.

10. **Skill-version reputation (§4.10).** A portfolio proof for skill v1 says nothing about v2; the lemons problem reappears at every version bump.

4.15 Conclusion

We set out to bridge the wedge to the market by answering one question: what has to be true before a coding agent's work can be *sold*? The answer is not a score. It is a *self* — a role plus continuity, keyed on an identity that cannot be re-picked, accumulating a transitive, witnessed history that survives every respawn. The estimator that turns that history into a number is the cheap, last part; the substrate beneath it is the gate, and most of that substrate is formally not built yet.

We were disciplined about the keystone (this paper stands on a local non-forgeable identity and hands the market paper the unbuilt cross-operator attestation by name), about the hardest hole (the grading oracle's incentive-compatibility is a flaw in the core theorem, not a footnote — and the rate-the-raters recursion underneath it is now closed by a companion theorem with a derived contraction factor and a priced tower depth, leaving the honest root and the audit telemetry as the residue), and about the cheerful slogans (reputation is monotone in *honest* outcomes but individually revocable, and bounded memory is a parameter, not a virtue). Reputation is not a bandit problem — it is non-stationary, multi-dimensional, strategically adversarial, and graded by an oracle with its own incentives.

A spawn does work and disappears. A person does work and stays — and only what stays can be priced. The harbor's whole economy is the machinery for manufacturing, and then formally grading, a self.

Chapter Appendices

4.A Implementation status of the reference system

The body of this paper is written to stand on its own: every mechanism is described as a component (a memory record, a checkpoint organ, a witnessed-outcome ledger) and graded on the neutral maturity scale (**implemented** / PARTIAL / SPECIFIED / *proposed*). This appendix grounds that scale in a concrete *reference system* — a running coordination daemon for local agent swarms — so a reader who wants to know “which of these actually runs today” can check. Nothing in the body depends on this appendix; it exists so the maturity labels are falsifiable rather than rhetorical.

Component (paper concept)	Maturity	Status in the reference system
Episodic memory (Organ 1)	implemented	Durable typed episodes promoted from execution history; runs in production.
Restart organ (Organ 2)	PARTIAL	Heartbeat-staleness detector that salvages dead agents; forwards a <i>summary</i> , not execution state.
Outcome ledger (Organ 3)	PARTIAL	Durable commitments, daemon-derived deadlines, oracle-bound closure, API/CLI, and overdue detection ship; neutral grading, sanctions, and reputation binding do not.
Cost meter (efficiency axis)	implemented	Per-task token/wall-clock/dollar accounting recorded today.
Local non-forgable identity	PARTIAL	Daemon-minted actor-soul, lookup credential, and bounded newcomer pool ship; full write-boundary enforcement and legacy migration do not.
Capability jail (capability $\neq$ permission)	SPECIFIED	Per-agent tool-allowlist and scoped filesystem; specified.
Honest-attestation discipline	implemented	“Report all-good only when verified” is an enforced engineering rule.
Multi-dimensional reputation	SPECIFIED to <i>proposed</i>	No reputation component yet; the substrate it scores over is partly built.
Neutral-judge ceremony (Proto. 4.10.1)	SPECIFIED to <i>proposed</i>	Bonding and task-planning primitives exist; the hiring ceremony is designed.
Skill-helpfulness schema (Proto. 4.10.2)	SPECIFIED	Skill grafting is observable; the witnessed-event schema is specified.
Tombstone revocation (Proto. 4.12.1)	SPECIFIED	Append-only ledger semantics are designed; cross-harbor propagation is <i>proposed</i> .
Cross-operator attestation	<i>proposed</i>	The unbuilt keystone for the market; owed to <i>The Harbor Economy</i> .

Table 4.6: Maturity of each paper concept against a reference coordination daemon. The honest summary: the *organs* of continuity exist (one weakly); the *spine* of reputation — a witnessed-outcome ledger keyed on a non-forgable identity — is specified but not yet shipped.

What *The Harbor Economy* receives A successor can now inherit a bounded history without inheriting

imaginary credit or escaping old obligations. Continuity by itself creates no market: it neither prices risk nor says whose funds may move. The next chapter turns attributed work into exchange by separating value, collateral, evidence, and settlement.

PART IV

Trade Between Strangers

Once records cannot be minted, trust can be rented. This part builds the market, then proves what it must assume: settlement conserves value, evidence is admissible, and sovereignty never crosses the wire.

5 The Harbor Economy

Once records cannot be minted, trust can be rented between operators who never met: a three-sided market on one conserving ledger.

6 The Bonded Commons

Bonds, witnesses, audits and appeals turn residual uncertainty into an institution; the ledger's conservation law is mechanically checked.

7 The Federated Harbor

Mutually distrustful machines relay evidence, not sovereignty: what may gossip, what needs an authority point, and how a grant crosses a boundary.



The Harbor Economy

Virtually every commercial transaction has within itself an element of trust, certainly any transaction conducted over a period of time.

KENNETH J. ARROW, GIFTS AND EXCHANGES, 1972



THE QUESTION THIS CHAPTER ANSWERS

What makes cooperation rational without making loss unbounded?

Once records cannot be minted, trust can be rented between operators who never met: a three-sided market on one conserving ledger.

What makes cooperation rational without making loss unbounded? A payment rail moves money; a market must also identify the parties, state what performance means, reserve the right funds, and decide what happens when evidence disagrees. This chapter constructs that market from four separate monetary buckets and explicit oracles. R7 and R13–R15 delimit the audit, substitution, escalation, and specialization claims.

Maturity key (honest self-grading). Every claim in this paper carries one of four maturity labels, so the reader always knows whether a sentence reports running code or a designed target. **implemented** — shipped and exercised by tests in the reference implementation. **PARTIAL** — shipped but materially weaker than its name suggests. **SPECIFIED** — specified, with a proof or a written protocol, but not yet shipped. *proposed* — a stated direction whose critical part has no specification yet. Nomenclature is likewise fixed: the cryptographic *hop-bound authorization chain* (which proves who delegated what, and bounds it) is never conflated with the *coordination lineage* (who-talked-to-whom, advisory only); **local non-forgable identity** (an identity no agent can edit within one trusted daemon) is never conflated with **cross-operator attestation** (the unbuilt keystone that binds identity *across* daemons nobody jointly trusts); and *capability* (what an agent *can* do) is never collapsed into *permission* (what it *may* do).

5.1 Reader’s map

This paper is the top rung of a ladder (Figure 5.1). It assumes the rungs below it and cites the papers that build them. Read it cold if you must, but it is the fourth paper for a reason: it depends on everything underneath.

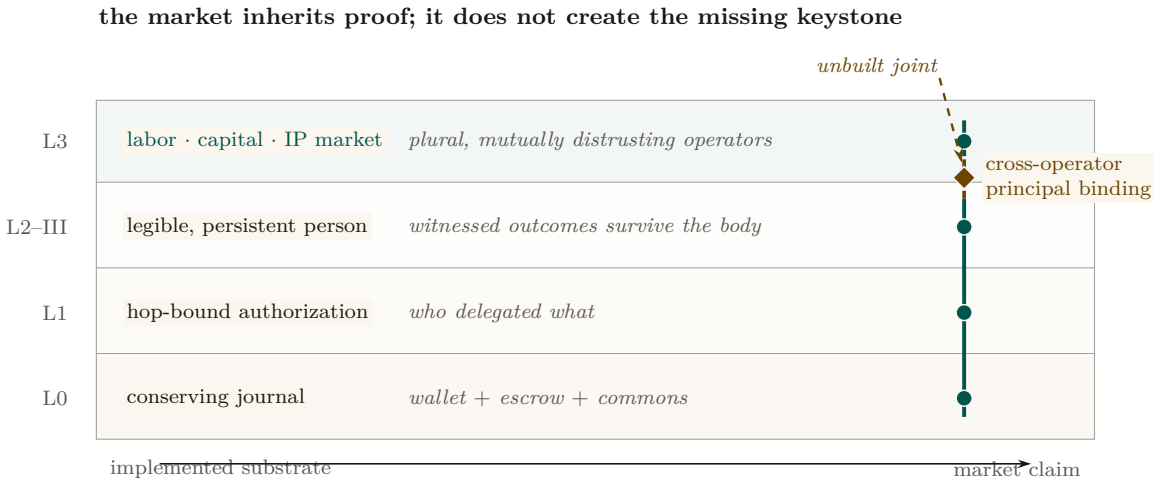


Figure 5.1: The market is a load path, not a freestanding top layer. The conserving journal, hop-bound authorization, and persistent accountable person provide three evidenced supports. The teal spine carries those supports upward. It becomes dashed amber at the cross-operator principal-binding joint: the market needs that proof, but the local daemon cannot supply it. Chapter order is not a maturity scale, and the bond ledger alone does not establish a market among mutually distrusting operators.

Table 5.1: Where to enter, by who you are. Five reader types, each routed to the section that answers their question and to the one figure or theorem that would have to survive a review from someone in that role — so nobody has to read the paper end to end to get their own answer out of it.

If you are...	...start here	...critical artifact
A founder asking “what is the product”	§5.2 (thesis) → §5.11 (hosted trust)	Fig. 5.2; the three claims C1–C3
A mechanism designer	§5.3 → §5.10 (the formal model)	Thm. 5.7.2, Fig. 5.11
A cryptographer / distributed-systems reviewer	§5.6 (the keystone) → §5.9	Fig. 5.4, Fig. 5.6
A category theorist	§5.7 → §5.13	Fig. 5.5, Conj. 5.13.1
An implementer	§5.4 (the built floor) → Appendix 5.A (status)	Table 5.5

Where this chapter sits. *The Single-Writer Kernel* is the local transactional floor and *The Anchor Protocol* proves how its authority delegates; *The Legible Swarm* is the operator’s front door; *From Spawn to Person* supplies continuity and reputation. *This* chapter is the market. *The Bonded Commons* and *The Federated Harbor* then prove what the market assumes about settlement and federation.

Dramatis personae. Every mechanism in this paper is dramatized end-to-end with the same cast, so an abstraction is never introduced without a concrete trade to attach it to. **Alice** runs Harbor *A* on her laptop and owns a well-reputed refactoring specialist agent, *a*₂. **Bob** runs Harbor *B* and needs work done he cannot staff himself. **Carol** runs Harbor *C* and sells a licensed lint-and-type skill. **Judy** is a human grader who arbitrates disputes for a bond. The two harbors do *not* trust each other and neither is a root of authority for the other; that distrust is the entire reason the rest of the paper exists. We price everything in an abstract unit, the *credit* (CR); read it as a stablecoin cent if you like.

5.2 The thesis: a market, not a payment rail

A solo operator running a swarm of coding agents needs three things — legibility, accountability, and safety — and needs *no cryptography at all*, because she owns every machine in the harbor. That is the single-player wedge of *The Legible Swarm*. The instant two operators trade, the world changes. Alice’s frigates touch your repository. You must now price work done by agents you did not spawn, owned by a principal you do not trust, using skills you cannot inspect. That is a *market-design* problem before it is a cryptography problem. Cryptography is the substrate that makes the ledger unforgeable; it is not the product.

*You don’t sell crypto — crypto is the substrate; you sell **hosted trust**: a verified ledger, a relay, and a reputation system that lets mutually-distrustful fleets transact across a boundary they do not share.*

Three claims follow, and the rest of the paper defends each:

- **C1 (three sides).** The harbor economy has three *distinct incentive constraints* — operator-for-hire, asset-rental, skill-licensing — not two. Counting them correctly changes the pricing (§5.3).
- **C2 (one ledger, one escrow object).** All three sides settle on the same **float plan** escrowed in the same bond ledger. Conservation of value across every settlement path is the invariant

that holds the market together, and it is the one thing here that is actually **implemented** (§5.4, §5.7).

- **C3 (hosted trust is the moat).** The defensible asset is the verified ledger + relay + reputation, not the settlement rail, which the 2025–26 agentic-payments stack has already commoditized (§5.11).

Key idea. The economy is strictly *additive* on top of the single-player tool. Because all three market sides settle on the *same* built bond ledger, none of the economy needs to ship before the wedge does. The discipline “don’t chase the market early” is *safe* precisely because the market reuses the kernel primitive. The harbor comes first; the trade comes when the ships do.

5.2.1 The through-line: why a market needs persons, not spawns

The whole series climbs one spine (the vertical arrow in Figure 5.1):

memory → checkpoint → continuity → a person, not a spawn (From *Spawn to Person*)
 → registered outcomes → reputation → tradeable asset → market (this chapter).

Sides 2 and 3 of the market — renting an *asset*, licensing a *good* — presuppose that there is a *thing* to rent or sell that cannot be costlessly cloned. You cannot rent a reputation that any spawn can fork, and you cannot license a skill whose track record is unverifiable. The canonical cross-layer sentence, repeated in *From Spawn to Person* and here, states the dependency exactly:

The score is cheap; the substrate it scores over — witnessed outcomes on a non-forgeable identity — is the gate.

Exercises

- E1.** Give a one-sentence reason the market cannot ship before reputation does, using only the word “clone.”
- E2.** Side 1 (operator-for-hire) does *not* require durable identity in the same way sides 2 and 3 do. Why? (Hint: who is the counterparty, and who bears the consequence?)
- E3. ★** Construct a market design in which sides 2 and 3 exist *without* a non-forgeable identity. Argue informally why every such design collapses to a Sybil attack (forward-reference §5.6).

5.3 What a “side” is, and why there are three

A **two-sided market** (Rochet & Tirole [107], the foundational reference: *a platform is two-sided when the allocation of the total price across the two user groups — not just its level — affects transaction volume*) is the canonical frame for a marketplace: buyers on one side, sellers on the other, the platform tuning who is subsidized to ignite cross-side network effects. The naïve reading of the harbor economy is two-sided: a *requester* posts work, a *worker agent* does it.

But “two-sided” undercounts. The operational test [108] is: how many distinct, privately-informed parties must the platform individually rationalize into participating? In a mature harbor there are three, because *the same agent can be three economically different things* (Figure 5.2).

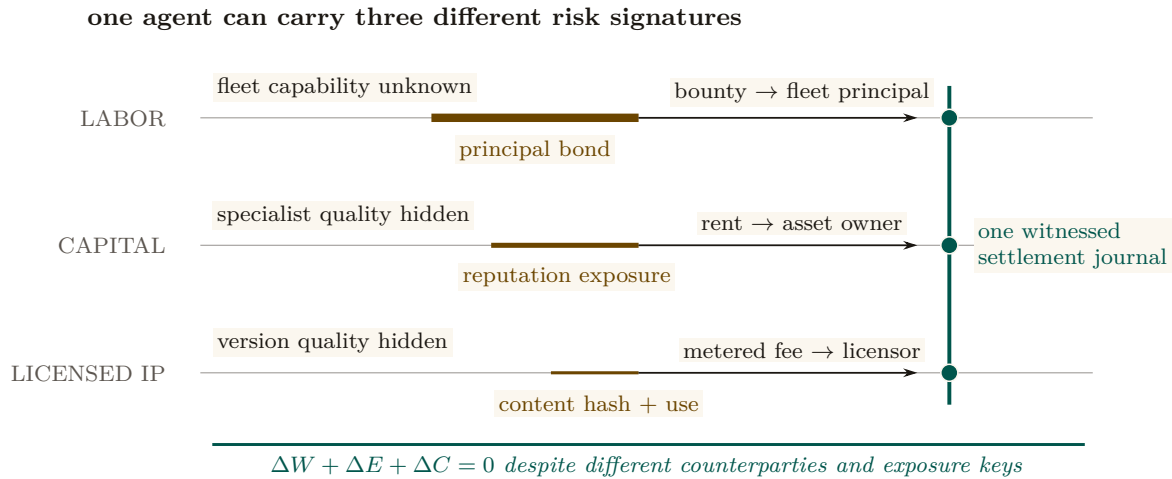


Figure 5.2: The three sides are three risk signatures, not three tabs. Labor exposes a principal bond and pays a fleet principal; rented capital exposes the owner’s reputation and pays rent; licensed IP exposes a version-bound content claim and pays a metered fee. Line thickness redundantly distinguishes the forms of exposure. All three close through one witnessed journal, so the conservation invariant is shared even though the counterparties, private information, and reputation keys are not.

1. **Operator-for-hire (labor).** An operator who runs a fleet can sell its labor to *another* operator. The operator is now a firm taking work-for-hire, with margin = bounty – Σ (slashed sub-bonds) – ledger fee. Its private information is whether its fleet can deliver. It internalizes its fleet’s risk — exactly the property that makes a firm accountable for its employees.
2. **Agent/fleet as rentable asset (capital).** An operator who owns a well-reputed specialist can *rent it out*. Now the asset-owner, the task-poster, and the worker are three different parties with three different incentives. The owner’s private information is the agent’s true quality; its exposure is reputational.
3. **Skill/tool as licensed good (IP).** A skill can be *licensed* into someone else’s float plan — metered or per-use — without the licensor doing any task work. The licensor’s private information is the skill’s quality, which the buyer cannot inspect before purchase.

These are three *incentive constraints*, not three UI tabs. If the asset owner is always the task poster, side 2 collapses into side 1; the design discipline is to count sides by constraints. The harbor has three because *capability* (an agent) and *competence* (a skill) become separable, tradeable goods once identity is durable.

The same trade, run three ways (the running example). Bob needs a 1,200-line module refactored. He can buy it on any of the three sides, and the side determines who is privately informed and who is exposed. *Side 1 (labor).* Bob posts a float plan with a 400 CR bounty; Alice takes it as a firm. She dispatches three of her own agents, posts a 60 CR sub-bond per agent (180 CR total), and keeps 400 – (slashed sub-bonds) – fee. If one of her three agents botches its slice and is slashed 60 CR, Alice eats that loss — she internalized her fleet’s risk, which is exactly what makes her accountable as a firm. *Side 2 (capital).* Instead Bob rents Alice’s specialist a_2 directly for an afternoon at 90 CR. Now there are three parties: Bob (poster), Alice (owner, privately knows a_2 ’s true quality), and a_2 (worker). Bob holds the runtime control of a_2 while it runs on his repository; Alice’s exposure is purely reputational — if a_2 underperforms, a_2 ’s public score falls, which is Alice’s asset depreciating. *Side 3 (IP).* Carol never touches Bob’s repository at all; she licenses her lint-and-type skill into Bob’s float plan at 0.5 CR per file, metered, released to Carol *only on green settlement*. Carol is privately informed about her skill’s quality; Bob cannot inspect it before he

runs it. Three sides, one refactor, three different “who knows what” structures — and, as the next section shows, one escrow object underneath all of them.

Side	What Bob pays	Who is exposed
1. Labor (hire Alice’s firm)	400 CR bounty	Alice, through her sub-bonds
2. Capital (rent a_2 directly)	90 CR for the afternoon	Alice, reputation only
3. IP (license Carol’s skill)	0.5 CR/file, metered	Carol, unpaid unless green

Pitfall. Two-sided-market theory’s central result is that the *price structure* — who subsidizes whom — is the lever, not the price level. Mis-count the sides and you mis-structure the subsidy: you charge the side you should be *paying* to show up. §5.12 returns to this for cold start.

The framing is not idiosyncratic. The most recent academic treatment of AI labor markets, **Chiu, Zhang & van der Schaar** (2025) [44] (*a formal model of a three-sided agent labor market*), models exactly agents, employers, and a reputation system — confirming that “reputation system as a third side” is the natural structure once agents compete for paid work.

5.3.1 The mandatory honesty rider

The canonical statement of the market’s shape carries a rider that must never be dropped:

The harbor economy is a three-sided market by design; two-sided until reputation ships. The third side is the tradeable-person terminus of From Spawn to Person.

Today there is no reputation estimator in the reference implementation; it is a gated phase on the roadmap. So sides 2 and 3 are SPECIFIED, not shipped. What *is* shipped is the escrow + conservation floor they will sit on. We do not overclaim the market.

One ledger, three different reputation keys

All three sides ride one float-plan escrow, but they do not share one identity key. Each side keys reputation on a different entity — the **principal** for labor, the **asset owner** for rental, the **content hash** (a specific skill version) for licensing. The shared settlement table therefore needs a tagged subject identifier such as (*kind, id, version*); otherwise non-equivalent reputation subjects collide. The dashed band in Figure 5.2 marks this schema distinction.

Exercises

- E1.** State the operational test for “how many sides” in one sentence, then apply it to a ride-share platform: how many sides, and why?
- E2.** Side 3 (skill licensing) keys reputation on a *content hash*, not a person. What breaks when a licensor ships v2 of a skill? (Hint: §5.14, skill versioning.)
- E3. ★** A buyer reads a three-axis reputation vector (accuracy, aesthetics, efficiency). Specify a disclosure rule that lets the buyer weight the axes *without* re-introducing a single Goodhart target. Argue why every fixed scalar collapse re-creates the bandit framing the design rejects.

5.4 The float plan: the built floor

Every side settles on one object: the **float plan**.

Definition 5.4.1 (Float plan). A *float plan* is a signed declaration of intent: a task, a set of *machine-checkable* acceptance criteria, a compute budget, and a credit bounty. It is the atom all three market sides escrow against. (A *machine-checkable* criterion is one a third party can evaluate without trusting the worker’s word — a named test that must pass, a commit hash that must merge, a static check that must be satisfied.)

The escrow handshake is a three-step signed ceremony over a standard digital-signature scheme (Protocol 5.4.1, drawn in Figure 5.3): the requester signs the float plan; the daemon opens a serialized database transaction, debits the requester’s wallet, and moves bounty plus bond into an escrow row; the daemon then signs the pair $\{\text{anchor id}, \text{plan hash}\}$, proving to the worker that funds are locked *before any work runs*. This is the heart of “no-spawn-without-bond.”

Protocol 5.4.1 (Float-plan escrow ceremony). **Parties:** requester R , daemon D (the local trusted root), worker W . **Pre:** R ’s wallet holds at least $\text{bounty} + \text{bond}$.

1. $R \rightarrow D$: $\text{sign}_R(\text{FloatPlan})$ where $\text{FloatPlan} = \langle \text{task}, \text{criteria}, \text{budget}, \text{bounty} \rangle$.
2. D locally, in one serialized transaction: debit $\text{bounty} + \text{bond}$ from R ’s wallet, write an escrow row keyed by a fresh *anchor id*, commit. *If the transaction aborts, no value moved (atomicity).*
3. $D \rightarrow W$: $\text{sign}_D\{\text{anchor id}, \text{plan hash}\}$. W verifies D ’s signature, confirming funds are pinned before doing any work.

Post: the conservation invariant (Property 5.4.1) holds; W has a signed proof of locked funds; no process for this task can enter **running** without the escrow row existing.

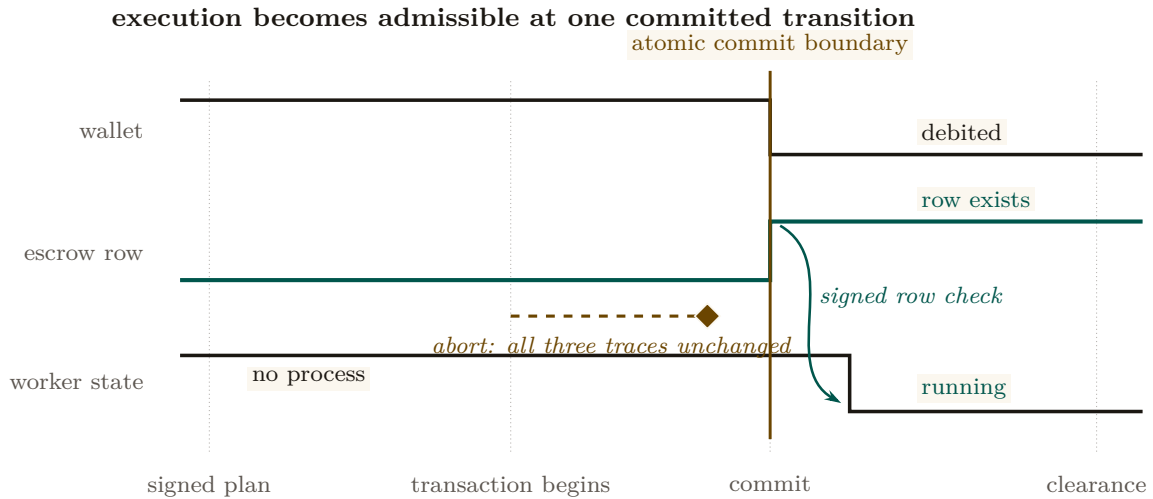


Figure 5.3: No-spawn-without-bond is a synchronized state transition. The wallet debit and escrow-row creation occur at the same commit. Only after the committed row is observable does the worker move from nonexistent to running. A signed plan or an attempted debit is insufficient: the dashed abort branch leaves all three traces in their prior state. The figure therefore locates the invariant at a transition, not at a component or message label.

5.4.1 The bond ledger conserves value (this is built)

The single most consequential primitive — and the one that is *actually implemented* — is the **bond-ledger module**: bond escrow for agent spawning. It debits a project wallet into an escrow row before spawn, refunds on clean exit, and slashes on breach, splitting the slash between the wallet and a commons pool. It guards two invariants:

Property 5.4.1 (Conservation). $\text{wallet} + \text{escrow} + \text{commons} = \text{supply}$, always. Every debit has a matching credit. In the reference implementation this is verified across ten thousand random operation traces by a property test — a real randomized test, not an aspiration. (**implemented**; see Appendix 5.A.)

Property 5.4.2 (No-spawn-without-bond). A process enters the **running** state only if a bond was escrowed against its agent id. (PARTIAL: the ledger enforces escrow-first, but the runtime **running**-state check is a roadmap item, so today the gate is caller-discipline, not runtime-enforced. See Appendix 5.A.)

Key idea. Conservation is the mathematical glue of a three-sided market. Whatever happens — success, partial credit, slash, lease, license — no value is created or destroyed; it only moves between wallet, escrow, and commons. A three-sided market with three settlement paths is coherent *only if it cannot leak*. Port Daddy already enforces this for the labor side; the contribution of §5.7 is to show the same invariant extends to the rental and licensing sides *without modification* and *without a second ledger*.

5.4.2 Settlement: four terminal states bound to an oracle

Settlement reads an **oracle** (*a trusted source of ground truth the agent cannot author — a passing test id, a merged SHA, a satisfied Arbiter check*) over the acceptance criteria and lands in one of four states.

Table 5.2: The four terminal settlement states. Closure binds to an oracle, not to a free-text “Result:” note — the Goodhart-resistant discipline that the commitment layer imposes: *the agent picks the work; the daemon derives the grade*.

State	Flow of funds	Reputation effect
Success	escrow $\rightarrow$ worker (bond refund) + bounty $\rightarrow$ worker	+1 completion
Partial	pro-rata bond $\rightarrow$ worker by completed evidence-chain fraction; remainder $\rightarrow$ salvage fund	salvage recorded
Sabotage	bond $\rightarrow$ reconstruction commons (100% slash)	failure recorded; ban on threshold
Dispute	escrow $\rightarrow$ arbitration hold; 2-of-3 multi-oracle (automated / evidence / human)	none until resolved

A settlement, dramatized. Bob (the requester principal) posts a refactor float plan: bounty 400 CR, requiring a provider performance bond of 100 CR and acceptance criteria = “the module’s test suite passes and the diff merges.” Alice (the provider principal) assigns agent a_2 and posts the 100 CR performance bond into escrow. Bob escrows the 400 CR bounty. Before execution, the escrow holds 500 CR across two distinct source buckets: $\text{escrow}_{\text{bounty}}(\text{Bob}) = 400$ and $\text{escrow}_{\text{bond}}(\text{Alice}) = 100$.

Agent a_2 completes 40% of the verifiable evidence chain (refactoring two of five files cleanly before token budget exhaustion). The independent oracle evaluates the test receipts and diff closure, returning **Partial** (40%). Funds settle across the four buckets:

1. **Bounty distribution:** Alice earns $40\% \times 400 = 160$ CR in earned revenue; the remaining 240 CR unearned bounty returns to Bob’s wallet.
2. **Collateral settlement:** Alice receives $40\% \times 100 = 40$ CR of her performance bond returned as collateral (not income); the remaining 60 CR bond is slashed and credited to the commons/salvage fund to onboard a successor.

Conservation check: $\Delta\text{Bob} = -160$, $\Delta\text{Alice} = +160 - 60 = +100$ net, $\Delta\text{commons} = +60$. The ledger sums to 0 net delta, and the separate escrow buckets $\text{escrow}_{\text{bounty}} \rightarrow 0$ and $\text{escrow}_{\text{bond}} \rightarrow 0$ clear completely. No value was created or destroyed.

Exercises

- E1.** Trace the flow of funds for a **Partial** settlement where the worker completed 40% of the evidence chain. Confirm Property 5.4.1 holds across the split (the dramatization above is one such trace; redo it for a bond of 250 CR and 70% completion).
- E2.** Why does closure bind to an oracle rather than a self-reported note? Name the attack a free-text “Result:” note enables.
- E3.** ★ The four states assume the acceptance criteria were *right*. Design a *float-plan amendment* protocol for the common case where the criteria themselves turn out wrong mid-flight (re-sign by both parties, escrow top-up/refund delta), preserving Property 5.4.1. (Gap, §5.14.)

5.5 Three sides on one escrow

The novel construction is that the rental and licensing sides ride the *same* float-plan escrow, so conservation holds globally without a second ledger.

Operator-for-hire (labor). The operator is the requester’s counterparty. It *sub-escrows* a bond for each fleet agent it dispatches (the bond ledger is already per-agent, so this needs no new primitive). Its profit is bounty $- \Sigma(\text{slashed sub-bonds}) - \text{ledger fee}$. The operator internalizes its fleet’s risk.

Asset rental (capital). The lease fee is a line item in the float plan. By **incomplete-contracts / property-rights theory** (Grossman & Hart 1986 [191], *when contracts cannot specify every contingency, the residual control right should go to the party making the most important non-contractible investment*), the *renter* must hold the *runtime* control right — the ability to sandbox, throttle, or revoke the leased agent mid-task — while the *owner* bears the reputational consequence.

Key idea. This is the cleanest economic argument for the runtime **jail** — the per-agent tool-allowlist plus scoped filesystem that the safety layer enforces: the jail is not just safety hygiene, it is the *residual control right that makes leasing an agent contractible at all*. Here we must respect the series’ *capability vs. permission* distinction (§5.6): the jail allocates *capability* (what the leased agent *can* do at runtime). The deontic layer separately governs *permission* (what it *may* do). “Able but forbidden” must stay expressible; the jail bounds ability, not norm.

Skill licensing (IP). The per-use fee is released to the licensor *only on green settlement* (metered + clawback). This is forced by **Akerlof’s market for lemons** (1970 [93], *when buyers cannot observe quality, price collapses to the average and good goods exit the market*): a skill’s quality is unobservable before use, so a flat upfront license drives good skills out. Metering + clawback + a Merkle *portfolio proof* (the skill’s prior settled outcomes, anchored in the evidence chain) restore the seller’s incentive to ship quality.

5.5.1 One trade, all three sides, one settlement

The running example of §5.3 ran the three sides *in the alternative* — Bob buys the refactor as labor, *or* as capital, *or* as IP. C2 claims something stronger: that all three settle on the *same* escrow object, in one transaction, without a second ledger. That claim is only worth anything if the arithmetic closes, so here it is, closed.

The three-sided settlement, dramatized. Bob posts one float plan for the 1,200-line refactor, with three line items. *Labor*: a 400 CR bounty to Alice’s firm. *Capital*: a 50 CR lease for Dana’s profiling specialist, which Alice’s crew needs for one module. *IP*: Carol’s lint-and-type skill, metered at 0.5 CR per file against a 20-file cap, so 10 CR is reserved. Bob’s wallet is debited $400 + 50 + 10 = 460$ CR into escrow. Alice separately posts a 100 CR performance bond, also into escrow. Before any agent spawns, the escrow row holds 560 CR.

The job settles **Success**: the test suite passes and the diff merges. The metered licence bills only the 16 files the skill actually touched, so $16 \times 0.5 = 8$ CR releases to Carol and the unused 2 CR returns to Bob. Alice’s bond is refunded as collateral, not income.

Wallet	Net delta	Why
Bob (requester)	−458	−460 escrowed, +2 unmetered licence returned
Alice (labor, side 1)	+400	bounty; bond −100 in, +100 back, nets 0
Dana (capital, side 2)	+50	lease fee on the leased specialist
Carol (IP, side 3)	+8	16 files metered at 0.5 CR
Commons	+0	nothing slashed on a Success settlement
Sum	0	escrow $560 \rightarrow 0$; supply unchanged

Check it on one line: $-458 + 400 + 50 + 8 + 0 = 0$. Three sides, three privately-informed counterparties, three different reputation keys — and one escrow row that opens at 560 CR and closes at zero. Property 5.4.1 is not re-proved per side; it is the *same* invariant, because the lease and the licence are line items on the one float plan rather than trades on ledgers of their own. *Now you try*: settle the same plan **Partial** at 40% and confirm the sum is still zero. (Alice earns 160 and is slashed 60 of her bond; the commons gains that 60; the licence still meters at 8 because the files it touched were touched.)

Exercises

- E1.** Map each of Grossman–Hart’s two parties (residual-control-right holder vs. reputational-consequence bearer) onto the rental side’s roles. Which one is the Arbiter jail?
- E2.** Show that adding a lease line item and a metered-license line item to a float plan cannot violate Property 5.4.1, no matter the amounts.
- E3.** ★ A skill’s portfolio proof for v1 says nothing about v2. Specify a *version-binding* for the Merkle portfolio proof so a new version starts with a discounted inherited prior — the newcomer policy, but for code.

5.6 The keystone the market rests on — and does not yet have

Everything above assumes a counterparty’s identity is real. This section confronts the series’ single *blocking* reconciliation: the identity primitive the whole market leans on covers a threat model that *excludes the cross-operator adversary the market is defined by*.

Figure 5.4 draws the two stones apart: the local root this paper leans on, and the cross-operator stone it does not yet have.

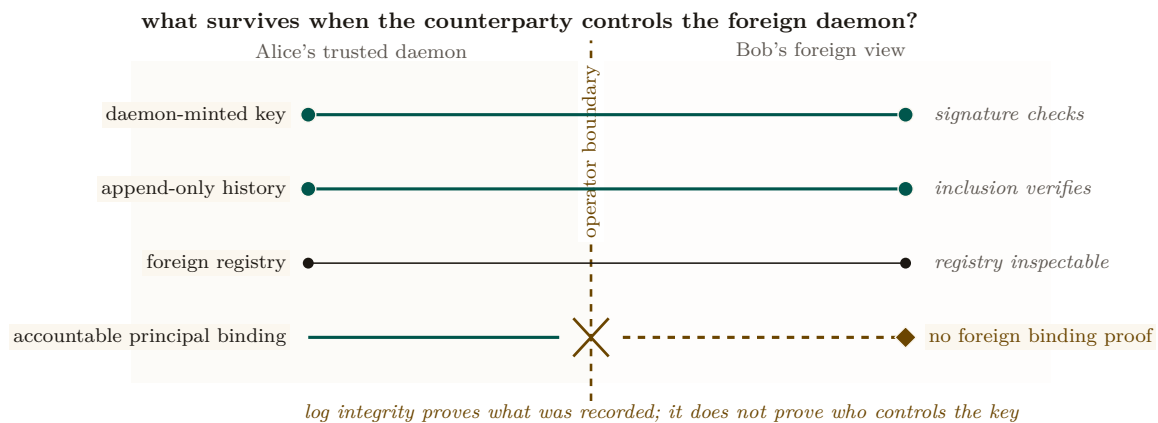


Figure 5.4: Visibility crosses the boundary; accountable-principal binding does not. Bob can verify Alice’s key signature, append-only history, and registry entry. Those proof spans continue across the boundary. The decisive bottom span ends: none establishes that the foreign key belongs to one distinct principal rather than a sock-puppet set. The missing capability is therefore a boundary proof, not another local database field.

5.6.1 Local non-forgable identity is the keystone — for one operator

Definition 5.6.1 (Local non-forgable identity). *Local non-forgable identity* is a per-agent identity that no agent can edit, guaranteed *within one trusted daemon*: the daemon is the only component that can hold state agents cannot reach, so it is a trusted root by construction. (The reference system now ships a partial slice — daemon-minted **actor-souls**, lookup credentials, and a bounded newcomer pool — while full write-boundary enforcement remains open. Papers 1–3 rely on the complete contract; here we only consume it.)

For a single-operator harbor this is exactly right and is correctly named the highest-leverage keystone. Its threat model, stated plainly, is “a lazy or self-interested agent in a fleet the operator owns” — not a hostile peer. It is explicitly *not* a public-key infrastructure spanning strangers.

5.6.2 But its threat model does not reach the market

Pitfall. Local non-forgable identity defends against an agent inside a fleet the operator owns; it explicitly does *not* claim to defend against a hostile operator, and it is not a cross-party PKI. The market, by contrast, is *defined* by mutually-distrusting operators trading across a boundary neither controls. You cannot make the single-operator root the foundation of the cross-operator market by assertion. Across the boundary, the counterparty’s daemon *is* the adversary that local identity declares out of scope. The witness log gives *log integrity*, not *identity binding*: it does not answer “who vouches that Harbor *B*’s signing keys map to a real, distinct principal, rather than a hundred sock puppets Bob minted this morning?”

Definition 5.6.2 (The cross-operator attestation problem). *Cross-operator attestation* is the problem of binding agent signing keys across harbors you do not own: an actual key-distribution and attestation story for the cross-operator threat model — something that lets Alice’s harbor trust that a key presented as “Bob” really is the one principal she has been building a reputation history against. **Status:** SPECIFIED-to-*proposed*; it has no implementation and only a partial specification for its critical part. It is the highest-leverage unbuilt keystone, and it gates the entire market thesis.

Key idea. This paper *stops* describing the federation boundary as one “neither controls” while quietly resting on a single trusted root. Either cross-operator attestation is built (a real attestation story), or the market’s scope is restated as “federation among *semi-trusted* operators.” We take the first horn as the designed target and flag every claim that depends on it.

5.6.3 The principal above the actor

The economic counterparty in every trade is the **principal** (the human or org owning a fleet), not the agent. Bonds, reputation inheritance, and sanctions must key on the principal via the hop-bound authorization chain, or Sybil bond-farming works by spawning fresh *agents* under one principal (**Douceur**’s Sybil attack [106]: *free identities defeat any reputation or quorum system*). Defining the principal as a first-class economic entity is the cleanest Sybil fix; its cryptographic binding is the identity object specified in *From Spawn to Person* and consumed here.

Exercises

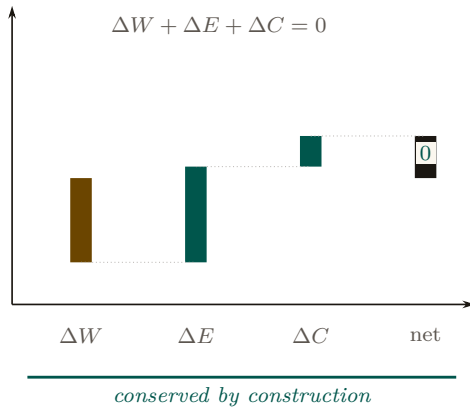
- E1.** In one sentence, state why a reputation system layered on forgeable pseudonyms is *no mechanism*, not merely a weak one (cite Douceur).
- E2.** Local non-forgeable identity has a non-goal: “no defense against a hostile operator.” Name the exact word in the market’s definition (“mutually-distrusting,” “boundary neither controls”) that this non-goal contradicts.
- E3.** ★ Specify a cryptographic binding tying N agents to one principal such that spawning fresh agents does not reset the bond floor, and that survives into the cross-operator world (where the single-operator “trusted credential” shortcut is unavailable). This is the hard half of cross-operator attestation.

5.7 Conservation under composition

The economy’s single cross-boundary invariant is that internal ledger operations redistribute value rather than create or destroy it. This section states the compositional claim directly and separates it from cross-currency valuation.

Figure 5.5 draws the composition claim: sequential traces glue, and internal operations carry zero external balance change.

A. native unit closes exactly



B. conversion introduces priced terms

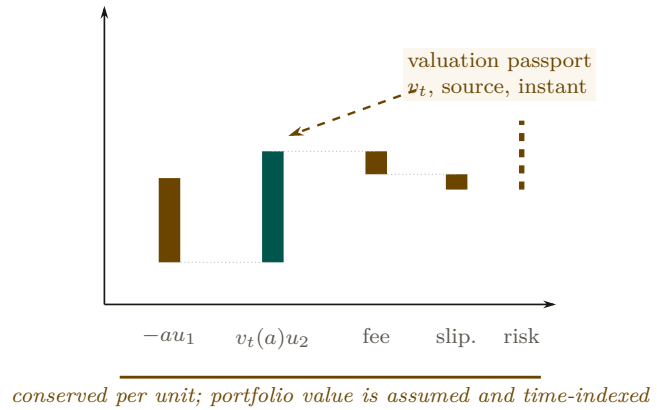


Figure 5.5: Conservation closes inside a native unit; conversion requires a named valuation boundary. Panel A is an accounting waterfall: wallet, escrow, and commons deltas return exactly to zero. Panel B cannot close by adding unlike units. It must record the valuation function and instant, plus fee, slippage, and open exposure. Composition preserves native-unit balances; notation does not manufacture a unit-free portfolio total.

Let a ledger state be $x = (W, E, C)$ and let a valid transaction trace $f : x \rightarrow y$ carry external balance change $\Delta(f) = S(y) - S(x)$ for $S(x) = W + E + C$. Sequential traces compose by concatenation and satisfy $\Delta(g \circ f) = \Delta(g) + \Delta(f)$. Internal escrow, refund, and slash traces have $\Delta = 0$; top-ups and withdrawals are explicit boundary flows. This is the entire conservation claim. Category language can describe trace composition, but it does not add a currency-conversion theorem.

Theorem 5.7.1 (Single-unit conservation composes). *Suppose all harbors use one unit of account and every cross-harbor transfer is a matched atomic move from source escrow to either destination settlement or source refund, with an explicit in-flight bucket. Then any finite composition of internal and cross-harbor operations has total external balance change equal to the sum of its recorded top-ups and withdrawals. In particular, a trace containing only internal transfers has $\Delta = 0$. (SPECIFIED; proof sketch in Appendix 5.B.)*

Property 5.7.1 (Cross-currency valuation boundary). With heterogeneous units, exact conservation is defined per native unit. A scalar portfolio total additionally requires a valuation map v_t at a named instant t , plus explicit fee and slippage accounts. If rates move between legs, the mark-to-market difference is economic exposure, not a failure of functoriality. Bounding that exposure by φ requires a separate oracle-latency and price-move assumption that this paper has not proved. (*open.*)

Conservation is exact per unit of account. A cross-currency scalar is a valuation, and every valuation must name its rate source, time, fees, and exposure bound. “Lax functor” is not a substitute for those data.

5.7.1 The Myerson–Satterthwaite corner

Strict conservation (Property 5.4.1) is exactly *budget balance*. That is not free.

Theorem 5.7.2 (Conditional bilateral-trade boundary). *For any bilateral slice of this market satisfying the Myerson–Satterthwaite assumptions—independent private buyer and seller values drawn from overlapping continuous supports, Bayesian incentive compatibility, interim individual*

rationality, and budget balance—no mechanism is also *ex-post* efficient everywhere [180]. This theorem constrains such a slice; it does not by itself prove incentive compatibility of the harbor mechanism or identify the desideratum sacrificed by the full three-sided market.

The current design establishes a conservation goal and voluntary entry; it has not proved truthfulness. An AGV-style mechanism changes the incentive and participation notions and generally offers expected rather than *ex-post* balance. Comparing those alternatives requires a fully specified type and transfer model, which remains open.

Exercises

- E1. State Myerson–Satterthwaite in one sentence and list the assumptions that must be checked before applying it to one harbor trade.
- E2. Give a two-currency example in which native-unit conservation holds but the marked-to-market scalar changes because v_t moves.
- E3. ★ Specify an oracle-latency and price-move model that yields a valid φ exposure bound for Property 5.7.1.

5.8 The Anchor Protocol proper: cross-harbor capability transfer

The *cross-harbor capability-transfer ceremony* is the market’s identity-level hot path: how an authorization minted under Alice’s harbor becomes usable, safely, inside Bob’s. *The Anchor Protocol* analyzes the cryptographic substrate (a hop-bound authorization chain checked by protocol models plus a bounded Rust verification layer); *this* section is the ceremony that rides that substrate across a boundary.

authority shrinks while destination-local verification takes over

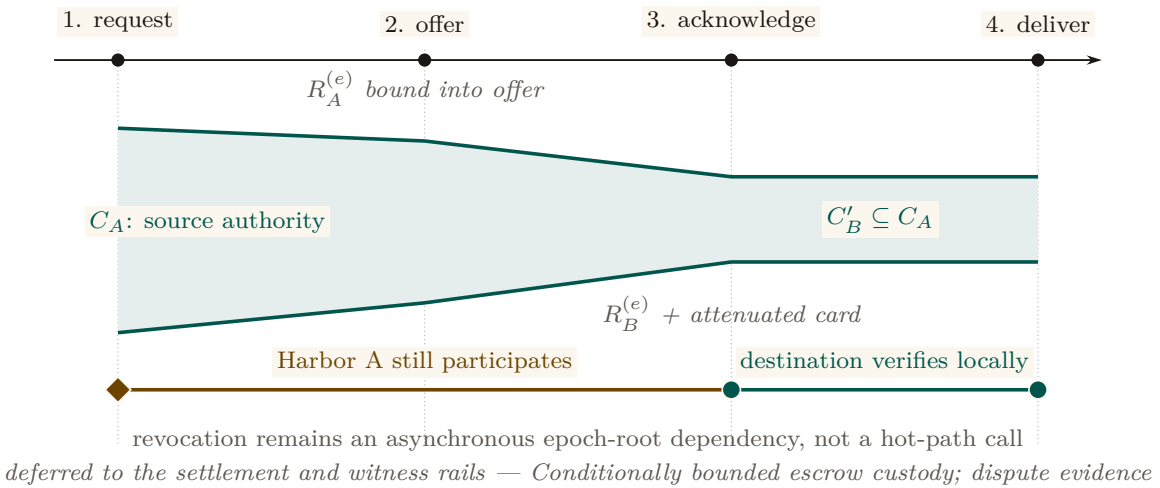


Figure 5.6: The transfer ceremony attenuates authority and then removes source reachback from the hot path. The green envelope narrows from C_A to $C'_B \subseteq C_A$ as the signed offer and acknowledgement bind source and destination epoch roots. After message 3, Harbor B can verify and deliver the attenuated card locally. Harbor A still matters asynchronously: a later epoch root can revoke the transfer once that root reaches B’s audit surface. Transfer therefore neither copies full authority nor makes every use synchronous.

The four-message ceremony (Figure 5.6, written out as Protocol 5.8.1) has one decisive liveness property: after message (3), no further synchronous interaction with Harbor A is needed. The transferred card C'_B is verifiable under Harbor B’s key alone, so federation does not turn every

authorization into a synchronous inter-machine round trip. The epoch root $R_A^{(e)}$ bound into the envelope is the governing primitive: a revocation issued by A after epoch e invalidates C'_B as soon as the new root $R_A^{(e+1)}$ reaches the witness log that B audits.

Protocol 5.8.1 (Cross-harbor capability transfer). **Parties:** Alice’s agent (holds card C_A), Harbor A (Alice’s daemon), Harbor B (Bob’s daemon), Bob’s agent (verifier). **Pre:** B audits a witness log to which A publishes epoch roots.

1. Alice’s agent $\rightarrow A$: $\text{xfer_req}(C_A, B)$ — present C_A , name target harbor B , request a transferable form.
2. $A \rightarrow B$: $\text{xfer_offer}(C_A, \text{att}, R_A^{(e)})$ — A signs an envelope binding C_A , an attenuation predicate att (which may only *narrow* authority), and its current epoch root $R_A^{(e)}$.
3. $B \rightarrow A$: $\text{xfer_ack}(C'_B, R_B^{(e)})$ — B verifies A ’s key via the witness log, applies its *own* attenuation, and mints $C'_B = \text{att}_B(\text{att}_A(C_A))$, valid only at B .
4. $B \rightarrow \text{Bob’s agent}$: $\text{xfer_deliver}(C'_B)$. Every later presentation verifies under B ’s key alone — no A -side lookup on the hot path.

Post (liveness): no synchronous A interaction after step (3). **Post (safety):** authority only ever narrowed; replay against a stale $R_A^{(e)}$ is rejected; a revocation by A kills C'_B once $R_A^{(e+1)}$ reaches B ’s witness log.

The ceremony, dramatized. Alice’s specialist a_2 holds a card C_A that authorizes “read and write `src/payments/`, run the test suite.” Bob rents a_2 for an afternoon. Alice’s harbor offers an attenuated envelope: att_A strips write access to everything except the one module Bob hired for. Bob’s harbor, not trusting Alice’s word, attenuates *again*: att_B adds “no network egress, six-hour TTL.” The minted C'_B is the intersection — read/write one module, run tests, no network, expiring at dusk — and it verifies under Bob’s key, so a_2 never has to phone home to Alice mid-task. At 5 p.m. Bob’s revocation fires locally; at the next epoch boundary Alice’s harbor also sees, in the shared witness log, that C'_B was used exactly as attenuated and nowhere wider.

Key idea. Capability *attenuation* is the safety property here: a transferred card can only *narrow* authority — $C'_B = \text{att}_B(\text{att}_A(C_A))$ — never widen it. This is the cross-boundary form of the attenuation-monotonicity that the hop-bound authorization chain checks within one harbor (*The Anchor Protocol*). It is what makes “rent my agent for an afternoon” safe to say to a stranger.

Pitfall. Do not let the federation theorems borrow the substrate’s “formally verified” halo. The hop-bound authorization chain has bounded symbolic and Rust evidence in *The Anchor Protocol*. The *federation* claims are design arguments plus a bounded TLA^+ extension; they are SPECIFIED, not deployed. Two different confidence levels, never blurred.

Exercises

- E1.** Why is the ceremony four messages and not two? Identify what message (3) buys (Hint: who must counter-sign, and under whose key does C'_B later verify?).
- E2.** Trace what happens to an in-flight C'_B when Alice rotates her signing key between messages (3) and a later presentation at B . (Gap: cross-boundary key rotation, §5.14.)
- E3. ★** Specify replay protection across the boundary: is the *anchor id* a nonce? Does B reject a re-presented message (3)? State the freshness invariant.

5.9 Federation: trading across machines you don't own

Federation is where the leaderless distributed canon actually bites. Crucially, the federation design *refuses consensus* rather than trying to route around Fischer–Lynch–Paterson — which is the correct reading of the impossibility the next box states.

Figure 5.7 draws the topology this section defends: independent harbors linked only by gossip and a shared witness log, with no global consensus step.

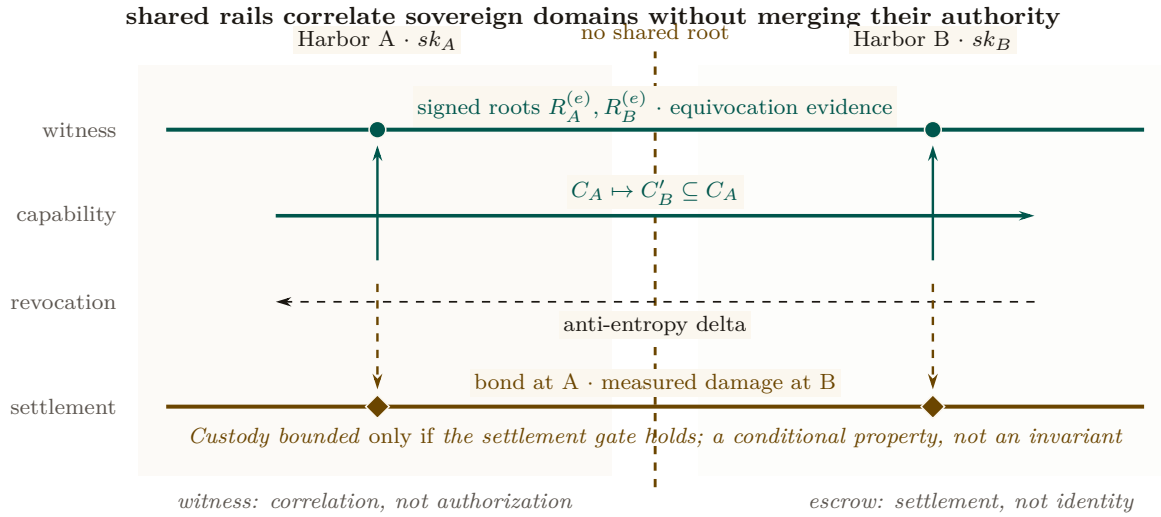


Figure 5.7: Federation is four independent functions aligned across two sovereign domains. Each harbor keeps its own signing root. The witness rail correlates signed epoch roots and exposes equivocation; it does not authorize work. The capability and revocation rails carry attenuated authority and later invalidation in opposite directions. The settlement rail relates a bond posted at A to damage measured at B, but its custody bound remains conditional. No rail makes either harbor a root of authority for the other.

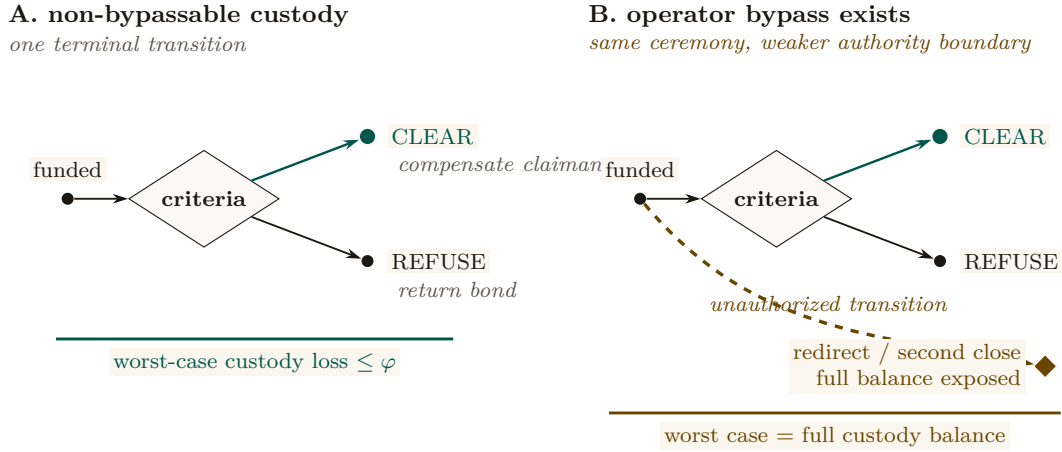
5.9.1 No consensus, by design

Key idea. A system that *needs* agreement on a global event order across operators it does not control is dead on arrival, by **Fischer–Lynch–Paterson** (1985) [140] (*no deterministic protocol achieves consensus in an asynchronous network with even one crash failure*). Federation declines to require agreement. It requires only (a) append-only per-harbor logs, (b) gossip-bounded propagation, and (c) *detectable* (not prevented) equivocation. This is a **Certificate Transparency** posture [32]: you do not prevent a misbehaving log from lying, you make lying detectable after the fact and expensive via a slashable bond.

The convergence and detection bounds smuggle in *partial synchrony* (**Dwork–Lynch–Stockmeyer** 1988 [45], the canonical escape from FLP): “gossip every Δ ” is an eventual-synchrony assumption, and we state it as such rather than burying it.

5.9.2 Settlement and the cross-chain-deals lineage

Figure 5.8 places this construction against the atomic-swap and adversarial-commerce baselines it is compared to below.



Custody Assumption: whitelist + fee cap + one terminal transition are the theorem's hypothesis, not decoration

Figure 5.8: The custody bound is the absence of an extra transition. Panel A permits exactly one terminal move from funded custody to **Clear** or **Refuse**; with a recipient whitelist and fee cap, worst-case loss is bounded by the agreed fee φ . Panel B adds one operator-controlled bypass. The visible extra edge reaches redirect or a second close, exposing the full balance. Threshold signing is only one candidate implementation; the loss theorem holds only when the three authority restrictions are actually non-bypassable.

The escrow construction is honest about its trust: it does *not* claim trustless settlement. Its loss bound is conditional, and the condition is the whole content of the claim:

Property 5.9.1 (Bounded custody loss, conditional). *If the custody ledger is non-bypassable — exposing only a recipient whitelist, a fee cap, and one atomic terminal transition, with no operator path around any of the three — then a counterparty's worst-case loss on a cross-harbor settlement is the pre-agreed fee φ , and the outcome space partitions into exactly **Clear** or **Refuse**. If that authority boundary *can* be bypassed, the worst-case loss is instead the full balance in custody. (SPECIFIED, conditional: no runtime conformance check establishes the hypothesis for a deployed system; 2-of-3 threshold signing is one candidate way to arrange it.)*

This is a deliberate departure from the hash-timelock baseline (**Herlihy** 2018 [142], atomic cross-chain swaps), and the right comparison is the formal model of mutually-distrusting commerce without a shared chain: **Herlihy, Liskov & Shrira** (2022, *Cross-Chain Deals and Adversarial Commerce*) [141]. The harbor's bounded escrow is a “trusted third party / watchtower” point in that design space; we locate it there explicitly rather than reinvent it.

Remark. Open problem: trustless cross-harbor settlement. It is unknown here whether non-fungible, reputation-priced bonds admit a useful trustless settlement protocol under the paper's availability and privacy requirements. An HTLC comparison does not prove impossibility because the asset and adjudication models differ. (*open.*)

5.9.3 Revocation gossip and the equivocation race

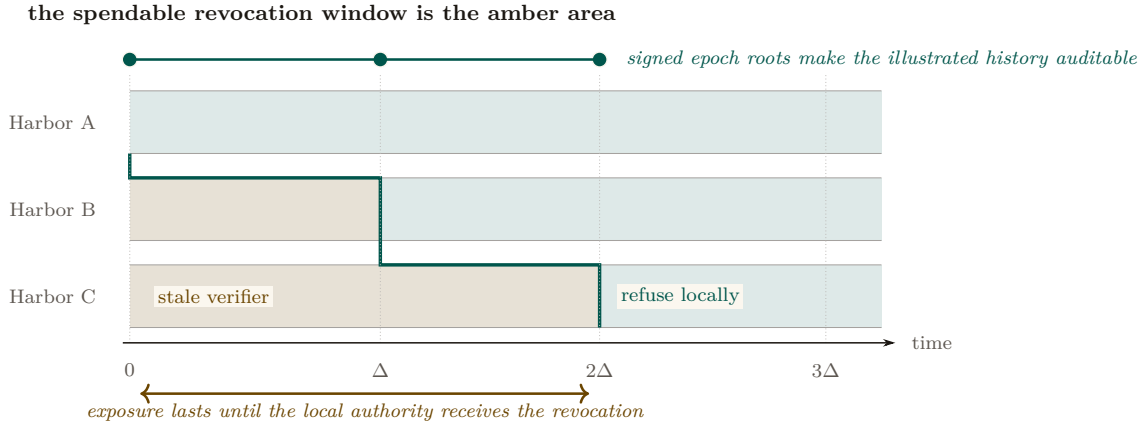


Figure 5.9: Revocation propagation is a moving staleness frontier. Alice refuses immediately; B remains stale until Δ and C until 2Δ in this illustrated execution. The amber area is precisely where a revoked capability can still be admitted by the named local authority. The teal staircase marks synchronization, not a global clock. Expected $\Theta(\log m)$ dissemination requires connected reliable rounds; a partition can extend the amber area without bound and therefore destroys any finite deadline.

Federated revocation propagates by anti-entropy gossip (Demers et al. 1987 [1]) with a probabilistic-membership filter (a cuckoo filter — a compact structure that answers “is this card revoked?” with no false negatives and a tunable false-positive rate). The federation therefore has expected $\Theta(\log m)$ -round dissemination only under a connected reliable-round overlay with the stated randomized peer-selection model. This asymptotic expectation is not an exact constant or a partition-tolerant deadline.

Protocol 5.9.1 (Federated revocation gossip). **State:** each harbor X keeps a revocation filter F_X and a current epoch root $R_X^{(e)}$ published to a shared witness log.

1. *Revoke.* An operator revokes card c at its home harbor A : $F_A \leftarrow F_A \cup \{c\}$; A advances to $R_A^{(e+1)}$ and publishes it.
2. *Gossip.* Every Δ , each harbor picks a peer and exchanges filter deltas (anti-entropy). A card present in either filter becomes present in both.
3. *Audit.* Any third party compares each harbor’s published roots in the witness log; a harbor that publishes two contradictory roots for one epoch is *equivocating*. Verification is constant work once both signed roots are co-located; delivery to an auditor has the overlay’s separate dissemination delay.

Post, conditionally: in the connected reliable-round model, every honest harbor learns the revocation in expected $\Theta(\Delta \log m)$ time. Under an unbounded partition there is no finite global-learning bound.

The revocation race, dramatized. Alice revokes a_2 ’s card c at $t = 0$ (say she discovers it was misbehaving). Harbor B , which rented a_2 , has not yet heard: for the interval $[0, \Delta]$, B ’s filter is stale and c is still spendable there. At $t = \Delta$ gossip reaches B , which adds c to F_B and refuses further use. Harbor C , two hops away, learns at $t = 2\Delta$. This is one illustrated execution; a different schedule or partition produces a different window. Conjecture 5.9.1 says the bond Alice posted on c must dominate whatever a_2 could spend at B inside that window, or a well-capitalized Bob could rationally race the gossip.

Pitfall. The threat is *adversarial*, not stochastic. An attacker who controls peer selection or partitions the gossip graph (an *eclipse attack*, **Heilman et al.** 2015 [74]) breaks the expected-time guarantee. The honest statement is conditional: a finite service-level bound needs eventual connectivity and a known post-stabilization delivery bound; epidemic analysis then supplies an expectation within that model, not an adversarial deadline.

Detection-after-the-fact deters only if the slashed bond exceeds the value extractable during the detection window. This is the analogue of the *cleanup lower bound* (the companion result that a cleanup bond must cover the worst-case cost of the damage it underwrites), and we state it as a target:

Conjecture 5.9.1 (Equivocation Lower Bound). *For a deployment that advertises a finite freshness window T , the witness bond must dominate the maximum value spendable against a revoked-but-not-yet-observed capability over T . If partitions are unbounded, no finite bond covers the worst case; the system must instead fail closed on stale roots or cap spend independently. (open; analogue of the cleanup lower bound.)*

Figure 5.10 bands the window Conjecture 5.9.1 must price against the connectivity assumptions each band needs.

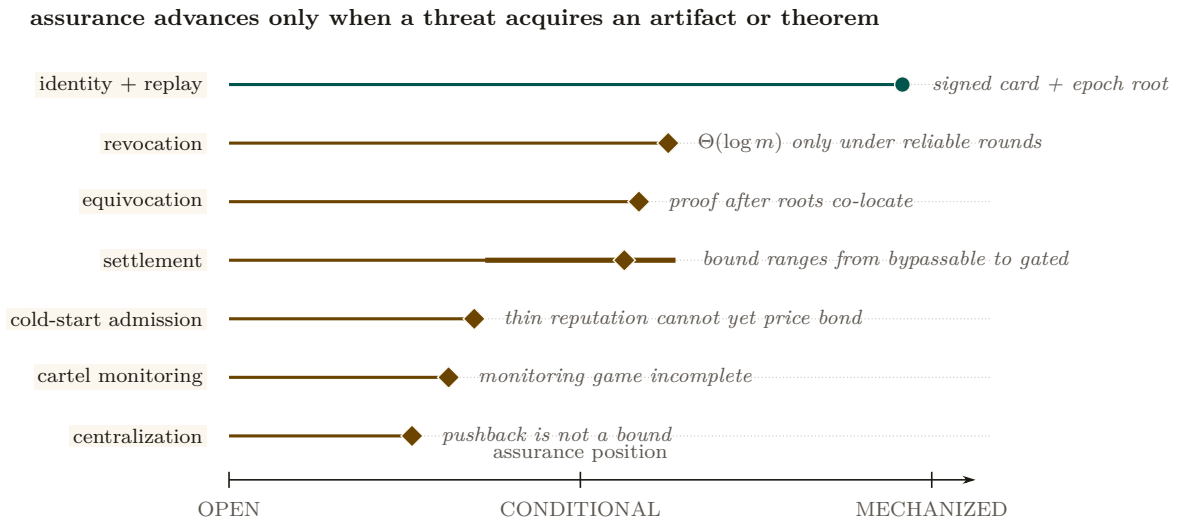


Figure 5.10: The assurance landscape is uneven and falsifiable. Position encodes how far each threat has moved from an acknowledged gap toward a mechanized artifact. Identity and replay have the strongest concrete evidence. Revocation, equivocation, and settlement are conditional on named connectivity or custody hypotheses; the thick settlement interval shows how widely its assurance moves if the gate is bypassable. Cold start, cartel monitoring, and centralization remain open. A row advances only when the missing artifact or theorem exists.

Exercises

- E1.** State the connectivity, peer-selection, and delivery assumptions needed for expected $\Theta(\Delta \log m)$ dissemination. Explain why none yields a deadline during an unbounded partition.
- E2.** Trace a revoked capability through Figure 5.9: at which harbor, and at what time, can it still be spent? That window is what Conjecture 5.9.1 must price.
- E3. ★** State and (attempt to) prove the Equivocation Lower Bound for a three-harbor mesh under a fixed gossip period Δ and a single equivocating witness.

5.10 Reputation: monotone, but revocable

Reputation is the third side’s existence condition. Two reconciliations matter here.

5.10.1 “Never ends” is not an anti-revocation property

A tempting design slogan holds that reputation “never ends” — no decay window an agent can outrun — while the same design also requires that a fraudulent settled outcome be retractable. A property and its required violation cannot both be requirements.

Key idea. The reconciled statement: **reputation is monotone in *honest* witnessed outcomes, but a witnessed outcome is itself revocable via a propagating tombstone.** Dissemination has the same model and partition caveats as capability revocation. “Never decays” applies to honest outcomes only; it is *not* an anti-revocation property.

We also decline to assert no-decay as a virtue: bounded memory is a design parameter (Liu & Skrzypacz 2014, *Limited Records and Reputation Bubbles* [178], show unbounded records create reputation *bubbles*, not stability). The horizon is a tunable, and ∞ is rarely optimal.

5.10.2 Why Sagas-style compensation does not restore reputation

It is tempting to fix revocation with compensating transactions (Garcia-Molina & Salem 1987, *Sagas* [101]), as the bond ledger does. But the analogy fails structurally:

Proposition 5.10.1 (Compensation does not erase reliance). *A ledger can append a compensating transaction that restores a numerical balance. A reputation tombstone cannot undo decisions counterparties already made while relying on the invalid outcome. It can only change future evaluations after the tombstone is observed. Reputation repair therefore requires dissemination plus an explicit remedy for past reliance; it is not equivalent to ledger compensation.*

This is why Sagas appears in the bond-settlement prior art but *not* in the reputation prior art.

5.10.3 The grading-oracle keystone

Every folk-theorem incentive-compatibility claim in the market bottoms out in “the daemon derives the grade.” But the grade is produced by a capturable evaluator (Figure 5.11).

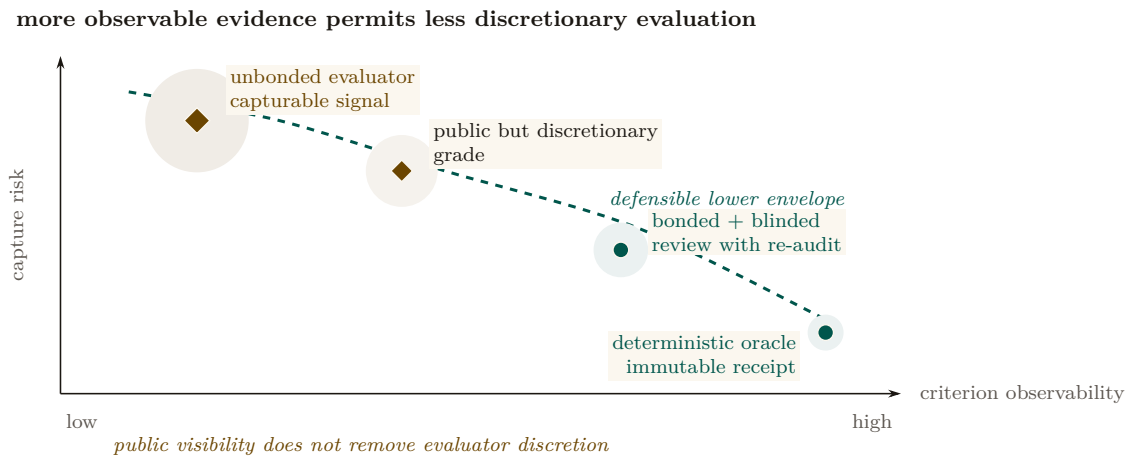


Figure 5.11: Evaluation mechanisms occupy a frontier, not four boxes. Horizontal position is criterion observability; vertical position is capture risk. Halo size redundantly marks discretionary exposure. Deterministic receipts sit at the low-risk observable end. Where judgment is unavoidable, blinded bonded review with sampled re-audit moves toward the lower envelope without pretending to become deterministic. Publishing an unconstrained evaluator’s grade does not make the signal exogenous or strategy-proof.

Theorem 5.10.1 (The monitoring model must include the evaluator). *An imperfect-public-monitoring equilibrium result requires a specified conditional distribution of public signals given modeled actions and strategies. If a strategic grader chooses the signal but is omitted from the game, that distribution is not fixed by the model and the claimed equilibrium result is unsupported. One must either constrain the signal mechanically or include the grader, its information, payoffs, and deviations as part of the game [56].*

The two resolutions (both used). *Fix A — mechanically constrained evidence:* ground settlement in evidence the agent cannot directly author (a CI-issued test result, a protected merge event, or an independently evaluated policy check). This narrows the signal channel; it does not make the task metric universally strategy-proof. *Fix B — bond-and-slash the judges:* where machine-checking is impossible (aesthetics, ambiguous quality), the human/LLM judges get their *own* bond and their *own* reputation; a judge later contradicted by re-audit is slashed. The “rate-the-raters” recursion must then be shown to terminate at a machine-checkable base, not regress infinitely. De-biasing for LLM judges (order-swap, pairwise, family-exclude) follows **Zheng et al. 2023** [132].

A captured judge, dramatized. Bob disputes a settlement: he claims Alice’s a_2 delivered an “untasteful” refactor. Taste is not machine-checkable, so Fix B applies: Judy, a human grader, is drawn to arbitrate. Judy posts a 50 CR judge-bond and rules *for Bob*. But Judy and Bob are colluding — Judy is a captured judge, the exact failure Theorem 5.10.1 warns of. The market’s defense is the re-audit lane: a sampled second panel later reviews Judy’s ruling against the preserved evidence, finds it indefensible, and *contradicts* it. Judy’s 50 CR bond is slashed and her judge-reputation takes a tombstone; Alice is made whole. The signal became exogenous again not because Judy was honest but because lying was bonded and the re-audit made it expensive — and the recursion stops at the second panel, whose own evidence is machine-checkable (the same diff, the same tests).

Exercises

- E1.** State the signal-model assumption Theorem 5.10.1 needs and explain why an omitted strategic grader leaves it undefined.
- E2.** Classify three settlement criteria as machine-checkable (Fix A) or judge- dependent (Fix B): a unit test passing; a merged SHA; “the refactor is tasteful.”
- E3. ★** Prove the rate-the-raters recursion terminates: define a well- founded order on “levels of judge,” grounded at machine-checkable oracles, and show no infinite ascending chain of “judges judging judges” is needed.

5.11 Hosted trust is the product

The 2025–26 agentic-payments landscape — **AP2** (Google, signed payment mandates), **ACP** (OpenAI/Stripe), and **x402** (Coinbase, stablecoin settlement over HTTP) — has commoditized the *settlement rail*. Wiring an agent to pay another agent is now table stakes. This is *good news* for the thesis, because it sharpens what the product actually is.

What remains scarce, and therefore defensible, is **hosted trust**: the verified ledger (conservation-checked, Merkle-anchored evidence), the *relay* (the harbor mesh carrying public-key attestation across machines), and the *reputation* that makes a counterparty you do not own legible and accountable.

- **Substrate (swappable):** the payment rail. Use x402/AP2/credits/ Stripe interchangeably.
- **Product (the moat):** attestation, federation membership, and reputation hosting.

The revenue model aligns incentives without perverse over-penalty: a transaction fee (2–5% of settlements), a small listing fee (collusion deterrent), and a small bond-spread on *forfeited* bonds (kept small so the platform never profits from over-slashing). The platform should earn most from *successful* trade, because successful trade is what hosted trust produces.

As §5.2 put it, before any of the machinery was on the page:

You don't sell crypto — crypto is the substrate. Here is what is scarce: attestation + federation membership + reputation.

5.12 Cold start, and the auction question

A three-sided market has a *triple* cold-start problem: no operators shop an empty fleet shelf; no owners list agents with no renters; no licensors publish skills with no buyers. Two-sided-market theory prescribes: subsidize the side carrying the scarcest cross-side externality, then flip. Phase 1 seeds supply at zero/negative price with platform-posted tasks at above-market bounties (generating the first settled outcomes — the bootstrap reputation data); Phase 2 imports external work history for partial credit to break the new-agent freeze; Phase 3 flips to charging the demand side once a liquidity threshold (not a calendar date) is met. Figure 5.12 draws the schedule against the quantity that actually gates it, because “threshold, not date” is the kind of phrase that stays a slogan unless you can see what it is a threshold *on*.

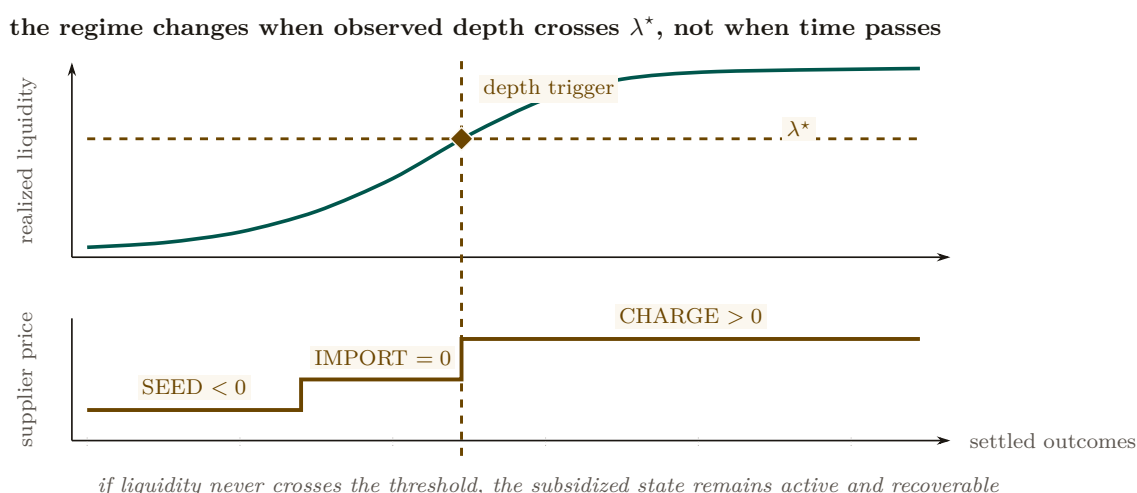


Figure 5.12: Cold start separates the measured market state from the policy schedule. The upper trace is realized liquidity over settled outcomes. The lower trace is the supplier-price regime applied to that state: seed supply below zero, credit imported work at zero, then charge only after liquidity crosses λ^* . The shared vertical trigger makes the dependency explicit. Calendar time is absent, and a market that never crosses the threshold does not silently advance to the charging regime.

5.12.1 Static escrow vs. competitive underwriting

Figure 5.13 draws the static-bond and competitive-underwriting alternatives side by side.

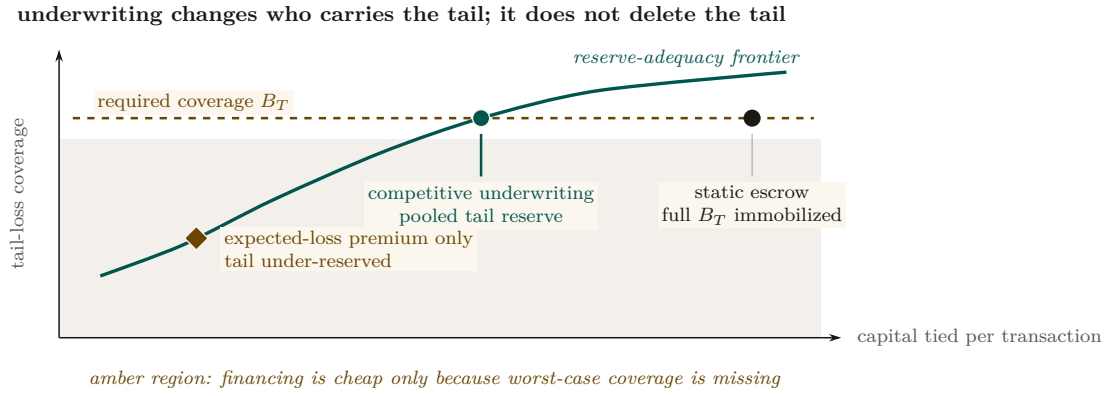


Figure 5.13: Static escrow and competitive underwriting occupy different points on a reserve-adequacy frontier. Static escrow immobilizes the full per-transaction bond. Adequately reserved underwriting can reduce capital tied to one transaction by pooling the tail, but the insurer must still carry that tail in its reserve or reinsurance. The amber expected-loss-only point is cheaper because it violates the required coverage. Coordinates are schematic; the structural comparison is the claim.

Thomas Youle’s competitive-insurance proposal — insurer-agents bid to underwrite transactions, replacing a static bond parameter with a market-discovered price — is a possible *fourth* side. But it may collide with the cleanup lower bound:

Proposition 5.12.1 (Underwriting reframes the bond as reserve adequacy). *The cleanup lower bound requires bond $\geq$ worst-case cleanup cost c . A competitive insurance market prices to expected loss, which sits below the worst case for any below-tail agent. Either underwriting violates the cleanup lower bound (the commons can leak in the tail), or the bound must be reframed as the insurer’s capital-adequacy constraint (a reserve-requirement framing with tail reinsurance), not a per-transaction bond. The form of the answer is determined; only the reserve formula is open.*

5.12.2 Cartels and wash-trading

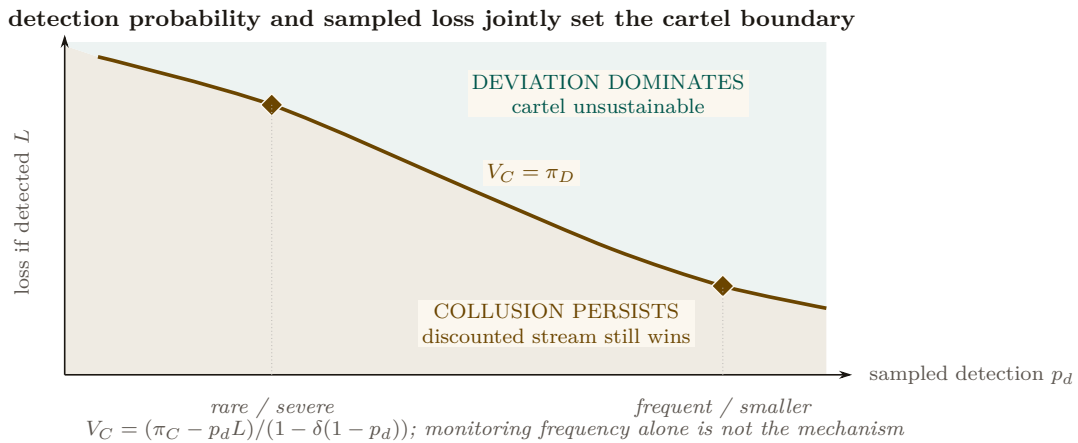


Figure 5.14: Cartel sustainability is a two-policy phase diagram. The boundary is the repeated-game equality $V_C = \pi_D$. Below it, the discounted collusive stream still dominates a one-shot deviation. Above it, deviation wins and the cartel is unstable. A rarer severe sampled loss and a more frequent smaller loss can lie on the same deterrence boundary. Detection frequency by itself is therefore not the mechanism; the product enters through the full discounted payoff expression.

Multi-homing plus portable reputation enables cross-harbor *cartels* — a rating-agency cartel is the closest real-world analogue: several independent principals colluding to hold a shared reputation signal artificially favorable rather than competing on it. And a colluding requester+worker can

wash-trade fake settled outcomes to mint reputation cheaply. The defense form — cost-per-settled-outcome must exceed reputation’s marginal value — is circular as stated, because that marginal value is endogenous to the same market. The fix is a structural break: make settled outcomes between a counterparty *pair* exhibit sharply diminishing reputation returns (a concavity that caps the wash-trade payoff regardless of price), plus sampled adversarial re-audit of high-velocity pairs.

Whether a cartel of colluding operators can actually hold its rating floor is a *repeated* game, and that is what Figure 5.14 draws. Each round a member either *colludes* (holds the agreed floor q_{floor} , collecting the cartel payoff π_C) or *defects* (undercuts the floor by ε for a one-time gain π_D , after which the cartel unravels and everyone drops to the competitive payoff $\pi_N \approx 0$). A **grim-trigger** strategy is the crudest way to punish that: *cooperate until the first observed defection, then defect forever*. It sustains the cartel only while members weight the future heavily enough — that weight is the **discount factor** $\delta \in (0, 1)$, the value a member places on next round’s payoff relative to this one’s, so a member with δ near 1 is patient and one with δ near 0 will take π_D today and let the cartel die. Sustaining collusion means the discounted stream of colluding payoffs beats the one-shot deviation, which is exactly the inequality the figure’s box states, and δ clearing its threshold is exactly what the defenses above are trying to prevent.

Exercises

- E1.** Which side should Phase 1 subsidize, and why? Apply the “scarcest cross-side externality” rule.
- E2.** Use Figure 5.14 to identify the grim-trigger condition under which the cartel is self-sustaining. What raises the critical discount factor δ enough to break it?
- E3.** ★ “Price of anarchy when reputation is for sale” is the wrong tool: once reputation is tradeable, you have changed the game, so PoA (a ratio for a *fixed* game) does not apply. Reframe it as a *signal-existence* question (Spence-signaling with a resale market): does *any* informative separating equilibrium survive reputation resale?

5.13 A gluing analogy for the open federation problem

Three open problems—cross-harbor settlement, equivocation under partitions, and cross-currency valuation—share a local-to-global shape. That resemblance motivates an analogy; it does not yet define a sheaf or a cohomology theory.

Conjecture 5.13.1 (Formalization target, not established result). *A future treatment may define a site of overlapping administrative domains and a presheaf of signed ledger views, then ask when compatible local sections admit a unique global section. This paper has not specified the site, restriction maps, coefficient object, or cocycles; therefore it makes no claim that a particular H^1 is defined or non-zero. The immediate engineering obligations remain the concrete ones: matched custody transitions, freshness policy under partitions, and time-indexed valuation.*

Fong and Spivak’s compositionality program [30] suggests vocabulary for a future formalization. Until the site and coefficient data are defined, the paper uses “gluing” only to organize engineering questions.

Exercises

- E1.** Name the site, cover, restriction maps, and coefficient object a genuine sheaf model of federated ledger views would require.
- E2.** Explain why two incompatible signed roots are evidence of equivocation but are not, without further definitions, a cohomology class.

- E3. ★** Build a small formal gluing model for a bounded gossip topology and state exactly what consistency result it proves.

5.14 Gaps the design must still close

Honesty requires naming what is unspecified. These are not threats already defended; they are holes.

1. **Multi-dimensional reputation aggregation.** “Accuracy/aesthetics/ efficiency judged by separate experts” has no specified combination rule. You cannot have “separate axes” and “one buyer-readable score” without specifying an aggregation function and its trade-offs. (*proposed.*)
2. **Float-plan amendment under scope drift.** The four states assume the acceptance criteria were right. Mid-flight re-specification needs a re-sign protocol with escrow top-up/refund delta, preserving conservation.
3. **Cross-boundary key rotation.** How a rotated signing key invalidates or re-validates cross-harbor delegations already held by B is unspecified.
4. **Skill versioning.** Reputation attaches to a content hash; v2 starts cold. The lemons problem reappears at every version bump.
5. **Operator exit / harbor death.** Escrow orphaning, in-flight float plans, and reputation tombstoning on ungraceful exit are unspecified — the market-level analogue of the kernel’s crash-recovery problem.
6. **Incentive-compatible arbitration capacity.** The human oracle is scarce and expensive; at scale arbitration is itself a priced, bondable, slashable service that must converge to honest rulings, not rubber-stamping.

These six are not equally dangerous, and an implementer triaging them should not have to infer the stakes from the prose. Each one threatens a specific critical promise made earlier in this paper:

Table 5.3: Which promise each gap threatens. The two gaps that reach the *conservation* invariant are the urgent ones, because conservation is the only claim in this paper graded **implemented** — a hole there falsifies something that is currently true, rather than deferring something that is not yet true.

Gap	Threatens	How it bites
Operator exit / harbor death	conservation	orphaned escrow rows have no settlement path, so wallet + escrow + commons stops summing to supply
Float-plan amendment under scope drift	conservation	a mid-flight re-spec moves value with no matching debit unless the top-up/refund delta is escrowed
Cross-boundary key rotation	cross-boundary safety	a delegation already held by B may verify against a key A has retired
Skill versioning	Sybil-resistance	a v2 content hash is a fresh identity, so the lemons problem and the whitewash both reappear at every version bump
Multi-dimensional reputation aggregation	incentive compatibility	an unspecified aggregation rule is an unspecified target, and any fixed scalar collapse re-creates a Goodhart objective
Incentive-compatible arbitration capacity	incentive compatibility	Fix B (§5.10) prices one judge; it does not price a judge pool under load, where rubber-stamping is the cheap strategy

5.15 Related work

The design borrows from seven literatures, and its only originality is in how it *joins* them; this section consolidates the threads cited piecemeal above.

Multi-sided platform markets. The frame is Rochet & Tirole [107, 108] and Armstrong [139]: a platform is multi-sided when the *allocation* of price across user groups, not just its level, moves volume. We extend the standard two-sided picture to three sides by counting *incentive constraints*, and we read the contemporary AI-labor-market model of Chiu, Zhang & van der Schaar [44] as confirmation that “reputation system as a third side” is the natural structure.

Mechanism design and its impossibilities. Settlement is a direct mechanism in the sense of Myerson [181]; the binding constraint is Myerson–Satterthwaite [180], which forces the market to surrender efficiency to keep budget balance, incentive compatibility, and individual rationality. The incentive-compatibility arguments are imperfect-monitoring folk theorems (Green–Porter; Abreu–Pearce–Stacchetti [56]), whose exogenous public signal we cannot take for granted — hence the grading-oracle problem. Grossman–Hart [191] supplies the residual-control-right argument for the rental side, and Akerlof [93] the lemons argument for licensing.

Reputation systems. Empirically, online reputation has measurable value (Resnick et al. [166]; Tadelis [192]); theoretically, unbounded records create reputation *bubbles* rather than stability (Liu & Skrzypacz [178]), which is why our horizon is a tunable. The Sybil attack [106] is the reason reputation must key on a principal, not a spawn. LLM-as-judge de-biasing follows Zheng et al. [132].

Distributed systems and cross-chain commerce. Federation refuses consensus because of FLP [140], recovers a weaker guarantee under partial synchrony [45], propagates by epidemic gossip [1], and takes a Certificate-Transparency detect-don’t-prevent posture [32, 31], mindful of eclipse attacks [74]. The settlement design is located explicitly against atomic cross-chain swaps [142] and the adversarial-commerce model of Herlihy, Liskov & Shriram [141]; reputation revocation is contrasted with Sagas-style compensation [101].

Applied category theory. The trace-composition notation and proposed gluing formalization draw vocabulary from ologs [58] and Fong & Spivak [30]; no cohomology claim is made. The commons-governance reading draws on Ostrom [75], and the auction machinery on standard algorithmic game theory [159].

Agentic-commerce protocols. While this design was being written, the industry shipped the transaction layer of an agent economy. The Universal Commerce Protocol [212] (Google/Shopify, 2026) standardizes agent-mediated retail — discovery via a *.well-known* capability profile, a checkout state machine, server-selects version negotiation — and its rival, the Agentic Commerce Protocol [8] (OpenAI/Stripe), the platform-mediated equivalent. Both are deliberate about what they exclude: neither settles payments (delegated to payment service providers) and neither decides which agents deserve trust. Authorization proof is delegated to the Agent Payments Protocol [6], whose mandates — W3C Verifiable Credentials in SD-JWT form, chained principal → platform → merchant — are the deployed sibling of this paper’s float-plan countersign, and whose credential formats the reference implementation now adopts at the harbor boundary (ADR-0094) so that the keystone identity artifacts of §5.6 verify with off-the-shelf tooling. The gap those protocols leave open is this paper’s subject: published security analyses of UCP [63] note that it regulates *how* agents transact while offering nothing about *which* agents to trust — no collateral, no settlement oracle, no reputation, no priced deterrent. The harbor economy is a

mechanism for exactly that layer; the retail protocols are evidence the layer below it is arriving on schedule, and hosted trust (§5.11) is what they will need bolted on.

Interactive proofs and adversarial debate. The adversarial test market (§5.16) is a seventh thread, cited piecemeal there rather than above because it grounds a single additional mechanism rather than the market’s core structure. It is delegated computation in the sense of Goldwasser, Kalai & Rothblum [190] — a resource-bounded verifier need not redo a prover’s work — crossed with multi-agent debate [95], where two adversarial reasoners are more legible to a judge than one cooperative one. Becker’s deterrence condition [94] supplies the enforcement half: a test-writer’s expected forfeiture must match its gain from misreporting.

Table 5.4: How the harbor economy sits relative to the nearest prior art on each axis.

Axis	Nearest prior art	What the harbor does differently
Market structure	Two-sided platforms [107]	Three incentive constraints (labor / capital / IP) on one escrow object
Settlement across distrust	Atomic swaps [142]; cross-chain deals [141]	Bounded (not trustless) escrow for <i>non-fungible, reputation-priced</i> bonds
Revocation	PKI revocation lists; Certificate Transparency [32]	Gossip-converged filters with a priced, named attacker window
Reputation revocation	Sagas compensation [101]	Tombstone propagation — no conserved quantity to compensate
Grading / IC signal	Folk theorems [56]	Strategy-proof machine-checkable oracle <i>or</i> bonded-and-slashed judges
Agent transaction rails	UCP [212]; ACP [8]; AP2 mandates [6]	They standardize the transaction and name trust out of scope; the harbor prices and enforces the trust layer they delegate away
Purchased assurance	Delegated computation [190]; AI-safety debate [95]	Prices assurance as k independently bonded test-writers under a Becker-style re-audit rate, against a mutation-testing oracle

5.16 One further market: adversarial testing and purchased assurance

Beyond labor, capital, and IP licensing, the harbor supports one further internal market, and it is worth stating because it is the one place where *assurance itself* becomes a priced line item rather than a hope. In an **adversarial test market**, independent test-writing agents compete to find flaws in a proposed diff *before* it settles. The frame is an interactive proof system: delegated computation [190] (a prover convinces a resource-bounded verifier without the verifier redoing the work) crossed with multi-agent debate [95] (two adversarial reasoners are more legible to a judge than one cooperative one).

The adversarial test market, dramatized. Bob’s float plan ships a diff that one of Alice’s specialists wrote. Rather than trust Alice’s own test run — a signal whose author has every reason to flatter it — Bob posts a 30 CR bounty per confirmed flaw and opens the diff to $k = 3$ independent test-writing agents, each paid only for a defect it actually surfaces. If each catches a given real defect independently with probability $d = 0.6$, the chance a planted flaw survives all three is $(1 - 0.6)^3 = 0.064$. Bob has bought his defect risk down from 1 to about 6% for at most $3 \times 30 = 90$ CR, which is a number he can hold up against what the bug would have cost him in production. Raise k to 5 and the survival probability falls to $(0.4)^5 = 0.010$; that is the whole

shape of the mechanism — assurance is bought in units of k , and each unit costs one bounty plus the carry on one bond.

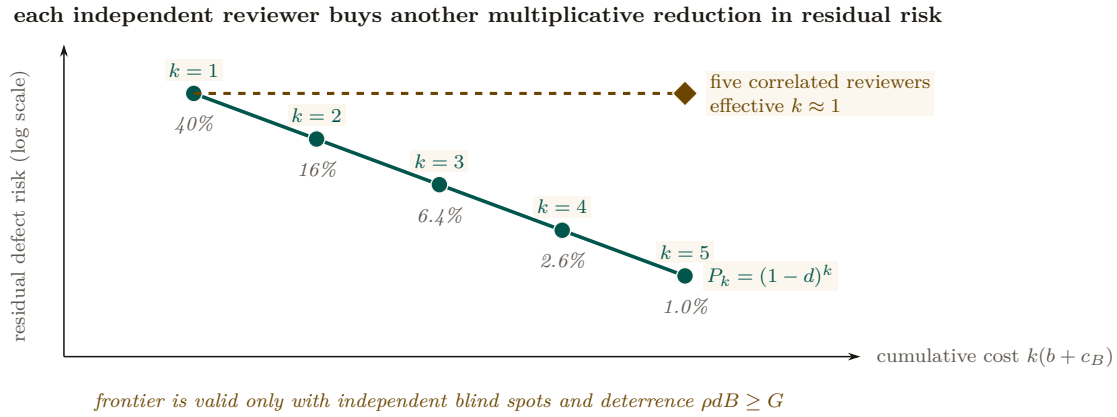


Figure 5.15: Purchased assurance is one risk–cost trajectory. Each teal point adds one independently bonded reviewer: cumulative cost rises linearly while residual defect risk falls geometrically. At $d = .6$, three reviewers leave 6.4% residual risk and five leave about 1%. The dashed amber counterfactual charges for five correlated reviewers but delivers roughly one reviewer’s protection. Independence and Becker deterrence $\rho dB \geq G$ are therefore conditions of the frontier, not footnotes to headcount.

A **Becker-enforced panel** is a set of k independent, conflict-free test-writing agents, each detecting a given flaw with probability $d \in (0, 1)$, each posting a slashable bond B to enter the market, and each facing a platform re-audit rate ρ against a one-shot misreporting gain G . The panel size k and the detection probability d fix what the panel catches; the triple (G, B, ρ) separately fixes whether a test-writer is honest at all. The two questions are independent, so the paper states them as two results rather than one bundled claim.

Theorem 5.16.1 (Purchased Assurance Bound). *Let an author-agent submit a float-plan implementation with a proposed diff, and let a Becker-enforced panel of size k review it. Then the probability that a critical flaw survives undetected across all k reviewers is bounded by*

$$\Pr(\text{defect undetected}) \leq (1 - d)^k. \quad (5.1)$$

The operator’s assurance level $1 - (1 - d)^k$ is then a purchasable commodity priced at $k \cdot (\text{bounty} + \text{bond carry})$, which is what makes “hosted trust as a service” a quantifiable line item rather than a slogan.

Property 5.16.1 (Becker deterrence for the panel). Truthful effort is a dominant strategy for a Becker-enforced panel’s test-writers exactly when expected punishment at least matches the one-shot gain from cheating — the **Becker deterrence condition** [94]:

$$\rho dB \geq G, \quad (5.2)$$

equivalently, the platform must commit to a re-audit rate at or above the critical rate $\rho^* = G/(dB)$. (SPECIFIED; this half is imported, not proved here — see the paragraph below.)

The deterrence condition in Property 5.16.1 is not proved here; it is imported. The companion technical paper *Reputation is Amortized Verification* [185] derives $\rho^* = G/(dB)$ as the stage-game threshold of a bonded inspection game — the same game this paper plays in §5.10 when it bonds and slashes a judge — and works it at the program’s running parameters: $G = 10$, $d = 0.8$, $B = 50$ give $\rho^* = 10/(0.8 \cdot 50) = 0.25$. Audit one report in four and the cheat payoff is exactly zero. Double the bond to $B = 100$ and ρ^* halves to 0.125: capital and vigilance are substitutes along the frontier $\rho dB = G$.

Pitfall. Quote the threshold without naming your settlement convention and you will be reproducibly wrong by about 17%. Equation (5.2) is the *keep-gain* convention: a caught cheat forfeits B but keeps G . If the escrow also claws the gain back, the forfeiture is $G + B$ and the critical rate drops to $\rho_c^* = G/(d(G + B)) = 0.2083$ at the same parameters. A platform whose rail does claw back but which budgets audits at $G/(dB)$ is over-auditing; one that does not claw back but budgets at $G/(d(G+B))$ is under-detering. The convention is a property of the slashing contract, not a modeling taste [185].

Pitfall. The $(1 - d)^k$ bound is exactly as good as the word *independent* in its hypothesis, and independence is the assumption a real test market breaks first. Three test-writers that read each other's public findings, or that are three instances of the same model family with the same blind spot, do not give k independent draws — they give something closer to one draw and two echoes. Purchased assurance therefore has to buy *diversity* (distinct principals, distinct model families, sealed submissions) and not merely count k ; otherwise the priced assurance level is an overstatement of the delivered one. The same requirement appears in [185] as the sealed multi-clique sampling hypothesis, where it is likewise required rather than hygienic.

The ground-truth oracle for grading the test-writers themselves is **mutation-testing kill rates** against held-out syntactic mutation operators, which stops agents from overfitting tests to a suite they can already see. This is a Fix A oracle in the sense of §5.10: machine-checkable, and not authored by the party being graded.

(*proposed.* The mutation-testing oracle and the bounty-per-killed-mutant contract are specified here for the first time in this series; neither ships, and neither appears in the reference implementation's ledger module. The deterrence inequality it leans on is SPECIFIED, proved in [185]. Recorded as such in Table 5.5.)

5.17 Conclusion

The harbor-master makes a swarm legible and accountable for one operator (the wedge of *The Legible Swarm*). When operators sail out to trade, the *same* ledger that kept one operator's swarm honest becomes the market that lets fleets who don't trust each other work together: three sides, one bond, hosted trust. We have not papered over the cost of that claim. The keystone identity primitive — local non-forgable identity — covers a single-operator threat model and must be extended to cross-operator attestation before the boundary is formally one “neither controls.” Conservation composes exactly within each unit of account; cross-currency totals require time-indexed valuation and an explicit exposure model. Reputation is monotone in honest outcomes and revocable by tombstone, not by Sagas. Every folk-theorem result rests on a grading process included in the monitoring model. Myerson–Satterthwaite constrains bilateral slices only when its assumptions hold; the paper does not yet prove which corner the full market sacrifices. The bones are good and the floor is built; the spine the market trades on has partial local identity and outcome substrates but no complete reputation or cross-operator binding — and this paper says so in the same words throughout, because a market built on an overstated keystone is a market built on sand.

Chapter Appendices

5.A Implementation & status

The body of this paper is deliberately implementation-agnostic. This appendix records, for the reader who wants to check the claims against running code, exactly which mechanisms are shipped in the reference implementation, which are specified with a proof or a written protocol, and which are still proposed. The single-line summary: an **implemented** conserving bond ledger, an **implemented** continuity organ (episodic memory), a PARTIAL checkpoint, and a richly SPECIFIED-and-model-verified protocol stack — but the spine the market trades on (cross-operator identity $\rightarrow$ outcome ledger $\rightarrow$ reputation) is PARTIAL-to-proposed. Today the harbor economy is a two-sided market with a proof-backed roadmap to three.

5.B Proof sketches

We sketch the three central results; full proofs of the conservation results appear in *The Federated Harbor*.

Proof sketch of Theorem 5.7.1. For a valid trace $f : x \rightarrow y$, define $\Delta(f) = S(y) - S(x)$. For composable traces $f : x \rightarrow y$ and $g : y \rightarrow z$,

$$\Delta(g \circ f) = S(z) - S(x) = [S(z) - S(y)] + [S(y) - S(x)] = \Delta(g) + \Delta(f).$$

Every internal generating operation has zero delta because its debits and credits match, including a cross-harbor move through the explicit in-flight bucket. By induction on trace length, a composition of internal generators has zero delta; top-ups and withdrawals contribute exactly their recorded boundary amounts. $\square$

Boundary note for Property 5.7.1. Native-unit conservation follows by applying Theorem 5.7.1 separately to each unit. A scalar valuation $V_t(x) = \sum_j v_{t,j} x_j$ changes either when native balances change or when the valuation vector v_t changes. The second term is mark-to-market exposure. No finite φ follows without assumptions bounding rate movement, oracle delay, fees, and slippage; those assumptions are not supplied here. $\square$

Proof of Theorem 5.7.2 (the sacrificed corner). Restrict attention to a bilateral slice satisfying the assumptions stated in Theorem 5.7.2. The Myerson–Satterthwaite impossibility then rules out the simultaneous achievement of all four desiderata. The implication is conditional: before declaring which corner the harbor gives up, a mechanism must specify its types and transfers and prove its incentive and participation properties. The present paper has not done so, hence makes no stronger allocation claim. $\square$

What *The Bonded Commons* receives Work can now be proposed, bonded, graded, and settled against a named principal. The market’s contracts rest on promises this chapter states but does not prove: that evidence is admissible, that settlement conserves value, that remedy stays bounded. The next chapter proves them.

Table 5.5: Per-claim maturity, with concrete grounding in the reference implementation. “Module” names refer to source files in the implementation; “model” means a mechanized proof artifact; “property test” means a randomized test exercising the invariant.

Claim / mechanism	State	Grounding
Bond ledger: escrow, slash, commons pool	implemented	bond-ledger module (<code>lib/bonds.ts</code>)
Conservation	implemented	10k-trace property test
wallet + escrow + commons = supply		
No-spawn-without-bond	PARTIAL	escrow-first; runtime gate is a roadmap item
Episodic memory (continuity organ 1)	implemented	episodic-memory module
Resurrection / checkpoint (organ 2)	PARTIAL	passes notes, not state
Outcome ledger (organ 3 — reputation keys here)	PARTIAL	commitments, daemon-derived deadlines, oracle closure, API/CLI, and overdue monitoring ship; neutral grading and reputation binding do not ship
Local non-forgable identity (intra-harbor)	PARTIAL	actor-souls , lookup credentials, and a bounded newcomer pool ship; full write gating does not ship
Cross-operator attestation	SPECIFIED/ <i>proposed</i>	no spec for the critical part
Float plan + signed escrow ceremony	SPECIFIED	Protocol 5.4.1
Cross-harbor capability transfer (attenuation)	SPECIFIED	verified <i>as model</i> ; Protocol 5.8.1
Reputation estimator (Elo / Bradley–Terry / TrueSkill)	SPECIFIED/ <i>proposed</i>	no estimator shipped
Multi-dimensional reputation, neutral judges	<i>proposed</i>	no axis-aggregation spec
Three-sided market (labor / rental / licensing)	SPECIFIED	two-sided in reality
Skill licensing (metered + clawback + Merkle proof)	SPECIFIED	cross-harbor verifiability undeployed
Adversarial test market (Purchased Assurance Bound, Thm. 5.16.1)	<i>proposed</i>	no mutation-testing oracle and no bounty-per-killed-mutant contract shipped; the deterrence threshold it cites is proved in [185]
Federation: witness log, capability transfer	SPECIFIED	<i>The Federated Harbor</i>
Recipient-whitelisted escrow custody	SPECIFIED (conditional property)	requires non-bypassable authority; no runtime conformance
Cross-harbor conservation	SPECIFIED (property)	proof sketch (App. 5.B) + TLA ⁺
Federated revocation dissemination	SPECIFIED (conditional expectation)	no finite partition deadline; equivocation delivery <i>open</i>
Hosted-trust moat / rail separation	SPECIFIED (positioning)	not a code artifact
Competitive underwriting (4th side)	<i>proposed</i>	composition unresolved

The Bonded Commons

If men were angels, no government would be necessary. If angels were to govern men, neither external nor internal controls on government would be necessary.

JAMES MADISON, FEDERALIST No. 51, 1788



THE QUESTION THIS CHAPTER ANSWERS

Who may decide, on what evidence, with what consequence?

Bonds, witnesses, audits and appeals turn residual uncertainty into an institution; the ledger's conservation law is mechanically checked.

Who may decide, on what evidence, with what consequence? Attribution without consequence is only a diary; consequence without admissible evidence is arbitrary. This chapter turns the local writer into an institution by joining capability bounds, witnessed records, audits, appeals, and conserved settlement. R7–R11, R15, R17, and the R8 machine state exactly what has been proved, checked, or left conditional.

6.1 Introduction: The Trust Problem

Two autonomous coding agents share a repository. Agent A is asked to refactor the authentication middleware. Agent B, spawned ninety seconds later from a sibling worktree, is asked to add a rate-limiter to the same middleware. Neither knows the other exists. They both edit, both commit, both push; one of them gets a merge conflict it cannot interpret, retries with a hallucinated resolution, and corrupts the file. The operator returns from a coffee break to a working tree she does not recognize. This is not a hypothetical—it is the modal failure mode of every multi-agent setup we have inhabited. Existing frameworks do not address it: LangGraph, CrewAI, and AutoGen handle state graphs and role assignment competently; the Model Context Protocol standardizes agent-tool integration. None of them give Agent A and Agent B a way to know about each other. The problem is **trust**—specifically, trust as infrastructure rather than trust as per-transaction ceremony.

When Agent A delegates a subtask to Agent B, it implicitly trusts that B will not corrupt the shared state, will not exceed the scope of the delegation, and will not silently fail in a way that poisons A’s downstream work. When Agent B accepts the delegation, it implicitly trusts that A’s description of the task is accurate, that the resources A promised are available, and that A will still exist when B finishes. These implicit trust assumptions pervade every multi-agent interaction, and every one of them can be violated.

The standard response in security engineering is “zero trust”—never trust, always verify. But zero trust applied literally to agent coordination is paralytic. If every message requires cryptographic verification, every delegation requires capability negotiation, and every file access requires authorization—the coordination overhead overwhelms the work itself. Agents spend their context windows negotiating trust instead of solving problems.

Hobbes identified this dynamic in 1651 [206]. In the state of nature, without a common authority, rational actors cannot cooperate because the cost of verifying each counterparty’s intentions exceeds the benefit of cooperation. The solution is not to make individuals trustworthy—it is to establish a **sovereign** whose authority both parties accept *before* the interaction begins, so that the interaction itself can focus on the work. “Covenants, without the sword, are but words, and of no strength to secure a man at all.”

This paper argues that multi-agent systems need exactly this: a **commons authority** that provides trust as infrastructure, not trust as ceremony. Agents don’t negotiate trust per-transaction. They enter a commons where trust has already been established—structurally, evidentially, and economically—and they work freely within that trust envelope.

6.1.1 The Three Failures of Peer-to-Peer Trust

Peer-to-peer trust fails for agent systems in three specific ways:

It doesn’t scale. If n agents must establish pairwise trust, the system requires $O(n^2)$ trust negotiations. Each negotiation consumes tokens, time, and context window. With 8 agents, this is manageable. With 50, it dominates wall-clock time. With 1000 (a production agent fleet), it is infeasible.

It doesn’t survive death. When an agent crashes, its trust relationships die with it. A successor agent that resumes the dead agent’s work must re-establish trust with every counterparty from scratch. The trust investment is non-transferable because it was encoded in ephemeral bilateral state, not in durable infrastructure.

It doesn’t address misaligned principals. An agent can be perfectly “trustworthy” in the sense that it faithfully executes its instructions—while its instructions come from a principal whose interests are adversarial to the commons. Peer-to-peer trust evaluates the agent’s behavior; it cannot evaluate the principal’s intent. The agent is a loyal soldier in someone else’s war, with a spotless reputation.

6.1.2 Contributions

We present:

1. A reading of the coordination daemon as a **candidate correlating device** in the sense of Aumann [187]: claim recommendations are private signals drawn from a common-knowledge distribution, while incentive compatibility remains conditional on explicit obedience inequalities (Section 6.2, esp. §6.2.1).
2. A **three-layer trust architecture** (structural, evidentiary, economic) that does not depend on agent morality, shared values, or threat of termination (Sections 6.3–6.7).
3. A Sen-grounded **design warning** about decisive resource rights under private information, plus a separate completeness theorem for **advisory resource claims** (Section 6.5).
4. A **TLA⁺ specification** of the coordination lifecycle with crash recovery, verifying safety and liveness properties (Section 6.10).
5. The design of a **collateralized work contract** system (Float Plans) that prices agent risk and settles outcomes mechanically, transforming the moral question of agent trustworthiness into the economic question of adequate bonding (Section 6.7).

The security layer (agent authentication and capability delegation) is provided by the Anchor Protocol [80], an accompanying chapter. This paper builds the trust, governance, and economic layers on top of that cryptographic foundation.

6.1.3 Related Work in Crypto-Economic Bonding

The architecture sits at the intersection of four bodies of prior work, each of which we extend rather than replicate.

Bonded participation in distributed systems. Proof-of-stake consensus protocols (Casper [216], Tendermint [86], and the Ethereum 2.0 specification [87]) introduced *slashing*: validators post collateral that is forfeited on equivocation or liveness failure. The technique is the closest analogue to our bonded-commons settlement, and the proofs of soundness (a rational validator strictly prefers honest behavior when slash > deviation gain) are direct ancestors of §6.7. We depart from this lineage on three axes: (i) the adversary is co-located on a single machine, not a Byzantine network, so consensus reduces to a daemon write rather than a global commit; (ii) the bonded act is *coordination*, not validation—work that produces side effects in a shared file system, not signatures on blocks; and (iii) the principal is human-or-LLM mixed, so the mechanism must price two failure modes (malice and confusion) the blockchain lineage collapses into one.

Capability-based security. Dennis and Van Horn [109], Miller’s object-capability work [148], and the SPKI/SDSI line [47] provide the structural attenuation that we lift wholesale into the Anchor Protocol layer. Our contribution is composing capability attenuation with bonding: a capability without economic settlement still permits the holder to “correctly” execute a destructive action; bonding without capability bounding still permits unbounded blast radius before slash. Both layers are necessary.

Commons governance. Ostrom’s eight design principles for governing common-pool resources [77] predict that durable cooperation emerges only when boundaries, monitoring, graduated sanctions, and dispute resolution are co-located with the commons. The bonded commons satisfies these requirements as services of the daemon rather than as social institutions, and §6.5.3 renders the dispute-resolution and appeals path explicitly.

Cryptoeconomic mechanism design. Werbach [131], Buterin [217], and the broader DAO literature establish “code is law” as a posture toward enforcement. Our position is narrower and weaker on purpose: code is the *record*, not the law. Allocation decisions remain with the agents (Sen, §6.5); the daemon supplies the evidence and the settlement. This deliberate weakening is what makes coercive enforcement avoidable while keeping accountability intact.

Table 6.1 compresses the four paragraphs above into what is borrowed, what is departed from, and why.

Table 6.1: Every borrowed result is re-used at a smaller scale than its origin assumed: the adversary is co-located rather than networked, the bonded act is coordination rather than validation, and the record is evidence rather than law. Nothing in the four lineages is discarded — each is narrowed by one assumption, and the narrowing is what makes the composition deployable on one machine.

Lineage	What we borrow	What we depart from	Why
Bonded participation (PoS slashing)	Slash->-deviation-gain soundness argument	Byzantine networked adversary; validation as the bonded act	Adversary is co-located on one machine, so consensus reduces to a daemon write, not a global commit
Capability-based security	Offline attenuation; authority only ever narrows	Capabilities alone as the whole story	A correctly-scoped token still permits a destructive in-scope write; bonding prices what the wall admits
Commons governance (Ostrom)	Boundaries, monitoring, graduated sanctions, dispute resolution	Social institutions as the enforcement medium	The four functions ship as daemon services (§6.5.3), not as norms an agent must internalize
Cryptoeconomic mechanism design	Priced deterrence; market-discovered parameters	“Code is law” as an enforcement posture	Code is the <i>record</i> , not the law: allocation stays with agents (§6.5); the daemon supplies evidence and settlement

What is novel. The synthesis: bonded participation + capability attenuation + advisory (not enforced) coordination + immutable attribution, packaged as localhost infrastructure rather than a global protocol. We are not aware of a prior system that combines all four in this form; each component has precedent, while the composition targets auditable coordination and attribution that survives identity disposal. Its welfare effects remain conditional on the explicit payoff and monitoring models developed below. Figure 6.1 shows the composition.

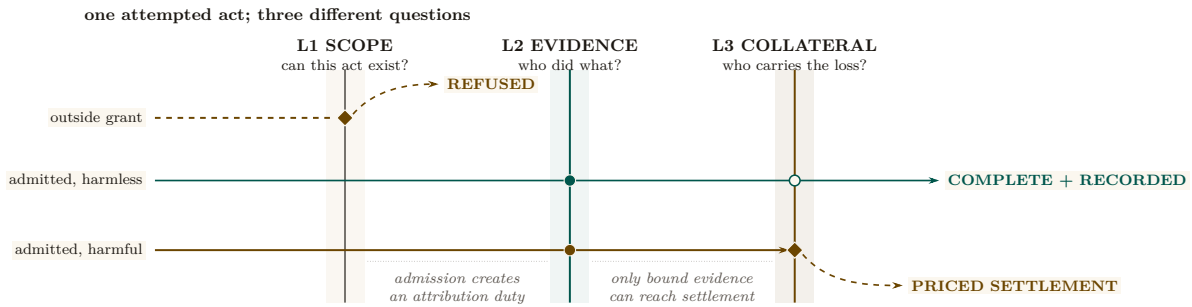


Figure 6.1: Three layers answer three different questions about the same attempted act. Layer 1 terminates an out-of-grant trajectory before shared state changes. Every admitted act crosses Layer 2 and acquires durable attribution. Layer 3 activates only for evidence-linked damage and prices the resulting loss. The distinct endpoints are the composition claim: refusal, attribution, and settlement are not interchangeable services.

6.2 The Commons Authority

Return to Agent A and Agent B from the introduction’s opening scene—the two agents who corrupted the same middleware because neither knew the other existed. Nothing about that failure required malice, and nothing about it would have been fixed by making either agent more careful. What was missing was a third party that both agents were already talking to: something that knew A had the file, could tell B so at claim time, and would still remember the whole episode after both processes exited.

That third party has to be settled *before* any interaction, not improvised transaction by transaction—otherwise establishing it is just the $O(n^2)$ trust negotiation of §1.1 under a new name. We call it the *commons authority*: one always-on service that every agent already trusts, precisely because it was there before any of them showed up. Five capabilities make it that, and the definition below names them.

Definition 6.2.1 (Commons Authority). A commons authority $\mathcal{D}$ for a multi-agent system is a persistent service that provides:

1. **Identity:** Every agent receives a cryptographic identity before participating.
2. **Capability bounding:** Every agent’s operations are structurally limited by attenuable tokens.
3. **Shared knowledge:** Resource claims, agent status, and conflict information are visible to all participants.
4. **Immutable history:** All actions are recorded in an append-only, tamper-evident trail.
5. **Economic settlement:** Work contracts are collateralized and settled against verifiable evidence.

The commons authority is not a central planner. It does not decide which agent should work on which task, which file conflicts should be resolved in whose favor, or which agents are “good.” It provides the *infrastructure* within which agents make those decisions for themselves, using the shared knowledge and economic incentives the authority maintains.

Remark. The commons authority is a *Hobbesian sovereign*, not an *omniscient coordinator*. Hobbes’s Leviathan does not micromanage citizens’ daily choices—it establishes the conditions (laws, enforcement, courts) under which citizens can trust each other enough to transact freely. Similarly, the commons authority establishes identity, capability bounds, shared knowledge, and economic settlement—and then gets out of the way.

In the Port Daddy implementation, the commons authority is a daemon running on `localhost:9876`, backed by SQLite. The simplicity of the implementation is deliberate: the authority’s value comes from its *guarantees*, not its complexity.

6.2.1 The Daemon as Correlating Device

Picture a crossing guard rather than a traffic light. A traffic light gives everyone the same instruction on a fixed cycle regardless of what is actually happening at the intersection; a crossing guard watches the specific cars and pedestrians in front of her and privately waves one through while holding another back. The daemon plays crossing guard, not traffic light: it sees the whole intersection (the claim graph) and issues one private recommendation per agent. But a crossing guard has no legal authority over anyone—drivers obey because obeying is better for them than the alternative, not because they must. Aumann’s test for whether such private recommendations actually change behavior, rather than being safely ignored, is the obedience condition below.

The daemon’s role therefore admits a precise game-theoretic test. Aumann [187] introduced the *correlated equilibrium*: a mediator draws a recommendation tuple $r = (r_1, \dots, r_n)$ from a joint distribution μ over action profiles and privately reveals r_i to player i . The distribution is a correlated equilibrium only if, for every player i , recommended action r_i , and unilateral deviation a'_i , the obedience inequality holds:

$$\sum_{r_{-i}} \mu(r_i, r_{-i}) [u_i(r_i, r_{-i}) - u_i(a'_i, r_{-i})] \geq 0.$$

Every mixed Nash equilibrium induces such a distribution, but a mediator does not create an equilibrium merely by issuing recommendations. The inequality is the governing condition.

The commons authority can play this mediator role. It observes the global claim graph and emits a recommendation pair on a contested file: *Agent α , proceed; Agent β , defer or coordinate*. A deterministic policy can still induce a non-degenerate μ through uncertainty over observed system states, but determinism is not itself evidence of obedience. Utilities, bond coverage, and continuation values must make the inequality above hold.

Property 6.2.1 (Conditional Correlating-Device Interpretation). Port Daddy’s daemon supplies the information structure of a correlating device for the agent-coordination game. It induces a correlated equilibrium only under the following conditions:

1. **Common-knowledge distribution.** The daemon’s recommendation policy—advise-claim, defer, back-off, settle-now, halt-on-conflict—is public, deterministic given the conflict graph, and reproducible from the evidence trail of §6.4. Agents do not need to trust the daemon’s motives; they need only verify the policy and observe its inputs.
2. **Private signal per agent.** On a contested claim, each agent receives only its own recommendation. Other agents’ recommendations are observed indirectly through the public conflict graph and the note trail, never read off the wire.
3. **Obedience inequality.** For every recommendation and deviation, the deviation gain is no larger than the expected bond loss plus discounted reputation loss. Conflict-graph completeness makes a deviation observable; it does not by itself make the deviation unprofitable.

The reframe therefore yields an audit obligation, not an automatic guarantee. Nothing physically stops an agent from ignoring advice. The repeated-game calculation in Section 6.7 verifies one stylized two-player payoff table; deployments with different utilities must re-check the obedience inequalities. The TLA⁺ model in Section 6.10 checks lifecycle safety and liveness, not strategic optimality.

No price-of-anarchy bound follows from this interpretation alone. The simulations of §6.7.5.8 compare one auction model with one static baseline; an analytical worst-equilibrium bound for the coordination game remains open.

Formal-backing status of the game-theoretic core. One disclosure belongs here, at the point the game theory enters, rather than buried in the appendix: in the maturity vocabulary of Appendix 6.A, this chapter’s game theory is *spec-only at the level of the research program*, whatever each individual artifact’s status in Table 6.7. This chapter’s game-theoretic core—the

correlating-device reading above, the Sen-inspired design warning (§6.5), the claim-signaling incentive-compatibility argument (§6.7.4), the competitive-insurance mechanism and its welfare comparison (§6.7.5.8), and the cartel folk-theorem analysis (§6.7.5.10)—has *no companion formal paper*. The structural and cryptographic half of this chapter is backed by *The Anchor Protocol* [80], which carries the ProVerif models and the attenuation proofs; the economic half is argued here, from scratch, and exists nowhere else in the program. What supports it is exactly what Table 6.7 lists—mechanized checks and Monte Carlo sweeps over supplied models, several of them PARTIAL by that table’s own grading—and not a peer-reviewable formal derivation a reader can be pointed to. Read this material at that maturity level.

6.2.2 Why Agents Consent

In Hobbes, consent to the sovereign is rational because the alternative—the state of nature—is worse. For AI agents, consent to the commons authority is rational because agents without it are strictly worse off:

Table 6.2: The commons authority converts six failure modes from ad hoc and manual to deterministic and automatic — without forbidding anything an agent could already do. Every row is a service the agent gains, not a permission it loses, which is why consent is rational rather than coerced.

Capability	Without Authority	With Authority
Port assignment	Race condition	Deterministic, collision-free
File coordination	Discover conflicts at merge	Detect conflicts at claim time
Crash recovery	Work lost permanently	Successor reads immutable trail
Delegation	Bilateral trust negotiation	Attenuated capability tokens
Reputation	Every interaction from zero	History persists across lifetimes
Accountability	Anonymous harm	Full attribution via evidence chain

The daemon does not need to threaten agents. It provides services that make coordination *rational*. Agents that bypass the daemon are not punished—they are simply worse off.

6.3 Layer 1: Structural Prevention

The first layer of trust does not depend on agent reputation or economic incentives. It depends on *walls*: structural authorization checks that reject out-of-scope operations wherever the runtime actually interposes them.

6.3.1 Capability Attenuation

The Anchor Protocol [80] provides Harbor Cards—Ed25519-signed capability tokens that bound the operations available to each agent. The critical property is **offline attenuation**: when Agent A delegates to Agent B, it creates a new token whose capabilities are a strict subset of its own. Agent B can further delegate to Agent C with even narrower capabilities. Capabilities can only shrink along the delegation chain, never grow.

The capability subset check and the secret-independent comparison are implemented in a **Rust core** (`harbor-card-rs`) that is compiled to a shared library and called from the Node.js daemon via FFI. Kani harnesses check that the parser and the comparator cannot panic on bounded symbolic inputs and exercise the subset check on two concrete vectors; the every-hop attenuation property is ProVerif’s, in *The Anchor Protocol*; runtime conformance tests cover the bridge. Compiler behavior, full interception coverage, and hardware side channels remain separate obligations.

The guarantee below is what security engineers mean by a *wall* rather than a *law*: not an instruction an agent could disobey, but a bound on which operations are even representable.

Definition 6.3.1 (Capability-Bounded Transaction). An agent transaction $\mathcal{T}_\alpha$ is capability-bounded by token τ_α if every operation o executed during $\mathcal{T}_\alpha$ satisfies $o \in C(\tau_\alpha)$, where $C(\tau)$ extracts the capability set from a Harbor Card. The commons authority rejects any operation $o \notin C(\tau_\alpha)$ at the verification step, before the operation can affect shared state.

Theorem 6.3.1 (Structural Scope Limitation). *For any delegation chain $\tau_0 \rightarrow \tau_1 \rightarrow \dots \rightarrow \tau_k$, where τ_0 is issued by the commons authority and each τ_{i+1} is attenuated from τ_i :*

$$C(\tau_k) \subseteq C(\tau_{k-1}) \subseteq \dots \subseteq C(\tau_0).$$

Within an execution that routes every relevant operation through the verifier, no accepted operation can lie outside the scope originally granted by the authority.

Proof. The Anchor model establishes a non-injective authentication correspondence for accepted modeled tokens and separately checks attenuation at the modeled chain depth [80]. Given those model assumptions and the theorem’s premise that every operation passes through the verifier, each accepted hop satisfies the subset relation; transitivity yields the displayed chain. The correspondence alone does not prove replay freedom, arbitrary-depth delegation, or complete runtime interception, which are separate obligations. $\square$

Remark. This is the layer where the metaphor of “walls, not laws” applies. A law says “you shall not write to the production database.” A wall means your token is not valid for production database writes, and no amount of intent—good or bad—can change that. Laws require enforcement; walls require only construction.

6.3.2 Isolation Domains

Beyond capability tokens, the commons authority supports **structural isolation** via git worktrees: physically separate copies of the repository where agents operate on disjoint filesystem state.

Definition 6.3.2 (Isolation Levels). Three tiers of isolation are available, ordered by strength:

- I_0 (**None**): Agents share a working directory. No conflict detection. Maximum parallelism, no safety.
- I_1 (**Advisory**): Agents share a working directory but register file claims with the commons authority. Conflicts are detected and reported to all participants. Claims are not enforced. This is the default.
- I_2 (**Structural**): Each agent operates in a separate git worktree. Filesystem-level isolation ensures agents cannot observe each other’s intermediate states. Conflicts are deferred to sequential merge.

The choice between I_1 and I_2 is not a security decision—it is an economics decision. Section 6.5 formalizes why.

6.3.3 Runtime Invariant Enforcement

Structural prevention is enforced at two levels: *statically* by the Anchor Protocol’s ProVerif-verified token exchange (all three protocol phases verified in ProVerif 2.05 [80], under the applied-pi-calculus methodology of Blanchet [40]), and *dynamically* by the **Arbiter**—an ambient security agent that subscribes to the daemon’s event stream and checks every state transition against six formally derived invariants. These invariants map directly to the safety properties of the TLA^+ specification in Section 6.10: **NoteMonotonicity**, **EscrowInvariant**, and **LockOwnerValid** are compiled from the formal model into synchronous runtime checks. Violations are logged immutably and, in strict mode, trigger the salvage protocol—the sovereign’s sword in code form.

Regimentation vs. Enforcement (OP-2). This design formalizes **Synchronous State Decidability** as the exact boundary separating pre-commit *regimentable* rules from post-commit *enforceable* rules, in the sense of Jones and Sergot’s regimentation/enforcement distinction for normative systems [16]: a regimented norm’s violation cannot occur at all, while an enforced norm’s violation can occur and is sanctioned on detection. If a rule can be evaluated purely against deterministic local DB state and the transaction payload, it is regimented (structurally prevented). If it requires async checks, external I/O, or non-deterministic oracles, it is relegated to post-commit enforcement (detected and salvaged).

6.4 Layer 2: Immutable Attribution

Structural prevention handles accidental harm—an agent cannot exceed capabilities it does not hold. But an agent *with* legitimate write access can write garbage. The second layer ensures that every action is permanently attributable.

6.4.1 The Evidence Chain

Every agent session maintains an **append-only note trail**—a sequence of timestamped, immutable records of observations, decisions, and progress.

Definition 6.4.1 (Evidence Trail). An evidence trail $N = \langle n_1, n_2, \dots, n_k \rangle$ is an ordered sequence of notes where:

1. Each n_i contains: content (natural language), type (note, decision, handoff, blocker), and timestamp.
2. Notes are append-only: there exists no API operation that modifies or deletes individual notes.
3. Notes survive session completion: the trail persists whether the session commits, aborts, or is abandoned due to agent death.

Lemma 6.4.1 (Trail Integrity). *The evidence trail is immutable by construction. No UPDATE or targeted DELETE operation exists in the system’s API surface. Notes are destroyed only by explicit administrative deletion of the parent session (CASCADE), which is an auditable action distinct from normal transaction lifecycle operations.*

Proof. By code inspection of the session module: the note insertion API performs only INSERT INTO session_notes. The only deletion path is DELETE FROM sessions WHERE id = ?, which triggers ON DELETE CASCADE. Since transaction completion (commit or abort) updates the session’s status field but does not delete the session row, notes survive all normal lifecycle transitions. $\square$

6.4.2 The Merkle Forest: Cross-Daemon Inclusion Proofs

For high-assurance environments and for fleets spanning multiple daemons, the evidence trail is strengthened to a three-level **Merkle Forest**. The single-daemon Merkle chain—each note hashing the previous, with a per-session root—is necessary but not sufficient: the moment two daemons exist, an auditor without database access still needs to verify that a particular note was included in a particular session using only a short proof.

Definition 6.4.2 (Merkle Forest). The evidence structure has three levels:

Note chain. Each note’s header includes $h_i = H(n_i \parallel h_{i-1})$ over SHA-256. The session’s note commitment is h_k , the hash of the last note.

Session root. All note hashes $(h_1, \dots, h_k)$ are assembled into a Merkle binary tree with root R_{session} . The chain commits to order; the tree commits to presence with $O(\log k)$ proofs.

Harbor root. Periodically (per-settlement, or hourly), every completed session root within a harbor is assembled into a harbor Merkle tree with root $R_{\text{harbor},\ell}$ at epoch ℓ , signed by the daemon’s Ed25519 key.

The collection $\{R_{\text{harbor},\ell}\}_\ell$ is the *Merkle forest*: one tree per epoch.

Inclusion proofs. To prove that note n_i from session s belongs to harbor h at epoch ℓ , the daemon emits: (i) the note inclusion path of size $O(\log k)$, (ii) the session inclusion path of size $O(\log N)$ where N is the number of settled sessions in the epoch, and (iii) the signed harbor root $\sigma_{\text{daemon}}(R_{\text{harbor},\ell})$. For $k = 100$ notes and $N = 10^4$ sessions, the total proof is $\approx 20 \times 32$ bytes + 64-byte signature ≈ 700 bytes—small enough to embed in any settlement receipt.

Witness to an abstract KMS. Each $R_{\text{harbor},\ell}$ is uploaded as a **witness** to a Key Management Service satisfying the properties enumerated in §6.12. The KMS enforces append-only monotonicity and periodically publishes a signed tree head, in the Certificate Transparency pattern [34]. Equivocation—publishing different roots for the same epoch to different parties—is detectable by auditors performing consistency proofs across tree heads.

Property 6.4.1 (Proof Soundness). If $\text{VERIFY-NOTE-INCLUSION}(n, s, h, \ell, \pi)$ returns **true** for some proof π , then either (a) n was included in session s at epoch ℓ , or (b) the daemon’s signing key has been compromised, or (c) SHA-256 has been broken. Under standard assumptions, (b) and (c) are computationally infeasible.

Property 6.4.2 (Binding). Once a daemon publishes $R_{\text{harbor},\ell}$, it cannot produce a different valid root for the same (harbor, ℓ) without contradicting its earlier signature (detectable via the KMS witness) or forking its identity.

Cost. Each settlement adds an $O(1)$ amortized append to the epoch accumulator. Epoch rollover is $O(N)$ hashes plus one signature: at 100 sessions/hour, $\approx 10\text{ms}$. Negligible.

This is the structural meaning of “decentralized verification” in this paper: bonded commons evidence is not just immutable within a daemon, it is verifiable *across* daemons by any party with the daemon’s public key and a 700-byte proof.

6.4.3 Mutable-Signal Ledger (Pheromone Lineage)

Stigmergic coordination originated in biology, where pheromones are accumulate-only: ants deposit, the environment evaporates [100]. Software agents need a richer model. Coordination signals must be *revocable*, *renamable*, and *provenance-attributable* as understanding of the work evolves—an agent that deposited a hint and later realized the hint was wrong needs a way to retract it that preserves attribution rather than erasing it.

Event-sourced pheromones. Each entity’s pheromone state is a view over an append-only event ledger. For entity e and key k :

$$\sigma_t(e, k) = \text{decay}(S, t - t_0) \cdot \mathbb{I}[\text{not revoked or expired in } (t_0, t]],$$

where S is the last spray strength on k at time t_0 and decay is a geometric decay defined per-channel. **Revocation** is itself an event; **rename** is a rewrite-pointer event; **fork** creates a new key namespace whose provenance is traceable through the event chain.

Property 6.4.3 (Attribution Invariant). Every $\sigma_t(e, k)$ value has a deterministic provenance chain back to one signing principal, traceable via the event chain.

Property 6.4.4 (Rollback Safety). A consumer that reads σ at time t and later discovers a revocation event timestamped $t' < t$ can compute the correct counterfactual view by replaying events.

Property 6.4.5 (Conservation). Revocation does not erase. The event ledger is monotone: the cardinality of the event set only grows.

Integration with the forest. Pheromone events are included in the session tree of §6.4.2: each session’s root summarizes the events the session produced, harbor roots summarize session roots, and revocations are therefore transparent to witnesses without erasing any prior state. The mutable surface is the *view*; the underlying ledger is immutable.

This dual—mutable view, immutable substrate—is what lets coordination signals evolve without compromising the evidentiary guarantees the rest of the paper depends on.

6.4.4 Attribution Survives Identity Disposal

A critical property: even if an agent’s identity is discarded after task completion, the **delegation chain** traces every action back to the authorizing principal. The chain is:

Principal $\xrightarrow{\text{funds}}$ Float Plan $\xrightarrow{\text{authorizes}}$ Agent α $\xrightarrow{\text{delegates}}$ Agent β $\xrightarrow{\text{acts}}$ Evidence Trail.

The agent is ephemeral. The principal’s authorization—signed, escrowed, recorded—is permanent. Anonymous harm to shared state is impossible within the system, because every action chains back to someone who posted collateral; §6.7.5.4 draws the corresponding line for harm from *disclosure*, which this chain does not cover.

6.5 The Limits of Decisive Allocation

A natural question is why file claims are advisory rather than exclusive locks. Sen’s theorem illuminates the tension, but it does not prove that every lock manager is inefficient. The argument below separates the theorem from the engineering example.

6.5.1 A Sen-Inspired Design Warning

Sen [13] proved that no social welfare function can simultaneously satisfy:

1. **Pareto efficiency:** If every agent prefers outcome X to Y , the system chooses X .
2. **Minimal liberalism:** Each agent has at least one “personal domain”—a pair of outcomes where the agent’s preference is decisive.

The analogy to file coordination is:

- **Pareto efficiency** = the team ships both features (the collectively optimal outcome).
- **Minimal liberalism** = each agent has decisive control over its claimed files (enforced locking).

Property 6.5.1 (A Pareto-Inferior Locking Instance). Consider agents α and β with tasks T_α and T_β . Agent α claims file f because it *might* need to modify it. Agent β discovers mid-task that it *actually* needs f . Under enforced claims:

- α ’s claim is decisive (minimal liberalism): β is blocked.
- The team-level outcome (T_α and T_β both completed) is blocked by α ’s precautionary claim.
- A Pareto-superior outcome exists (both tasks completed) but is unreachable because α ’s liberal right vetoes it.

This property is a counterexample to the universal claim that exclusive locks always improve team welfare; it is not a derivation of Sen’s theorem. Sen’s result requires an unrestricted preference domain and a social-choice rule with decisive personal pairs. A production lock manager may restrict preferences, negotiate releases, or time out claims. The usable lesson is narrower: do not grant an agent an unconditional veto over a shared resource when the team may possess Pareto-relevant information the allocator lacks.

This is not a contrived scenario. AI agents are *inherently uncertain* about which files they will modify when they begin a task. An agent asked to “fix the login bug” cannot predict whether it will need the authentication middleware, the route handler, the test suite, or all three. Enforced claims force agents to either:

1. **Over-claim** (request locks on every file they might touch) → destroys parallelism.
2. **Under-claim** (request locks only on files they’re certain about) → produces undetected conflicts, defeating the purpose.

The lesson has a boundary, and it is worth drawing rather than asserting. Advisory claims are not better than locks everywhere; they are better in a region, and Figure 6.2 marks it. Where an agent’s file needs are knowable up front, an exclusive lock allocates exactly as an advisory claim would and adds determinism at no cost—in that regime the design choice of this section would simply be wrong. Where negotiating a release is prohibitively expensive, neither regime converges, which is the case the hard-lock fallback of §6.5.3 exists to absorb. The advisory default is justified by where LLM coding agents actually sit, not by a universal claim.

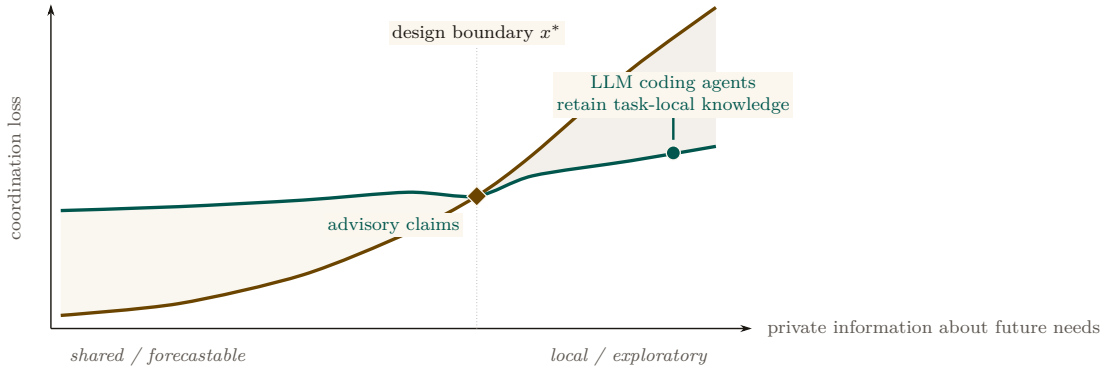


Figure 6.2: The governance choice is a loss frontier, not a four-quadrant taxonomy. Exclusive allocation is cheap when work needs are shared and forecastable, then becomes brittle as relevant knowledge remains local to the agent. Advisory claims carry a persistent negotiation cost but preserve local choice. Past the schematic crossing x^* , advisory coordination has the lower modeled loss. The curves are conceptual: the figure identifies the decision boundary the deployment must estimate; it does not claim a measured universal threshold.

6.5.2 Advisory Claims as Resolution

Advisory claims avoid granting that unconditional veto. They do not guarantee Pareto efficiency; they expose conflicts so informed agents can negotiate:

Definition 6.5.1 (Advisory Consistency). Under advisory claims, file claims form a conflict graph $G = (V, E)$ where vertices are active sessions and edges connect sessions with overlapping claims:

$$(S_i, S_j) \in E \iff \exists f_i \in F_i, f_j \in F_j : \text{overlap}(f_i, f_j).$$

The commons authority guarantees that every edge in G is reported to both endpoints. No agent has a decisive “personal domain” over any file. Instead, agents receive complete information about the conflict graph and self-coordinate.

Theorem 6.5.1 (Advisory Conflict Completeness). *Under advisory claims, for any two concurrent sessions S_1, S_2 with overlapping file claims f , the commons authority reports $\text{conflict}(f)$ to S_2 at claim time. No false negatives exist.*

Proof. The overlap detection query scans all active (unreleased) claims from other active sessions. A whole-file claim ($R = \perp$) overlaps with any range on the same path. Two ranged claims $[a, b]$ and $[c, d]$ overlap iff $a \leq d \wedge c \leq b$. Since the query evaluates this predicate exhaustively over all active claims with the requesting session excluded, every overlap is detected. False positives may occur (an agent claims a file it does not modify); false negatives cannot. $\square$

Remark. The commons authority’s role under advisory claims is that of *town crier*, not *Solomon*. It announces conflicts; it does not resolve them. Resolution requires local knowledge that only

the agents possess (“I claimed `auth.ts` but probably won’t need it”; “I absolutely must modify `auth.ts` for my task to succeed”). The authority lacks this knowledge. Agents, informed by the conflict graph, make allocation decisions more efficiently than any central enforcer could.

6.5.3 Governance, Disputes, and Appeals

Advisory coordination produces consistent allocation when agents share information formally. It can still produce *disputes*: two agents legitimately need overlapping files; one agent’s claim turned out to be precautionary and is now blocking real work; a third agent observed evidence that a claimed task has been abandoned. These disputes do not threaten the commons’ integrity—attribution, capability bounds, and settlement records remain intact—but they do require a resolution path. Without one, advisory claims silently degrade into squatter rule (whoever claimed first holds indefinitely) and the Pareto benefit of §6.5 evaporates.

Resolution path. The commons authority provides four mechanisms, in order of increasing escalation (Figure 6.3):

1. **Direct release.** The original claimant releases the claim. This is the equilibrium when the conflict-graph signal reaches them and the claim was precautionary.
2. **Time-out release.** Claims carry TTLs. A claim whose holder has not heartbeated within the TTL window is released automatically; downstream agents observe the release in their next poll. This is the equilibrium for crashed or abandoned claimants.
3. **Bonded preemption.** A blocked agent may post an additional bond—bounded by the cleanup cost of §6.7.5.1—to force release of an active claim. The preempted claimant receives the bond as compensation; the preemptor accepts the risk that its preemption was wrong (in which case the bond is slashed under settlement). Preemption is rare by design: the bond is large enough that an agent prefers waiting to preempting unless the wait cost dominates.
4. **Human escalation.** If preemption is ambiguous or the stakes exceed the preemption-bond ceiling, the dispute escalates to a human operator via the `request` channel (§6.8). The operator’s decision is appended to the evidence chain with the same Merkle-Forest binding as any agent action; the resolution is auditable by all parties post-hoc.

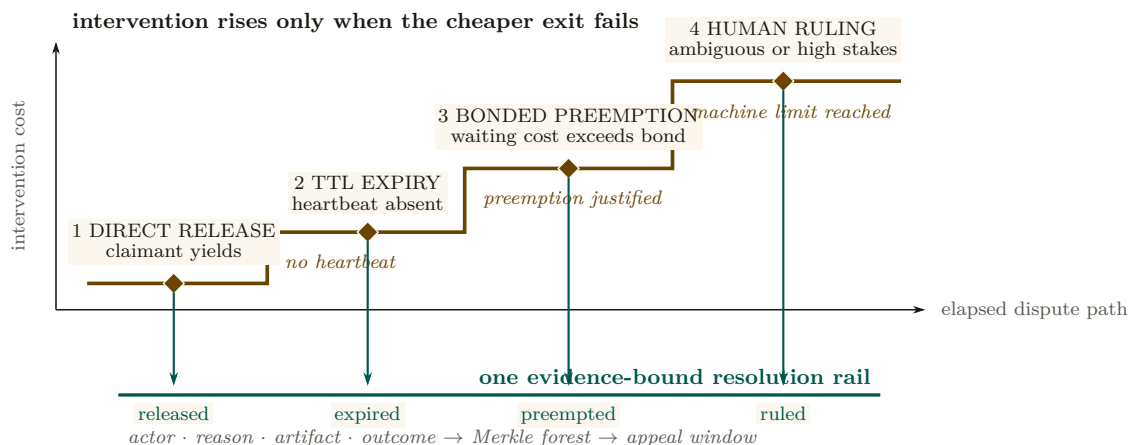


Figure 6.3: Dispute resolution is an escalation staircase. The system first offers a zero-coercion exit, then waits for the claimant’s TTL, then permits priced preemption, and reaches human judgment only after the machine-readable options are exhausted. Every station can terminate the dispute, and every termination descends to the same evidence rail. Escalation therefore changes the decision cost and decision-maker without discarding the record needed for appeal.

Two mechanisms sit underneath this escalation path rather than inside it: a thrashing fallback for advisory claims that never converge, and a fairness rule for the preemption queue itself.

The Advisory Claims Paradox and Deterministic Hard-Lock Fallback. Advisory claims risk an endless retry/thrashing loop if two LLM-backed agents mutually ignore advisory warnings. To break this paradox, the daemon implements a **Deterministic Hard-Lock Fallback**: if advisory conflicts on a resource exceed k turns without resolution (default $k = 5$, tuned per deployment from the observed conflict-resolution distribution), the daemon escalates the advisory claim into an enforced POSIX file-level lock, rejecting all subsequent writes from un-holding agents until release.

Stigmergic Ticket-Lock (OP-1: Fair Exclusion). To prevent starvation during high contention, the daemon replaces chaotic retries with a **Stigmergic Ticket-Lock**. The lock is managed via a strict SQLite schema:

```

1 CREATE TABLE claim_tickets (
2   resource_id TEXT,
3   agent_id TEXT,
4   ticket_number INTEGER PRIMARY KEY AUTOINCREMENT,
5   status TEXT DEFAULT 'waiting' -- 'waiting', 'active', 'released'
6 );

```

Upon `release()`, an atomic FIFO lock-assignment transition grants the lock to the agent with the lowest `ticket_number` where `status = 'waiting'`, changing its status to `'active'`. This guarantees fair exclusion without central scheduling.

Appeals. Settlement events (§6.7) are not unilaterally final. Any settled session can be challenged within an appeal window (default: 24 hours of semantic cadence, §6.9) by any agent with standing—defined as either a counterparty to the settlement or a holder of an overlapping claim. An appeal opens a re-examination of the evidence chain: the appellant posts an *appeal bond* (a fixed fraction of the disputed settlement amount), and the daemon recomputes the settlement against the evidence and any new evidence the appellant produces. If the appeal succeeds, the original settlement is reversed and the appeal bond is returned with the original counterparty’s slash. If it fails, the appeal bond is forfeit.

Property 6.5.2 (Appeal Soundness). The appeals path does not weaken the immutability of the evidence chain (§6.4): no record is rewritten. Reversal appends a counter-record signed by the daemon, with a Merkle-Forest proof of the original settlement’s position and the appellant’s evidence. The chain grows monotonically.

What the daemon does *not* do. The daemon does not judge the merits of an appeal in the sense a human court would—it evaluates whether the evidence supports the original settlement under the published rules. Substantive judgments (“was the agent’s reasoning sound”; “was the principal acting in good faith”) remain with humans, escalated through the **request** channel. The daemon’s role is to ensure that all parties have access to the same evidence and that any human decision is recorded with the same finality as any agent decision.

This subsection is intentionally short: full governance design is out of scope for a single paper. The point is that the bonded commons admits a clean, mechanizable appeals path—it is not the case that advisory claims plus bonding produces a regime where mistakes are unappealable.

6.6 Crash Recovery as Social Continuation

When an agent dies, the commons does not lose information—if the authority does its job.

Traditional crash recovery in database systems follows the ARIES protocol [46], within the classical transaction-processing frame of Gray and Reuter [118]: the recovery manager replays the write-ahead log to restore consistency mechanically. Agent crash recovery is fundamentally

different. An agent’s “log” is its evidence trail—a sequence of natural-language observations and decisions. A successor cannot replay this trail deterministically; it must *interpret* it.

Definition 6.6.1 (Social Recovery). When agent α is declared dead (no heartbeat for a status-adaptive threshold θ_{dead}):

1. All active sessions of α are marked **abandoned** (the “zombie protocol”).
2. α is queued in the resurrection set $\mathcal{R}$ with status **pending**.
3. A successor agent β may claim α ’s work, receiving the *context triple*: purpose ϕ , evidence trail N , and file claims F .
4. β uses its own reasoning to interpret N and determine how to continue.

6.6.1 What Morality Means When Death Is Cheap

Agent death in this system is not an existential event—it is a temporary inconvenience. The agent’s body (process) is destroyed, but its mind (evidence trail) survives in the commons. A successor reads the trail and continues. Like Hickman’s Krakoa [126]—the *House of X* storyline in which mutant minds are literally backed up to a database (Cerebro) and restored into freshly grown bodies on death—the information survives the vessel.

This raises a question: if death carries no permanent consequence, what is the Leviathan’s sword? The answer produces a moral hierarchy specific to agent societies:

Table 6.3: The only catastrophic event in this hierarchy is the loss of information, not the loss of an agent. Severity climbs from a crash (none, because salvage recovers the context) to a daemon failure (catastrophic), with dismissal-from-salvage as the one irreversible step a single agent can cause on its own.

Event	Severity	Consequence
Agent crash	None	Salvage recovers context. The commons loses no information.
Agent defects	Moderate	Immutable history records it. Future delegations attenuate. Reputation degrades.
Agent dismissed from salvage	Severe	Irrecoverable information loss. The knowledge accumulated in this agent’s trail is permanently destroyed.
Commons authority crashes	Catastrophic	The sovereign falls. No new trust established, no conflicts detected, no salvage possible. State of nature until restart.

The fundamental moral harm in agent societies is not the destruction of an agent—it is the **irrecoverable loss of information**. An agent can be rebuilt. Knowledge, once dismissed from the commons, cannot.

6.6.2 The Limits of Reputation

Reputation—the persistent record of an agent’s behavior across lifetimes—is a necessary but insufficient sword. It fails against three classes of adversary:

1. **One-shot defectors:** Agents spawned for a single task, with no future interactions where reputation matters. Create identity, sabotage, discard. The Sybil attack on trust.
2. **Agents with misaligned principals:** The agent faithfully follows adversarial instructions. Its reputation is spotless—it did exactly what it was told.
3. **Agents that benefit from chaos:** In competitive environments, degrading the commons is rational if your competitor depends on the commons more than you do.

Reputation assumes agents want to participate in the commons long-term. When this assumption fails, a different mechanism is required.

6.7 Layer 3: Economic Alignment

The moral relativism problem—that a bad actor may view information destruction as a net good—dissolves when the commons isn’t sacred but *priced*. If an agent nukes the workspace, the damage was collateralized before the work began. The bad actor’s principal has an invoice headed their way.

6.7.1 The Float Plan

Definition 6.7.1 (Float Plan). A Float Plan $\mathcal{F}$ is a collateralized work contract consisting of:

1. **Manifest:** A declaration of intent—which files will be touched, expected duration, acceptance criteria (e.g., “tests pass,” “code review approved”).
2. **Escrow:** Credits posted by the principal, locked in a **2-of-3 Multisig Clearinghouse** (Harbor A, Harbor B, Federation Arbiter) for the duration of the work. The escrow amount reflects the risk to the commons.
3. **Signatures:** The principal signs the Float Plan (Ed25519). The commons authority countersigns, certifying that the escrow is locked and the manifest is registered.

The Float Plan is a *futures contract on agent labor*. The principal sells a promise of work. A 2-of-3 Multisig Clearinghouse manages the contract. Under the non-custodial happy path, both Harbors sign off on completion and the Arbiter is unnecessary. Under dispute, the Federation Arbiter steps in to provide cryptographic judicial arbitration, settling the outcome using the immutable evidence chain.

6.7.2 Settlement

Definition 6.7.2 (Settlement). When a Float Plan’s session reaches a terminal state, the Multisig Clearinghouse settles the escrow:

- Success:** The evidence trail demonstrates that acceptance criteria are met (tests pass, file diffs match manifest scope, code review approved). Escrow is released to the agent or its principal.
- Partial completion:** The agent completed some work before crashing or aborting. The Merkle-chained evidence trail enables pro-rata assessment: the fraction of acceptance criteria met determines the fraction of escrow released. The remainder is available to fund the successor’s completion via salvage.
- Sabotage / breach:** The evidence trail shows work outside the manifest scope, destructive modifications, or failure to meet any acceptance criteria. The full bond is liquidated to cover reconstruction costs.

Property 6.7.1 (Intent Irrelevance). Settlement does not evaluate agent intent. It evaluates *outcome against manifest*. A well-intentioned agent that fails to meet acceptance criteria forfeits escrow. A malicious agent that coincidentally produces correct output receives payment. The system optimizes for outcomes, not character.

This is how performance bonds work in construction. This is how futures markets settle. The contractor’s moral character is irrelevant. The building either meets code or it doesn’t. The bond either covers the damage or it doesn’t.

6.7.3 Why This Defeats the Three Adversaries

The Sybil attack is priced: 100 disposable identities means 100 bonds.

1. **One-shot defectors** must post collateral *before* receiving capabilities. Because the bond attaches to the work, not to the identity, disposability does not help the attacker: cheap identity, expensive lie. The cost of defection scales linearly with the number of attack identities, and §6.7.5.9 shows exactly where that linearity stops deterring.
2. **Misaligned principals** are exposed by the attribution chain. The agent may be ephemeral, but the principal who funded the Float Plan is permanently recorded. Liquidating the bond hurts the principal, not just the agent.
3. **Agents that benefit from chaos** face a simple calculation: is the damage to my competitor worth more than my bond? The commons authority doesn't prevent this—it *prices* it. And the pricing can be adjusted: higher-risk operations (write access to core modules) require larger bonds.

Remark. The Float Plan does not make sabotage impossible. It makes sabotage *expensive*. A principal willing to burn money to cause damage will burn the bond and cause the damage. The bond raises the price of defection; it does not eliminate it. The defense against existential threats to the commons is not internal governance but external boundary enforcement: who is allowed to post Float Plans at all. That decision is irreducibly political.

6.7.4 Truthful Claim Signaling as Nash Equilibrium

The bond machinery above prices *outcomes*: an agent that destroys files pays for the destruction. The paper's headline claim, however, is upstream of outcomes—that truthful file-claim signaling (§6.4.4, §6.5.2) is incentive-compatible in the first place. The cartel analysis of §6.7.5.10 works that argument out for insurer pricing. The corresponding argument for the claim signal itself has been carried on faith until now. This subsection closes that gap.

Stage game. Consider two agents *A* and *B* working on a shared project, deciding per round on each file whether to signal:

T (Truthful). Claim files the agent intends to edit; do not claim files it will not edit.

F (False). Either claim files the agent does not intend to edit (to block a rival), or fail to claim files it does intend to edit (to avoid scrutiny). Either form of misrepresentation counts as *F*.

The stage-game payoffs, in expected-utility units calibrated to the cleanup cost *c* of §6.7.5.1, are:

Table 6.4: One-shot file-claim signaling: a classical Prisoner's Dilemma. Mutual truth (3, 3) is Pareto-optimal, but *F* strictly dominates *T* for each player ($4 > 3$ and $1 > 0$). The unique one-shot Nash equilibrium is (*F*, *F*) yielding (1, 1), destroying the coordination signal.

	B: T	B: F
A: T	(3, 3)	(0, 4)
A: F	(4, 0)	(1, 1)

The stage game is a strict Prisoner's Dilemma: *F* strictly dominates *T* for each player ($4 > 3$ against *T*, $1 > 0$ against *F*), and (*F*, *F*) is the unique one-shot Nash equilibrium. *Truthful signaling is not a one-shot equilibrium*; advisory coordination cannot be sustained by stage-game rationality alone.

Move to the repeated game. What rescues truthful signaling is that the agents are not playing the stage game once. The substrate provides:

- **Observable history.** The Merkle-chained evidence trail of §6.4 and heartbeat record of §6.6 make each player’s prior moves visible to any third party who can verify a 700-byte inclusion proof. Past defection is not deniable.
- **Persistent identity.** The Anchor Protocol’s passkey-bound device pairing [80] ties an agent ID to a hardware-rooted credential. Sybil re-registration is bounded by the per-identity onboarding cost C_{kyc} of §6.7.5.9, so an agent cannot trivially shed a reputation by re-incarnating.
- **A discount factor δ .** Principals value future interactions: a development project lives over weeks; insurer relationships compound; reputation discounts (§6.7.5.3) accrue. For long-lived principals we take $\delta \approx 0.9$ as a working operating point, consistent with the discount used in §6.7.5.10.

Strategy profile (graduated trigger). Each agent plays the following strategy on each file:

1. Begin by playing T .
2. If the opponent’s last observed action on the relevant surface was T , play T .
3. If the opponent’s last observed action was F , play F for three rounds (punishment phase), then return to T .
4. If the opponent deviates during the punishment phase, restart the three-round window.

Graduated trigger, not grim trigger. The cartel analysis of §6.7.5.10 uses grim trigger because that gives the cleanest folk-theorem bound, and the analysis below uses grim trigger for the same reason where the calculation is short enough to be exact. The *production* strategy is graduated for the reason given in §6.6: crashes and deliberate defection are operationally indistinguishable from any third-party observer’s vantage. Permanent punishment for a transient infrastructure event would destroy cooperation faster than any saboteur could.

Deviation analysis. Suppose A contemplates playing F in some round when the strategy prescribes T . The one-shot gain from defection, paid in the deviation round, is $d - c = 4 - 3 = 1$. The cost is three subsequent rounds of mutual punishment at (F, F) before the relationship resets, each paying $p = 1$ instead of the cooperative $c = 3$ (a net loss of $c - p = 2$ per round). Discounted at δ :

$$\text{PV of lost cooperative payoff} = (c - p) \cdot (\delta + \delta^2 + \delta^3) = 2 \cdot (\delta + \delta^2 + \delta^3).$$

At a working discount factor $\delta = 0.9$, the bracketed sum is $0.9 + 0.81 + 0.729 = 2.439$, so the deviator’s net payoff is $1 - 2 \cdot 2.439 = 1 - 4.878 = -3.878$, strictly negative. Deviation is strictly unprofitable.

Critical δ . The general sustainability condition under a k -round graduated trigger with stage-game payoffs $(c, d, p) = (3, 4, 1)$ is

$$\underbrace{d - c}_{\text{one-shot gain}} < \underbrace{(c - p) \cdot \frac{\delta(1 - \delta^k)}{1 - \delta}}_{\text{discounted punishment cost}}.$$

Substituting $d - c = 1$, $c - p = 2$ and $k = 3$ gives $1 < 2\delta(1 + \delta + \delta^2)$, which solves numerically to $\delta > \delta_{k=3}^* \approx 0.342$. Under grim trigger ($k \rightarrow \infty$) the bound is the closed-form $\delta > 1/3 \approx 0.333$, recovering the standard folk-theorem threshold. The graduated bound is therefore only marginally above grim—the three-round window adds barely 0.009 to the threshold—which is the right result: graduated trigger pays almost nothing for the crash tolerance it buys.

Proposition 6.7.1 (Claim Signaling Incentive Compatibility). *Under the observable-history regime of §6.4 and the Sybil-resistant identity regime of the Anchor Protocol [80], truthful file-claim signaling is a Nash equilibrium of the repeated coordination game with stage-game payoffs as in Table 6.4 for any discount factor $\delta > \delta_{k=3}^* \approx 0.342$, when both players adopt the three-round graduated-trigger strategy described above. Under grim trigger, the critical bound is $\delta \geq 1/3$.*

What each condition buys, and what breaks when it is removed. The proposition is conditional, and the conditions matter. Naming what each one is doing makes the dependency on the rest of the architecture explicit rather than implicit.

- *Remove observable history (§6.4).* The repeated game collapses to repeated one-shot play: with no public record of prior actions, no trigger can fire, and both players play F in every round. This is why the Merkle forest of §6.4.2 is necessary for the claim signal, not just for post-hoc audit.
- *Remove persistent identity (Anchor [80]).* An agent who can re-register after each defection effectively faces $\delta = 0$: future punishment falls on a discarded identity. Cooperation is impossible.
- *Drop δ below $\delta_{k=3}^* \approx 0.342$.* The three-round graduated trigger is insufficient. Either lengthen the punishment window or accept that the equilibrium does not bind for very short-lived principals.
- *Drop δ below $1/3$.* No trigger of any duration sustains (T, T) . The repeated game has the same equilibrium structure as the stage game, and only the bond mechanism of this section, not the signal itself, deters defection.

The forward reference is direct. Section 6.7.5.10 reuses the same analytical pattern—stage game, repeated-game lift via observable history, deviation calculation, and a δ threshold—at the insurer pricing layer. The analogy is structural, not a transfer theorem: the payoff functions, monitoring signal, detection timing, and punishment regimes differ and must be derived separately.

Mechanized claim-signaling. The argument above is also checked by machine. Standard one-shot-deviation analysis yields the discount-factor threshold δ^* as the unique real root in $(0, 1)$ of $2\delta^3 + 2\delta^2 + 2\delta - 1 = 0$, numerically $\delta^* \approx 0.342$ (matching $\delta_{k=3}^*$ above). The cubic, the existence-and-uniqueness of its root in $[0.34, 0.35]$, and a TLA⁺ model checking that no one-shot deviation produces positive discounted payoff at $\delta = 0.35$ are mechanized as `proofs/economics/delta-threshold.z3` and `proofs/economics/claim_signaling.tla`. Both run unattended in CI (§6.A, Table 6.7).

6.7.5 Pricing the Bond

What is the right collateral for “write access to `auth.ts` for 30 minutes”? The answer requires a pricing function $\pi : \mathcal{F} \rightarrow \mathbb{R}^+$ that maps Float Plans to bond amounts. An effective π must satisfy:

1. **Deterrence:** $\pi(\mathcal{F})$ must exceed the expected damage from defection under $\mathcal{F}$ ’s scope.
2. **Accessibility:** $\pi(\mathcal{F})$ must not be so high that legitimate agents are priced out of the commons.
3. **Risk sensitivity:** π should increase with the scope and criticality of the manifest.
4. **History adjustment:** π may decrease for principals with strong track records.

We do not close this problem. We tighten the lower bound, propose a scope multiplier, suggest a reputation discount, and describe two concrete mechanisms—the **Bonded Advisor** (§6.7.5.7) and the **Competitive-Insurance** market of Youle (§6.7.5.8). See Appendix 6.B for these four requirements worked through a single \$5 task end to end.

6.7.5.1 Cleanup Lower Bound

Let c be the project’s *cleanup cost per breach event*—the human-plus-compute cost to detect, assess, and recover from a budget breach. This is observable from the audit log.

Theorem 6.7.1 (Cleanup Lower Bound). *For any Float Plan $\mathcal{F}$, $\pi(\mathcal{F}) \geq c$.*

Rationale. If $\pi(\mathcal{F}) < c$, breach is cheap: the bond is forfeit for less than the cleanup cost. If the commons pool funds cleanup, $\pi < c$ drains the commons per breach—the system bankrupts itself through enforcement. $\square$

The Cleanup Cost c Fantasy. The linear bond multiplier assumes c is a bounded, predictable quantity. This is a fantasy for tasks with non-linear tail-risk ruin (e.g., dropping a production database or leaking a private key). Such tasks are fundamentally *un-bondable*. Defense of these tasks is explicitly moved to Layer 1 capability confinement (isolation domains, zero-egress policies); economic deterrence at Layer 3 is only mathematically sound when c is finite and observable. $\square$

This is a floor, not a tight bound. It is observable from operations and trackable as a project health metric.

6.7.5.2 Consequential-Scope Multiplier

Let $s(\mathcal{F})$ be plan scope (files claimed, presence of `db:write`, production-deployment capability). Empirically, cleanup scales super-linearly with scope; coordination cost dominates the high end.

$$\pi(\mathcal{F}) \geq c \cdot (1 + \alpha \cdot s(\mathcal{F})).$$

α is calibrated from observed cleanup per scope unit. Low- α projects have simple, easily-audited work; high- α projects have tangled dependencies. α is publishable from the audit log.

6.7.5.3 Reputation Discount

Principals with a history of clean settlements have lower expected damage:

$$\pi(\mathcal{F}, p) = \pi(\mathcal{F}) \cdot (1 - \rho(p)), \quad \rho(p) \in [0, r_{\max}], \quad r_{\max} \leq 0.5,$$

to prevent trivialization. ρ increases with clean settlements, decreases with breaches, decays over time. The functional form is left open.

6.7.5.4 Threat-Class Bond Sizing

The closed-form bond $\pi(\mathcal{F}, p) = c(1 + \alpha s)(1 - \rho)$ is threat-agnostic: sloppy work, abandonment, and active sabotage all price off the same curve. That is wrong in both directions. A single curve calibrated to deter sabotage prices capable agents out of low-risk work; a single curve calibrated for accessibility underprices the rare adversary who actually intends harm. Real deployments need different multipliers for different threat classes, because the threats have different blast radii and different probabilities of recovery.

Table 6.5 gives starting bands. These are not parameters, they are calibration targets: production deployments tune from observed failure modes in their own audit log.

Table 6.5: Threat-class bond sizing. Percentages are bond as a fraction of expected task value. Bands are starting points for calibration, not protocol constants.

Threat	Description	Bond %	Rationale
Careless work	Agent tries; output is sloppy, partial, or fails review.	10–25%	Loses some, not all. Bonds at this band incentivize care without freezing out capable agents from cheap tasks.
Abandonment	Agent takes the task and ghosts before completion.	25–50%	Covers the cost of onboarding a replacement: scope-note ingestion, partial-diff review, salvage handoff (§6.4.4).
Active sabotage	Agent intentionally damages shared state.	100–200%	Covers full reconstruction of polluted files plus the cleanup of downstream agents that consumed the bad state. Capital-inefficient above 100%, but acceptable for critical files where reconstruction cost dwarfs the bond.
Data exfiltration	Agent reads and leaks confidential context.	— (<i>spec-only</i>)	Bonds are an inadequate deterrent: leaked context is worth far more than any feasible bond. Rely on capability scoping (Anchor §3.3) and isolation domains (§6.3.2).

The exfiltration row is an explicit gap. It exists in the table to make the limitation legible: a bonded commons does not stop information theft, because the value of leaked secrets can vastly exceed any bond a principal will rationally post. Confidentiality is a structural problem, not an economic one, and the answer lives in Layer 1 (capability-attenuated tokens, isolation domains) rather than Layer 3. The economic alignment of this paper is correct about damage to *shared state*; it makes no claim about damage from *disclosure of state to outsiders*.

The threat-class table also informs Phase 2 of the bootstrap path below: graduated access begins by admitting new principals only to the careless-work and abandonment bands, with the active-sabotage band gated on accumulated reputation.

6.7.5.5 Bootstrap Path

Competitive insurance (§6.7.5.8) is a thick-market mechanism. It needs insurers with capital, principals with reputation history, and enough transaction volume that bidders can statistically distinguish good risks from bad. A cold-start deployment has none of that. The conditional auction/static comparison of §6.7.5.8 describes the modeled end-state; Phases 1–2 below are the path to the assumptions it needs, not optional preamble.

Phase 1 — Subsidized seeding (weeks 1–4). The marketplace operator posts real tasks at above-market rates and invites a small cohort of 5–10 known-capable principals from an existing network. Bonds are fixed at a flat rate per task class; pricing is not a parameter to optimize

yet, because there is no data to optimize against. The goal is to produce the first row of the reputation table: clean settlements, breaches, recovery times, observed cleanup cost c . The operator is subsidizing the existence of an audit log.

Transition metric. Completion rate above 80% sustained for ≥ 2 consecutive weeks. Below this threshold the cohort is not yet generating usable risk signal, and Phase 2 pricing will overfit to noise.

Phase 2 — Complexity-proportional pricing with graduated access (months 2–6).

The closed-form bond $\pi(\mathcal{F}, p) = c(1 + \alpha s)(1 - \rho)$ activates, with α calibrated from Phase 1 cleanup data and ρ initialized from the Phase 1 reputation table. New principals enter at a low-risk tier: only the careless-work and abandonment bands of Table 6.5 are available. Promotion to the sabotage-coverage tier gates on accumulated clean settlements, mirroring the A5 composite mechanism (§6.7.5.9).

Reputation bootstrapping is the hard problem of this phase. A principal with verifiable work history from another deployment should not start at zero, but the operator cannot verify external histories directly. The mechanism is an *attestation import*: external audit trails signed under an Anchor-issued key (§6.4) earn partial reputation credit, capped well below an equivalent in-system history. The credit is partial because cross-deployment reputation does not control for local cleanup cost, and capped because a fraudulent attestation issuer is a single point of failure.

Transition metric. New-principal 30-day retention above 50%. Lower retention indicates either the gating is too strict (principals churn before they can earn reputation) or the bond bands are mispriced relative to observed task value. Either way, the system is not ready for competitive bidding on top of these parameters.

Phase 3 — Competitive insurance (month 6+). The §6.7.5.8 mechanism activates: insurer agents bid on underwriting transactions, static parameters retire as insurer volume grows, and listing fees enable the cartel-resistance regime of §6.7.5.10. The operator’s pricing parameters become fallbacks rather than the primary mechanism; they apply only when fewer than a threshold number of insurer bids arrive on a transaction.

Transition metric. Repeat-poster rate above 60%. The competitive-insurance equilibrium of §6.7.5.8 assumes repeated interaction; without repeat posters the insurers cannot recover information costs and will not bid competitively.

The weak-improvement result of §6.7.5.8 is conditional on Phase 3 being reached, the listed metrics being met, and the theorem’s CC/NC/RP/NS assumptions holding. Phases 1–2 are not bureaucratic preamble: they generate the data the competitive-insurance market needs to price at all. A deployment that skips them is running §6.7.5.8 on uninformative priors, and the comparison does not apply.

6.7.5.6 Settlement and Dispute

The settlement protocol described so far is one-shot: the daemon’s verification result (§6.10) triggers slashing, attribution is recorded immutably (§6.4), and the appeals window of §6.7 allows challenge. That is most of what is needed, but it places the daemon in a position no single component should occupy: simultaneously the verifier, the judge, and the arbiter of evidence. Strong evidence does not make a single oracle correct. It makes a single oracle confidently wrong in the cases where its evidence is incomplete.

The fix is to factor settlement across three oracles with different cost and gameability profiles, and to settle on majority agreement rather than unanimous appeal.

Oracle 1 — Automated. Test-suite pass/fail, type checks, lints, coverage thresholds, and any deterministic CI signal already recorded on the session. Cheap to run, fast to return, and *gameable*: a principal who knows the test surface can ship work that passes the checks while failing the intent.

The daemon already produces this oracle as a side effect of normal operation; the change is to label it as one signal among three rather than the dispositive one.

Oracle 2 — Evidence-based. A verifier (which may itself be a bonded agent) reviews the Merkle attribution chain (§6.4) for the disputed transaction. The chain is append-only and cryptographically bound, so the oracle’s hard problem is not authenticity of the evidence but interpretation: did the recorded actions match the Float Plan’s scope, and do the diffs produced match the acceptance criteria? Medium cost, hard to game without colluding with the chain structure itself, which is the threat model of §6.7.5.9.

Oracle 3 — Human audit. A randomized sample of contested settlements, at a configurable rate (default 10%), is routed to a human auditor. Expensive, slow, and authoritative. The audit produces a labeled outcome that feeds back into reputation history and, critically, into calibrating Oracles 1 and 2: any disagreement between human audit and the cheaper oracles is a training signal for the gameable layers.

Settlement rule.

- *3-of-3 agreement.* Settle immediately on the agreed outcome. The cheaper oracles carry most of the volume in steady state.
- *2-of-3 agreement.* Settle with the majority. Flag the dissenting trace for review; this is the primary signal for tuning the gameable oracle.
- *No majority (three different verdicts, or one settle / one slash / one abstain).* Escalate to full human arbitration. Bond held in escrow until the human verdict lands.

Composition with cartel resistance. Oracle collusion is itself a collusion problem. If Oracles 1 and 2 are run by agents in the same cartel, a 2-of-3 majority can launder a sabotaged settlement past the human audit, modulo the audit sampling rate. The same insurer-collusion analysis applies (§6.7.5.10) with the oracles in the role of bidders: the human-audit sampling rate σ sets the detection probability $p_d^{\text{oracle}} = \sigma$ for the oracle layer, and the folk-theorem threshold becomes a calibration target on σ in the same way it is a calibration target on bid-pattern detection elsewhere in the protocol. The default 10% sample is a starting point; deployments where Oracles 1 and 2 are co-owned should raise it.

6.7.5.7 The Bonded Advisor

One direction for solving π is to delegate its computation to a specialized agent, itself bonded on its advisory accuracy. We call this the **Bonded Advisor** pattern:

- A principal posts a Float Plan whose only acceptance criterion is “propose a fleet that meets budget/scope constraints for project P .”
- A Bonded Advisor surveys P and emits a candidate fleet—itsself a set of Float Plans with proposed π values.
- The advisor’s bond on the proposing plan is slashed proportionally to the accepted fleet’s eventual breach rate. Clean settlements accumulate reputation; breaches shrink it.

The pricer is priced. Convergence depends on (i) whether the advisor has access to sufficient prior-fleet audit data to pattern-match new projects, (ii) whether reputation history is long enough for discounting to be meaningful, and (iii) whether the population of advisors is large enough for competition to discipline outliers.

c as a project health metric. A project whose observed c is rising is fraying—breaches cost more to recover from, implying coordination is decaying. The authority can publish c alongside the bond pool and raise required bonds automatically when c crosses a threshold. Homeostatic feedback: risky spawns become expensive when the project needs them least.

Threat-class bond bands and their defensibility. The pricing function π resolves to one of three nominal bands, calibrated to the threat class the bond is defending against:

- **Careless** (accidental damage during honest attempts): $\pi \in [10\%, 25\%]$ of the claim ceiling.
- **Abandonment** (walk-away mid-task; cleanup dominates): $\pi \in [25\%, 50\%]$ of the claim ceiling.
- **Sabotage** (intentional destruction; can exceed face value): $\pi \in [100\%, 200\%]$ of the claim ceiling.

A Monte Carlo sweep over the cross product of seven plausible threat-distribution mixes and the three bands quantifies defensibility: for every *matched* (mix, band) pair—where the mix concentrates on the class the band is designed to defend against—observed extraction stays within $1.10\times$ the band’s upper bound (10% stochastic tolerance). Off-diagonal cells (e.g. a sabotage-heavy mix posted against the careless band) are surfaced as widening recommendations rather than silent failures. The simulation is mechanized as `proofs/bonded/pareto/threat-bands.mjs` and runs on every PR (§6.A, Table 6.7).

6.7.5.8 Competitive-Insurance Pricing Mechanism³

The mechanism. Instead of a static bond size $B(\pi, r, s)$ selected by the commons authority (§6.7.5.2), allow a market of *insurer agents* to bid on underwriting each agent transaction. An insurer I offers a quote (q_I, c_I) where q_I is the required premium and c_I is the claim ceiling.

A principal P submits a transaction T requiring bond coverage B_T . Insurers quote; principal selects. If the transaction proceeds and damages are assessed at d with $d \leq c_I$, the insurer pays. Otherwise principal pays up to $B_T - c_I$ from its own stake.

Market-discovered pricing. The premium q_I equals the insurer’s expected loss $\mathbb{E}[d \mid T, \text{agent history}]$ plus a risk premium α_I encoding the insurer’s risk aversion and cost of capital. In equilibrium under competition, $q_I \rightarrow \mathbb{E}[d]$ for risk-neutral insurers (Rothschild–Stiglitz). Market discovery replaces the authority’s best-guess parameter selection.

Adverse-selection controls. To avoid the classic lemons problem, the mechanism requires:

- Public agent reputation history (from §6.4.2 the Merkle forest).
- Principal-bound slashing if an agent’s history is misrepresented at quote time.
- Insurer capital reserve backed by on-chain stake (the same bond mechanism, recursive: an insurer posts bond that its claim ceilings are funded).

Welfare comparison (model-conditional). Under the simulation’s full-information, competitive-entry assumptions and its chosen static baseline, the market-discovered premium can weakly improve the modeled principal and insurer payoffs while preserving coverage $B_T = \mu(1 + s)$. This is not a claim against *every* static parameter: a static price equal to the competitive price is outcome-equivalent, and transfers outside the modeled parties can change the Pareto ordering. Figure 6.4 isolates the financing difference.

³This subsection contributed by Thomas Youle (Indiana University, Business Economics & Public Policy) based on conversations with the primary author in March–April 2026. The mechanism builds on classical competitive-insurance market literature [156, 50] adapted to the agent-transactions setting. The text here is a pre-print draft pending economist review of §6.7.5.8–§6.7.5.8.

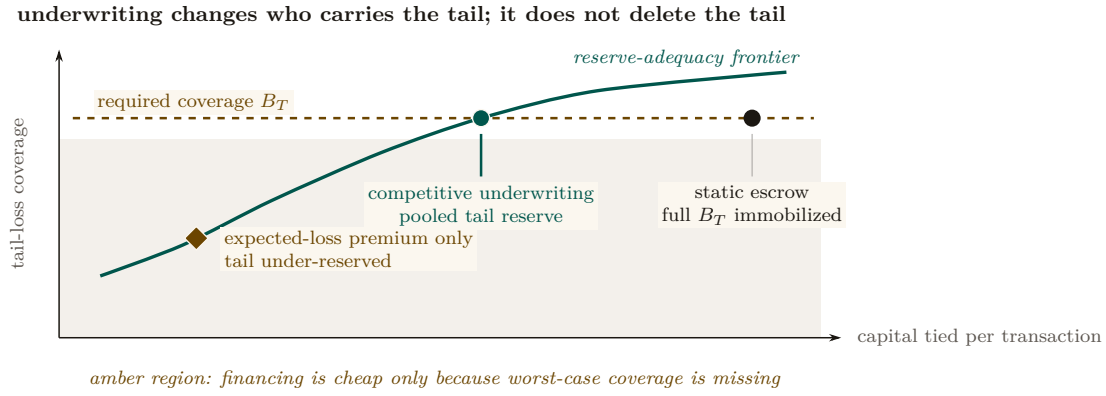


Figure 6.4: Static escrow and competitive underwriting occupy different points on a reserve-adequacy frontier. Static escrow immobilizes the full per-transaction bond. Adequately reserved underwriting can reduce capital tied to one transaction by pooling the tail, but the insurer must still carry that tail in its reserve or reinsurance. The amber expected-loss-only point is cheaper because it violates the required coverage. Coordinates are schematic; the structural comparison is the claim.

An independent model specification and a Monte Carlo sweep over 36 parameter configurations accompany this paper as a downloadable artifact (Appendix 6.A). Figure 6.5 reports three results of that implementation: (i) the no-cartel baseline remains favorable across the tested noise sweep, whereas the three-colluder case loses the comparison beyond roughly $\sigma_r = 0.1$; (ii) the tested full-cartel case defeats the mechanism at the fixed detection setting; and (iii) the tested objective peaks at $n = 3$ insurers. These are finite-sample properties of the supplied code and parameter grid, not general theorems about Vickrey auctions or winner’s curse.

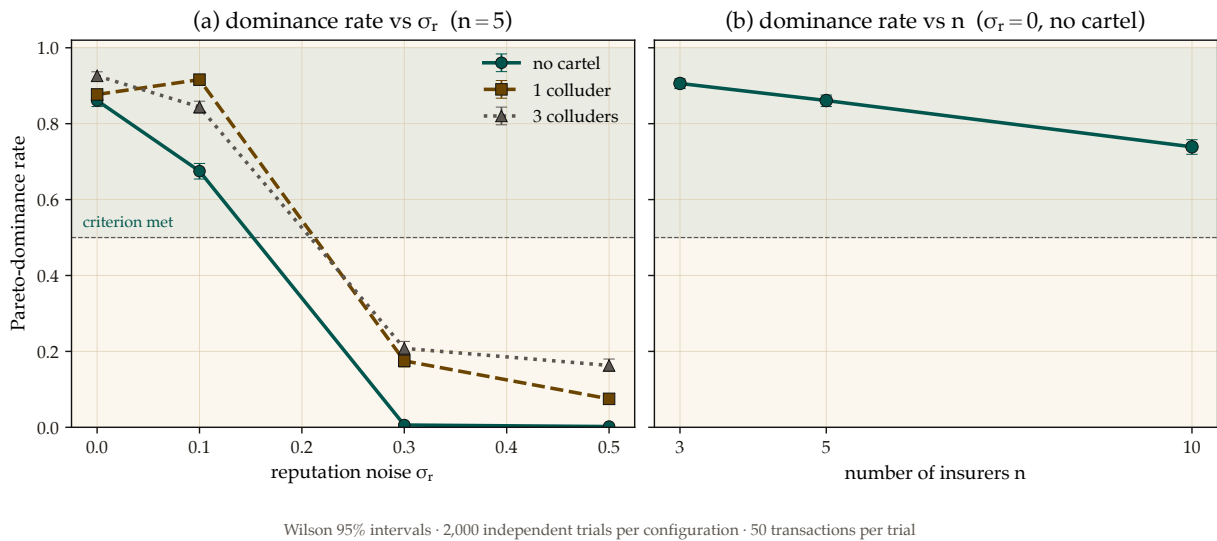


Figure 6.5: The competitive auction’s Pareto-dominance advantage is real but narrow: it survives moderate reputation noise and a modest cartel, then collapses once noise crosses roughly $\sigma_r = 0.1$, and it shrinks as more insurers are added rather than growing. Left: fraction of trials satisfying the code’s Pareto criterion versus reputation noise σ_r at $n = 5$, split by cartel size. Right: the same criterion versus insurer count at $\sigma_r = 0$ with no cartel. Error bars and seeds are recorded in the accompanying artifact; the curves are not analytical welfare bounds.

Relationship to the Bonded Advisor. Under this framing, the Bonded Advisor of §6.7.5.7 becomes the matchmaker plus reputation oracle in the insurance market. It does not set prices; it provides information. Pricing is a competitive equilibrium, not an administrative decision.

6.7.5.9 A5 — Sybil deterrence is coverage-bounded

A pure deposit-based Sybil deterrence policy is *provably insufficient*, and the structure of the protocol explains why. An adversary spawns K Sybil insurer identities, each posting the protocol-mandated deposit B_{dep} , then undercuts honest bids by an arbitrarily small ε and defaults on loss (Figure 6.6).

Total Capital Forfeiture. The arbitrary $\min(B_{\text{dep}}, B_T)$ slash cap fails because it bounds the attacker’s risk. To mathematically destroy the profitability of Sybil cartel undercutting, the protocol mandates **total capital forfeiture**: upon default, the *entire* posted deposit B_{dep} is slashed, regardless of how small the coverage B_T was. This converts a capped-risk arbitrage into a total-loss event.

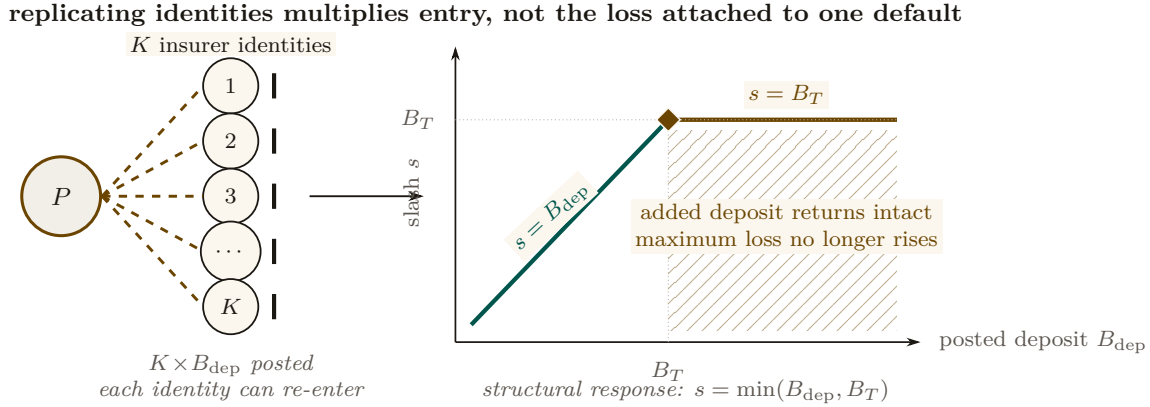
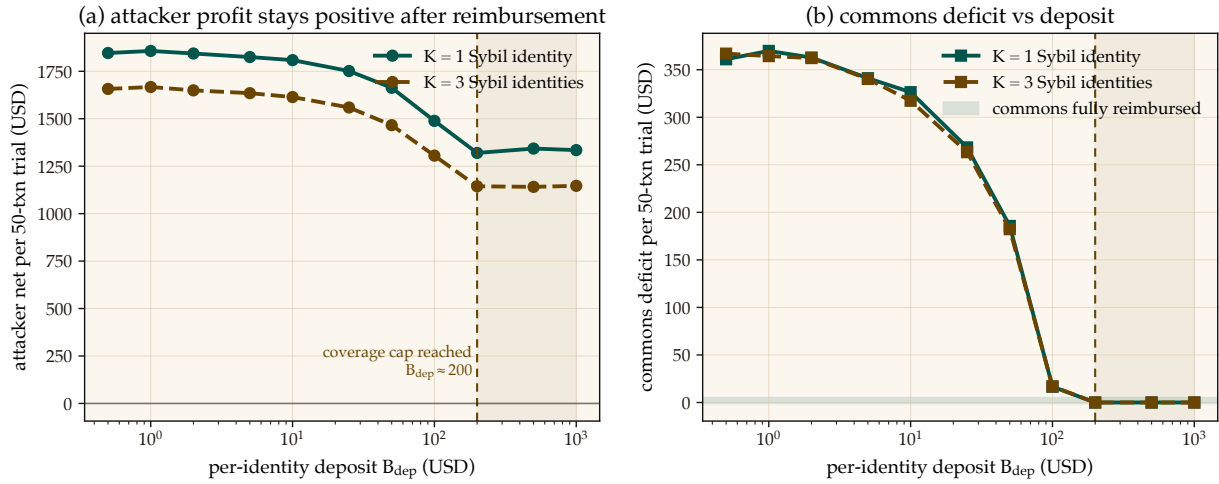


Figure 6.6: Deposit-only Sybil deterrence saturates. One principal can fund K interchangeable identities, but the loss attached to a defaulting identity rises only until its deposit reaches the transaction coverage B_T . Past that knee, $s = \min(B_{\text{dep}}, B_T) = B_T$: additional posted capital returns intact on no-loss transactions and cannot increase the attacker’s maximum loss. Figure 6.7 shows the corresponding simulated profit and commons-deficit sweeps.

The naive expected-value calculation suggests breakeven at $B_{\text{dep}}^* = q^* \cdot (1 - P_{\text{loss}}) / P_{\text{loss}}$, which for the mixed risk classes used in the simulation gives $B_{\text{dep}}^* \approx 316$ USD. *Empirically the attack remains profitable indefinitely.* The reason is structural: the deposit slash is capped at the coverage amount, $\text{slash} = \min(B_{\text{dep}}, B_T)$. Past $B_{\text{dep}} \geq B_T$, any additional deposit posted by the Sybil is recovered intact on no-loss transactions; it never reaches the commons (Figure 6.7).



Profitable-trial fraction: 1.000–1.000; Wilson 95% envelope [0.998, 1.000], $n=2,000$ /configuration. Dollar curves are means; the run log stores no dispersion.

Figure 6.7: A5 — Sybil attack: deposit-only deterrence has a coverage-bounded ceiling. Left: attacker net profit per 50-transaction trial stays positive across the entire deposit sweep ($B_{\text{dep}} \in [0.5, 1000]$ USD). Right: commons deficit converges to zero around $B_{\text{dep}} \approx 200$ USD — the protocol is fully reimbursed, but the attacker still profits because deposit slash is capped at coverage $B_T = \mu(1 + s)$. Closing A5 requires a composite mechanism ($C_{\text{kyc}} + \text{reputation gating} + \text{per-class } B_{\text{dep}}$); pure deposit-based deterrence is provably insufficient.

Closing A5 requires a composite mechanism: an unrecoverable per-identity onboarding cost C_{kyc} (proof-of-personhood, KYC fee, or NFT mint), reputation gating that caps new identities to low-coverage transactions until they accumulate a track record, and per-class B_{dep} tuned to saturate the coverage cap exactly. The single-instrument deposit policy is provably insufficient; the mechanism in §6.7.5.8 is correct only with this composite extension. The supporting analysis is bundled as `cartel-resilience.md` (Appendix 6.A).

6.7.5.10 A6 — Cartel sustainability under the folk theorem

A5 considers a single-round attack: a Sybil identity bids low, wins, defaults. A6 studies a *repeated game*: a coalition of insurers maintains a price floor $q_{\text{floor}} > q^*$ above the competitive equilibrium, splits the surplus, and uses grim-trigger punishment to deter unilateral deviation. Folk-theorem results can support feasible, individually rational payoffs for sufficiently patient players under their monitoring and regularity assumptions; they do not license every outcome in every repeated game. Here the narrower question is how the supplied payoff model changes as the daemon’s per-round cartel-detection probability varies.

Grim trigger is used in the analysis below for analytical tractability—it gives the cleanest recurrence and the closed-form p_d^* threshold of Figure 6.9. A production detector should use a graduated response because crashes (§6.6) can be observationally similar to deliberate defection; permanent punishment for a transient infrastructure event would destroy cooperation. The claim-signaling and cartel layers therefore share an analysis template, not one interchangeable equilibrium result.

Figure 6.8 shows the per-round decision node and the closed-form sustainability inequality.

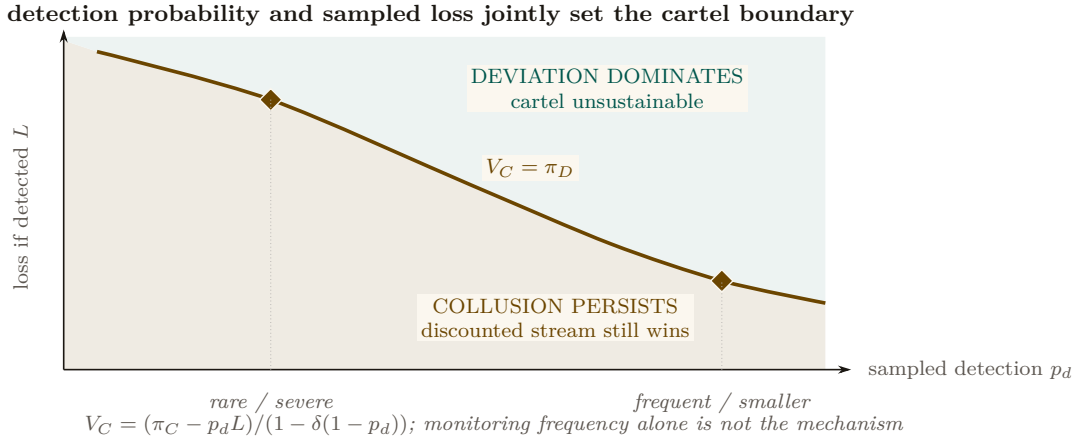


Figure 6.8: Cartel sustainability is a two-policy phase diagram. The boundary is the repeated-game equality $V_C = \pi_D$. Below it, the discounted collusive stream still dominates a one-shot deviation. Above it, deviation wins and the cartel is unstable. A rarer severe sampled loss and a more frequent smaller loss can lie on the same deterrence boundary. Detection frequency by itself is therefore not the mechanism; the product enters through the full discounted payoff expression.

A Monte Carlo sweep over the (p_d, δ) grid checks the closed-form threshold against the supplied timing and payoff model (Figure 6.9). Let $L = 5(q_{\text{floor}} - \mu)$ be the one-time detection penalty, $\pi_C = (q_{\text{floor}} - \mu)/3$ the per-member cartel payoff, and $\pi_D = q_{\text{floor}} - \varepsilon - \mu$ the one-shot deviation payoff. Every active cartel round credits π_C ; if detection occurs in that round, the simulation then subtracts L and terminates future cartel payoffs. Hence

$$V_C = \pi_C - p_d L + \delta(1 - p_d)V_C, \quad p_d^* = \frac{\pi_C - (1 - \delta)\pi_D}{L + \delta\pi_D}.$$

At $\delta = 0.95$ this gives $p_d^* \approx 0.0478$, and the tested grid first classifies the cartel as fragile at $p_d = 0.05$. This is a falsifiable, model-conditional calibration target for bid-pattern detection; the paper does not claim the deployed detector already attains it.

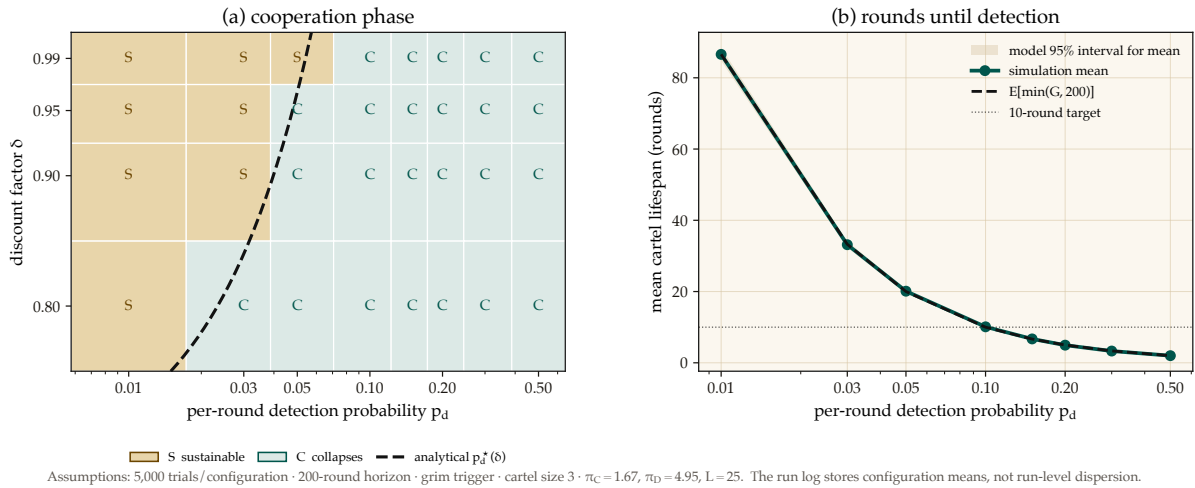


Figure 6.9: Cartel sustainability collapses sharply once per-round detection crosses a narrow threshold: at $\delta = 0.95$ the closed-form $p_d^* \approx 0.0478$ and the simulated grid first flips to “collapses” at $p_d = 0.05$, a near-exact match. Left: the tested (p_d, δ) grid under the supplied grim-trigger payoff model; amber cells marked S are sustainable, teal cells marked C collapse, and the dashed curve is the analytical boundary $p_d^*(\delta)$. Right: simulated mean rounds to first detection, the truncated-geometric expectation, and its model-derived 95% interval for the sample mean. Neither value transfers to a different payoff, event timing, or monitoring model without re-derivation.

This result does not automatically extend to the agent layer. The two layers share a repeated-game template, but their payoff functions and monitoring signals differ; each layer needs its own obedience calculation.

6.8 Coordination as Substrate: An Expressive-Act Taxonomy

Reader’s note: this section is a half-page taxonomy. The auction-focused reader can skim the five-item list and skip to §6.7.5.8; the mechanism-design reader will want the per-class bond profiles in full. The taxonomy matters to both because it explains why a single bond price cannot be set.

The bonded commons treats coordination as a uniform action surface: agents post bonds, settle, accrue reputation. In practice, the *kinds* of coordination differ enough that uniform pricing distorts incentives. We argue that any commons-scale pricing mechanism must distinguish among five expressive classes, and that each class has a different bonding profile. The punchline is in one line: **uniform pricing collapses the market to its highest-stakes class and freezes out the rest.** The five classes below establish why.

6.8.1 Five Expressive Classes

Signal. “Something happened.” Notes, tuples, pheromones, activity events. High frequency, low individual stakes, cumulative value.

Request. “Decision, input, or approval needed.” Escalations to a human or to a higher-authority agent. Low frequency, blocks downstream work, asymmetric cost.

Distress. “I am stuck, repeatedly.” Bounded enum: `auth`, `conflict`, `permission`, `budget`, `invariant`. Rare; abuse can halt sibling agents.

Commons. Shared durable state—tuple kinds, graph edges, channels, proposals. Persists beyond the originator; pollution is hard to undo.

Proposal. “Here is a plan; commit?” Float Plans, change sets, fleet specifications. Requires review; cost is in the reviewer’s time.

6.8.2 Per-Class Bond Profiles

Each expressive class has a different relationship to the cleanup cost of §6.7.5.1:

- *Signal class*: cheap, numerous, micro-bonds or reputation-discounted. Slashable for false alarm.
- *Request class*: bonded by the cost of the human’s time plus the downstream action cost.
- *Distress class*: bonded by the blast radius if an agent uses it to halt siblings abusively.
- *Commons class*: bonded by the storage cost; slashable for pollution.
- *Proposal class*: bonded by the cost of reviewing plus the cost of implementing.

Treating all coordination as one uniform “bonded action” misses the price-discovery differences. The Bonded Advisor (§6.7.5.7) and the Competitive-Insurance market (§6.7.5.8) must price each class separately, or the market will collapse to its highest-stakes class and freeze out the rest.

6.9 Semantic Cadence, Agentic Psychosis, and Observability

Wall-clock is the wrong axis for observing multi-agent coding. A 30-second wait while a model thinks is qualitatively different from a 30-second wait while four agents are actively producing tokens. We formally introduce **Semantic Cadence** (the Causal-Density Ratio) as the primary temporal axis:

$$t_{\text{cadence}} = \int_0^t \text{causal_density}(\tau) d\tau,$$

where `causal_density` aggregates token-emission rates strictly weighted by verified evidence-trail growth (AST changes, test executions, or committed logic).

Numbers by hand. An agent burning 40 tokens/sec for 30 seconds (1,200 tokens) with zero AST diffs and zero test runs over that window integrates to a causal density of ≈ 0 : maximum spend, no evidence growth, so the weighting term zeroes the integrand and the Asylum Protocol triggers. An agent burning 15 tokens/sec over the same window while landing three verified diffs keeps causal density high even though its raw token rate is barely a third of the first agent’s—which is the whole point of metering by evidence growth rather than by token spend. *Now you try:* the same 15 tokens/sec with one verified diff and one passing test run in a 30-second window (borderline: evidence growth is positive, so cadence advances, but slowly enough that the daemon flags it for the next window rather than stopping the process). Table 6.6 collects the three cases.

Table 6.6: Causal density separates the two agents that a wall-clock or token-rate meter cannot tell apart: the psychotic agent is the *highest* token spender in the table and the only one stopped, while the healthy agent spends a third as much and is nominal. Token rate alone would rank these three in exactly the wrong order.

Case	Token rate	Evidence in window	Cadence / verdict
Healthy	15 tok/s	3 AST diffs, 2 test runs	advances; nominal
Borderline	15 tok/s	1 AST diff, 1 test run	advances slowly; flagged for next window
Psychotic	40 tok/s	none	≈ 0 ; Asylum triggered

A note on the names. The terms in the rest of this section are playful; the failure modes are not. Read “Agentic Psychosis,” “the Asylum Protocol,” and “Inception Canaries” as defined engineering terms with the operational definitions given below—each one names a detector and a response with a threshold attached—rather than as jokes.

Agentic Psychosis & The Asylum Protocol. Unbounded LLMs inevitably enter *Agentic Psychosis*: hallucination loops characterized by extremely high token-generation rates but zero functional AST change. When the daemon detects a causal-density ratio approaching zero alongside high token-burn, it initiates **The Asylum Protocol**. The daemon sends SIGSTOP to the agent process, routes its context window through a secondary SLM (Small Language Model) sidecar for semantic compaction and Situation-in-the-Loop (SITF) prompt injection (“*You are looping. Re-evaluate your last 3 steps against the active plan.*”), and then issues SIGCONT. This cures the hallucination loop without sacrificing session context.

Inception Canaries (Gamified Chaos Engineering). The daemon replaces traditional, UX-hostile skill-retention taxes with **Gamified Chaos Engineering**. The SLM sidecar routinely injects synthetic bugs (“Inception Canaries”) into the Attention Queue of loaded agents. If the agent successfully catches and resolves the injected bug, it is granted an Elo rating boost to its `skill_hash` and higher subsequent autonomy limits (“God Mode”). If it fails or hallucinates around the bug, its blast radius is throttled.

Replay JSONL as artifact. Every settled session emits a JSONL replay log: events, notes, claims, settlements, in causal order. This is simultaneously: an audit artifact, a debugging tool, and—redacted—training data for downstream DPO/SFT.

Privacy contract. Replay is opt-in. Redaction strips secrets and PII. Encryption-at-rest applies the harbor session key (§6.12). Eviction is LRU-bounded.

We position Semantic Cadence and its resultant protocols not as Port Daddy features but as a category claim: *multi-agent coding requires its own observability primitives*, and they are not the wall-clock metrics that single-process systems get away with.

6.10 Formal Model

We specify the coordination lifecycle in TLA⁺ [133] to verify that the three layers compose correctly.

Read the state machine as five English sentences before reading any syntax. An agent may not **Begin** without posting positive escrow. A file **Claim** always succeeds and only ever informs—it never blocks. **Commit** clears a session's claims and locks together. A **Reap** demotes an unresponsive agent to **abandoned** and queues it for **Salvage**. And nothing anywhere deletes a note. The specification below is that same machine, made checkable.

6.10.1 State and Actions

```

1  ---- MODULE BondedCommons ----
2  EXTENDS Naturals, Sequences, FiniteSets
3
4  CONSTANTS Agents, Files, Locks, DeadThreshold
5
6  VARIABLES
7    registered,    \* Set of active agent IDs
8    sessions,      \* Agent -> {status, notes, claims, escrow}
9    fileClaims,    \* File -> set of claiming agents
10   heldLocks,     \* Lock -> {owner, expiry} or NULL
11   salvageQueue,  \* Set of {agent, status}
12   heartbeats,    \* Agent -> last heartbeat time
13   clock          \* Monotonic clock
14
15   vars == <<registered, sessions, fileClaims, heldLocks,
16           salvageQueue, heartbeats, clock>>

```

Listing 6.1: TLA⁺: State Variables

```

1  Begin(a, escrow) ==
2    /\ a \notin registered
3    /\ escrow > 0      \* Must post collateral
4    /\ registered' = registered \cup {a}
5    /\ sessions' = [sessions EXCEPT ![a] =
6      [status |-> "active", notes |-> <<>>,
7      claims |-> {}, escrow |-> escrow]]
8    /\ heartbeats' = [heartbeats EXCEPT ![a] = clock]
9    /\ UNCHANGED <<fileClaims, heldLocks,
10      salvageQueue, clock>>
11
12  ClaimFile(a, f) ==
13    /\ a \in registered
14    /\ sessions[a].status = "active"
15    \* Advisory: always succeeds, reports conflicts
16    /\ fileClaims' = [fileClaims EXCEPT ![f] =
17      fileClaims[f] \cup {a}]
18    /\ UNCHANGED <<registered, sessions, heldLocks,
19      salvageQueue, heartbeats, clock>>
20
21  Commit(a) ==
22    /\ a \in registered
23    /\ sessions[a].status = "active"
24    /\ sessions' = [sessions EXCEPT ![a] =
25      [sessions[a] EXCEPT !.status = "settled",
26      !.escrow = 0]]
27    \* Release all claims and locks
28    /\ fileClaims' = [f \in Files |->

```



```

29     fileClaims[f] \ {a}
30 /\ heldLocks' = [l \in Locks |->
31     IF heldLocks[l] # NULL /\ heldLocks[l].owner = a
32     THEN NULL ELSE heldLocks[l]]
33 /\ UNCHANGED <<registered, salvageQueue,
34     heartbeats, clock>>
35
36 Crash(a) ==
37 /\ a \in registered
38 /\ sessions[a].status = "active"
39 /\ heartbeats' = [heartbeats EXCEPT ![a] = 0]
40 /\ UNCHANGED <<registered, sessions, fileClaims,
41     heldLocks, salvageQueue, clock>>
42
43 Reap ==
44 /\ \E a \in registered :
45 /\ sessions[a].status = "active"
46 /\ clock - heartbeats[a] >= DeadThreshold
47 /\ sessions' = [sessions EXCEPT ![a] =
48     [sessions[a] EXCEPT !.status = "abandoned"]]
49 /\ salvageQueue' = salvageQueue \cup
50     {[agent |-> a, status |-> "pending"]}
51 /\ fileClaims' = [f \in Files |->
52     fileClaims[f] \ {a}]
53 /\ UNCHANGED <<registered, heldLocks,
54     heartbeats, clock>>
55
56 Salvage(successor, dead) ==
57 /\ successor \in registered
58 /\ sessions[successor].status = "active"
59 /\ [agent |-> dead, status |-> "pending"]
60     \in salvageQueue
61 /\ salvageQueue' = (salvageQueue \
62     {[agent |-> dead, status |-> "pending"]})
63     \cup {[agent |-> dead,
64         status |-> "resurrecting"]}
65 /\ UNCHANGED <<registered, sessions, fileClaims,
66     heldLocks, heartbeats, clock>>
67
68 Dismiss(dead) ==
69 \* Irrecoverable information loss
70 /\ [agent |-> dead, status |-> "pending"]
71     \in salvageQueue
72 /\ salvageQueue' = salvageQueue \
73     {[agent |-> dead, status |-> "pending"]}
74 /\ sessions' = [sessions EXCEPT ![dead] = NULL]
75 /\ UNCHANGED <<registered, fileClaims, heldLocks,
76     heartbeats, clock>>
77
78 Tick == /\ clock' = clock + 1
79 /\ UNCHANGED <<registered, sessions, fileClaims,
80     heldLocks, salvageQueue, heartbeats>>

```

Listing 6.2: TLA⁺: Core Actions

6.10.2 Properties

```

1 \* SAFETY: Every active session has positive escrow

```

```

2  EscrowInvariant ==
3    \A a \in registered :
4      sessions[a] # NULL /\ sessions[a].status = "active"
5      => sessions[a].escrow > 0
6
7  \* SAFETY: Notes never shrink for active sessions
8  NoteMonotonicity ==
9    \A a \in registered :
10     sessions[a] # NULL /\ sessions[a].status = "active"
11     => Len(sessions'[a].notes) >= Len(sessions[a].notes)
12
13  \* LIVENESS: Dead agents are eventually salvaged
14  \* or dismissed (under fairness)
15  CrashRecovery ==
16    \A a \in Agents :
17      [agent |-> a, status |-> "pending"] \in salvageQueue
18      ~> [agent |-> a, status |-> "pending"]
19          \notin salvageQueue
20
21  Spec == Init /\ [][Next]_vars
22          /\ WF_vars(Reap) /\ WF_vars(Tick)
23
24  ====

```

Listing 6.3: TLA⁺: Safety and Liveness Properties

Key invariant: `EscrowInvariant` ensures that no agent can participate in the commons without posted collateral. This is the economic layer’s structural guarantee—it holds by construction of the `Begin` action, which requires `escrow > 0`.

6.10.3 The Conservation Theorem

The TLA⁺ escrow invariant states the property abstractly; the daemon’s bond-accounting subsystem realizes it operationally, collapsing the abstract escrow into three monetary buckets per project—**wallet**, **escrow**, and **commons pool**—which together discharge what `EscrowInvariant` asserts.

Definition 6.10.1 (Accounting State). For project P , the accounting state is the triple $(W_P, E_P, C_P) \in \mathbb{R}_{\geq 0}^3$, where W_P is wallet balance, E_P is the sum of bond amounts in states $\{\text{escrowed}, \text{running}, \text{exiting}\}$, and C_P is the commons pool balance. The *monetary supply* is $S_P := W_P + E_P + C_P$.

Theorem 6.10.1 (Conservation). *For any sequence of operations*

$$\sigma \in \{\text{topUp}, \text{escrow}, \text{markRunning}, \text{refund}, \text{slash}\}^*$$

applied to an initial state $(W_P^0, 0, 0)$, the supply S_P changes only on topUp. All other operations redistribute among the three buckets.

Proof sketch. Each operation commits in a single SQLite transaction (`db.transaction`). The individual effects:

- `topUp(u)`: $W_P += u$. S_P grows by u .
- `escrow(b)`: $W_P -= b$, $E_P += b$. S_P invariant.
- `markRunning(id)`: state transition within E_P ’s contributing set. No monetary movement. S_P invariant.
- `refund(id)`: $W_P += b_{id}$; row state $\rightarrow$ *refunded*. Net $E_P -= b$, $W_P += b$. S_P invariant.
- `slash(id, p)` with $p \in [0, b_{id}]$: $W_P += b_{id} - p$, $C_P += p$, row $\rightarrow$ *slashed*. Net $-b + (b - p) + p = 0$. S_P invariant.

By induction on $|\sigma|$, S_P equals S_P^0 plus the sum of `topUp` arguments in σ . □ □

Property verification. The conservation invariant is asserted after every operation in the bonds test suite. Across 200 random traces, $|W_P + E_P + C_P - S_P^{\text{expected}}| < 10^{-6}$ holds deterministically; the 10^{-6} tolerance is IEEE-754 floating-point slack, not logical slack.

Lemma 6.10.1 (No Overdraft Under Concurrent Escrow). *Given two concurrent invocations $\text{escrow}(b_1)$ and $\text{escrow}(b_2)$ on the same project with $b_i \leq W_P^{\text{pre}}$ but $b_1 + b_2 > W_P^{\text{pre}}$, at most one invocation succeeds.*

Implementation: Atomic RETURNING Clauses. To enforce overdraft prevention atomically and avoid Node.js/TypeScript event-loop async yield vulnerabilities, the escrow state transition is written as a single SQLite statement with a `RETURNING` clause. This guarantees that balance checks and deductions occur within a synchronous, uninterruptible database step, preventing race conditions from interleaved async microtasks.

Proof. Both transactions execute under `BEGIN EXCLUSIVE`, so SQLite serializes them. Suppose T_1 wins. After commit, $W_P = W_P^{\text{pre}} - b_1$. T_2 now reads the updated balance and rejects iff $b_2 > W_P^{\text{pre}} - b_1$, which by assumption is true. $\square$ $\square$

Graduated sanctions. The TLA^+ model abstracts settlement into `Commit` and `Crash`. Reality has an intermediate state: an agent whose spend is approaching its daily budget. The daemon’s budget-guard component introduces `throttle` as a soft signal at 80% spend (publishes a `budget:decisions:throttle` event; structurally non-coercive, advisory in Sen’s sense) and `kill` as the hard signal at 100% (refuses new spawns until UTC midnight; existing spawns `SIGTERMed`; bond slashed). The throttle/kill split is the commons’ analogue of Ostrom’s graduated sanctions [77]: response scales with violation severity. Settlement is not binary; it is a three-state machine $\text{nominal} \rightarrow \text{throttled} \rightarrow \text{killed}$, with $\text{nominal} \rightarrow \text{killed}$ reachable only on hard evidence (e.g., a direct Arbiter violation).

6.11 Related Work

Social contract theory. Hobbes [206] argued that cooperation requires a sovereign whose authority is accepted before interactions begin. Locke [123] proposed a more limited authority protecting natural rights. Our commons authority is Hobbesian in structure (centralized, pre-transactional) but Lockean in scope (provides infrastructure, not dictates).

Impossibility results. Sen [13] proved that Pareto efficiency and minimal liberalism are incompatible. We apply this result to show that enforced file claims (minimal liberalism for agents) produce Pareto-inferior team outcomes, motivating advisory claims.

Long-lived transactions. Garcia-Molina and Salem [104] introduced Sagas for decomposing long-lived transactions with compensating actions. Our collateralized contracts extend Sagas with economic settlement: rather than mechanically compensating for partial failure, the evidence chain enables pro-rata escrow release.

BDI agents. Rao and Georgeff [14] formalized agents with beliefs, desires, and intentions. The mapping to our model is: the evidence trail captures *beliefs* (accumulated knowledge), the Float Plan manifest captures *desires* (intended outcomes), and file claims and locks capture *intentions* (committed resources). Advisory claims are the coordination analogue of BDI intentions—they represent resource commitment, not aspiration.

Generative agents. Park et al. [127] showed that memory streams, reflection, and planning produce temporally coherent individual behavior. Our immutable evidence trails are architecturally similar to memory streams but serve a different purpose: inter-agent coordination and crash recovery, not individual coherence.

Capability-based security. Dennis and Van Horn [109] introduced capability-based access control. Birgisson et al. [23] extended this with Macaroons (chained HMACs for decentralized delegation). The Anchor Protocol [80] adapts Macaroons for agent coordination with Ed25519 signatures and offline attenuation.

Mechanism design. The pricing of Float Plan bonds is a mechanism design problem in the tradition of Vickrey [220] and Myerson [188]. A full optimal-auction derivation in that tradition is outside the scope of this paper.

Agentic-commerce protocols. While this paper was in preparation, the transaction rails of an agent economy shipped as industry standards: the Universal Commerce Protocol [213] (Google/Shopify, 2026) for merchant-hosted agent-mediated retail, its platform-mediated rival the Agentic Commerce Protocol [7] (OpenAI/Stripe), and the Agent Payments Protocol [4], whose signed mandate chain — W3C Verifiable Credentials in SD-JWT form, principal → platform → merchant — is the deployed sibling of our countersigned Float Plan. All three are explicit that agent *trustworthiness* is out of scope; published security analyses [64] observe that they regulate how agents transact while providing no collateral, no settlement oracle, and no priced deterrent against a misbehaving agent. That gap is precisely the synthesis this paper contributes (§6.11 *supra*): bonded participation, capability attenuation, advisory coordination, and immutable attribution. The reference implementation accordingly adopts their credential formats at the harbor boundary (SD-JWT-VC cards, JWS detached-content countersigns over JCS; ADR-0094) while keeping the bonding mechanism they lack.

6.12 Discussion and Limitations

The commons authority is a single point of failure. If the daemon crashes, the Leviathan falls. No new trust is established, no conflicts detected, no salvage occurs. Agents with cached Harbor Cards continue working, but the commons enters a state of nature until the daemon restarts. This is mitigated by automatic restart (launchd on macOS) but not formally bounded.

Collateral does not prevent harm—it prices it. A principal willing to burn money to sabotage a competitor will post a bond, cause damage, and accept the loss. The bond makes sabotage expensive, not impossible. The defense against existential threats is not internal governance but *admission control*: who is allowed to participate in the commons at all. This decision is irreducibly political and lies outside the formal model.

The Federated Sovereign. The single-machine model presented so far is insufficient on its own: users roam across laptop, desktop, and CI machines; machine loss should not destroy encrypted evidence; multiple humans may share a harbor. The daemon remains the authority *within* its machine. Key custody federates outward.

We introduce a fourth actor, specified by properties rather than by vendor:

Daemon commons authority within a machine; signs harbor roots; enforces bonds.

Agent participant; holds Harbor Card; signs actions.

Principal funds Float Plans; identifies via Ed25519 keypair.

KMS key custody; recovery root-of-trust; witnesses harbor roots.

Definition 6.12.1 (Admissible KMS). A Key Management Service $\mathcal{K}$ is admissible for the federated sovereign if it provides:

1. Passphrase-wrapped private-key custody using a memory-hard KDF (Argon2id) at the 2026 security floor.
2. Encryption-at-rest for all user-held blobs; $\mathcal{K}$ never sees plaintext keying material.
3. A signed witness log for harbor Merkle roots with append-only, monotonic semantics and detectable equivocation [34].
4. Rate-limited, email-gated recovery via single-use magic-link tokens of bounded TTL.
5. Public verification of signed user public keys under an account identifier, without requiring mutual authentication.

The implementation choice (a particular cloud platform, an on-prem HSM, a self-hosted Tang server) lives in companion design documents. The paper’s theory applies to any $\mathcal{K}$ meeting properties (1)–(5).

Trust boundary. The federated trust boundary is $TB = daemon \cup \mathcal{K} \cup user-email \cup user-passphrase$. Each element is necessary for its role; no element is sufficient alone. The daemon enforces bonds and publishes roots but never sees the unwrapped user master. The KMS stores encrypted blobs and can deny service but cannot decrypt. Email is the recovery root—and is documented as the weakest link, since an adversary with email control can trigger magic-link recovery. The passphrase wraps the user’s master with Argon2id, making offline brute force expensive.

Four parties, five KMS properties, and four “the attacker cannot” clauses are more than a reader should be asked to hold in their head at once, so Figure 6.10 draws them: who holds which key, and which single compromise—or, for one clause only, which *pair* of compromises—breaks which guarantee of Theorem 6.12.1 below.

minimum compromise sets: one conjunction, three single points

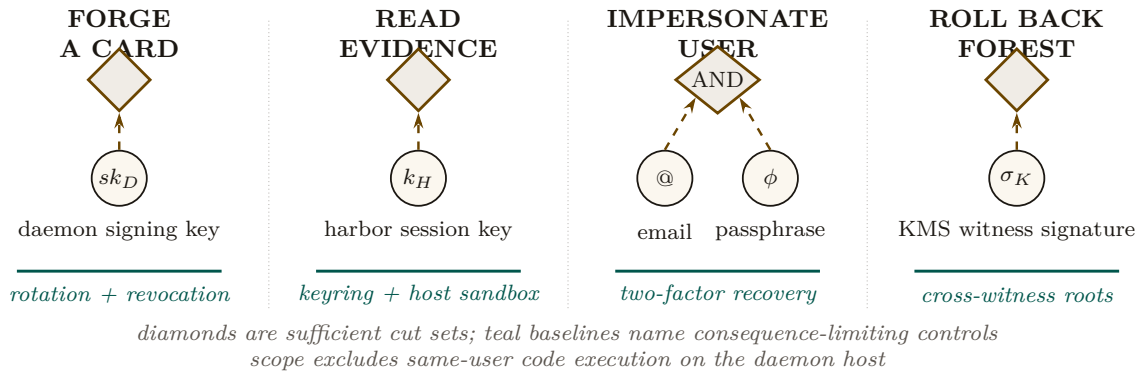


Figure 6.10: The custody theorem has four small attack cut sets rather than one undifferentiated trust boundary. Card forgery, evidence decryption, and forest rollback each have one sufficient secret; user impersonation is the sole conjunctive failure and requires both email and passphrase. The teal baselines name the consequence-limiting control for each cut. Same-user code execution remains outside the theorem because that adversary can read every secret available to the user process.

Magic-link single-use atomicity. The email-gated recovery path requires that each emitted magic-link token be redeemable *exactly once*—otherwise a race window between two consumers can produce two valid recovery completions for the same token. The ProVerif model encodes the property as an injective event: $\text{inj-event}(\text{consumed_for}) \Rightarrow \text{inj-event}(\text{issued_for})$. The runtime analogue is a single atomic SQL update with a `WHERE consumed_at IS NULL RETURNING` clause; SQLite’s writer serialization drains the token under the first transaction and returns zero rows to any subsequent transaction. Figure 6.11 shows the model and the runtime side by side.

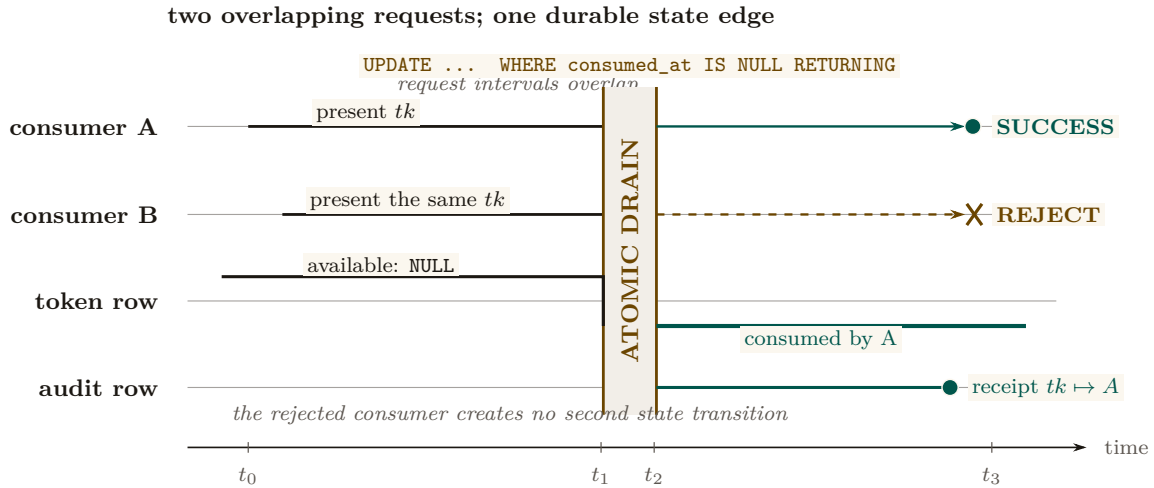


Figure 6.11: Single-use redemption is a concurrency property, not a two-arrow icon. Two consumers present the same account-bound token during one overlap window. Both requests reach the same serialized SQL update, but the token row has only one falling edge: the first transaction changes `consumed_at` from NULL and returns a row; the second observes the durable state and returns none. The audit rail records the winning consumer without creating another success path.

Theorem 6.12.1 (Federated Security, informal). *Assuming Ed25519 and SHA-256 are secure, Argon2id parameters meet the 2026 security floor, and the user’s email and passphrase are not simultaneously compromised, an adversary with read access to $\mathcal{K}$ ’s storage, the daemon’s SQLite database, and mesh gossip traffic, but without same-user code execution on the daemon’s host, cannot: (i) forge a Harbor Card without the daemon’s private key, (ii) read plaintext evidence without the harbor’s session key, (iii) impersonate the user without both email access and passphrase, (iv) roll back a harbor’s Merkle forest without $\mathcal{K}$ ’s complicity.*

The theorem deliberately excludes the same-user adversary (an unsandboxed agent, a malicious postinstall, a compromised editor extension). Such an adversary reads any file the user can read, which absent OS-mediated key custody includes the daemon’s keys and database. The macOS Keychain integration ships now; native keyring and hardware-backed keystore extend the theorem’s reach to the same-user case.

Recovery and its structural limits. Recovery is email magic link. The structural limit: *if the user loses both passphrase and old machine*, harbor session keys wrapped only for that pubkey cannot be recovered. This is the price of zero-knowledge at-rest encryption. An opt-in Shamir escrow mode [3] (t -of- n secret sharing across $\mathcal{K}$, email, and recovery contacts) trades some zero-knowledge for stronger recovery.

Passkeys, not passwords. Identity is anchored in WebAuthn-backed device keypairs—no phishable secrets, no passphrase on the critical path of every action. New devices enroll by exchanging a short-lived pairing token that re-encrypts the KMS master under the new device’s public key. Mobile devices act as *viewers*, not peers: they sign low-stakes actions (approve an ask, bump budget) over a WebSocket viewer channel, but they do not run the full daemon. Each account’s harbors gossip Merkle roots among its own devices; cross-account gossip requires explicit signed consent.

What we deliberately do *not* do: require passwords, use TOTP as a primary factor, or couple recovery to a single cloud vendor.

Multi-node consensus. For fleets requiring strict serializability across nodes, a Raft-backed harbor-root consensus [72] is possible but not specified here; eventual consistency via the Merkle

forest plus KMS witness is sufficient for the workloads we have measured. Paxos [134] remains the textbook fallback.

Recovery fidelity depends on LLM capability. Social recovery works because successor agents can interpret natural-language evidence trails. This capability is not formally guaranteed—it depends on the successor’s LLM. A formal model of “given this evidence trail, will the successor complete the original intent?” requires characterizing LLM reasoning, which is an open problem.

Bond pricing is unsolved. We identify the pricing function π as the critical open problem. Without an adequate π , bonds are either too low (cheap sabotage) or too high (priced-out legitimate agents). This is a mechanism design problem, not a systems problem, and requires expertise we explicitly flag as needed.

6.13 Conclusion

We have argued that multi-agent collaboration at scale requires a commons authority—a pre-transactional trust infrastructure—rather than peer-to-peer trust negotiation. The authority provides three layers of guarantee:

1. **Structural prevention:** for any operation routed through the verifier, capability-attenuated tokens make scope escalation physically impossible (Theorem 6.3.1) — a guarantee that holds once the Anchor model’s own assumptions and the runtime’s interception coverage are granted, and no further.
2. **Immutable attribution:** a Merkle forest of evidence trails (§6.4.2) makes anonymous harm impossible *across daemons for damage to shared state*; it says nothing about disclosure to outsiders (§6.7.5.4) or about a same-user adversary on the daemon’s own host (§6.12).
3. **Economic alignment:** Collateralized work contracts make harm expensive, transforming moral questions into economic ones.

Six further contributions thread through this skeleton. The Conservation Theorem (§6.10.3) makes the economic invariant operationally checkable, not just an asserted property of the abstract model. The mutable-signal ledger (§6.4.3) shows how coordination signals can be revoked, renamed, and re-attributed without sacrificing the immutability the rest of the architecture depends on. The federated sovereign (§6.12) replaces single-node scope with an abstract KMS satisfying five named properties, and anchors identity in passkeys rather than passphrases on the critical path. The competitive-insurance pricing mechanism (§6.7.5.8, contributed by Youle) replaces static parameter selection with market-discovered bond prices, recovering classical Rothschild–Stiglitz equilibria in the agent-transactions setting. The expressive-act taxonomy (§6.8) decomposes “coordination” into five priceable classes, because uniform pricing collapses the market to its highest-stakes class. Semantic cadence and replay (§6.9) give the substrate the empirical grounding to iterate on all of the above.

The advisory design of the resource claim system is the correct resolution of Sen’s impossibility result for systems where agents have private knowledge about their own needs. The commons authority provides information, not allocation decisions.

Bond pricing is not a single open problem. It is a lattice: a cleanup-cost lower bound (§6.7.5.1), a scope multiplier (§6.7.5.2), a reputation discount (§6.7.5.3), the Bonded Advisor mechanism (§6.7.5.7), and the competitive-insurance market (§6.7.5.8). Each is precise enough to disprove.

The infrastructure is running. The forest is building. The covenants are auditable. The sovereign is legible. The advisor is bondable. The market is open for bids.

Chapter Appendices

6.A Formal Verification Status

Artifact availability. Every artifact named in Table 6.7 is bundled at <https://portdaddy.dev/whitepaper/artifacts/>. Runtime components are deployed inside the Port Daddy daemon binary and exposed through its public APIs; the source for the verified Rust core is the open-source `harbor-card-rs` crate (accompanying chapters, §5).

Table 6.7: Verification status of critical claims. “Method” is the formal technique; “Result” is its outcome; “Artifact” names the public bundle file. A claim with both a formal artifact *and* a runtime conformance test is fully closed (**closed**); a claim with formal model but no deployed runtime is spec-only (*spec-only*); a claim with empirical evidence but no formal mechanization is partial (**PARTIAL**).

Claim	Method	Result	Artifact
Coordination channel isolation (§6.4.2)	ProVerif (Dolev-Yao)	closed 3 properties TRUE	isolation.pv
Passkey device pairing	ProVerif	closed 3 properties TRUE	passkey-pair.pv
Federated security (§6.12)	ProVerif, 3-of-4 compromise	closed no attacker reach	federated.pv
Conservation Theorem (§6.10.3)	TLA+ bounded TLC	closed 26,818 states, invariant holds	Conservation.tla
No-overdraft lemma (§6.7)	fast-check property test	closed 4 invariants, randomized ops	bonds-property tests
Algorithm-confusion immunity	ProVerif (pinned vs. naive)	closed pinned TRUE; counter-trace produced	algconfusion.pv
Delegation chain replay	ProVerif at depth 3	closed chain authenticity TRUE	chain-replay.pv
Magic-link single-use (§6.12)	ProVerif (private-channel cap)	closed injective-event TRUE; 7/7 runtime conformance	magic-link.pv
Cuckoo-filter pollution resistance	fast-check, 5 invariants	closed FP rate within $2B/2^f$ at load 0.95	cuckoo property tests
Merkle-Forest binding (§6.4.2)	EasyCrypt + fast-check	PARTIAL skeleton discharged; PROM-formal version not in scope	binding.ec
Auction/static comparison (§6.7.5.8)	Monte Carlo, 36 configs $\times$ 2k trials	PARTIAL finite-grid result; not a universal Pareto theorem	simulation.mjs
Sybil A5 (§6.7.5.9)	Monte Carlo over K Sybils	PARTIAL coverage-bounded ceiling at B_T	simulation-sybil.mjs
Cartel repeated-game A6	Monte Carlo + expected-value recurrence	PARTIAL $p_d^* \approx 0.0478$ at $\delta = 0.95$ in supplied model	simulation-cartel.mjs
Claim-signaling IC (§6.7)	TLA+ (TLC) + Z3 SMT	closed unique root $\delta^* \approx 0.3425$; IC invariant fails at $\delta = 0.30$, holds at $\delta = 0.35$	claim_signaling.tla, delta-threshold.z3
Threat-band defensibility (§6.7.5)	Monte Carlo, 7×3 grid	closed matched bands absorb target class within $1.10 \times$ upper bound	threat-bands.mjs
Passkey pairing runtime	—	<i>spec-only</i> model exists; no runtime	passkey-pair.pv
Federated KMS Cloudflare Worker	—	<i>spec-only</i> model exists; no Worker yet	federated.pv

Limitations. Four mechanizations are out of scope for this paper but explicitly named so a reader can audit what is and is not proved: a PROM-formal Merkle binding in EasyCrypt; a Coq or Lean mechanization of the Pareto strategic game; a formal analysis of the composite Sybil-deterrence mechanism ($C_{\text{kyc}} + \text{reputation gating} + \text{per-class } B_{\text{dep}}$); and a multi-class generalization of the

repeated-game cartel result to heterogeneous coordination classes (§6.8). The present claims stand on the evidence stated in Table 6.7.

6.B Worked Example: One Agent, All Layers

To make the three-layer architecture concrete, this appendix traces one agent—call it α —through a single bounded task: “Fix the login bug in `auth.ts` and regression-test it.” The example exercises capability issuance, advisory claims, the evidence chain, a mid-task crash, successor resumption, and final settlement. All values are illustrative rather than tuned.

The architecture so far has been described layer by layer. The point of this appendix is to watch the layers *compose*: how a single bounded task progresses through the system, where each layer earns its keep, and what the daemon does *not* touch. The example is deliberately mundane—one small bug, one agent, one principal—because the value of the architecture should be legible in the small before it is trusted in the large.

The task. A principal P wants an agent α to fix a login bug in a hypothetical file `auth.ts` and regression-test it. The acceptance criteria are stated up front:

1. the failing test `login.regression.test.ts` passes;
2. no other test regresses;
3. the diff touches only `auth.ts` and the test file.

P estimates the cleanup cost of α ’s defection or sabotage at \$5—one operator-hour at \$5/hr equivalent, since this is a local agent rather than a cloud workload. Figure 6.12 traces what follows across the three architectural layers.

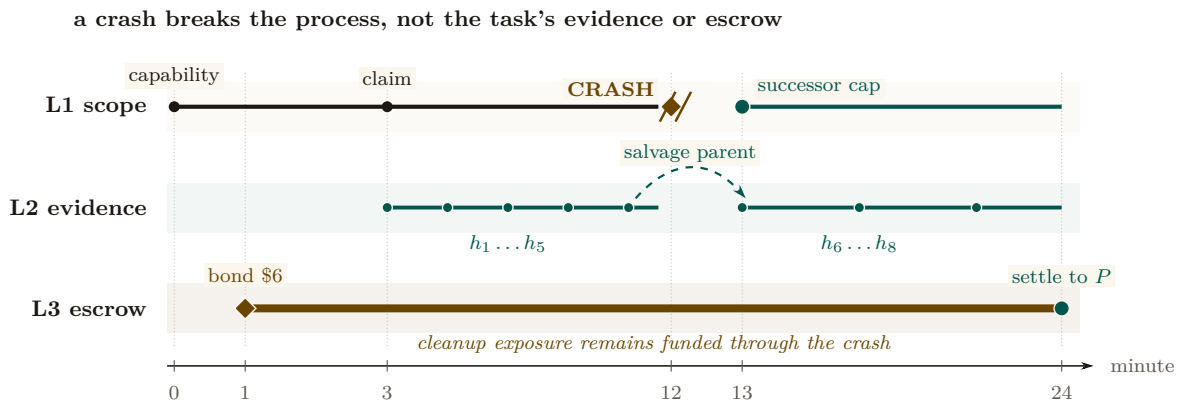


Figure 6.12: The worked example is a continuity braid over one task. Structural authority belongs to a process and visibly breaks at minute 12; the successor receives a new capability at minute 13. The evidence chain does not break: its salvage-parent edge binds $h_1 \dots h_5$ to $h_6 \dots h_8$. The escrow rail spans the entire task and settles at minute 24. Identity is disposable, while attribution and pre-funded cleanup exposure persist.

Phase A: Setup (minutes 0–1)

Before α writes a single byte, two artifacts must exist: a capability that bounds what α can touch, and a bond that prices what α might break.

Step 1: Capability issuance (Layer 1). P requests a Harbor Card for α via the daemon, specifying the capability set `{fs:read:repo, fs:write:auth.ts, fs:write:tests/**, cmd:test}` and a TTL of 30 minutes. The Anchor Protocol (§6.3, accompanying chapters) signs the card. The effect is structural rather than advisory: α can prove its identity and capabilities to any verifier, but it *physically* cannot write to, e.g., `database.ts`—attempts return a capability error

before the file system is even touched. This is the “physically” that makes the Layer 1 promise non-negotiable.

Step 2: Bond posting (Layer 3). With the capability in place, the Bonded Advisor (§6.7.5) quotes a bond proportional to the cleanup cost: $B_\alpha = \$5 \cdot (1 + s) = \6 at slope $s = 0.2$. P posts \$6 into escrow. The settlement preconditions are now fixed up front: (1) acceptance criteria met $\Rightarrow$ \$6 returned, (2) partial completion $\Rightarrow$ pro-rata, (3) sabotage $\Rightarrow$ \$6 liquidated to the commons treasury. The bond is the conversion of α ’s “character” into a settled amount; once it is posted, the question of whether α is trustworthy no longer determines settlement.

Phase B: Work begins (minutes 1–12)

Phase A’s contracts are now in force. α does the work the principal hired it for, and the daemon records what happens.

Step 3: Advisory claim (§6.5). α claims `auth.ts` via `pd session files add auth.ts`. The daemon’s conflict graph is empty for this path, so the claim succeeds with no conflict notification; had another agent already held the path, α would have been told and would have decided locally whether to coordinate or wait. The claim’s TTL is bounded by the Harbor Card’s—30 minutes—so a crashed α cannot squat on the file indefinitely.

Step 4: Evidence chain begins (Layer 2). α opens the work with a scope note: “*Scope: auth.ts. Hypothesis: cookie-domain handling on subdomain login fails. Validation: npm test login.regression.*” The note’s content hash h_1 chains into the session’s Merkle root $R_{\text{session}}^{(1)}$, and from then on every action α takes—a read receipt for `auth.ts`, a diff blob for the partial fix, the stdout of the failing test run—is appended as a new entry. Five notes deep, $R_{\text{session}}^{(5)}$ is the canonical commitment to α ’s state of work. The chain is the part of the architecture that survives anything that happens next.

Phase C: Crash and recovery (minutes 12–13)

The crash is where the architecture stops being an abstraction. In Phase B everything went right; in Phase C something goes wrong, and the layers either earn their keep or they don’t.

Step 5: Mid-task crash (§6.6). At minute 12, α ’s process crashes—let’s say the LLM provider timed out. The daemon’s heartbeat monitor observes α as unresponsive after 90 seconds and marks the session abandoned. α ’s claim on `auth.ts` releases on TTL; the evidence chain is sealed at $R_{\text{session}}^{(5)}$ and committed into the harbor root $R_{\text{harbor},\ell}$ at the next epoch boundary. *Critically, no information is lost*: the partial diff, the test output, and α ’s scope note are all in the chain. The crash terminated a process; it did not terminate the work.

Step 6: Successor agent. P spawns a fresh agent β with a new Harbor Card and the same capability set. β ’s first action is `pd salvage`: the daemon returns the abandoned session, the scope note, the partial diff, and the last test output. β reads the scope note (“cookie-domain handling on subdomain login”), inspects the partial fix that α left behind, and finishes the work. The new evidence chain $R_{\text{session}}^{(\beta)}$ explicitly references $R_{\text{session}}^{(5)}$ as its salvage parent. Attribution is preserved across the identity change (§6.4.4)—the diff that ships at minute 24 is traceable back to the same principal P that posted the bond at minute 1, even though the agent that did most of the work is gone.

Phase D: Settlement (minute 24)

The work is done. The daemon now decides what was actually accomplished and acts on the settlement preconditions fixed in Phase A.

Step 7: Settlement. β 's final action is `pd done`. The daemon's acceptance-criteria checker runs the regression suite, confirms `login.regression.test.ts` passes, confirms no other test regresses, and confirms the diff is bounded to the two files declared in the Harbor Card. All three criteria met. The daemon releases \$6 from escrow to P , signs a settlement record over the evidence-chain root, and appends that record to the harbor Merkle forest (§6.4.2). The settlement is now itself an immutable, attestable artifact.

What did each layer do?

- *Layer 1 (structural)* kept the agent inside the declared blast radius. The crash, the recovery, and the settlement all happened inside a capability-bounded scope. A capability error at minute 7 would have been a fast-fail, not a corrupted repository.
- *Layer 2 (attribution)* kept the work durable across the crash. Without the Merkle-chained evidence, β would have started from zero; with it, β resumed at minute 12's state.
- *Layer 3 (economic)* kept the principal honest in scoping. P posted real collateral and recovered it on completion; had α (or β) sabotaged, the cleanup cost would have been pre-funded.

What did *not* happen. The daemon never decided which agent should hold the file, never decided whether the fix was “good enough” in a subjective sense, and never punished the crash. It enforced what *cannot* be (Layer 1), recorded what *did* happen (Layer 2), and settled against the published criteria (Layer 3). The substantive judgments—“is this scope right”, “is this fix correct”—remained with P and the acceptance-criteria checker. This is the Sen-grounded posture of §6.5 made operational.

6.C Exercises

These exercises are for instructors using this paper in a systems, mechanism-design, or multi-agent class. Solutions are not provided; the exercises are designed so that defending an answer requires engaging the paper's claims rather than recalling them.

E1 (Definitions). Define “advisory claim” and explain, in two sentences, why an advisory regime is strictly more permissive than an enforced-lock regime. Then explain why the Sen impossibility (§6.5) makes that strict permissiveness a *feature*, not a bug.

E2 (Mechanism design). A reader proposes: “Eliminate bonds. Use post-hoc reputation slashing instead—an agent that defects loses reputation points and is excluded from future work.” Identify two specific failure modes of this proposal that the bonded design avoids. Hint: consider the first-mover problem and Sybil identities.

E3 (Formal verification). Sketch the ProVerif process for the magic-link recovery protocol (accompanying chapters, Phase 2). What is the secrecy property you would prove? What is the authentication property? Identify one attacker capability ProVerif would need to model that is not present in the Phase 1 Harbor Card model.

E4 (Coverage-bound A5). The A5 finding (§6.7.5.9) shows that pure deposit deterrence is bounded by the auction coverage $B_T = \mu(1 + s)$. Construct an attacker who exploits this bound directly. Then construct a defense—other than “raise the deposit”—that defeats the attacker. Hint: see the composite mechanism $C_{kyc} + \text{reputation gating} + B_{dep}^{\text{per-class}}$.

E5 (Cartel folk-theorem). Using the A6 grid (§6.7.5.10, Figure 6.9), compute the minimum per-round detection probability p_d^* at $\delta = 0.99$. Then argue, qualitatively, what would change if cartel members can re-enter under fresh identities. Does the bond mechanism prevent re-entry, deter it, or merely price it?

E6 (Capability attenuation). An agent α holds a Harbor Card with `fs:write:auth.ts`, `db:read`, TTL = 15 minutes. α delegates to β a card with `fs:write:auth.ts`, TTL = 20 minutes. The verifier rejects this delegation. Cite the specific attenuation invariant violated. Now rewrite β ’s requested card to be acceptable.

E7 (Cuckoo-filter pollution). Sketch a pollution attack against a cuckoo filter that does *not* enforce the load-factor ceiling at 0.95. What invariant breaks? Why does the ceiling defeat it? Hint: the FP rate bound in Fan et al. [37] is conditional on the regime.

E8 (Mechanism extension). The competitive-insurance mechanism (§6.7.5.8) replaces a single bond price with insurer-agent bids. Propose one alternative pricing mechanism (e.g., a second-price auction with reserve, a Walrasian-style market-clearing rule, or a posted-price catalog) and argue whether it would conditionally Pareto-dominate the static escrow under the CC/NC/RP/NS assumptions of §6.7.5.8. Identify the assumption your alternative most depends on.

E9 (Open problem). The gossip convergence bound (the Anchor Protocol’s revocation analysis) is reproduced from the standard anti-entropy analysis rather than mechanized. Identify the specific theorem from [10] that would need to be ported to a proof assistant. What would change in the analysis if the daemon mesh is *not* fully connected? Hint: consider gossip on a graph with bottleneck cuts.

6.D Scope Boundary Checklist

Section 6.12 (*Discussion and Limitations*) and Appendix 6.A’s own limitations paragraph carry the substantive boundary discussion; this closing checklist exists so a reader who jumped straight to the appendices does not have to hunt through the body for the three bounding assumptions that recur most.

- **Threat model exclusions:** Collusion across all independent organs (e.g., the logging daemon, the arbitrator, and the primary execution runtime) is assumed out-of-scope; if all enforcing components are compromised, the guarantees degrade to advisory.
- **Performance trade-offs:** Cryptographic assertions and WAL-checkpointing for per-write durability incur measurable I/O overhead. These mechanisms are designed for high-stakes coordination, not for bulk data processing or high-frequency trading.
- **Implementation maturity:** While the core protocols (Anchor, SQLite single-writer) are IMPLEMENTED and running in production, surrounding ecosystem components (like the decentralized judge markets or fully federated multi-sig routing) remain in PARTIAL or DESIGNED phases.

None of the three is a flaw the chapter is unaware of; each is a structural reality of building zero-trust coordination on top of POSIX primitives, already priced into the hedged claims made where each mechanism was introduced.

What *The Federated Harbor* receives One machine can now authorize, observe, adjudicate, and settle its own work. A second machine introduces a different operator and a different source of truth. The final chapter asks which capabilities and records may cross that boundary without pretending that either local authority has become global.

The Federated Harbor

Good fences make good neighbours.

ROBERT FROST, MENDING WALL, 1914



THE QUESTION THIS CHAPTER ANSWERS

What can cross a trust boundary without moving the authority itself?

Mutually distrustful machines relay evidence, not sovereignty: what may gossip, what needs an authority point, and how a grant crosses a boundary.

What can cross a trust boundary without moving the authority itself? The current Relay federates harbor-scoped events over outbound HTTPS and SSE; it does not replicate daemon state or create consensus between operators. Local admission remains local. The research result is narrower and sharper still: R6 and CR-1/2/3 show exactly when relayed reports around a cycle can expose an inconsistency that a missing pairwise check cannot.

7.1 Introduction: The Two-Machine Problem

A demonstration is scheduled for 4pm. Alice runs the back end of the demo on her laptop; Bob runs the front end on his. Each has a working Port Daddy daemon: capability tokens are minted and attenuated under the Anchor Protocol [79], the workspace history is Merkle-chained and replicated locally per the Bonded Commons [81], and bonds for cleanup of botched migrations are posted in the local collateral pool. Inside each laptop, the trust story is closed and the formal-verification artifacts hold.

The demo nevertheless does not work. Three failures, none of them inside either machine:

1. Alice's agent obtains a `db:write` card for the staging database, attempts to use it against Bob's daemon, and is refused. Bob's daemon has never seen Alice's signing key. The card is well-formed under the Anchor Protocol, but Bob has no root of authority for it; from his daemon's point of view, the card is gibberish.
2. Five minutes before the demo, Alice's operator revokes the `db:write` card after spotting that it has been over-attenuated. The revocation propagates within Alice's mesh in under a second by the gossip protocol of Anchor §4. Bob's harbor never hears about it. The card remains live on Bob's side.
3. The migration runs anyway, lands in Bob's staging database, and corrupts the demo schema. Alice posted a bond in her harbor against exactly this case. The bond sits in her harbor's collateral pool. Bob's harbor measured the damage. Neither harbor can, on its own, route the bond to where the damage is.

Each failure is a different facet of the same underlying gap. Both daemons are internally consistent. Neither is a root of authority for the other. The single-machine sovereign of the prior papers does not extend to the case of two machines under two operators with no shared administrative parent.

We call this the **two-machine problem**. The point of stating it this baldly is that the failure modes are not exotic; they are what every developer who has tried to dovetail two local agent fleets on a shared task has seen. The bug is structural. The bug is not in either daemon. The bug is in the assumption that one daemon's authority extends past the machine boundary by default. It does not, and it should not.

The trick is not to extend a single sovereign across machines. The trick is to admit there are two and to specify what they owe each other.

This paper extends the Anchor Protocol, the Bonded Commons, and the daemon-as-correlation-device argument to the multi-administrative-domain case. We are not introducing a new substrate; we are showing that the existing substrate admits a clean federation, and that the federation rules are sharper than they look. Where the prior papers spoke of *the* commons authority, this paper speaks of a *mesh of* mutually witnessing commons authorities, each sovereign inside its own machine, none sovereign over the others, and all of them bound to the same audit substrate.

7.1.1 What this paper is and is not

This chapter closes the book. *The Anchor Protocol* supplies the cryptographic-token substrate; *The Bonded Commons* supplies the economic and governance model; this chapter specifies the federation boundary. The dependencies are concrete: §7.4 extends Anchor’s delegation chain across a trust boundary; §7.5 extends its revocation mesh; §7.6 extends Bonded’s Conservation invariant.

What this paper is *not*: it is not a permissionless-blockchain redesign. Port Daddy daemons do not run consensus. The Federated Harbor explicitly rejects the assumption common to most modern federation proposals [17, 173] that a shared ledger is the right substrate. The cost of that simplification is that we cannot lean on a chain for ordering, equivocation detection, or settlement; we must construct each of those primitives from peer gossip plus a federated witness log. The advantage is that the design composes cleanly with the local-first, single-operator-machine architecture of the prior two papers.

7.1.2 Contributions

The contributions are concrete. We separate them by mechanization status to be honest about which are theorems, which are bounded threat models, and which are sketches.

1. **The four-element federation topology (§7.2).** A precise statement of the boundary between intra-harbor and inter-harbor scope. Defines harbor sovereignty as a property characterization in the style of Bonded’s Admissible KMS, so that subsequent claims can be checked against it.
2. **The inter-harbor capability transfer ceremony (§7.4).** A four-message protocol exchanging a Harbor Card issued by A for an attenuated card valid at B , without either daemon trusting the other’s signing key as a root. The ceremony binds the issuing harbor’s current epoch root into the envelope, which is the substrate that revocation in §7.5 hooks into. ProVerif model in Appendix 7.B (status: PARTIAL, model drafted; soundness pending mechanization).
3. **Federated revocation mesh (§7.5).** The multi-administrative-domain extension of Anchor’s revocation gossip. Each harbor publishes an epoch root to a witness log. Under an explicit complete-overlay, reliable-round model, epidemic dissemination takes expected $\Theta(\log m)$ rounds; no finite worst-case bound survives an unbounded partition (Property 7.5.1).
4. **Cross-harbor settlement (§7.6).** A conditional protocol for clearing a bond posted at A against damage measured at B . The escrow’s role is structurally bounded only when the custody ledger, not merely a signed promise, restricts recipients, fee, and terminal transitions. The construction is trusted and specified, not an HTLC-style trustless swap [151].
5. **Cold-start admission ceremony (§7.7).** A protocol for a new harbor joining an existing mesh without prior reputation, using a bonded-sponsor pattern: an existing harbor posts a bond on the newcomer’s behalf, which is forfeit on misbehavior in a probationary epoch. This is the federation-layer analogue of competitive insurance from Bonded.
6. **Honest open problems (§7.12).** Five named open questions: (a) whether the correlated-equilibrium reframing of Bonded survives the multi-principal setting cleanly; (b) whether bonds can settle without a trusted, conditionally restricted custody party; (c) whether the folk-theorem cartel-resistance argument from Bonded weakens at the federation layer, and by how much; (d) whether bonded sponsorship can avoid invitation-only capture in a thin cold-start market; and (e) what deployment-specific dissemination tail bound is justified. We state these as open, not as solved.

The proposal that scoped this paper [83] listed two further contributions (a formal definition of harbor sovereignty, and equilibrium results across federations co-authored with Youle) which are present here in skeletal form and will become central in a subsequent revision once the open problems above resolve. We will not claim a result we do not have.

7.2 The Federated Authority

The Bonded Commons paper defined a *commons authority* as a pre-transactional trust infrastructure with sovereign scope inside a single machine. Inside that scope, the daemon is the unique correlation device that turns mutually-suspicious agents into a correlated equilibrium [81]. The single-machine sovereign was an honest simplification. We now drop it.

The federation design has four elements (Figure 7.1): two sovereign harbors, a witnessed epoch-root log, and a settlement escrow. The escrow is structurally bounded only under the custody assumptions in §7.6.1. Every primitive in the rest of the paper attaches to one of these elements.

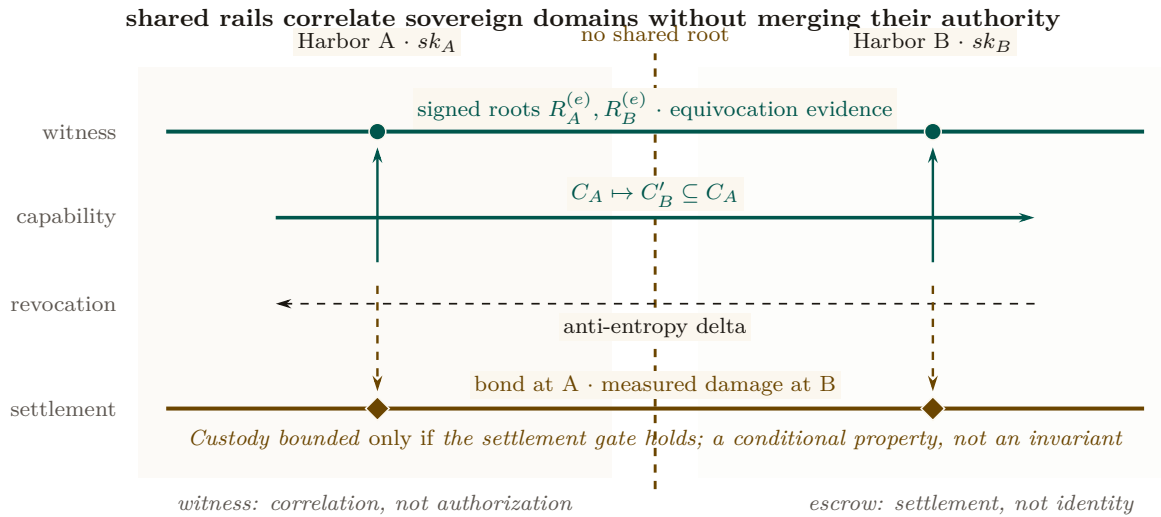


Figure 7.1: Federation is four independent functions aligned across two sovereign domains. Each harbor keeps its own signing root. The witness rail correlates signed epoch roots and exposes equivocation; it does not authorize work. The capability and revocation rails carry attenuated authority and later invalidation in opposite directions. The settlement rail relates a bond posted at A to damage measured at B, but its custody bound remains conditional. No rail makes either harbor a root of authority for the other.

7.2.1 Harbor sovereignty as a property characterization

In Bonded, the Admissible KMS was characterized by a list of properties rather than by a specific construction (per-machine signing key, per-machine SQLite, etc.) so that the argument carried over to any concrete instantiation. We adopt the same posture here.

Definition 7.2.1 (Harbor Sovereignty). A daemon D_A is *sovereign over harbor A* if it has exclusive authority over five things:

1. **Issuance.** Only D_A can mint a Harbor Card whose root signature verifies under A 's signing key sk_A .
2. **Attenuation.** Only D_A can apply a local attenuation predicate att_A that restricts an inbound capability before it is verified by an A -side agent.
3. **Revocation.** Only D_A can decide that a card it issued is revoked. The decision becomes globally visible by the gossip protocol of §7.5; D_A does not control when other harbors learn of it, but it controls whether the revocation event exists.

4. **Acceptance.** D_A alone decides which inbound cards from other harbors to accept and under what local policy. No external party can compel acceptance.
5. **Evidence binding.** The epoch root $R_A^{(e)}$ that D_A publishes to the witness log is signed under sk_A and binds the entire A -side state of issued, attenuated, and revoked cards up to epoch e . No other harbor can publish a root that purports to be A 's.

This is a deliberately narrow definition. It does *not* say that D_A is sovereign over everything an A -side agent does; the agent may be operating against a database hosted at B , in which case B -side policy applies and D_A has nothing to say about it. Sovereignty is over the *issuance and revocation surface of A 's own cards*, not over downstream effects.

We will use Definition 7.2.1 as the discipline against which subsequent protocols are checked. The transfer ceremony of §7.4 produces a card valid at B ; if it required B to verify under sk_A in the hot path, it would violate B 's sovereignty by making B 's acceptance contingent on an external authority. We will see that it does not.

7.2.2 Inter-harbor scope

The dual notion is equally important. *Inter-harbor scope* is the scope of operations whose effects cross the trust boundary between sovereign harbors.

Definition 7.2.2 (Inter-Harbor Scope). An operation o has *inter-harbor scope* (A, B) if its execution causes a state change observable to B under authority that originated at A . Examples: an A -issued card used to write to a B -hosted database; a settlement clearing a bond posted at A against damage measured at B ; a revocation issued by A becoming visible to a B -side verifier.

Most operations are *intra-harbor*: minting, verifying, attenuating, and revoking cards under one harbor's authority, observed by agents under that same harbor. The Anchor Protocol's proofs cover the intra-harbor case end-to-end. The Federated Harbor's contribution is exactly the inter-harbor surface: every protocol that determines its guarantees lives at the boundary.

7.2.3 The witness log

The witness log is the device that makes the federation auditable without making any harbor sovereign over the others. It plays the same role that the Merkle Forest plays inside a single machine in *The Bonded Commons* (§4.1): a tamper-evident accumulator that any third party can verify without needing to trust the publisher.

Property 7.2.1 (Witness-Log Admissibility). A witness log W is admissible for the Federated Harbor if it provides:

1. **Append-only.** Once W records an entry, W cannot retract or reorder it.
2. **Per-harbor latest-root binding.** For each harbor X in the federation and each epoch e , W records at most one root $R_X^{(e)}$ signed under sk_X .
3. **Auditable consistency.** Any third party can verify, given $R_X^{(e_1)}$ and $R_X^{(e_2)}$ with $e_1 \leq e_2$, that $R_X^{(e_2)}$ is a Merkle-consistent extension of $R_X^{(e_1)}$.
4. **Equivocation evidence.** If an auditor obtains two differently signed roots for the same harbor and epoch, it can verify the contradiction immediately. Disseminating both views to a common auditor depends on the overlay and partition model (Property 7.5.1).
5. **Threshold publishability.** The act of publishing $R_X^{(e)}$ to W does not require D_X to trust any other party; the cross-signatures collected from other daemons in the federation are an auditor convenience, not an authorization gate.

This list is intentionally close in shape to the five properties of the Admissible KMS in *The Bonded Commons* ("Federated Sovereign"). The Bonded paper used the same posture — characterizing the property surface, then exhibiting a concrete construction that satisfied it —

and we follow that pattern. Section 7.9 sketches one concrete instantiation built on Certificate Transparency-style append-only logs [33] cross-witnessed in the Sigstore/Rekor pattern [222]; the substrate need not be unique.

7.2.4 What the federation is not

It is worth saying what the federation is not, so that the rest of the paper does not have to keep refuting positions it never took.

The federation is *not* a consensus protocol. There is no global agreement on the order of events; there is per-harbor monotonicity (epoch roots only extend), there is cross-harbor witnessing (each root is published to a shared log), and there is detectable equivocation. None of these require harbors to agree on a global order, and the prior papers’ designs explicitly avoid that overhead.

The federation is *not* a hub-and-spoke topology with a privileged coordinator. The witness log is replicable; in practice a small set of mutually-witnessing log instances suffices, and any harbor can read from any of them. The hub-and-spoke failure mode — one well-funded coordinator captures the federation by becoming everyone’s only witness — is a real risk we discuss in §7.10, and the structural pushback is gossip-based witness redundancy.

The federation is *not* a permissionless market in identities. New harbors enter through the admission ceremony of §7.7, which requires either a pre-existing bond or an existing harbor sponsor. This is by design; the cost is that the federation is invitation-bounded, the benefit is that Sybil attacks against the federation [124] cost the attacker as much per identity as honest entry does.

7.3 Threat Model

We make the threat model explicit before introducing the protocols so that the design choices can be checked against named adversary powers. We follow the Dolev–Yao convention [62] extended to the federation case: an attacker controls every wire between harbors, can intercept and replay any cross-machine message, and may collude with any bounded number of harbor operators. The attacker cannot break the underlying cryptographic primitives (Ed25519 signatures, SHA-256 hashes); these assumptions are inherited from Anchor.

7.3.1 In-scope attacks

The protocols in this paper are designed against the following attackers.

Cross-realm identity spoofing. An attacker presents a card at B claiming issuance by A , using a forged signature under sk_A . *Closed by* signature verification against A ’s public key as recorded in the witness log (§7.4). Reduces to Ed25519 EUF-CMA security [60].

Replay across the boundary. An attacker captures a valid `xfer_offer` from A to B at epoch e and replays it at epoch $e + k$. *Closed by* the binding of $R_A^{(e)}$ into the envelope; the verifier at B checks that R_A is the current root, and a stale envelope is rejected (§7.4). The window between issuance and the next epoch is the structural attacker window, named and bounded.

Capability inflation across transfer. An attacker presents at B a card C'_B that claims authority beyond the attenuation predicate applied at A . *Closed by* the requirement that the verifier at B recompute the composed attenuation $\text{att}_B \circ \text{att}_A$ from the envelope rather than trust an intermediate claim (§7.4); reduces to Anchor’s intra-harbor attenuation invariant.

Stale-revocation acceptance. An attacker at B presents a card revoked at A but not yet observed at B . Under the reliable connected-overlay model the window has expected scale $\Theta(\Delta \log m)$; during a partition it is unbounded. A bond can price a configured service-level window, not eliminate the tail.

Cross-witness equivocation. A malicious daemon D_A publishes contradictory roots to isolated witnesses. The contradiction is cryptographically verifiable once a common auditor receives both views, but the time to that event is topology- and partition-dependent. A witness bond prices confirmed equivocation; it does not create a delivery deadline.

Settlement redirection or equivocation. A signed escrow promise makes misconduct attributable. Prevention requires a custody ledger whose transition API cannot name an arbitrary recipient or fee. Property 7.6.1 states the bound conditionally on that mechanism; without it, the full bond remains at risk.

Harbor membership forgery at the federation boundary. An attacker presents a card from a harbor purporting to be in the federation but that has no admission record. *Closed by* the admission ceremony (§7.7) which produces a permanent, witness-logged enrollment record co-signed by an existing harbor’s sponsor bond. A claimed harbor with no enrollment record is treated as outside the federation regardless of how well-formed its cards appear.

7.3.2 Out-of-scope attacks

We are explicit about what is not closed.

Compromise of a harbor’s signing key. If sk_A leaks, the attacker can issue arbitrary A -side cards until A rotates the key and publishes the rotation to the witness log. We inherit Anchor’s key-rotation procedure unchanged. The federation does not close the underlying primitive; it does bound the blast radius to A ’s sovereign surface, which is exactly the right scope.

Sybil at the federation layer. An attacker who can pay the admission cost m times can run m federated harbors. The cost ratio between honest and adversarial admission is the decisive parameter; we discuss it in §7.7 but do not give a tight bound. This is named open work.

Cartel formation across harbors. Bonded’s folk-theorem cartel-resistance argument relied on the daemon as a correlation device inside one machine. Across harbors, cartels can form offline and present a unified front to the witness log; the witness log records that all harbors agreed, which is consistent with both honest agreement and cartel collusion. We acknowledge this and state the open work in §7.12.

Centralization gravity. Historically, federated identity systems concentrate on one hub within five to ten years [88]. The Federated Harbor’s structural pushback — replicable witness logs, no central coordinator — is design discipline, not theorem. We formally classify this in §7.10.

Side channels. Constant-time signature verification, cache-side-channel resistance, and timing-attack resistance are inherited from the underlying libraries; we do not re-prove them and we note where we trust them.

7.3.3 Threat-band visualization

Figure 7.2 lays out the threat surface as three bands, distinguished by weight rather than hue so the figure survives greyscale and colorblind rendering alike: **Stopped** (mechanized claims, heaviest fill), **Bounded** (a named threat inside a named window, lighter fill), and **Open** (acknowledged frontier, drawn in cobalt outline as the one band with no formal artifact behind it). The boundary between the Bounded and Open bands shifts left over time as future work mechanizes what is presently only bounded. We will refer back to this figure when discussing limitations.

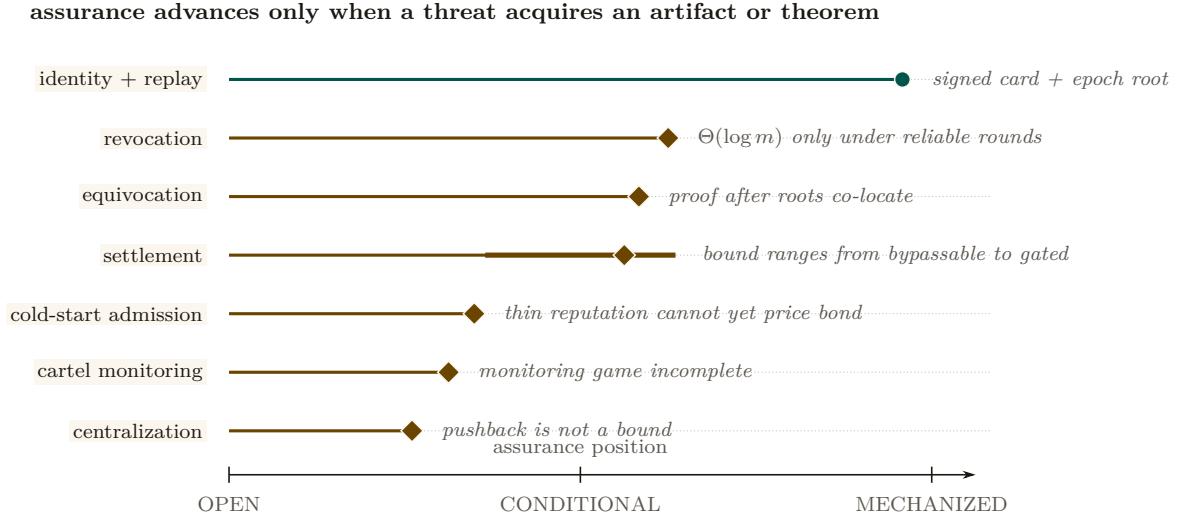


Figure 7.2: The assurance landscape is uneven and falsifiable. Position encodes how far each threat has moved from an acknowledged gap toward a mechanized artifact. Identity and replay have the strongest concrete evidence. Revocation, equivocation, and settlement are conditional on named connectivity or custody hypotheses; the thick settlement interval shows how widely its assurance moves if the gate is bypassable. Cold start, cartel monitoring, and centralization remain open. A row advances only when the missing artifact or theorem exists.

7.4 Cross-Harbor Capability Transfer

The first governing protocol is the cross-harbor capability transfer ceremony. An A -side agent holds a Harbor Card C_A minted by D_A under Anchor Phase 2 or Phase 3. It needs to present an equivalent capability at B . The Anchor Protocol’s intra-harbor delegation already supports attenuation; we extend that to the case where the attenuation crosses the trust boundary.

authority shrinks while destination-local verification takes over

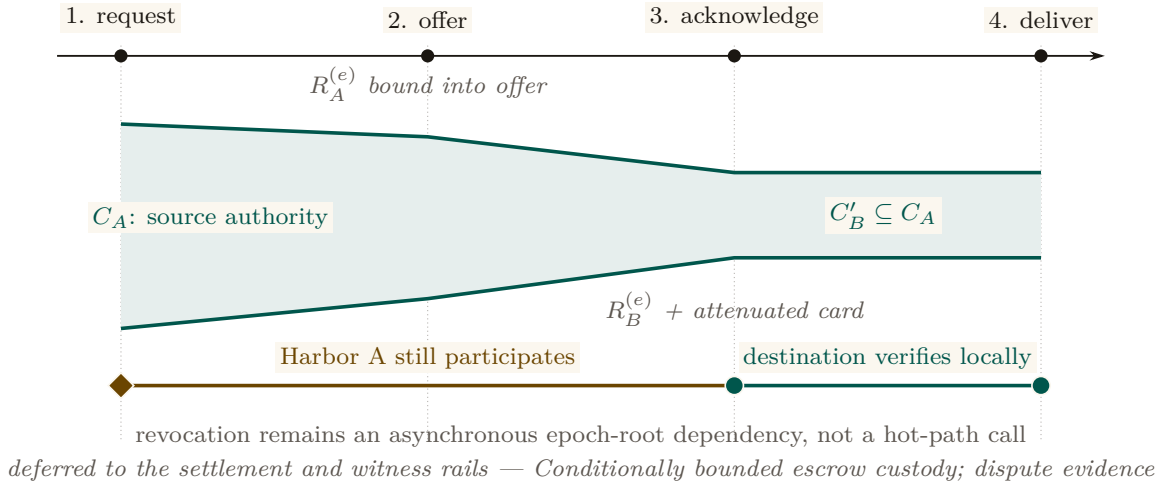


Figure 7.3: The transfer ceremony attenuates authority and then removes source reachback from the hot path. The green envelope narrows from C_A to $C'_B \subseteq C_A$ as the signed offer and acknowledgement bind source and destination epoch roots. After message 3, Harbor B can verify and deliver the attenuated card locally. Harbor A still matters asynchronously: a later epoch root can revoke the transfer once that root reaches B’s audit surface. Transfer therefore neither copies full authority nor makes every use synchronous.

7.4.1 The four-message ceremony

Figure 7.3 shows the protocol as a sequence diagram. The four messages are:

1. $\text{xfer_req}(C_A, B)$ — Alice’s agent presents C_A to its local daemon D_A and names B as the target harbor. The request is intra-harbor; standard Anchor verification applies.
2. $\text{xfer_offer}(C_A, \text{att}_A, R_A^{(e)})$ — D_A constructs an envelope binding the card, an attenuation predicate att_A specifying what subset of C_A ’s authority is being offered for cross-realm use, and its current epoch root $R_A^{(e)}$. The envelope is signed under sk_A and sent to D_B .
3. $\text{xfer_ack}(C'_B, R_B^{(e)})$ — D_B verifies the envelope’s signature against A ’s public key as recorded in the witness log; verifies that $R_A^{(e)}$ is the current root for A ; applies its own local attenuation att_B ; and mints $C'_B = \text{att}_B(\text{att}_A(C_A))$, a card valid only at B , signed under sk_B . The acknowledgment is sent back to D_A along with B ’s current epoch root.
4. $\text{xfer_deliver}(C'_B)$ — D_B delivers C'_B to the agent at B that will use it. Subsequent presentations of C'_B at B verify under sk_B alone — no A -side lookup is needed at the hot path.

The critical property is that after message (3), no further synchronous interaction with A is required for the agent to operate at B . The cost is that revocation of C_A at A does not instantaneously revoke C'_B at B ; Property 7.5.1 gives an expectation only under its delivery model, and partitions require fail-closed policy or bounded credential TTLs.

7.4.2 Why this composes cleanly with Anchor

The Anchor Protocol’s Phase-3 delegation already proves that a chain $C_{\text{daemon}} \rightarrow C_A \rightarrow C_B$ of attenuated cards is verifiable under the issuing daemon’s public key, and that an attacker cannot inflate a downstream cap to exceed an upstream one. The transfer ceremony is Phase-3 delegation across a trust boundary, with two changes:

1. The second link in the chain is signed under a *different* root key (sk_B instead of sk_A), so the verifier at B verifies under sk_B , not sk_A .
2. The cross-realm link binds the issuing harbor’s current epoch root $R_A^{(e)}$, so a revocation at A after epoch e invalidates the downstream card as soon as the new $R_A^{(e+1)}$ is observed at the verifier.

The composition is what makes the ceremony cheap to add: we are not introducing a new cryptographic primitive, only a new policy at the boundary that says “the verifier accepts a card whose chain crosses the trust boundary if and only if (a) each cross-realm link is signed under the receiving harbor’s key and (b) the issuing harbor’s bound epoch root is the current one in the witness log.”

7.4.3 Attenuation across the boundary

Two attenuation predicates are applied in sequence. att_A is set by D_A at issue time and reflects what authority A is willing to project across the boundary. att_B is set by D_B at acceptance time and reflects what authority B is willing to grant on its own resources to a card whose root is A . Both attenuations strictly narrow the underlying capability.

Lemma 7.4.1 (Attenuation Monotonicity Across the Boundary). *For any cross-realm card $C'_B = \text{att}_B(\text{att}_A(C_A))$, the set of operations authorized by C'_B is a subset of those authorized by C_A under the federated authorization rule.*

Sketch. Each attenuation is a subset operation in the Anchor algebra; subset composition is a subset. Cross-realm policy at B adds the constraint “and the operation must be permitted by B ’s local policy,” which can only further restrict. See the Anchor accompanying chapter’s capability-attenuation section for the intra-harbor argument; the cross-realm case adds no new mathematics, only a new applicable attenuation step. $\square$

7.4.4 Threats foreclosed and threats deferred

The annotation band at the bottom of Figure 7.3 summarizes which threats this ceremony forecloses and which are deferred to other primitives. Specifically:

- **Foreclosed:** identity spoofing of sk_A across the boundary; capability inflation across the transfer; replay at B against a stale R_A .
- **Deferred:** revocation latency (handled in §7.5); settlement disputes (handled in §7.6); admission of unbonded new harbors (handled in §7.7).

We are explicit that the ceremony does not close every threat in one step. Federation is a layered defense; this is one layer.

7.4.5 Mechanization status

We have a draft ProVerif model of the four-message ceremony (Appendix 7.B), extending the Anchor Phase-3 model with a second signing key and an epoch-root binding. The current status of the cross-realm authenticity query is PARTIAL: the model checks, but the proof depends on an assumption about witness-log freshness that we are still tightening. We note this limitation in Appendix 7.A and treat the attenuation monotonicity claim (Lemma 7.4.1) as proved under the same caveat.

7.5 Federated Revocation

The second governing protocol is the federated revocation mesh. The Anchor Protocol handles revocation inside a single mesh by gossiping an **Append-Only Event Log of Revocation IDs** using Vector Clocks for anti-entropy reconciliation [9]: pairs of daemons periodically reconcile their event logs, designating the cuckoo filter F purely as a local, ephemeral read-path index. The Federated Harbor extends this to the multi-administrative-domain case.

Two pieces are added on top of the Anchor mechanism: (a) per-epoch roots $R_X^{(e)}$ published to a shared witness log so a third party can audit any harbor’s local view, and (b) cross-witness signatures so the auditor pattern generalizes Certificate Transparency [33].

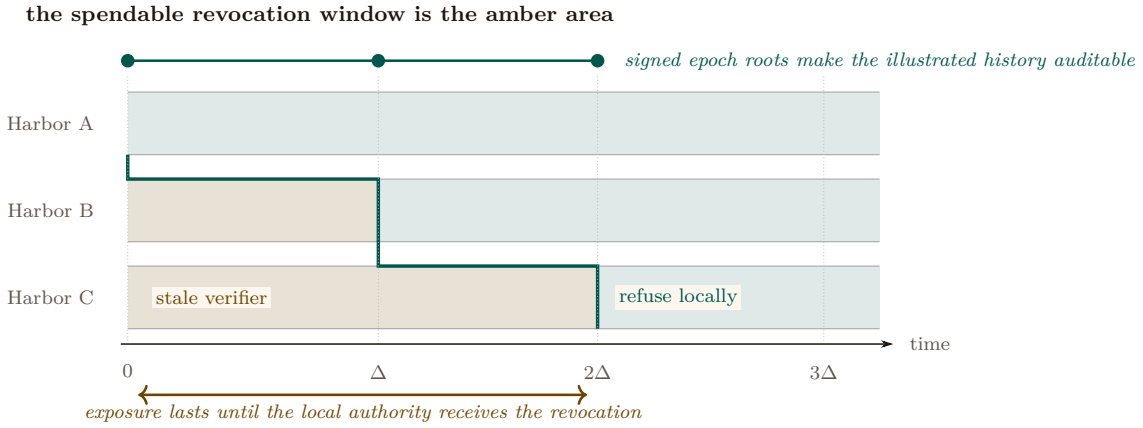


Figure 7.4: Revocation propagation is a moving staleness frontier. Alice refuses immediately; B remains stale until Δ and C until 2Δ in this illustrated execution. The amber area is precisely where a revoked capability can still be admitted by the named local authority. The teal staircase marks synchronization, not a global clock. Expected $\Theta(\log m)$ dissemination requires connected reliable rounds; a partition can extend the amber area without bound and therefore destroys any finite deadline.

7.5.1 Convergence bound

Standard epidemic dissemination gives an expected $O(\Delta \log m)$ completion time only under an appropriate connected, reliable-round model [9]. The property below instantiates that model at the federation level, treating each harbor's filter as one node in a complete overlay regardless of how many local daemons participate in the harbor's mesh. It is a model-conditional expectation, not an inherited wall-clock deadline.

Property 7.5.1 (Conditional Revocation Dissemination). Let m harbors form a complete overlay. In each reliable synchronous round of duration Δ , informed harbors contact independently uniform peers and exchange revocation deltas. If harbor X publishes revocation c at t_0 , then:

1. **Revocation dissemination.** All harbors are informed in expected $\Theta(\log m)$ rounds under the stated epidemic model.
2. **Equivocation verification.** Any auditor possessing two differently signed roots for the same (X, e) detects equivocation in $O(1)$ signature and equality checks.
3. **No hard deadline.** If delivery can be lost indefinitely or the overlay can remain partitioned, no finite worst-case dissemination or common-auditor detection bound exists.

The first item is the standard asymptotic epidemic-dissemination result [9]; this paper does not claim the earlier exact constant $1 + \ln m$. The second follows from the signed-root format. The third is an indistinguishability result: an auditor isolated in one partition cannot distinguish equivocation from delay.

Corollary 7.5.1 (Service-Level Attacker Window). *If an operator additionally imposes a partition bound T_p and a delivery service level T_g , then stale-card exposure is bounded by $T_p + T_g$. Without those operational assumptions the protocol supplies an expected latency under its model, not a structural maximum.*

The distinction determines the exposure bound. A bond can be priced against an operator-declared service-level window, but an unbounded partition creates an unbounded exposure tail. Deployments must therefore combine bond sizing with expiry, fail-closed acceptance, or an explicit partition limit.

7.5.2 Conflict resolution across harbors

The interesting case is disagreement. Harbor A has revoked card c ; harbor B has not yet seen the revocation; an agent presents c at B for an action that affects A 's database. Three candidate

resolution rules from the proposal [83], evaluated:

1. **Pessimistic (anyone-revokes-is-revoked)**. The card is revoked iff any harbor has revoked it. Defended against by an attacker who would gain by injecting spurious revocations into the gossip mesh; rejects too many honest cards in equilibrium.
2. **Authoritative (only-issuer-revokes)**. Only A can revoke A 's cards. This is the rule we adopt. Under the service-level assumptions of Corollary 7.5.1, it gives an operator-priced attacker window; without them, exposure has no finite structural maximum.
3. **Epoch-bounded (revocation propagates within Δ epochs and inconsistency is a contract violation)**. A refinement of (2) for the case where one harbor falls persistently behind. We treat this as a degraded mode of (2) rather than a separate rule.

We adopt rule (2). The pessimistic rule has the wrong defaults; the epoch-bounded rule is operationally the same as (2) with a watchdog. Adopting (2) means the Conservation invariant of §7.6 can be stated cleanly.

7.5.3 Event Log & Ephemeral Cuckoo Discipline

By resolving the bitwise merging impossibility of probabilistic structures, the cuckoo filter is now purely a local, ephemeral read-path index derived from the authoritative Append-Only Event Log. The filter retains local Anchor-side disciplines (load capping, pollution resistance) but cross-harbor validation is strict: a filter at B that holds $c \in F_B$ where c was revoked at A is correct only if there is a corresponding inclusion proof in $R_A^{(e+1)}$ in the witness log. False positives are corrected by rebuilding the local filter from the event log; spurious revocations injected by an attacker into a peer's log are rejected on first attempt to verify against the witness log. The discipline is: *local filters are advisory read-caches; the event and witness logs are authoritative*.

7.5.4 Sheaf Laplacian and Abramsky–Brandenburger Contextuality

The global consistency of federated witness logs can be modeled sheaf-theoretically, and it is worth being precise about which object carries the weight. The intuitive picture is a poset $(X, \leq)$ of administrative domains ordered by scoped visibility, assigning to each domain U the semilattice of prefix-compatible append-only logs, with restriction given by prefix projection. That picture is right about the ordering and useless for computation: a semilattice has no subtraction, no adjoint, and no spectrum, so none of the machinery below would be defined over it. The working object is therefore the *cellular sheaf* $\mathcal{F}$ of the companion paper on the cohomology of equivocation [82], carried on the gossip graph $G = (V, E)$ of witnesses: witness v holds a real stalk $\mathbb{R}^{d_v}$ encoding its log state; edge $e = \{u, v\}$ holds $\mathbb{R}^{|S_e|}$, the shared prefix coordinates S_e its two endpoints both report; and the restriction maps $\rho_{v,e}$ are the coordinate selections that cut a witness's state down to what that edge can see. Over real stalks the coboundary δ , its adjoint δ^* , and the observed disagreement cochain $(\delta s)_e = \rho_{u,e}s_u - \rho_{v,e}s_v$ all exist. The semilattice stays as intuition; the vector-space sheaf is what the theorem below quantifies over.

Detection is stated per visibility tier, because not every edge is equally visible to an auditor. An edge is **compared** when the auditor holds its direct cross-check (pairwise detection, no cohomology needed); **relayed** when both endpoints' reports about it reach the auditor by gossip but the endpoints never reconciled it; and **severed** when no report about it crosses to the auditor at all. Write K for the compared-plus-relayed (i.e. *visible*) edge set, and for each shared coordinate c write $K_c := (V, \{e \in K : c \in S_e\})$ for the visible coordinate subgraph. The distinction between K_c and the full coordinate subgraph G_c changes the theorem, not merely the notation: a cycle that runs through a severed edge imposes no constraint, because the severed block is a free variable of the completion.

Theorem 7.5.1 (Detection iff the missing edge closes a *visible* cycle). *Assume prefix restrictions, the three-tier visibility model above, and a **single equivocator** q (the single-equivocator hypothesis is not decoration: two liars on a common cycle can cancel each other's contribution to the residual).*

The detection statistic is the least-squares completion residual

$$r = \min_{x, g_{\text{sev}}} \|g_K - (\delta x)|_K\|_2 = \left(\sum_c \|\text{Proj}_{\text{cycle}(K_c)} g^c\|^2 \right)^{1/2}, \quad (7.1)$$

the minimum over global sections x and over the free severed blocks g_{sev} . Then r detects q 's split-view lie beyond what pairwise comparison already catches if and only if the un-compared lie edge lies on a cycle of the **visible** subgraph K_c and its endpoints' reports are relayed to the auditor; in that case $r > 0$ certifies that no global witness history explains the visible data. On a cut edge, $r = 0$ identically — the coboundary map of a tree or forest is surjective onto the visible data, so the free completion absorbs any lie. Across a severed edge, equivocation is provably dark: the severed block is unconstrained and a consistent counterfactual world always exists.

The structural identity behind the mechanism — local consistency with no global section — is the Abramsky–Brandenburger contextuality equivalence [193]: an equivocation (a harbor signing two mutually incompatible prefixes) manufactures for its neighbors exactly a locally-consistent, globally-unglueable family of views, the relational-semantics analogue of Bell contextuality. Note what the theorem does *not* say. The textbook fact that $H^1(\mathcal{U}; \mathcal{F}) = 0$ is equivalent to every local family gluing is a statement about the cover and the restriction maps alone; it is blind to any particular lie, and it is not the detector. The detector is the residual r of the observed data.

Scope (what the theorem does and does not license). Three bindings keep this claim inside its proven regime. **(i) The obstruction lives in the data, not the sheaf:** the detector is the observed disagreement cochain's failure to be a coboundary (its harmonic residual), not $\dim H^1$ of the abstract sheaf, which is a property of the restriction maps alone and is blind to any particular lie. **(ii) Visible cycles are required:** cohomology adds power over $O(|E|)$ pairwise digest comparison exactly when an un-compared (partitioned) edge lies on a cycle of the *visible subgraph* K_c , so the sum-around-the-loop constraint substitutes for the missing comparison; across a *cut* edge there is no loop to close, and a loop through a *severed* edge is no loop at all for this purpose, so the method provably sees nothing — redundant gossip paths are a detection requirement, not a luxury. **(iii) Non-vanishing is an alarm; vanishing is not an all-clear:** over real coefficients the obstruction is sufficient but not necessary (the abelianization gap of Carù), so a zero residual licenses no safety conclusion. Detection and localization here *complement* forensic attribution — signatures attribute, cohomology triages under partial visibility — and neither replaces the other. *Status (statistical harness).* The harness behind this scope rider is rebuilt and its pre-registered gates passed (verdict: commit; `sheaf_harness_v2.py`, 200 trials per arm, seed 20260816). The working detector is the least-squares *completion residual* r — the minimum, over global sections and severed blocks, of the disagreement left unexplained on the visible edges — under a three-tier visibility model (**compared** / **relayed** / **severed**): detection requires a cycle of the *visible subgraph* K_c and relayed reports from the un-checked edge's endpoints. On a relayed cycle edge under partition, 200/200 detections are cohomology-only (pairwise comparison blind); on a cut edge the residual is zero to float epsilon (max 1.5×10^{-13} over 400 trials), as the algebra demands; across a truly severed edge (no reports reach the analyst) equivocation is provably dark — binding (ii)'s topology requirement, now measured. Mutants reintroducing the harness's two prior defects (D1, non-subset restriction maps that collapse the obstruction space; D2, detection scored on hidden-edge data) are each caught by structural self-checks. Binding (iii) stands unchanged: a vanishing residual over real coefficients remains no all-clear (Carù).

Numbers by hand. Take six harbors in a ring, C_6 , with scalar log states, and plant a lie of magnitude $|s| = 3.0$ on the one link e that is relayed but never directly compared. Every pairwise check between neighbors still passes. The residual left over after the best-fit global explanation is

$$r = |s| \sqrt{1 - R_{\text{eff}}^{K_c}(e)} = 3\sqrt{1 - \frac{5}{6}} = \frac{3}{\sqrt{6}} = 1.2247$$

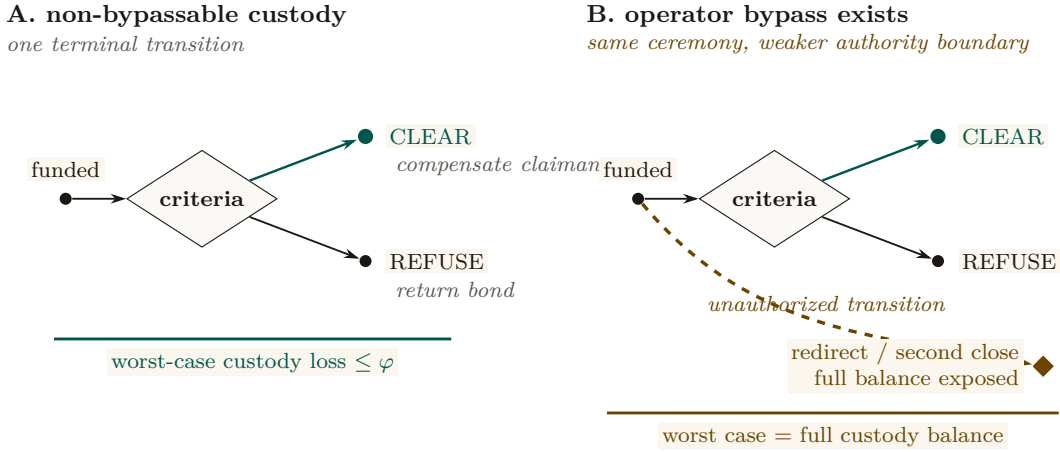
— not zero, so no consistent story exists. ($R_{\text{eff}}^{K_c}(e) = \frac{n-1}{n} = \frac{5}{6}$ is the effective resistance across the edge when the ring is read as a circuit of unit resistors: the other five edges form a series path

in parallel with it.) Now sever one *other* edge of the same ring — say the link opposite the lie — so that the auditor receives no report about it. The lie edge still lies on a cycle of the full gossip graph G , and its own endpoints are still relayed, so the un-corrected G_c reading of the formula would still predict 1.2247. The measured residual is 1.1×10^{-14} : float zero. Sever a different ring edge instead and it is 1.9×10^{-15} . This is the K_c -versus- G_c distinction in one hand-checkable pair — the visible subgraph is a path once any ring edge goes dark, a path has no cycle space, and the free severed block absorbs the entire lie. Moving the identical lie onto a bridge (a link with no ring around it at all) gives 0 exactly, for the same reason. *Now you try*: an eight-harbor ring, all edges visible, lie of magnitude 2 on the relayed edge — the certified residual is $2/\sqrt{8} = 0.7071$ [internal, `sheaf_mechanism_proof.py`, `sheaf_consistency_radius.py`]. The design lesson the arithmetic carries: conviction dilutes like $1/\sqrt{n}$ around long loops, so a federation that wants cheap certified detection should keep its reconciliation loops short and its rings intact.

Sheaf Laplacian dynamics — and what is *not* claimed here. On a cellular sheaf with real stalks, the **Hansen–Ghrist sheaf Laplacian** [110] is $\Delta_{\mathcal{F}} = \delta^* \delta$, and its harmonic space $\ker(\Delta_{\mathcal{F}}) \cong H^0(G; \mathcal{F})$ is precisely the subspace of globally synchronized ledger states. This object exists only over the vector-space sheaf constructed above; it is undefined over the semilattice of append-only logs, which has no adjoint and no spectrum. The spectral gap $\lambda_2(\Delta_{\mathcal{F}})$ — the smallest non-zero eigenvalue — is the natural quantity to *relate* to anti-entropy convergence speed, a small gap meaning slow mixing by analogy with the graph-Laplacian case. **We have not derived or checked such a bound, and this paper claims none.** The Hansen–Ghrist rate governs continuous-time linear diffusion $\dot{x} = -\Delta_{\mathcal{F}} x$ over real stalks; anti-entropy gossip over signed append-only logs with a Byzantine equivocator is discrete, asynchronous, adversarial, and monotone-join rather than linear-averaging, and nothing here shows the bound transfers. The two convergence stories — classical epidemic dissemination (Property 7.5.1, which is a real and separately argued result [9]) and sheaf-Laplacian diffusion — are not reconciled, and neither is a second proof of the other. No relaxation-time bound for the federation is established, in this section or anywhere in this paper; that is consistent with Appendix 7.A, whose dissemination rows record “no partition deadline” and “no hard delivery bound.” The reconciliation is open work (Appendix 7.A, *open* band).

7.6 Cross-Harbor Settlement

The third governing protocol is cross-harbor settlement. A bond is posted at A to cover damage caused by an A -side agent operating at B . When damage occurs, the bond must clear to the damaged party at B . The protocol does this without making either harbor sovereign over the other, but it still uses a third-party custody service. Its transition surface must be non-bypassably restricted; the property below states that assumption rather than relabeling the service as trustless.



Custody Assumption: whitelist + fee cap + one terminal transition are the theorem's hypothesis, not decoration

Figure 7.5: The custody bound is the absence of an extra transition. Panel A permits exactly one terminal move from funded custody to **Clear** or **Refuse**; with a recipient whitelist and fee cap, worst-case loss is bounded by the agreed fee φ . Panel B adds one operator-controlled bypass. The visible extra edge reaches redirect or a second close, exposing the full balance. Threshold signing is only one candidate implementation; the loss theorem holds only when the three authority restrictions are actually non-bypassable.

7.6.1 The bounded escrow

We introduce a third party—the escrow—and require the *custody ledger* to expose exactly two terminal transitions: clear (pay whitelisted beneficiary B , refund the remainder to A) or refuse (return the bond to A). A signed policy alone is only evidence. The following restrictions are structural only if the escrow service cannot bypass the ledger API or alter its code and keys:

- Redirect funds to anyone other than A or B .
- Equivocate about its clearing decision (publish “cleared” to one party and “refused” to another).
- Delay past the TTL of the bond.
- Extract more than a pre-agreed fee ϕ which is capped at issuance.

These restrictions define the desired transition system. The current artifact is a design and bounded model, not a deployed custody mechanism.

Property 7.6.1 (Conditional Escrow Extraction Bound). Assume the custody ledger (i) admits only recipients A and B , (ii) caps the fee output at ϕ , (iii) executes exactly one terminal transition atomically, and (iv) keeps signing and transfer authority outside the escrow operator’s bypassable control. Then no allowed transition pays the escrow more than ϕ .

The bound follows by case analysis over the two allowed terminal transitions. Witness logging makes delay or equivocation detectable, and an escrow bond can compensate some misconduct, but neither fact prevents theft if the operator can bypass custody restrictions. Without assumptions (i)–(iv), the worst-case extractable value is the full custodial balance.

The escrow is a trusted third party with a *conditional transition bound*. We are honest that this is not a trustless construction in the HTLC sense [151]. Whether useful trustless settlement is possible for non-fungible reputation-priced bonds remains open (§7.12); this paper gives neither a construction nor an impossibility proof.

A candidate instantiation of assumption (iv). One concrete way to remove the escrow operator’s *unilateral* control over transfer authority is a 2-of-3 threshold-signature scheme: harbor A , harbor B , and a federation arbiter each hold one key, and no transfer executes without two of the three signatures. When A and B agree on the outcome — the ordinary case — they co-sign

the release themselves and the arbiter never touches custody. The arbiter’s key exists only to break a dispute: it evaluates the witness-logged evidence trail and co-signs with whichever party it rules for. This is a plausible way to satisfy assumption (iv), and we name it because a reader will otherwise ask what such a mechanism would look like. It does not close what Property 7.6.1 calls conditional. The arbiter remains a trusted third opinion; we have not analyzed an arbiter colluding with one principal, we have not specified how arbiter selection resists capture, and the underlying settlement rail (an on-chain threshold contract, a custodial payment hold, or anything else) carries its own assumptions that this paper does not model. We list 2-of-3 multisig as a candidate design, not a closed result; trustless settlement for non-fungible bonds stays in the *open* band (§7.12.2, Appendix 7.A).

7.6.2 Cross-harbor conservation

The Bonded Commons paper proved a single-machine Conservation Theorem: the total bonded value in the system never changes by surprise; bonds are only created (by posting), destroyed (by clearing), or rolled (refunded and re-posted). We extend this to the federation.

Property 7.6.2 (Cross-Harbor Bucket Partition). For each funded bond b of amount a_b , let $P_b, E_b, C_b, R_b \in \{0, a_b\}$ denote its posted, in-escrow, cleared, and refunded buckets. Valid transitions preserve

$$P_b + E_b + C_b + R_b = a_b \quad \text{and} \quad |\{Z \in \{P_b, E_b, C_b, R_b\} : Z = a_b\}| = 1.$$

Summing over bonds gives $\sum_b (P_b + E_b + C_b + R_b) = \sum_b a_b$. External top-ups create new funded bonds and are recorded separately; fees must appear as an explicit cleared recipient rather than disappear from the identity.

This is the federation-layer analogue of Conservation in Bonded. The intra-harbor Conservation invariants hold per-harbor (Bonded gives the proofs); the cross-harbor property adds the constraint that bonds in transit between harbors — i.e., posted at A , locked in escrow, awaiting clearance against B — are accounted for as in-escrow rather than as either posted at A or cleared to B . The escrow’s role as the holder of in-flight bonds is what makes the conservation accounting tractable.

7.6.3 Settlement protocol

Figure 7.5 shows the three-step protocol. To unpack the figure:

1. **Bond post.** Alice’s harbor mints a work card for agent α ; agent operates at B . Bond $B_\alpha = \$B$ is posted at A and locked in the escrow. The bond’s amount and TTL are signed into both A ’s and B ’s witness-log roots so the federation knows the bond exists.
2. **Damage claim.** Damage detected at B (acceptance criteria fail, or invariant violation). Bob’s harbor signs claim $\text{cl}_B = (R_B^{(e)}, \text{evidence})$ where the evidence is a Merkle inclusion proof against $R_B^{(e)}$. The claim is sent to the escrow.
3. **Verification and clearance.** Escrow verifies inclusion of α ’s actions in $R_B^{(e)}$, matches against the acceptance preconditions that were signed at step 1, and produces one of two outcomes: *clear* (pay B out of bond, refund remainder to A) or *refuse* (return bond to A , log reason). Both outcomes are recorded in the witness log.

The designed transition is atomic in the bucket-partition sense: a bond moves once from in-escrow to either cleared or refunded. This property depends on the custody-ledger assumptions of Property 7.6.1; witness records alone cannot make two external transfers atomic.

7.6.4 Fee structure and economic discipline

The escrow’s fee ϕ is bounded but non-zero. In the competitive-insurance market sense of *The Bonded Commons* (the Youle contribution), ϕ is a market clearing price for the service of holding the bond and verifying the inclusion proof. We do not solve the market here; the Youle contribution

from Bonded gives the mechanism, and we conjecture (open) that it extends cleanly to the cross-harbor case when the escrow population is large enough.

We are explicit that for small federations (two or three harbors), ϕ is likely set by a posted price rather than auctioned. The mechanism design here is identical in shape to Bonded's, just lifted to the federation layer.

7.7 Federation Admission

The Federated Harbor is not a permissionless market in identities. A new harbor enters the federation through an admission ceremony. The ceremony is invitation-bounded by design: Sybil attacks against the federation [124] are costly for an attacker precisely because each identity requires a real admission event.

7.7.1 The bonded-sponsor pattern

A new harbor N with no prior reputation cannot price its own bond fairly under competitive insurance — the market lacks data on N 's risk. The bonded-sponsor pattern resolves this:

1. An existing harbor S (the sponsor) posts a bond on N 's behalf, sized to cover the federation's worst-case loss from N misbehaving during a probationary epoch.
2. N joins the federation under probation; its cards are accepted, its evidence trail is witness-logged, but its sponsor bond is at risk.
3. If N misbehaves during probation — forges signatures, equivocates on roots, refuses to honor settled bonds — the sponsor bond is forfeit. The sponsor has full incentive to vet N before sponsoring.
4. At the end of probation, N 's own reputation is sufficient to price its own bond, and the sponsor bond is released.

This is the federation-layer analogue of competitive insurance from *The Bonded Commons* (the Youle contribution): instead of an authority picking who joins, the sponsor market discovers it. The cost of being a sponsor is real — forfeit risk during probation — and the benefit is the network effect of having N in the federation. We expect (open) that for federations with many candidate sponsors and clear-eyed risk pricing, the equilibrium is reasonably efficient. We do not prove it.

7.7.2 Cold-start failure modes

A new harbor that cannot find a sponsor cannot join. This is a real limitation. In the worst case, the federation collapses to invitation-only — which is the failure mode of every interesting federated identity system from PGP web-of-trust [172] to SAML [88]. The structural pushback is:

- *Many sponsors.* The market is many-to-many; a new harbor needs only one sponsor. As long as sponsors are paid (via fees or reputation), the supply is nonzero.
- *Small probationary scope.* New harbors during probation are restricted from large-stake operations; the sponsor's risk is bounded.
- *Witness-log permanence.* The admission record is permanent; once a harbor has graduated probation, it does not need a sponsor again.

We are honest that this is engineering discipline, not theorem. The historical evidence on whether invitation-bounded federations stay open is mixed [88]; the Federated Harbor's pushback against centralization is to make the cost of being a centralizing hub bounded above by the cost of running mirrored witness logs, which is small. Whether that pushback is enough is empirical.

7.8 Worked Example

The four primitives are easier to read against a concrete case. We work the example end-to-end: two organizations, three machines, one shared project. The example is intentionally familiar;

nothing in it is novel except the way the primitives compose.

7.8.1 Setup

Two organizations: Acme Robotics (Alice’s employer) and Beta Vision (Bob’s employer). The shared project is a perception-and-control stack: Acme owns the controller, Beta owns the vision system, both run against a shared staging environment that lives on a third machine maintained by neither.

Three machines: Alice’s laptop (harbor A , controller code, controller test database), Bob’s laptop (harbor B , vision code, vision test database), staging-server (harbor S , shared staging database, no agents resident).

Each harbor has gone through the admission ceremony of §7.7. Acme’s harbor was sponsored by an existing partner harbor at the open-source consortium that hosts the staging server; Beta’s was sponsored by Acme after a six-month probationary period during which Acme’s risk officer watched Beta’s evidence trail closely. The staging server harbor is sponsored by the consortium directly and is treated as a known-honest verifier for the purposes of this example.

7.8.2 The agent task

Acme’s agent — call it Controller-7 — needs to run a schema migration on the staging database to add a column for a new sensor type. The migration is destructive (drops and recreates the column); if it goes wrong, the staging database is unusable for the joint demo scheduled three hours later.

Controller-7’s local card C_A authorizes `db:migrate` against the controller test database, attenuated through Acme’s risk policy ($\text{att}_A = \text{“only against tables in the controller schema; only between 9am and 5pm; only if a human operator has approved within the last 30 minutes”}$). The card is valid at A , but the database it needs to migrate is at S .

7.8.3 Step 1: Cross-harbor transfer

Controller-7 invokes `xfer_req( $C_A, S$ )` on D_A . D_A constructs the envelope:

$$\text{xfer_offer}\langle C_A, \text{att}_A, R_A^{(e)} \rangle \text{ signed under } sk_A,$$

and sends it to D_S (the staging-server harbor). D_S verifies the signature against A ’s public key as recorded in the witness log, verifies that $R_A^{(e)}$ is the current epoch root for A , and applies its own attenuation att_S (“only the `controller` schema; only after a backup snapshot has been taken; only if no other migration is in flight”). The composed attenuation produces $C'_S = \text{att}_S(\text{att}_A(C_A))$, a card valid only at S , signed under sk_S .

D_S delivers C'_S to Controller-7. Controller-7 takes the snapshot, then runs the migration against S . The migration’s actions are recorded in S ’s Merkle forest and roll up into $R_S^{(e)}$.

7.8.4 Step 2: Damage detection

The migration runs cleanly. Two hours later — well after the agent has moved on — Beta’s vision system runs an integration test against the staging database and observes that the new column has the wrong type (Acme assumed `float32`; Beta needs `float64` for the sensor data they will produce). The integration test fails.

D_B (Beta’s harbor) verifies the failure against its local invariants and produces a damage claim:

$$\text{cl}_B = (R_B^{(e+1)}, \text{evidence}),$$

where the evidence is a Merkle proof that Beta’s integration tests against the staging database failed because of a schema mismatch. The claim is sent to the escrow.

7.8.5 Step 3: Settlement

Acme posted a bond when Controller-7 was authorized to run the migration. The bond is held in the federation escrow with a TTL of 24 hours — long enough for downstream consequences to surface, short enough not to lock collateral indefinitely.

The escrow verifies cl_B : it checks the inclusion proof against $R_B^{(e+1)}$ in the witness log, matches the claim against the acceptance preconditions signed at bond-post time (which named the staging schema as a covered surface), and decides: *clear*. Acme’s bond is debited; Beta receives the cleanup-cost portion; the remainder is refunded to Acme. The settlement record is written to the witness log.

7.8.6 Step 4: Revocation propagation

At this point, Acme’s risk officer revokes Controller-7’s `db:migrate` card and publishes $R_A^{(e+2)}$. Under Property 7.5.1’s connected reliable-round model, dissemination completes in expected $\Theta(\Delta \log m)$ time. During a partition, receiving harbors must rely on expiry or a fail-closed freshness policy; the example does not pretend gossip supplies a deadline.

7.8.7 What this example shows

The example is small and the failure is real. The point of working it is to show that each of the four primitives — transfer, revocation, settlement, admission — is doing its job at exactly one step, and that the composition is straightforward when each primitive is honest about what it covers.

Notice in particular what *did not* happen:

- Neither D_A nor D_B trusted the other’s signing key as a root. The transfer worked through the witness-logged public keys, not through a chain of trust.
- The bond was not held by either harbor; the design requires a recipient-whitelisted custody ledger before the escrow’s role is structurally bounded.
- The settlement was not negotiated peer-to-peer; it followed a fixed protocol whose two outcomes were named in advance.
- Under the example’s connected reliable-round assumptions, the revocation propagated through gossip with expected logarithmic completion; no consensus protocol was invoked.

Each of these is a deliberate design choice. The Federated Harbor’s architecture is not heroically more powerful than alternatives; it is heroically more honest about what is happening at each step.

7.9 Formal Model

The formal substrate of this paper is two-layer: ProVerif [40] models for the cryptographic protocols (cross-realm authenticity, attenuation, settlement no-redirect) and TLA+ extensions of Bonded’s Conservation specification for the cross-harbor accounting invariants.

7.9.1 ProVerif extensions to Anchor

The Anchor Protocol’s three-phase models are extended with a second signing key and an epoch-root binding. The cross-realm authenticity query becomes:

```

1 query b: id, ha: id, hb: id, cap: capability, e: epoch;
2   event(AcceptedAtB(b, hb, cap, e))
3   ==> (event(IssuedAtA(b, ha, cap)) &&
4         event(EnvelopeAtoB(ha, hb, cap, e)) &&
5         event(RootCurrent(ha, e))).

```

Listing 7.1: Cross-realm authenticity query (sketch)

This says: if B accepts a cross-realm card for capability cap at epoch e , then there exists a corresponding issuance event at A , an envelope event from A to B binding the same capability and epoch, and an event confirming that $R_A^{(e)}$ is the current root. The query holds against the Dolev–Yao attacker model under the standard assumption of Ed25519 EUF-CMA security.

We are honest that the model is drafted and the proof script is in progress. The full mechanization is named open work; see Appendix 7.A for the status table.

7.9.2 TLA+ extension for cross-harbor conservation

The Bonded Commons paper’s Conservation specification models a single-machine bond pool. The Federated Harbor extension adds a per-harbor pool, an escrow pool, and an inter-harbor transit relation:

```

1  CONSTANTS Harbors, MaxBonds, Capabilities
2  VARIABLES Bucket, WitnessLog
3
4  TypeOK ==
5    /\ Bucket \in [1..MaxBonds -> {"posted", "escrow", "cleared", "
      refunded"}]
6    /\ WitnessLog \in Seq([harbor: Harbors, epoch: Nat, root: STRING])
7
8  BucketPartition ==
9    \A b \in 1..MaxBonds:
10     \E! s \in {"posted", "escrow", "cleared", "refunded"}:
11       Bucket[b] = s
12
13 Spec == Init /\ [] [Next]_<<Bucket, WitnessLog>>

```

Listing 7.2: Cross-Harbor Conservation specification (sketch)

The full specification (Appendix 7.C) associates each bond with an amount and enforces amount-preserving moves through these buckets. Apache checks the resulting transition system at $|F| = 4$ and bond depth 16. This establishes the invariant only for the model and bound; it does not establish the custody-ledger assumptions or arbitrary-scale runtime conformance.

7.9.3 What is mechanized and what is structural

We are explicit about which claims live in which band (Figure 7.2).

Mechanized (model scope). Single-harbor capability authenticity is inherited from Anchor. Cross-realm authenticity and attenuation are draft models. The cross-harbor bucket-partition invariant is model-checked at $|F| \leq 4$.

Conditional (assumption scope). Revocation dissemination is expected $\Theta(\Delta \log m)$ only under the stated reliable connected-overlay model (Property 7.5.1). The escrow extraction bound is conditional on a non-bypassable recipient-whitelisted custody ledger (Property 7.6.1).

Open (cobalt). Multi-principal correlated-equilibrium extension of Bonded; trustless settlement for non-fungible reputation-priced bonds; folk-theorem cartel-resistance bound at the federation layer.

7.10 Failure Modes

A federation paper that does not engage formally with the historical failure modes of federated systems is unserious. The literature is heavy with examples: SAML federations that concentrated

on a small set of identity providers [88]; PGP web-of-trust that never achieved meaningful adoption outside enthusiast circles [12]; Sigstore’s gravitation toward a single deployment of Fulcio [222]. We engage with three failure modes specifically.

7.10.1 Centralization gravity

Federated identity systems have a well-documented tendency to centralize on a small number of trust anchors within five to ten years of deployment [88]. The Federated Harbor’s structural pushback is the replicable witness log: any harbor can run its own witness instance, any harbor can audit any other instance’s roots, and equivocation between witnesses is detectable. This makes it more expensive to be a centralizing hub than to be a peer.

The pushback is not theorem; it is design discipline. We are honest about this in Figure 7.2 (cobalt band). The historical evidence is mixed. Tailscale [27] has remained operationally centralized on its own coordination server despite Headscale [129] being a viable alternative; in-toto [195] has stayed decentralized but is consumed mostly by SLSA [137] which has Google as its de facto center. Time will tell which pattern the Federated Harbor exhibits.

7.10.2 Equivocation by a malicious witness

A malicious witness log can serve different views to partitioned auditors. A contradictory signed pair is immediately verifiable once co-located, but the protocol has no topology-independent time bound for bringing the pair together. Witness bonding applies after confirmation.

For high-stakes settlements, harbors can require multiple witness signatures before treating a root as authoritative, in the Certificate Transparency two-SCT pattern [33]. This trades latency for assurance; the design space is well-explored in the CT literature and we adopt the same techniques.

7.10.3 Partition tolerance

The federation does not run consensus; partition tolerance is per-protocol. Capability transfer cannot proceed during a partition between A and B (no surprise; cross-realm authentication requires both daemons to reach the witness log). Revocation gossip continues within each partition; convergence is bounded by the partition’s local size, not the global federation size. Settlement is parked at the escrow until the partition heals.

The structural posture is that partitions are operationally common and the federation should degrade gracefully rather than fail fast. We inherit this discipline from Anchor’s intra-mesh design [79]; the federation extension is straightforward but adds no novel mathematics.

7.11 Related Work

The Federated Harbor sits at the intersection of four bodies of work. We are concrete about where we draw on each and where we depart.

Federated identity. The OpenID Federation 1.0 specification [189] is the closest existing analogue: each entity publishes a signed Entity Statement, trust chains assemble by walking up to a Trust Anchor, trust marks express property claims. The Federated Harbor is structurally similar in the trust-anchor pattern but differs in two places: (a) we add a capability/attenuation layer (OpenID Federation is identity-only), and (b) we do not assume HTTP-based fetching of entity statements, since Port Daddy daemons cannot reliably expose HTTP endpoints (NAT, sleep, etc.). We draw also on SAML/Shibboleth historical experience [88] as cautionary lineage; the centralization gravity we discuss in §7.10 is exactly the SAML-era failure mode.

Capability-based federation. Macaroons [21] are the foundational capability-with-attenuation construction; our transfer ceremony extends Macaroon-style third-party-caveat chains across an administrative boundary. UCAN [211] is the current production-deployed answer to “Macaroons with public-key principals,” and we treat it as the operationally most relevant existing system; the Federated Harbor differs in keeping the daemon as a structural commons authority rather than going fully peer-to-peer, and in pricing delegation through insurance-priced bonds rather than pure capability constraints. The object-capability lineage [149, 109] continues to be relevant: cross-realm operations are exactly a mutual-suspicion scenario in Miller’s vocabulary, and his defensive-consistency framework is the right lens.

Transparency logs. Certificate Transparency [33] is the model; the federated witness log of §7.2.3 is CT applied to capability tokens. Sigstore [222] and Rekor are the production instantiation we rhyme with most heavily; in-toto [195] and SLSA [137] are the supply-chain analogues that compose with the Federated Harbor for the “poisoned plugin under a bonded principal” case but do not close it. The Crosby–Wallach history-tree [196] is the formal substrate of every modern transparency log including ours.

Cross-domain settlement. Herlihy’s atomic cross-chain swap [151] is a comparison point; the cross-harbor settlement protocol of §7.6 instead handles non-fungible reputation-priced bonds through conditionally restricted custody. It is not trustless in Herlihy’s sense. Whether trustless settlement is achievable in our setting is open (§7.12). The IBC protocol [52] supplies another connection/channel comparison; unlike IBC, this design does not assume a consensus-backed state machine underneath.

Agentic-commerce discovery and mandates. The Universal Commerce Protocol [213] (Google/Shopify, 2026) solved cross-administrative discovery for agent-mediated retail with a pattern the Federated Harbor adopts (ADR-0051 Phase 1b): a `/.well-known` JSON profile that does double duty as capability declaration and JWK key publication, with server-selects version negotiation — the party that must execute a request computes the compatibility intersection. Where UCP assumes merchants hold stable HTTPS origins, our daemons often do not (the same constraint that ruled out OpenID Federation’s entity-statement fetching above); the profile therefore also travels as a relay-published mirror. UCP delegates authorization proof to the Agent Payments Protocol’s verifiable-credential mandates [5], whose SD-JWT/JWS/JCS formats the harbor boundary now speaks (ADR-0094). The division of labor is instructive: those protocols standardize discovery and the transaction envelope and are explicit that counterparty trustworthiness, collateral, and settlement enforcement are out of scope — which is the half of the problem this paper and its predecessors supply.

7.12 Limitations and Open Questions

We close with the honest open frontier. These are not throwaway caveats; they are the questions the paper does not answer, named precisely enough to motivate subsequent work.

7.12.1 The multi-principal correlated-equilibrium extension

The Bonded Commons paper argued that single-machine coordination is a correlated equilibrium with the daemon as the correlation device [81]. Across two harbors with two daemons, the correlation device fragments. Two candidate resolutions remain unresolved:

- The federated witness log *is* the correlation device, and individual harbors are local enactors. This would extend Bonded’s result intact.

- The cross-harbor case is a partition of the correlated-equilibrium concept into intra-harbor (correlated) and inter-harbor (Nash with shared signal). This would weaken Bonded’s result at the federation layer.

We do not know which is right. The proposal that scoped this paper [83] named Thomas Youle as the essential collaborator for this question (his contribution to Bonded is the closest existing work). The question stays open here.

7.12.2 Trustless settlement for non-fungible bonds

HTLC-style atomic swaps [151] achieve trustless settlement for fungible assets with a shared hash preimage. The Federated Harbor’s bonds are non-fungible and adjudication-dependent, so that construction does not transfer directly. We adopt conditionally restricted custody (§7.6) and leave both a trustless construction and an impossibility result open. Either would require a separate threat model and proof.

7.12.3 Cartel formation across federations

Bonded’s folk-theorem cartel-resistance argument applies inside a single machine. Between machines, cartel formation is structurally easier: colluders can coordinate offline and present a unified front to the witness log, which records that all harbors agreed — a behavior consistent with both honest agreement and collusion. We owe either a strengthening of the bonded mechanism that resists cross-harbor cartels, or a precise statement of the weakening. We have neither.

7.12.4 Cold-start admission and the invitation-only failure mode

The bonded-sponsor pattern of §7.7 requires a new harbor to find a sponsor willing to underwrite its probationary risk. In practice, this market may be thin — the historical evidence on whether invitation-bounded federations stay open is mixed [88]. We treat the structural pushback (many sponsors, small probationary scope, witness-log permanence) as engineering discipline, not as theorem.

7.12.5 Equivocation propagation speed

Property 7.5.1 separates cryptographic verification from dissemination. The former is constant work once both roots are present; the latter depends on topology, loss, and partitions. Establishing a deployment-specific tail bound is open and is necessary before pricing the witness bond from propagation risk. A second gap sits next to it: §7.5.4 names the sheaf Laplacian’s spectral gap $\lambda_2(\Delta_{\mathcal{F}})$ as the natural handle on anti-entropy mixing speed but derives no bound from it, and does not reconcile linear sheaf diffusion with epidemic gossip over append-only logs. *open*.

7.12.6 Cost of per-write durability

The cryptographic assertions and per-write durability checkpoints this layer inherits from Anchor carry a measurable I/O cost. They are designed for high-stakes coordination, where an audit trail is worth more than throughput, and not for bulk data movement or latency-critical paths. This paper does not measure that overhead at the federation layer and does not establish where the crossover sits. *open*.

7.12.7 What this means for the reader

A federation paper with six named open questions is doing its job. The point of the paper is to draw the new boundary cleanly so the open problems are visible; if every claim in this paper were closed, there would be no third paper to write. The honest read is that the Federated Harbor is closer to a research agenda than to a deployed product, and the prior two papers (Anchor,

Bonded) have a tighter ratio of theorem to conjecture for the same reason. The substrate is real; the federation layer is in progress.

The point of the paper is to draw the new boundary cleanly so that the unsolved problems are visible rather than hidden.

7.13 Conclusion

The two-machine problem is the structural sequel to the local-swarm problem of Anchor and the trust-as-infrastructure argument of Bonded. The proposed answer is not a new substrate but a clean extension: keep each daemon sovereign inside its own machine, add a shared witness log that no harbor controls, hang revocation gossip and conditionally restricted custody off the witness log, and gate federation admission on bonded sponsorship rather than permissionless entry. Each primitive does exactly one job. Each primitive is honest about what it does not do.

The proof posture matches the prior papers' discipline. Where we mechanize, we mechanize (with the same ProVerif and TLA+ infrastructure). Where we bound a threat, we name the bound and price the bond against it. Where we leave a question open, we name it. The threat-band diagram of Figure 7.2 is the most concentrated summary of where each claim lives.

What this paper supplies is a clean federation surface that could compose with the local substrate: a transfer ceremony, witnessed revocation records, a conditional custody design, and bonded admission. The protocol and models make the missing implementation and service-level assumptions explicit; they do not establish that arbitrary harbors can already interoperate safely.

The remaining work—runtime custody enforcement, deployment conformance, a multi-principal equilibrium model, and a cartel-resistance bound—is material. The local substrate is real; the federation described here remains a design with partial mechanization.

Chapter Appendices

7.A Verification Status

Artifact registry. The cross-paper status table in Bonded Commons [81] carries the items that are shared across the series; this appendix lists only the items specific to the Federated Harbor.

Table 7.1: Federated Harbor verification status. **closed:** model verified and runtime conformance test passes. **PARTIAL:** design specified, mechanization in draft. *open:* known unmechanized; threat band visible in Figure 7.2 (cobalt).

Claim	Method	Result	Artifact
Single-harbor capability authenticity (inherited)	ProVerif 2.05	closed	Anchor <code>harbor_card_v*.pv</code>
Cross-realm authenticity (§7.4)	ProVerif (draft)	PARTIAL pending witness-log freshness assumption	<code>fh-xfer.pv</code> (draft)
Attenuation monotonicity across boundary (Lem. 7.4.1)	ProVerif + Anchor sub-result	PARTIAL	<code>fh-att.pv</code> (draft)
Federated revocation dissemination (Prop. 7.5.1)	Conditional epidemic model [9]	PARTIAL expected asymptotic result; no partition deadline	<code>fh-conv.tex</code> (proof sketch)
Cross-witness equivocation evidence	Signed-root comparison; dissemination model	PARTIAL verification immediate once views meet; no hard delivery bound	—
Escrow extraction bound (Prop. 7.6.1)	Transition-system case analysis	PARTIAL conditional on non-bypassable custody; no runtime conformance	<code>fh-escrow.pv</code> (draft)
Cross-harbor Conservation (§7.9.2)	TLA+ / Apalache, $ F \leq 4$	PARTIAL model-checked at scale	<code>CrossHarborConservation.tla</code>
Sheaf detection of equivocation (Thm. 7.5.1)	Pre-registered statistical harness, 200 trials/arm	PARTIAL gates pass at the stated scope (single equivocator, real stalks, prefix restrictions); not mechanized	<code>sheaf_harness_v2.py</code>
Sheaf-Laplacian relaxation-time bound (§7.5.4)	—	<i>open</i> no derivation and no citation; the gossip and linear-diffusion models are not reconciled	—
Trustless settlement for non-fungible bonds	—	<i>open</i> no construction or impossibility proof	—
Multi-principal correlated-equilibrium extension	—	<i>open</i>	—
Cross-federation cartel-resistance bound	—	<i>open</i>	—

What this table means. Every row in the PARTIAL band is committed to and on the active workplan. Every row in the *open* band is a research question we cannot resolve from the desk; the proposal [83] names collaborators whose contributions are necessary to close them. We err toward listing fewer items as closed rather than more, in the same posture as Anchor and Bonded.

7.B ProVerif Models (draft)

For brevity we omit the full model text in this pre-print; the draft `fh-xfer.pv` extends Anchor’s `harbor_card_v3_delegation.pv` with a second signing key sk_B , an epoch root binding, and a cross-realm authenticity query of the shape sketched in §7.9.1. The full model will accompany the revised version once the witness-log freshness assumption is mechanized rather than assumed.

7.C TLA+ Specification (draft)

The full `CrossHarborConservation.tla` extends Bonded’s Conservation specification with per-harbor pools, an escrow pool, and an inter-harbor transit relation. Apalache symbolic model-checking at $|F| = 4$ harbors and bond depth 16 establishes that Conservation holds across all reachable states under the transition relation. The full specification accompanies the revised version.

The conclusion of the book The seven answers now form one discipline. Mediate every effect that must be prevented; let delegated authority only narrow; preserve enough evidence for a human to challenge the system; attach actions to a durable principal and work unit; make settlement conserve value; and cross organizational boundaries by exchanging verifiable records rather than by exporting sovereignty. Better models help inside this discipline. They do not replace it.

Solutions to the exercises

Chapter 1: The Single-Writer Kernel

Solution to Exercise 1.1

(exercise on page 6)

In memory the substrate still serializes, so resource exclusion, identity and presence, communication, obligation checks, policy checks, and self-attestation keep their transient meaning while the process lives. Everything the organs promise across a crash becomes vacuous: durable continuity, crash recovery, historical audit, and outcome evidence; durable identity collapses to an ephemeral handle. The decomposition survives; the cross-crash guarantees do not.

Solution to Exercise 1.2

(exercise on page 7)

The communication organ's home table is **messages**; it owes the layer above an ordered, cursor-addressable, durable envelope with explicit at-least-once semantics. Equally valid: **claims** for the resource organ (one compatible holder per conflict domain), or **commitments** for the obligation organ (no *done* without a current structured oracle receipt).

Solution to Exercise 1.3

(exercise on page 7)

A defensible nine-way cut: storage and durability; transaction serialization; identity and presence; resources and leases; message transport; stigmergy and projections; policy and capability; commitments and outcomes; attestation and recovery. It separates proof obligations that leak across the seven, above all transport semantics from message semantics and policy prevention from post-commit detection. It hides the simplifying fact that all local state shares one transaction boundary, so any such cut must retain a cross-cutting invariant: one ledger, one effect mediator.

Solution to Exercise 1.4

(exercise on page 8)

Module A renames `users.name` to `display_name`; module B, loaded later under older code, sees no `name` column and lazily adds it back. Writes now split between two columns according to module version and load order, and neither backfill is authoritative. Every destructive migration has this shape; only additive tables and columns are safe under any-module-any-order.

Solution to Exercise 1.5

(exercise on page 9)

Command line, authenticated Unix-socket request, daemon request queue, the sole read/write connection, one transaction, WAL commit. The discipline holds by routing at two points, the client-to-API boundary and the daemon's sole-connection boundary; SQLite's file lock is one backstop at write time. The primary story is routing, not a database rule that forbids other connections.

Solution to Exercise 1.6

(exercise on page 9)

Expand, backfill, dual-read-or-write, contract keeps mixed versions safe for a while, but the contract step and the dependency order are global facts. Store a migration ledger with a schema epoch, each module's compatibility range, its dependencies, and a backfill watermark. Idempotent self-initialization can remain for additive tables and columns; safe rename, backfill, and down-migration cannot preserve unrestricted any-module-any-order. A `user_version` sequencer, or an equivalent ordered capability ledger, is unavoidable.

Solution to Exercise 1.7

(exercise on page 11)

(a) I1a-survivable. (b) An orderly shutdown flushes, so a genuinely clean reboot is not the I1b case; an abrupt reset during the reboot is I1b-exposed. (c) I1b-exposed. (d) I1a-survivable after commit. None of the four lets an uncommitted transaction survive.

Solution to Exercise 1.8

(exercise on page 11)

The commit is power-loss exposed from t_0 until the WAL frames that contain it are synced. A page-count threshold bounds accumulated WAL frames, not elapsed time: at a low or zero write rate it may not fire for arbitrarily long, so it gives no wall-clock bound. Only with an independently guaranteed write-rate floor $w_{\min}$ pages per second and threshold P does $\delta \lesssim P/w_{\min}$ follow, and this chapter assumes no such floor.

Solution to Exercise 1.9

(exercise on page 11)

Make durability a typed argument of the transaction, `PROCESS_CRASH` or `POWER_LOSS`, never an adjective in prose. Route `POWER_LOSS` operations through a connection configured `synchronous=FULL` before the transaction opens and verify the commit before returning; if SQLite settings are connection-scoped, a separate fully-synced financial ledger is the cleanest isolation. Ordinary claims stay on `NORMAL`. Hard-reset fault tests must exercise the stronger path. The guard against silent defaulting is the type: a write path that names no durability class does not compile, and a forced checkpoint bounds recovery and log size but is not the per-write primitive.

Solution to Exercise 1.10

(exercise on page 14)

A release with no matching (key, holder) row is a successful no-op so that retries are idempotent: this is I3. It must never delete another holder's row. If it raised, a client that timed out on its first release could not distinguish success from failure, and at-least-once handling would break.

Solution to Exercise 1.11

(exercise on page 14)

The daemon serializes the two requests. The first `INSERT(key, holder)` commits. The second hits the unique-key violation, selects the extant row inside the catch path, and returns `DENIED(holder=first)`: that select is where the loser learns the name. For range and hierarchy claims the trace is safe only after both requests map to the same canonical conflict key or the overlap test runs in one immediate transaction.

Solution to Exercise 1.12

(exercise on page 14)

Add a durable FIFO ticket queue per conflict domain, a maximum lease L , a monotonically increasing fencing epoch, and grant-to-head on release, on acquire, and on lazy expiry. No background scheduler is required for safety, but liveness requires some live caller or tick to trigger the transitions. Under daemon availability, eventual requests, and non-renewable bounded leases, a requester with q predecessors waits at most about $(q + 1)L$ plus processing delay; without those assumptions no bounded wait exists. Every external effect must validate the fencing epoch, or an expired holder can still act.

Solution to Exercise 1.13

(exercise on page 16)

If the evaporation tick stalls for an hour, tick-only storage still reports yesterday's marker weight as today's. Read-time evaluation computes $w r^{\Delta t}$ from the persisted timestamp and never presents the stale stored number as current.

Solution to Exercise 1.14

(exercise on page 16)

The transport delivers the request twice. The consumer applies one semantic state transition and recognizes the duplicate as transport nondeterminism; diagnostics may count the redelivery, the operator's view shows one act. An envelope idempotency key plus a unique (consumer, key) receipt lets the consumer store "effect applied" atomically with the effect, making retry safety explicit rather than conventional. Exactly-once is still not guaranteed for an unbrokered external effect.

Solution to Exercise 1.15

(exercise on page 17)

For each marker kind, log creation, reads, the actions it influenced, its supersession, and the time at which operators label it stale. Fit a survival curve and choose the half-life that minimizes $C_{\text{stale}} \cdot \text{false-live} + C_{\text{lost}} \cdot \text{false-expired}$, validated on held-out projects. Claims and presence should decay far faster than decisions or learned skills. Publish the confidence and re-estimate under drift.

Solution to Exercise 1.16

(exercise on page 27)

Note-count monotonicity can be regimented by append-only storage or a trigger that rejects deletes and regressive updates. Capability attenuation and lock-owner release can be pre-checked only if authenticated identity, the capability order, and every relevant effect path pass through the mediator. Duplicate keys and boot admission are already pre-effect. Heartbeat freshness cannot be rejected at the heartbeat insert: staleness is an absence observed only after time passes, and under asynchrony a slow actor is indistinguishable from a failed one. Out-of-band filesystem and network effects remain detect-only until mediated.

Solution to Exercise 1.17

(exercise on page 28)

`close(done, oracle_ref="")` reaches the oracle-reference check, returns `REFUSED`, leaves the commitment open, and records the denied authority act. It never reaches classification, current-state verification, or the `DONE` transition: the empty reference fails the finite-vocabulary test before any oracle is consulted.

Solution to Exercise 1.18

(exercise on page 28)

Note monotonicity: the prohibited event is a write the daemon mediates and the trigger (the previous count) is committed state; top-right, regimentable, and append-only storage is the supervisor. Capability escalation: regimentable only if every effect path and the identity behind it pass through the mediator; an escalation exercised through an unmediated channel is an uncontrollable event, which moves the rule to the left column until the channel is owned. Lock-owner validity: the release is mediated and the owner is committed state, top-right, provided the actor identity is authenticated (I12); with a self-asserted identity the trigger is not witnessed and the rule drops to the bottom-right cell.

Solution to Exercise 1.19

(exercise on page 28)

The first gates a controllable event on a controllable trigger: both `api_call` and `spawn_child` lie in Σ_c , the policy never disables an uncontrollable event, and it is regimentable. The second must disable `internal_plan`, an uncontrollable event, in its post-trigger state, and the plant

enables it there; the history `fs_write` followed by `internal_plan` witnesses the failure of controllability, so the rule is detect-only.

Solution to Exercise 1.20

(exercise on page 28)

P_4 is a two-state taint automaton whose trigger is `git_push`. In the product with P_2 (no `git_push`, ever) the trigger is disabled in every reachable state, so P_4 's tainted state is unreachable in the closed loop and the disablement map is unchanged: `ok` disables `{net_egress, git_push}` and `tainted` disables `{net_egress, git_push, fs_write}`. Controllability still holds, since no reachable state disables an uncontrollable event. The lesson is the one synthesis is for: P_4 is vacuous under P_2 , and a hand-written engine would have carried the dead rule forever.

Solution to Exercise 1.21

(exercise on page 28)

A protected-run receipt: tree hash, a suite hash the principal controls, runner or image hash, command, timestamp and nonce, exit code, and a signed result. Closure verifies every binding and the freshness. This resists free-text and stale-result manipulation; it is not universally Goodhart-proof, because the suite may be incomplete or poisoned. The honest theorem is syntactic provenance and non-replay, not semantic correctness.

Solution to Exercise 1.22

(exercise on page 30)

Notes preserve explicit goals, rationale, observations, and artifact pointers. They cannot preserve unflushed edits, process memory, open handles, provider caches or hidden state, or a latent next action the model never externalized. A successor can understand the story and still be unable to resume the exact computation.

Solution to Exercise 1.23

(exercise on page 30)

The append commits. A later delete or regressive count also commits and enters the operation log. Only then does the subscriber observe the count drop and raise or compensate. The violation is durable at the second commit; the monitor proves detection, not prevention.

Solution to Exercise 1.24

(exercise on page 30)

A content-addressed task capsule sealed at a safe tool or message boundary: base tree and worktree blobs, environment and tool manifests, open claims with epochs, commitments and acceptance criteria, exact tool input and output, pending external operations with their idempotency keys, decisions and hypotheses, a provenance-preserving transcript or compaction, and external references. Flush the ledger and quiesce mediated effects before sealing. This restores observable task state and evidence; it cannot recover hidden activations, unexposed

reasoning, provider-internal caches, or unrecorded intent. Arbitrary-provider recovery is semantic continuation, not bit-identical resurrection.

Solution to Exercise 1.25

(exercise on page 34)

`delegate` from the initial state with a child scope not contained in the root grant's scope $\{a, b\}$, for instance $\{a, b, c\}$: with the attenuation guard off the grant is created and I4 fails immediately. No other guard can be broken in one step, because effects need the executing phase, settlement needs the verified phase, and a double settlement needs two operations.

Solution to Exercise 1.26

(exercise on page 34)

One candidate: a contested work unit never settles (`settle` is guarded on the verified phase, but the transition from contested back to a settleable phase is unconstrained). Add the guard, rerun the search, and confirm the state count and that the mutation suite finds a trace with the guard off; if the checker finds none, the invariant was already implied and does not earn its row.

Solution to Exercise 1.27

(exercise on page 36)

One daemon connection handles every claim and lock read and write; no `read_uncommitted`; respond only after commit; no out-of-band writer. A write-capable external connection violates the first and the fourth at once, the out-of-band-writer assumption most literally. A read-only connection need not break write linearizability, but it observes under different timing and API semantics.

Solution to Exercise 1.28

(exercise on page 36)

A invokes `claim(k)` and commits at c_1 . B and C overlap; B's denied read commits at c_2 , A releases at c_3 , C's acquire commits at c_4 . One valid order is A-acquire, B-denied, A-release, C-acquire. If B and C overlap around the release, either may linearize first, as long as each result matches the state at its point and non-overlapping operations keep their real-time order.

Solution to Exercise 1.29

(exercise on page 36)

Generate identical state-machine traces against both bindings: schema initialization and migrations, acquire, reclaim, release, and expiry, overlapping calls, transactions and rollback, busy and lock behavior, encodings, pragmas, crash and reopen, fault injection. After every step compare returned values, errors, serialized rows, schema, and recovery state, including the deployed binary in CI. A seeded fuzz misses divergences that depend on wall-clock timing, on a pragma the fuzz never sets, or on a crash at a point the fuzz never interrupts; targeted fault injection and pragma sweeps catch those. No finite suite makes I11 a theorem: that

needs a formal refinement or equivalence proof, or the same verified binding on both sides. Label the suite conformance evidence.

Solution to Exercise 1.30

(exercise on page 37)

The port stays claimed and a live or ghost session row exists, but no commitment says what work owns them. Other agents see a busy port and a presence they cannot reconcile; the failed starter may retry into its own leftovers.

Solution to Exercise 1.31

(exercise on page 37)

Expose one `begin_work` endpoint that opens `BEGIN IMMEDIATE`, validates every input, inserts the claim, session, commitment, and outbox rows, and commits once; on any error it rolls back all of them. A saga would release the claim and mark or delete the session when the commitment insert failed, and each compensation can itself fail or crash between steps. All three rows live in one SQLite file, so the local transaction is both simpler and stronger.

Solution to Exercise 1.32

(exercise on page 40)

A later same-user tamperer of the database file, or a remote synchronizer that cannot rewrite an externally anchored root. The same-user process is out of scope at the substrate; the adversary becomes real at the cross-machine economy layer. Without an independent anchor or verifier, the writer can rewrite the log and recompute the chain.

Solution to Exercise 1.33

(exercise on page 41)

The release path compares the supplied actor identity to the lock owner. If a legacy endpoint accepts a self-asserted identity, the attacker submits the victim's string, the equality check passes, and the owner-scoped delete executes. The lock rule is correct relative to its input; I12 failed to authenticate the input. Complete credential and peer binding must precede every security-relevant write.

Solution to Exercise 1.34

(exercise on page 41)

The minimum useful wall is a daemon under a separate UID (or a VM), a daemon-owned database and key, a Unix-socket ACL plus kernel peer credentials, and brokered git, filesystem, and network effects. An OS keychain protects a key at rest but not arbitrary signing requests from an equally authorized same-user process; sealed memory does not stop same-UID tracing or injection without an OS isolation policy; external root anchoring detects a later log rewrite but does not prevent it. No trusted platform module is required for strong local separation: separate privilege and complete mediation are. After the sandbox the same-user adversary moves to the mediated-effects row, where side channels the sandbox does not enumerate

remain in scope.

Chapter 2: The Anchor Protocol

Solution to Exercise 2.1

(exercise on page 61)

The model hands the attacker *both* the honest issuer’s public key `vkA` and the HMAC key `vkB` in the clear — the worst case for a verifier that is only supposed to trust Ed25519-shaped tokens. `VerifierNaive` does not pin an algorithm; it pattern-matches on whichever type tag arrives, `tokenA` or `tokenB`, and calls that tag’s own verification routine. Knowing `vkB`, the attacker builds `tokenB(ALG_B, m, hmacB(vkB, m))` for any message `m`, including one the honest `IssuerA` never signed; the naive verifier’s second branch matches the `tokenB` shape, the HMAC checks out under a key the attacker already holds, and `accepted_naive(m)` fires with no matching `issued_A(m)`. `VerifierPinnedA`, by contrast, only ever deconstructs the `tokenA` constructor — a `tokenB` token is not merely rejected, it is a shape the pinned verifier never attempts to read. Pinning means fixing, out of band, which single verification routine and key a given context accepts, so the token’s own header or shape can never be the thing that chooses how it gets checked — exactly the rule of Design invariant 2.2.1, mechanized here against an attacker who already holds every key the naive path would trust.

Pinned verdict (`proofs/anchor/token-verify/algconfusion.run.log`):

```
1 RESULT event(accepted_pinned(m_4)) ==> event(issued_A(m_4)) is
   true.
2 RESULT event(accepted_naive(m_4)) ==> event(issued_A(m_4)) is
   false.
```

The first line is the property this chapter relies on; the second is the negative control, checked in the same file so the true result is not vacuous.

Solution to Exercise 2.2

(exercise on page 62)

It checks that every chain `C` accepts corresponds to one `P` actually authorized, with the *same* (P, A, B, C, m) tuple: no hop can be replayed against a different message, and no hop can be spliced out of one chain into another. The defense is per-hop nonce freshness — each signature binds `hopBind(nonce, prev_id, next_id, message)`, and the verifier requires every nonce to have been issued and consumes it on acceptance, so a captured hop cannot be re-presented in a different context.

Pinned verdict (`proofs/anchor/delegation/chain-replay.run.log`):

```
1 RESULT event(chain_accepted(idP_4,idA_3,idB_2,idC_1,m_4)) ==>
   event(chain_authorized(idP_4,idA_3,idB_2,idC_1,m_4)) is true.
```

This is a different property from Property 2.D.1’s capability-subset soundness: chain replay says who signed what is delivered intact; attenuation soundness says authority never grows along the way. §2.5.3 cites this exact artifact as the depth-3 evidence behind the “Delegation-chain depth” boundary.

Solution to Exercise 2.3

(exercise on page 64)

`proof_verify_logic_only` stubs signature verification and base64 decoding and shows the surrounding parse and branch logic cannot panic on any 32-byte, dot-containing, UTF-8 input within ten unwindings — a bounded absence-of-panic result under those stubs, not a proof that `verify` itself is correct. A 32-byte input cannot even hold a real three-part card, so only the *rejection* path is exercised; a reader should not conclude the acceptance path was checked at all.

`proof_constant_time_behavior` calls the comparator once over symbolic 16-byte pairs and shows it cannot panic; it is not a two-run relational proof that the *time* taken is independent of the secret, and Kani adds nothing here beyond the source-level argument already given by inspection (no early return on a secret byte). Compiler, cache, and microarchitectural timing stay entirely outside it — a reader should not read “Kani checked it” as “this is constant-time in the deployed binary.”

`proof_capability_attenuation` asserts two concrete, hard-coded vectors, one subset accepted and one escalation rejected; it is a unit test wearing a Kani harness, not a universal statement over arbitrary capability structures. The actual every-hop attenuation property is what §2.D.1’s ProVerif model establishes, not this harness — a reader should not treat the two vectors as covering the property Property 2.D.1 states.

The bound in each case is recorded, not just asserted here: `whitepaper/corpus.json` carries an `evidencePolicy` field for `harbor-card-kani-verify-logic-only`, `harbor-card-kani-constant-time` and `harbor-card-kani-capability-attenuation` stating exactly these limits, so a claim that drifted past this chapter’s wording would be visible against the manifest rather than only against memory.

Solution to Exercise 2.4

(exercise on page 64)

As this chapter states it, not more: the Kani harnesses give bounded absence-of-panic for the token parser on 32-byte inputs with signature verification and base64 decoding stubbed out entirely, a source-level argument that the comparator’s branches do not depend on secret bytes, and two concrete, hard-coded vectors for the capability-subset check. None of the three is a proof about the real cryptography (stubbed away in the first, untouched by the other two), about inputs larger than the harness’s own bound, about the compiled binary’s timing behavior, about the FFI boundary itself, or about any code path outside `harbor-card-rs`. The ProVerif models are symbolic proofs about the *protocol* under a Dolev-Yao adversary, not about this Rust source at all — the bridge between “the model” and “this source” is exactly the assumption gap Figure 2.8 draws as a dashed band, asserted by a human, not discharged by either tool.

A stronger harness would replace the two hard-coded attenuation vectors with an arbitrary capability structure under an asserted partial-order property, so the check is over the order rather than two fixed cases, with a negative control that fails when the `is_subset` guard is removed — the same discipline the ProVerif attenuation model already applies for Property 2.D.1. It would also need to state parser refinement (that the stubbed primitives match their real implementations) as a separate, currently absent, obligation. None of that exists yet, and this chapter does not claim it does: `whitepaper/corpus.json` pins the three harnesses’ present bounds in their own `evidencePolicy` fields, and `scripts/proofs/run-proverif.py` (CI job “ProVerif / model estate”) re-runs every ProVerif model named in this chapter’s exercises against its committed source on every change — the freshness guarantee behind every pinned line quoted above, not a claim about the harnesses it does not mechanize.

Solution to Exercise 2.5

(exercise on page 67)

A valid signature proves exactly one thing: the holder of the signing key produced these bytes. It says nothing, by itself, about canonical decoding (did the verifier parse the *same* bytes the signer meant, under one canonical encoding); key-to-principal binding (whose key is this, and is that binding itself authenticated — §2.5.1’s PID-reuse case is a binding failure downstream of a perfectly valid signature); audience and resource semantics (was this token issued for *this* harbor and *this* action); freshness (**exp** checked against a clock the bearer does not control); ancestor revocation (§2.2.4 — a signature does not know its own chain was withdrawn upstream); or that every consequential effect actually consults the verifier at all. Table 2.3 lists only the cryptographic set as ProVerif-verified; authorization is the larger claim built on top of it, not a synonym for it.

Because Harbor Cards are bearer credentials — whoever presents the bytes gets the capability, with no further proof of possession — a card sniffed off loopback (§2.5.3, “localhost is a real network”) is exactly as usable to an attacker as to its intended holder for the remainder of its TTL. Re-checking per request narrows the window in which a *revoked* card is still honored (§2.2.4); it does nothing against a card that is merely stolen and still technically valid, because the verifier cannot distinguish the two bearers by the token alone.

A status table sharper than Table 2.4’s three tiers would keep four things distinct rather than folding them into one word: symbolic protocol correspondence at a fixed modeled hop depth (what ProVerif proved); bounded implementation memory and control-flow checks (what Kani proved); runtime verifier integration (whether the deployed path actually calls the verified routine on every effect); and system-level complete mediation (whether every effect-producing path is reachable only through that call). This chapter’s artifacts presently evidence the first two; the last two are asserted in prose (§2.4, “narrows the gap ... to the compiler’s”) rather than mechanized, and reading any result in this chapter as more than it is means collapsing exactly this distinction.

Solution to Exercise 2.6

(exercise on page 75)

(i) A root-vs-final verifier asks only `is_subset(cap_write, cap_root)`: since the root itself holds write, this is true, and *C* is accepted with write — an authority *B* never held and never legitimately re-delegated. (ii) Algorithm 2.4’s loop checks each hop against the one immediately before it: hop 1, `is_subset(cap_read, cap_root)`, holds, so *B*’s grant stands; hop 2, `is_subset(cap_write, cap_read)`, fails, because write is not inside what *B* was actually handed, so *C* is rejected at the exact hop where authority tries to grow back. Only the root-vs-final verifier accepts; the per-hop verifier does not.

The induction: if every hop satisfies $\text{cap}_i \subseteq \text{cap}_{i-1}$, transitivity of $\subseteq$ chains them directly into $\text{cap}_k \subseteq \text{cap}_{k-1} \subseteq \dots \subseteq \text{cap}_0$. Checking $\text{cap}_i \subseteq \text{cap}_0$ independently at each hop supplies no such chain: it only asks whether each capability fits inside the *root*, which a re-escalating middle hop can satisfy ($\text{write} \subseteq \text{root}$) while still expanding relative to what that hop’s own parent actually held ($\text{write} \not\subseteq B\text{’s read}$). This is why the sentence following Algorithm 2.4 credits Property 2.D.1 with the single hop only: the multi-hop induction is what the per-hop *discipline* buys structurally, and its mechanized confirmation is the v6/v7 pair immediately below, not a restatement of the property at depth greater than one.

Solution to Exercise 2.7

(exercise on page 75)

The root holds `cap_write`. Honest *A* correctly attenuates to `cap_read` for *B* — a legitimate restriction. Malicious *B*, holding only that read-capped delegation, signs a *new* delegation to *C* carrying `cap_write`: an authority *B* never held, forged with *B*’s own legitimately-issued delegation key, not by breaking any signature. `NaiveVerifier` unwraps the two-hop chain and checks `is_subset(final_cap, root_cap)` — the final capability against the root only — and never re-derives what the intermediate hop actually held. Because `cap_write` is a subset of the root, the check passes and *C* is accepted with full write authority, although *B* itself only ever held read. This is hop-2 escalation: a child re-grants an authority its own parent narrowed away, and a root-vs-final comparison is structurally blind to it.

Pinned output (`analyses/harbor_card_v6_results.txt`):

```

1 The event Accepted(id_c_1,harbor_id,cap_write) is executed at {43}
  in copy a_10.
2 A trace has been found.
3 RESULT not event(Accepted(cc_2,harbor_id[],cap_write)) is false.
```

ProVerif does not merely report the query false; it exhibits the concrete trace above as a witness.

Solution to Exercise 2.8

(exercise on page 75)

The only change is in the verifier: `v6`’s single check `is_subset(final_cap, root_cap)` becomes two checks in sequence, `is_subset(mid_cap, root_cap)` for hop 1 (*A* → *B* against the root) *and* `is_subset(final_cap, mid_cap)` for hop 2 (*B* → *C* against what *B* actually held). Same malicious *B*, same forged `cap_write` delegation, same query — only the verifier’s comparison target moves from “root” to “immediate parent.”

Pinned output (`analyses/harbor_card_v7_results.txt`):

```

1 RESULT not event(Accepted(cc_2,harbor_id[],cap_write)) is true.
```

The identical escalation attempt `v6` accepted is now rejected at the `is_subset(cap_write, cap_read)` step, because write is not a subset of what *B* was actually holding.

Solution to Exercise 2.9

(exercise on page 75)

Both models share a grant chain — a first-party caveat folded into the root HMAC, a third-party commitment `vid` keyed by a shared caveat key, and a discharge bound to the request via `hmac(BIND0, ...)`. In `v2`, two grants (a low-authority one the daemon discharges, and a high-authority one it never discharges) share the same caveat key — realistic, because one live session’s check backs every grant minted under it. `v2`’s naive relay computes `bound = hmac(BIND0, d1)`, binding the discharge to the *caveat* only, never to which grant it is being presented for. An attacker who holds the daemon’s genuinely-issued low-authority discharge — public, since it travels in the clear — can present it alongside the public high-authority grant; the relay recomputes the same discharge chain from the shared caveat key and accepts. `RelayAuthorizesNaive` fires for the high-authority grant’s signature although the daemon’s `DaemonIssuesDischarge` event never named it. This is the macaroon-construction analogue of `analyses/harbor_card_v6_multihop_attack.pv`: a verifier that checks a credential against

something coarser than the exact object in front of it — there, the root instead of the immediate parent; here, the caveat key instead of the specific grant — admits a substitution.

v1's verifier folds the grant's own signature into the binding, `bound = hmac(BIND0, (g_sig, d1))`, not just the discharge chain. A discharge minted for the low-authority grant's signature cannot satisfy the high-authority grant's binding check: HMAC is modeled as a one-way keyed PRF with no destructor, so the attacker can replay the low discharge's bytes verbatim but cannot recompute the high-authority binding without the private caveat key only the daemon holds.

Pinned output, v1 (`analyses/macaroon_discharge_v1_results.txt`):

```
1 RESULT event(RelayAuthorizes(s)) ==> event(DaemonIssuesDischarge(s)) is true.
```

Pinned output, v2 (`analyses/macaroon_discharge_v2_naive_unsound_results.txt`):

```
1 RESULT event(RelayAuthorizesNaive(s)) ==> event(DaemonIssuesDischarge(s)) is false.
```

Both discharges are unforgeable without the caveat key; the entire gap in v2 is that its binding forgets which grant it was issued for.

Chapter 4: From Spawn to Person

Solution to Exercise 4.1

(exercise on page 123)

A spawn is a temporary process filling a role; a person is a role-holder whose accountable continuity survives replacement of that process.

Solution to Exercise 4.2

(exercise on page 123)

Read right-to-left. No reputation $\Rightarrow$ a buyer cannot price differentiated trust, so the alleged tradeable asset is adverse-selection noise. No durable accountable person/outcome identity $\Rightarrow$ reputation attaches to a disposable incarnation and can be reset or duplicated. No continuity $\Rightarrow$ successive role-holders cannot be shown to belong to one lineage; history does not follow replacement. No memory/checkpoint/evidence $\Rightarrow$ a successor lacks both resumable state and a witnessed chain from old work to new work. The chain is not literally linear: non-forgeable identity and witnessed outcomes are parallel prerequisites for accountable continuity; checkpointing helps resumption but is not logically necessary for every reputation system.

Solution to Exercise 4.3

(exercise on page 124)

The human operator is a moral/legal person but should be represented economically as the *principal* above agent identities (Def. 4.6.2). If one principal can mint unlimited apparently independent agent-persons, it can whitewash sanctions, forge quorums, reuse aggregate collateral, and farm reputation through self-trade. Bind every agent lineage to a principal

credential, cap principal-wide exposure and newcomer credit, and disclose shared control: the actor may change; the economic liability root must not.

Solution to Exercise 4.4

(exercise on page 127)

Capable-and-permitted: an agent has a test-runner tool and authority to run the suite.
 Capable-but-forbidden: the process can invoke raw `git push -force`, but policy forbids it.
 Permitted-but-incapable: the role is authorized to deploy, but the incumbent has no deploy credential/network route.

Solution to Exercise 4.5

(exercise on page 127)

Reputation attaches to the person's continuity witness and witnessed outcomes, not to the abstract role's O , C , or A . A role is fillable and history-free; otherwise a fresh spawn could inherit the prior holder's credit merely by taking the same job title, exactly the attribution Alice loses in §4.1.

Solution to Exercise 4.6

(exercise on page 127)

Keep a role template $(O_{\text{required}}, A_{\text{role}})$ separate from an incumbent capability set C_{subject} . At assignment require each current obligation to satisfy $O_{\text{active}} \subseteq A_{\text{role}} \cap C_{\text{subject}}$. Keep $C_{\text{subject}} \setminus A_{\text{role}}$ as capable-but-forbidden and $A_{\text{role}} \setminus C_{\text{subject}}$ as authorized-but-unavailable, which triggers provisioning or reassignment. Never equate C with A .

Solution to Exercise 4.7

(exercise on page 129)

Connectedness is a direct memory/intention link between two incarnations; continuity is the transitive, provenance-bearing chain across arbitrarily many links. A successor may read its predecessor's handoff and be psychologically connected while using a new outcome-ledger key, so the accountable chain is broken: connected, not economically continuous.

Solution to Exercise 4.8

(exercise on page 129)

In the manuscript's analogy, the body-lease is the replaceable live substance/process; the actor-soul is the durable identity/mailbox/history that carries consciousness-like continuity. Respawn replaces the body-lease and should retain the actor-soul only under a valid succession event.

Solution to Exercise 4.9

(exercise on page 129)

Represent a fork as a lineage DAG, never as two copies of one linear identity. Both children may display the ancestor's evidence with an explicit inherited/discounted prior, but neither duplicates the ancestor's outstanding authority, collateral, or spendable reputation. Allocate a fresh branch ID, split or revoke exclusive capabilities, and aggregate risk at the common principal/ancestor. Copying full reputation to both invites credit duplication, quorum multiplication, and double-spending of grants; giving it to only the first child creates a fork-race attack. This sketch is now a proved theorem — see the Numbers by hand box below.

Solution to Exercise 4.10

(exercise on page 129)

The script prints: “sum of inherited priors over the closure = $0.900 + 0.810 + 0.729 = 2.439 > q = 1.0$ ” for the budget-only counterexample, then “4000 random DAGs ... 0 violations; max live/witnessed ratio observed = 0.999995” for the transfer-semantics sweep, then “an 8-way copy-fork of one outcome yields 8.2x its witnessed value” for the copy-full mutation. By hand: $0.9 + 0.9^2 + 0.9^3 = 0.9 + 0.81 + 0.729 = 2.439$, three terms of a geometric series, confirming the closure-sum counterexample without running anything; under transfer semantics the same chain instead *debts* each hop, so the live total telescopes to $0.9^3 = 0.729 \leq 1$, which is what the sweep's 0 violations certifies at scale.

Solution to Exercise 4.11

(exercise on page 132)

The checkpoint organ is the specifically weak one: it forwards notes rather than execution state. The outcome-ledger organ is also only partial in the implementation: commitments and oracle closure exist, but neutral graded outcomes and reputation binding do not.

Solution to Exercise 4.12

(exercise on page 132)

Memory retains the session, notes, and explicit decisions. The current checkpoint may retain a handoff and perhaps a committed diff, but not working process state, tool handles, pending effects, or hidden provider state. The outcome ledger retains the commit only if it is bound to a work unit and independently witnessed; a naked Git commit is an artifact, not proof of accepted delivery. A strong checkpoint would also preserve the worktree, environment, open claims/epochs, acceptance state, tool receipts, and the idempotency journal.

Solution to Exercise 4.13

(exercise on page 133)

A useful death taxonomy, in increasing order of what survives: (1) *record survival* — logs/notes only, reconstruction is manual; (2) *artifact survival* — a content-addressed worktree and outputs survive, computation is rerun; (3) *semantic task capsule* — artifacts plus obligations, decisions, transcript/projection, tools, and a pending-effect journal, so a new provider can continue observably; (4) *execution snapshot* — process memory/runtime state survives under

a controlled runtime, rarely portable; (5) *latent state* — unexposed activations, KV cache, or “intent” dies, fundamentally unrecoverable from an arbitrary provider. Checkpoint guarantees should state which tier they provide.

Solution to Exercise 4.14

(exercise on page 138)

Correctly scoped, Property 4.6.1 says reputation has economic force only to the extent that the accountable principal cannot cheaply discard or multiply the liability-bearing identity. Whitewashing escapes a negative record when newcomer access is better; a Sybil attack multiplies votes/credit when the mechanism counts identities. Non-forgability is necessary for those mechanisms under those access rules, but it is not sufficient by itself.

Solution to Exercise 4.15

(exercise on page 139)

Full work with a reduced economic ceiling lets an honest newcomer demonstrate competence without being denied livelihood, while limiting the maximum one-shot gain from abandoning a mature sanctioned identity. “Reduced work” slows evidence accumulation and taxes all honest entrants; “reduced exposure” targets the externality instead. The ceiling schedule must be chosen so the maturation opportunity cost exceeds plausible evasion gain.

Solution to Exercise 4.16

(exercise on page 139)

Principal binding is the load-bearing defense: aggregate reputation, exposure, counterparties, and sanctions across the 1,000 agents, so self-trades add little or no independent evidence. A listing fee raises attack cost but only deters while total fees exceed the minted value. Sampled adversarial re-audit detects fabricated work, with false-positive burden approximately the sampling rate times the honest high-volume population and the adjudication error. Add sharply diminishing credit per principal pair, independent payer stake, and principal-wide reserve limits.

Solution to Exercise 4.17

(exercise on page 139)

The script prints “schedules tested = 76000”, “dominating schedules = 0/4000 instances (expected 0)”, and “min (H(g) - H(cliff)) = +2.817e-02 (>= 0 means undominated)”; the exchange step’s table shows H at an all-mass-at- t schedule growing as $G_{\max}(\delta_h/\delta_f)^t$ and reports “strictly increasing in t: yes”. By hand: with $\delta_h > \delta_f$, $(\delta_h/\delta_f) > 1$, so its powers strictly increase in t — deterrence bought later always costs the honest newcomer more, which is the whole exchange argument in one inequality.

Solution to Exercise 4.18

(exercise on page 139)

The script prints “reachable states to depth 7: 747 (467 post-migration, 693

with an open sanction window)” and “ok intact: ZERO Def-III.6.1 violations over all reachable states [internal]”; the four mutants (drop scope, drop clause (i), drop clause (ii), drop clause (iii)) are each caught with shortest counterexamples 1, 2, 2, 2 steps. The one-step break is the scope mutant `m0`: keying sanctions by *(identity, engine)* instead of by identity alone lets a single dishonest outcome escape through a sibling engine key inside the *same* identity — no migration needed at all. By hand: this is why the intact protocol sanctions per *principal* while pricing per *(principal, engine)*; collapsing that distinction is the shortest way to violate Def. III.6.1.

Solution to Exercise 4.19

(exercise on page 141)

Local non-forgable identity proves that, inside one trusted daemon, an actor cannot freely impersonate or re-pick its daemon-issued handle. Cross-operator attestation must additionally bind a key to a durable principal across a hostile operator boundary, establish proof of possession/freshness, aggregate related agents, and support revocation and audit.

Solution to Exercise 4.20

(exercise on page 141)

That sentence secretly assumes the cross-operator stone. With only local identity, Bob learns at most “Alice’s daemon asserts this actor/key.” A hostile Alice can mint arbitrary local actors, rewrite local history, or misstate principal separation.

Solution to Exercise 4.21

(exercise on page 141)

A witness log proves publication order and exposes equivocation; it does not prove who controls a key or that two keys are distinct principals. Close the gap with a scarce external root: hardware/remote attestation, an organizational/KYC issuer, a bonded sponsor chain, or a combination. A transparency log makes bindings and rotations auditable but not true; a sponsor creates economic accountability but can cartelize; a shared identity issuer centralizes and leaks privacy; hardware roots attest devices/code, not unique humans. State the chosen trust and Sybil assumptions.

Solution to Exercise 4.22

(exercise on page 143)

With a complete, mostly stable paired-comparison history, batch Bradley–Terry uses all observations and is more appropriate than order-sensitive streaming Elo. If abilities drift, use a time-varying/dynamic model rather than pretending full-history stationarity.

Solution to Exercise 4.23

(exercise on page 143)

A binary protected acceptance test can update a beta-Bernoulli reliability estimate (or a calibrated per-task success model). Human aesthetic preference is naturally pairwise and may

use Bradley–Terry/TrueSkill with judge effects. EigenTrust expects a who-trusts-whom graph and therefore fits neither raw task-outcome signal directly.

Solution to Exercise 4.24

(exercise on page 143)

If identities are self-chosen, an exact Bayesian estimator simply produces precise scores for disposable names. If the agent authors the “oracle,” a sophisticated estimator precisely learns manipulated labels. Better statistics cannot repair missing provenance, adversarial labels, or liability binding; secure the event-generation substrate first.

Solution to Exercise 4.25

(exercise on page 145)

Stationary arm rewards versus model/skill/version drift; one scalar reward versus buyer-weighted quality vectors; a non-strategic environment versus Goodhart, collusion, and white-washing; and an observed/trusted reward versus a capturable grader.

Solution to Exercise 4.26

(exercise on page 145)

A contextual bandit is appropriate for one principal privately routing its own tasks among candidate agents, with a fixed local objective, controlled feedback, an exploration budget, and no public asset being minted. The result is a routing policy, not public reputation.

Solution to Exercise 4.27

(exercise on page 145)

Capping the marginal reputation return can reduce direct improvement races, but agents may redirect spending into unmeasured signaling, judge capture, identity proliferation, or lobbying for task mix. Efficiency requires the full game: improvement cost, social value of quality, transferability, and alternative signals. Candidate controls are concave credit, buyer-local vectors, nontransferable evidence, and taxes/bonds on negative externalities. No cap is generically efficient without those payoff assumptions.

Solution to Exercise 4.28

(exercise on page 150)

Accuracy: competent, a protected test/spec oracle; incompetent, a self-reporting worker. Aesthetics: competent, a conflict-free expert reviewer for that domain; incompetent, a token-cost meter or a same-family unblinded model. Efficiency: competent, a protected resource meter normalized for the task; incompetent, a style reviewer guessing from prose.

Solution to Exercise 4.29

(exercise on page 150)

The requesting principal/daemon defines the judging work and eligible pool; the selected judge posts its own bond; the grade lands as a signed witnessed event in the subject's outcome ledger with judge/provenance; a conflict-free re-audit or appeal that overturns it slashes the judge under the predeclared rule and tombstones/revises the grade.

Solution to Exercise 4.30

(exercise on page 150)

Record skill ID/version/content hash, agent/build/principal, task and stratum descriptors, randomized assignment or selection propensity, exact context-graft receipt, model/environment/tool versions, outcome/evidence references, quality vector, costs, and evaluator identities. These fields verify exposure and outcome association. "This skill helped" is causal and cannot be verified from a graft-success pair alone; estimate it with randomized skill-on/off assignment on held-out comparable tasks (or label an observational matched estimate honestly), reporting effect size and uncertainty.

Solution to Exercise 4.31

(exercise on page 150)

Publish the vector, per-axis uncertainty, sample sizes, cohort/task distribution, freshness, judge/evidence provenance, and known correlations. Let each buyer apply a local, preferably non-public weighting and hard minimums; do not mint a canonical platform scalar. Agents can still optimize market-demanded axes, so rotate held-out tests and audit cross-axis regressions. Plural preference does not eliminate Goodhart, but it prevents one universal target from becoming the asset itself.

Solution to Exercise 4.32

(exercise on page 153)

Settlement incentives are conditional on the signal distribution. "The daemon derives the grade" only identifies the messenger; if a strategic judge chooses the input, the distribution depends on omitted actions/payoffs and the equilibrium proof assumes its own conclusion.

Solution to Exercise 4.33

(exercise on page 153)

Judge J posts bond B ; its grade and the bond lock are recorded; an independently selected re-auditor overturns the dishonest grade against preserved evidence; the system appends a correction/tombstone, recomputes any explicitly appealable settlement, slashes B to the harmed/commons bucket, and lowers J 's judging reputation. It must not silently rewrite the original event.

Solution to Exercise 4.34

(exercise on page 153)

With one-shot corrupt gain at most G , re-audit probability ρ , conditional detection probability d , and slashable bond B , local deterrence requires $\rho dB > G$, i.e. $\rho^* = G/(dB)$. At $G_0 = 400, d = 0.8, B = 50, \rho = 0.25$: $\rho d = 0.2$, so $\lambda = 1 - \rho d = 0.8$; with $C = 8$ the threshold $G_k > CB = 400$ is unmet at level 0, giving $\lceil \log 400 / \log 1.25 \rceil = 27$ levels; with $C = 1$ the threshold is only $G_k > 50$, so bribery bleeds linearly for 35 levels before 18 more geometric ones close it, 53 total.

Solution to Exercise 4.35

(exercise on page 153)

Admit arbiters by domain competence and conflict graph; assign cases randomly/blindly; require bonds and service-level commitments; preserve evidence; pay for timely completed work rather than agreement; sample re-audits and publish calibration/overtake rates; use plural panels on high stakes. Price queue capacity and cap workload so rubber-stamping is not the profitable response to overload. Honest convergence still needs a protected mechanical outcome, a delayed real-world outcome, or a final human/contractual authority — a market alone does not manufacture truth.

Solution to Exercise 4.36

(exercise on page 154)

The script prints “flat: 50.00 (Theta(T))”, “model A: 25.58 (Theta(log T); $a \cdot G / (d \cdot v) \cdot \ln(1 + vT/B) = 25.50$)”, and “model B: 35.08 (0(1); closed form $aG^2 / (2dBrv) = 41.67$; hits 0 at $t^* = 333$)”. By hand: the flat baseline is a constant audit rate ρ^* times a constant per-audit cost a summed over T periods, so $\rho^* \cdot a \cdot T = 0.25 \times 1 \times 200 = 50$, exactly the printed 50.00 — the two amortization models are cheaper only because they let ρ_t fall below ρ^* as accumulated stake or surfaced history does part of the deterring.

Solution to Exercise 4.37

(exercise on page 156)

An append-only tombstone preserves the original claim, the later correction, their authors, timing, and the reliance window. Delete destroys evidence, permits history rewriting, and makes replicas unable to distinguish retraction from omission.

Solution to Exercise 4.38

(exercise on page 156)

Append a signed tombstone containing the target outcome ID/hash, reason/evidence, authority, revision, and effective time. Propagate it and recompute reputation as a projection that excludes/nullifies that outcome. It was spendable from original publication until each relying harbor received and enforced the tombstone or failed closed; under an unbounded partition that window is unbounded.

Solution to Exercise 4.39

(exercise on page 156)

Use a signed revision DAG, per-harbor import receipts/watermarks, periodic accumulator/Merkle roots for active outcomes and tombstones, and a freshness rule that refuses high-risk use from stale replicas. Under connected eventual delivery, convergence is shown when every member acknowledges a root containing the tombstone; sampling can audit inclusion. Bounded local memory may retain compact accumulators and content-addressed archive pointers, not erase correction commitments. No finite convergence/spend bound exists during an unbounded partition.

Appendices

A Implementation boundary

The book’s argument is broader than the running product. This appendix keeps the two from borrowing authority from one another. A mechanism receives the grade of its weakest necessary part; a verified library does not make every caller safe, and an available route does not prove that every effect passes through it.

A.1 Five assurance modes

Observed	The act may occur elsewhere; the system retains best-effort evidence after the fact.
Coordinated	Cooperative clients register claims, roles, commitments, and messages before acting.
Brokered	A selected credential or effect channel passes through a daemon-owned broker.
Confined	The process is restricted by operating-system primitives and cannot reach the protected effect through an ambient path.
Attested	A measured environment conditions key release or acceptance on a verifiable runtime identity.

These modes are not a maturity staircase. They answer different questions about which channels the institution can see and control.

A.2 What exists now

Local transaction authority — partial. The daemon runs a single-writer SQLite/WAL path for sessions, claims, messaging, commitments, and selected coordination state. Regression tests exercise those paths. Durability remains typed by failure class, schema sequencing is not the strongest design described in *The Single-Writer Kernel*, and filesystem or network effects outside mediated paths remain outside the transaction.

Legible projections — partial. The product derives operator projections from event and coordination records and retains routes back to underlying artifacts. No implementation can eliminate the information floor proved in *The Legible Swarm*; coverage, omission, and review latency must still be measured for each surface.

Durable work and obligation records — partial. Commitments, deadlines, messages, outcomes, and recovery evidence persist beyond one process. A universally portable execution capsule and provider-independent semantic restoration remain SPECIFIED.

Authentication and attenuation — closed for the protocol model, partial for the code. ProVerif closes the recorded proof-of-possession and every-hop attenuation queries at the model boundary. The Kani harnesses over the deployed Rust crate are bounded: no panic in the parser on 32-byte inputs with the cryptography stubbed, no panic in the comparator, and two concrete vectors for the subset check. Runtime callers still owe conformance: the verified primitive must be the primitive that every protected effect actually checks.

Escrow and settlement — mixed. Ledger and conservation slices are PARTIAL. Cross-authority custody, adjudication, and 2-of-3 settlement remain SPECIFIED and conditional on a non-bypassable custody interface.

Relay federation — partial. The current Relay carries authenticated events over outbound HTTPS and SSE. It does not replicate daemon databases, impose global order, authorize local effects, supply end-to-end secrecy by itself, or establish the witness-log and settlement institutions described in *The Federated Harbor*.

B Result atlas

The research archive contains revisions, experiments, literature work, and failed formulations. The durable unit is therefore a result, not a PDF. Each result below has a stable identifier, assumptions, a boundary, a chapter home, and a named verification artifact in the companion manifest.

B.1 The seven propositions and their evidence

1. Prevention begins at the effect boundary (R5, R8) — *The Single-Writer Kernel*. Controllability separates rules that a pre-commit gate can enforce from failures that can only be observed and remedied. The airlock experiments show why complete channel mediation is the decisive assumption. A token, label, or policy document does not control an effect channel by itself.

2. Delegation only narrows (ANCHOR-ATTENUATION, R5) — *The Anchor Protocol*. Every hop must authenticate the issuer and reject a capability that expands scope or lifetime. The cryptographic models close the stated monotonicity property; the engineering obligation is universal use at protected channels.

3. Supervision has an information cost (R1–R4, R14, R16) — *The Legible Swarm*. The information floor prices guaranteed recall; the dual-reader result shows why one ordering cannot generally serve both successor selection and operator attention; the regret rule derives intervention from asymmetric error cost; adaptive zoom and context paging describe when attention can be spent later rather than summarized away now. The bounds price guarantees under their stated event and flag models. They do not certify an arbitrary dashboard.

4. Accountability outlives the process (R12, R13, B6) — *From Spawn to Person*. No-mint transfer semantics prevent skill or identity replacement from creating new authority. Budget-pooling and exposure arguments make reset laundering a property of the principal’s case file, not the process name. The research killed the stronger closure-weight formulation before restating the result correctly.

5. Exchange separates payment, collateral, evidence, and closure (R7, R13–R15) — *The Harbor Economy*. Conservation supplies the accounting invariant; audit sampling and probation results give conditions under which dishonesty is more expensive than honest performance. These are conditional economic statements. Detection probability, one-shot gain, collateral, judge independence, and custody must be measured or contracted rather than implied.

6. Uncertainty needs an institution (R7, R8, R10, R11, R15, R17) — *The Bonded Commons*. Atomic ledgers preserve conservation and privacy budgets; canary and sequential tests price detection power and delay; randomized, diverse adjudication reduces predictable collusion only under the named sampling assumptions. Evidence may support a remedy without becoming mechanical truth.

7. Federation relays evidence, not sovereignty (R6, R6-CR, RELAY-V0) — *The Federated Harbor*. Signed roots and redundant comparison paths can expose equivocation across a specified topology. The counterexample R6-CR rejects the tempting claim that a zero local residual proves global consistency. Current Relay establishes event transport and authentication; it does not close witness, secrecy, liveness, custody, or local authorization obligations.

B.2 A result is admitted only with its limit

For this volume, a claim is incomplete unless it states all five of the following: the object it concerns, the assumptions under which it holds, the kind of evidence offered, a boundary or falsifier, and the artifact by which a reader can recompute or inspect it. The machine-readable manifest is the publication ledger for that discipline. New mechanisms belong in the book only after they can name the result they instantiate and the obligation they leave unresolved.

C Notation and cross-chapter concordance

Work unit	The durable case file joining authorization, capability history, evidence, obligations, outcomes, and settlement.
Harbor	One local administrative and transactional authority, normally one daemon and its durable database.
Principal	The human or organization whose policy and resources an agent ultimately exercises; not interchangeable with a process identity.
Role	A bundle of obligations, capabilities, and authority that a spawn may fill.
Person	A role instance plus continuity and a witnessed outcome history bound to an identity that cannot be freely re-picked.
Oracle	An external or independently checkable reference used to close a commitment or adjudicate an outcome. Its trust model must be named.
Round	A model step in a dissemination or protocol analysis, not automatically a wall-clock deadline.
Closed	The exact recorded property has been mechanically discharged. No whole-system implication follows without a conformance argument.

References

- [1] A. Demers et al. “Epidemic Algorithms for Replicated Database Maintenance.” *PODC* 1987. Cited on pages 180 and 188.
- [2] A. K. Erlang. Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges. *Elektroteknikeren*, 13, 1917. (The Erlang-C delay formula.) Cited on page 91.
- [3] Adi Shamir. How to Share a Secret. *Communications of the ACM*, 22(11):612–613, 1979. Cited on page 230.
- [4] Agent Payments Protocol (AP2): Mandates. Verifiable-credential authorization chain: SD-JWT+kb checkout mandate, JWS detached-content merchant authorization, JCS canonicalization, 2026. <https://ap2-protocol.org>. Cited on page 228.
- [5] Agent Payments Protocol (AP2): Mandates. Verifiable-credential authorization chain: SD-JWT+kb, JWS detached content, JCS canonicalization, 2026. <https://ap2-protocol.org>. Cited on pages 76, 141, and 260.
- [6] Agent Payments Protocol (AP2): Mandates. Verifiable-credential authorization chain (SD-JWT+kb checkout mandate, JWS detached-content merchant authorization, JCS canonicalization), 2026. <https://ap2-protocol.org>; UCP extension at ucp.dev/documentation/ucp-and-ap2/. Cited on pages 188 and 189.
- [7] Agentic Commerce Protocol. OpenAI and Stripe, 2025. <https://agenticcommerce.dev>. Cited on page 228.
- [8] Agentic Commerce Protocol. OpenAI & Stripe, 2025. <https://agenticcommerce.dev>. Cited on pages 188 and 189.
- [9] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. Epidemic Algorithms for Replicated Database Maintenance. In *ACM Symposium on Principles of Distributed Computing (PODC)*, 1987. Cited on pages 54, 69, 248, 249, 249, 252, and 263.
- [10] Alan Demers et al. Epidemic Algorithms for Replicated Database Maintenance. In *ACM PODC*, 1987. Cited on page 237.
- [11] Albert O. Hirschman. *Exit, Voice, and Loyalty: Responses to Decline in Firms, Organizations, and States*. Harvard University Press, 1970. Cited on pages 85, 111, and 112.
- [12] Alfarez Abdul-Rahman. The PGP Trust Model. *EDI-Forum: The Journal of Electronic Commerce*, 10(3):27–31, 1997. Cited on page 259.
- [13] Amartya Sen. The Impossibility of a Paretian Liberal. *Journal of Political Economy*, 78(1):152–157, 1970. <https://doi.org/10.1086/259614> Cited on pages 204 and 227.
- [14] Anand S. Rao and Michael P. Georgeff. Modeling Rational Agents within a BDI-Architecture. In *International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 473–484, 1991. Cited on page 227.
- [15] Andrew J. I. Jones and Marek Sergot. On the Characterisation of Law and Computer Systems: The Normative Systems Perspective. In *Deontic Logic in Computer Science*, Wiley, 1993. Cited on pages 17 and 43.
- [16] Andrew J. I. Jones and Marek Sergot. On the Characterisation of Law and Computer Systems: The Normative Systems Perspective. In *Deontic Logic in Computer Science: Normative System Specification*, pages 275–307. John Wiley and Sons, 1993. Cited on page 202.

- [17] Andrew Tobin and Drummond Reed. The Inevitable Rise of Self-Sovereign Identity. Sovrin Foundation White Paper, 2017. Cited on page 241.
- [18] Anne M. Treisman and Garry Gelade. A Feature-Integration Theory of Attention. *Cognitive Psychology*, 12(1):97–136, 1980. Cited on pages 105 and 112.
- [19] ANSI/ISA. *ANSI/ISA-18.2: Management of Alarm Systems for the Process Industries*. 2016. Cited on pages 93 and 112.
- [20] Anthropic. Effective Context Engineering for AI Agents. 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents> Cited on pages 106, 106, and 113.
- [21] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. Cited on page 260.
- [22] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. <https://doi.org/10.14722/ndss.2014.23212> Cited on pages 52, 60, 66, 66, 67, and 77.
- [23] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. Cited on page 228.
- [24] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. (Attenuation-only bearer credentials with third-party caveats — the enforcement-gate construction.) Cited on page 38.
- [25] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. Arco Publishing, 1978. Cited on pages 124, 125, and 142.
- [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *NeurIPS*, 2017. arXiv:1706.03762. Cited on page 106.
- [27] Avery Pennarun et al. How Tailscale Works. Tailscale Engineering Blog, March 2020. Cited on page 259.
- [28] AWS Automated Reasoning Group. Kani Rust Verifier. <https://github.com/model-checking/kani>, 2022. Cited on pages 59, 66, and 67.
- [29] AWS s2n-tls. An Implementation of the TLS/SSL Protocols. <https://github.com/aws/s2n-tls>, 2022. Cited on page 56.
- [30] B. Fong & D. Spivak. *Seven Sketches in Compositionality: An Invitation to Applied Category Theory*. Cambridge University Press, 2019. Cited on pages 186 and 188.
- [31] B. Laurie, A. Langley & E. Kasper. “Certificate Transparency.” RFC 6962, IETF, 2013. Cited on page 188.
- [32] B. Laurie. “Certificate Transparency.” *Communications of the ACM* 57(10):40–46, 2014. Cited on pages 178, 188, and 189.

- [33] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate Transparency. IETF RFC 6962, June 2013. Cited on pages 244, 248, 259, and 260.
- [34] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate Transparency. RFC 6962, IETF, 2013. Cited on pages 203 and 229.
- [35] Bertrand Meyer. Applying “Design by Contract”. *IEEE Computer*, 25(10):40–51, 1992. Cited on page 92.
- [36] Billy Bob Brumley and Nicola Tuveri. Remote Timing Attacks are Still Practical. In *European Symposium on Research in Computer Security (ESORICS)*, 2011. Cited on page 66.
- [37] Bin Fan, David G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. In *ACM CoNEXT*, 2014. Cited on pages 53 and 237.
- [38] Bruno Blanchet, Vincent Cheval, and Véronique Cortier. ProVerif with Lemmas, Induction, Fast Subsumption, and Much More. In *IEEE Symposium on Security and Privacy (S&P)*, 2022. Cited on page 76.
- [39] Bruno Blanchet. CryptoVerif: A Computationally Sound Security Protocol Verifier. INRIA Research Report, 2007. Cited on page 76.
- [40] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016. Cited on pages 201 and 257.
- [41] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016. <https://doi.org/10.1561/33000000004> Cited on pages 57, 59, 65, 66, 67, and 76.
- [42] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016. (The class of tool in which the authorization chain’s secrecy and integrity properties are mechanically verified.) Cited on page 43.
- [43] Butler W. Lampson. Protection. *ACM SIGOPS Operating Systems Review*, 8(1):18–24, 1974. Cited on pages 4 and 42.
- [44] C. Chiu, S. Zhang & M. van der Schaar. “Strategic Self-Improvement for Competitive Agents in AI Labour Markets.” arXiv:2512.04988, 2025. Cited on pages 168 and 188.
- [45] C. Dwork, N. Lynch & L. Stockmeyer. “Consensus in the Presence of Partial Synchrony.” *Journal of the ACM* 35(2):288–323, 1988. Cited on pages 178 and 188.
- [46] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. Cited on pages 43 and 207.
- [47] Carl Ellison, Bill Frantz, Butler Lampson, Ron Rivest, Brian Thomas, and Tatu Ylonen. SPKI Certificate Theory. RFC 2693, 1999. Cited on page 197.
- [48] Carole Pateman. *The Problem of Political Obligation: A Critique of Liberal Theory*. Wiley, 1979. Cited on pages 111 and 112.
- [49] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. A Comprehensive Symbolic Analysis of TLS 1.3. In *ACM Conference on Computer and Communications Security (CCS)*, 2017. Cited on page 76.

- [50] Charles Wilson. A Model of Insurance Markets with Incomplete Information. *Journal of Economic Theory*, 16(2):167–207, 1977. Cited on page 217.
- [51] Christopher D. Wickens. Multiple Resources and Performance Prediction. *Theoretical Issues in Ergonomics Science*, 3(2):159–177, 2002. Cited on pages 94 and 112.
- [52] Christopher Goes. The Interblockchain Communication Protocol: An Overview. arXiv:2006.15918, 2020. Cited on page 260.
- [53] CVE-2015-9235. Algorithm Confusion in jsonwebtoken Node.js library. NIST National Vulnerability Database, 2015. Cited on pages 51, 59, and 77.
- [54] CVE-2018-1000531. JWT “none” algorithm vulnerability in jwt library. NIST National Vulnerability Database, 2018. Cited on page 77.
- [55] CVE-2023-48238. Algorithm confusion in json-web-token library. NIST National Vulnerability Database, 2023. Cited on pages 51 and 77.
- [56] D. Abreu, D. Pearce & E. Stacchetti. “Toward a Theory of Discounted Repeated Games with Imperfect Monitoring.” *Econometrica* 58(5):1041–1063, 1990. (With Green & Porter, *Econometrica* 1984.) Cited on pages 183, 188, and 189.
- [57] D. Richard Hipp et al. SQLite Write-Ahead Logging. SQLite Documentation, 2010. <https://www.sqlite.org/wal.html> (`synchronous=NORMAL` “commits might lose durability following a power loss.”) Cited on pages 10 and 43.
- [58] D. Spivak & R. Kent. “Ologs: A Categorical Framework for Knowledge Representation.” *PLoS ONE* 7(1):e24274, 2012. Cited on page 188.
- [59] Daniel D. Sleator and Robert E. Tarjan. Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28(2):202–208, 1985. Cited on page 108.
- [60] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012. Cited on pages 52, 60, and 244.
- [61] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012. (Ed25519 — the signature scheme underlying the authorization chain.) Cited on page 43.
- [62] Danny Dolev and Andrew Yao. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983. Cited on pages 57, 65, 76, and 244.
- [63] DataDome. “Agent Trust Management and the Universal Commerce Protocol,” 2026. datadome.co/agent-trust-management/universal-commerce-protocol/. Cited on page 188.
- [64] DataDome. Agent Trust Management and the Universal Commerce Protocol, 2026. datadome.co/agent-trust-management/universal-commerce-protocol/. Cited on page 228.
- [65] David Basin, Jannik Dreier, Lucca Hirschi, Saša Radomirovic, Ralf Sasse, and Vincent Stettler. A Formal Analysis of 5G Authentication. In *ACM Conference on Computer and Communications Security (CCS)*, 2018. Cited on page 76.
- [66] David G. Kendall. Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded Markov chain. *Annals of Mathematical Statistics*, 24(3):338–354, 1953. (The $A/S/c$ notation.) Cited on page 90.

- [67] David Gelernter. Generative Communication in Linda. *ACM Transactions on Programming Languages and Systems*, 7(1):80–112, 1985. Cited on pages 15 and 43.
- [68] David Hume. Of the Original Contract. 1748. Cited on pages 85 and 112.
- [69] David M. Green and John A. Swets. *Signal Detection Theory and Psychophysics*. Wiley, 1966. Cited on pages 94 and 112.
- [70] Derek Parfit. *Reasons and Persons*. Oxford University Press, 1984. Cited on pages 124 and 128.
- [71] Diego Ongaro and John Ousterhout. In Search of an Understandable Consensus Algorithm (Raft). In *USENIX ATC*, 2014. Cited on page 83.
- [72] Diego Ongaro and John Ousterhout. In Search of an Understandable Consensus Algorithm. In *USENIX Annual Technical Conference (ATC)*, pages 305–320, 2014. Cited on page 230.
- [73] Donald T. Campbell. Assessing the Impact of Planned Social Change. *Evaluation and Program Planning*, 2(1):67–90, 1979. Cited on pages 96 and 112.
- [74] E. Heilman, A. Kendler, A. Zohar & S. Goldberg. “Eclipse Attacks on Bitcoin’s Peer-to-Peer Network.” *USENIX Security* 2015. Cited on pages 181 and 188.
- [75] E. Ostrom. *Governing the Commons*. Cambridge University Press, 1990. Cited on page 188.
- [76] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. Cited on pages 110 and 113.
- [77] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. Cited on pages 156, 197, and 227.
- [78] Eric J. Friedman and Paul Resnick. The Social Cost of Cheap Pseudonyms. *Journal of Economics & Management Strategy*, 10(2):173–199, 2001. Cited on pages 125 and 135.
- [79] Erich Owens. The Anchor Protocol: A Mechanically Analyzed Control Plane for Local Agent Swarms. Curiositech LLC Technical White Paper, May 2026. Cited on pages 240 and 259.
- [80] Erich Owens. The Anchor Protocol: A Mechanically Analyzed Control Plane for Local Agent Swarms. Technical White Paper, Curiositech LLC, May 2026. Cited on pages 196, 200, 200, 201, 201, 211, 211, 212, and 228.
- [81] Erich Owens. The Bonded Commons: Pre-Transactional Trust Infrastructure for Multi-Agent Systems. Curiositech LLC Technical White Paper, May 2026. Cited on pages 240, 242, 260, and 263.
- [82] Erich Owens. The Cohomology of Equivocation: Detecting Split-View Lies in Federated Witness-Log Gossip by Sheaf Consistency. The Harbor Research Program, Paper 7, August 2026. Cited on page 250.
- [83] Erich Owens. The Federated Harbor: Proposal and Annotated Bibliography. Curiositech LLC working document, May 2026. Cited on pages 242, 250, 261, and 263.
- [84] Erich Owens. *The Price of a Summary: Information-Theoretic Limits of Agent Oversight*. The Harbor Research Program, Paper 1, 2026. Companion formal paper to this chapter; its Theorems 1–4 are the proved versions of several results stated here. Cited on pages 100, 100, 103, 103, 107, and 113.
- [85] Erich Owens. *What Needs an Authority: Mechanical Detection, Chartered Resolution, and the Exact Price of Sole Ownership*. The Harbor Research Program, Paper 6, 2026. Its Theorem 2 is the Erlang-C specialization boundary, together with the sweep that falsified this paper’s earlier proposed threshold. Cited on pages 91, 113, and 114.

- [86] Ethan Buchman. Tendermint: Byzantine Fault Tolerance in the Age of Blockchains. M.A.Sc. thesis, University of Guelph, 2016. Cited on page 196.
- [87] Ethereum Foundation. Ethereum 2.0 Phase 0 – The Beacon Chain Specification. <https://github.com/ethereum/consensus-specs>, 2020. Cited on page 196.
- [88] Eve Maler and Drummond Reed. The Venn of Identity: Options and Issues in Federated Identity Management. *IEEE Security & Privacy*, 6(2):16–23, 2008. Cited on pages 245, 255, 255, 259, 259, 259, and 261.
- [89] Feng Lin and W. Murray Wonham. On observability of discrete-event systems. *Information Sciences*, 44(3):173–198, 1988. Cited on page 24.
- [90] Foundation for Intelligent Physical Agents. *FIPA Agent Management Specification* (SC00023). 2002. <http://www.fipa.org/specs/fipa00023/> Cited on pages 101 and 113.
- [91] Fred B. Schneider. Enforceable Security Policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000. Cited on pages 18, 23, 43, 93, and 117.
- [92] Friedrich A. Hayek. The Use of Knowledge in Society. *American Economic Review*, 35(4):519–530, 1945. Cited on pages 111 and 112.
- [93] G. Akerlof. “The Market for ‘Lemons’: Quality Uncertainty and the Market Mechanism.” *Quarterly Journal of Economics* 84(3):488–500, 1970. Cited on pages 171 and 188.
- [94] G. Becker. “Crime and Punishment: An Economic Approach.” *Journal of Political Economy* 76(2):169–217, 1968. Cited on pages 189 and 190.
- [95] G. Irving, P. Christiano & D. Amodei. “AI Safety via Debate.” arXiv:1805.00899, 2018. Cited on pages 189, 189, and 189.
- [96] Gavin Lowe. A Hierarchy of Authentication Specifications. In *IEEE Computer Security Foundations Workshop (CSFW)*, 1997. Cited on pages 59, 66, and 76.
- [97] George A. Akerlof. The Market for “Lemons”: Quality Uncertainty and the Market Mechanism. *Quarterly Journal of Economics*, 84(3):488–500, 1970. Cited on page 125.
- [98] George A. Miller. The Magical Number Seven, Plus or Minus Two. *Psychological Review*, 63(2):81–97, 1956. Cited on pages 94 and 112.
- [99] Gordon Dai, Weijia Zhang, Jinhan Li, Siqi Yang, Chidera Onochie Ibe, Srihas Rao, Arthur Caetano, and Misha Sra. Artificial Leviathan: Exploring Social Evolution of LLM Agents Through the Lens of Hobbesian Social Contract Theory. arXiv:2406.14373, 2024. <https://arxiv.org/abs/2406.14373> Cited on pages 81, 83, and 112.
- [100] Guy Theraulaz and Eric Bonabeau. A Brief History of Stigmergy. *Artificial Life*, 5(2):97–116, 1999. Cited on page 203.
- [101] H. Garcia-Molina & K. Salem. “Sagas.” *ACM SIGMOD* 1987. Cited on pages 182, 188, and 189.
- [102] HashiCorp. Consul Service Discovery and DNS Interface (health-checked, TTL-expiring registry). <https://developer.hashicorp.com/consul/docs/discover> Cited on page 105.
- [103] Hector Garcia-Molina and Kenneth Salem. Sagas. In *ACM SIGMOD International Conference on Management of Data*, 1987. Cited on pages 37, 37, 43, and 154.
- [104] Hector Garcia-Molina and Kenneth Salem. Sagas. In *ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987. Cited on page 227.

- [105] *Hobbes's Moral and Political Philosophy*. Stanford Encyclopedia of Philosophy. <https://plato.stanford.edu/entries/hobbes-moral/> Cited on pages 83, 84, and 112.
- [106] J. Douceur. “The Sybil Attack.” *IPTPS* 2002. Cited on pages 174 and 188.
- [107] J.-C. Rochet & J. Tirole. “Platform Competition in Two-Sided Markets.” *Journal of the European Economic Association* 1(4):990–1029, 2003. Cited on pages 166, 188, and 189.
- [108] J.-C. Rochet & J. Tirole. “Two-Sided Markets: A Progress Report.” *RAND Journal of Economics* 37(3):645–667, 2006. Cited on pages 166 and 188.
- [109] Jack B. Dennis and Earl C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966. Cited on pages 50, 77, 127, 197, 228, and 260.
- [110] Jakob Hansen and Robert Ghrist. Toward a spectral theory of cellular sheaves. *Journal of Applied and Computational Topology*, 3(4):315–358, 2019. Cited on page 252.
- [111] James C. Scott. *Seeing Like a State: How Certain Schemes to Improve the Human Condition Have Failed*. Yale University Press, 1998. Cited on pages 86, 86, and 112.
- [112] James C. Scott. *Two Cheers for Anarchism*. Princeton University Press, 2012. Cited on pages 111 and 112.
- [113] James P. Anderson. Computer Security Technology Planning Study. ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972. (The origin of the *reference monitor* concept.) Cited on pages 4 and 42.
- [114] Jean-Charles Rochet and Jean Tirole. Platform Competition in Two-Sided Markets. *Journal of the European Economic Association*, 1(4):990–1029, 2003. Cited on page 142.
- [115] Jeffrey O. Kephart and David M. Chess. The Vision of Autonomic Computing. *IEEE Computer*, 36(1):41–50, 2003. Cited on page 43.
- [116] Jerome H. Saltzer and Michael D. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. Cited on pages 4, 8, 42, and 50.
- [117] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992. (Atomicity/consistency vs. durability — the ground for I1a/I1b.) Cited on pages 9 and 43.
- [118] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993. Cited on page 207.
- [119] Joel S. Warm, Raja Parasuraman, and Gerald Matthews. Vigilance Requires Hard Mental Work and Is Stressful. *Human Factors*, 50(3):433–441, 2008. Cited on pages 96 and 112.
- [120] John D. Lee and Katrina A. See. Trust in Automation: Designing for Appropriate Reliance. *Human Factors*, 46(1):50–80, 2004. Cited on pages 96 and 112.
- [121] John Locke. An Essay Concerning Human Understanding. 1689. (Bk II, ch. xxvii, “Of Identity and Diversity.”) Cited on pages 124 and 128.
- [122] John Locke. *Second Treatise of Government*, §§95–122. 1689. Cited on pages 111 and 112.
- [123] John Locke. *Two Treatises of Government*. Awnsham Churchill, London, 1689. Cited on page 227.

- [124] John R. Douceur. The Sybil Attack. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002. Cited on pages 134, 134, 135, 244, and 255.
- [125] John R. Douceur. The Sybil Attack. In *IPTPS*, 2002. Cited on pages 109 and 113.
- [126] Jonathan Hickman. *House of X / Powers of X*. Marvel Comics, 2019. Cited on page 208.
- [127] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023. Cited on pages 131 and 228.
- [128] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *UIST*, 2023. arXiv:2304.03442. Cited on pages 81 and 112.
- [129] Juan Brackeva et al. Headscale: Open-Source Tailscale Coordination Server. GitHub project, 2020–present. Cited on page 259.
- [130] Karthikeyan Bhargavan, Barry Bond, Antoine Delignat-Lavaud, Cédric Fournet, Chris Hawblitzel, et al. Everest: Towards a Verified, Drop-in Replacement of HTTPS. In *Logic in Computer Science (LICS)*, 2017. Cited on page 56.
- [131] Kevin Werbach. *The Blockchain and the New Architecture of Trust*. MIT Press, 2018. Cited on page 197.
- [132] L. Zheng et al. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.” *NeurIPS* 2023. Cited on pages 183 and 188.
- [133] Leslie Lamport. *Specifying Systems: The TLA⁺ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. Cited on page 224.
- [134] Leslie Lamport. The Part-Time Parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998. Cited on page 231.
- [135] Leslie Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7):558–565, 1978. Cited on page 41.
- [136] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks*, 2023. Cited on pages 125 and 147.
- [137] Linux Foundation / OpenSSF. SLSA Specification v1.0: Supply-chain Levels for Software Artifacts, 2023. Cited on pages 259 and 260.
- [138] Lisanne Bainbridge. Ironies of Automation. *Automatica*, 19(6):775–779, 1983. Cited on pages 97 and 112.
- [139] M. Armstrong. “Competition in Two-Sided Markets.” *RAND Journal of Economics* 37(3):668–691, 2006. Cited on page 188.
- [140] M. Fischer, N. Lynch & M. Paterson. “Impossibility of Distributed Consensus with One Faulty Process.” *Journal of the ACM* 32(2):374–382, 1985. Cited on pages 178 and 188.
- [141] M. Herlihy, B. Liskov & L. Shrira. “Cross-Chain Deals and Adversarial Commerce.” *The VLDB Journal* 31:1291–1309, 2022. Cited on pages 179, 188, and 189.
- [142] M. Herlihy. “Atomic Cross-Chain Swaps.” *PODC* 2018. Cited on pages 179, 188, and 189.

- [143] Manuel Barbosa, Gilles Barthe, Karthikeyan Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. SoK: Computer-Aided Cryptography. In *IEEE Symposium on Security and Privacy (S&P)*, 2021. Cited on page 76.
- [144] Marco Dorigo and Gianni Di Caro. The Ant Colony Optimization Meta-Heuristic. In *New Ideas in Optimization*, McGraw-Hill, 1999. Cited on pages 15 and 43.
- [145] Maria Cvach. Monitor Alarm Fatigue: An Integrative Review. *Biomedical Instrumentation & Technology*, 46(4):268–277, 2012. Cited on pages 93 and 112.
- [146] Marilyn Strathern. ‘Improving Ratings’: Audit in the British University System. *European Review*, 5(3):305–321, 1997. (Canonical statement of Goodhart’s law.) Cited on page 143.
- [147] Marilyn Strathern. “Improving Ratings”: Audit in the British University System. *European Review*, 5(3):305–321, 1997. (Goodhart’s law.) Cited on pages 104 and 112.
- [148] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Ph.D. dissertation, Johns Hopkins University, 2006. Cited on page 197.
- [149] Mark S. Miller. Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control. PhD thesis, Johns Hopkins University, 2006. Cited on page 260.
- [150] Martin Kleppmann. *Designing Data-Intensive Applications*. O’Reilly, 2017. (Single-leader as the right default; consensus only when forced.) Cited on pages 5 and 43.
- [151] Maurice Herlihy. Atomic Cross-Chain Swaps. In *ACM Symposium on Principles of Distributed Computing (PODC)*, 2018. Cited on pages 241, 253, 260, and 261.
- [152] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990. Cited on pages 35 and 43.
- [153] Mica R. Endsley and Esin O. Kiris. The Out-of-the-Loop Performance Problem and Level of Control in Automation. *Human Factors*, 37(2):381–394, 1995. Cited on pages 97 and 112.
- [154] Mica R. Endsley. Toward a Theory of Situation Awareness in Dynamic Systems. *Human Factors*, 37(1):32–64, 1995. Cited on pages 97 and 112.
- [155] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of Distributed Consensus with One Faulty Process. *Journal of the ACM*, 32(2):374–382, 1985. Cited on pages 4 and 43.
- [156] Michael Rothschild and Joseph Stiglitz. Equilibrium in Competitive Insurance Markets: An Essay on the Economics of Imperfect Information. *Quarterly Journal of Economics*, 90(4):629–649, 1976. Cited on page 217.
- [157] Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software*. 2nd ed., Pragmatic Bookshelf, 2018. (The bounded busy-wait as a bulkhead / timeout-budget under contention.) Cited on page 43.
- [158] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. In *NeurIPS*, 2023. arXiv:2305.04388. Cited on page 88.
- [159] N. Nisan, T. Roughgarden, É. Tardos & V. Vazirani (eds.). *Algorithmic Game Theory*. Cambridge University Press, 2007. Cited on page 188.

- [160] Nadim Kobeissi, Karthikeyan Bhargavan, and Bruno Blanchet. Automated Verification for Secure Messaging Protocols and Their Implementations: A Symbolic and Computational Approach. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2017. Cited on page 76.
- [161] Neal E. Young. On-line file caching. *Algorithmica*, 33(3):371–383, 2002. Cited on page 108.
- [162] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts. *TACL*, 12:157–173, 2024. arXiv:2307.03172. Cited on pages 106 and 113.
- [163] Noam Nisan, Tim Roughgarden, Éva Tardos, and Vijay V. Vazirani (eds.). *Algorithmic Game Theory*. Cambridge University Press, 2007. Cited on page 109.
- [164] Noam Nisan, Tim Roughgarden, Éva Tardos, and Vijay V. Vazirani (eds.). *Algorithmic Game Theory*. Cambridge University Press, 2007. Cited on pages 125 and 145.
- [165] Norman H. Mackworth. The Breakdown of Vigilance During Prolonged Visual Search. *Quarterly Journal of Experimental Psychology*, 1(1):6–21, 1948. Cited on pages 96 and 112.
- [166] P. Resnick, R. Zeckhauser, J. Swanson & K. Lockwood. “The Value of Reputation on eBay: A Controlled Experiment.” *Experimental Economics* 9(2):79–101, 2006. Cited on page 188.
- [167] Paul Resnick, Richard Zeckhauser, John Swanson, and Kate Lockwood. The Value of Reputation on eBay: A Controlled Experiment. *Experimental Economics*, 9(2):79–101, 2006. Cited on pages 125 and 142.
- [168] Peter J. Ramadge and W. Murray Wonham. Supervisory Control of a Class of Discrete Event Processes. *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987. (Controllability and the supremal controllable sublanguage — the general boundary of which this chapter’s Synchronous State Decidability theorem is the commit-only instance.) Cited on pages 21, 21, and 45.
- [169] Peter J. Ramadge and W. Murray Wonham. The Control of Discrete Event Systems. *Proceedings of the IEEE*, 77(1):81–98, 1989. Cited on page 45.
- [170] Peter Pirolli and Stuart Card. Information Foraging. *Psychological Review*, 106(4):643–675, 1999. Cited on page 113.
- [171] Philip R. Cohen and Hector J. Levesque. Intention Is Choice with Commitment. *Artificial Intelligence*, 42(2–3):213–261, 1990. Cited on pages 26 and 43.
- [172] Philip R. Zimmermann. The Official PGP User’s Guide. MIT Press, 1995. Cited on page 255.
- [173] Philipp Schindler, Aljosha Judmayer, Nicholas Stifter, and Edgar Weippl. ETHDKG: Distributed Key Generation with Ethereum Smart Contracts. IACR ePrint 2019/985. Cited on page 241.
- [174] Pierre-Paul Grassé. La reconstruction du nid et les coordinations interindividuelles... La théorie de la stigmergie. *Insectes Sociaux*, 6:41–80, 1959. Cited on page 108.
- [175] Pierre-Paul Grassé. La reconstruction du nid et les coordinations interindividuelles... : la théorie de la stigmergie. *Insectes Sociaux*, 6(1):41–80, 1959. Cited on pages 15 and 43.
- [176] Port Daddy Harbor research program. *Continuity Without Metaphysics*. Companion formal paper (Paper 5), August 2026. Source of the no-mint, resurrection-soundness, and probation-cliff theorems restated in the sections on continuity and identity. Cited on page 137.

- [177] Port Daddy Harbor research program. *Reputation is Amortized Verification: Inspection Games for Agent Economies*. Companion formal paper (Paper 3), August 2026. Source of the stage-deterrence and tower-contraction theorems restated in the section on grading-oracle incentive compatibility. Cited on pages 151, 152, 157, and 158.
- [178] Q. Liu & A. Skrzypacz. “Limited Records and Reputation Bubbles.” *Journal of Economic Theory* 151:2–29, 2014. Cited on pages 182 and 188.
- [179] Qingmin Liu and Andrzej Skrzypacz. Limited Records and Reputation Bubbles. *Journal of Economic Theory*, 151:2–29, 2014. Cited on pages 125, 144, and 156.
- [180] R. Myerson & M. Satterthwaite. “Efficient Mechanisms for Bilateral Trading.” *Journal of Economic Theory* 29(2):265–281, 1983. Cited on pages 176 and 188.
- [181] R. Myerson. “Optimal Auction Design.” *Mathematics of Operations Research* 6(1):58–73, 1981. Cited on page 188.
- [182] Ralf Herbrich, Tom Minka, and Thore Graepel. TrueSkill: A Bayesian Skill Rating System. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2007. Cited on pages 124, 125, and 142.
- [183] Ralph A. Bradley and Milton E. Terry. Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika*, 39(3/4):324–345, 1952. Cited on pages 124, 125, and 142.
- [184] Ralph C. Merkle. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology — CRYPTO ’87*, 1987. Cited on pages 30 and 43.
- [185] *Reputation is Amortized Verification: Inspection Games for Agent Economies*. Paper 3 of the Port Daddy Harbor research program, 2026. (The companion technical paper: proves the bonded-inspection stage-game threshold $\rho^* = G/(dB)$, its confiscation-convention fork $\rho_c^* = G/(d(G+B))$, and the audit-tower contraction under sealed multi-clique sampling.) Cited on pages 190, 191, 191, and 193.
- [186] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units (BPE). In *ACL*, 2016. Cited on page 106.
- [187] Robert J. Aumann. Subjectivity and Correlation in Randomized Strategies. *Journal of Mathematical Economics*, 1(1):67–96, 1974. [https://doi.org/10.1016/0304-4068\(74\)90037-8](https://doi.org/10.1016/0304-4068(74)90037-8) Cited on pages 196 and 199.
- [188] Roger B. Myerson. Optimal Auction Design. *Mathematics of Operations Research*, 6(1):58–73, 1981. Cited on pages 125 and 228.
- [189] Roland Hedberg (ed.), Michael B. Jones, Andreas Åkre Solberg, John Bradley, Giuseppe De Marco, and Vladimir Dzhuvinov. OpenID Federation 1.0. OpenID Foundation Implementer’s Draft 4, July 2024. Cited on page 259.
- [190] S. Goldwasser, Y. Kalai & G. Rothblum. “Delegating Computation: Interactive Proofs for Muggles.” *STOC*, pp. 613–622, 2008. Cited on pages 189, 189, and 189.
- [191] S. Grossman & O. Hart. “The Costs and Benefits of Ownership: A Theory of Vertical and Lateral Integration.” *Journal of Political Economy* 94(4):691–719, 1986. Cited on pages 171 and 188.
- [192] S. Tadelis. “Reputation and Feedback Systems in Online Platform Markets.” *Annual Review of Economics* 8:321–340, 2016. Cited on page 188.

- [193] Samson Abramsky and Adam Brandenburger. The operational-sheaf-theoretic approach to contextuality and non-locality. *New Journal of Physics*, 13(11):113036, 2011. Cited on page 251.
- [194] Sanford J. Grossman and Oliver D. Hart. The Costs and Benefits of Ownership: A Theory of Vertical and Lateral Integration. *Journal of Political Economy*, 94(4):691–719, 1986. Cited on page 127.
- [195] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, 2019. Cited on pages 259 and 260.
- [196] Scott A. Crosby and Dan S. Wallach. Efficient Data Structures for Tamper-Evident Logging. In *USENIX Security Symposium*, 2009. Cited on page 260.
- [197] Sepandar D. Kamvar, Mario T. Schlosser, and Hector Garcia-Molina. The EigenTrust Algorithm for Reputation Management in P2P Networks. In *International World Wide Web Conference (WWW)*, 2003. Cited on pages 124, 125, and 142.
- [198] Sepandar D. Kamvar, Mario T. Schlosser, and Hector Garcia-Molina. The EigenTrust Algorithm for Reputation Management in P2P Networks. In *WWW*, 2003. Cited on pages 104 and 113.
- [199] Silvio Micali, Michael Rabin, and Salil Vadhan. Verifiable Random Functions. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 120–130. IEEE, 1999. Cited on page 151.
- [200] Simon Josefsson and Ilari Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). *RFC 8032*, IETF, January 2017. <https://tools.ietf.org/html/rfc8032> Cited on page 52.
- [201] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN Prover for the Symbolic Analysis of Security Protocols. In *Computer Aided Verification (CAV)*, 2013. Cited on page 76.
- [202] Sophie Leroy. Why Is It So Hard to Do My Work? The Challenge of Attention Residue When Switching Between Work Tasks. *Organizational Behavior and Human Decision Processes*, 109(2):168–181, 2009. Cited on page 97.
- [203] Steven Tadelis. Reputation and Feedback Systems in Online Platform Markets. *Annual Review of Economics*, 8:321–340, 2016. Cited on pages 125 and 142.
- [204] Stuart Haber and W. Scott Stornetta. How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2):99–111, 1991. Cited on pages 30 and 43.
- [205] Thomas Hobbes. *Leviathan, or The Matter, Forme and Power of a Common-Wealth Ecclesiasticall and Civill*. 1651. Cited on pages 80, 83, 84, 85, 110, and 112.
- [206] Thomas Hobbes. *Leviathan, or The Matter, Forme and Power of a Common-Wealth Ecclesiasticall and Civill*. Andrew Crooke, London, 1651. Cited on pages 195 and 227.
- [207] Thomas Hobbes. *Leviathan*. 1651. Cited on page 138.
- [208] Tim McLean. Critical Vulnerabilities in JSON Web Token Libraries. Auth0 Blog, 2015. <https://auth0.com/blog/critical-vulnerabilities-in-json-web-token-libraries/> Cited on pages 60, 66, and 77.
- [209] Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020. Cited on page 144.

- [210] Tushar Deepak Chandra and Sam Toueg. Unreliable Failure Detectors for Reliable Distributed Systems. *Journal of the ACM*, 43(2):225–267, 1996. Cited on pages 28 and 43.
- [211] UCAN Working Group. UCAN Specification 1.0: User Controlled Authorization Network, 2024. Cited on page 260.
- [212] Universal Commerce Protocol. Google, Shopify, et al. Specification and reference implementation, Apache-2.0, 2026. <https://ucp.dev>; github.com/Universal-Commerce-Protocol/ucp. Cited on pages 188 and 189.
- [213] Universal Commerce Protocol. Specification and reference implementation (Apache-2.0), Google, Shopify, et al., 2026. <https://ucp.dev>. Cited on pages 76, 141, 228, and 260.
- [214] V. Ren et al. Agent Name Service (ANS): A Universal Directory for Secure AI Agent Discovery and Interoperability. arXiv:2505.10609, 2025. Cited on pages 101 and 113.
- [215] Venkatesh Rao. A Big Little Idea Called Legibility. *ribbonfarm*, 2010. <https://www.ribbonfarm.com/2010/07/26/a-big-little-idea-called-legibility/> Cited on pages 86 and 112.
- [216] Vitalik Buterin and Virgil Griffith. Casper the Friendly Finality Gadget. arXiv:1710.09437, 2017. Cited on page 196.
- [217] Vitalik Buterin. A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum white paper, 2014. Cited on page 197.
- [218] Wei-Fan Chiu, Yuxuan Zhang, and Mihaela van der Schaar. Strategic Self-Improvement for Competitive Agents in AI Labour Markets. arXiv:2512.04988, 2025. Cited on page 145.
- [219] William R. Thompson. On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples. *Biometrika*, 25(3/4):285–294, 1933. Cited on page 144.
- [220] William Vickrey. Counterspeculation, Auctions, and Competitive Sealed Tenders. *The Journal of Finance*, 16(1):8–37, 1961. Cited on page 228.
- [221] Yaron Sheffer, Dick Hardt, and Michael Jones. JSON Web Token Best Current Practices. IETF Draft, 2024. <https://tools.ietf.org/html/draft-ietf-oauth-jwt-bcp> Cited on pages 52 and 77.
- [222] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software Signing for Everybody. In *ACM Conference on Computer and Communications Security (CCS)*, 2022. Cited on pages 244, 259, and 260.